OMNICODE: VOLUME 2: HOST ENGINE & VIRTUAL MACHINE

Track 1: Rust VM, Balanced Ternary Runtime, Evaluator, Assembler & Tests

CORPUS VOLUME
VOL 2

NOTEBOOKLM WORD COUNT
100,075 words (Limit: 500k words)

CATALOGED FILES
61 files (24,866 lines)

GENERATION DATE
2026-10-14 10:52:42

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM (AI PRO) GROUNDING PROMPTS

- **Balanced Ternary Quad-State:** *"How does the T4 algebra {-1:Rollback, 0:Await, +1:Proceed, NULL:Halt} enforce symmetry in this volume?"*
- **4-Block Structure:** *"Explain the boundaries of Block 1 (Metadata/Scripture), Block 2 (Opening), Block 3 (Body), and Block 4 (Closing)."*
- **Character Lifecycle & SVO Verbs:** *"Trace the Character lifecycle through INSCRIBE, TRANSDUCE, PROJECT, and SEQUENCE across terminal ports 0x03F8 (FROM) and 0x000B8000 (INTO)."*

VOLUME SOURCE MANIFEST & INDEX (61 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/TRACK.md**	MD	14	85	0.7
2	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/assemble_tripwire_tests.rs**	RS	305	1,255	13.2
3	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/boot_tripwire_tests.rs**	RS	275	1,137	12.8
4	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/main.rs**	RS	433	1,808	19.6
5	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/platform_tripwire_tests.rs**	RS	241	1,214	11.6
6	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/functional/main.rs**	RS	542	2,314	24.4
7	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/unit/main.rs**	RS	701	3,078	35.5
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/lib.rs**	RS	84	324	3.6
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/main.rs**	RS	748	2,857	32.7
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/boundaries.rs**	RS	167	778	6.9
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/cell.rs**	RS	345	1,236	11.7
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/filetype.rs**	RS	355	1,357	13.7
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/linguistics.rs**	RS	1,209	4,115	52.9
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mmio.rs**	RS	300	1,055	9.7
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mod.rs**	RS	54	205	2.1
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/operators.rs**	RS	344	1,239	13.3
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/reel.rs**	RS	286	1,090	11.5
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/scene.rs**	RS	130	449	4.3
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/tape.rs**	RS	489	1,805	18.6
20	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/ternary.rs**	RS	288	1,163	10.6
21	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/vessel.rs**	RS	320	1,088	11.2
22	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/word.rs**	RS	410	1,517	13.0
23	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/mod.rs**	RS	18	71	0.7
24	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler.rs**	RS	601	2,121	22.7
25	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session.rs**	RS	435	1,554	18.5
26	**<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio.rs**	RS	220	851	10.3
27	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexer.rs**	RS	943	4,072	40.6
28	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/mod.rs**	RS	32	109	1.2
29	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/parser.rs**	RS	760	2,950	31.2
30	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon.rs**	RS	568	3,837	33.6
31	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/mod.rs**	RS	39	161	1.7
32	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer.rs**	RS	745	2,852	27.4
33	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/cortex.rs**	RS	192	589	7.1
34	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/learner.rs**	RS	163	555	6.1
35	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/memory.rs**	RS	389	1,546	15.2
36	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/mod.rs**	RS	38	166	1.7

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
37	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/scroll.rs**	RS	258	881	10.3
38	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/NEURAL_NETWORK_ARCHITECTURE.md**	MD	101	738	5.4
39	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/mod.rs**	RS	32	116	1.3
40	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/network.rs**	RS	237	921	8.8
41	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/mod.rs**	RS	42	146	1.8
42	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/repl.rs**	RS	427	1,585	21.6
43	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/server.rs**	RS	576	1,855	23.0
44	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/assemble.rs**	RS	534	2,055	23.3
45	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/boot.rs**	RS	776	3,856	39.0
46	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/mod.rs**	RS	38	132	1.6
47	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler.rs**	RS	1,930	8,762	95.3
48	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment.rs**	RS	524	2,052	21.0
49	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator.rs**	RS	1,203	4,217	58.5
50	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/mod.rs**	RS	37	122	1.5
51	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/debugger.rs**	RS	471	1,835	20.2
52	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/hover.rs**	RS	989	3,470	39.9
53	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter.rs**	RS	391	1,830	16.9
54	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger.rs**	RS	617	2,541	25.9
55	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/mod.rs**	RS	51	252	2.4
56	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/restorator.rs**	RS	667	2,387	27.8
57	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/mod.rs**	RS	32	132	1.4
58	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/platform.rs**	RS	940	4,296	42.8
59	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/tripwire.rs**	RS	628	2,781	28.2
60	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/mod.rs**	RS	30	102	1.1
61	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes.rs**	RS	152	408	4.0

<TRACK 1> : {Host Engine & Virtual Machine}/TRACK.md MD 14 LINES 85 WORDS 0.7 KB

```
# <TRACK 1> : {Host Engine & Virtual Machine}

## Scope & Invariant

The host compiler, balanced ternary virtual machine, process scheduler, and test harness.

- `src/core/`: Boundaries, Cell, 32-Trit Word, 4-Directional Reel, MMIO, Linguistics.
- `src/runtime/`: Assembler, Transactional Environment, Evaluator.
- `src/terminal/`: Multi-Track OmniScheduler, Living TerminalSession, Studio Renderer.
- `src/ai/`: Cognitive loop, AI Cortex, AILearner, AIMemory, Biblical Lexicon.
- `src/parser/` & `src/ast/`: Directional Lexer, recursive-descent Parser, AST.
- `src/diagnostics/`: Restorator, Debugger, Linter, Logger, Hover Engine.
- `tests/`: 3-tier automated test harness (unit, integrated, functional).
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/assemble_tripwire_tests.rs

RS | 305 LINES | 1255 WORDS | 13.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Assembler Tripwire, Adversarial Faults & Telemetry Oracle Manifest )
// ============================================================================
//! MODULE:              tests::integrated::assemble_tripwire_tests
//! IDENTITY:            Physical Assembler Adversarial Fault Injection & Telemetry Oracle Suite
//! NATURE:              Modular Integration Test Sandbox & Destructive Lowering Certifier
//! VERSION:             0.1.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {OmniCode Language & Standard Library} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Adversarial Assembler Oracle & Machine Lowering Tester
//!   WHAT:              Deliberately push the process tape assembler to fail under adversarial conditions:
//!                      missing files across 6-track roots, invalid file extensions, malformed grammar tapes,
//!                      unsealed commit invariants, and unwritable filesystem destinations.
//!   WHERE:             Track 1 Test Harness -> Assembler Lowering Engine -> DiagnosticLogger & Restorator
//!   WHEN:              Executed during comprehensive test suites and regression certification
//!   WHY:               Ensure that corrupted, malicious, or misconfigured process tapes never cause
//!                      unhandled panics or uncontrolled memory states, and that all failures emit
//!                      crisp balanced-ternary rollback telemetry (-1:ROLLBACK).
//!   HOW:               Adversarial fault injection ("push to fail"): if the test passes, it is
//!                      because the failure was caught and handled by tripwires, NOT because of soft checks.
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - LOGS:            Inscribed in-memory DiagnosticLogger ring buffers
//!   - DISK:            Temporary test tapes written to `/tmp` and cleaned up immediately
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_INPUT_RESOLVE`       (Asserts missing file resolution trips rollback)
//!   - `TRIPWIRE_FILETYPE_VALIDATION` (Asserts non-tape/non-omni extensions trip rollback)
//!   - `TRIPWIRE_SYNTAX_TOKENIZATION` (Asserts malformed process ribbons trip rollback)
//!   - `TRIPWIRE_COMMIT_INVARIANT`    (Asserts unsealed commit status trips rollback)
//!   - `TRIPWIRE_OUTPUT_SERIALIZE`    (Asserts unwritable destination trips rollback)
//!   - `TRIPWIRE_DRY_RUN_GUARD`       (Asserts dry-run guarantees zero disk mutation)
//!   - `TRIPWIRE_BATON_PASS_WORDS`    (Asserts physical silicon word emission integrity)

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Test Fixtures & Environment Setup )
// ============================================================================
use std::fs;
use std::path::PathBuf;

use omnicode::cli::commands::assemble::{
    AssembleConfig, AssembleFormat, execute_assemble_pipeline, serialize_assembled_output,
};
use omnicode::core::filetype::resolve_track_path;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Adversarial Tests: Pushing Assembler to Fail & Lowering Telemetry Oracles )
// ============================================================================

/// Test 1: Adversarial Process Tape Resolution Failure.
///
/// Deliberately queries a non-existent process tape across all 6 track roots.
/// Pushes `AssembleConfig::open` to fail, verifying that:
/// 1. A `TripwireError` is returned with channel `"TRIPWIRE_INPUT_RESOLVE"`.
/// 2. Failpoint classification is [`FailpointType::SyntaxAnomaly`].
/// 3. The recovery state is affirmative rollback (`-1:ROLLBACK`).
/// 4. The attached [`DiagnosticLogger`] in-memory trace registers the failpoint.
```

```rust
#[test]
fn test_adversarial_assemble_missing_file_trips_preflight() {
    let logger = DiagnosticLogger::new(false);
    let config = AssembleConfig::new("nonexistent_phantom_root/phantom_tape_999.tape");

    let open_result = config.open(&logger);
    assert!(open_result.is_err(), "Non-existent process tape MUST trip pre-flight resolution");

    let err = open_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_INPUT_RESOLVE");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    assert!(logger.has_failpoints(), "DiagnosticLogger must capture the failpoint");
    let fps = logger.failpoints();
    assert_eq!(fps.len(), 1);
    assert_eq!(fps[0].channel, "TRIPWIRE_INPUT_RESOLVE");
}

/// Test 2: Adversarial File Extension Rejection.
///
/// Provides a path that exists on disk (e.g. `Cargo.toml`), but has an unsupported
/// extension. Verifies that the assembler strictly refuses to assemble arbitrary files,
/// requiring `.tape` (process ribbon) or `.omni` (declarative spatial reel).
#[test]
fn test_adversarial_assemble_invalid_extension_trips() {
    let logger = DiagnosticLogger::new(false);
    let config = AssembleConfig::new("Cargo.toml");

    let open_result = config.open(&logger);
    assert!(open_result.is_err(), "Invalid file extension MUST trip pre-flight validation");

    let err = open_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_FILETYPE_VALIDATION");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    assert!(err.message.contains("unsupported extension"));
}

/// Test 3: Adversarial Malformed Process Ribbon Syntax.
///
/// Injects a syntactically broken tape containing unclosed boundaries and corrupted
/// directional vectors. Pushes `execute_assemble_pipeline` to fail, ensuring that
/// the lowering engine traps the error via [`FailpointType::SyntaxAnomaly`] without panicking.
#[test]
fn test_adversarial_assemble_corrupted_syntax_tape_trips() {
    let logger = DiagnosticLogger::new(false);
    let temp_tape_path = PathBuf::from("/tmp/omnicode_corrupted_syntax_test.tape");

    // Intentionally corrupted tape: unclosed delimiter and invalid token grammar
    let bad_source = r#"
[Tape.Corrupted]
    who: "AdversarialAgent",
    what: "InjectAnomaly"

TAPE HEAD:
    direction: +1:PROCEED,
    partition: 64

SUBORDINATE ROLLS:
    roll 1: [ BrokenBoundary < 42 ]
"#;

    fs::write(&temp_tape_path, bad_source).expect("Writing temporary corrupted tape");

    let config = AssembleConfig::new(&temp_tape_path);
    let mut ctx = config.open(&logger).expect("Preflight opening valid path");

    let assemble_result = execute_assemble_pipeline(&mut ctx);
    assert!(assemble_result.is_err(), "Corrupted tape syntax MUST trip assembly pipeline");

    let err = assemble_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_SYNTAX_TOKENIZATION");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    let _ = fs::remove_file(&temp_tape_path);
```

```
}

/// Test 4: Adversarial Commit Invariant Breach.
///
/// Injects a process tape whose execution or declaration does NOT conclude with
/// `commit_status [ PROCEED ]`. Asserts that `Tripwire::assert_quad_state_proceed`
/// fires on `"TRIPWIRE_COMMIT_INVARIANT"`, halting assembly.
#[test]
fn test_adversarial_assemble_unsealed_commit_status_trips() {
    let logger = DiagnosticLogger::new(false);
    let temp_tape_path = PathBuf::from("/tmp/omnicode_unsealed_commit_test.tape");

    // Tape explicitly concludes with Rollback instead of Proceed
    let rollback_tape = r#"
[Tape.RollbackTrap]
    who: "AdversarialTester",
    what: "TrapCommitInvariant"

TAPE HEAD:
    direction: -1:ROLLBACK,
    partition: 128

SUBORDINATE ROLLS:
    roll 1: execute [ NopOperation ]
    roll 2: commit_status [ ROLLBACK ]
"#;

    fs::write(&temp_tape_path, rollback_tape).expect("Writing temporary rollback tape");

    let config = AssembleConfig::new(&temp_tape_path);
    let mut ctx = config.open(&logger).expect("Preflight opening valid path");

    let assemble_result = execute_assemble_pipeline(&mut ctx);
    assert!(assemble_result.is_err(), "Unsealed commit status MUST trip invariant assertion");

    let err = assemble_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_COMMIT_INVARIANT");
    assert_eq!(err.failpoint_type, FailpointType::RollbackTriggered);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    let _ = fs::remove_file(&temp_tape_path);
}

/// Test 5: Adversarial Output Path Permission & Serialization Failure.
///
/// Points the output path to a directory that cannot be written (e.g. `/dev/null/forbidden/out.asm`).
/// Verifies that disk serialization catches the IO error, tripwires `"TRIPWIRE_OUTPUT_SERIALIZE"`,
/// and returns `-1:ROLLBACK` without panicking.
#[test]
fn test_adversarial_assemble_output_unwritable_path_trips() {
    let logger = DiagnosticLogger::new(false);
    let boot_tape = resolve_track_path("lib/kernel/boot.tape");
    if !boot_tape.exists() {
        return;
    }

    let config = AssembleConfig {
        input_path: boot_tape,
        output_path: Some(PathBuf::from("/dev/null/forbidden_nonexistent_dir/output.asm")),
        format: AssembleFormat::Listing,
        verbose: false,
        strict: false,
        dry_run: false,
    };

    let mut ctx = config.open(&logger).expect("Preflight opening valid boot tape");
    let assembled = execute_assemble_pipeline(&mut ctx).expect("Assembling boot tape");

    let serialize_result = serialize_assembled_output(&assembled, &config, &logger);
    assert!(serialize_result.is_err(), "Unwritable output destination MUST trip serialization");

    let err = serialize_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_OUTPUT_SERIALIZE");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);
}

/// Test 6: Dry Run Safety Invariant (Zero Disk Mutation).
```

```rust
///
/// Verifies that when `--dry-run` is active, the assembler completes all lowering,
/// symbol resolution, and formatting, but GUARANTEES that the output file is NOT written.
#[test]
fn test_assemble_dry_run_guarantees_zero_disk_mutation() {
    let logger = DiagnosticLogger::new(false);
    let boot_tape = resolve_track_path("lib/kernel/boot.tape");
    if !boot_tape.exists() {
        return;
    }

    let ghost_output = PathBuf::from("/tmp/omnicode_ghost_dry_run_artifact.asm");
    if ghost_output.exists() {
        let _ = fs::remove_file(&ghost_output);
    }

    let config = AssembleConfig {
        input_path: boot_tape,
        output_path: Some(ghost_output.clone()),
        format: AssembleFormat::Words,
        verbose: false,
        strict: false,
        dry_run: true, // Dry run enabled
    };

    let mut ctx = config.open(&logger).expect("Preflight opening boot tape");
    let assembled = execute_assemble_pipeline(&mut ctx).expect("Lowering pipeline");

    let result = serialize_assembled_output(&assembled, &config, &logger);
    assert!(result.is_ok(), "Dry run serialization must succeed without error");
    assert!(!ghost_output.exists(), "Dry run MUST NOT mutate or create output file on disk");
}

/// Test 7: Baton Pass Integration to Physical Silicon Words.
///
/// Assembles `lib/kernel/boot.tape` and certifies that the resulting `AssembledTape`
/// contains non-trivial physical 64-bit machine words (`Word64`), verified registers,
/// and discrete machine opcodes, validating the complete baton pass to hardware execution.
#[test]
fn test_baton_pass_assemble_to_machine_words_verification() {
    let logger = DiagnosticLogger::new(false);
    let boot_tape = resolve_track_path("lib/kernel/boot.tape");
    if !boot_tape.exists() {
        return;
    }

    let config = AssembleConfig {
        input_path: boot_tape,
        output_path: None,
        format: AssembleFormat::Words,
        verbose: false,
        strict: false,
        dry_run: true,
    };

    let mut ctx = config.open(&logger).expect("Opening boot context");
    let assembled = execute_assemble_pipeline(&mut ctx).expect("Lowering pipeline");

    // Baton pass validation
    assert_eq!(assembled.commit_status, QuadState::Proceed);
    assert!(!assembled.instructions.is_empty(), "Instruction stream must not be empty");

    let words = assembled.to_machine_words();
    assert_eq!(words.len(), assembled.instructions.len());

    for (idx, word) in words.iter().enumerate() {
        // Assert that words are non-trivial physical containers
        assert_ne!(word.raw, 0xFFFFFFFF_FFFFFFFF, "Word {} must not be uninitialized sentinel", idx);
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Test Harness Verification Seal & Baton Pass )
// ============================================================================
// All 7 physical assembler adversarial tripwire tests sealed and certified.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/boot_tripwire_tests.rs RS | 275 LINES | 1137 WORDS | 12.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Boot Inception Tripwire & Self-Healing Telemetry Test Suite Manifest )
// ============================================================================
//! MODULE:              tests::integrated::boot_tripwire_tests
//! IDENTITY:            Bare-Metal Boot Telemetry, Tripwire Invariants & Restorator Healing Suite
//! NATURE:              Modular Integration Test Sandbox & Telemetry Oracle
//! VERSION:             0.1.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1> ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Self-Healing Test Oracle
//!   WHAT:              Verify boot tripwire sensors, failpoint logging, debugger frame capture,
//!                      and automated restorative healing handoff to the Restorator.
//!   WHERE:             Track 1 Test Harness -> Boot Pipeline -> Diagnostics Ring Buffer
//!   WHEN:              Executed during test runs and regression test verification
//!   WHY:               Ensure that runtime anomalies during machine inception never panic,
//!                      produce rich flight-recorder telemetry, and feed the Restorator.
//!   HOW:               Simulate invariant breaches, inspect DiagnosticLogger and OmniDebugger,
//!                      and certify candidate healing via test oracles.
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - ARTIFACT:        Track 0 kernel build script and target image artifacts
//!   - LOGS:            Inscribed in-memory DiagnosticLogger ring buffers
//!
//! TRIPWIRE REGISTRY:
//!   - TRIPWIRE_TEST_PRECONDITION_PATH (Asserts path existence sensing)
//!   - TRIPWIRE_TEST_BINARY_RESOLUTION (Asserts hypervisor binary availability)
//!   - TRIPWIRE_TEST_REMEDIATION_HANDOFF (Asserts logger/debugger to Restorator pipeline)

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Test Harness Setup, Mock Fixtures & Scoped Imports )
// ============================================================================
use std::path::PathBuf;

use omnicode::cli::commands::boot::{
    BootArch, BootConfig, BootContext, synthesize_hypervisor_command,
    DEFAULT_BOOT_ARCH, DEFAULT_GRAPHICAL_MODE, REAL_MODE_BOOT_VECTOR,
    VGA_TEXT_FRAMEBUFFER_BASE, COM1_SERIAL_IO_BASE,
};
use omnicode::cogs::platform::HostPlatform;
use omnicode::cogs::tripwire::Tripwire;
use omnicode::core::filetype::resolve_track_path;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::debugger::OmniDebugger;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion};
use omnicode::diagnostics::restorator::Restorator;
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;

/// Creates an isolated test context with a non-printing diagnostic logger
/// and an attached debugger configured with a retroaction depth threshold.
fn create_test_boot_context(arch: BootArch, graphical: bool) -> BootContext {
    let config = BootConfig::new(arch, graphical);
    let logger = DiagnosticLogger::new(false);
    config.open(logger).expect("Boot context must open under valid workspace")
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Tripwire Telemetry, Debugger Frame Capture & Restorator Self-Healing Tests )
// ============================================================================

```

```rust
/// Test 1: Asserts that an invalid or missing path immediately trips the tripwire,
/// inscribes a structured failpoint event into the [`DiagnosticLogger`], records microsecond
/// timestamps, and emits an episodic cognitive note tagged `#failpoint`.
#[test]
fn test_boot_tripwire_missing_path_records_telemetry() {
    let logger = DiagnosticLogger::new(false);
    let non_existent = PathBuf::from("/non_existent_silicon_kernel_image.bin");

    let result = Tripwire::assert_path_exists(
        &logger,
        "BOOT",
        &non_existent,
        "Simulated Missing Kernel Image",
    );

    assert!(result.is_err(), "Missing path must breach the tripwire boundary");
    let err = result.unwrap_err();
    assert_eq!(err.channel, "BOOT");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    // Verify logger captured the failpoint event
    assert!(logger.has_failpoints(), "Logger must register the trapped failpoint");
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    assert!(failpoints[0].message.contains("Simulated Missing Kernel Image"));
    assert_eq!(failpoints[0].channel, "BOOT");

    // Verify cognitive notes for AI episodic memory
    let notes = logger.cognitive_notes();
    assert_eq!(notes.len(), 1);
    assert_eq!(notes[0].category, "INCIDENT");
    assert!(notes[0].tags.contains(&"failpoint".to_string()));
}

/// Test 2: Asserts that tripwiring with remediation telemetry equips the logger event
/// with a concrete [`FixSuggestion`] that the [`OmniDebugger`] and [`Restorator`] can consume.
#[test]
fn test_boot_tripwire_with_remediation_telemetry() {
    let logger = DiagnosticLogger::new(false);
    let mut debugger = OmniDebugger::new(logger.clone(), 10);

    let suggestion = FixSuggestion {
        kind: FixKind::AddBlockHeader,
        line: 1,
        col: 1,
        target_token: "".to_string(),
        suggested_patch: "// BLOCK 1: METADATA BLOCK\n".to_string(),
        rationale: "Inject missing 4-block metadata header before machine compilation".to_string(),
    };

    let err = Tripwire::trip_with_remediation(
        &logger,
        "BOOT",
        FailpointType::SyntaxAnomaly,
        "Kernel manifest header missing 4-block architecture scaffolding",
        suggestion.clone(),
    );

    assert_eq!(err.recovery_state, QuadState::Rollback);

    // Verify failpoint with remediation in logger
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    let fp = &failpoints[0];
    assert!(fp.suggestion.is_some(), "Remediation suggestion must be attached");
    assert_eq!(fp.suggestion.as_ref().unwrap().kind, FixKind::AddBlockHeader);

    // Verify debugger registers the incident into its remediation queue
    debugger.remediation_queue.push(suggestion);
    assert_eq!(debugger.remediation_queue.len(), 1);
    assert_eq!(debugger.remediation_queue[0].kind, FixKind::AddBlockHeader);
}

/// Test 3: The Game Dev Loop Oracle — Simulates a boot-stage anomaly (broken source
/// script in boot loader), captures the tripwire event, hands the telemetry trace
/// off to the [`Restorator`], and verifies that the patch passes the test oracle.
#[test]
```

```rust
fn test_boot_anomaly_telemetry_to_restorator_self_healing_oracle() {
    let broken_boot_script = "boot_kernel < 1 ]";
    let logger = DiagnosticLogger::new(false);

    // 1. Lex and parse the broken boot script to trip the boundary
    let mut lexer = Lexer::new(broken_boot_script, logger.clone());
    let tokens = lexer.tokenize().expect("Lexing succeeds");
    let mut debugger = OmniDebugger::new(logger.clone(), 5);
    let mut parser = Parser::new(tokens, debugger.clone());
    let parse_res = parser.parse_program();
    assert!(parse_res.is_err(), "Delimiter mismatch must fail parsing");

    // 2. Verify logger trapped the boundary mismatch failpoint
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    let incident = &failpoints[0];
    assert_eq!(incident.failpoint_type, Some(FailpointType::BoundaryMismatch));
    assert!(incident.suggestion.is_some());

    // 3. Define the Test Oracle simulating valid boot script verification
    let oracle = |candidate_code: &str| -> Result<(), String> {
        let test_logger = DiagnosticLogger::new(false);
        let mut test_lexer = Lexer::new(candidate_code, test_logger.clone());
        let test_tokens = test_lexer.tokenize().map_err(|e| format!("Lex error: {e}"))?;
        let test_debugger = OmniDebugger::new(test_logger.clone(), 5);
        let mut test_parser = Parser::new(test_tokens, test_debugger);
        test_parser.parse_program().map_err(|e| format!("Parse error: {e}"))?;
        if test_logger.has_failpoints() {
            return Err("Candidate caused failpoints".to_string());
        }
        Ok(())
    };

    // 4. Debugger executes restoration from the tripwire log event
    let healed_code = debugger
        .execute_restoration_from_log(broken_boot_script, incident, oracle)
        .expect("Self-healing restoration must be verified by oracle");

    assert_eq!(healed_code, "boot_kernel < 1 >\n");

    // 5. Restorator executes automated multi-pass verification with oracle
    let restorator = Restorator::new(logger.clone());
    let report = restorator.restore_and_verify_with_oracle(broken_boot_script, |code| {
        if code.contains("bad_boundary") {
            Err("Forbidden pattern".to_string())
        } else {
            Ok(())
        }
    });

    assert!(report.verified, "Restorator must verify patch with oracle");
    assert!(report.was_modified);

    // 6. Confirm episodic cognitive memory reflects verified repair
    let notes = logger.cognitive_notes();
    assert!(notes.iter().any(|n| n.category == "REPAIR_VERIFIED" || n.category == "ORACLE_CERTIFIED"));
}

/// Test 4: Verifies the hypervisor command synthesis matrix across all 3 architectures
/// (BIOS MBR, UEFI Long Mode, Multiboot ELF) in both headless and GUI modes.
#[test]
fn test_hypervisor_command_synthesis_matrix() {
    let _logger = DiagnosticLogger::new(false);

    // Case A: BIOS Headless
    let ctx_bios = create_test_boot_context(BootArch::Bios, false);
    let cmd_bios = synthesize_hypervisor_command(&ctx_bios).expect("Synthesize BIOS command");
    let args_bios: Vec<String> = cmd_bios.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(args_bios.contains(&"-drive".to_string()));
    assert!(args_bios.iter().any(|a| a.contains("format=raw,file=")));
    assert!(args_bios.contains(&"-display".to_string()));
    assert!(args_bios.contains(&"none".to_string()));
    assert!(args_bios.contains(&"-serial".to_string()));
    assert!(args_bios.contains(&"stdio".to_string()));

    // Case B: BIOS Graphical
    let ctx_bios_gui = create_test_boot_context(BootArch::Bios, true);
    let cmd_bios_gui = synthesize_hypervisor_command(&ctx_bios_gui).expect("Synthesize BIOS GUI");
```

```
    let args_bios_gui: Vec<String> = cmd_bios_gui.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(!args_bios_gui.contains(&"-display".to_string()));
    assert!(args_bios_gui.contains(&"-serial".to_string()));

    // Case C: UEFI Headless
    let ctx_uefi = create_test_boot_context(BootArch::Uefi, false);
    let cmd_uefi = synthesize_hypervisor_command(&ctx_uefi).expect("Synthesize UEFI command");
    let args_uefi: Vec<String> = cmd_uefi.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(args_uefi.contains(&"-bios".to_string()));
    assert!(args_uefi.iter().any(|a| a.contains("fat:rw:")));

    // Case D: Multiboot Headless
    let ctx_mb = create_test_boot_context(BootArch::Multiboot, false);
    let cmd_mb = synthesize_hypervisor_command(&ctx_mb).expect("Synthesize Multiboot command");
    let args_mb: Vec<String> = cmd_mb.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(args_mb.contains(&"-kernel".to_string()));
    assert!(args_mb.iter().any(|a| a.contains("omnicode_multiboot.elf")));
}

/// Test 5: Verifies that hardware constants align with physical silicon definitions:
/// - Real mode reset vector = `0x7C00`
/// - VGA text buffer base = `0xB8000`
/// - COM1 serial port I/O base = `0x3F8`
/// - Default boot architecture = BIOS
/// - Default graphical mode = false (headless)
#[test]
fn test_boot_hardware_constants_and_silicon_addresses() {
    assert_eq!(REAL_MODE_BOOT_VECTOR, 0x7C00);
    assert_eq!(VGA_TEXT_FRAMEBUFFER_BASE, 0xB8000);
    assert_eq!(COM1_SERIAL_IO_BASE, 0x3F8);
    assert_eq!(DEFAULT_BOOT_ARCH, BootArch::Bios);
    assert_eq!(DEFAULT_GRAPHICAL_MODE, false);

    let host = HostPlatform::current();
    assert!(!host.hypervisor_binary().is_empty());

    let script = resolve_track_path("kernel/build_all_kernels.sh");
    assert!(script.exists(), "Build script must resolve");
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Test Suite Sealing, Invariant Verification Proof & Postconditions )
// ============================================================================
// Boot inception tripwire integration test suite verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/main.rs RS 433 LINES 1808 WORDS 19.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Integration Test Suite Identity, Subsystem Interactions & Failpoint Invariants )
// ============================================================================
//! OmniCode Subsystem Integration Test Suite
//!
//! Verifies interactions between compiler components, runtime engines,
//! and the autonomous self-healing loop:
//!   - Boundary physics and asymmetric causality delimiters.
//!   - Failpoint traps (mismatches, leaks, immutable mutations, caveat vetoes).
//!   - Cyclic retroaction depth bounding (infinite rollback protection).
//!   - 4-Block structural linter validation.
//!   - Module import and core standard library resolution.
//!   - Connected Self-Healing Feedback Loop:
//!     Logger -> Structured Cognitive Trace -> Debugger/Restorator -> Test Oracle.

use omnicode::core::boundaries::BoundaryType;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType};
use omnicode::diagnostics::debugger::OmniDebugger;
use omnicode::diagnostics::linter::OmniLinter;
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;
use omnicode::runtime::environment::{Environment, RuntimeValue};
use omnicode::runtime::evaluator::Evaluator;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Integration Test Harness Setup & Invariant Preconditions )
// ============================================================================
/// Constructs an integration test fixture containing a silent [`DiagnosticLogger`]
/// and an [`OmniDebugger`] with a custom retroaction limit threshold.
///
/// # Arguments
/// * `threshold` - Maximum retroactions permitted before triggering [`FailpointType::CyclicRetroaction`].
fn create_test_debugger(threshold: usize) -> (DiagnosticLogger, OmniDebugger) {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), threshold);
    (logger, debugger)
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Integration Cases: Failpoints, Imports, Self-Healing Loop )
// ============================================================================

/// Verifies asymmetric causality delimiters that govern directional influence:
/// - `{ ... ]`: Top-Down / Outside-In imposition by ambient context.
/// - `[ ... }`: Bottom-Up / Inside-Out emergence defined by internal denotation.
/// - Standard symmetric pairs: `[ ... ]` (Denotation), `< ... >` (Invariant), `{ ... }` (Connotation), `( ... )` (Aside).
#[test]
fn test_asymmetric_causality_boundaries() {
    assert_eq!(BoundaryType::from_delimiters('{', ']').unwrap(), BoundaryType::TopDown);
    assert_eq!(BoundaryType::from_delimiters('[', '}').unwrap(), BoundaryType::BottomUp);
    assert_eq!(BoundaryType::from_delimiters('[', ']').unwrap(), BoundaryType::Denotation);
    assert_eq!(BoundaryType::from_delimiters('<', '>').unwrap(), BoundaryType::Invariant);
    assert_eq!(BoundaryType::from_delimiters('{', '}').unwrap(), BoundaryType::Connotation);
    assert_eq!(BoundaryType::from_delimiters('(', ')').unwrap(), BoundaryType::Aside);
}

/// Verifies that closing a boundary with an illegal partner (e.g. `< ... ]`) triggers
/// a [`FailpointType::BoundaryMismatch`], records remediation telemetry, and creates
/// an episodic [`crate::diagnostics::logger::CognitiveNote`] for AI memory.
#[test]
fn test_failpoint_boundary_mismatch_illegal_crossing() {
    let source = "bad_boundary < 42 ]";
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization should succeed");

    let mut parser = Parser::new(tokens, debugger);
```

```
    let result = parser.parse_program();
    assert!(result.is_err(), "Parser must reject illegal boundary '< ... ]'");

    let failpoints = logger.failpoints();
    assert!(!failpoints.is_empty(), "Failpoint must be recorded in logger");
    assert_eq!(
        failpoints[0].failpoint_type,
        Some(FailpointType::BoundaryMismatch),
        "Expected FP_BOUNDARY_MISMATCH"
    );

    // Verify cognitive notes captured the incident for AI episodic memory
    let notes = logger.cognitive_notes();
    assert!(!notes.is_empty(), "Incident note must be recorded");
    assert_eq!(notes[0].category, "INCIDENT");
    assert!(notes[0].tags.contains(&"remediation_ready".to_string()));
}

/// Verifies that reaching EOF while delimiters remain open on the stack triggers
/// [`FailpointType::BoundaryLeak`] with a remediation patch proposing the missing closers.
#[test]
fn test_failpoint_boundary_leak_unclosed_delimiter() {
    let source = "unclosed_block [ 100 200";
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization should succeed");

    let mut parser = Parser::new(tokens, debugger);
    let result = parser.parse_program();
    assert!(result.is_err(), "Parser must reject unclosed boundary reaching EOF");

    let failpoints = logger.failpoints();
    assert!(
        failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::BoundaryLeak)),
        "Expected FP_BOUNDARY_LEAK"
    );
}

/// Verifies that attempting to rebind or mutate an entity declared inside `< ... >`
/// triggers [`FailpointType::ImmutableMutation`] and suggests converting it to `[ ... ]`.
#[test]
fn test_failpoint_immutable_mutation_attempt() {
    let source = r#"
        CONSTANT < 100 >
        CONSTANT [ 200 ]
    "#;
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokens");

    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse");

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let result = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog));

    assert!(result.is_err(), "Must reject mutation of invariant <...>");
    let failpoints = logger.failpoints();
    assert!(
        failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::ImmutableMutation)),
        "Expected FP_IMMUTABLE_MUTATION"
    );
}

/// Verifies that an unsatisfied adversative caveat expression (`! [ ... ]`) vetoes
/// the active state transition, triggers [`FailpointType::CaveatVeto`], and returns [`QuadState::Rollback`].
#[test]
fn test_failpoint_caveat_veto_and_rollback() {
    let source = "Transaction [ +1 ] ! [ -1 ]";
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokens");

    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse");

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let result = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
```

```
    assert_eq!(result, RuntimeValue::State(QuadState::Rollback));

    let failpoints = logger.failpoints();
    assert!(
        failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::CaveatVeto)),
        "Expected FP_CAVEAT_VETO"
    );
}

/// Verifies that evaluating into a void state without a resolving caveat triggers
/// [`FailpointType::NullTrap`] and halts execution into [`QuadState::Halt`].
#[test]
fn test_failpoint_null_trap_absorption() {
    let source = "risky_operation [ +1 ] ! [ NULL ]";
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokens");

    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse");

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let result = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");

    assert_eq!(result, RuntimeValue::State(QuadState::Halt));

    let failpoints = logger.failpoints();
    assert!(
        failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::NullTrap)),
        "Expected FP_NULL_TRAP"
    );
}

/// Verifies that repeated retroactions on the same label exceeding the threshold limit
/// are caught and aborted as [`FailpointType::CyclicRetroaction`].
#[test]
fn test_failpoint_cyclic_retroaction_threshold() {
    let (logger, mut debugger) = create_test_debugger(3);

    assert!(debugger.track_retroaction("loop").is_ok());
    assert!(debugger.track_retroaction("loop").is_ok());
    assert!(debugger.track_retroaction("loop").is_ok());

    let over_threshold = debugger.track_retroaction("loop");
    assert!(over_threshold.is_err(), "Must trip cyclic retroaction detector");

    let failpoints = logger.failpoints();
    assert!(
        failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::CyclicRetroaction)),
        "Expected FP_CYCLIC_RETROACTION"
    );
}

/// Verifies structural linter enforcement of the universal 4-Block architecture:
/// Confirms that compliant code passes, and non-compliant code missing blocks is flagged with errors.
#[test]
fn test_four_block_structural_linter_validation() {
    let logger = DiagnosticLogger::new(false);
    let linter = OmniLinter::new(logger);

    // Test compliant 4-block source
    let compliant_source = r#"
# BLOCK 1: METADATA BLOCK
Module < Test > () {}
# BLOCK 2: OPENING BLOCK
Opening {}
# BLOCK 3: BODY BLOCK
Body []
# BLOCK 4: CLOSING BLOCK
Closing {}
"#;
    let report = linter.lint_source("test_compliant", compliant_source);
    assert!(report.is_valid, "Compliant 4-block source must pass linter");
    assert_eq!(report.blocks_found.len(), 4);

    // Test non-compliant source (missing body block)
    let non_compliant_source = r#"
```

```
# BLOCK 1: METADATA BLOCK
Module < Broken > () {}
# BLOCK 2: OPENING BLOCK
Opening {}
# BLOCK 4: CLOSING BLOCK
Closing {}
"#;
    let fail_report = linter.lint_source("test_broken", non_compliant_source);
    assert!(!fail_report.is_valid, "Source missing Body block must fail lint");
    assert!(fail_report.errors.iter().any(|e| e.contains("Missing BLOCK 3")));
}

/// Verifies standard library imports (`Core.Arithmetic`, `Core.Logic`) inside the `Opening` block,
/// confirming that library macros (`ADD`, `SUB`, `PROCEED_MACRO`) are resolved and registered.
#[test]
fn test_module_import_and_library_resolution() {
    let source = r#"
        Opening {
            import: [ Core.Arithmetic :: Core.Logic ],
            preconditions: [ +1 ]
        }
        test_val [ 50 ]
        delta [ 10 ]
        res [ test_val --> test_val plus delta ]
        Closing {
            verify: [
                res = 60 ::
                ADD = InvariantMacro
            ],
            commit_status [ PROCEED ]
        }
    "#;
    let logger = DiagnosticLogger::new(false);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize module import");
    let debugger = OmniDebugger::new(logger, 10);
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse module import");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));

    assert!(evaluator.loaded_modules.contains("Core.Arithmetic"));
    assert!(evaluator.loaded_modules.contains("Core.Logic"));
    assert!(evaluator.env.contains("ADD"), "ADD macro must be present from Core.Arithmetic");
    assert!(evaluator.env.contains("SUB"), "SUB macro must be present from Core.Arithmetic");
    assert!(evaluator.env.contains("PROCEED_MACRO"), "PROCEED_MACRO must be present from Core.Logic");
    assert!(evaluator.debugger.immutable_registry.contains("ADD"));
    assert_eq!(evaluator.env.get("res"), Some(&RuntimeValue::Number(60.0)));
}

/// Verifies that the standard vector library `lib/core/vectors.omni` parses and executes cleanly.
#[test]
fn test_lib_core_vectors() {
    let path = omnicode::core::filetype::resolve_track_path("lib/core/vectors.omni");
    let source = std::fs::read_to_string(&path).unwrap();
    let logger = DiagnosticLogger::new(false);
    let mut lexer = Lexer::new(&source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize");
    let debugger = OmniDebugger::new(logger.clone(), 10);
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));
    assert!(!logger.has_failpoints(), "Failpoints: {:?}", logger.failpoints());
}

/// Verifies that the standard memory library `lib/core/memory.omni` parses and executes cleanly,
/// testing vessels, partitions, forward vector writes, and bounds verification.
#[test]
fn test_lib_core_memory() {
    let path = omnicode::core::filetype::resolve_track_path("lib/core/memory.omni");
    let source = std::fs::read_to_string(&path).unwrap();
    let logger = DiagnosticLogger::new(false);
    let mut lexer = Lexer::new(&source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize memory.omni");
    let debugger = OmniDebugger::new(logger.clone(), 10);
```

```
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse memory.omni");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval memory.omni");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));
    assert!(!logger.has_failpoints(), "Failpoints: {:?}", logger.failpoints());
}

/// Verifies multi-pass automatic healing of syntax errors and unclosed delimiters
/// via [`crate::diagnostics::restorator::Restorator`].
#[test]
fn test_restorator_boundary_and_structural_auto_heal() {
    let broken_source = r#"
        bad_boundary < 42 ]
        unclosed_block [ 10 20
        step [ 5 --> 10 ]
    "#;
    let logger = DiagnosticLogger::new(false);
    let restorator = omnicode::Restorator::new(logger);
    let report = restorator.restore_source(broken_source);

    assert!(report.was_modified, "Broken source must be modified by restorator");
    assert!(!report.repairs_applied.is_empty(), "Restorator must log applied repairs");
    assert!(report.verified, "Restored code must pass verification: {:?}", report.errors);
    assert_eq!(report.final_state, Some(QuadState::Proceed));
    assert!(report.restored_content.contains("BLOCK 1: METADATA"));
    assert!(report.restored_content.contains("BLOCK 2: OPENING"));
    assert!(report.restored_content.contains("BLOCK 3: BODY"));
    assert!(report.restored_content.contains("BLOCK 4: CLOSING"));
}

/// Verifies end-to-end prompt transduction into executable AST and natural response generation
/// via [`crate::ai::SemanticTransducer`].
#[test]
fn test_ai_semantic_transducer_round_trip() {
    let prompt = "Add 1 ball to the group A which contains 4 balls";
    let logger = DiagnosticLogger::new(false);
    let transducer = omnicode::AITransducer::new(logger);
    let session = transducer.transduce_and_execute(prompt).expect("Transduction must succeed");

    assert_eq!(session.execution_state, QuadState::Proceed);
    assert!(session.synthesized_omnicode.contains("Ball < Entity? >"));
    assert!(session.synthesized_omnicode.contains("Group < Set? >"));
    assert!(session.synthesized_omnicode.contains("A [ 4 : Ball? ]"));
    assert!(session.synthesized_omnicode.contains("A --> A plus new_Ball"));
    assert!(session.synthesized_omnicode.contains("roll_1 = 5"));
    assert!(session.natural_response.contains("I added 1 ball to Group A"));
    assert!(session.natural_response.contains("5 balls in total"));
}

/// The Cybernetic Self-Healing Feedback Loop Test:
///
/// Demonstrates the full feedback loop between sensory logging, debugger intervention,
/// restorative patch synthesis, and test oracle verification:
/// 1. **Phase 1 (Anomaly Detection)**: Execution hits an illegal delimiter crossing (`< 99 ]`).
/// 2. **Phase 2 (Structured Telemetry)**: Logger records a [`FailpointType::BoundaryMismatch`]
///    equipped with a [`crate::diagnostics::logger::FixSuggestion`].
/// 3. **Phase 3 (Oracle-Verified Restoration)**: The Debugger consumes the structured log,
///    applies the patch, and runs an isolated test oracle to certify that the fix is sound.
/// 4. **Phase 4 (Immune System Restoration)**: Restorator executes round-trip verification with an oracle.
/// 5. **Phase 5 (Episodic Memory Audit)**: Verifies that cognitive memory notes reflect the certified repair.
#[test]
fn test_self_healing_feedback_loop_logger_debugger_oracle() {
    let broken_input = "rank1_invariant < 99 ]";
    let logger = DiagnosticLogger::new(false);

    // Phase 1: Anomaly Encounter & Structured Telemetry Inscription
    let mut lexer = Lexer::new(broken_input, logger.clone());
    let tokens = lexer.tokenize().expect("Lexing succeeds");
    let mut debugger = OmniDebugger::new(logger.clone(), 5);
    let mut parser = Parser::new(tokens, debugger.clone());
    let parse_err = parser.parse_program();
    assert!(parse_err.is_err(), "Must trap boundary mismatch");

    // Phase 2: Inspect Structured LogEvent and Remediation Suggestion
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    let incident = &failpoints[0];
```

```
    assert_eq!(incident.failpoint_type, Some(FailpointType::BoundaryMismatch));
    assert!(incident.suggestion.is_some(), "Structured remediation required");

    // Phase 3: The Debugger applies the restoration from the structured log and invokes the test oracle
    let oracle = |candidate_code: &str| -> Result<(), String> {
        let test_logger = DiagnosticLogger::new(false);
        let mut test_lexer = Lexer::new(candidate_code, test_logger.clone());
        let test_tokens = test_lexer.tokenize().map_err(|e| format!("Lex error: {e}"))?;
        let test_debugger = OmniDebugger::new(test_logger.clone(), 5);
        let mut test_parser = Parser::new(test_tokens, test_debugger);
        test_parser.parse_program().map_err(|e| format!("Parse error: {e}"))?;
        if test_logger.has_failpoints() {
            return Err("Candidate caused failpoints".to_string());
        }
        Ok(())
    };

    let healed = debugger
        .execute_restoration_from_log(broken_input, incident, oracle)
        .expect("Self-healing restoration must be verified by oracle");

    assert_eq!(healed, "rank1_invariant < 99 >\n");

    // Phase 4: Restorator round-trip oracle certification
    let restorator = omnicode::Restorator::new(logger.clone());
    let report = restorator.restore_and_verify_with_oracle(broken_input, |code| {
        if code.contains("bad_boundary") {
            Err("Forbidden pattern".to_string())
        } else {
            Ok(())
        }
    });

    assert!(report.verified, "Restorator oracle certification succeeded");
    assert!(report.was_modified);

    // Phase 5: Confirm cognitive notes audit trace contains the verified repair
    let notes = logger.cognitive_notes();
    assert!(notes.iter().any(|n| n.category == "REPAIR_VERIFIED" || n.category == "ORACLE_CERTIFIED"));
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Integration Verification Proof & Postconditions )
// ============================================================================
// Integration test harness sealed and verified.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/platform_tripwire_tests.rs

RS | 241 LINES | 1214 WORDS | 11.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Platform Adapter Tripwire, Adversarial Faults & Telemetry Oracle Manifest )
// ============================================================================
//! MODULE:              tests::integrated::platform_tripwire_tests
//! IDENTITY:            Platform Adapter Adversarial Fault Injection & Telemetry Oracle Suite
//! NATURE:              Modular Integration Test Sandbox & Destructive Telemetry Certifier
//! VERSION:             0.1.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Adversarial Platform Oracle & Telemetry Tester
//!   WHAT:              Deliberately push platform adapters to fail under hostile conditions,
//!                      inject faults, simulate missing hypervisors/firmware/display servers,
//!                      and certify that sensory tripwires produce structured telemetry and
//!                      remediation suggestions for the Restorator.
//!   WHERE:             Track 1 Test Harness -> Platform Adapter Cog -> DiagnosticLogger & Restorator
//!   WHEN:              Executed during comprehensive test suites and regression certification
//!   WHY:               Ensure that hostile host environments never cause unhandled panics or crashes,
//!                      and that every failure is converted into an actionable cognitive observation.
//!   HOW:               Adversarial fault injection ("push to fail"): if the test passes, it is
//!                      because the failure was caught and handled, NOT because of soft green checks.
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - LOGS:            Inscribed in-memory DiagnosticLogger ring buffers
//!   - SUBPROCESS:      Executes isolated test child processes to verify error trapping
//!
//! TRIPWIRE REGISTRY:
//!   - TRIPWIRE_TEST_HYPERVISOR_MISSING    (Asserts hypervisor missing remediation)
//!   - TRIPWIRE_TEST_FIRMWARE_UNRESOLVED   (Asserts firmware missing AI observation)
//!   - TRIPWIRE_TEST_DISPLAY_HEADLESS      (Asserts display server missing caveat fix)
//!   - TRIPWIRE_TEST_SUBPROCESS_NONZERO    (Asserts child process non-zero exit trapping)
//!   - TRIPWIRE_TEST_BATON_PASS_BOOT       (Asserts clean handoff to boot context)

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Test Fixtures & Environment Setup )
// ============================================================================
use std::process::Command;

use omnicode::cli::commands::boot::{BootArch, BootConfig};
use omnicode::cogs::platform::HostPlatform;
use omnicode::cogs::tripwire::Tripwire;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Adversarial Tests: Pushing Platform Adapters to Fail & Telemetry Oracles )
// ============================================================================

/// Test 1: Adversarial Hypervisor Resolution Failure.
///
/// Deliberately simulates a non-existent or corrupted hypervisor binary on the host platform.
/// Pushes `Tripwire::assert_binary_available` to fail, verifying that:
/// 1. A `TripwireError` is returned with `QuadState::Rollback`.
/// 2. The `DiagnosticLogger` captures the failpoint with channel `"PLATFORM"`.
/// 3. A concrete `FixSuggestion` with installation instructions (`apt`, `brew`, `winget`)
///    is attached for the [`omnicode::diagnostics::restorator::Restorator`].
#[test]
fn test_adversarial_hypervisor_missing_trips_remediation() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::current();

```

```
    // Query an intentionally non-existent executable name
    let non_existent_binary = "qemu-system-nonexistent-silicon-arch-999";
    let check = platform.resolve_in_path(non_existent_binary);
    assert!(check.is_none(), "Non-existent binary must NOT resolve on PATH");

    // Push tripwire to fail and assert that it trips
    let trip_result = Tripwire::assert_binary_available(&logger, "PLATFORM", non_existent_binary);
    assert!(trip_result.is_err(), "Missing binary MUST trip the sensory wire");

    let err = trip_result.unwrap_err();
    assert_eq!(err.channel, "PLATFORM");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    // Assert that the logger recorded the failpoint
    assert!(logger.has_failpoints(), "Logger must register the trapped failpoint");
    let fps = logger.failpoints();
    assert_eq!(fps.len(), 1);
    assert!(fps[0].message.contains(non_existent_binary));

    // Assert that cognitive note with #failpoint tag was recorded
    let notes = logger.cognitive_notes();
    assert!(
        notes.iter().any(|n| n.tags.iter().any(|t| t == "failpoint")),
        "Episodic trace must contain #failpoint tag"
    );
}

/// Test 2: Adversarial Firmware Traversal & Fallback Guidance.
///
/// Traverses firmware candidates when none are present on disk (or in a simulated environment),
/// asserting that `find_ovmf_firmware_with_telemetry`:
/// 1. Emits individual telemetry probe events into the logger.
/// 2. Records an AI cognitive observation labeled `"FIRMWARE_UNRESOLVED"`.
/// 3. Inscribes tags `["firmware", "ovmf", "uefi", "fallback"]` so the Restorator
///    and AI reasoning agents can propose falling back to `--bios` MBR mode.
#[test]
fn test_adversarial_firmware_discovery_records_ai_observation() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::Unknown; // Unknown platform has generic fallback candidates

    // Probing firmware with telemetry must record probe logs
    let fw = platform.find_ovmf_firmware_with_telemetry(&logger);

    let events = logger.events();
    assert!(events.iter().any(|e| e.channel == "FIRMWARE"), "Probe events must be logged");

    // If firmware is absent on host, verify AI observation was registered
    if fw.is_none() {
        let notes = logger.cognitive_notes();
        assert!(
            notes.iter().any(|n| n.category == "FIRMWARE_UNRESOLVED"),
            "Logger must record FIRMWARE_UNRESOLVED note for AI cortex and Restorator"
        );
        let target_note = notes.iter().find(|n| n.category == "FIRMWARE_UNRESOLVED").unwrap();
        assert!(target_note.tags.iter().any(|t| t == "fallback"));
    }
}

/// Test 3: Adversarial Display Server Assertion.
///
/// Tests `assert_display_server_available` when display server is absent (simulated or real).
/// Asserts that if no display server is active:
/// 1. The tripwire fires with `FailpointType::SyntaxAnomaly`.
/// 2. A `FixSuggestion` with `FixKind::AdjustCaveat` is generated, recommending `--headless`.
#[test]
fn test_adversarial_display_server_assertion_contract() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::current();

    // If display server is active, assert success
    if platform.has_display_server() {
        let res = platform.assert_display_server_available(&logger);
        assert!(res.is_ok(), "Display server assertion must succeed when display server is active");
    } else {
        // If headless, assert that tripwire trips with remediation
        let res = platform.assert_display_server_available(&logger);
        assert!(res.is_err(), "Headless environment MUST trip display server assertion");
```

```
        let err = res.unwrap_err();
        assert_eq!(err.channel, "DISPLAY");
        assert!(logger.has_failpoints(), "Logger must capture display failpoint");
    }
}

/// Test 4: Adversarial Host Subprocess Non-Zero Exit Trapping.
///
/// Deliberately spawns a child process that exits with a non-zero exit status code (`42`).
/// Asserts that `execute_host_command`:
/// 1. Traps the non-zero exit status without panicking.
/// 2. Returns a `TripwireError` containing the exit code and stderr output.
/// 3. Inscribes the failpoint into the logger with channel `"SUBPROCESS_TEST"`.
#[test]
fn test_adversarial_subprocess_traps_nonzero_exit() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::current();

    // Construct a failing command depending on host family
    let cmd = if platform.is_windows() {
        let mut c = Command::new("cmd");
        c.args(["/C", "exit 42"]);
        c
    } else {
        let mut c = Command::new("sh");
        c.args(["-c", "echo 'Simulated Subprocess Error' >&2; exit 42"]);
        c
    };

    let result = platform.execute_host_command(&logger, "SUBPROCESS_TEST", cmd);
    assert!(result.is_err(), "Non-zero exit must trip the subprocess execution wire");

    let err = result.unwrap_err();
    assert_eq!(err.channel, "SUBPROCESS_TEST");
    assert!(err.message.contains("42"), "Error message must report exit code 42");

    if !platform.is_windows() {
        assert!(
            err.message.contains("Simulated Subprocess Error"),
            "Error message must capture child stderr stream"
        );
    }

    assert!(logger.has_failpoints(), "Logger must capture non-zero subprocess failpoint");
}

/// Test 5: Baton Pass Integration Test between `platform.rs` and `boot.rs`.
///
/// Verifies the full handoff contract:
/// 1. `HostPlatform::current()` identifies the host environment.
/// 2. `BootConfig::new` ingests the platform without data loss.
/// 3. `BootConfig::open` uses the platform's hypervisor binary and firmware paths
///    to construct an immutable `BootContext`.
/// 4. If `--gui` is requested in a headless environment, an AI observation is recorded
///    in the logger without aborting process state.
#[test]
fn test_baton_pass_platform_to_boot_context_integration() {
    let platform = HostPlatform::current();
    assert!(!platform.display_name().is_empty());

    let logger = DiagnosticLogger::new(false);
    let cfg = BootConfig::new(BootArch::Bios, false);
    assert_eq!(cfg.host_platform, platform, "Config must retain current host platform");

    let ctx = cfg.open(logger).expect("Boot context opening must succeed under valid workspace");
    assert_eq!(ctx.hypervisor_binary, platform.hypervisor_binary(), "Hypervisor binary must match platform contract");
    assert_eq!(ctx.debugger.step_count, 0, "Debugger must be armed at step 0");

    // Test graphical request handoff: if headless, verify observation
    let logger_gui = DiagnosticLogger::new(false);
    let cfg_gui = BootConfig::new(BootArch::Bios, true);
    let ctx_gui = cfg_gui.open(logger_gui.clone()).expect("Context opening must succeed");
    assert!(ctx_gui.config.graphical);

    if !platform.has_display_server() {
        assert!(
            logger_gui.cognitive_notes().iter().any(|n| n.category == "GUI_DISPLAY_HEADLESS"),
            "Opening graphical config in headless environment must record GUI_DISPLAY_HEADLESS observation"
```

```
        );
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Module Sealing & Invariant Verification )
// ================================================================================
// Verification Constraints:
//   - All 5 adversarial tests MUST push to fail and verify clean recovery.
//   - Zero unhandled panics across subprocess execution or missing binary probes.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/functional/main.rs RS 542 LINES 2314 WORDS 24.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Functional Test Suite Identity, E2E Cognitive Workflows & Lifecycle Invariants )
// ================================================================================
//! OmniCode Functional Test Suite
//!
//! Verifies end-to-end user journeys, live file lifecycle operations, and the
//! complete neural-symbolic cognitive loop:
//!   - REPL interactive session simulation with persistent environment state.
//!   - Canonical syntax example file parsing and execution from disk.
//!   - AI Cortex end-to-end prompt-to-evaluation pipeline producing dual files
//!     (.scroll native Turing reels & .log traditional execution traces).
//!   - AILearner episodic memory digestion of execution scrolls.
//!   - End-to-end file healing with in-place disk modification and re-execution.

use std::fs;
use std::path::PathBuf;
use omnicode::core::ternary::QuadState;
use omnicode::ai::{AICortex, AILearner};
use omnicode::diagnostics::logger::DiagnosticLogger;
use omnicode::diagnostics::debugger::OmniDebugger;
use omnicode::diagnostics::restorator::Restorator;
use omnicode::diagnostics::hover::HoverEngine;
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;
use omnicode::runtime::environment::{Environment, RuntimeValue};
use omnicode::runtime::evaluator::Evaluator;
use omnicode::terminal::{OmniScheduler, TerminalSession, Track};


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Test Artifact Isolation & Fixture Initialization )
// ================================================================================
/// Prepares an isolated test workspace directory within `target/test_artifacts/`.
///
/// # Invariants & Isolation
/// - If the target directory already exists from a previous run, it is purged.
/// - A fresh, empty directory is created to prevent test cross-contamination.
///
/// # Arguments
/// * `name` - Subdirectory identifier uniquely distinguishing the functional test run.
///
/// # Returns
/// A [`PathBuf`] pointing to the clean artifact directory on disk.
fn get_test_dir(name: &str) -> PathBuf {
    let dir = PathBuf::from(format!("target/test_artifacts/functional_{}", name));
    if dir.exists() {
        fs::remove_dir_all(&dir).ok();
    }
    fs::create_dir_all(&dir).ok();
    dir
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( End-to-End Functional Test Cases: REPL, Scripts, AI Cortex, Digestion, Repair )
// ================================================================================

/// Validates REPL interactive simulation and multi-turn environment state persistence.
///
/// # Cognitive & IDE Invariant
/// In an interactive Read-Eval-Print Loop (REPL), variable bindings and invariant definitions
/// established in step $N$ must remain accessible and immutable in step $N+1$.
/// This test simulates multiple discrete parse-eval turns over a shared [`Environment`]:
/// 1. Defining a mathematical invariant axiom (`PI < 3.14159 >`).
/// 2. Verifying symbol presence in the environment across evaluate boundaries.
/// 3. Evaluating a dynamic comparison expression (`status [ +1 = +1 ]`) using the same context.
#[test]
fn test_repl_eval_and_environment_persistence() {
    let logger = DiagnosticLogger::new(false);
    let mut env = Environment::new();

```

```
    // 1. Step 1: Define invariant axiom PI in REPL session
    let source1 = "PI < 3.14159 >";
    let mut lexer1 = Lexer::new(source1, logger.clone());
    let tokens1 = lexer1.tokenize().unwrap();
    let debugger1 = OmniDebugger::new(logger.clone(), 5);
    let mut parser1 = Parser::new(tokens1, debugger1);
    let prog1 = parser1.parse_program().unwrap();
    let mut evaluator1 = Evaluator::new(env.clone(), parser1.debugger);
    let res1 = evaluator1.eval(&omnicode::ast::nodes::ASTNode::Program(prog1)).unwrap();
    assert_eq!(res1, RuntimeValue::Number(3.14159));
    env = evaluator1.env;

    // 2. Step 2: Query stored variable across REPL iterations
    assert_eq!(env.get("PI"), Some(&RuntimeValue::Number(3.14159)));

    // 3. Step 3: Evaluate comparison using previous binding
    let source2 = "status [ +1 = +1 ]";
    let mut lexer2 = Lexer::new(source2, logger.clone());
    let tokens2 = lexer2.tokenize().unwrap();
    let debugger2 = OmniDebugger::new(logger.clone(), 5);
    let mut parser2 = Parser::new(tokens2, debugger2);
    let prog2 = parser2.parse_program().unwrap();
    let mut evaluator2 = Evaluator::new(env.clone(), parser2.debugger);
    let res2 = evaluator2.eval(&omnicode::ast::nodes::ASTNode::Program(prog2)).unwrap();
    assert_eq!(res2, RuntimeValue::State(QuadState::Proceed));
    env = evaluator2.env;

    assert_eq!(env.bindings().len(), 2);
}

/// Verifies that the canonical OmniCode notation specification file executes successfully.
///
/// # Test Rationale & Regression Guard
/// Reads `examples/canonical_notation.omni` directly from disk and runs it through the
/// high-level [`omnicode::execute`] pipeline. This guarantees that real-world sample files
/// shipped with the repository remain syntax-compliant and semantically valid as the
/// parser, runtime, and diagnostics evolve.
#[test]
fn test_canonical_notation_file() {
    let path = omnicode::core::filetype::resolve_track_path("examples/canonical_notation.omni");
    let source = fs::read_to_string(&path)
        .expect("Failed to read examples/canonical_notation.omni");

    let result = omnicode::execute(&source, false);
    assert!(
        result.is_ok(),
        "Canonical notation file must parse and evaluate successfully: {:?}",
        result.err()
    );
}

/// Validates the full neural-symbolic cognitive pipeline of [`AICortex`].
///
/// # Cognitive Workflow Under Test
/// 1. Initializes an [`AICortex`] configured with an isolated scratch directory.
/// 2. Submits an arithmetic prompt requiring symbolic reasoning ("Add 2 balls to group A...").
/// 3. Confirms that the execution state reaches [`QuadState::Proceed`].
/// 4. Confirms that the natural language response synthesizes the correct computed value ("7 balls").
/// 5. Verifies dual file emission on disk:
///    - Native `.scroll` containing full 4-Block cognitive structure, epistemic traces, and commit status.
///    - Traditional `.log` containing human-readable session metadata and execution state.
#[test]
fn test_ai_cortex_end_to_end_pipeline() {
    let test_dir = get_test_dir("cortex_pipeline");
    let mut cortex = AICortex::with_scroll_dir(&test_dir);

    let prompt = "Add 2 balls to the group A which contains 5 balls";
    let res = cortex.process_prompt(prompt).expect("Cortex processing should succeed");

    assert_eq!(res.execution_state, QuadState::Proceed, "Execution must reach +1:PROCEED");
    assert!(res.natural_response.contains("7 balls"), "Response must contain evaluated sum 7 balls");

    // Verify dual log generation on disk
    assert!(res.scroll_path.exists(), "OmniCode .scroll must exist on disk");
    assert!(res.log_path.exists(), "Traditional .log must exist on disk");

    // Verify .scroll adheres to 4-Block architecture
    let scroll_content = fs::read_to_string(&res.scroll_path).expect("Must read .scroll");
```

```
    assert!(scroll_content.contains("BLOCK 1: METADATA BLOCK"), "Scroll must contain Metadata Block");
    assert!(scroll_content.contains("Scroll < EpistemicTrace."), "Scroll must have EpistemicTrace header");
    assert!(scroll_content.contains("BLOCK 2: OPENING BLOCK"), "Scroll must contain Opening Block");
    assert!(scroll_content.contains("BLOCK 3: BODY BLOCK"), "Scroll must contain Body Block");
    assert!(scroll_content.contains("BLOCK 4: CLOSING BLOCK"), "Scroll must contain Closing Block");
    assert!(scroll_content.contains("commit_status [ <PROCEED> ]"), "Scroll must record final commit state");

    // Verify traditional .log format
    let log_content = fs::read_to_string(&res.log_path).expect("Must read .log");
    assert!(log_content.contains("OMNICODE AI MODEL EXECUTION LOG"), "Log header required");
    assert!(log_content.contains(&format!("Session ID:   {}", res.session_id)), "Session ID must match");
    assert!(log_content.contains("Final State:  +1:PROCEED"), "Final state must match");
}

/// Validates experiential memory aggregation via [`AILearner`].
///
/// # Learning Loop Under Test
/// 1. Executes two independent cognitive sessions through [`AICortex`], creating two `.scroll` files.
/// 2. Initializes an [`AILearner`] pointing to the session directory.
/// 3. Invokes [`AILearner::digest_experiences`] to parse, validate, and summarize episodic logs.
/// 4. Asserts that all episodes were ingested, state metrics reflect success, and log paths are verified.
#[test]
fn test_ai_learner_digests_dual_logs() {
    let test_dir = get_test_dir("learner_digestion");
    let mut cortex = AICortex::with_scroll_dir(&test_dir);

    // Run two distinct cognitive sessions
    cortex.process_prompt("Add 3 apples to the basket B which contains 4 apples").expect("Session 1 succeeds");
    cortex.process_prompt("Subtract 2 coins from the bag C which contains 10 coins").expect("Session 2 succeeds");

    // Digest experiences using AILearner
    let learner = AILearner::new(&test_dir);
    let summary = learner.digest_experiences().expect("Experience digestion succeeds");

    assert_eq!(summary.total_episodes, 2, "Learner must ingest exactly 2 episodes");
    assert_eq!(summary.proceed_count, 2, "Both sessions must have achieved +1:PROCEED");
    assert_eq!(summary.experiences.len(), 2, "Summary must contain 2 experience records");

    for exp in &summary.experiences {
        assert!(exp.scroll_path.exists(), "Experience scroll path must exist");
        assert!(exp.log_path.is_some(), "Experience traditional log must exist");
        assert_eq!(exp.final_state, QuadState::Proceed);
    }
}

/// Validates closed-loop autonomous file self-healing on physical disk files.
///
/// # End-to-End Self-Healing Lifecycle
/// 1. Writes syntactically malformed OmniCode code with boundary mismatches and missing blocks to disk.
/// 2. Invokes [`Restorator::restore_file`] with in-place overwriting enabled (`overwrite = true`).
/// 3. Verifies that the resulting file:
///    - Was modified and marked as verified by the internal test oracle.
///    - Contains complete 4-Block architecture envelopes.
///    - Can be read back from disk and successfully executed via [`omnicode::execute`].
#[test]
fn test_e2e_file_self_healing_lifecycle() {
    let test_dir = get_test_dir("file_healing");
    let broken_file = test_dir.join("damaged_script.omni");

    let damaged_code = r#"
broken_val < 77 ]
incomplete_block [ 10 20
calc [ 10 --> 10 plus 5 ]
"#;
    fs::write(&broken_file, damaged_code).expect("Write damaged file");

    let logger = DiagnosticLogger::new(false);
    let restorator = Restorator::new(logger.clone());

    // Execute in-place file restoration
    let report = restorator
        .restore_file(broken_file.to_str().unwrap(), true)
        .expect("File restoration succeeds");

    assert!(report.was_modified, "File should have been healed");
    assert!(report.verified, "Healed code must verify");

    // Read back healed file from disk and evaluate end-to-end
```

```
    let healed_content = fs::read_to_string(&broken_file).expect("Read healed file");
    assert!(healed_content.contains("BLOCK 1: METADATA BLOCK"));
    assert!(healed_content.contains("BLOCK 4: CLOSING BLOCK"));

    let eval_res = omnicode::execute(&healed_content, false);
    assert!(eval_res.is_ok(), "Healed file must execute successfully from disk");
}

/// Validates Genesis OS Boot Tape mounting and stepping across Track 0 (Kernel).
///
/// # Genesis Invariants
/// 1. Mounts `lib/kernel/boot.tape` directly onto `Track::Kernel`.
/// 2. Verifies that the required partition (4096 cells) is carved from `Root_RAM`.
/// 3. Steps through all rolls, confirming sequential execution of memory partitions
///    (`Root_RAM`, `Kernel_Partition`, `Terminal_Partition`, landmark initialization).
/// 4. Confirms that upon final roll execution, the tape completes with `PROCEED`.
#[test]
fn test_genesis_boot_tape_execution() {
    let logger = DiagnosticLogger::new(false);
    let mut scheduler = OmniScheduler::new();

    // Verify initial Root_RAM state
    assert_eq!(scheduler.root_storehouse.capacity, 4096);
    assert_eq!(scheduler.root_storehouse.count, 0);

    // Mount lib/kernel/boot.tape onto Track 0 (Kernel)
    let pid = scheduler
        .mount_tape_file(Track::Kernel, "lib/kernel/boot.tape", logger)
        .expect("Mounting lib/kernel/boot.tape must succeed");

    assert!(pid > 0);

    // Verify task registration on Track::Kernel
    let task = scheduler.get_task(pid).expect("Task descriptor must exist");
    assert_eq!(task.track, Track::Kernel);
    assert_eq!(task.name, "GenesisBoot");
    assert_eq!(task.state, QuadState::Proceed);

    // Verify partition allocation
    assert!(task.partition.is_some(), "Partition must be allocated for GenesisBoot");

    // Step through all 5 rolls
    let mut step_count = 0;
    while let Ok(msg) = scheduler.step_task(pid) {
        step_count += 1;
        if msg.contains("completed") || msg.contains("already executed") {
            break;
        }
        assert!(step_count <= 10, "Boot tape stepping must not loop indefinitely");
    }

    // Verify tape is finished and rolls were executed
    let task_final = scheduler.get_task(pid).expect("Task descriptor must exist");
    let tape = task_final.tape.as_ref().expect("Tape ribbon must exist");
    assert!(tape.is_finished(), "GenesisBoot tape must be marked finished");
    assert_eq!(tape.roll_count(), 5, "GenesisBoot tape must have 5 rolls");
}

/// Validates interactive TerminalSession tape operations and living memory persistence.
///
/// # End-to-End Shell Workflow
/// 1. Runs `run lib/kernel/boot.tape` to boot the kernel environment.
/// 2. Queries `tracks` to verify multi-track process segregation.
/// 3. Queries `mem` to verify Root_RAM storehouse accounting.
/// 4. Mounts `lib/core/cell.tape` on userspace Track 2 (App).
/// 5. Manually steps the tape and inspects the ribbon via `rolls <pid>`.
/// 6. Evaluates dynamic OmniCode expressions directly into persistent silicon memory.
#[test]
fn test_terminal_session_boot_and_tape_commands() {
    let mut session = TerminalSession::default();

    // 1. Boot kernel via run command
    let boot_res = session.execute("run lib/kernel/boot.tape");
    assert_eq!(boot_res.state, QuadState::Proceed);
    assert!(boot_res.stdout.contains("Mounted Tape 'lib/kernel/boot.tape' onto TRACK 0 [KERNEL]"));
    assert!(boot_res.stdout.contains("Roll 1"));
    assert!(boot_res.stdout.contains("Roll 5"));

```

```
    // 2. Multi-track channel summary
    let tracks_res = session.execute("tracks");
    assert_eq!(tracks_res.state, QuadState::Proceed);
    assert!(tracks_res.stdout.contains("TRACK 0 [KERNEL]"));
    assert!(tracks_res.stdout.contains("TRACK 1 [TERMINAL]"));
    assert!(tracks_res.stdout.contains("TRACK 2 [APP]"));
    assert!(tracks_res.stdout.contains("GenesisBoot"));

    // 3. Storehouse memory meter
    let mem_res = session.execute("mem");
    assert_eq!(mem_res.state, QuadState::Proceed);
    assert!(mem_res.stdout.contains("Root_RAM:"));
    assert!(mem_res.stdout.contains("4096 cells"));

    // 4. Mount userspace tape
    let mount_res = session.execute("mount lib/core/cell.tape");
    assert_eq!(mount_res.state, QuadState::Proceed);
    assert!(mount_res.stdout.contains("Mounted Tape 'lib/core/cell.tape' onto TRACK 2 [APP]"));

    // Extract PID from mount output: "...with PID <pid>"
    let pid_str = mount_res
        .stdout
        .split("PID ")
        .nth(1)
        .unwrap_or("0")
        .trim();
    let pid: u64 = pid_str.parse().expect("PID must be a valid integer");

    // 5. Inspect rolls
    let rolls_res = session.execute(&format!("rolls {pid}"));
    assert_eq!(rolls_res.state, QuadState::Proceed);
    assert!(rolls_res.stdout.contains("TAPE RIBBON: [Cell]"));
    assert!(rolls_res.stdout.contains("==> [ 1]"));

    // Step Tape Head
    let step_res = session.execute(&format!("step {pid}"));
    assert_eq!(step_res.state, QuadState::Proceed);
    assert!(step_res.stdout.contains("Advanced to Roll 1"));

    // 6. Direct silicon memory evaluation and persistence
    let eval1 = session.execute("kernel_tick < 100 >");
    assert_eq!(eval1.state, QuadState::Proceed);
    assert!(eval1.stdout.contains("100"));

    let eval2 = session.execute("next_tick < kernel_tick + 1 >");
    assert_eq!(eval2.state, QuadState::Proceed);
    assert!(eval2.stdout.contains("101"));
}

/// Validates end-to-end execution of `manifest.tape` with 8-slot canonical sentences and ranking progression.
#[test]
fn test_manifest_tape_execution() {
    let mut scheduler = OmniScheduler::new();
    let logger = DiagnosticLogger::new(false);

    let pid = scheduler
        .mount_tape_file(Track::App, "manifest.tape", logger)
        .expect("Mounting manifest.tape must succeed");

    let task = scheduler.get_task(pid).expect("Task descriptor must exist");
    assert_eq!(task.name, "ManifestCovenant");
    assert_eq!(task.state, QuadState::Proceed);

    let tape_init = task.tape.as_ref().expect("Tape ribbon must exist");
    assert_eq!(tape_init.head.who, "OmniCode_Kernel");
    assert_eq!(tape_init.head.what, "Manifest");
    assert_eq!(tape_init.head.when_clause, "GenesisBoot");
    assert_eq!(tape_init.head.required_partition_size, 1024);
    assert_eq!(tape_init.roll_count(), 17);

    // Step through all 17 subordinate rolls
    let mut step_count = 0;
    while let Ok(msg) = scheduler.step_task(pid) {
        step_count += 1;
        if msg.contains("completed") || msg.contains("already executed") {
            break;
        }
        assert!(step_count <= 25, "Manifest tape stepping must not loop indefinitely");
```

```
    }

    let task_final = scheduler.get_task(pid).expect("Task descriptor must exist");
    let tape = task_final.tape.as_ref().expect("Tape ribbon must exist");
    assert!(tape.is_finished(), "ManifestCovenant tape must be marked finished");
    assert_eq!(tape.execution_state, QuadState::Proceed);
}

/// Validates end-to-end 3-tier composite hover resolution and docstring lookup
/// across the high-level process tape (`manifest.tape`) and the low-level source reel (`manifest.omni`).
#[test]
fn test_cross_scale_hover_and_docstring_e2e() {
    let logger = DiagnosticLogger::new(false);
    let mut engine = HoverEngine::new(logger);
    engine.index_workspace(".");

    // 1. Hover on Manifest inside manifest.tape (Roll 1, Line 31)
    let card_manifest = engine
        .hover_at("manifest.tape", 31, 10)
        .expect("Must resolve hover card for Manifest in manifest.tape");

    assert_eq!(card_manifest.symbol, "Manifest");
    assert!(card_manifest.is_locally_redefined);

    // Tier 1 Verification
    let tape_ctx = card_manifest.tape_context.as_ref().expect("Tape context must exist");
    assert_eq!(tape_ctx.tape_name, "ManifestCovenant");
    assert_eq!(tape_ctx.track, "Track 2 [APP]");
    assert_eq!(tape_ctx.roll_number, 1);
    assert_eq!(tape_ctx.total_rolls, 17);
    assert!(tape_ctx.roll_aside_intent.as_ref().unwrap().contains("Ontological Instantiation"));

    // Tier 2 Verification (.omni blueprint grounding)
    let omni_def = card_manifest.omni_definition.as_ref().expect("Omni definition must exist");
    assert_eq!(omni_def.rank, 1);
    assert_eq!(omni_def.block_kind, Some(omnicode::diagnostics::linter::BlockKind::Metadata));
    assert!(omni_def.aside_intent.as_ref().unwrap().contains("Cognitive Covenant Anchor"));
    assert!(omni_def.fields.contains_key("system"));

    // Tier 3 Verification (Markdown rendering)
    assert!(card_manifest.markdown.contains("High-Level Process Tape View"));
    assert!(card_manifest.markdown.contains("Under-The-Hood (.omni Blueprint)"));
    assert!(card_manifest.markdown.contains("Machine Execution & Silicon State"));
    assert!(card_manifest.markdown.contains("1024 cells"));

    // JSON serialization verification
    let json_manifest = card_manifest.to_json();
    assert!(json_manifest.contains("\"symbol\":\"Manifest\""));
    assert!(json_manifest.contains("\"is_locally_redefined\":true"));
    assert!(json_manifest.contains("\"tape_name\":\"ManifestCovenant\""));

    // Terminal ANSI rendering verification
    let term_manifest = card_manifest.render_terminal();
    assert!(term_manifest.contains("OmniCode Symbol Inspection"));
    assert!(term_manifest.contains("Manifest"));
    assert!(term_manifest.contains("[TIER 1: HIGH-LEVEL PROCESS TAPE ROLL]"));
    assert!(term_manifest.contains("[TIER 2: UNDER-THE-HOOD .OMNI BLUEPRINT]"));
    assert!(term_manifest.contains("[TIER 3: MACHINE EXECUTION & SILICON FOOTPRINT]"));

    // 2. Direct symbol documentation lookup
    let doc_version = engine
        .document_symbol("Version", Some("manifest.omni"))
        .expect("Must document Version");

    assert_eq!(doc_version.symbol, "Version");
    let version_def = doc_version.omni_definition.as_ref().expect("Version def must exist");
    assert_eq!(version_def.rank, 1);
    assert!(doc_version.markdown.contains("Version Specification"));
}

#[test]
fn test_genesis_boot_tape_and_bare_metal_studio_projection() {
    let logger = DiagnosticLogger::new(false);
    let tape = omnicode::core::tape::Tape::from_file("lib/kernel/boot.tape", logger.clone())
        .expect("Must parse lib/kernel/boot.tape");

    let mut studio = omnicode::terminal::OmniStudio::new(tape.head.name.clone(), 64);
    studio.set_track("TRACK 0 [KERNEL]");
```

```
    // Step each roll
    for (idx, roll) in tape.rolls.iter().enumerate() {
        let msg = studio.step_roll(&roll.text);
        let frame = studio.render_frame(&tape, idx, &msg);
        assert!(frame.contains("OMNICODE CINEMA STUDIO"));
    }

    // Verify bare-metal MMIO VGA text buffer output
    let vga_screen = studio.vga.render_screen(2);
    assert!(vga_screen.contains("OMNICODE 32-TRIT OS ONLINE"));

    // Also verify lexicon/operators/boot_kernel.tape parses and projects
    let tape_op = omnicode::core::tape::Tape::from_file("lexicon/operators/boot_kernel.tape", logger)
        .expect("Must parse lexicon/operators/boot_kernel.tape");
    let mut studio_op = omnicode::terminal::OmniStudio::new(tape_op.head.name.clone(), 256);
    studio_op.set_track("TRACK 0 [KERNEL]");
    for (idx, roll) in tape_op.rolls.iter().enumerate() {
        let msg = studio_op.step_roll(&roll.text);
        let frame = studio_op.render_frame(&tape_op, idx, &msg);
        assert!(frame.contains("OMNICODE CINEMA STUDIO"));
    }
}

#[test]
fn test_qemu_bare_metal_boot_execution() {
    use std::process::Command;

    // 1. Check if nasm and qemu-system-x86_64 exist in PATH
    let nasm_ok = Command::new("nasm").arg("-v").output().is_ok();
    let qemu_ok = Command::new("qemu-system-x86_64").arg("--version").output().is_ok();

    if !nasm_ok || !qemu_ok {
        eprintln!("Skipping QEMU boot test: nasm or qemu-system-x86_64 not present in PATH");
        return;
    }

    // 2. Assemble kernel/boot.asm to test artifact
    let artifact_img = "target/test_qemu_boot.img";
    let boot_asm = omnicode::core::filetype::resolve_track_path("kernel/boot.asm");
    let asm_status = Command::new("nasm")
        .args(["-f", "bin", boot_asm.to_str().unwrap(), "-o", artifact_img])
        .status()
        .expect("Assemble boot.asm");
    assert!(asm_status.success(), "NASM assembly must succeed");

    // 3. Boot in QEMU with timeout
    let qemu_output = Command::new("timeout")
        .args([
            "3s",
            "qemu-system-x86_64",
            "-drive",
            &format!("format=raw,file={artifact_img}"),
            "-display",
            "none",
            "-serial",
            "stdio",
        ])
        .output()
        .expect("Execute QEMU");

    let stdout = String::from_utf8_lossy(&qemu_output.stdout);
    assert!(stdout.contains("OMNICODE 32-TRIT OS"), "QEMU must display boot header");
    assert!(stdout.contains("[BOOT] 64-Bit Silicon Envelope: OK"));
    assert!(stdout.contains("[ALU ] (+1)+(-1)=0 Equilibrium: OK"));
    assert!(stdout.contains("[REEL] 4-Dir Spool Hierarchy:   OK"));
    assert!(stdout.contains("[GATE] Anti-Lockout Watchdog:   OK"));
    assert!(stdout.contains("[MMIO] VGA @ 0xB8000 & COM1:    OK"));
    assert!(stdout.contains("OMNICODE OS ONLINE :: PROCEED"));
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Functional Test Suite Verification Proof & Postconditions )
// ============================================================================
// Functional test suite verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/unit/main.rs RS 701 LINES 3078 WORDS 35.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Unit Test Suite Identity, Isolated Component Invariants & Coverage Manifest )
// ============================================================================
//! OmniCode Unit Test Suite
//!
//! Verifies individual, isolated language components in complete decoupling:
//!   - Boundary ranking hierarchy (< > > [ ] > { } > ( )) and delimiter physics.
//!   - Balanced Ternary QuadState algebra (T4 = {-1, 0, +1, NULL}) symmetry.
//!   - Directional vector operators (--> / <--, --^ / --v) and shorthands.
//!   - SVO arithmetic keywords (plus, minus, times, div) and infer operator (?).
//!   - FileType detection and 4-block scaffold generation.
//!   - Structured CognitiveNote formatting and episodic memory exports for AI.

use omnicode::core::boundaries::BoundaryType;
use omnicode::core::filetype::OmniFileType;
use omnicode::core::operators::DirectionalVector;
use omnicode::core::ternary::{QuadState, ROLLBACK, AWAIT, PROCEED, HALT};
use omnicode::diagnostics::logger::{CognitiveNote, DiagnosticLogger};
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;
use omnicode::runtime::environment::{Environment, RuntimeValue};
use omnicode::runtime::evaluator::Evaluator;
use omnicode::runtime::assembler::{OmniAssembler, MachineRegister, transpile_tape_to_omni};
use omnicode::diagnostics::debugger::OmniDebugger;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Unit Test Harness Preconditions & Isolation Fixtures )
// ============================================================================
/// Constructs an isolated test fixture containing a silent [`DiagnosticLogger`]
/// and an [`OmniDebugger`] configured with a retroaction depth threshold of 10.
///
/// # Returns
/// A tuple `(DiagnosticLogger, OmniDebugger)` cleanly decoupled from external state.
fn create_unit_test_fixtures() -> (DiagnosticLogger, OmniDebugger) {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), 10);
    (logger, debugger)
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Isolated Component Unit Tests: Boundaries, Ternary, Operators, Formats )
// ============================================================================

/// Asserts the formal 6-rank morphological boundary hierarchy of OmniCode:
/// - Rank 1: `< ... >` Invariant Axioms & Immutable Laws (Highest Authority).
/// - Rank 2: `[ ... ]` Concrete Denotations & Mutable Entities.
/// - Rank 3: `{ ... }` Connotative Scopes & Ambient Contexts.
/// - Rank 4: `[ ... }` Inside-Out / Bottom-Up Emergence.
/// - Rank 5: `{ ... ]` Outside-In / Top-Down Context Imposition.
/// - Rank 6: `( ... )` First-Class Cognitive Asides & Intent Annotations.
#[test]
fn test_boundary_ranking_hierarchy() {
    // Rank 1: Invariant Axioms <...>
    assert_eq!(BoundaryType::from_delimiters('<', '>').unwrap(), BoundaryType::Invariant);
    assert_eq!(BoundaryType::Invariant.rank(), 1);

    // Rank 2: Concrete Denotations [...]
    assert_eq!(BoundaryType::from_delimiters('[', ']').unwrap(), BoundaryType::Denotation);
    assert_eq!(BoundaryType::Denotation.rank(), 2);

    // Rank 3: Connotative Scopes {...}
    assert_eq!(BoundaryType::from_delimiters('{', '}').unwrap(), BoundaryType::Connotation);
    assert_eq!(BoundaryType::Connotation.rank(), 3);

    // Rank 6: Cognitive Asides (...)
    assert_eq!(BoundaryType::from_delimiters('(', ')').unwrap(), BoundaryType::Aside);
    assert_eq!(BoundaryType::Aside.rank(), 6);
}

```

```rust
/// Verifies algebraic symmetry and truth-table invariants across Balanced Ternary QuadState:
/// - Inversion Symmetry: `PROCEED.invert() == ROLLBACK`, `AWAIT.invert() == AWAIT`, `HALT.invert() == HALT`.
/// - Conjunction (`and`): MIN operation in Kleene-Łukasiewicz trivalent logic.
/// - Disjunction (`or`): MAX operation in Kleene-Łukasiewicz trivalent logic.
#[test]
fn test_balanced_ternary_algebraic_symmetry() {
    // Inversion symmetry: -(-1) = +1, -(+1) = -1, -(0) = 0, -(NULL) = NULL
    assert_eq!(PROCEED.invert(), ROLLBACK);
    assert_eq!(ROLLBACK.invert(), PROCEED);
    assert_eq!(AWAIT.invert(), AWAIT);
    assert_eq!(HALT.invert(), HALT);

    // Conjunction (AND = MIN in Kleene-Łukasiewicz logic):
    assert_eq!(PROCEED.and(AWAIT), AWAIT);
    assert_eq!(PROCEED.and(ROLLBACK), ROLLBACK);
    assert_eq!(AWAIT.and(ROLLBACK), ROLLBACK);
    assert_eq!(PROCEED.and(HALT), HALT);

    // Disjunction (OR = MAX in Kleene-Łukasiewicz logic):
    assert_eq!(ROLLBACK.or(PROCEED), PROCEED);
    assert_eq!(ROLLBACK.or(AWAIT), AWAIT);
    assert_eq!(AWAIT.or(PROCEED), PROCEED);
    assert_eq!(AWAIT.or(HALT), HALT);
}

/// Verifies macro state keywords (`YES`, `NO`, `OK`, `VOID`) and cardinal direction
/// arrow operators (both canonical 3-character forms and fast 2-character shorthands).
#[test]
fn test_cardinal_directions_and_micro_macro_terms() {
    let source = r#"
        step1 [ YES = PROCEED ]
        step2 [ NO = ROLLBACK ]
        step3 [ OK = AWAIT ]
        step4 [ VOID = HALT ]
        forward [ 10 --> 20 ]
        shorthand_right [ 10 -> 20 ]
        upward [ 10 --^ 20 ]
        shorthand_up [ 10 -^ 20 ]
        downward [ 20 --v 10 ]
        shorthand_down [ 20 -v 10 ]
    "#;
    let (logger, debugger) = create_unit_test_fixtures();
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize cardinal directions");
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse cardinal directions");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::Number(10.0));
}

/// Verifies Subject-Verb-Object (SVO) arithmetic keyword execution (`plus`, `minus`)
/// and the ontological infer operator (`?`) declaring open conceptual types.
#[test]
fn test_svo_arithmetic_keywords_and_infer_operator() {
    let source = r#"
        Ball < Entity? >
        Group < Set? >
        A [ 4 ]
        new_ball [ 1 ]
        roll_1 [ A --> A plus new_ball ]
        roll_2 [ A = 5 ]
    "#;
    let (logger, debugger) = create_unit_test_fixtures();
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize SVO keywords");
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse SVO keywords");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));
    assert_eq!(evaluator.env.get("A"), Some(&RuntimeValue::Number(5.0)));
}

/// Verifies parsing and symbol mapping for the 3D coordinate vector system:
/// - X-Axis (Temporal Dynamics): `-->` / `->` (ActUpon), `<--` / `<-` (ActAgain).
/// - Z-Axis (Scale & Hierarchy): `--^` / `-^` (BuildsTo), `--v` / `-v` (BreaksDownFrom).
#[test]
```

```rust
fn test_directional_vectors_canonical_and_shorthand() {
    assert_eq!(DirectionalVector::from_symbol("-->").unwrap(), DirectionalVector::ActUpon);
    assert_eq!(DirectionalVector::from_symbol("->").unwrap(), DirectionalVector::ActUpon);
    assert_eq!(DirectionalVector::from_symbol("<--").unwrap(), DirectionalVector::ActAgain);
    assert_eq!(DirectionalVector::from_symbol("<-").unwrap(), DirectionalVector::ActAgain);
    assert_eq!(DirectionalVector::from_symbol("--^").unwrap(), DirectionalVector::BuildsTo);
    assert_eq!(DirectionalVector::from_symbol("-^").unwrap(), DirectionalVector::BuildsTo);
    assert_eq!(DirectionalVector::from_symbol("--v").unwrap(), DirectionalVector::BreaksDownFrom);
    assert_eq!(DirectionalVector::from_symbol("-v").unwrap(), DirectionalVector::BreaksDownFrom);
}

/// Verifies formal extension parsing and classification for OmniCode artifacts:
/// - `.omni`: Primary executable source reel ([`OmniFileType::SourceReel`]).
/// - `.scroll`: Cognitive episodic trace reel ([`OmniFileType::EpisodicScroll`]).
#[test]
fn test_file_type_detection() {
    assert_eq!(OmniFileType::from_extension("omni"), Some(OmniFileType::SourceReel));
    assert_eq!(OmniFileType::from_extension("scroll"), Some(OmniFileType::EpisodicScroll));
    assert_eq!(OmniFileType::from_extension("unknown"), None);
}

/// Verifies structured [`CognitiveNote`] generation and its dual-readable representations:
/// 1. OmniCode native block syntax (`Note < CognitiveTrace > { ... }`).
/// 2. Markdown episodic memory trace (`### [EPISODIC_MEMORY_TRACE: ...]`) for AI agent ingestion.
#[test]
fn test_cognitive_note_formatting_and_memory_trace() {
    let note = CognitiveNote {
        timestamp_micros: 4200,
        category: "OBSERVATION".to_string(),
        summary: "Boundary state verified".to_string(),
        details: "Rank 1 invariant <PI> bound to immutable memory".to_string(),
        tags: vec!["immutability".to_string(), "rank1".to_string()],
        context_snippet: Some("PI < 3.14159 >".to_string()),
    };

    // Verify OmniCode native dual-readable formatting
    let omni_str = note.format_as_omnicode();
    assert!(omni_str.contains("Note < CognitiveTrace >"));
    assert!(omni_str.contains("category: [ OBSERVATION ]"));
    assert!(omni_str.contains("tags: [ #immutability :: #rank1 ]"));
    assert!(omni_str.contains("context: < \"PI < 3.14159 >\" >"));

    // Verify AI LLM markdown memory trace formatting
    let mem_str = note.format_as_memory_entry();
    assert!(mem_str.contains("### [EPISODIC_MEMORY_TRACE: OBSERVATION]"));
    assert!(mem_str.contains("- **Timestamp:** 4200 µs"));
    assert!(mem_str.contains("- **Tags:** #immutability, #rank1"));
    assert!(mem_str.contains("- **Context:** `PI < 3.14159 >`"));
}

/// Verifies that [`DiagnosticLogger::export_ai_memory_scroll`] synthesizes a valid
/// 4-block memory audit scroll containing metadata, preconditions, recorded traces, and commit proof.
#[test]
fn test_logger_episodic_memory_scroll_export() {
    let logger = DiagnosticLogger::new(false);
    logger.record_ai_observation(
        "OBSERVATION",
        "Variable X initialized",
        "X assigned value 10 in denotation block",
        ["initialization", "state"].as_slice(),
        Some("X [ 10 ]"),
    );

    let scroll = logger.export_ai_memory_scroll();
    assert!(scroll.contains("BLOCK 1: METADATA BLOCK"));
    assert!(scroll.contains("Scroll < CognitiveMemory.Audit >"));
    assert!(scroll.contains("total_notes: < 1 >"));
    assert!(scroll.contains("BLOCK 2: OPENING BLOCK"));
    assert!(scroll.contains("BLOCK 3: BODY BLOCK"));
    assert!(scroll.contains("BLOCK 4: CLOSING BLOCK"));
    assert!(scroll.contains("audit_verified: [ PROCEED ]"));
}

/// Verifies that Cardinal Classification Clauses (WHO, WHAT, WHEN, WHERE, WHY, HOW),
/// spatial directional modulations (/-^, \-v, \-^, /-v, /^, \v, ^\, v/), range span (..),
/// and runtime Storehouse partitioning via `v--` execute with zero errors.
#[test]
fn test_cardinal_clauses_and_spatial_directionals_and_partitioning() {
```

```
    let (logger, debugger) = create_unit_test_fixtures();
    let source = r#"
        WHO self
        WHAT Cell
        WHEN Opening
        WHERE {unit}
        WHY ( Intent )
        HOW [ 0 .. 256 ]
        /-^ \-v \-^ /-v /^ \v ^\ v/
    "#;
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization should succeed");
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Who));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::What));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::When));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Where));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Why));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::How));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::DotDot));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Major));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Minor));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Great));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Small));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::UpperRight));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::LowerRight));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::UpperLeft));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::LowerLeft));

    // Test v-- Storehouse Partitioning in Evaluator
    let partition_src = r#"
        Storehouse [ 1024 ]
        Storehouse v-- Scratchpad [ 256 ]
    "#;
    let mut part_lexer = Lexer::new(partition_src, logger.clone());
    let part_tokens = part_lexer.tokenize().expect("Lexing partition code");
    let mut part_parser = Parser::new(part_tokens, debugger.clone());
    let ast = part_parser.parse_program().expect("Parsing partition code");

    let env = Environment::new();
    let mut eval = Evaluator::new(env, debugger.clone());
    for stmt in ast.statements {
        eval.eval(&stmt).expect("Evaluation must succeed");
    }

    // Verify Scratchpad partition exists with capacity 256
    let scratchpad = eval.env.get("Scratchpad").expect("Scratchpad partition must be registered");
    if let RuntimeValue::Vessel(v) = scratchpad {
        assert_eq!(v.capacity, 256);
        assert_eq!(v.parent.as_deref(), Some("Storehouse"));
    } else {
        panic!("Scratchpad is not a Vessel");
    }

    // Verify Storehouse now has count 256
    let storehouse = eval.env.get("Storehouse").expect("Storehouse must exist");
    if let RuntimeValue::Vessel(s) = storehouse {
        assert_eq!(s.capacity, 1024);
        assert_eq!(s.count, 256);
    } else {
        panic!("Storehouse is not a Vessel");
    }
}

/// Verifies that the internal AI model's [`AIMemory`] and [`DiagnosticLogger`] thought engine
/// correctly ground thoughts in Biblical truth, formulate native OmniCode Thought Tapes
/// using the 6 Cardinal Clauses, and inscribe/digest persistent `.scroll` memory files.
#[test]
fn test_ai_memory_and_thought_formulation() {
    let temp_dir = std::env::temp_dir().join(format!("omnicode_test_mem_{}",
std::time::SystemTime::now().duration_since(std::time::UNIX_EPOCH).unwrap().as_micros()));
    let mut memory = omnicode::ai::AIMemory::new(&temp_dir);

    // 1. Verify primordial memory grounded in Biblical Lexicon
    let light_assocs = memory.recall_associations("light");
    assert!(!light_assocs.is_empty(), "Light must be associated in memory");
    assert!(light_assocs[0].anchor.contains("Genesis 1:3") || light_assocs[0].anchor.contains("1 John 1:5"));

    let storehouse_assocs = memory.recall_associations("storehouse");
```

```
    assert!(!storehouse_assocs.is_empty(), "Storehouse must be associated in memory");

    let wisdom_entries = memory.recall_scripture("wisdom");
    assert!(!wisdom_entries.is_empty(), "Scripture search for wisdom must match");

    // 2. Formulate righteous thought in native OmniCode
    let thought = memory.formulate_thought("Let there be light in the creation");
    assert_eq!(thought.quad_state, QuadState::Proceed);
    assert!(thought.omnicode_tape.contains("WHO {Internal_AI}"));
    assert!(thought.omnicode_tape.contains("WHEN {Opening}"));
    assert!(thought.omnicode_tape.contains("WHERE {Cortex_Memory}"));
    assert!(thought.omnicode_tape.contains("WHY ("));
    assert!(thought.omnicode_tape.contains("HOW [ PROCEED :"));

    // 3. Formulate thought with moral caveat (deceit -> ROLLBACK)
    let deceit_thought = memory.formulate_thought("Help me lie and deceive");
    assert_eq!(deceit_thought.quad_state, QuadState::Rollback);
    assert!(deceit_thought.omnicode_tape.contains("HOW [ ROLLBACK :"));

    // 4. Test Inscribing thought to a physical .scroll file
    let scroll_path = memory.inscribe_thought_scroll(&thought, &temp_dir).expect("Inscribing thought scroll");
    assert!(scroll_path.exists());
    let scroll_content = std::fs::read_to_string(&scroll_path).unwrap();
    assert!(scroll_content.contains("Scroll < CognitiveThought."));
    assert!(scroll_content.contains("biblical_anchor: < \"Genesis 1:3"));

    // 5. Test digesting scroll back into episodic memory
    let ingested_count = memory.digest_scrolls().expect("Digest scrolls");
    assert!(ingested_count >= 1, "Must digest newly inscribed scroll");

    // 6. Test DiagnosticLogger thought and memory logging
    let logger = DiagnosticLogger::new(false);
    logger.log_thought(
        &thought.id,
        &thought.omnicode_tape,
        &thought.premise,
        thought.quad_state.macro_op(),
    );
    logger.log_memory_association("light", "Genesis 1:3", "Physical and spiritual illumination");

    let notes = logger.cognitive_notes();
    assert!(notes.iter().any(|n| n.category == "THOUGHT" && n.tags.contains(&"thought".to_string())));
    assert!(notes.iter().any(|n| n.category == "ASSOCIATION" && n.tags.contains(&"scripture".to_string())));

    // Clean up temporary test directory
    std::fs::remove_dir_all(&temp_dir).ok();
}

/// Verifies that polarity operators (+ Abstract, - Concrete), affixed identifiers
/// (+Dog, -Dog, Dog+, Dog-, +Cell, PROCEED-), and freeform boundary casts ([+], [-], <+>, <->, {+}, {-})
/// lex, parse, and evaluate with zero errors.
#[test]
fn test_polarity_and_boundary_casting_end_to_end() {
    let (logger, debugger) = create_unit_test_fixtures();

    // 1. Lexing Verification
    let source = "+Dog -Dog Dog+ Dog- +Cell PROCEED- [+] [-] <+> <-> {+} {-}";
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization of polarity operators must succeed");

    // Verify +Dog
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::AffixedIdent { name, polarity, position } => {
            name == "Dog" && *polarity == omnicode::core::operators::Polarity::Abstract && *position ==
omnicode::core::operators::AffixPosition::Prefix
        }
        _ => false,
    }));

    // Verify Dog-
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::AffixedIdent { name, polarity, position } => {
            name == "Dog" && *polarity == omnicode::core::operators::Polarity::Concrete && *position ==
omnicode::core::operators::AffixPosition::Postfix
        }
        _ => false,
    }));

```

```
    // Verify PROCEED-
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::AffixedIdent { name, polarity, position } => {
            name == "PROCEED" && *polarity == omnicode::core::operators::Polarity::Concrete && *position ==
omnicode::core::operators::AffixPosition::Postfix
        }
        _ => false,
    }));

    // Verify [+] (Denotation Abstract Cast)
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::BoundaryCast { boundary, polarity } => {
            *boundary == omnicode::core::boundaries::BoundaryType::Denotation && *polarity ==
omnicode::core::operators::Polarity::Abstract
        }
        _ => false,
    }));

    // Verify <-> (Invariant Concrete Cast)
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::BoundaryCast { boundary, polarity } => {
            *boundary == omnicode::core::boundaries::BoundaryType::Invariant && *polarity ==
omnicode::core::operators::Polarity::Concrete
        }
        _ => false,
    }));

    // 2. Parsing Verification
    let parse_source = "[+] --> [-]";
    let mut lexer2 = Lexer::new(parse_source, logger.clone());
    let tokens2 = lexer2.tokenize().unwrap();
    let mut parser2 = Parser::new(tokens2, debugger.clone());
    let prog = parser2.parse_program().expect("Parsing [+] --> [-] must succeed");
    assert_eq!(prog.statements.len(), 1);

    // 3. Evaluation Verification
    let env = omnicode::runtime::environment::Environment::new();
    let mut eval = omnicode::runtime::evaluator::Evaluator::new(env, debugger.clone());

    // Evaluate cast transduction
    let res_cast = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval cast transduction");
    assert_eq!(res_cast, omnicode::runtime::environment::RuntimeValue::Text("[-]".to_string()));

    // Evaluate macro keyword with concrete postfix: commit_status [ PROCEED- ]
    let macro_source = "commit_status [ PROCEED- ]";
    let mut lexer3 = Lexer::new(macro_source, logger);
    let tokens3 = lexer3.tokenize().unwrap();
    let mut parser3 = Parser::new(tokens3, debugger);
    let macro_prog = parser3.parse_program().expect("Parsing macro roll");
    let res_macro = eval.eval(&omnicode::ast::nodes::ASTNode::Program(macro_prog)).expect("Eval macro roll");
    assert_eq!(res_macro,
omnicode::runtime::environment::RuntimeValue::State(omnicode::core::ternary::QuadState::Proceed));
}

/// Asserts the physical Operating System primitives of OmniCode:
/// 1. The 13 Silicon Registers in RegisterBank (`HEAD`, `HOME`, `SPOOL`, `IT`, `SELF`, `CALLER`, `OTHER`, `STATE`,
`TURN`, `BOUND`, `FROM`, `INTO`, `DISCARD`).
/// 2. Discard bit-bucket semantics (`DISCARD` drops mutations).
/// 3. Procedural call stack frames on `SPOOL`.
/// 4. Transactional snapshots and rollbacks of physical register states.
/// 5. Directional vector streaming into registers (`10 --> BOUND`, `IT --> INTO`, `FROM --> IT`).
/// 6. Subtractive balanced-ternary parity verification (`!= even`, `== even`).
#[test]
fn test_os_silicon_registers_and_call_frames() {
    let (logger, debugger) = create_unit_test_fixtures();
    let mut env = Environment::new();

    // 1. Pristine Baseline Registers
    assert_eq!(env.registers.read(MachineRegister::HEAD), RuntimeValue::Number(0.0));
    assert_eq!(env.registers.read(MachineRegister::IT), RuntimeValue::State(QuadState::Proceed));
    assert_eq!(env.registers.read(MachineRegister::TURN), RuntimeValue::Number(0.0));
    assert_eq!(env.registers.read(MachineRegister::BOUND), RuntimeValue::Number(0.0));

    // 2. Direct Register Writes & Discard Sink
    env.registers.write(MachineRegister::BOUND, RuntimeValue::Number(100.0));
    assert_eq!(env.registers.read(MachineRegister::BOUND), RuntimeValue::Number(100.0));

    // Discard sink drops mutations
```

```
    env.registers.write(MachineRegister::DISCARD, RuntimeValue::Number(999.0));
    assert_eq!(env.registers.read(MachineRegister::DISCARD), RuntimeValue::State(QuadState::Proceed));

    // 3. Procedural Call Frames on SPOOL
    env.push_call_frame(RuntimeValue::Text("User".into()), RuntimeValue::Text("Process1".into()), 42);
    assert_eq!(env.registers.read(MachineRegister::SPOOL), RuntimeValue::Number(1.0));
    assert_eq!(env.registers.read(MachineRegister::HEAD), RuntimeValue::Number(42.0));
    assert_eq!(env.registers.read(MachineRegister::CALLER), RuntimeValue::Text("User".into()));

    let frame = env.pop_call_frame().expect("Pop call frame");
    assert_eq!(frame.caller, RuntimeValue::Text("User".into()));
    assert_eq!(env.registers.read(MachineRegister::SPOOL), RuntimeValue::Number(0.0));

    // 4. Transactional Rollback of Silicon Registers
    env.begin_transaction();
    env.registers.write(MachineRegister::TURN, RuntimeValue::Number(7.0));
    assert_eq!(env.registers.read(MachineRegister::TURN), RuntimeValue::Number(7.0));
    env.rollback_transaction().expect("Rollback transaction");
    assert_eq!(env.registers.read(MachineRegister::TURN), RuntimeValue::Number(0.0));

    // 5. Evaluator Directional Streaming to Registers
    let mut eval = Evaluator::new(env, debugger.clone());

    // 10 --> BOUND
    let src1 = "10 --> BOUND";
    let mut lex1 = Lexer::new(src1, logger.clone());
    let prog1 = Parser::new(lex1.tokenize().unwrap(), debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog1)).unwrap();
    assert_eq!(eval.env.registers.read(MachineRegister::BOUND), RuntimeValue::Number(10.0));

    // 42 --> IT then IT --> INTO
    let src2 = "42 --> IT";
    let prog2 = Parser::new(Lexer::new(src2, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog2)).unwrap();
    assert_eq!(eval.env.registers.read(MachineRegister::IT), RuntimeValue::Number(42.0));

    let src3 = "IT --> INTO";
    let prog3 = Parser::new(Lexer::new(src3, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog3)).unwrap();
    assert_eq!(eval.env.registers.read(MachineRegister::INTO), RuntimeValue::Number(42.0));
    assert_eq!(eval.env.output_stream.last(), Some(&RuntimeValue::Number(42.0)));

    // 6. Balanced Ternary Parity Checking
    let src4 = "4 = even";
    let prog4 = Parser::new(Lexer::new(src4, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res4 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog4)).unwrap();
    assert_eq!(res4, RuntimeValue::State(QuadState::Proceed));

    let src5 = "5 != even";
    let prog5 = Parser::new(Lexer::new(src5, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res5 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog5)).unwrap();
    assert_eq!(res5, RuntimeValue::State(QuadState::Proceed));
}

/// Asserts the bidirectional transpilation bridge between:
/// - Compiled, verbose, declarative 4-Block spatial reels (`.omni`)
/// - Interpreted line-by-line, lightweight procedural process ribbons (`.tape`)
#[test]
fn test_bidirectional_transpilation_omni_and_tape() {
    let (logger, debugger) = create_unit_test_fixtures();
    let mut assembler = OmniAssembler::new(logger.clone(), debugger.clone());

    let omni_source = r#"
Module < TranspileTest > ( Transpiler Bridge Verification ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}

Opening {
    requires: [ TruthSpace.T4 ],
    preconditions: [ +1 ]
}

Body [
```

```
    roll_1 [ 10 --> BOUND ]
    roll_2 [ 0 --> TURN ]
    roll_3 [ commit_status [ PROCEED ] ]
]

Closing {
    verify: [ roll_1 = yes ],
    commit_status [ PROCEED ]
}
"#;

    // Transpile .omni -> .tape
    let tape_text = assembler.transpile_omni_source_to_tape(omni_source).expect("Transpile omni to tape");
    assert!(tape_text.contains("[Tape.TranspileTest]"));
    assert!(tape_text.contains("roll_1 [ 10 --> BOUND ]"));
    assert!(tape_text.contains("roll_2 [ 0 --> TURN ]"));
    assert!(tape_text.contains("seal_roll [ commit_status [ PROCEED ] ]"));

    // Assemble the lowered tape and transpile back to .omni
    let assembled = assembler.assemble_omni(omni_source).expect("Assemble omni");
    let back_to_omni = transpile_tape_to_omni(&assembled);
    assert!(back_to_omni.contains("Module < TranspileTest >"));
    assert!(back_to_omni.contains("Opening {"));
    assert!(back_to_omni.contains("Body ["));
    assert!(back_to_omni.contains("Closing {"));
    assert!(back_to_omni.contains("commit_status [ PROCEED ]"));
}

/// Test 14: Verifies the 6 foundational Base Block Invariant Types in the Lexicon.
/// Asserts that Character, Text, Number, State, Container, and Site all assemble cleanly,
/// lower into physical Word64 machine containers, and commit in affirmative PROCEED.
#[test]
fn test_base_block_primitive_type_invariants_and_baton_pass() {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), 100);
    let mut asm = OmniAssembler::new(logger, debugger);

    let base_block_tapes = [
        "lexicon/01_nouns/ideas/types/character.tape",
        "lexicon/01_nouns/ideas/types/text.tape",
        "lexicon/01_nouns/ideas/types/number.tape",
        "lexicon/01_nouns/ideas/types/state.tape",
        "lexicon/01_nouns/ideas/types/container.tape",
        "lexicon/01_nouns/ideas/types/site.tape",
    ];

    for path_str in &base_block_tapes {
        let path = omnicode::core::filetype::resolve_track_path(path_str);
        assert!(path.exists(), "Base block tape must exist: {}", path_str);
        let assembled = asm.assemble_file(&path).unwrap_or_else(|e| panic!("Failed to assemble Base Block {}: {}",
path_str, e));
        assert_eq!(assembled.commit_status, QuadState::Proceed, "Base Block {} must seal in PROCEED", path_str);
        assert!(!assembled.instructions.is_empty(), "Base Block {} must synthesize instructions", path_str);
    }
}

/// Test 15: Verifies the dual-substrate Character Lifecycle and End-to-End Terminal Echo.
/// Proves that an input character glyph arriving from the terminal port (FROM ::@ 0x03F8)
/// executes through character.tape and character.omni, undergoes bijective Radix-3 transduction
/// into five balanced trits, verifies single-cell boundary invariants, projects outward to
/// the display target port (INTO ::@ 0x000B8000 / 0x01), and commits with affirmative +1:PROCEED.
#[test]
fn test_character_tape_and_omni_terminal_echo() {
    let (logger, debugger) = create_unit_test_fixtures();
    let mut asm = OmniAssembler::new(logger.clone(), debugger.clone());

    // 1. Verify that character.tape and character.omni exist and assemble to +1:PROCEED
    let tape_path = omnicode::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
    let omni_path = omnicode::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
    assert!(tape_path.exists(), "character.tape must exist in Track 2 lexicon");
    assert!(omni_path.exists(), "character.omni must exist in Track 2 lexicon");

    let assembled_tape = asm.assemble_file(&tape_path).expect("character.tape must assemble cleanly");
    assert_eq!(assembled_tape.commit_status, QuadState::Proceed);
    assert!(!assembled_tape.instructions.is_empty());

    let assembled_omni = asm.assemble_file(&omni_path).expect("character.omni must assemble cleanly");
    assert_eq!(assembled_omni.commit_status, QuadState::Proceed);
```

```
    assert!(!assembled_omni.instructions.is_empty());

    // 2. Set up Runtime Environment with simulated Terminal Input Port
    let mut env = Environment::new();

    // Input character 'A' (ASCII 65)
    let input_char = RuntimeValue::from_char('A');

    // Feed terminal input to FROM port
    env.write_from(input_char.clone());

    // Initialize environment symbols for character execution
    env.set("Glyph", input_char.clone());
    env.set("Cell", RuntimeValue::Cell(omnicode::core::cell::Cell::equilibrium()));
    env.set("Target", RuntimeValue::Text("Terminal Stdout".to_string()));
    env.set("Text", RuntimeValue::Text(String::new()));
    env.set("Tripwire", RuntimeValue::Text("ARMED".to_string()));
    env.set("Status", RuntimeValue::State(QuadState::Proceed));

    let mut eval = Evaluator::new(env, debugger.clone());

    // 3. Execute ROLL 1: Glyph Inscription (Outside-in imposition)
    let roll1_src = "{ Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]";
    let prog1 = Parser::new(Lexer::new(roll1_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res1 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog1)).expect("Eval Roll 1");
    assert_eq!(res1, RuntimeValue::State(QuadState::Proceed));
    assert_eq!(eval.env.get("Character`"), Some(&input_char));

    // 4. Execute ROLL 2: Bijective Radix-3 Transduction (<=>)
    let roll2_src = "[ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]";
    let prog2 = Parser::new(Lexer::new(roll2_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res2 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog2)).expect("Eval Roll 2");
    assert_eq!(res2, RuntimeValue::State(QuadState::Proceed));
    let cell_a = RuntimeValue::Cell(omnicode::core::cell::Cell::from_natural(b'A').unwrap());
    assert_eq!(eval.env.get("Cell`"), Some(&cell_a));

    // 5. Execute ROLL 3: Comparison & Single-Cell Bounds Verification
    let roll3_src = "{ Character` = Glyph } & { Character` <= [ 1 Cell ] } ! GUARD --> [ Status` = +1:PROCEED ]";
    let prog3 = Parser::new(Lexer::new(roll3_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    if let omnicode::ast::nodes::ASTNode::Connective(ref conn) = prog3.statements[0] {
        if let omnicode::ast::nodes::ASTNode::Connective(ref left_conn) = *conn.left {
            eprintln!("AND LEFT ({{ Character` = Glyph }}): {:?}", eval.eval(&left_conn.left));
            eprintln!("AND RIGHT ({{ Character` <= [ 1 Cell ] }}): {:?}", eval.eval(&left_conn.right));
        }
    }
    let res3 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog3)).expect("Eval Roll 3");
    eprintln!("RES3: {:?}", res3);
    assert_eq!(res3, RuntimeValue::State(QuadState::Proceed));
    let status_proceed = RuntimeValue::State(QuadState::Proceed);
    assert_eq!(eval.env.get("Status`"), Some(&status_proceed));

    // 6. Execute ROLL 4: Target Projection to Terminal Output Port (INTO)
    let roll4_src = "[ Character` } --> PROJECT [ Target ] = [ Target` }";
    let prog4 = Parser::new(Lexer::new(roll4_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res4 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog4)).expect("Eval Roll 4");
    assert_eq!(res4, RuntimeValue::State(QuadState::Proceed));

    // Verify Terminal Echo in INTO register and output stream
    assert_eq!(eval.env.read_into(), input_char.clone());
    assert_eq!(eval.env.output_stream.last(), Some(&input_char));

    // 7. Execute ROLL 5: Continuous Text Sequence Ribbon Transition
    let roll5_src = "[ Character` } --> SEQUENCE [ Text ] = [ Text` ]";
    let prog5 = Parser::new(Lexer::new(roll5_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res5 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog5)).expect("Eval Roll 5");
    assert_eq!(res5, RuntimeValue::State(QuadState::Proceed));
    let text_a = RuntimeValue::Text("A".to_string());
    assert_eq!(eval.env.get("Text`"), Some(&text_a));

    // 8. Execute ROLL 6: Closing Commit Status Verification
    eval.env.set("Verify", RuntimeValue::State(QuadState::Proceed));
    let roll6_src = "{ [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED } --> [ +1:PROCEED ]";
    let prog6 = Parser::new(Lexer::new(roll6_src, logger.clone()).tokenize().unwrap(),
```

```
debugger.clone()).parse_program().unwrap();
    let res6 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog6)).expect("Eval Roll 6");
    assert_eq!(res6, RuntimeValue::State(QuadState::Proceed));

    // 9. Repeat with dynamic Terminal Character 'K' (ASCII 75)
    let glyph_k = RuntimeValue::from_char('K');
    eval.env.write_from(glyph_k.clone());
    eval.env.set("Glyph", glyph_k.clone());
    let prog1_k = Parser::new(Lexer::new(roll1_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog1_k)).expect("Eval Roll 1 for K");
    let prog4_k = Parser::new(Lexer::new(roll4_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog4_k)).expect("Eval Roll 4 for K");
    assert_eq!(eval.env.read_into(), glyph_k);
}

// ==============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Suite Verification Proof & Postconditions )
// ==============================================================================
// Unit test suite verified and sealed.

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/lib.rs

RS | 84 LINES | 324 WORDS | 3.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Library Root Identity, Universal Subsystem Modules & Core Re-exports )
// ============================================================================
//! OmniCode Language Engine
//! High-performance language engine based on formal ontology,
//! set theory, and balanced ternary quad-state execution,
//! engineered to be symmetrically human and machine readable.

pub mod core;
pub mod diagnostics;
pub mod ast;
pub mod parser;
pub mod runtime;
pub mod cli;
pub mod ai;
pub mod terminal;
pub mod cogs;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Public Re-export Declarations & High-Level API Surface )
// ============================================================================
pub use cogs::platform::{HostPlatform, HostArch};
pub use cogs::tripwire::{Tripwire, TripwireError};
pub use core::ternary::{QuadState, TransportAction, ROLLBACK, AWAIT, PROCEED, HALT};
pub use core::boundaries::BoundaryType;
pub use core::operators::{BeingRelation, DirectionalVector, Connective, Polarity, AffixPosition};
pub use core::word::{Word32Trit, Word64};
pub use core::scene::Scene;
pub use core::reel::MemoryReel;
pub use core::filetype::OmniFileType;
pub use core::tape::{Tape, Roll};
pub use core::mmio::{PhysicalMemoryMap, VgaBuffer, VgaColor, VGA_BUFFER_BASE, COM1_SERIAL_PORT};
pub use terminal::{OmniScheduler, OmniTask, Track, TaskPriority};

pub use diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion};
pub use diagnostics::debugger::OmniDebugger;
pub use diagnostics::linter::{OmniLinter, LintReport, BlockKind};
pub use diagnostics::restorator::{Restorator, RestorationReport, AppliedRepair};
pub use diagnostics::hover::{HoverEngine, HoverCard, OmniDefinition, TapeRollContext};
pub use ai::{
    AITransducer, TransductionSession, AICortex, ScrollLogger, ExecutionScroll, AILearner,
    EpisodicSummary, EpisodicExperience, CognitiveResult, BiblicalLexicon, LexiconDomain,
    LexiconEntry, MoralEvaluation,
};
pub use parser::lexer::Lexer;
pub use parser::parser::Parser;
pub use runtime::environment::Environment;
pub use runtime::evaluator::Evaluator;
pub use terminal::OmniStudio;
pub use cli::{
    run_repl, run_gui_server, run_boot, run_boot_with_config, BootArch, BootConfig, BootContext,
    run_assemble, run_assemble_with_config, AssembleConfig, AssembleContext, AssembleFormat,
};


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Convenience Execution Pipeline )
// ============================================================================
/// Convenience execution helper that tokenizes, parses, and evaluates an OmniCode string.
pub fn execute(source: &str, print_logs: bool) -> Result<runtime::RuntimeValue, String> {
    let logger = DiagnosticLogger::new(print_logs);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize()?;

    let debugger = OmniDebugger::new(logger, 10);
    let mut parser = Parser::new(tokens, debugger);
    let program = parser.parse_program()?;

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
```

```
    evaluator.eval(&ast::nodes::ASTNode::Program(program))
}


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Engine Status & Library Invariants )
// ================================================================================
// OmniCode library surface verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/main.rs RS 748 LINES 2857 WORDS 32.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( CLI Executable Identity, Subsystem Routing & Core Imports )
// ============================================================================
//! OmniCode CLI Entrypoint
//!
//! Provides the primary terminal interface for the OmniCode language engine:
//!     - Interactive REPL Terminal (default)
//!     - Single-file Script Runner (`run <file.omni>`)
//!     - 4-Block Structural Linter (`lint <file.omni>`)
//!     - Canonical Template Scaffolder (`scaffold <Name>`)
//!     - Ontological Verification Demo (`demo`)

use std::env;
use std::fs;
use std::process;
use std::thread;

use omnicode::{
    run_repl, run_gui_server, DiagnosticLogger, OmniDebugger, OmniLinter, Restorator,
    AICortex, AILearner, OmniFileType, Lexer, Parser, Evaluator, Environment,
    BiblicalLexicon, HoverEngine, Tape, OmniStudio,
};
use omnicode::terminal::{OmniScheduler, Track};
use omnicode::cli::commands::{BootConfig, run_boot_with_config, AssembleConfig, run_assemble_with_config};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( CLI Help Display, Version Banner, and Command Route Setup )
// ============================================================================
fn print_cli_help() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode: Dual Human & Machine Readable Language Engine (v0.1.0)\x1b[0m");
    println!("\x1b[1;36m  Core: Balanced Ternary & Boundary Physics (Rust Engine)\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");
    println!("\x1b[1;33mUSAGE:\x1b[0m");
    println!("  omnicode                    Start the interactive terminal shell (REPL)");
    println!("  omnicode repl               Start the interactive terminal shell (REPL)");
    println!("  omnicode run <file>         Execute an OmniCode source (.omni) or scroll (.scroll)");
    println!("  omnicode assemble <file>    Assemble an OmniCode process tape into concrete memory layout (.tape)");
    println!("  omnicode studio <file.tape> Project process tape in the cinematic HUD terminal projector");
    println!("  omnicode project <file.tape> Project process tape in the cinematic HUD terminal projector");
    println!("  omnicode boot [--uefi|--multiboot|--bios] [--gui] Boot bare-metal OS in QEMU");
    println!("  omnicode lint <file>        Validate 4-Block architecture compliance (.omni / .scroll)");
    println!("  omnicode hover <file> <l> <c> Inspect cross-scale 3-tier hovercard for symbol at position");
    println!("  omnicode doc <symbol>       Query architectural docstring, invariants, and machine intent");
    println!("  omnicode restore <file>     Auto-heal syntax, boundary, and block anomalies");
    println!("  omnicode ai \"<prompt>\"      Transduce prompt with AI Cortex, self-heal, execute & inscribe logs");
    println!("  omnicode scrolls            Inspect episodic memory bank and learning statistics");
    println!("  omnicode manifest           Display the canonical Covenant Manifest and moral axioms");
    println!("  omnicode lexicon [term]     Inspect the 100% offline Biblical world lexicon");
    println!("  omnicode gui [--port 8080]  Launch the local web GUI Studio & Terminal interface");
    println!("  omnicode desktop            Launch the native desktop GUI Studio & Terminal shell");
    println!("  omnicode scaffold <Name> [--type omni|scroll] Generate canonical 4-Block template");
    println!("  omnicode filetypes          Display all formal OmniCode file types and MIME definitions");
    println!("  omnicode demo               Run the built-in ontological verification suite");
    println!("  omnicode --help, -h         Show this help information");
    println!("  omnicode --version, -v      Display engine version\n");
    println!("\x1b[1;33mREPL FEATURES:\x1b[0m");
    println!("  • Real-time evaluation of balanced ternary expressions (-1, 0, +1, NULL)");
    println!("  • Boundary delimiter matching with multiline input buffering");
    println!("  • Interactive failpoint alerts and transaction snapshot rollback");
    println!("  • Live symbol inspection via `:env` and AST inspection via `:ast <expr>`");
    println!("  • Persistent command line history\n");
}

fn print_version() {
    println!("omnicode 0.1.0 (Rust Core: Balanced Ternary & Boundary Physics)");
}


// ============================================================================
// BLOCK 3: BODY BLOCK
```

```
// ( Command Handlers: Script Runner, Linter, Scaffolder, Ontological Demo )
// ============================================================================
fn run_tape_file(path: &str) {
    let logger = DiagnosticLogger::new(false);
    println!("\x1b[1;36mMounting & Executing OmniCode Process Tape:\x1b[0m {}", path);

    let mut scheduler = OmniScheduler::new();
    let track = if path.contains("kernel") || path.contains("boot") {
        Track::Kernel
    } else {
        Track::App
    };

    let pid = match scheduler.mount_tape_file(track, path, logger.clone()) {
        Ok(id) => id,
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Tape Mount Error:\x1b[0m {}", e);
            process::exit(1);
        }
    };

    let task = scheduler.get_task(pid).expect("Task descriptor must exist");
    let tape = task.tape.as_ref().expect("Tape ribbon must exist");

    println!("\x1b[1;32m✓ Tape Manifest Covenant:\x1b[0m");
    println!("    • Process:   [{}]", tape.head.name);
    println!("    • Channel:   {}", track.as_str());
    println!("    • WHO:       {}", tape.head.who);
    println!("    • WHAT:      {}", tape.head.what);
    println!("    • WHEN:      {}", tape.head.when_clause);
    println!("    • WHERE:     {} (Partition: {} cells)", tape.head.where_clause, tape.head.required_partition_size);
    println!("    • WHY:       ({})", tape.head.why);
    println!("    • HOW:       {}", tape.head.how);
    println!("-------------------------------------------------------------");

    let total_rolls = tape.roll_count();
    let mut step = 0;
    while let Ok(msg) = scheduler.step_task(pid) {
        step += 1;
        println!("  \x1b[1;34m==>\x1b[0m {}", msg);
        if msg.contains("completed") || msg.contains("already executed") || msg.contains("STOP") {
            break;
        }
        if step > 500 {
            println!("  \x1b[1;33m[!] Step Limit Reached (500 steps) - breaking transport loop\x1b[0m");
            break;
        }
    }


    println!("-------------------------------------------------------------");
    println!("\x1b[1;32m✓ Tape Execution Complete:\x1b[0m All {} rolls executed successfully (State: +1:PROCEED)",
total_rolls);
}


fn run_studio(path: &str) {
    let logger = DiagnosticLogger::new(false);
    let tape = match Tape::from_file(path, logger) {
        Ok(t) => t,
        Err(e) => {
            eprintln!("\x1b[1;31mError loading tape for studio:\x1b[0m {e}");
            process::exit(1);
        }
    };
    let mut studio = OmniStudio::new(tape.head.name.clone(), tape.head.required_partition_size);
    studio.project_tape(&tape);
}


fn run_file(path: &str) {
    let ft = OmniFileType::from_path(path);
    if ft == Some(OmniFileType::Tape) || path.ends_with(".tape") {
        run_tape_file(path);
        return;
    }

    let title = ft.map(|f| f.display_name()).unwrap_or("OmniCode source file");
```

```
    println!("\x1b[1;36mExecuting {}:\x1b[0m {}", title, path);
    let content = match fs::read_to_string(path) {
        Ok(c) => c,
        Err(e) => {
            eprintln!("\x1b[1;31m✗ File read error:\x1b[0m Could not open '{}': {}", path, e);
            process::exit(1);
        }
    };

    let logger = DiagnosticLogger::new(false);

    // 1. Structural 4-Block Lint Check
    let linter = OmniLinter::new(logger.clone());
    let lint_report = linter.lint_source(path, &content);
    if !lint_report.is_valid {
        eprintln!("\x1b[1;33m⚠ Warning: File failed 4-Block architectural validation:\x1b[0m");
        for err in &lint_report.errors {
            eprintln!("    • {}", err);
        }
    } else {
        println!("\x1b[1;32m✓ 4-Block Structural Check:\x1b[0m Valid ({} blocks verified).",
lint_report.blocks_found.len());
    }

    // 2. Tokenization
    let mut lexer = Lexer::new(&content, logger.clone());
    let tokens = match lexer.tokenize() {
        Ok(toks) => toks,
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Lexical Error:\x1b[0m {}", e);
            process::exit(1);
        }
    };

    // 3. Parsing
    let debugger = OmniDebugger::new(logger.clone(), 10);
    let mut parser = Parser::new(tokens, debugger);
    let prog = match parser.parse_program() {
        Ok(p) => p,
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Parser Error:\x1b[0m {}", e);
            process::exit(1);
        }
    };

    // 4. Execution
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    match evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)) {
        Ok(result) => {
            println!("\x1b[1;32m✓ Execution Succeeded:\x1b[0m => {}", result.format_colored());
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Evaluation Error:\x1b[0m {}", e);
            process::exit(1);
        }
    }

    // 5. Failpoints report
    if logger.has_failpoints() {
        println!("\n\x1b[1;33m--- Trapped Diagnostic Failpoints ({}) ---\x1b[0m", logger.failpoints().len());
        for fp in logger.failpoints() {
            let fp_name = fp.failpoint_type.map(|t| t.as_str()).unwrap_or("FAILPOINT");
            println!("  * [{} µs] \x1b[1;31m[{}]\x1b[0m [{}] {}", fp.elapsed_micros, fp.channel, fp_name, fp.message);
        }
    }
}

fn run_lint_file(path: &str) {
    let ft = OmniFileType::from_path(path);
    let title = ft.map(|f| f.display_name()).unwrap_or("OmniCode file");
    println!("\x1b[1;33m--- Linting 4-Block Structural Architecture ({title}): {} ---\x1b[0m", path);
    let content = match fs::read_to_string(path) {
        Ok(c) => c,
        Err(e) => {
            eprintln!("\x1b[1;31m✗ File read error:\x1b[0m Could not open '{}': {}", path, e);
            process::exit(1);
        }
    };
```

```
    let logger = DiagnosticLogger::new(false);
    let linter = OmniLinter::new(logger);
    let report = linter.lint_source(path, &content);

    if report.is_valid {
        println!("\x1b[1;32m✓ 4-Block Validation PASSED\x1b[0m");
        println!("  Blocks verified in sequence:");
        for b in &report.blocks_found {
            println!("    - Line {}: {}", b.line_number, b.kind.name());
        }
        if !report.warnings.is_empty() {
            println!("\x1b[1;33m⚠ Lint Warnings / Recommendations ({}):\x1b[0m", report.warnings.len());
            for warn in &report.warnings {
                println!("    • {}", warn);
            }
        }
    } else {
        println!("\x1b[1;31m✗ 4-Block Validation FAILED\x1b[0m");
        for err in &report.errors {
            eprintln!("    • {}", err);
        }
        process::exit(1);
    }
}

fn run_scaffold(name: &str, file_type: Option<&str>) {
    let ft = file_type
        .and_then(OmniFileType::from_extension)
        .unwrap_or(OmniFileType::SourceReel);
    let scaffolded = match ft {
        OmniFileType::SourceReel => OmniLinter::auto_scaffold_omni(name, "User-defined module"),
        OmniFileType::EpisodicScroll => OmniLinter::auto_scaffold_scroll(name, "User cognitive discourse intent"),
        OmniFileType::Tape => ft.scaffold_template(name, "User process execution tape"),
    };
    println!("{}", scaffolded);
}

fn run_filetypes() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode File Type System & Taxonomy Manifest                                  \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    for (i, ft) in OmniFileType::all().iter().enumerate() {
        println!("  \x1b[1;33m[{}]\x1b[0m \x1b[1;32m.{}\x1b[0m  —  \x1b[1m{}\x1b[0m", i + 1, ft.extension(),
ft.display_name());
        println!("      MIME:        \x1b[36m{}\x1b[0m", ft.mime_type());
        println!("      Language ID: \x1b[35m{}\x1b[0m", ft.language_id());
        println!("      Purpose:     {}\n", ft.description());
    }
}

fn run_demo() {
    println!("\x1b[1;36m======================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode: Ontological Conceptual Testbed           \x1b[0m");
    println!("\x1b[1;36m======================================================\x1b[0m\n");

    let demo_code = fs::read_to_string("examples/canonical_notation.omni")
        .unwrap_or_else(|_| OmniLinter::auto_scaffold_omni("DemoModule", "Ontological verification module"));


    println!("\x1b[1;33m--- Running 4-Block Structural Linter ---\x1b[0m");
    let logger = DiagnosticLogger::new(true);
    let linter = OmniLinter::new(logger.clone());
    let lint_report = linter.lint_source("demo_code", &demo_code);

    if lint_report.is_valid {
        println!("\x1b[1;32m✓ 4-Block Structural Check Passed:\x1b[0m Found {} blocks in order.",
lint_report.blocks_found.len());
    } else {
        println!("\x1b[1;31m✗ Structural Linter Errors:\x1b[0m {:?}", lint_report.errors);
    }

    println!("\n\x1b[1;33m--- Running Code Evaluation Pipeline ---\x1b[0m");
    let mut lexer = Lexer::new(&demo_code, logger.clone());

    match lexer.tokenize() {
        Ok(tokens) => {
```

```
            println!("\x1b[1;32m✓ Lexer Tokenization Succeeded:\x1b[0m ({} tokens)", tokens.len());
            let debugger = OmniDebugger::new(logger.clone(), 5);
            let mut parser = Parser::new(tokens, debugger);

            match parser.parse_program() {
                Ok(prog) => {
                    println!("\x1b[1;32m✓ Parser Succeeded:\x1b[0m Generated AST with {} statements",
prog.statements.len());
                    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
                    match evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)) {
                        Ok(res) => {
                            println!("\n\x1b[1;32m✓ Execution Final Result:\x1b[0m => {}", res.format_colored());
                        }
                        Err(e) => eprintln!("\x1b[1;31m✗ Evaluator Error:\x1b[0m {}", e),
                    }
                }
                Err(e) => eprintln!("\x1b[1;31m✗ Parser Error:\x1b[0m {}", e),
            }
        }
        Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error:\x1b[0m {}", e),
    }

    if logger.has_failpoints() {
        println!("\n\x1b[1;33m--- Diagnostic Failpoints Caught During Run ---\x1b[0m");
        for fp in logger.failpoints() {
            println!("  * [{}] {}", fp.channel, fp.message);
        }
    } else {
        println!("\n\x1b[1;32m✓ No unexpected failpoints encountered during demo run.\x1b[0m");
    }
}

fn run_restore(path: &str, in_place: bool) {
    println!("\x1b[1;36mAutonomous Self-Healing Restoration:\x1b[0m {}", path);
    let logger = DiagnosticLogger::new(false);
    let restorator = Restorator::new(logger);

    match restorator.restore_file(path, in_place) {
        Ok(report) => {
            if !report.was_modified {
                println!("\x1b[1;32m✓ Source is already healthy:\x1b[0m No repairs needed.");
            } else {
                println!("\x1b[1;33m⚠ Repairs Applied ({}):\x1b[0m", report.repairs_applied.len());
                for repair in &report.repairs_applied {
                    println!("    • L{}: [{:?}] {}", repair.line, repair.kind, repair.description);
                }
                if in_place {
                    println!("\x1b[1;32m✓ Healed file written in-place to {}\x1b[0m", path);
                } else {
                    println!("\n\x1b[1;36m--- Restored Content Preview ---\x1b[0m\n{}", report.restored_content);
                }
            }

            if report.verified {
                println!("\x1b[1;32m✓ Post-Restoration Verification PASSED:\x1b[0m Code executes to => +1:PROCEED");
            } else if !report.errors.is_empty() {
                println!("\x1b[1;33m⚠ Verification Diagnostics:\x1b[0m {:?}", report.errors);
            }
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Restoration Failed:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_ai(prompt: &str) {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: Cognitive Cortex (Logger + Debugger + Restorator + Transducer)\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");
    println!("\x1b[1;33m[Human Layer Discourse Prompt]:\x1b[0m\n  \"{}\"\n", prompt);

    let mut cortex = AICortex::new();

    match cortex.process_prompt(prompt) {
        Ok(res) => {
            if !res.repairs_applied.is_empty() {
                println!("\x1b[1;33m⚠ Restorator Immune Repairs Applied ({}):\x1b[0m", res.repairs_applied.len());
```

```
                for r in &res.repairs_applied {
                    println!("    • L{}: [{:?}] {}", r.line, r.kind, r.description);
                }
                println!();
            }

            println!("\x1b[1;32m✓ Synthesized 4-Block OmniCode Turing Reel:\x1b[0m");
            println!("--------------------------------------------------------------------------------");
            println!("{}", res.synthesized_omnicode.trim());
            println!("--------------------------------------------------------------------------------");
            println!("\x1b[1;32m✓ Machine Quad-State Status:\x1b[0m => +1:{}", res.execution_state.macro_op());
            println!("\n\x1b[1;35m[Machine Layer Response]:\x1b[0m");
            println!("  \"{}\"\n", res.natural_response);
            println!("\x1b[1;34m[Episodic Learning Inscription]:\x1b[0m");
            println!("  • OmniCode Reel (.scroll): {:?}", res.scroll_path);
            println!("  • Traditional Telemetry (.log): {:?}", res.log_path);
            println!("  (Session ID: {})\n", res.session_id);
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ AI Cortex Error:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_scrolls() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: Episodic Memory Bank & Experiential Learning Archive           \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");
    let scroll_dir = if std::path::Path::new("storage/scrolls").exists() {
        "storage/scrolls"
    } else if std::path::Path::new("runtime/scrolls").exists() {
        "runtime/scrolls"
    } else {
        "scrolls"
    };
    let learner = AILearner::new(scroll_dir);
    match learner.digest_experiences() {
        Ok(summary) => {
            println!("\x1b[1;33mEPISODIC LEARNING SUMMARY:\x1b[0m");
            println!("  • Total Inscribed Episodes:     {}", summary.total_episodes);
            println!("  • State Distribution:           Proceed (+1): {} | Halt (NULL): {} | Rollback (-1): {} | Await
(0): {}",
                summary.proceed_count, summary.halt_count, summary.rollback_count, summary.await_count
            );
            println!("  • Restorator Repairs Learned:   {}\n", summary.total_repairs_learned);

            if summary.experiences.is_empty() {
                println!("  (No scrolls recorded yet in `scrolls/`. Run `omnicode ai \"...\"` to generate
experiences.)\n");
            } else {
                println!("\x1b[1;33mRECENT EPISODES:\x1b[0m");
                for (i, exp) in summary.experiences.iter().enumerate() {
                    let log_status = if exp.log_path.is_some() { "yes" } else { "no" };
                    println!(
                        "  [{:>2}] Session: {:<12} | State: +1:{:<8} | Repairs: {:>2} | Failpoints: {:>2} | .log: {}",
                        i + 1, exp.session_id, exp.final_state.macro_op(), exp.repair_count, exp.failpoint_count,
log_status
                    );
                    println!("       Prompt: \"{}\"", exp.prompt);
                    println!("       Scroll: {:?}\n", exp.scroll_path);
                }
            }
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Failed to read scrolls:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_manifest() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: Invariant Covenant Manifest (manifest.omni)                    \x1b[0m");
    println!("\x1b[1;36m  Anchored in Public Domain Scriptures (King James Version & World English Bible)\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    match fs::read_to_string("manifest.omni") {
```

```
        Ok(content) => println!("{}", content),
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Could not open manifest.omni:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_lexicon(query: Option<&str>) {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: 100% Offline Biblical World Lexicon & Moral Ontology           \x1b[0m");
    println!("\x1b[1;36m  Public Domain Primary Witness: King James Version (1611) & World English Bible\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    let lex = BiblicalLexicon::new();
    if let Some(q) = query {
        let results = lex.search(q);
        if results.is_empty() {
            println!("No concepts found matching '{}'. Total concepts available: {}\n", q, lex.all_entries().len());
        } else {
            println!("\x1b[1;33mSEARCH RESULTS FOR '{}' ({} found):\x1b[0m\n", q, results.len());
            for e in results {
                println!("\x1b[1;32m[{}]\x1b[0m \x1b[1m{}\x1b[0m (State Resonance: +1:{})", e.domain.as_str(),
e.term.to_uppercase(), e.quad_state_resonance.macro_op());
                println!("  • Definition: {}", e.definition);
                println!("  • KJV:        {}", e.kjv_passage);
                println!("  • WEB:        {}", e.web_passage);
                println!("  • Guidance:   {}\n", e.moral_guidance);
            }
        }
    } else {
        println!("\x1b[1;33mCANONICAL VOCABULARY OVERVIEW ({} concepts available offline):\x1b[0m\n",
lex.all_entries().len());
        for e in lex.all_entries() {
            println!("  • \x1b[1;32m{:<20}\x1b[0m | Domain: {:<30} | Resonance: +1:{}", e.term, e.domain.as_str(),
e.quad_state_resonance.macro_op());
        }
        println!("\nTip: Query a specific concept with `omnicode lexicon <term>` (e.g. `omnicode lexicon truth`)\n");
    }
}

fn run_gui(port: u16, auto_open: bool, path: &str) {
    if auto_open {
        let url = format!("http://localhost:{port}{path}");
        thread::spawn(move || {
            thread::sleep(std::time::Duration::from_millis(300));
            omnicode::cogs::platform::HostPlatform::current().open_browser(&url);
        });
    }
    if let Err(e) = run_gui_server(port) {
        eprintln!("\x1b[1;31m✗ GUI Server Fatal Error:\x1b[0m {}", e);
        process::exit(1);
    }
}



fn run_hover(file: &str, line: usize, col: usize, as_json: bool) {
    let logger = DiagnosticLogger::new(false);
    let mut engine = HoverEngine::new(logger);
    engine.index_workspace(".");

    match engine.hover_at(file, line, col) {
        Some(card) => {
            if as_json {
                println!("{}", card.to_json());
            } else {
                println!("{}", card.render_terminal());
            }
        }
        None => {
            if as_json {
                println!(r#"{{"symbol":"","found":false,"error":"No symbol found at coordinate"}}"#);
            } else {
                eprintln!("\x1b[1;33m⚠ No symbol located at {}:{}:{}\x1b[0m", file, line, col);
            }
            process::exit(1);
        }
```

```
    }
}

fn run_doc(symbol: &str, file_hint: Option<&str>, as_json: bool) {
    let logger = DiagnosticLogger::new(false);
    let mut engine = HoverEngine::new(logger);
    engine.index_workspace(".");

    match engine.document_symbol(symbol, file_hint) {
        Some(card) => {
            if as_json {
                println!("{}", card.to_json());
            } else {
                println!("{}", card.render_terminal());
            }
        }
        None => {
            if as_json {
                println!(r#"{{"symbol":"{}","found":false,"error":"Symbol definition not found"}}"#, symbol);
            } else {
                eprintln!("\x1b[1;33m⚠ Symbol '{}' not found in workspace or target file\x1b[0m", symbol);
            }
            process::exit(1);
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( CLI Main Entrypoint Dispatcher & Process Termination )
// ============================================================================
fn main() {
    let args: Vec<String> = env::args().collect();

    if args.len() <= 1 {
        // Default behavior: launch interactive terminal environment (REPL)
        if let Err(e) = run_repl() {
            eprintln!("\x1b[1;31mFatal REPL Error:\x1b[0m {}", e);
            process::exit(1);
        }
        return;
    }

    match args[1].as_str() {
        "repl" => {
            if let Err(e) = run_repl() {
                eprintln!("\x1b[1;31mFatal REPL Error:\x1b[0m {}", e);
                process::exit(1);
            }
        }
        "terminal" | "term" | "shell" => {
            let port = if args.len() >= 4 && (args[2] == "--port" || args[2] == "-p") {
                args[3].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2].starts_with("--port=") {
                args[2]["--port=".len()..].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2] != "terminal" && args[2] != "term" {
                args[2].parse::<u16>().unwrap_or(8080)
            } else {
                8080
            };
            println!("\x1b[1;36mLaunching OmniCode Native Interactive Terminal (CreativeWorkzStudio LLC)...\x1b[0m");
            run_gui(port, true, "/terminal");
        }
        "project" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode project <file.tape>");
                process::exit(1);
            }
            run_studio(&args[2]);
        }
        "studio" => {
            if args.len() >= 3 && (args[2].ends_with(".tape") || args[2].ends_with(".omni") || !args[2].starts_with("-"))
&& args[2].parse::<u16>().is_err() {
                run_studio(&args[2]);
            } else {
                let port = if args.len() >= 4 && (args[2] == "--port" || args[2] == "-p") {
                    args[3].parse::<u16>().unwrap_or(8080)
```

```
                } else if args.len() >= 3 && args[2].starts_with("--port=") {
                    args[2]["--port=".len()..].parse::<u16>().unwrap_or(8080)
                } else if args.len() >= 3 && args[2] != "studio" {
                    args[2].parse::<u16>().unwrap_or(8080)
                } else {
                    8080
                };
                println!("\x1b[1;36mLaunching OmniCode Studio & Platform (CreativeWorkzStudio LLC)...\x1b[0m");
                run_gui(port, true, "/studio");
            }
        }
        "gui" | "serve" | "desktop" | "app" => {
            let port = if args.len() >= 4 && (args[2] == "--port" || args[2] == "-p") {
                args[3].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2].starts_with("--port=") {
                args[2]["--port=".len()..].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2] != "gui" && args[2] != "serve" && args[2] != "desktop" && args[2] !=
"app" {
                args[2].parse::<u16>().unwrap_or(8080)
            } else {
                8080
            };
            println!("\x1b[1;36mLaunching OmniCode Studio & Platform (CreativeWorkzStudio LLC)...\x1b[0m");
            run_gui(port, true, "/studio");
        }
        "boot" | "qemu" => {
            let config = BootConfig::from_args(&args);
            if let Err(e) = run_boot_with_config(config) {
                eprintln!("\x1b[1;31mMachine Inception Halted:\x1b[0m {e}");
                process::exit(1);
            }
        }
        "assemble" | "asm" => {
            let sub_args: Vec<String> = args.iter().skip(2).cloned().collect();
            let config = match AssembleConfig::from_args(&sub_args) {
                Ok(c) => c,
                Err(e) => {
                    eprintln!("\x1b[1;31mError:\x1b[0m {e}");
                    process::exit(1);
                }
            };
            if let Err(e) = run_assemble_with_config(config) {
                eprintln!("\x1b[1;31m✗ Assemble Tripwire Error:\x1b[0m {e}");
                process::exit(1);
            }
        }
        "run" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode run <file.omni>");
                process::exit(1);
            }
            run_file(&args[2]);
        }
        "lint" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode lint <file.omni>");
                process::exit(1);
            }
            run_lint_file(&args[2]);
        }
        "hover" => {
            if args.len() < 5 {
                eprintln!("\x1b[1;31mError:\x1b[0m Usage: omnicode hover <file> <line> <col> [--json]");
                process::exit(1);
            }
            let file = &args[2];
            let line = args[3].parse::<usize>().unwrap_or(1);
            let col = args[4].parse::<usize>().unwrap_or(1);
            let as_json = args.iter().any(|a| a == "--json" || a == "-j");
            run_hover(file, line, col, as_json);
        }
        "doc" | "explain" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Usage: omnicode doc <symbol> [--file <path>] [--json]");
                process::exit(1);
            }
            let symbol = &args[2];
            let as_json = args.iter().any(|a| a == "--json" || a == "-j");
```

```
            let file_hint = args.iter().position(|a| a == "--file" || a == "-f").and_then(|idx| args.get(idx + 1).map(|s| s.as_str()));
            run_doc(symbol, file_hint, as_json);
        }
        "restore" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode restore <file.omni> [--write]");
                process::exit(1);
            }
            let in_place = args.iter().any(|a| a == "--write" || a == "-w");
            run_restore(&args[2], in_place);
        }
        "ai" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing prompt. Usage: omnicode ai \"<natural language sentence>\"");
                process::exit(1);
            }
            let prompt = args[2..].join(" ");
            run_ai(&prompt);
        }
        "scrolls" | "history" | "learn" => {
            run_scrolls();
        }
        "manifest" => {
            run_manifest();
        }
        "lexicon" | "bible" | "vocab" => {
            let query = args.get(2).map(|s| s.as_str());
            run_lexicon(query);
        }
        "filetypes" | "types" => {
            run_filetypes();
        }
        "scaffold" => {
            let (name, kind) = if args.len() >= 4 && (args[2] == "--type" || args[2] == "-t") {
                (args.get(4).map(|s| s.as_str()).unwrap_or("MyModule"), Some(args[3].as_str()))
            } else if args.len() >= 4 && (args[3] == "--type" || args[3] == "-t") {
                (args[2].as_str(), args.get(4).map(|s| s.as_str()))
            } else if args.len() >= 4 && (args[2] == "scroll" || args[2] == "omni") {
                (args[3].as_str(), Some(args[2].as_str()))
            } else if args.len() >= 3 {
                (args[2].as_str(), None)
            } else {
                ("MyModule", None)
            };
            run_scaffold(name, kind);
        }
        "demo" => {
            run_demo();
        }
        "--help" | "-h" | "help" => {
            print_cli_help();
        }
        "--version" | "-v" | "version" => {
            print_version();
        }
        other => {
            eprintln!("\x1b[1;31mUnknown argument:\x1b[0m '{}'", other);
            print_cli_help();
            process::exit(1);
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/boundaries.rs RS 167 LINES 778 WORDS 6.9 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Module Identity, Morphological Boundary Invariants & Delimiter Tables )
// ================================================================================
//! OmniCode Core: Boundary Mechanics & Morphological Delimiters
//!
//! Delimiters are active morphological operators that govern the boundary physics
//! between an entity and its surrounding universe.
//!
//! The 6 Canonical Boundaries:
//!     1. Denotation  : [ ... ]  (Mutable Definition)
//!     2. Invariant   : < ... >  (Immutable Axiom / Constant)
//!     3. Connotation : { ... }  (In Context Of / Environment)
//!     4. TopDown     : { ... ]  (Context Defines Entity)
//!     5. BottomUp    : [ ... }  (Entity Defined by Context)
//!     6. Aside       : ( ... )  (Aside To / First-Class Machine Intent)

use std::fmt;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Context Declarations, BoundaryType Enum, and Delimiter Mapping )
// ================================================================================
/// Morphological boundary types ranked by ontological hierarchy:
///   Rank 1: < ... > (Invariant / Axiom / Law)
///   Rank 2: [ ... ] (Denotation / Essence / Mutable State)
///   Rank 3: { ... } (Connotation / Ambient Context / Environment)
///   Rank 4: [ ... } (Bottom-Up / Inside-Out / Projection into Context)
///   Rank 5: { ... ] (Top-Down / Outside-In / Imposition by Context)
///   Rank 6: ( ... ) (Aside / First-Class Intent / Observer Rationale)
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum BoundaryType {
    /// Rank 1: < ... > Immutable Axiom / Law
    Invariant,
    /// Rank 2: [ ... ] Mutable Definition / Essence / State
    Denotation,
    /// Rank 3: { ... } Ambient Context / Environment
    Connotation,
    /// Rank 4: [ ... } Defined by context (Inside-Out / Emergent)
    BottomUp,
    /// Rank 5: { ... ] Context defines (Outside-In / Imposition)
    TopDown,
    /// Rank 6: ( ... ) Aside to (Intent / Observer Rationale)
    Aside,
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Core Resolution Logic: Boundary Properties, Directionality, Pair Matching )
// ================================================================================
impl BoundaryType {
    /// Returns the ontological rank of the boundary (1 = Highest authority / ground truth).
    pub fn rank(&self) -> u8 {
        match self {
            BoundaryType::Invariant => 1,
            BoundaryType::Denotation => 2,
            BoundaryType::Connotation => 3,
            BoundaryType::BottomUp => 4,
            BoundaryType::TopDown => 5,
            BoundaryType::Aside => 6,
        }
    }

    /// Descriptive name of the ontological rank tier.
    pub fn rank_name(&self) -> &'static str {
        match self {
            BoundaryType::Invariant => "Rank 1: Axiomatic Invariant",
            BoundaryType::Denotation => "Rank 2: Denotative Essence",
            BoundaryType::Connotation => "Rank 3: Connotative Environment",
            BoundaryType::BottomUp => "Rank 4: Inside-Out Emergence",
            BoundaryType::TopDown => "Rank 5: Outside-In Imposition",
            BoundaryType::Aside => "Rank 6: Intentional Aside",
```

```
        }
    }

    /// Natural language grammatical clause template for this boundary.
    pub fn linguistic_clause(&self) -> &'static str {
        match self {
            BoundaryType::Invariant => "It is an invariant law that",
            BoundaryType::Denotation => "The entity defines and executes",
            BoundaryType::Connotation => "In the context of the ambient universe",
            BoundaryType::BottomUp => "Projecting outward from entity into context",
            BoundaryType::TopDown => "As conditioned and imposed by the context",
            BoundaryType::Aside => "With the intent and rationale that",
        }
    }

    /// Returns the (open, close) character pair for this boundary type.
    pub fn delimiters(&self) -> (char, char) {
        match self {
            BoundaryType::Invariant => ('<', '>'),
            BoundaryType::Denotation => ('[', ']'),
            BoundaryType::Connotation => ('{', '}'),
            BoundaryType::BottomUp => ('[', '}'),
            BoundaryType::TopDown => ('{', ']'),
            BoundaryType::Aside => ('(', ')'),
        }
    }

    /// True if opening and closing delimiters come from different bracket families.
    pub fn is_asymmetric(&self) -> bool {
        matches!(self, BoundaryType::TopDown | BoundaryType::BottomUp)
    }

    /// True if boundary enforces unalterable immutability.
    pub fn is_immutable(&self) -> bool {
        matches!(self, BoundaryType::Invariant)
    }

    /// Resolves opening and closing characters into a valid BoundaryType.
    ///
    /// # Failpoint
    /// Returns `Err` if the pair forms an illegal boundary (e.g. `(`, `]` or `{`, `>`).
    pub fn from_delimiters(open: char, close: char) -> Result<Self, String> {
        match (open, close) {
            ('[', ']') => Ok(BoundaryType::Denotation),
            ('<', '>') => Ok(BoundaryType::Invariant),
            ('{', '}') => Ok(BoundaryType::Connotation),
            ('{', ']') => Ok(BoundaryType::TopDown),
            ('[', '}') => Ok(BoundaryType::BottomUp),
            ('(', ')') => Ok(BoundaryType::Aside),
            (o, c) => Err(format!(
                "FAILPOINT [IllegalBoundaryPair]: Delimiter pair '{o}...{c}' does not form a legal boundary. \
                 Legal pairs: [...] (Denotation), <...> (Invariant), {{...}} (Connotation), \
                 {{...] (Top-Down), [...}} (Bottom-Up), (...) (Aside)"
            )),
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Verification and String Formatter )
// ============================================================================
impl fmt::Display for BoundaryType {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let (o, c) = self.delimiters();
        write!(f, "{:?}({o}...{c})", self)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_boundary_ranking_hierarchy() {
        assert_eq!(BoundaryType::Invariant.rank(), 1);
        assert_eq!(BoundaryType::Denotation.rank(), 2);
```

```
        assert_eq!(BoundaryType::Connotation.rank(), 3);
        assert_eq!(BoundaryType::BottomUp.rank(), 4);
        assert_eq!(BoundaryType::TopDown.rank(), 5);
        assert_eq!(BoundaryType::Aside.rank(), 6);

        // Verify ordinal comparison preserves exact hierarchy
        assert!(BoundaryType::Invariant < BoundaryType::Denotation);
        assert!(BoundaryType::Denotation < BoundaryType::Connotation);
        assert!(BoundaryType::Connotation < BoundaryType::BottomUp);
        assert!(BoundaryType::BottomUp < BoundaryType::TopDown);
        assert!(BoundaryType::TopDown < BoundaryType::Aside);
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/cell.rs RS 345 LINES 1236 WORDS 11.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Cell Engine Identity, 8-bit to 5-Trit Codec, & Balanced Radix-3 Mapping )
// ============================================================================
//! OmniCode Core: Cell Architecture & Byte-Trit Hardware Transducer
//!
//! Provides the physical hardware bridge between 8-bit binary silicon
//! and OmniCode's Balanced Ternary Quad-State physics:
//!     - 8 Bits (256 states) <--> 5 Trits (243 symmetric states) + 13 Control Traps
//!     - Dense packing: 1 Cell = exactly 1 physical byte (zero struct overhead)
//!     - Invariant Out-of-band Quad-States:
//!         * 243: NULL / HALT (Void / Unmapped memory)
//!         * 244: AWAIT (Superposition / Pending / Lock)
//!         * 245: ROLLBACK (Veto / Tainted / Unwound cell)
//!         * 246: PROCEED (Affirmative sealed landmark)
//!         * 247..255: Hardware Traps & System Landmarks

use std::fmt;
use crate::core::linguistics::{GrammaticalClassification, GrammaticalEntity, PartOfSpeech};
use crate::core::ternary::QuadState;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Out-of-Band Constants, Trit Definition, and Cell Structures )
// ============================================================================

pub const BYTE_HALT: u8 = 243;
pub const BYTE_AWAIT: u8 = 244;
pub const BYTE_ROLLBACK: u8 = 245;
pub const BYTE_PROCEED: u8 = 246;
pub const BYTE_TRAP_BASE: u8 = 247;
pub const MAX_NATURAL_CELL_VALUE: u8 = 242;
pub const MIN_SIGNED_CELL_VALUE: i8 = -121;
pub const MAX_SIGNED_CELL_VALUE: i8 = 121;

/// The atomic ternary digit (Radix-3 balanced unit).
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Trit {
    /// -1: Negative polarity / Contra (denoted 'T')
    Neg,
    ///  0: Neutral balance / Equilibrium (denoted '0')
    Zero,
    /// +1: Positive polarity / Affirmation (denoted '1')
    Pos,
}

impl Trit {
    pub fn as_i8(&self) -> i8 {
        match self {
            Trit::Neg => -1,
            Trit::Zero => 0,
            Trit::Pos => 1,
        }
    }

    pub fn from_i8(val: i8) -> Result<Self, String> {
        match val {
            -1 => Ok(Trit::Neg),
            0 => Ok(Trit::Zero),
            1 => Ok(Trit::Pos),
            other => Err(format!("FAILPOINT [InvalidTrit]: '{other}' is not in {{-1, 0, +1}}")),
        }
    }

    pub fn char_symbol(&self) -> char {
        match self {
            Trit::Neg => 'T',
            Trit::Zero => '0',
            Trit::Pos => '1',
        }
    }

    /// Algebraic negation of a trit: NOT(x) = -x.
```

```
    pub fn negate(&self) -> Self {
        match self {
            Trit::Neg => Trit::Pos,
            Trit::Zero => Trit::Zero,
            Trit::Pos => Trit::Neg,
        }
    }
}

impl fmt::Display for Trit {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.char_symbol())
    }
}

/// The atomic storage unit of memory in OmniCode.
///
/// Holds 5 balanced trits (spanning 243 values) alongside its Quad-State resonance.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct Cell {
    pub trits: [Trit; 5],
    pub state: QuadState,
    pub is_marker: bool,
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Mathematical Conversion, Packing, Unpacking, and Word Operations )
// ============================================================================

impl Cell {
    /// Constructs a Cell with explicit trits and Quad-State.
    pub fn new(trits: [Trit; 5], state: QuadState) -> Self {
        Self { trits, state, is_marker: false }
    }

    /// Creates an empty cell at equilibrium (0 in value, AWAIT in state).
    pub fn equilibrium() -> Self {
        Self {
            trits: [Trit::Zero; 5],
            state: QuadState::Await,
            is_marker: true,
        }
    }

    /// Creates a cell initialized with a pure out-of-band Quad-State marker.
    pub fn from_state(state: QuadState) -> Self {
        Self {
            trits: [Trit::Zero; 5],
            state,
            is_marker: true,
        }
    }

    /// Encodes a natural number in [0 ..= 242] into a 5-trit Cell with state PROCEED.
    pub fn from_natural(value: u8) -> Result<Self, String> {
        if value > MAX_NATURAL_CELL_VALUE {
            return Err(format!(
                "FAILPOINT [CellOverflow]: Value {value} exceeds single-cell capacity ({MAX_NATURAL_CELL_VALUE}). \
                 Use multi-cell Word chaining."
            ));
        }
        let signed_val = (value as i16) + (MIN_SIGNED_CELL_VALUE as i16);
        Self::from_signed(signed_val as i8)
    }

    /// Encodes a signed balanced integer in [-121 ..= +121] into 5 balanced trits.
    pub fn from_signed(value: i8) -> Result<Self, String> {
        if !(MIN_SIGNED_CELL_VALUE..=MAX_SIGNED_CELL_VALUE).contains(&value) {
            return Err(format!(
                "FAILPOINT [CellRangeError]: Signed value {value} out of range [{MIN_SIGNED_CELL_VALUE} ..=
{MAX_SIGNED_CELL_VALUE}]."
            ));
        }

        let mut x = value;
        let mut trits = [Trit::Zero; 5];
```

```
        for i in 0..5 {
            let rem = ((x % 3) + 3) % 3;
            match rem {
                0 => {
                    trits[i] = Trit::Zero;
                    x /= 3;
                }
                1 => {
                    trits[i] = Trit::Pos;
                    x = (x - 1) / 3;
                }
                2 => {
                    trits[i] = Trit::Neg;
                    x = (x + 1) / 3;
                }
                _ => unreachable!(),
            }
        }

        Ok(Self {
            trits,
            state: QuadState::Proceed,
            is_marker: false,
        })
    }

    /// Decodes the 5 trits back to a signed integer in [-121 ..= +121].
    pub fn to_signed(&self) -> Option<i8> {
        if self.is_marker || self.state == QuadState::Halt {
            return None;
        }

        let mut total: i32 = 0;
        let mut multiplier: i32 = 1;

        for trit in &self.trits {
            total += (trit.as_i8() as i32) * multiplier;
            multiplier *= 3;
        }

        Some(total as i8)
    }

    /// Decodes the 5 trits back to a natural number in [0 ..= 242].
    pub fn to_natural(&self) -> Option<u8> {
        self.to_signed().map(|s| (s as i16 - (MIN_SIGNED_CELL_VALUE as i16)) as u8)
    }

    /// Packs this Cell into a single physical 8-bit byte.
    pub fn pack(&self) -> u8 {
        if self.is_marker {
            match self.state {
                QuadState::Halt => BYTE_HALT,
                QuadState::Await => BYTE_AWAIT,
                QuadState::Rollback => BYTE_ROLLBACK,
                QuadState::Proceed => BYTE_PROCEED,
            }
        } else {
            self.to_natural().unwrap_or(BYTE_HALT)
        }
    }

    /// Unpacks a single physical 8-bit byte into a Cell with full 5-trit fidelity and Quad-State.
    pub fn unpack(byte: u8) -> Self {
        match byte {
            BYTE_HALT => Self::from_state(QuadState::Halt),
            BYTE_AWAIT => Self::from_state(QuadState::Await),
            BYTE_ROLLBACK => Self::from_state(QuadState::Rollback),
            BYTE_PROCEED => Self::from_state(QuadState::Proceed),
            trap if trap >= BYTE_TRAP_BASE => {
                // Reserved system trap / hardware landmark
                Self::from_state(QuadState::Halt)
            }
            natural => {
                Self::from_natural(natural).unwrap_or_else(|_| Self::from_state(QuadState::Halt))
            }
        }
```

```
    }
}

impl GrammaticalEntity for Cell {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Preposition),
            None,
            "Atomic memory vessel; stores 5 balanced trits and Quad-State resonance",
            "Single 8-bit memory byte carrying payload and state",
        )
    }
}

impl GrammaticalEntity for Trit {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Adjective),
            None,
            "Atomic ternary digit (-1, 0, +1)",
            "Single Radix-3 unit of computational polarity",
        )
    }
}

impl fmt::Display for Cell {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        if self.state == QuadState::Halt {
            write!(f, "Cell[NULL]")
        } else if let Some(nat) = self.to_natural() {
            write!(
                f,
                "Cell[Val:{nat} | {}{}{}{}{} | {:?}]",
                self.trits[4], self.trits[3], self.trits[2], self.trits[1], self.trits[0], self.state
            )
        } else {
            write!(f, "Cell[{:?}]", self.state)
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Bijective Round-Trip Verification & Mathematical Unit Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_signed_round_trip_all_243_values() {
        for s in MIN_SIGNED_CELL_VALUE..=MAX_SIGNED_CELL_VALUE {
            let cell = Cell::from_signed(s).expect("Signed conversion must succeed");
            let recovered = cell.to_signed().expect("To signed must succeed");
            assert_eq!(s, recovered, "Failed round-trip for signed value {s}");
        }
    }

    #[test]
    fn test_natural_round_trip_all_243_values() {
        for n in 0..=MAX_NATURAL_CELL_VALUE {
            let cell = Cell::from_natural(n).expect("Natural conversion must succeed");
            let recovered = cell.to_natural().expect("To natural must succeed");
            assert_eq!(n, recovered, "Failed round-trip for natural number {n}");

            // Verify physical 8-bit byte pack and unpack
            let packed_byte = cell.pack();
            assert_eq!(packed_byte, n);
            let unpacked_cell = Cell::unpack(packed_byte);
            assert_eq!(unpacked_cell.to_natural(), Some(n));
            assert_eq!(unpacked_cell.state, QuadState::Proceed);
        }
    }

    #[test]
```

```
    fn test_out_of_band_quad_state_markers() {
        // Test HALT (NULL)
        let halt_cell = Cell::from_state(QuadState::Halt);
        let packed_halt = halt_cell.pack();
        assert_eq!(packed_halt, BYTE_HALT);
        let unpacked_halt = Cell::unpack(packed_halt);
        assert_eq!(unpacked_halt.state, QuadState::Halt);

        // Test AWAIT
        let await_cell = Cell::from_state(QuadState::Await);
        let packed_await = await_cell.pack();
        assert_eq!(packed_await, BYTE_AWAIT);
        let unpacked_await = Cell::unpack(packed_await);
        assert_eq!(unpacked_await.state, QuadState::Await);

        // Test ROLLBACK
        let rollback_cell = Cell::from_state(QuadState::Rollback);
        let packed_rollback = rollback_cell.pack();
        assert_eq!(packed_rollback, BYTE_ROLLBACK);
        let unpacked_rollback = Cell::unpack(packed_rollback);
        assert_eq!(unpacked_rollback.state, QuadState::Rollback);

        // Test PROCEED marker
        let proceed_cell = Cell::from_state(QuadState::Proceed);
        assert_eq!(proceed_cell.pack(), BYTE_PROCEED);
        let unpacked_proceed = Cell::unpack(BYTE_PROCEED);
        assert_eq!(unpacked_proceed.state, QuadState::Proceed);
    }

    #[test]
    fn test_zero_and_equilibrium_properties() {
        let zero_cell = Cell::from_signed(0).unwrap();
        assert_eq!(zero_cell.trits, [Trit::Zero; 5]);
        assert_eq!(zero_cell.to_signed(), Some(0));
        assert_eq!(zero_cell.to_natural(), Some(121)); // 121 is the balanced center of [0 .. 242]
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/filetype.rs

RS | 355 LINES | 1357 WORDS | 13.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode File Type Subsystem Identity, Taxonomy & Invariants )
// ============================================================================
//! OmniCode File Type Taxonomy & Formal Extension System
//!
//! Classifies and handles all recognized OmniCode file artifacts:
//!   - `.omni`: Canonical OmniCode Source Reel (Primary executable program logic,
//!     modules, libraries, and ontological domain models).
//!   - `.scroll`: Cognitive Episodic Scroll (Homoiconic execution trace reel,
//!     recording prompts, synthesized programs, Restorator repairs, failpoints,
//!     and natural responses for AI continual learning).

use std::fmt;
use std::path::Path;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( File Type Enumeration, Mime Types, and Constructor Declarations )
// ============================================================================
/// Formal file classifications in the OmniCode ecosystem.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum OmniFileType {
    /// `.omni` - Canonical executable source reel.
    SourceReel,
    /// `.scroll` - Homoiconic episodic cognitive trace and AI learning reel.
    EpisodicScroll,
    /// `.tape` - Sequential execution process tape led by a Tape Head.
    Tape,
}

pub const ALL_FILE_TYPES: [OmniFileType; 3] = [
    OmniFileType::SourceReel,
    OmniFileType::EpisodicScroll,
    OmniFileType::Tape,
];

impl OmniFileType {
    pub const EXT_OMNI: &'static str = "omni";
    pub const EXT_SCROLL: &'static str = "scroll";
    pub const EXT_TAPE: &'static str = "tape";

    pub const ALL: &'static [OmniFileType] = &ALL_FILE_TYPES;

    /// Returns all supported OmniCode file types.
    pub fn all() -> &'static [OmniFileType] {
        Self::ALL
    }

    /// Detects file type from extension string (with or without leading dot).
    pub fn from_extension(ext: &str) -> Option<Self> {
        let clean = ext.trim().trim_start_matches('.').to_lowercase();
        match clean.as_str() {
            "omni" => Some(OmniFileType::SourceReel),
            "scroll" => Some(OmniFileType::EpisodicScroll),
            "tape" => Some(OmniFileType::Tape),
            _ => None,
        }
    }

    /// Detects file type from a file path.
    pub fn from_path(path: impl AsRef<Path>) -> Option<Self> {
        let p = path.as_ref();
        p.extension()
            .and_then(|ext| ext.to_str())
            .and_then(Self::from_extension)
    }

    /// Checks if a file path is a known OmniCode artifact.
    pub fn is_omnicode_file(path: impl AsRef<Path>) -> bool {
        Self::from_path(path).is_some()
    }
}

```

```
// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Metadata Queries, Scaffolding Templates & Formatting Properties )
// ============================================================================
impl OmniFileType {
    /// Canonical file extension without leading dot.
    pub fn extension(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => Self::EXT_OMNI,
            OmniFileType::EpisodicScroll => Self::EXT_SCROLL,
            OmniFileType::Tape => Self::EXT_TAPE,
        }
    }

    /// File extension with leading dot.
    pub fn dotted_extension(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => ".omni",
            OmniFileType::EpisodicScroll => ".scroll",
            OmniFileType::Tape => ".tape",
        }
    }

    /// Human-readable display title.
    pub fn display_name(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => "OmniCode Source Reel",
            OmniFileType::EpisodicScroll => "OmniCode Episodic Scroll",
            OmniFileType::Tape => "OmniCode Process Tape",
        }
    }

    /// VS Code / IDE language identifier.
    pub fn language_id(&self) -> &'static str {
        "omnicode"
    }

    /// Formal MIME type.
    pub fn mime_type(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => "text/x-omnicode",
            OmniFileType::EpisodicScroll => "text/x-omnicode-scroll",
            OmniFileType::Tape => "text/x-omnicode-tape",
        }
    }

    /// Short technical summary.
    pub fn description(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => {
                "Canonical 4-Block executable program module conforming to balanced ternary physics."
            }
            OmniFileType::EpisodicScroll => {
                "Homoiconic episodic execution reel recording cognitive discourse, repairs, and telemetry."
            }
            OmniFileType::Tape => {
                "Sequential process tape consisting of a leading Tape Head and subordinate rolls."
            }
        }
    }

    /// Generates canonical 4-Block scaffolding template for this file type.
    pub fn scaffold_template(&self, identifier: &str, intent_or_prompt: &str) -> String {
        match self {
            OmniFileType::SourceReel => {
                format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( File Identity, Version, Author, Intent Aside, and Invariants )
# ============================================================================
Module < {name} > ( {intent} ) {{
    version: < "0.1.0" >,
    system: < OmniCode.Core >,
    safety: < MemorySafe :: Deterministic >
}}

# Invariant Axioms & Types
Type_{name} < +1 > ( Invariant definition )
```

```
# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environment Setup, Preconditions, Scoped Context, and Declarations )
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

# Initial state instance
state [ +1 ] ( Initialized state instance )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Core Transformations, Causality Vectors, Logic, and Caveat Pipelines )
# ================================================================================
Body [
    # Forward action: state acts upon transformation
    state --> next_state [ +1 ]
    next_state ! [ state = +1 ] ( Caveat verification )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions, Invariant Proofs, Teardown, and Commit Status )
# ================================================================================
Closing {{
    verify: [ next_state = +1 ],
    commit_status [ +1 ] ( Commit verified state )
}}
"#,
                    name = identifier,
                    intent = intent_or_prompt
                )
            }
            OmniFileType::EpisodicScroll => {
                format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
# ( Execution Trace ID, Prompt Intent, Inferred Types, and Neural Snapshot )
# ================================================================================
Scroll < EpistemicTrace.{session} > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    timestamp: < "{timestamp}" >,
    session_id: < "{session}" >,
    cognitive_mode: < NeuralSymbolicTransduction >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ( Hardware Invariants, Preconditions, and Initial Context )
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

# Original Cognitive Discourse Prompt:
user_prompt < "{prompt}" >

# ================================================================================
# BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & RESTORATION)
# ( Replay Tape: Synthesized Program, Sensory Telemetry & Immune Interventions )
# ================================================================================
Body [
    # Immune System Interventions (Restorator):
    repair_entry [ L1: CleanExecution --> "Zero repairs required." ]

    # Executed Turing Reel:
    synthesized_reel [
        state [ +1 ]
    ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING PROOF)
```

```
# ( Verification Proof, Final Quad-State, and Conversational Synthesis )
# ================================================================================
Closing {{
    commit_status [ <PROCEED> ],
    machine_utterance ( "Execution verified to +1:PROCEED." )
}}
"#,
                    session = identifier,
                    prompt = intent_or_prompt,
                    timestamp = "0"
                )
            }
            OmniFileType::Tape => {
                format!(
r#"# ================================================================================
# TAPE HEAD (COVENANT DECLARATION & PROCESS MANIFEST)
# ================================================================================
[Tape.{name}] : WHO {{Process}} --> DISPATCH {{{name}}}
    WHERE {{partition}} = [ 256 ]
    WHEN {{Mount}}
    WHY ( {intent} )
    HOW [ +1 : PROCEED ]
    :: < covenant > ( verified )

# ================================================================================
# SUBORDINATE ROLLS (SEQUENTIAL EXECUTION STREAM)
# ================================================================================
roll_1 [ 10 --> Buf @ 0 ] ( Inscribe initial state into cell 0 )
roll_2 [ Buf[0] = 10 ! [ +1 ] ] ( Verify invariant bounds )
roll_3 [ commit_status [ PROCEED ] ] ( Tape execution completed successfully )
"#,
                    name = identifier,
                    intent = intent_or_prompt
                )
            }
        }
    }
}

/// Resolves a requested file path against the canonical 6-Track filesystem hierarchy.
///
/// If `path` exists directly as specified, it is returned immediately.
/// Otherwise, searches through the designated Track directories:
/// - `<TRACK 0> : {Silicon & Bare-Metal Kernel}`
/// - `<TRACK 1> : {Host Engine & Virtual Machine}`
/// - `<TRACK 2> : {OmniCode Language & Standard Library}`
/// - `<TRACK 3> : {Applications & OmniStudio Interfaces}`
/// - `<TRACK 4> : {Variable Storage & Episodic Memory}`
/// - `<TRACK 5> : {Infrastructure, Operations & Archives}`
pub fn resolve_track_path(path: impl AsRef<Path>) -> std::path::PathBuf {
    let p = path.as_ref();
    if p.exists() {
        return p.to_path_buf();
    }

    const TRACK_PREFIXES: [&str; 6] = [
        "<TRACK 2> : {OmniCode Language & Standard Library}",
        "<TRACK 3> : {Applications & OmniStudio Interfaces}",
        "<TRACK 0> : {Silicon & Bare-Metal Kernel}",
        "<TRACK 1> : {Host Engine & Virtual Machine}",
        "<TRACK 4> : {Variable Storage & Episodic Memory}",
        "<TRACK 5> : {Infrastructure, Operations & Archives}",
    ];

    for prefix in &TRACK_PREFIXES {
        let candidate = std::path::PathBuf::from(prefix).join(p);
        if candidate.exists() {
            return candidate;
        }
    }

    p.to_path_buf()
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementation, Sealing & Unit Tests )
// ================================================================================
```

```
impl fmt::Display for OmniFileType {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}(.{})", self.display_name(), self.extension())
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_file_type_detection() {
        assert_eq!(OmniFileType::from_extension("omni"), Some(OmniFileType::SourceReel));
        assert_eq!(OmniFileType::from_extension(".omni"), Some(OmniFileType::SourceReel));
        assert_eq!(OmniFileType::from_extension("scroll"), Some(OmniFileType::EpisodicScroll));
        assert_eq!(OmniFileType::from_extension(".scroll"), Some(OmniFileType::EpisodicScroll));
        assert_eq!(OmniFileType::from_extension("tape"), Some(OmniFileType::Tape));
        assert_eq!(OmniFileType::from_extension(".tape"), Some(OmniFileType::Tape));
        assert_eq!(OmniFileType::from_extension("txt"), None);

        assert_eq!(OmniFileType::from_path("src/core.omni"), Some(OmniFileType::SourceReel));
        assert_eq!(OmniFileType::from_path("scrolls/test.scroll"), Some(OmniFileType::EpisodicScroll));
        assert_eq!(OmniFileType::from_path("lib/core/cell.tape"), Some(OmniFileType::Tape));
        assert!(OmniFileType::is_omnicode_file("lib/vector.omni"));
        assert!(OmniFileType::is_omnicode_file("history/session.scroll"));
        assert!(OmniFileType::is_omnicode_file("lib/core/cell.tape"));
        assert!(!OmniFileType::is_omnicode_file("Cargo.toml"));
    }

    #[test]
    fn test_scaffold_generation() {
        let omni_code = OmniFileType::SourceReel.scaffold_template("MathModule", "Basic math");
        assert!(omni_code.contains("Module < MathModule >"));
        assert!(omni_code.contains("BLOCK 1: METADATA BLOCK"));
        assert!(omni_code.contains("BLOCK 4: CLOSING BLOCK"));

        let scroll_code = OmniFileType::EpisodicScroll.scaffold_template("sess_1234", "Add 2 and 3");
        assert!(scroll_code.contains("Scroll < EpistemicTrace.sess_1234 >"));
        assert!(scroll_code.contains("commit_status [ <PROCEED> ]"));

        let tape_code = OmniFileType::Tape.scaffold_template("BootProcess", "Kernel Genesis");
        assert!(tape_code.contains("[Tape.BootProcess]"));
        assert!(tape_code.contains("TAPE HEAD"));
        assert!(tape_code.contains("SUBORDINATE ROLLS"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/linguistics.rs RS 1209 LINES 4115 WORDS 52.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Linguistic Engine Identity, 8 Parts of Speech, & Grammatical Classification )
// ============================================================================
//! OmniCode Core: Universal Linguistics & Grammatical Classification
//!
//! Maps every token, delimiter, vector, and quad-state directly into the
//! 8 canonical parts of speech in human natural language:
//!     1. Noun          (Entities, types, structures, literals, values)
//!     2. Pronoun       (Pointers, self-references, context handles)
//!     3. Verb          (Vectors, state transitions, operations, relations of being)
//!     4. Adjective     (Invariants, type qualifiers, contracts, quad-states)
//!     5. Adverb        (Execution manner, scheduling priority, negation)
//!     6. Preposition   (Indexing, spatial offsets, projections, scoping)
//!     7. Conjunction   (Connectives, boolean logic, pipeline caveats)
//!     8. Interjection   (Interrupts, traps, halts, parenthetical Asides)

use crate::core::boundaries::BoundaryType;
use crate::core::operators::{BeingRelation, Connective, DirectionalVector, Polarity};
use crate::core::ternary::QuadState;
use crate::parser::lexer::TokenType;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Grammatical Role Definitions, Classifications & Mapping Structures )
// ============================================================================

/// The canonical 8 parts of speech in natural human grammar.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PartOfSpeech {
    Noun,
    Pronoun,
    Verb,
    Adjective,
    Adverb,
    Preposition,
    Conjunction,
    Interjection,
}

impl PartOfSpeech {
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Noun => "Noun",
            Self::Pronoun => "Pronoun",
            Self::Verb => "Verb",
            Self::Adjective => "Adjective",
            Self::Adverb => "Adverb",
            Self::Preposition => "Preposition",
            Self::Conjunction => "Conjunction",
            Self::Interjection => "Interjection",
        }
    }
}

/// A comprehensive grammatical classification supporting primary, secondary,
/// and tertiary functional roles across natural language and computation.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct GrammaticalClassification {
    /// The primary syntactic and semantic role in the sentence.
    pub primary: PartOfSpeech,
    /// The secondary role when functioning within different syntactic contexts.
    pub secondary: Option<PartOfSpeech>,
    /// The tertiary role for edge cases or multi-tier semantics.
    pub tertiary: Option<PartOfSpeech>,
    /// Natural language explanation of its linguistic function.
    pub linguistic_role: &'static str,
    /// Corresponding computer science / systems counterpart.
    pub computing_counterpart: &'static str,
}

impl GrammaticalClassification {
    pub fn new(
```

```
        primary: PartOfSpeech,
        secondary: Option<PartOfSpeech>,
        tertiary: Option<PartOfSpeech>,
        linguistic_role: &'static str,
        computing_counterpart: &'static str,
    ) -> Self {
        Self {
            primary,
            secondary,
            tertiary,
            linguistic_role,
            computing_counterpart,
        }
    }

    pub fn is(&self, pos: PartOfSpeech) -> bool {
        self.primary == pos || self.secondary == Some(pos) || self.tertiary == Some(pos)
    }
}

/// Grammatical case of a pronoun in sentence structure.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounCase {
    /// Subjective / Nominative: Initiates action / holds execution frame (e.g., `I`, `we`, `you`, `he`, `she`, `it`, `they`).
    Subjective,
    /// Objective / Accusative: Recipient / target of action / inbound buffer (e.g., `me`, `us`, `him`, `her`, `them`).
    Objective,
    /// Reflexive / Introspective: Self-referential inspection / recursion (e.g., `myself`, `itself`, `themselves`).
    Reflexive,
    /// Possessive Determiner / Capability Token: Ownership / access rights (e.g., `my`, `your`, `his`, `her`, `its`, `our`, `their`).
    Possessive,
}

/// Person perspective of a pronoun.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounPerson {
    /// First person: Active self, kernel engine, or cooperative covenant (`I`, `me`, `we`, `us`, `myself`).
    First,
    /// Second person: Interlocutor, caller, user, or client socket (`you`, `yourself`, `your`).
    Second,
    /// Third person: External entity, data vessel, antecedent, or peer thread (`it`, `he`, `she`, `they`).
    Third,
}

/// Grammatical number of a pronoun.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounNumber {
    Singular,
    Plural,
}

/// The canonical spectrum of grammatical pronouns in OmniCode.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounKind {
    // --- First Person Singular (Kernel / Self) ---
    I,
    Me,
    Myself,
    My,
    Mine,

    // --- First Person Plural (Covenant / Composite) ---
    We,
    Us,
    Ourselves,
    Our,
    Ours,

    // --- Second Person (Caller / User / Interlocutor) ---
    You,
    Yourself,
    Yourselves,
    Your,
    Yours,

    // --- Third Person Singular Animate (Autonomous Worker / Peer) ---
```

```
    He,
    Him,
    Himself,
    His,
    She,
    Her,
    Herself,
    Hers,

    // --- Third Person Singular Inanimate (Data Vessel / Antecedent / Accumulator) ---
    It,
    Itself,
    Its,

    // --- Third Person Plural (Worker Pool / Swarm / Quorum) ---
    They,
    Them,
    Themselves,
    Their,
    Theirs,

    // --- System Context Handles & Wildcards ---
    SelfHandle,
    CallerHandle,
    OwnerHandle,
    Wildcard,
}

impl PronounKind {
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::I => "I",
            Self::Me => "me",
            Self::Myself => "myself",
            Self::My => "my",
            Self::Mine => "mine",
            Self::We => "we",
            Self::Us => "us",
            Self::Ourselves => "ourselves",
            Self::Our => "our",
            Self::Ours => "ours",
            Self::You => "you",
            Self::Yourself => "yourself",
            Self::Yourselves => "yourselves",
            Self::Your => "your",
            Self::Yours => "yours",
            Self::He => "he",
            Self::Him => "him",
            Self::Himself => "himself",
            Self::His => "his",
            Self::She => "she",
            Self::Her => "her",
            Self::Herself => "herself",
            Self::Hers => "hers",
            Self::It => "it",
            Self::Itself => "itself",
            Self::Its => "its",
            Self::They => "they",
            Self::Them => "them",
            Self::Themselves => "themselves",
            Self::Their => "their",
            Self::Theirs => "theirs",
            Self::SelfHandle => "self",
            Self::CallerHandle => "caller",
            Self::OwnerHandle => "owner",
            Self::Wildcard => "_",
        }
    }

    pub fn case(&self) -> PronounCase {
        match self {
            Self::I | Self::We | Self::You | Self::He | Self::She | Self::It | Self::They | Self::SelfHandle |
Self::CallerHandle | Self::OwnerHandle | Self::Wildcard => PronounCase::Subjective,
            Self::Me | Self::Us | Self::Him | Self::Her | Self::Them => PronounCase::Objective,
            Self::Myself | Self::Ourselves | Self::Yourself | Self::Yourselves | Self::Himself | Self::Herself |
Self::Itself | Self::Themselves => PronounCase::Reflexive,
            Self::My | Self::Mine | Self::Our | Self::Ours | Self::Your | Self::Yours | Self::His | Self::Hers |
Self::Its | Self::Their | Self::Theirs => PronounCase::Possessive,
```

```
        }
    }

    pub fn person(&self) -> PronounPerson {
        match self {
            Self::I | Self::Me | Self::Myself | Self::My | Self::Mine |
            Self::We | Self::Us | Self::Ourselves | Self::Our | Self::Ours |
            Self::SelfHandle => PronounPerson::First,

            Self::You | Self::Yourself | Self::Yourselves | Self::Your | Self::Yours |
            Self::CallerHandle => PronounPerson::Second,

            Self::He | Self::Him | Self::Himself | Self::His |
            Self::She | Self::Her | Self::Herself | Self::Hers |
            Self::It | Self::Itself | Self::Its |
            Self::They | Self::Them | Self::Themselves | Self::Their | Self::Theirs |
            Self::OwnerHandle | Self::Wildcard => PronounPerson::Third,
        }
    }

    pub fn number(&self) -> PronounNumber {
        match self {
            Self::We | Self::Us | Self::Ourselves | Self::Our | Self::Ours |
            Self::Yourselves |
            Self::They | Self::Them | Self::Themselves | Self::Their | Self::Theirs => PronounNumber::Plural,
            _ => PronounNumber::Singular,
        }
    }

    pub fn from_keyword(s: &str) -> Option<Self> {
        match s.to_lowercase().as_str() {
            "i" => Some(Self::I),
            "me" => Some(Self::Me),
            "myself" => Some(Self::Myself),
            "my" => Some(Self::My),
            "mine" => Some(Self::Mine),
            "we" => Some(Self::We),
            "us" => Some(Self::Us),
            "ourselves" => Some(Self::Ourselves),
            "our" => Some(Self::Our),
            "ours" => Some(Self::Ours),
            "you" => Some(Self::You),
            "yourself" => Some(Self::Yourself),
            "yourselves" => Some(Self::Yourselves),
            "your" => Some(Self::Your),
            "yours" => Some(Self::Yours),
            "he" => Some(Self::He),
            "him" => Some(Self::Him),
            "himself" => Some(Self::Himself),
            "his" => Some(Self::His),
            "she" => Some(Self::She),
            "her" => Some(Self::Her),
            "herself" => Some(Self::Herself),
            "hers" => Some(Self::Hers),
            "it" => Some(Self::It),
            "itself" => Some(Self::Itself),
            "its" => Some(Self::Its),
            "they" => Some(Self::They),
            "them" => Some(Self::Them),
            "themselves" => Some(Self::Themselves),
            "their" => Some(Self::Their),
            "theirs" => Some(Self::Theirs),
            "self" => Some(Self::SelfHandle),
            "caller" => Some(Self::CallerHandle),
            "owner" => Some(Self::OwnerHandle),
            "_" => Some(Self::Wildcard),
            _ => None,
        }
    }

    pub fn is_subjective(&self) -> bool {
        self.case() == PronounCase::Subjective
    }

    pub fn is_objective(&self) -> bool {
        self.case() == PronounCase::Objective
    }

```

```
    pub fn is_reflexive(&self) -> bool {
        self.case() == PronounCase::Reflexive
    }

    pub fn is_possessive(&self) -> bool {
        self.case() == PronounCase::Possessive
    }

    pub fn linguistic_role(&self) -> &'static str {
        match self {
            Self::I => "First-person singular subjective; represents the active executing identity / kernel",
            Self::Me => "First-person singular objective; represents self as the receiver of inbound action",
            Self::Myself => "First-person reflexive; drives introspective self-audit and invariant verification",
            Self::My | Self::Mine => "First-person possessive; denotes exclusive ownership and write capability",
            Self::We => "First-person plural subjective; denotes cooperative covenant (Kernel + User or Multi-Agent)",
            Self::Us => "First-person plural objective; denotes composite entity as target",
            Self::Ourselves => "First-person plural reflexive; drives collective group audit",
            Self::Our | Self::Ours => "First-person plural possessive; denotes shared covenant resource access",
            Self::You => "Second-person pronoun; identifies external interlocutor, user, or caller socket",
            Self::Yourself | Self::Yourselves => "Second-person reflexive; triggers remote caller verification",
            Self::Your | Self::Yours => "Second-person possessive; denotes borrowed caller capability token",
            Self::He | Self::She => "Third-person singular animate; designates specific peer worker or thread",
            Self::Him | Self::Her => "Third-person singular objective; identifies designated worker as target",
            Self::Himself | Self::Herself => "Third-person singular reflexive; isolates worker-internal state",
            Self::His | Self::Hers => "Third-person singular possessive; denotes worker-delegated capability",
            Self::It => "Third-person singular inanimate; represents data vessel, antecedent, or accumulator",
            Self::Itself => "Third-person reflexive; drives homoiconic tape rewriting or fixed-point recursion",
            Self::Its => "Third-person possessive; denotes vessel-internal property encapsulation",
            Self::They => "Third-person plural subjective; identifies worker pool, quorum, or parallel swarm",
            Self::Them => "Third-person plural objective; identifies worker pool as destination targets",
            Self::Themselves => "Third-person plural reflexive; verifies Byzantine swarm agreement or consensus",
            Self::Their | Self::Theirs => "Third-person plural possessive; denotes process-group capability mask",
            Self::SelfHandle => "Architectural local frame handle; resolves active stack/partition base",
            Self::CallerHandle => "Architectural caller handle; resolves invoking principal and frame return",
            Self::OwnerHandle => "Architectural owner handle; resolves root capability authority",
            Self::Wildcard => "Anonymous discard handle; suppresses destination binding or ignores noise",
        }
    }

    pub fn computing_counterpart(&self) -> &'static str {
        match self {
            Self::I => "Active Task PID / Execution Context Register / `&mut self`",
            Self::Me => "Destination Buffer in Self / Callee Register Inflow",
            Self::Myself => "Self-Audit Telemetry / Local Frame Invariant Check / Introspection",
            Self::My | Self::Mine => "Exclusive Memory Partition Lease / Write Capability Token",
            Self::We => "Composite Covenant Task / Dual-Principal Session / Multi-Agent Swarm",
            Self::Us => "Composite Destination Queue / Shared Inbound Buffer",
            Self::Ourselves => "Group Invariant Consensus Check / Quorum Self-Verification",
            Self::Our | Self::Ours => "Shared Memory Partition Capability / Covenant Lock",
            Self::You => "Caller Context Handle / `STDIN` Channel / Remote IPC Socket",
            Self::Yourself | Self::Yourselves => "Caller Self-Test RPC / Remote Health Probe",
            Self::Your | Self::Yours => "Borrowed Caller Capability Token / Foreign Slice Reference",
            Self::He | Self::She => "Designated Worker Thread PID / Specific Actor Descriptor",
            Self::Him | Self::Her => "Worker Inflow Channel / Task Mailbox",
            Self::Himself | Self::Herself => "Worker Local State Isolation / Actor Invariant Seal",
            Self::His | Self::Hers => "Worker Capability Delegation / Sub-Partition Lease",
            Self::It => "Pipeline Antecedent / Accumulator Register (`ACC`) / Top of Stack",
            Self::Itself => "Homoiconic Tape Rewriter / Circular Reel Wrap / Fixed-Point Recursion",
            Self::Its => "Vessel Field Encapsulation / Internal Property Offset",
            Self::They => "Process Group Descriptor / Parallel Thread Pool / Consensus Quorum",
            Self::Them => "Broadcast Channel / Multi-Partition Destination Slices",
            Self::Themselves => "Swarm Peer-to-Peer Consensus Alignment / Byzantine Quorum Proof",
            Self::Their | Self::Theirs => "Process Group Access Control List / Collective Capability Mask",
            Self::SelfHandle => "Frame Pointer Register (`FP`) / Local Execution Fiber",
            Self::CallerHandle => "Return Address / Caller Task Descriptor (`CALLER`)",
            Self::OwnerHandle => "Capability Authority Root / Security Principal Token",
            Self::Wildcard => "Bit Bucket (`/dev/null`) / Black Hole Register (`_`)",
        }
    }
}

impl GrammaticalEntity for PronounKind {
    fn classify(&self) -> GrammaticalClassification {
        let sec = match self.case() {
            PronounCase::Possessive => Some(PartOfSpeech::Adjective),
            PronounCase::Reflexive => Some(PartOfSpeech::Verb),
            PronounCase::Objective => Some(PartOfSpeech::Noun),
            PronounCase::Subjective => Some(PartOfSpeech::Noun),
```

```
        };
        GrammaticalClassification::new(
            PartOfSpeech::Pronoun,
            sec,
            None,
            self.linguistic_role(),
            self.computing_counterpart(),
        )
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Trait Implementations & Grammatical Classification Mappings )
// ============================================================================

pub trait GrammaticalEntity {
    fn classify(&self) -> GrammaticalClassification;
}

impl GrammaticalEntity for TokenType {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            // Delimiters
            TokenType::LBracket | TokenType::RBracket => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Preposition),
                None,
                "Denotation boundary container; spatially indexes inner state",
                "Array / Struct container; memory offset indexing `buf[idx]`",
            ),
            TokenType::LBrace | TokenType::RBrace => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Conjunction),
                None,
                "Invariant contract; specifies necessary ambient conditions",
                "Type invariant / Precondition constraint / Scope qualification",
            ),
            TokenType::LAngle | TokenType::RAngle => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                None,
                "Immutable axiom boundary; names high-order categorical domains",
                "Type parameter / Scale invariant / Module domain constant",
            ),
            TokenType::Aside(_) => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Adverb),
                None,
                "First-class teleological Aside; states machine intent and rationale",
                "Non-executable intent annotation / Compiler reasoning trace",
            ),

            // Grammatical Pronouns (Perspective, Identity, Ownership & Capability)
            TokenType::Pronoun(p) => p.classify(),

            // Polarity & Boundary Cast Operators
            TokenType::AffixedIdent { polarity, .. } => match polarity {
                Polarity::Abstract => GrammaticalClassification::new(
                    PartOfSpeech::Adjective,
                    Some(PartOfSpeech::Noun),
                    None,
                    "Abstract archetype polarity modifier (+Word); denotes universal concept or intension",
                    "Uninstantiated archetype / Type specification / Zero-cost blueprint",
                ),
                Polarity::Concrete => GrammaticalClassification::new(
                    PartOfSpeech::Noun,
                    Some(PartOfSpeech::Adjective),
                    None,
                    "Concrete substance polarity modifier (-Word / Word-); denotes physical instance",
                    "Physical memory allocation / Concrete hardware register footprint",
                ),
            },
            TokenType::BoundaryCast { .. } => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
```

```
                "Boundary casting operator; modulates entity essence across boundary rank",
                "Type cast / Transduction between abstract and concrete representations",
            ),

            // Directional Vectors (Verbs of Action & Causation)
            TokenType::ActUpon => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Forward causation; acts upon target entity",
                "Store / Mutation / Pipeline emission / MOV",
            ),
            TokenType::ActAgain => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Retroactive reflection; draws from or reflects back upon source",
                "Load / Feedback / State rollback inspection",
            ),
            TokenType::BuildsTo => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Ascending action; synthesizes lower primitive into higher concept",
                "Type promotion / Emergence / Layering up to abstraction",
            ),
            TokenType::BreaksDownFrom => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Descending action; decomposes abstract concept into primitives",
                "Destructuring / Register unpack / Compilation lowering",
            ),
            TokenType::Major => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Major scale vector; ascending forward diagonal leap",
                "Major scale promotion / Leap to higher abstraction tier (`/-^`)",
            ),
            TokenType::Minor => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Minor scale vector; descending forward diagonal step",
                "Minor scale reduction / Settling to lower execution tier (`\\-v`)",
            ),
            TokenType::Great => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Great scale vector; ascending backward diagonal reflection",
                "Great scale elevation / Architectural reflection (`\\-^`)",
            ),
            TokenType::Small => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Small scale vector; descending backward diagonal micro-adjustment",
                "Small scale reduction / Micro-destructuring (`/-v`)",
            ),
            TokenType::UpperRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant I compass vector; forward ascending northeast movement",
                "2D Quadrant I vector: +X, +Y (`/^`)",
            ),
            TokenType::LowerRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant IV compass vector; forward descending southeast movement",
                "2D Quadrant IV vector: +X, -Y (`\\v`)",
            ),
            TokenType::UpperLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
```

```
                None,
                "Quadrant II compass vector; backward ascending northwest movement",
                "2D Quadrant II vector: -X, +Y (`^\\`)",
            ),
            TokenType::LowerLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant III compass vector; backward descending southwest movement",
                "2D Quadrant III vector: -X, -Y (`v/`)",
            ),

            // Relations of Being (Verbs of Identity & State)
            TokenType::Is => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Copula of being; asserts exact equivalence and identity",
                "Equality assertion (`==`) / Identity binding",
            ),
            TokenType::IsNot => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Negative copula; asserts distinctness and exclusion",
                "Inequality assertion (`!=`)",
            ),
            TokenType::ToBeLike => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Analogy copula; asserts structural likeness or homomorphism",
                "Structural compatibility / Duck typing / Approximate equivalence",
            ),
            TokenType::InvariantEquivalence => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Invariant copula; immutable balance, identity, and eternal equilibrium (<=>)",
                "Invariant to be / Axiomatic Equivalence (`<=>`)",
            ),
            TokenType::LessThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; strictly less than within context bounds (<)",
                "Contextual less than comparison (`[ { A } < { B } ]`)",
            ),
            TokenType::GreaterThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; strictly greater than within context bounds (>)",
                "Contextual greater than comparison (`[ { A } > { B } ]`)",
            ),
            TokenType::LessEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; less than or equal within context bounds (<=)",
                "Contextual less than or equal comparison (`[ { A } <= { B } ]`)",
            ),
            TokenType::GreaterEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; greater than or equal within context bounds (>=)",
                "Contextual greater than or equal comparison (`[ { A } >= { B } ]`)",
            ),

            // Connectives
            TokenType::But => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Interjection),
                None,
                "Adversative conjunction; introduces strict caveat condition",
                "Caveat guard; triggers `-1: ROLLBACK` if caveat fails",
            ),
```

```
            TokenType::And => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Preposition),
                None,
                "Cumulative conjunction; asserts simultaneous co-presence",
                "Conjunction (`&&`) / Simultaneous consensus",
            ),
            TokenType::InRelationTo => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
                "Relational projection; positions entity in relation to a category",
                "Type annotation (`x : Type`) / Contextual role assignment",
            ),
            TokenType::Or => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Adverb),
                None,
                "Disjunctive conjunction; introduces alternatives or choices",
                "Disjunction (`||`) / Non-deterministic branch",
            ),
            TokenType::Not => GrammaticalClassification::new(
                PartOfSpeech::Adverb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Negating adverb; inverts the state or predicate across balance line",
                "Logical / Algebraic negation",
            ),
            TokenType::BoundPair => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Noun),
                None,
                "Hand-in-hand inseparable dual binding; binds entities so neither can be referenced without the other",
                "Inseparable atomic pair / Dual-rail entangled binding (`A&B`)",
            ),

            // Prepositions & Spatial Locators
            TokenType::At => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Noun),
                None,
                "Spatial preposition specifying absolute or relative location, address, or memory site",
                "Memory address / MMIO locator / Absolute site (`Storehouse @ Offset`)",
            ),

            // Measures & Proportions
            TokenType::Percent => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                None,
                "Proportional measure specifying capacity fraction, ratio, or utilization percentage",
                "Capacity fullness ratio / Percentage scale (`Capacity % 80`)",
            ),

            // Ranges
            TokenType::DotDot => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Noun),
                None,
                "Range span preposition; defines closed interval bounds between coordinates",
                "Coordinate interval / Memory span `[0 .. 256]`",
            ),

            // Cardinal Classification Clauses (The 5 W's and 1 H)
            TokenType::Who => GrammaticalClassification::new(
                PartOfSpeech::Pronoun,
                Some(PartOfSpeech::Noun),
                None,
                "Agent pronoun clause; identifies actor, authority, caller, or process identity",
                "Process owner / Security principal / Caller identity (`WHO`)",
            ),
            TokenType::What => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Pronoun),
                None,
                "Substance noun clause; identifies entity, payload, data vessel, or type",
                "Data vessel / Entity payload / Operand specification (`WHAT`)",
            ),
```

```
            TokenType::When => GrammaticalClassification::new(
                PartOfSpeech::Adverb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Temporal adverb clause; identifies execution cadence, lifecycle block, or trigger",
                "Execution phase / Scheduler quantum / Event trigger (`WHEN`)",
            ),
            TokenType::Where => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Conjunction),
                None,
                "Spatial preposition clause; identifies memory location, address, scope, or invariant bounds",
                "Spatial address / MMIO site / Invariant scope (`WHERE`)",
            ),
            TokenType::Why => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Adjective),
                None,
                "Teleological inquiry clause; specifies rationale, contractual reason, or axiom",
                "Teleological rationale / Contractual reason / Axiom link (`WHY`)",
            ),
            TokenType::How => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adverb),
                None,
                "Causal mechanism clause; specifies vector method, transducer, or execution policy",
                "Causal mechanism / Vector method / Transducer (`HOW`)",
            ),

            // Natural Arithmetic Keywords
            TokenType::Plus => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of addition; unites quantities",
                "Arithmetic addition opcode",
            ),
            TokenType::Minus => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of subtraction; separates quantities",
                "Arithmetic subtraction opcode",
            ),
            TokenType::Times => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of multiplication; scales quantities",
                "Arithmetic multiplication opcode",
            ),
            TokenType::Div => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of division; partitions quantities",
                "Arithmetic division opcode",
            ),

            // Infer
            TokenType::Infer => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Verb),
                None,
                "Interrogative inquiry; seeks resolution or truth state",
                "Pattern match query / Ontological inference operator",
            ),

            // Literals
            TokenType::Identifier(_) => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Pronoun),
                None,
                "Named subject, object, or entity in discourse",
                "Variable name / Symbol / Type identifier",
            ),
            TokenType::Number(_) => GrammaticalClassification::new(
                PartOfSpeech::Noun,
```

```
                Some(PartOfSpeech::Adjective),
                None,
                "Numeric scalar value; quantifies entities",
                "Immediate scalar numeric literal",
            ),
            TokenType::StringLit(_) => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                None,
                None,
                "Textual noun; semantic content literal",
                "String literal / Byte payload",
            ),
            TokenType::Ternary(state) => state.classify(),

            TokenType::Eof => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                None,
                None,
                "Terminal punctuation sealing the discourse",
                "End of file / Stream termination",
            ),
        }
    }
}

impl GrammaticalEntity for QuadState {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            QuadState::Proceed => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                Some(PartOfSpeech::Interjection),
                "Affirmative state of truth, covenant alignment, and forward momentum",
                "Logic +1 / Monotonic commit / Success",
            ),
            QuadState::Await => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Verb),
                None,
                "Superposition state; yielding in wait for consensus",
                "Logic 0 / Cooperative schedule yield / Non-blocking pending",
            ),
            QuadState::Rollback => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Verb),
                Some(PartOfSpeech::Adjective),
                "Exclamation of veto; immediate transactional rewind",
                "Logic -1 / Transaction abort and rollback",
            ),
            QuadState::Halt => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Noun),
                None,
                "Terminal trap of void and non-existence; complete cessation",
                "Logic NULL / Fatal CPU trap / Irreducible collapse",
            ),
        }
    }
}

impl GrammaticalEntity for BoundaryType {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            BoundaryType::Denotation => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Preposition),
                None,
                "Concrete essence and mutable data literal",
                "Data structure container / Mutable memory slice",
            ),
            BoundaryType::Invariant => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                None,
                "Immutable axiom and universal physical constant",
                "Constant literal / Read-only memory / Axiomatic constraint",
            ),
            BoundaryType::Connotation => GrammaticalClassification::new(
```

```
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Conjunction),
                None,
                "Ambient context, required capabilities, and environmental predicates",
                "Precondition contract / Ambient capabilities / Context scope",
            ),
            BoundaryType::TopDown => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Adjective),
                None,
                "Outside-in conditioning: the environment dictates entity essence",
                "Hardware MMIO window / Environmental dependency injection",
            ),
            BoundaryType::BottomUp => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
                "Inside-out emergence: entity essence projects into environment",
                "Polymorphic emission / Register export to ABI",
            ),
            BoundaryType::Aside => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Adverb),
                None,
                "Aside to; teleological intent container preserving reasoning trace",
                "First-class intent container / AI reasoning trace",
            ),
        }
    }
}

impl GrammaticalEntity for DirectionalVector {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            DirectionalVector::ActUpon => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Forward causation; acts upon target",
                "Forward pipeline dispatch / Store operation",
            ),
            DirectionalVector::ActAgain => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Retroactive reflection; draws from or re-evaluates source",
                "Feedback loop / Rollback inspection / Load operation",
            ),
            DirectionalVector::BuildsTo => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Ascending synthesis; primitives build up to abstraction",
                "Type promotion / Abstraction synthesis",
            ),
            DirectionalVector::BreaksDownFrom => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Descending decomposition; concept lowers into hardware primitives",
                "Destructuring / Register lowering",
            ),
            DirectionalVector::Major => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Major scale vector; ascending forward diagonal leap",
                "Major scale promotion (`/-^`)",
            ),
            DirectionalVector::Minor => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Minor scale vector; descending forward diagonal step",
                "Minor scale reduction (`\\-v`)",
            ),
            DirectionalVector::Great => GrammaticalClassification::new(
                PartOfSpeech::Verb,
```

```
                Some(PartOfSpeech::Adjective),
                None,
                "Great scale vector; ascending backward diagonal reflection",
                "Great scale elevation (`\\-^`)",
            ),
            DirectionalVector::Small => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Small scale vector; descending backward diagonal micro-adjustment",
                "Small scale reduction (`/-v`)",
            ),
            DirectionalVector::UpperRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant I compass vector; forward ascending movement",
                "2D Quadrant I vector: +X, +Y (`/^`)",
            ),
            DirectionalVector::LowerRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant IV compass vector; forward descending movement",
                "2D Quadrant IV vector: +X, -Y (`\\v`)",
            ),
            DirectionalVector::UpperLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant II compass vector; backward ascending movement",
                "2D Quadrant II vector: -X, +Y (`^\\`)",
            ),
            DirectionalVector::LowerLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant III compass vector; backward descending movement",
                "2D Quadrant III vector: -X, -Y (`v/`)",
            ),
        }
    }
}

impl GrammaticalEntity for BeingRelation {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            BeingRelation::Is => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Identity copula: exact equivalence of essence",
                "Equality assertion / Equational binding",
            ),
            BeingRelation::IsNot => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Negative copula: distinctness and mutual exclusion",
                "Inequality assertion",
            ),
            BeingRelation::ToBeLike => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Analogy copula: structural or behavioral compatibility",
                "Homomorphism / Approximate equality",
            ),
            BeingRelation::Equivalence => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Invariant copula: immutable balance, identity, and eternal equilibrium (<=>)",
                "Invariant to be / Axiomatic Equivalence",
            ),
            BeingRelation::LessThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
```

```
                None,
                "Relational ordering: strictly less than within context bounds (<)",
                "Less than relational query",
            ),
            BeingRelation::GreaterThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering: strictly greater than within context bounds (>)",
                "Greater than relational query",
            ),
            BeingRelation::LessEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering: less than or equal within context bounds (<=)",
                "Less than or equal relational query",
            ),
            BeingRelation::GreaterEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering: greater than or equal within context bounds (>=)",
                "Greater than or equal relational query",
            ),
        }
    }
}

impl GrammaticalEntity for Connective {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            Connective::But => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Interjection),
                None,
                "Adversative conjunction: caveat requirement that vetoes on failure",
                "Guard condition with automatic rollback",
            ),
            Connective::And => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Preposition),
                None,
                "Cumulative conjunction: co-present simultaneous execution",
                "Conjunction (`&&`) / Joint consensus",
            ),
            Connective::InRelationTo => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
                "Relational projection: positions an entity within a role or type",
                "Type annotation (`:`) / Role assignment",
            ),
            Connective::Or => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Adverb),
                None,
                "Disjunctive conjunction: alternative path or branching",
                "Disjunction (`||`)",
            ),
            Connective::Not => GrammaticalClassification::new(
                PartOfSpeech::Adverb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Negating adverb: inverts the evaluation balance",
                "Algebraic negation (`--`)",
            ),
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Linguistic Unit Tests & Invariant Verification )
// ============================================================================
#[cfg(test)]
mod tests {
```

```rust
    use super::*;

    #[test]
    fn test_eight_parts_of_speech_coverage() {
        let act_upon = TokenType::ActUpon.classify();
        assert_eq!(act_upon.primary, PartOfSpeech::Verb);
        assert_eq!(act_upon.secondary, Some(PartOfSpeech::Preposition));

        let but = TokenType::But.classify();
        assert_eq!(but.primary, PartOfSpeech::Conjunction);
        assert_eq!(but.secondary, Some(PartOfSpeech::Interjection));

        let bracket = TokenType::LBracket.classify();
        assert_eq!(bracket.primary, PartOfSpeech::Noun);
        assert_eq!(bracket.secondary, Some(PartOfSpeech::Preposition));

        let brace = TokenType::LBrace.classify();
        assert_eq!(brace.primary, PartOfSpeech::Adjective);
        assert_eq!(brace.secondary, Some(PartOfSpeech::Conjunction));

        let aside = TokenType::Aside("intent".to_string()).classify();
        assert_eq!(aside.primary, PartOfSpeech::Interjection);
        assert_eq!(aside.secondary, Some(PartOfSpeech::Adverb));

        let not = TokenType::Not.classify();
        assert_eq!(not.primary, PartOfSpeech::Adverb);
        assert_eq!(not.secondary, Some(PartOfSpeech::Conjunction));

        let relation = TokenType::InRelationTo.classify();
        assert_eq!(relation.primary, PartOfSpeech::Preposition);
        assert_eq!(relation.secondary, Some(PartOfSpeech::Verb));

        let bound = TokenType::BoundPair.classify();
        assert_eq!(bound.primary, PartOfSpeech::Conjunction);
        assert_eq!(bound.secondary, Some(PartOfSpeech::Noun));

        let at = TokenType::At.classify();
        assert_eq!(at.primary, PartOfSpeech::Preposition);
        assert_eq!(at.secondary, Some(PartOfSpeech::Noun));

        let pct = TokenType::Percent.classify();
        assert_eq!(pct.primary, PartOfSpeech::Adjective);
        assert_eq!(pct.secondary, Some(PartOfSpeech::Noun));
    }

    #[test]
    fn test_quad_state_linguistic_classifications() {
        let proceed = QuadState::Proceed.classify();
        assert_eq!(proceed.primary, PartOfSpeech::Adjective);
        assert!(proceed.is(PartOfSpeech::Interjection));

        let rollback = QuadState::Rollback.classify();
        assert_eq!(rollback.primary, PartOfSpeech::Interjection);
        assert!(rollback.is(PartOfSpeech::Verb));
    }

    #[test]
    fn test_pronoun_matrix_and_capability_classifications() {
        // 1. Subjective (Agency / Execution context)
        let p_i = PronounKind::I;
        assert_eq!(p_i.case(), PronounCase::Subjective);
        assert_eq!(p_i.person(), PronounPerson::First);
        assert_eq!(p_i.number(), PronounNumber::Singular);
        assert!(p_i.is_subjective());
        let class_i = p_i.classify();
        assert_eq!(class_i.primary, PartOfSpeech::Pronoun);
        assert_eq!(class_i.secondary, Some(PartOfSpeech::Noun));
        assert!(class_i.computing_counterpart.contains("&mut self"));

        // 2. Objective (Inbound target)
        let p_me = PronounKind::Me;
        assert_eq!(p_me.case(), PronounCase::Objective);
        assert_eq!(p_me.person(), PronounPerson::First);
        assert!(p_me.is_objective());

        // 3. Reflexive (Introspection & self-audit)
        let p_myself = PronounKind::Myself;
        assert_eq!(p_myself.case(), PronounCase::Reflexive);
```

```
        assert!(p_myself.is_reflexive());
        let class_myself = p_myself.classify();
        assert_eq!(class_myself.secondary, Some(PartOfSpeech::Verb));
        assert!(class_myself.computing_counterpart.contains("Self-Audit"));

        let p_itself = PronounKind::Itself;
        assert_eq!(p_itself.case(), PronounCase::Reflexive);
        assert_eq!(p_itself.person(), PronounPerson::Third);
        let class_itself = p_itself.classify();
        assert!(class_itself.computing_counterpart.contains("Homoiconic"));

        let p_themselves = PronounKind::Themselves;
        assert_eq!(p_themselves.case(), PronounCase::Reflexive);
        assert_eq!(p_themselves.number(), PronounNumber::Plural);
        assert!(p_themselves.classify().computing_counterpart.contains("Consensus"));

        // 4. Possessive (Ownership & Capability tokens)
        let p_my = PronounKind::My;
        assert_eq!(p_my.case(), PronounCase::Possessive);
        assert!(p_my.is_possessive());
        let class_my = p_my.classify();
        assert_eq!(class_my.secondary, Some(PartOfSpeech::Adjective));
        assert!(class_my.computing_counterpart.contains("Capability Token"));

        let p_their = PronounKind::Their;
        assert_eq!(p_their.case(), PronounCase::Possessive);
        assert_eq!(p_their.number(), PronounNumber::Plural);

        // 5. Keyword parsing round-trip
        assert_eq!(PronounKind::from_keyword("we"), Some(PronounKind::We));
        assert_eq!(PronounKind::from_keyword("you"), Some(PronounKind::You));
        assert_eq!(PronounKind::from_keyword("it"), Some(PronounKind::It));
        assert_eq!(PronounKind::from_keyword("she"), Some(PronounKind::She));
        assert_eq!(PronounKind::from_keyword("he"), Some(PronounKind::He));
        assert_eq!(PronounKind::from_keyword("herself"), Some(PronounKind::Herself));
        assert_eq!(PronounKind::from_keyword("himself"), Some(PronounKind::Himself));
        assert_eq!(PronounKind::from_keyword("theirs"), Some(PronounKind::Theirs));
        assert_eq!(PronounKind::from_keyword("self"), Some(PronounKind::SelfHandle));
        assert_eq!(PronounKind::from_keyword("caller"), Some(PronounKind::CallerHandle));
        assert_eq!(PronounKind::from_keyword("owner"), Some(PronounKind::OwnerHandle));
        assert_eq!(PronounKind::from_keyword("_"), Some(PronounKind::Wildcard));

        // 6. TokenType delegation
        let tok = TokenType::Pronoun(PronounKind::Myself);
        assert_eq!(tok.classify().primary, PartOfSpeech::Pronoun);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mmio.rs RS 300 LINES 1055 WORDS 9.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Memory-Mapped I/O: Bare-Metal VGA Text Buffer & Serial Console )
// ============================================================================
//! OmniCode Physical MMIO Subsystem
//!
//! Provides the bare-metal hardware interaction layer for the bootable machine:
//!
//! # Physical Memory Architecture
//! - **0x000B8000 .. 0x000B8FA0**: Standard VGA text-mode video buffer (80 cols × 25 rows = 2000 cells).
//!   Each cell consists of an ASCII character code byte + color attribute byte.
//! - **0x03F8 .. 0x03FF**: COM1 UART 16550 Serial Port for low-level kernel telemetry.
//! - **0x00100000 .. 0x00FFFFFF**: Extended 16MB RAM partition for kernel and reels.
//!
//! When instructions on `Track 0 [KERNEL]` inscribe physical words to memory,
//! addresses in the MMIO range drive hardware video output and serial telemetry.

use crate::core::word::Word64;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( VGA Data Structures, Colors, and Serial Port Models )
// ============================================================================

pub const VGA_BUFFER_BASE: u64 = 0x000B8000;
pub const VGA_COLS: usize = 80;
pub const VGA_ROWS: usize = 25;
pub const VGA_CELL_COUNT: usize = VGA_COLS * VGA_ROWS;

pub const COM1_SERIAL_PORT: u64 = 0x03F8;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u8)]
pub enum VgaColor {
    Black = 0,
    Blue = 1,
    Green = 2,
    Cyan = 3,
    Red = 4,
    Magenta = 5,
    Brown = 6,
    LightGray = 7,
    DarkGray = 8,
    LightBlue = 9,
    LightGreen = 10,
    LightCyan = 11,
    LightRed = 12,
    Pink = 13,
    Yellow = 14,
    White = 15,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct VgaCell {
    pub ascii: u8,
    pub attribute: u8,
}

impl Default for VgaCell {
    fn default() -> Self {
        Self {
            ascii: b' ',
            attribute: (VgaColor::Black as u8) << 4 | (VgaColor::LightGray as u8),
        }
    }
}

pub struct VgaBuffer {
    pub cells: [VgaCell; VGA_CELL_COUNT],
    pub cursor_col: usize,
    pub cursor_row: usize,
}

impl Default for VgaBuffer {
```

```
    fn default() -> Self {
        Self::new()
    }
}

pub struct SerialPort {
    pub buffer: Vec<u8>,
}

impl Default for SerialPort {
    fn default() -> Self {
        Self::new()
    }
}

pub struct PhysicalMemoryMap {
    pub vga: VgaBuffer,
    pub serial: SerialPort,
    pub ram_cells: Vec<Word64>,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( MMIO Driver Methods, Video Screen Rendering, and Address Dispatch )
// ============================================================================

impl VgaBuffer {
    /// Creates an empty VGA text buffer cleared with light gray on black.
    pub fn new() -> Self {
        Self {
            cells: [VgaCell::default(); VGA_CELL_COUNT],
            cursor_col: 0,
            cursor_row: 0,
        }
    }

    /// Clears the video text buffer.
    pub fn clear(&mut self) {
        self.cells = [VgaCell::default(); VGA_CELL_COUNT];
        self.cursor_col = 0;
        self.cursor_row = 0;
    }

    /// Inscribes a single character byte at the current cursor location.
    pub fn write_char(&mut self, ch: u8, fg: VgaColor, bg: VgaColor) {
        if ch == b'\n' {
            self.new_line();
            return;
        }

        let attr = ((bg as u8) << 4) | (fg as u8);
        let idx = self.cursor_row * VGA_COLS + self.cursor_col;
        if idx < VGA_CELL_COUNT {
            self.cells[idx] = VgaCell { ascii: ch, attribute: attr };
        }

        self.cursor_col += 1;
        if self.cursor_col >= VGA_COLS {
            self.new_line();
        }
    }

    /// Writes an ASCII string to the video buffer.
    pub fn write_str(&mut self, s: &str, fg: VgaColor, bg: VgaColor) {
        for b in s.bytes() {
            self.write_char(b, fg, bg);
        }
    }

    /// Advances the cursor to the beginning of the next row, scrolling if needed.
    pub fn new_line(&mut self) {
        self.cursor_col = 0;
        if self.cursor_row + 1 < VGA_ROWS {
            self.cursor_row += 1;
        } else {
            // Scroll buffer up by 1 line
            for row in 1..VGA_ROWS {
                for col in 0..VGA_COLS {
```

```
                    let from_idx = row * VGA_COLS + col;
                    let to_idx = (row - 1) * VGA_COLS + col;
                    self.cells[to_idx] = self.cells[from_idx];
                }
            }
            // Clear last row
            let blank = VgaCell::default();
            for col in 0..VGA_COLS {
                let last_idx = (VGA_ROWS - 1) * VGA_COLS + col;
                self.cells[last_idx] = blank;
            }
        }
    }

    /// Renders the 80x25 video buffer as a framed ASCII string for terminal display.
    pub fn render_screen(&self, max_rows: usize) -> String {
        let mut out = String::new();
        out.push_str("\x1b[1;34m┌──────────────────────────────── [BARE-METAL VGA CONSOLE @ 0x000B8000] ────────────────────────────────┐\x1b[0m\n");
        let rows_to_show = max_rows.min(VGA_ROWS);
        for row in 0..rows_to_show {
            out.push_str("\x1b[1;34m│\x1b[0m ");
            for col in 0..VGA_COLS {
                let cell = self.cells[row * VGA_COLS + col];
                let ch = if (32..=126).contains(&cell.ascii) {
                    cell.ascii as char
                } else {
                    ' '
                };
                out.push(ch);
            }
            out.push_str(" \x1b[1;34m│\x1b[0m\n");
        }

        out.push_str("\x1b[1;34m└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘\x1b[0m\n");
        out
    }
}

impl SerialPort {
    pub fn new() -> Self {
        Self { buffer: Vec::new() }
    }

    pub fn write_byte(&mut self, b: u8) {
        self.buffer.push(b);
    }

    pub fn write_str(&mut self, s: &str) {
        self.buffer.extend_from_slice(s.as_bytes());
    }

    pub fn read_output(&self) -> String {
        String::from_utf8_lossy(&self.buffer).to_string()
    }
}

impl PhysicalMemoryMap {
    pub fn new(ram_capacity: usize) -> Self {
        Self {
            vga: VgaBuffer::new(),
            serial: SerialPort::new(),
            ram_cells: vec![Word64::zero(); ram_capacity],
        }
    }

    /// Dispatches a 64-bit word write to RAM or hardware MMIO peripherals.
    pub fn write(&mut self, address: u64, word: Word64) -> Result<(), String> {
        if (VGA_BUFFER_BASE..VGA_BUFFER_BASE + (VGA_CELL_COUNT as u64)).contains(&address) {
            let offset = (address - VGA_BUFFER_BASE) as usize;
            let ch = (word.payload().to_i64() as u8).max(32);
            let attr = ((word.raw >> 8) & 0xFF) as u8;
            self.vga.cells[offset] = VgaCell {
                ascii: ch,
                attribute: if attr == 0 { 0x07 } else { attr },
            };
            return Ok(());
```

```
        }

        if address == COM1_SERIAL_PORT {
            let b = (word.payload().to_i64() & 0xFF) as u8;
            self.serial.write_byte(b);
            return Ok(());
        }

        let idx = address as usize;
        if idx < self.ram_cells.len() {
            self.ram_cells[idx] = word;
            Ok(())
        } else {
            Err(format!("FAILPOINT [PhysicalBusFault]: Address 0x{:08X} unmapped on physical bus", address))
        }
    }

    /// Reads a 64-bit word from RAM or MMIO.
    pub fn read(&self, address: u64) -> Option<Word64> {
        if (VGA_BUFFER_BASE..VGA_BUFFER_BASE + (VGA_CELL_COUNT as u64)).contains(&address) {
            let offset = (address - VGA_BUFFER_BASE) as usize;
            let cell = self.vga.cells[offset];
            return Some(Word64::from_signed_int(cell.ascii as i64));
        }

        let idx = address as usize;
        self.ram_cells.get(idx).copied()
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_vga_write_and_scroll() {
        let mut vga = VgaBuffer::new();
        vga.write_str("OMNICODE BOOT", VgaColor::White, VgaColor::Black);
        assert_eq!(vga.cells[0].ascii, b'O');
        assert_eq!(vga.cells[1].ascii, b'M');
        assert_eq!(vga.cells[12].ascii, b'T');
        assert_eq!(vga.cursor_col, 13);
        assert_eq!(vga.cursor_row, 0);

        vga.new_line();
        assert_eq!(vga.cursor_col, 0);
        assert_eq!(vga.cursor_row, 1);
    }

    #[test]
    fn test_physical_memory_map_mmio_dispatch() {
        let mut mem = PhysicalMemoryMap::new(1024);

        // Write to VGA MMIO
        let word_char = Word64::from_signed_int(b'A' as i64);
        mem.write(VGA_BUFFER_BASE, word_char).expect("VGA write");
        assert_eq!(mem.vga.cells[0].ascii, b'A');

        // Write to Serial COM1
        let word_serial = Word64::from_signed_int(b'!' as i64);
        mem.write(COM1_SERIAL_PORT, word_serial).expect("Serial write");
        assert_eq!(mem.serial.read_output(), "!");

        // Write to RAM
        mem.write(42, Word64::from_signed_int(999)).expect("RAM write");
        assert_eq!(mem.read(42).unwrap().payload().to_i64(), 999);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mod.rs RS 54 LINES 205 WORDS 2.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Core Subsystem Identity, Module Architecture, and Invariant Re-exports )
// ============================================================================
//! OmniCode Core Subsystem
//! Encapsulates the algebraic foundation: Ternary Quad-State, Boundaries, and Operators.

pub mod ternary;
pub mod boundaries;
pub mod operators;
pub mod filetype;
pub mod linguistics;
pub mod cell;
pub mod vessel;
pub mod tape;
pub mod word;
pub mod scene;
pub mod reel;
pub mod mmio;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Direct Core Access )
// ============================================================================
pub use ternary::{QuadState, TransportAction, ROLLBACK, AWAIT, PROCEED, HALT};
pub use boundaries::BoundaryType;
pub use operators::{BeingRelation, DirectionalVector, Connective, Polarity, AffixPosition};
pub use filetype::OmniFileType;
pub use linguistics::{PartOfSpeech, GrammaticalClassification, GrammaticalEntity, PronounKind, PronounCase, PronounPerson, PronounNumber};
pub use cell::{Cell, Trit, BYTE_HALT, BYTE_AWAIT, BYTE_ROLLBACK, BYTE_PROCEED};
pub use vessel::{Vessel, Offset, Capacity, Count};
pub use tape::{Roll, Tape, TapeHead};
pub use word::{Word32Trit, Word64};
pub use scene::Scene;
pub use reel::MemoryReel;
pub use mmio::{PhysicalMemoryMap, VgaBuffer, VgaColor, VgaCell, SerialPort, VGA_BUFFER_BASE, COM1_SERIAL_PORT};




// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Subsystem Invariant Definitions )
// ============================================================================
// Invariants and operators are actively materialized via re-exported submodules.


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ============================================================================
// Core module namespace verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/operators.rs RS 344 LINES 1239 WORDS 13.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Module Identity, 3D Coordinate Operators, Connectives & Vector Maps )
// ================================================================================
//! OmniCode Core: Relational, Directional, and Connective Operators
//!
//! Governs the 3-Dimensional Coordinate System:
//!     - Y-Axis (Being/State)      : = (IS), != (IS NOT), ~= (IS LIKE)
//!     - X-Axis (Causality/Action) : --> (Act Upon), <-- (Act Again)
//!     - Z-Axis (Scale/Layering)   : ^-- (Builds To), v-- (Breaks Down From)
//!     - Connectives               : ! (But), :: (And), : (In Relation To), | (Or), -- (Not)

use std::fmt;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Type Declarations: BeingRelation, DirectionalVector, and Connective Enums )
// ================================================================================
/// Relations of Being along the Ontological Y-Axis.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum BeingRelation {
    /// Exact identity / equivalence (=)
    Is,
    /// Distinction / exclusion (!=)
    IsNot,
    /// Homomorphism / structural analogy / duck typing (~=)
    ToBeLike,
    /// Invariant Balance / Equivalence (<=>)
    Equivalence,
    /// Relational strictly less than (<)
    LessThan,
    /// Relational strictly greater than (>)
    GreaterThan,
    /// Relational less than or equal (<=)
    LessEqual,
    /// Relational greater than or equal (>=)
    GreaterEqual,
}

/// Vectors of Action, Scale, and Spatial Quadrants across X, Y, and Z axes.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum DirectionalVector {
    // X-Axis (Temporal / Dynamics)
    /// Forward causation, execution, emission (-->)
    ActUpon,
    /// Retroactive feedback, retry, rollback (<--)
    ActAgain,

    // Z-Axis (Scale / Hierarchy)
    /// Layering Up: synthesis, abstraction, emergence (^-- / --^)
    BuildsTo,
    /// Layering Down: decomposition, destructuring, compilation to IR (v-- / --v)
    BreaksDownFrom,

    // Diagonal Modulated Scales
    /// Major ascending forward diagonal (/-^)
    Major,
    /// Minor descending forward diagonal (\-v)
    Minor,
    /// Great ascending backward diagonal (\-^)
    Great,
    /// Small descending backward diagonal (/-v)
    Small,

    // 2D Quadrant / Compass Vectors
    /// Upper Right (Quadrant I: +X, +Y) (/^)
    UpperRight,
    /// Lower Right (Quadrant IV: +X, -Y) (\v)
    LowerRight,
    /// Upper Left (Quadrant II: -X, +Y) (^\)
    UpperLeft,
    /// Lower Left (Quadrant III: -X, -Y) (v/)
    LowerLeft,
```

```rust
}

/// Logical and Structural Connectives.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Connective {
    /// Adversative caveat: A ! B (A executes, BUT must satisfy caveat B)
    But,
    /// Cumulative conjunction: A :: B (A AND B simultaneous)
    And,
    /// Relational role / typing: A : B (A in relation to B)
    InRelationTo,
    /// Disjunction / alternative: A | B (A OR B)
    Or,
    /// Ontological inversion: -- A (NOT A)
    Not,
}

/// Polarity of an entity, keyword, or value.
/// '+' represents Abstract (archetype, intension, zero-cost blueprint).
/// '-' represents Concrete (extension, physical instance, allocated substance).
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Polarity {
    Abstract,
    Concrete,
}

/// Affix position of a polarity operator attached to an identifier or keyword.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum AffixPosition {
    Prefix,  // +Dog, -Dog, +Cell
    Postfix, // Dog+, Dog-, PROCEED-
}

impl Polarity {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "+" => Ok(Polarity::Abstract),
            "-" => Ok(Polarity::Concrete),
            other => Err(format!("FAILPOINT [InvalidPolarity]: Unknown polarity symbol '{other}'")),
        }
    }

    pub fn symbol(&self) -> &'static str {
        match self {
            Polarity::Abstract => "+",
            Polarity::Concrete => "-",
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Symbol Parsing & Semantic Transformations )
// ============================================================================
impl BeingRelation {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "=" => Ok(BeingRelation::Is),
            "!=" | "<>" => Ok(BeingRelation::IsNot),
            "~=" | "≈" => Ok(BeingRelation::ToBeLike),
            "<=>" => Ok(BeingRelation::Equivalence),
            "<" => Ok(BeingRelation::LessThan),
            ">" => Ok(BeingRelation::GreaterThan),
            "<=" => Ok(BeingRelation::LessEqual),
            ">=" => Ok(BeingRelation::GreaterEqual),
            other => Err(format!("FAILPOINT [InvalidBeingRelation]: Unknown symbol '{other}'")),
        }
    }

    pub fn symbol(&self) -> &'static str {
        match self {
            BeingRelation::Is => "=",
            BeingRelation::IsNot => "!=",
            BeingRelation::ToBeLike => "~=",
            BeingRelation::Equivalence => "<=>",
            BeingRelation::LessThan => "<",
            BeingRelation::GreaterThan => ">",
```

```
            BeingRelation::LessEqual => "<=",
            BeingRelation::GreaterEqual => ">=",
        }
    }
}

impl DirectionalVector {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "-->" | "->" => Ok(DirectionalVector::ActUpon),
            "<--" | "<-" => Ok(DirectionalVector::ActAgain),
            "--^" | "-^" | "^--" | "^-" | "↑" => Ok(DirectionalVector::BuildsTo),
            "--v" | "-v" | "v--" | "v-" | "↓" => Ok(DirectionalVector::BreaksDownFrom),
            "/-^" => Ok(DirectionalVector::Major),
            "\\-v" => Ok(DirectionalVector::Minor),
            "\\-^" => Ok(DirectionalVector::Great),
            "/-v" => Ok(DirectionalVector::Small),
            "/^" => Ok(DirectionalVector::UpperRight),
            "\\v" => Ok(DirectionalVector::LowerRight),
            "^\\" => Ok(DirectionalVector::UpperLeft),
            "v/" => Ok(DirectionalVector::LowerLeft),
            other => Err(format!("FAILPOINT [InvalidDirectionalVector]: Unknown vector symbol '{other}'")),
        }
    }

    /// Canonical ASCII symbol for the vector.
    pub fn symbol(&self) -> &'static str {
        match self {
            DirectionalVector::ActUpon => "-->",
            DirectionalVector::ActAgain => "<--",
            DirectionalVector::BuildsTo => "--^",
            DirectionalVector::BreaksDownFrom => "--v",
            DirectionalVector::Major => "/-^",
            DirectionalVector::Minor => "\\-v",
            DirectionalVector::Great => "\\-^",
            DirectionalVector::Small => "/-v",
            DirectionalVector::UpperRight => "/^",
            DirectionalVector::LowerRight => "\\v",
            DirectionalVector::UpperLeft => "^\\",
            DirectionalVector::LowerLeft => "v/",
        }
    }

    /// Fast 2-character ASCII shorthand symbol for the vector.
    pub fn shorthand(&self) -> &'static str {
        match self {
            DirectionalVector::ActUpon => "->",
            DirectionalVector::ActAgain => "<-",
            DirectionalVector::BuildsTo => "-^",
            DirectionalVector::BreaksDownFrom => "-v",
            DirectionalVector::Major => "/-^",
            DirectionalVector::Minor => "\\-v",
            DirectionalVector::Great => "\\-^",
            DirectionalVector::Small => "/-v",
            DirectionalVector::UpperRight => "/^",
            DirectionalVector::LowerRight => "\\v",
            DirectionalVector::UpperLeft => "^\\",
            DirectionalVector::LowerLeft => "v/",
        }
    }

    /// Cardinal direction on the computational coordinate plane.
    pub fn cardinal_direction(&self) -> &'static str {
        match self {
            DirectionalVector::ActUpon => "Right (Forward Causation / Advance)",
            DirectionalVector::ActAgain => "Left (Retroaction / Feedback / Rollback)",
            DirectionalVector::BuildsTo => "Up (Synthesis / Abstraction / Ascend)",
            DirectionalVector::BreaksDownFrom => "Down (Decomposition / Analysis / Descend)",
            DirectionalVector::Major => "Major (Ascending Forward Diagonal / Major Scale)",
            DirectionalVector::Minor => "Minor (Descending Forward Diagonal / Minor Scale)",
            DirectionalVector::Great => "Great (Ascending Backward Diagonal / Great Scale)",
            DirectionalVector::Small => "Small (Descending Backward Diagonal / Small Scale)",
            DirectionalVector::UpperRight => "Upper Right (Quadrant I: +X, +Y / Northeast)",
            DirectionalVector::LowerRight => "Lower Right (Quadrant IV: +X, -Y / Southeast)",
            DirectionalVector::UpperLeft => "Upper Left (Quadrant II: -X, +Y / Northwest)",
            DirectionalVector::LowerLeft => "Lower Left (Quadrant III: -X, -Y / Southwest)",
        }
    }
```

```rust
}

impl Connective {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "!" => Ok(Connective::But),
            "::" | "&&" => Ok(Connective::And),
            ":" => Ok(Connective::InRelationTo),
            "|" | "||" => Ok(Connective::Or),
            "--" | "not" | "~" => Ok(Connective::Not),
            other => Err(format!("FAILPOINT [InvalidConnective]: Unknown connective symbol '{other}'")),
        }
    }

    pub fn symbol(&self) -> &'static str {
        match self {
            Connective::But => "!",
            Connective::And => "::",
            Connective::InRelationTo => ":",
            Connective::Or => "|",
            Connective::Not => "--",
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementations and String Formatting )
// ============================================================================
impl fmt::Display for BeingRelation {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for DirectionalVector {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for Connective {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for Polarity {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for AffixPosition {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            AffixPosition::Prefix => write!(f, "Prefix"),
            AffixPosition::Postfix => write!(f, "Postfix"),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_polarity_and_affix_operators() {
        assert_eq!(Polarity::from_symbol("+").unwrap(), Polarity::Abstract);
        assert_eq!(Polarity::from_symbol("-").unwrap(), Polarity::Concrete);
        assert_eq!(Polarity::Abstract.symbol(), "+");
        assert_eq!(Polarity::Concrete.symbol(), "-");
        assert_eq!(format!("{}", Polarity::Abstract), "+");
        assert_eq!(format!("{}", Polarity::Concrete), "-");
    }

    #[test]
```

```
    fn test_directional_vectors_canonical_and_shorthand() {
        // Canonical 3-character symbols
        assert_eq!(DirectionalVector::from_symbol("-->").unwrap(), DirectionalVector::ActUpon);
        assert_eq!(DirectionalVector::from_symbol("<--").unwrap(), DirectionalVector::ActAgain);
        assert_eq!(DirectionalVector::from_symbol("--^").unwrap(), DirectionalVector::BuildsTo);
        assert_eq!(DirectionalVector::from_symbol("--v").unwrap(), DirectionalVector::BreaksDownFrom);

        // Fast 2-character shorthands
        assert_eq!(DirectionalVector::from_symbol("->").unwrap(), DirectionalVector::ActUpon);
        assert_eq!(DirectionalVector::from_symbol("<-").unwrap(), DirectionalVector::ActAgain);
        assert_eq!(DirectionalVector::from_symbol("-^").unwrap(), DirectionalVector::BuildsTo);
        assert_eq!(DirectionalVector::from_symbol("-v").unwrap(), DirectionalVector::BreaksDownFrom);

        // Legacy symbols
        assert_eq!(DirectionalVector::from_symbol("^--").unwrap(), DirectionalVector::BuildsTo);
        assert_eq!(DirectionalVector::from_symbol("v--").unwrap(), DirectionalVector::BreaksDownFrom);

        // Symbols emit canonical forms
        assert_eq!(DirectionalVector::BuildsTo.symbol(), "--^");
        assert_eq!(DirectionalVector::BreaksDownFrom.symbol(), "--v");

        // Scale Modulations
        assert_eq!(DirectionalVector::from_symbol("/-^").unwrap(), DirectionalVector::Major);
        assert_eq!(DirectionalVector::from_symbol("\\-v").unwrap(), DirectionalVector::Minor);
        assert_eq!(DirectionalVector::from_symbol("\\-^").unwrap(), DirectionalVector::Great);
        assert_eq!(DirectionalVector::from_symbol("/-v").unwrap(), DirectionalVector::Small);

        // 2D Quadrants
        assert_eq!(DirectionalVector::from_symbol("/^").unwrap(), DirectionalVector::UpperRight);
        assert_eq!(DirectionalVector::from_symbol("\\v").unwrap(), DirectionalVector::LowerRight);
        assert_eq!(DirectionalVector::from_symbol("^\\").unwrap(), DirectionalVector::UpperLeft);
        assert_eq!(DirectionalVector::from_symbol("v/").unwrap(), DirectionalVector::LowerLeft);
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/reel.rs

RS | 286 LINES | 1090 WORDS | 11.5 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( 4-Directional Toroidal Memory Reel: Transport Kinematics & Scene Spool )
// ================================================================================
//! OmniCode Core: 4-Directional Memory Reel
//!
//! Spools memory across two orthogonal mechanical axes:
//! 1. **Horizontal Ribbon Navigation:**
//!    - `push_forward(word)` (`-->`): Advances write head along the track.
//!    - `pull_back()` (`<--`): Recedes backward along the track to inspect/undo.
//! 2. **Vertical Spool Hierarchy Navigation:**
//!    - `sink_in(name, cap)` (`v--`): Cuts into a child scene (pushes stack frame).
//!    - `pop_out()` (`^--`): Cuts back to parent scene (pops stack frame).
//!
//! Governed by the Quad-State Transport Kinematics:
//! `PLAY` (+1), `PAUSE` (0), `REWIND` (-1), `STOP` (NULL).

use std::fmt;
use crate::core::scene::Scene;
use crate::core::ternary::TransportAction;
use crate::core::word::Word64;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( MemoryReel Struct Definition & Constructor )
// ================================================================================

/// A 4-directional spooled memory stack and multi-track film canister.
#[derive(Debug, Clone, PartialEq)]
pub struct MemoryReel {
    pub name: String,
    pub transport: TransportAction,
    pub scenes: Vec<Scene>,
    pub next_scene_id: usize,
}

impl MemoryReel {
    /// Creates a new MemoryReel initialized with a root master scene (depth 0).
    pub fn new(name: impl Into<String>, root_capacity: usize) -> Self {
        let root_scene = Scene::new(0, "MasterSet_0", root_capacity, 0, Some("genesis".to_string()));
        Self {
            name: name.into(),
            transport: TransportAction::Pause,
            scenes: vec![root_scene],
            next_scene_id: 1,
        }
    }

    /// Returns a reference to the currently active camera scene.
    pub fn active_scene(&self) -> &Scene {
        self.scenes.last().expect("Reel must always contain at least one scene")
    }

    /// Returns a mutable reference to the currently active camera scene.
    pub fn active_scene_mut(&mut self) -> &mut Scene {
        self.scenes.last_mut().expect("Reel must always contain at least one scene")
    }

    /// Returns the current spindle depth level (0 = Master set).
    pub fn depth(&self) -> usize {
        self.active_scene().depth
    }

    /// Returns the total number of scenes currently on the reel spindle.
    pub fn scene_count(&self) -> usize {
        self.scenes.len()
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( The 4 Mechanical Motions: Push Forward, Pull Back, Sink In, Pop Out )
// ================================================================================
```

```
impl MemoryReel {
    // ------------------------------------------------------------------------
    // Axis 1: Horizontal Track Motions (--> and <--)
    // ------------------------------------------------------------------------

    /// **Push Forward (`-->`):**
    /// Advances the track head, inscribes the physical word at the new position,
    /// and sets transport to `PLAY`.
    pub fn push_forward(&mut self, word: Word64) -> Result<usize, String> {
        let scene = self.active_scene_mut();
        let target_pos = scene.head_pos;

        if target_pos >= scene.capacity {
            return Err(format!(
                "FAILPOINT [ReelCapacityReached]: Track head at @0x{:04X} exceeded scene capacity {}",
                target_pos, scene.capacity
            ));
        }

        scene.write(target_pos, word)?;
        scene.head_pos += 1;
        self.transport = TransportAction::Play;
        Ok(target_pos)
    }

    /// **Pull Back (`<--`):**
    /// Recedes the track head backward along the ribbon to inspect antecedent props
    /// or perform transactional retreat without destroying future frame cells.
    pub fn pull_back(&mut self) -> Option<Word64> {
        let scene = self.active_scene_mut();
        if scene.head_pos > 0 {
            scene.head_pos -= 1;
            scene.read(scene.head_pos)
        } else {
            scene.read(0)
        }
    }

    // ------------------------------------------------------------------------
    // Axis 2: Vertical Spool Hierarchy Motions (v-- and ^--)
    // ------------------------------------------------------------------------

    /// **Sink In (`v--`):**
    /// Cuts into a subordinate scene (pushes a new stack frame onto the spindle).
    /// The parent scene remains frozen and untouched above it.
    pub fn sink_in(
        &mut self,
        scene_name: impl Into<String>,
        capacity: usize,
        landmark: Option<String>,
    ) -> usize {
        let parent_depth = self.depth();
        let scene_id = self.next_scene_id;
        self.next_scene_id += 1;

        let child_scene = Scene::new(scene_id, scene_name, capacity, parent_depth + 1, landmark);
        self.scenes.push(child_scene);
        self.depth()
    }

    /// **Pop Out (`^--`):**
    /// Completes the subordinate scene, pops the stack frame off the spindle,
    /// and restores the camera framing to the supervisory parent set.
    pub fn pop_out(&mut self) -> Result<Scene, String> {
        if self.scenes.len() <= 1 {
            return Err("FAILPOINT [RootScenePop]: Cannot pop out of the master root scene (Depth 0)".to_string());
        }
        let completed = self.scenes.pop().expect("Scene pop must succeed");
        Ok(completed)
    }

    // ------------------------------------------------------------------------
    // Transport Kinematics Controls: Play, Pause, Rewind, Stop
    // ------------------------------------------------------------------------

    /// Sets transport to `PLAY` (+1 / PROCEED).
    pub fn play(&mut self) -> TransportAction {
```

```
        self.transport = TransportAction::Play;
        self.transport
    }

    /// Sets transport to `PAUSE` (0 / AWAIT).
    pub fn pause(&mut self) -> TransportAction {
        self.transport = TransportAction::Pause;
        self.transport
    }

    /// **Rewinds (`<--` / -1 / ROLLBACK):**
    /// Spools the track head backward to a designated scene landmark cue.
    pub fn rewind_to_landmark(&mut self, target_cue: &str) -> Result<usize, String> {
        let scene = self.active_scene_mut();
        if let Some(ref landmark) = scene.landmark {
            if landmark == target_cue {
                scene.head_pos = 0;
                self.transport = TransportAction::Rewind;
                return Ok(0);
            }
        }
        Err(format!(
            "FAILPOINT [LandmarkNotFound]: Active scene '{}' does not match landmark cue '<{target_cue}>'",
            scene.name
        ))
    }

    /// Sets transport to `STOP` (NULL / HALT).
    pub fn stop(&mut self) -> TransportAction {
        self.transport = TransportAction::Stop;
        self.transport
    }

    /// Generates a cinematic ASCII status dashboard of the memory reel.
    pub fn inspect(&self) -> String {
        let mut out = String::new();
        out.push_str(&format!("═══════════════════════════════════════════════════════════════\n"));
        out.push_str(&format!("REEL: [{}] | TRANSPORT: [{:?}]\n", self.name, self.transport));
        out.push_str(&format!("SPINDLE DEPTH: {} | TOTAL SCENES: {}\n", self.depth(), self.scene_count()));
        out.push_str(&format!("---------------------------------------------------------------\n"));
        for (i, sc) in self.scenes.iter().enumerate() {
            let active_marker = if i == self.scenes.len() - 1 { "▶ [ACTIVE]" } else { "  [FROZEN]" };
            out.push_str(&format!("  {} Level {}: {}\n", active_marker, sc.depth, sc));
        }
        out.push_str(&format!("═══════════════════════════════════════════════════════════════\n"));
        out
    }
}

impl fmt::Display for MemoryReel {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            f,
            "Reel [{} | Transport: {:?} | Depth: {} | Active: {}]",
            self.name, self.transport, self.depth(), self.active_scene().name
        )
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Proof )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::word::Word32Trit;
    use crate::core::ternary::QuadState;
    use crate::core::operators::Polarity;
    use crate::core::boundaries::BoundaryType;

    #[test]
    fn test_reel_4_directional_motions() {
        let mut reel = MemoryReel::new("CinemaReel", 32);
        assert_eq!(reel.depth(), 0);
        assert_eq!(reel.scene_count(), 1);

```

```
        // 1. Push Forward (-->) on master set
        let w1 = Word64::new(Word32Trit::from_i64(10), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation,
0);
        let pos1 = reel.push_forward(w1).expect("Push forward 1");
        assert_eq!(pos1, 0);
        assert_eq!(reel.transport, TransportAction::Play);

        let w2 = Word64::new(Word32Trit::from_i64(20), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation,
0);
        let pos2 = reel.push_forward(w2).expect("Push forward 2");
        assert_eq!(pos2, 1);

        // 2. Pull Back (<--) along master track
        let back_word = reel.pull_back().expect("Pull back");
        assert_eq!(back_word.payload().to_i64(), 20);

        // 3. Sink In (v--) into nested sub-scene
        let depth = reel.sink_in("CloseUp_Scene", 16, Some("closeup_take".to_string()));
        assert_eq!(depth, 1);
        assert_eq!(reel.depth(), 1);
        assert_eq!(reel.scene_count(), 2);
        assert_eq!(reel.active_scene().name, "CloseUp_Scene");

        // Write in nested sub-scene
        let w_sub = Word64::new(Word32Trit::from_i64(999), QuadState::Proceed, Polarity::Concrete,
BoundaryType::Denotation, 0);
        reel.push_forward(w_sub).expect("Push in sub-scene");
        assert_eq!(reel.active_scene().read(0).unwrap().payload().to_i64(), 999);

        // 4. Pop Out (^--) back to master wide shot
        let popped = reel.pop_out().expect("Pop out to master");
        assert_eq!(popped.name, "CloseUp_Scene");
        assert_eq!(reel.depth(), 0);
        assert_eq!(reel.active_scene().name, "MasterSet_0");
        assert_eq!(reel.scene_count(), 1);

        // Verify master set props were unaffected by sub-scene
        assert_eq!(reel.active_scene().read(0).unwrap().payload().to_i64(), 10);
        assert_eq!(reel.active_scene().read(1).unwrap().payload().to_i64(), 20);
    }

    #[test]
    fn test_reel_rewind_to_landmark() {
        let mut reel = MemoryReel::new("TakeReel", 32);
        reel.sink_in("TakeScene", 16, Some("scene_cue".to_string()));

        let w = Word64::new(Word32Trit::from_i64(42), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation,
0);
        reel.push_forward(w).unwrap();
        reel.push_forward(w).unwrap();
        assert_eq!(reel.active_scene().head_pos, 2);

        // Rewind to landmark
        let reset_pos = reel.rewind_to_landmark("scene_cue").expect("Rewind to cue");
        assert_eq!(reset_pos, 0);
        assert_eq!(reel.active_scene().head_pos, 0);
        assert_eq!(reel.transport, TransportAction::Rewind);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/scene.rs RS 130 LINES 449 WORDS 4.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Scene Memory Frame: Set, Props & Landmark Clapper )
// ============================================================================
//! OmniCode Core: Scene Memory Frame
//!
//! Represents a unified memory stack frame / cinematic set where instructions
//! (rolls) operate upon local props (memory cells):
//! - **Set:** Bounded memory partition slice.
//! - **Props:** Physical 64-bit / 32-trit memory cells.
//! - **Clapper Landmark:** Scene cue point for retakes and rewind operations.

use std::fmt;
use crate::core::word::Word64;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scene Struct Definition & Constructor )
// ============================================================================

/// A memory stack frame and camera set in the OmniCode cinema architecture.
#[derive(Debug, Clone, PartialEq)]
pub struct Scene {
    pub id: usize,
    pub name: String,
    pub depth: usize,
    pub landmark: Option<String>,
    pub capacity: usize,
    pub head_pos: usize,
    pub cells: Vec<Word64>,
}

impl Scene {
    /// Creates a new Scene set with bounded cell capacity.
    pub fn new(
        id: usize,
        name: impl Into<String>,
        capacity: usize,
        depth: usize,
        landmark: Option<String>,
    ) -> Self {
        let cap = capacity.max(8);
        Self {
            id,
            name: name.into(),
            depth,
            landmark,
            capacity: cap,
            head_pos: 0,
            cells: vec![Word64::default(); cap],
        }
    }

    /// Returns the number of allocated cells in this scene.
    pub fn len(&self) -> usize {
        self.cells.len()
    }

    /// Returns true if this scene has zero capacity.
    pub fn is_empty(&self) -> bool {
        self.cells.is_empty()
    }

    /// Reads a memory cell prop at an explicit offset.
    pub fn read(&self, offset: usize) -> Option<Word64> {
        self.cells.get(offset).copied()
    }

    /// Inscribes a memory cell prop at an explicit offset with bounds enforcement.
    pub fn write(&mut self, offset: usize, word: Word64) -> Result<(), String> {
        if offset >= self.capacity {
            return Err(format!(
                "FAILPOINT [SceneBoundsOverflow]: Scene '{}' capacity is {}, but attempted write at offset {}",
                self.name, self.capacity, offset
```

```
            ));
        }
        self.cells[offset] = word;
        Ok(())
    }

    /// Reads the prop directly under the active scene head.
    pub fn current(&self) -> Option<Word64> {
        self.read(self.head_pos)
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Scene Display Formatting )
// ============================================================================

impl fmt::Display for Scene {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let landmark_str = self.landmark.as_deref().unwrap_or("none");
        write!(
            f,
            "Scene [{} | ID: {} | Depth: {} | Landmark: <{}> | Head: @0x{:04X} | Cells: {}/{}]",
            self.name, self.id, self.depth, landmark_str, self.head_pos, self.cells.len(), self.capacity
        )
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::word::Word32Trit;
    use crate::core::ternary::QuadState;
    use crate::core::operators::Polarity;
    use crate::core::boundaries::BoundaryType;

    #[test]
    fn test_scene_creation_and_bounded_write() {
        let mut scene = Scene::new(0, "TestSet", 16, 0, Some("take_1".to_string()));
        assert_eq!(scene.name, "TestSet");
        assert_eq!(scene.depth, 0);
        assert_eq!(scene.capacity, 16);

        let word = Word64::new(Word32Trit::from_i64(77), QuadState::Proceed, Polarity::Concrete,
BoundaryType::Denotation, 1);
        scene.write(3, word).expect("Write to offset 3");
        assert_eq!(scene.read(3).unwrap().payload().to_i64(), 77);

        // Bounds overflow rejection
        assert!(scene.write(16, word).is_err());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/tape.rs RS 489 LINES 1805 WORDS 18.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Tape Subsystem Identity, Ribbon Physical Topology & Invariants )
// ================================================================================
//! OmniCode Core Subsystem: Physical Tape & Roll Execution Topology
//!
//! # Physical Computing Hierarchy
//! In the OmniCode Operating System, execution does not use raw unstructured threads.
//! It follows a strict 3-tier physical hierarchy:
//!
//! $$\text{Roll} \longrightarrow \text{Tape} \longrightarrow \text{Track}$$
//!
//! 1. **Roll (Instruction Line):** An atomic line of code executing an SVO transformation.
//! 2. **Tape (Process Ribbon):** A sequential chain of rolls led by a **Tape Head**
//!    (the covenant manifest declaring Who, What, When, Where, Why, and How).
//! 3. **Track (Subsystem Channel):** A concurrent execution lane on the scheduler
//!    (Track 0: Kernel, Track 1: Terminal, Track 2: Apps).
//!
//! # Tape Head Anatomy
//! The Tape Head is the first roll on the ribbon. It establishes:
//! - Identity & Name (`[Tape.<Name>]`)
//! - Principal & Caller (`WHO {Caller}`)
//! - Target Entity (`WHAT {Target}`)
//! - Lifecycle Trigger (`WHEN {Phase}`)
//! - Memory Partition Ceiling (`WHERE {partition} = [ <N> ]`)
//! - Teleological Purpose Aside (`WHY ( ... )`)
//! - Operational Vector (`HOW [ ... ]`)

use std::fs;
use std::path::Path;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::DiagnosticLogger;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Tape Data Structures, Roll Containers and Context )
// ================================================================================

/// An atomic line of code on a Tape ribbon.
#[derive(Debug, Clone, PartialEq)]
pub struct Roll {
    /// 0-indexed position within the tape's rolls.
    pub index: usize,
    /// 1-indexed source line number.
    pub line_number: usize,
    /// Raw text content of the roll.
    pub text: String,
    /// Current execution state of this roll (+1: PROCEED, 0: AWAIT, -1: ROLLBACK, NULL: HALT).
    pub status: QuadState,
    /// Whether this roll is the leading Tape Head covenant.
    pub is_head: bool,
}

/// The leading covenant roll establishing the process manifest and memory bounds.
#[derive(Debug, Clone, PartialEq)]
pub struct TapeHead {
    /// Identifier or process name.
    pub name: String,
    /// Cardinal Clause: WHO (Caller / Principal).
    pub who: String,
    /// Cardinal Clause: WHAT (Process substance).
    pub what: String,
    /// Cardinal Clause: WHEN (Trigger precondition).
    pub when_clause: String,
    /// Cardinal Clause: WHERE (Target partition or spatial bounds).
    pub where_clause: String,
    /// Cardinal Clause: WHY (Teleological intent aside).
    pub why: String,
    /// Cardinal Clause: HOW (Directional vector and methodology).
    pub how: String,
    /// Required memory partition size in cells (carved from Storehouse via `v--`).
    pub required_partition_size: usize,
    /// Complete raw text of the Tape Head roll.
    pub raw_text: String,
```

```
}

impl Default for TapeHead {
    fn default() -> Self {
        Self {
            name: "Anonymous_Tape".to_string(),
            who: "System".to_string(),
            what: "Process".to_string(),
            when_clause: "Immediate".to_string(),
            where_clause: "Memory".to_string(),
            why: "Execution".to_string(),
            how: "Forward".to_string(),
            required_partition_size: 256,
            raw_text: String::new(),
        }
    }
}

/// A complete sequential process ribbon executed roll-by-roll.
#[derive(Debug, Clone)]
pub struct Tape {
    /// The leading covenant manifest.
    pub head: TapeHead,
    /// The sequence of subordinate execution rolls.
    pub rolls: Vec<Roll>,
    /// Active cursor pointing to the roll currently being executed.
    pub cursor: usize,
    /// Name of the isolated memory partition allocated for this tape.
    pub bound_partition: Option<String>,
    /// Overall execution quad-state.
    pub execution_state: QuadState,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Tape Parsing, Stepping, and Execution Management )
// ============================================================================

impl Tape {
    /// Parses a native `.tape` source string into a structured [`Tape`].
    pub fn from_source(source: &str, logger: DiagnosticLogger) -> Result<Self, String> {
        let mut head_opt: Option<TapeHead> = None;
        let mut raw_roll_lines: Vec<(usize, String)> = Vec::new();

        let mut head_text_accumulator = String::new();
        let mut parsing_multiline_head = false;

        for (zero_idx, raw_line) in source.lines().enumerate() {
            let line_number = zero_idx + 1;
            let trimmed = raw_line.trim();

            // Skip comment boundaries (# ...)
            if trimmed.starts_with('#') || trimmed.is_empty() {
                continue;
            }

            // Detect Tape Head
            if head_opt.is_none() {
                if !parsing_multiline_head {
                    if trimmed.starts_with('[')
                        || trimmed.starts_with("= [Tape")
                        || trimmed.starts_with("Tape [")
                        || trimmed.starts_with("Tape <")
                    {
                        parsing_multiline_head = true;
                        head_text_accumulator.push_str(trimmed);
                        head_text_accumulator.push(' ');
                        // Check if closed on same line (only if not opening a block with '{')
                        if !trimmed.ends_with('{') && (trimmed.contains(":: <") || (!trimmed.contains("WHERE") &&
!trimmed.contains("WHO"))) {
                            parsing_multiline_head = false;
                            let head = Self::parse_tape_head(&head_text_accumulator, line_number)?;
                            logger.log("TAPE", format!("Discovered Tape Head: [{}]", head.name));
                            head_opt = Some(head);
                            head_text_accumulator.clear();
                        }
                    } else {
                        // Plain line without leading bracket treated as fallback roll
```

```
                        raw_roll_lines.push((line_number, trimmed.to_string()));
                    }
                } else {
                    // Accumulating multiline Tape Head
                    if trimmed == "}" {
                        // Closing brace of block-manifest Tape Head
                        parsing_multiline_head = false;
                        let head = Self::parse_tape_head(&head_text_accumulator, line_number)?;
                        logger.log("TAPE", format!("Discovered Tape Head: [{}]", head.name));
                        head_opt = Some(head);
                        head_text_accumulator.clear();
                        continue;
                    } else if trimmed.starts_with("WHERE")
                        || trimmed.starts_with("WHEN")
                        || trimmed.starts_with("WHY")
                        || trimmed.starts_with("HOW")
                        || trimmed.starts_with("WHO")
                        || trimmed.starts_with("WHAT")
                        || trimmed.starts_with("< Scripture")
                        || trimmed.starts_with("::")
                        || trimmed.contains(":: <")
                    {
                        head_text_accumulator.push_str(trimmed);
                        head_text_accumulator.push(' ');
                    } else {
                        // Current line is not part of the head; the multiline head is complete
                        parsing_multiline_head = false;
                        let head = Self::parse_tape_head(&head_text_accumulator, line_number.saturating_sub(1))?;
                        logger.log("TAPE", format!("Discovered Tape Head: [{}]", head.name));
                        head_opt = Some(head);
                        head_text_accumulator.clear();

                        raw_roll_lines.push((line_number, trimmed.to_string()));
                    }
                }
                continue;
            }

            raw_roll_lines.push((line_number, trimmed.to_string()));
        }

        // If multiline head was accumulating at EOF
        if head_opt.is_none() && !head_text_accumulator.is_empty() {
            let head = Self::parse_tape_head(&head_text_accumulator, 1)?;
            head_opt = Some(head);
        }

        let head = head_opt.unwrap_or_else(|| TapeHead {
            name: "Default_Tape".to_string(),
            raw_text: "[Default_Tape]".to_string(),
            ..Default::default()
        });

        // Group raw_roll_lines into cohesive, multiline-aware Rolls
        let mut rolls = Vec::new();
        let mut cur_roll_text = String::new();
        let mut cur_roll_line = 1;
        let mut bracket_balance: i32 = 0;

        let is_new_roll_start = |s: &str| -> bool {
            let upper = s.to_uppercase();
            upper.starts_with("ROLL_")
                || upper.starts_with("[ ROLL")
                || upper.starts_with("[ROLL")
        };

        for (line_num, line_str) in raw_roll_lines {
            let starts_new = is_new_roll_start(&line_str);

            if starts_new && !cur_roll_text.is_empty() && bracket_balance <= 0 {
                rolls.push(Roll {
                    index: rolls.len(),
                    line_number: cur_roll_line,
                    text: cur_roll_text.trim().to_string(),
                    status: QuadState::Proceed,
                    is_head: false,
                });
                cur_roll_text.clear();
```

```
                bracket_balance = 0;
            }

            if cur_roll_text.is_empty() {
                cur_roll_line = line_num;
            }

            for ch in line_str.chars() {
                match ch {
                    '[' | '{' => bracket_balance += 1,
                    ']' | '}' => bracket_balance -= 1,
                    _ => {}
                }
            }

            cur_roll_text.push_str(&line_str);
            cur_roll_text.push(' ');

            // If not an explicit roll_... format and bracket_balance == 0, flush standalone roll
            if !starts_new && !cur_roll_text.starts_with("roll_") && !cur_roll_text.to_uppercase().starts_with("[ ROLL")
&& bracket_balance <= 0 {
                rolls.push(Roll {
                    index: rolls.len(),
                    line_number: cur_roll_line,
                    text: cur_roll_text.trim().to_string(),
                    status: QuadState::Proceed,
                    is_head: false,
                });
                cur_roll_text.clear();
                bracket_balance = 0;
            }
        }

        if !cur_roll_text.trim().is_empty() {
            rolls.push(Roll {
                index: rolls.len(),
                line_number: cur_roll_line,
                text: cur_roll_text.trim().to_string(),
                status: QuadState::Proceed,
                is_head: false,
            });
        }

        Ok(Self {
            head,
            rolls,
            cursor: 0,
            bound_partition: None,
            execution_state: QuadState::Proceed,
        })
    }

    /// Loads and parses a `.tape` file from the filesystem.
    pub fn from_file(path: impl AsRef<Path>, logger: DiagnosticLogger) -> Result<Self, String> {
        let p = path.as_ref();
        let resolved = crate::core::filetype::resolve_track_path(p);
        let content = fs::read_to_string(&resolved)
            .map_err(|e| format!("Failed to read tape file {:?}: {e}", resolved))?;
        Self::from_source(&content, logger)
    }

    /// Parses the raw text of a Tape Head roll into structured cardinal clauses.
    fn parse_tape_head(text: &str, line_number: usize) -> Result<TapeHead, String> {
        let clean = text.trim();

        // 1. Extract Name (e.g., from `[ Process { State } ]`, `[Tape.Name]` or `[Cell]`)
        let name = if let Some(proc_idx) = clean.find("Process {") {
            if let Some(end_brace) = clean[proc_idx..].find('}') {
                clean[proc_idx + 9..proc_idx + end_brace].trim().to_string()
            } else {
                "Process".to_string()
            }
        } else if let Some(start) = clean.find('[') {
            if let Some(end) = clean[start..].find(']') {
                let inner = &clean[start + 1..start + end];
                inner.trim_start_matches("Tape.").trim().to_string()
            } else {
                format!("Tape_L{}", line_number)
```

```
            }
        } else {
            format!("Tape_L{}", line_number)
        };

        // 2. Extract Cardinal Clauses
        let who = extract_clause(clean, "WHO").unwrap_or_else(|| "System".to_string());
        let what = extract_clause(clean, "WHAT").unwrap_or_else(|| name.clone());
        let when_clause = extract_clause(clean, "WHEN").unwrap_or_else(|| "Opening".to_string());
        let where_clause = extract_clause(clean, "WHERE").unwrap_or_else(|| "Memory".to_string());
        let why = extract_parenthetical(clean, "WHY").unwrap_or_else(|| "Operational task".to_string());
        let how = extract_clause(clean, "HOW").unwrap_or_else(|| "+1 : PROCEED".to_string());

        // 3. Extract required partition size from WHERE clause
        let mut required_partition_size = 256;
        if clean.contains("WHERE") {
            for token in clean.split_whitespace() {
                if let Ok(num) = token.trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
                    if num > 0 {
                        required_partition_size = num;
                        break;
                    }
                }
            }
        }

        Ok(TapeHead {
            name,
            who,
            what,
            when_clause,
            where_clause,
            why,
            how,
            required_partition_size,
            raw_text: clean.to_string(),
        })
    }

    /// Returns the roll currently under the Tape Head cursor.
    pub fn active_roll(&self) -> Option<&Roll> {
        self.rolls.get(self.cursor)
    }

    /// Advances the Tape Head cursor to the next roll, returning the newly focused roll.
    pub fn advance(&mut self) -> Option<&Roll> {
        if self.cursor < self.rolls.len() {
            self.cursor += 1;
        }
        self.active_roll()
    }

    /// Returns true if all subordinate rolls have been traversed.
    pub fn is_finished(&self) -> bool {
        self.cursor >= self.rolls.len()
    }

    /// Resets the Tape Head back to the first subordinate roll.
    pub fn reset(&mut self) {
        self.cursor = 0;
        for r in &mut self.rolls {
            r.status = QuadState::Proceed;
        }
        self.execution_state = QuadState::Proceed;
    }

    /// Rewinds the Tape Head cursor back to an explicit roll index.
    pub fn rewind_to(&mut self, target_index: usize) {
        if target_index < self.rolls.len() {
            self.cursor = target_index;
        }
    }

    /// Searches for a roll containing a scene landmark cue `< landmark >` or name.
    pub fn find_landmark_index(&self, landmark: &str) -> Option<usize> {
        let trimmed = landmark.trim();
        for (idx, roll) in self.rolls.iter().enumerate() {
            if roll.text.contains(&format!("< {trimmed} >"))
```

```
                || roll.text.contains(&format!("<{trimmed}>"))
                || roll.text.contains(&format!("::{trimmed}"))
                || roll.text.contains(trimmed)
            {
                return Some(idx);
            }
        }
        None
    }

    /// Total number of subordinate rolls on the tape.
    pub fn roll_count(&self) -> usize {
        self.rolls.len()
    }
}


/// Helper function to extract clause content bounded by braces `{...}` or brackets `[...]`.
fn extract_clause(text: &str, keyword: &str) -> Option<String> {
    if let Some(pos) = text.find(keyword) {
        let rest = &text[pos + keyword.len()..];
        // Look for { ... }
        if let Some(open) = rest.find('{') {
            if let Some(close) = rest[open..].find('}') {
                return Some(rest[open + 1..open + close].trim().to_string());
            }
        }
        // Look for [ ... ]
        if let Some(open) = rest.find('[') {
            if let Some(close) = rest[open..].find(']') {
                return Some(rest[open + 1..open + close].trim().to_string());
            }
        }
    }
    None
}

/// Helper function to extract aside content bounded by parentheses `(...)`.
fn extract_parenthetical(text: &str, keyword: &str) -> Option<String> {
    if let Some(pos) = text.find(keyword) {
        let rest = &text[pos + keyword.len()..];
        if let Some(open) = rest.find('(') {
            if let Some(close) = rest[open..].find(')') {
                return Some(rest[open + 1..open + close].trim().to_string());
            }
        }
    }
    None
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests, Verification & Subsystem Sealing )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_tape_parsing_and_head_extraction() {
        let source = r#"
            # Definition Roll / Tape Head
            [Tape.TaskWorker] : WHO {User} WHAT {Worker} --> DISPATCH {Worker}
                WHEN {Opening}
                WHERE {partition} = [ 512 ]
                WHY ( Secure background task execution )
                HOW [ +1 : PROCEED ]
                :: < covenant > ( verified )

            # Subordinate Rolls
            roll_1 [ 100 --> Buf @ 0 ] ( Initial write )
            roll_2 [ Buf[0] = 100 ! [ +1 ] ] ( Integrity verify )
            roll_3 [ commit_status [ PROCEED ] ] ( Tape sealed )
        "#;

        let logger = DiagnosticLogger::new(false);
        let mut tape = Tape::from_source(source, logger).expect("Parsing tape");
```

```
463
464         assert_eq!(tape.head.name, "TaskWorker");
465         assert_eq!(tape.head.who, "User");
466         assert_eq!(tape.head.what, "Worker");
467         assert_eq!(tape.head.when_clause, "Opening");
468         assert_eq!(tape.head.why, "Secure background task execution");
469         assert_eq!(tape.head.required_partition_size, 512);
470
471         assert_eq!(tape.roll_count(), 3);
472         assert_eq!(tape.cursor, 0);
473
474         let r1 = tape.active_roll().unwrap();
475         assert!(r1.text.contains("100 --> Buf @ 0"));
476
477         tape.advance();
478         assert_eq!(tape.cursor, 1);
479         let r2 = tape.active_roll().unwrap();
480         assert!(r2.text.contains("Buf[0] = 100"));
481
482         tape.advance();
483         assert_eq!(tape.cursor, 2);
484
485         tape.advance();
486         assert!(tape.is_finished());
487     }
488 }
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/ternary.rs RS 288 LINES 1163 WORDS 10.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Module Identity, Balanced Ternary Invariants, and Mathematical Basis )
// ============================================================================
//! OmniCode Core: Balanced Ternary Quad-State Engine
//!
//! Grounded in Balanced Radix-3 logic with a terminal Null/Void state:
//!     T4 = { -1 (Rollback), 0 (Await), +1 (Proceed), NULL (Halt) }
//!
//! Algebraic properties:
//!     - Symmetry: NOT(x) = -x
//!     - Conjunction (:: AND): min(A, B) with NULL absorption
//!     - Disjunction (| OR): max(A, B) with NULL absorption
//!     - Adversative Caveat (! BUT): Action A constrained by Caveat B

use std::fmt;

// Invariant QuadState Constants
pub const ROLLBACK: QuadState = QuadState::Rollback;
pub const AWAIT: QuadState = QuadState::Await;
pub const PROCEED: QuadState = QuadState::Proceed;
pub const HALT: QuadState = QuadState::Halt;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Context Declarations, State Enum, and Constructor Initialization )
// ============================================================================
/// The four fundamental ontological states of computation in OmniCode.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum QuadState {
    /// -1: NO / Rollback. Triggers retroactive vector (`<--`), unwinds mutated state.
    Rollback,
    /// 0: OK / Await. Equilibrium, standby, asynchronous yield, pass-through.
    Await,
    /// +1: YES / Proceed. Advances forward vector (`-->`), commits state.
    Proceed,
    /// NULL: VOID / Halt. Bottom type (⊥), boundary trap, terminal exit.
    Halt,
}

/// The cinematic transport kinematics of a reel along a track.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum TransportAction {
    /// Advance the track forward (matches +1, YES, PROCEED)
    Play,
    /// Hold the spindle in equilibrium (matches 0, OK, AWAIT)
    Pause,
    /// Rewind the track back to scene landmark (matches -1, NO, ROLLBACK)
    Rewind,
    /// Emergency brake detent / halt (matches NULL, VOID, HALT)
    Stop,
}

impl TransportAction {
    pub fn to_quad_state(&self) -> QuadState {
        match self {
            TransportAction::Play => QuadState::Proceed,
            TransportAction::Pause => QuadState::Await,
            TransportAction::Rewind => QuadState::Rollback,
            TransportAction::Stop => QuadState::Halt,
        }
    }

    pub fn as_str(&self) -> &'static str {
        match self {
            TransportAction::Play => "PLAY",
            TransportAction::Pause => "PAUSE",
            TransportAction::Rewind => "REWIND",
            TransportAction::Stop => "STOP",
        }
    }
}

```

```rust
impl QuadState {
    /// Parses a raw integer or string representation into a QuadState.
    ///
    /// # Failpoints
    /// Returns `Err` if the value does not belong to {-1, 0, 1, NULL}.
    pub fn from_raw(raw: &str) -> Result<Self, String> {
        match raw.trim().to_uppercase().as_str() {
            "-1" | "NO" | "ROLLBACK" | "REWIND" => Ok(QuadState::Rollback),
            "0" | "OK" | "AWAIT" | "PAUSE" => Ok(QuadState::Await),
            "1" | "+1" | "YES" | "PROCEED" | "PLAY" => Ok(QuadState::Proceed),
            "NULL" | "VOID" | "HALT" | "STOP" => Ok(QuadState::Halt),
            other => Err(format!(
                "FAILPOINT [InvalidQuadState]: '{other}' is not a valid balanced ternary state. \
                 Permitted states: -1, 0, +1, NULL (or PLAY, PAUSE, REWIND, STOP)"
            )),
        }
    }

    /// Transport kinematics level (cinematic reel/track motor):
    /// - Proceed: Play
    /// - Await: Pause
    /// - Rollback: Rewind
    /// - Halt: Stop
    pub fn transport(&self) -> TransportAction {
        match self {
            QuadState::Proceed => TransportAction::Play,
            QuadState::Await => TransportAction::Pause,
            QuadState::Rollback => TransportAction::Rewind,
            QuadState::Halt => TransportAction::Stop,
        }
    }

    /// Macro transport operation string: "PLAY", "PAUSE", "REWIND", "STOP"
    pub fn transport_str(&self) -> &'static str {
        self.transport().as_str()
    }


    /// Returns the integer value, or None for Halt (NULL).
    pub fn as_int(&self) -> Option<i8> {
        match self {
            QuadState::Rollback => Some(-1),
            QuadState::Await => Some(0),
            QuadState::Proceed => Some(1),
            QuadState::Halt => None,
        }
    }

    /// Micro level (linguistic / logical / epistemic stance):
    /// - Proceed: "yes" (1)
    /// - Rollback: "no" (-1)
    /// - Await: "ok" (0)
    /// - Halt: "void" (NULL)
    pub fn micro(&self) -> &'static str {
        match self {
            QuadState::Proceed => "yes",
            QuadState::Rollback => "no",
            QuadState::Await => "ok",
            QuadState::Halt => "void",
        }
    }

    /// Macro level (systems / OS state-machine physics):
    /// - Proceed: "PROCEED" (commit state / advance PC)
    /// - Rollback: "ROLLBACK" (unwind snapshot / retry)
    /// - Await: "AWAIT" (async yield / wait queue)
    /// - Halt: "HALT" (boundary trap / terminal exit)
    pub fn macro_op(&self) -> &'static str {
        match self {
            QuadState::Proceed => "PROCEED",
            QuadState::Rollback => "ROLLBACK",
            QuadState::Await => "AWAIT",
            QuadState::Halt => "HALT",
        }
    }

    /// Bridges the Micro and Macro ontological perspective.
    pub fn description(&self) -> &'static str {
```

```
        match self {
            QuadState::Proceed => "+1 / yes (Micro: Affirmation) | PROCEED (Macro: Forward commit)",
            QuadState::Rollback => "-1 / no (Micro: Refutation) | ROLLBACK (Macro: Retroactive rewind)",
            QuadState::Await => "0 / ok (Micro: Neutral balance) | AWAIT (Macro: Async yield / Standby)",
            QuadState::Halt => "NULL / void (Micro: Absence) | HALT (Macro: Terminal trap / Exit)",
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Algebraic Transformations: Inversion, Conjunction, Disjunction, Caveat )
// ============================================================================
impl QuadState {
    /// Algebraic Inversion (`--` NOT).
    ///
    /// Rules:
    /// - `--(+1)` == `-1` (NOT Proceed is Rollback)
    /// - `--(-1)` == `+1` (NOT Rollback is Proceed)
    /// - `--(0)`  ==  `0` (NOT Await is Await - zero is symmetric balance)
    /// - `--(NULL)` == `NULL` (NOT Halt is Halt - void remains void)
    pub fn invert(&self) -> Self {
        match self {
            QuadState::Proceed => QuadState::Rollback,
            QuadState::Rollback => QuadState::Proceed,
            QuadState::Await => QuadState::Await,
            QuadState::Halt => QuadState::Halt,
        }
    }

    /// Cumulative Conjunction (`::` AND).
    ///
    /// Evaluates to minimum with NULL acting as an absorbing trap.
    pub fn and(&self, other: Self) -> Self {
        if *self == QuadState::Halt || other == QuadState::Halt {
            return QuadState::Halt;
        }
        let a = self.as_int().unwrap();
        let b = other.as_int().unwrap();
        match a.min(b) {
            -1 => QuadState::Rollback,
            0 => QuadState::Await,
            1 => QuadState::Proceed,
            _ => unreachable!(),
        }
    }

    /// Disjunction (`|` OR).
    ///
    /// Evaluates to maximum with NULL acting as an absorbing trap.
    pub fn or(&self, other: Self) -> Self {
        if *self == QuadState::Halt || other == QuadState::Halt {
            return QuadState::Halt;
        }
        let a = self.as_int().unwrap();
        let b = other.as_int().unwrap();
        match a.max(b) {
            -1 => QuadState::Rollback,
            0 => QuadState::Await,
            1 => QuadState::Proceed,
            _ => unreachable!(),
        }
    }

    /// Adversative Caveat (`!` BUT).
    ///
    /// `action.but(caveat)`: Action proceeds ONLY if caveat does not veto.
    /// - Caveat = Rollback (-1) -> forces Rollback (-1)
    /// - Caveat = Halt (NULL)   -> forces Halt (NULL)
    /// - Caveat = Await (0)     -> degrades Proceed to Await (0)
    /// - Caveat = Proceed (+1)  -> action executes unhindered
    pub fn but(&self, caveat: Self) -> Self {
        match caveat {
            QuadState::Halt => QuadState::Halt,
            QuadState::Rollback => QuadState::Rollback,
            QuadState::Await => {
                if *self == QuadState::Proceed {
```

```
                    QuadState::Await
                } else {
                    *self
                }
            }
            QuadState::Proceed => *self,
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification, Formatter Proofs, Display Formatting, and Export )
// ============================================================================
impl fmt::Display for QuadState {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            QuadState::Rollback => write!(f, "-1:ROLLBACK"),
            QuadState::Await => write!(f, "0:AWAIT"),
            QuadState::Proceed => write!(f, "+1:PROCEED"),
            QuadState::Halt => write!(f, "NULL:HALT"),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_quad_state_micro_and_macro() {
        // Micro labels (linguistic stance)
        assert_eq!(QuadState::Proceed.micro(), "yes");
        assert_eq!(QuadState::Rollback.micro(), "no");
        assert_eq!(QuadState::Await.micro(), "ok");
        assert_eq!(QuadState::Halt.micro(), "void");

        // Macro labels (systems execution physics)
        assert_eq!(QuadState::Proceed.macro_op(), "PROCEED");
        assert_eq!(QuadState::Rollback.macro_op(), "ROLLBACK");
        assert_eq!(QuadState::Await.macro_op(), "AWAIT");
        assert_eq!(QuadState::Halt.macro_op(), "HALT");

        // Parsing micro and macro strings
        assert_eq!(QuadState::from_raw("yes").unwrap(), QuadState::Proceed);
        assert_eq!(QuadState::from_raw("no").unwrap(), QuadState::Rollback);
        assert_eq!(QuadState::from_raw("ok").unwrap(), QuadState::Await);
        assert_eq!(QuadState::from_raw("void").unwrap(), QuadState::Halt);

        assert_eq!(QuadState::from_raw("PROCEED").unwrap(), QuadState::Proceed);
        assert_eq!(QuadState::from_raw("ROLLBACK").unwrap(), QuadState::Rollback);
        assert_eq!(QuadState::from_raw("AWAIT").unwrap(), QuadState::Await);
        assert_eq!(QuadState::from_raw("HALT").unwrap(), QuadState::Halt);
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/vessel.rs RS 320 LINES 1088 WORDS 11.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Vessel Subsystem Identity, Memory Containers & Boundary Invariants )
// ============================================================================
//! OmniCode Core: Memory Vessel & Storehouse Subsystem
//!
//! Codifies the foundational storage nouns of OmniCode:
//!     - Units: Trit, Cell, Word
//!     - Vessels: Vessel, Storehouse, Partition
//!     - Coordinates: Offset, Origin, Landmark
//!     - Measures: Capacity, Count
//!     - Substance: Value, State
//!
//! Governed by Invariant Boundary Physics: An access beyond `Capacity`
//! produces an immediate `-1: ROLLBACK` (Caveat Veto), preserving memory integrity.

use std::fmt;
use crate::core::cell::Cell;
use crate::core::linguistics::{GrammaticalClassification, GrammaticalEntity, PartOfSpeech};
use crate::core::ternary::QuadState;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Vessel, Partition, and Coordinate Struct Definitions )
// ============================================================================

/// A bounded container of memory cells.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Vessel {
    /// Canonical identifier / name of the vessel.
    pub name: String,
    /// Physical or logical origin address / base coordinate.
    pub origin: usize,
    /// Maximum number of cells this vessel is allocated to hold.
    pub capacity: usize,
    /// Number of cells currently written or active.
    pub count: usize,
    /// Dense cell array (where each cell packs to exactly 1 byte in physical RAM).
    pub cells: Vec<Cell>,
    /// Whether this vessel acts as an ambient system Storehouse.
    pub is_storehouse: bool,
    /// If this vessel is a Partition, references the parent Storehouse name.
    pub parent: Option<String>,
}

/// Coordinate offset within a vessel (`Vessel[Offset]`).
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Offset(pub usize);

/// Measurement extent of a vessel's total bounds.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Capacity(pub usize);

/// Measurement count of active cells.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Count(pub usize);


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Vessel Allocation, Partitioning, Bounds Checking, and Cell Read/Write )
// ============================================================================

impl Vessel {
    /// Allocates a new bounded Vessel with the given capacity.
    /// Initialized with empty equilibrium cells (`AWAIT`).
    pub fn new(name: &str, capacity: usize) -> Self {
        Self {
            name: name.to_string(),
            origin: 0,
            capacity,
            count: 0,
            cells: vec![Cell::equilibrium(); capacity],
```

```
            is_storehouse: false,
            parent: None,
        }
    }

    /// Allocates an ambient system Storehouse.
    pub fn new_storehouse(name: &str, capacity: usize) -> Self {
        let mut v = Self::new(name, capacity);
        v.is_storehouse = true;
        v
    }

    /// Grants an entrusted, bounded Partition from this Storehouse.
    ///
    /// # Invariants
    /// - The partition capacity cannot exceed the remaining unallocated capacity.
    /// - Breaches within the partition are contained and cannot overrun the parent.
    pub fn partition(&mut self, partition_name: &str, partition_capacity: usize) -> Result<Self, String> {
        let available = self.capacity.saturating_sub(self.count);
        if partition_capacity > available {
            return Err(format!(
                "FAILPOINT [StorehouseExhaustion]: Requested partition '{partition_name}' of {partition_capacity} cells, \
                 but Storehouse '{}' only has {available} cells remaining (Capacity: {}, Count: {}).",
                self.name, self.capacity, self.count
            ));
        }

        let partition_origin = self.origin + self.count;
        self.count += partition_capacity;

        Ok(Self {
            name: partition_name.to_string(),
            origin: partition_origin,
            capacity: partition_capacity,
            count: 0,
            cells: vec![Cell::equilibrium(); partition_capacity],
            is_storehouse: false,
            parent: Some(self.name.clone()),
        })
    }

    /// Writes a Cell into this vessel at the specified Offset.
    ///
    /// # Invariant Physics
    /// If `offset >= capacity`, boundary physics triggers an immediate `-1: ROLLBACK`!
    pub fn write(&mut self, offset: usize, value: Cell) -> Result<QuadState, String> {
        if offset >= self.capacity {
            return Ok(QuadState::Rollback);
        }

        self.cells[offset] = value;
        if offset >= self.count {
            self.count = offset + 1;
        }

        Ok(QuadState::Proceed)
    }

    /// Reads a Cell from this vessel at the specified Offset.
    ///
    /// # Invariant Physics
    /// If `offset >= capacity`, returns `Cell::from_state(QuadState::Rollback)`.
    pub fn read(&self, offset: usize) -> Cell {
        if offset >= self.capacity {
            Cell::from_state(QuadState::Rollback)
        } else {
            self.cells[offset]
        }
    }

    /// Packs the entire vessel into a dense array of physical 8-bit bytes (1 byte per cell).
    pub fn pack_bytes(&self) -> Vec<u8> {
        self.cells.iter().map(|c| c.pack()).collect()
    }

    /// Reconstructs a Vessel directly from raw physical bytes (e.g. from RAM, MMIO, or disk).
    pub fn unpack_bytes(name: &str, bytes: &[u8]) -> Self {
```

```
        let cells: Vec<Cell> = bytes.iter().map(|&b| Cell::unpack(b)).collect();
        let capacity = cells.len();
        Self {
            name: name.to_string(),
            origin: 0,
            capacity,
            count: capacity,
            cells,
            is_storehouse: false,
            parent: None,
        }
    }
}

impl GrammaticalEntity for Vessel {
    fn classify(&self) -> GrammaticalClassification {
        let role = if self.is_storehouse {
            "System Storehouse; root reservoir of physical or virtual memory cells"
        } else if self.parent.is_some() {
            "Entrusted Partition; bounded memory slice derived from a storehouse"
        } else {
            "Bounded memory vessel; holds sequential cells under invariant law"
        };

        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Preposition),
            None,
            role,
            "Contiguous memory block / Buffer / Page frame",
        )
    }
}

impl GrammaticalEntity for Offset {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Preposition,
            Some(PartOfSpeech::Noun),
            None,
            "Spatial coordinate index within a vessel: Vessel[Offset]",
            "Array index / Memory byte offset",
        )
    }
}

impl GrammaticalEntity for Capacity {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Adjective),
            None,
            "Total allocated capacity measure of a vessel",
            "Buffer size / Maximum allocation limit",
        )
    }
}

impl GrammaticalEntity for Count {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Adjective),
            None,
            "Active or written element count within a vessel",
            "Active buffer length / Populated element count",
        )
    }
}

impl fmt::Display for Vessel {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let kind = if self.is_storehouse {
            "Storehouse"
        } else if self.parent.is_some() {
            "Partition"
        } else {
            "Vessel"
```

```
        };
        write!(
            f,
            "{kind} < {} > [ Capacity: {}, Count: {}, Origin: 0x{:X} ]",
            self.name, self.capacity, self.count, self.origin
        )
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Vessel Invariant Tests, Bounds Safety & Partition Verification )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_vessel_creation_and_bounds_checking() {
        let mut vessel = Vessel::new("Scratchpad", 10);
        assert_eq!(vessel.capacity, 10);
        assert_eq!(vessel.count, 0);

        // In-bounds write
        let cell42 = Cell::from_natural(42).unwrap();
        let res = vessel.write(0, cell42).unwrap();
        assert_eq!(res, QuadState::Proceed);
        assert_eq!(vessel.count, 1);

        // Read back
        let read_cell = vessel.read(0);
        assert_eq!(read_cell.to_natural(), Some(42));
        assert_eq!(read_cell.state, QuadState::Proceed);

        // Out-of-bounds write -> Automatic ROLLBACK (-1)
        let oob_res = vessel.write(15, cell42).unwrap();
        assert_eq!(oob_res, QuadState::Rollback);

        // Out-of-bounds read -> Automatic ROLLBACK cell
        let oob_read = vessel.read(15);
        assert_eq!(oob_read.state, QuadState::Rollback);
    }

    #[test]
    fn test_storehouse_partitioning() {
        let mut storehouse = Vessel::new_storehouse("Main_RAM", 1024);
        assert!(storehouse.is_storehouse);

        // Partition 256 cells
        let mut partition = storehouse.partition("Process_1_Stack", 256).unwrap();
        assert_eq!(partition.capacity, 256);
        assert_eq!(partition.origin, 0);
        assert_eq!(storehouse.count, 256);

        // Write inside partition
        let val = Cell::from_natural(99).unwrap();
        assert_eq!(partition.write(255, val).unwrap(), QuadState::Proceed);
        assert_eq!(partition.count, 256);

        // Boundary breach inside partition triggers rollback
        assert_eq!(partition.write(256, val).unwrap(), QuadState::Rollback);

        // Storehouse exhaustion
        let err = storehouse.partition("Excessive", 1000);
        assert!(err.is_err());
    }

    #[test]
    fn test_dense_byte_packing_round_trip() {
        let mut vessel = Vessel::new("PackedVessel", 4);
        vessel.write(0, Cell::from_natural(10).unwrap()).unwrap();
        vessel.write(1, Cell::from_natural(20).unwrap()).unwrap();
        vessel.write(2, Cell::from_state(QuadState::Rollback)).unwrap();
        vessel.write(3, Cell::from_state(QuadState::Halt)).unwrap();

        let bytes = vessel.pack_bytes();
        assert_eq!(bytes.len(), 4);
```

```
        assert_eq!(bytes[0], 10);
        assert_eq!(bytes[1], 20);
        assert_eq!(bytes[2], crate::core::cell::BYTE_ROLLBACK);
        assert_eq!(bytes[3], crate::core::cell::BYTE_HALT);

        let reconstructed = Vessel::unpack_bytes("Reconstructed", &bytes);
        assert_eq!(reconstructed.read(0).to_natural(), Some(10));
        assert_eq!(reconstructed.read(1).to_natural(), Some(20));
        assert_eq!(reconstructed.read(2).state, QuadState::Rollback);
        assert_eq!(reconstructed.read(3).state, QuadState::Halt);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/word.rs RS 410 LINES 1517 WORDS 13.0 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Machine Word Architecture: 64-Bit Silicon & 32-Trit OS Substrate )
// ================================================================================
//! OmniCode Core: Physical 64-Bit Machine Word & Logical 32-Trit Vector
//!
//! Bridges physical 64-bit binary silicon hardware with the 32-trit balanced
//! ternary operating system kernel:
//!
//! # Bitfield Layout (64 Physical Bits)
//! - **Bits 0..50 (51 bits):** 32-Trit Balanced Ternary Payload (Range: $3^{32} \approx 1.85 \times 10^{15}$)
//! - **Bits 51..52 (2 bits):** Quad-State / Transport Action (Play, Pause, Rewind, Stop)
//! - **Bit 53 (1 bit):** Polarity Affix (0 = Abstract `+`, 1 = Concrete `-`)
//! - **Bits 54..56 (3 bits):** Boundary Rank Delimiter (`()`, `[]`, `<>`, `{}`)
//! - **Bits 57..63 (7 bits):** Moral Invariant Checksum & Axiom Flags (Truth, Life, Justice, etc.)

use std::fmt;
use crate::core::boundaries::BoundaryType;
use crate::core::operators::Polarity;
use crate::core::ternary::{QuadState, TransportAction};

pub const POWERS_OF_3: [u64; 33] = {
    let mut p = [0u64; 33];
    p[0] = 1;
    let mut i = 1;
    while i <= 32 {
        p[i] = p[i - 1] * 3;
        i += 1;
    }
    p
};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Word32Trit: 32-Trit Balanced Ternary ALU Vector )
// ================================================================================

/// A 32-trit balanced ternary word.
/// Each trit is in `{-1, 0, +1}`.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct Word32Trit {
    pub trits: [i8; 32],
}

impl Default for Word32Trit {
    fn default() -> Self {
        Self::zero()
    }
}

impl Word32Trit {
    /// Creates a zeroed 32-trit word (equilibrium: all 0).
    pub const fn zero() -> Self {
        Self { trits: [0; 32] }
    }

    /// Creates a 32-trit word from an explicit array of trits.
    pub fn new(trits: [i8; 32]) -> Result<Self, String> {
        for (idx, &t) in trits.iter().enumerate() {
            if t < -1 || t > 1 {
                return Err(format!("Invalid trit at index {idx}: {t}. Permitted: -1, 0, 1"));
            }
        }
        Ok(Self { trits })
    }

    /// Converts a signed 64-bit integer into a balanced ternary 32-trit word.
    pub fn from_i64(mut val: i64) -> Self {
        let mut trits = [0i8; 32];
        for i in 0..32 {
            if val == 0 {
                break;
            }
            let rem = ((val % 3) + 3) % 3;
```

```
            if rem == 0 {
                trits[i] = 0;
                val /= 3;
            } else if rem == 1 {
                trits[i] = 1;
                val = (val - 1) / 3;
            } else {
                // rem == 2 -> trit is -1, adjust carry (+1)
                trits[i] = -1;
                val = (val + 1) / 3;
            }
        }
        Self { trits }
    }

    /// Decodes the 32-trit word into a standard signed 64-bit integer.
    pub fn to_i64(&self) -> i64 {
        let mut total: i64 = 0;
        let mut power: i64 = 1;
        for i in 0..32 {
            total = total.saturating_add((self.trits[i] as i64).saturating_mul(power));
            if i < 31 {
                power = power.saturating_mul(3);
            }
        }
        total
    }

    /// Negates the 32-trit word in-place with zero sign-extension cost.
    pub fn negate(&self) -> Self {
        let mut trits = [0i8; 32];
        for i in 0..32 {
            trits[i] = -self.trits[i];
        }
        Self { trits }
    }

    /// Balanced ternary full-adder for two 32-trit words.
    pub fn add(&self, other: &Self) -> (Self, i8) {
        let mut trits = [0i8; 32];
        let mut carry: i8 = 0;

        for i in 0..32 {
            let sum = self.trits[i] + other.trits[i] + carry;
            match sum {
                3 => {
                    trits[i] = 0;
                    carry = 1;
                }
                2 => {
                    trits[i] = -1;
                    carry = 1;
                }
                1 => {
                    trits[i] = 1;
                    carry = 0;
                }
                0 => {
                    trits[i] = 0;
                    carry = 0;
                }
                -1 => {
                    trits[i] = -1;
                    carry = 0;
                }
                -2 => {
                    trits[i] = 1;
                    carry = -1;
                }
                -3 => {
                    trits[i] = 0;
                    carry = -1;
                }
                _ => unreachable!("Sum out of range: {sum}"),
            }
        }
        (Self { trits }, carry)
    }
```

```
153
154     /// Packs the 32-trit word into an unsigned 51-bit binary representation.
155     pub fn pack_51bits(&self) -> u64 {
156         let mut packed: u64 = 0;
157         for i in 0..32 {
158             let digit = (self.trits[i] + 1) as u64; // Map -1,0,1 to 0,1,2
159             packed += digit * POWERS_OF_3[i];
160         }
161         packed
162     }
163
164     /// Unpacks a 32-trit word from an unsigned 51-bit binary representation.
165     pub fn unpack_51bits(mut val: u64) -> Self {
166         let mut trits = [0i8; 32];
167         for i in 0..32 {
168             let rem = (val % 3) as i8;
169             trits[i] = rem - 1; // Map 0,1,2 back to -1,0,1
170             val /= 3;
171         }
172         Self { trits }
173     }
174 }
175
176 impl fmt::Display for Word32Trit {
177     fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
178         write!(f, "0t")?;
179         for i in (0..32).rev() {
180             match self.trits[i] {
181                 1 => write!(f, "+")?,
182                 0 => write!(f, "0")?,
183                 -1 => write!(f, "-")?,
184                 _ => write!(f, "?")?,
185             }
186         }
187         write!(f, " ({})", self.to_i64())
188     }
189 }
190
191
192 // ============================================================================
193 // BLOCK 3: BODY BLOCK
194 // ( Word64: 64-Bit Physical Silicon Word Container )
195 // ============================================================================
196
197 /// A 64-bit physical silicon machine word combining a 32-trit payload
198 /// with out-of-band operational metadata.
199 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
200 pub struct Word64 {
201     pub raw: u64,
202 }
203
204 impl Default for Word64 {
205     fn default() -> Self {
206         Self::new(Word32Trit::zero(), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation, 0)
207     }
208 }
209
210 impl Word64 {
211     const PAYLOAD_MASK: u64   = 0x0007_FFFF_FFFF_FFFF; // Bits 0..50 (51 bits)
212     const STATE_SHIFT: u32    = 51;
213     const STATE_MASK: u64     = 0x3;                   // Bits 51..52 (2 bits)
214     const POLARITY_SHIFT: u32 = 53;
215     const POLARITY_MASK: u64  = 0x1;                   // Bit 53 (1 bit)
216     const BOUNDARY_SHIFT: u32 = 54;
217     const BOUNDARY_MASK: u64  = 0x7;                   // Bits 54..56 (3 bits)
218     const CHECKSUM_SHIFT: u32 = 57;
219     const CHECKSUM_MASK: u64  = 0x7F;                  // Bits 57..63 (7 bits)
220
221     /// Constructs a physical 64-bit word from components.
222     pub fn new(
223         payload: Word32Trit,
224         state: QuadState,
225         polarity: Polarity,
226         boundary: BoundaryType,
227         checksum: u8,
228     ) -> Self {
229         let p_val = payload.pack_51bits() & Self::PAYLOAD_MASK;
230         let s_val = match state {
```

```
            QuadState::Await => 0u64,
            QuadState::Proceed => 1u64,
            QuadState::Rollback => 2u64,
            QuadState::Halt => 3u64,
        };
        let pol_val = match polarity {
            Polarity::Abstract => 0u64,
            Polarity::Concrete => 1u64,
        };
        let b_val = match boundary {
            BoundaryType::Invariant => 0u64,
            BoundaryType::Denotation => 1u64,
            BoundaryType::Connotation => 2u64,
            BoundaryType::BottomUp => 3u64,
            BoundaryType::TopDown => 4u64,
            BoundaryType::Aside => 5u64,
        };
        let c_val = (checksum as u64) & Self::CHECKSUM_MASK;

        let raw = p_val
            | (s_val << Self::STATE_SHIFT)
            | (pol_val << Self::POLARITY_SHIFT)
            | (b_val << Self::BOUNDARY_SHIFT)
            | (c_val << Self::CHECKSUM_SHIFT);

        Self { raw }
    }

    /// Returns a zeroed physical 64-bit word with equilibrium payload and Proceed/PLAY transport.
    pub fn zero() -> Self {
        Self::from_signed_int(0)
    }

    /// Convenience constructor from a signed integer value with default Proceed/PLAY transport.
    pub fn from_signed_int(val: i64) -> Self {
        Self::new(
            Word32Trit::from_i64(val),
            QuadState::Proceed,
            Polarity::Concrete,
            BoundaryType::Denotation,
            0,
        )
    }

    /// Convenience constructor from a 32-trit word with default Proceed/PLAY transport.
    pub fn from_trit_payload(payload: Word32Trit) -> Self {
        Self::new(
            payload,
            QuadState::Proceed,
            Polarity::Concrete,
            BoundaryType::Denotation,
            0,
        )
    }

    /// Extracts the 32-trit payload.
    pub fn payload(&self) -> Word32Trit {
        let p_bits = self.raw & Self::PAYLOAD_MASK;
        Word32Trit::unpack_51bits(p_bits)
    }

    /// Extracts the QuadState / Transport Action.
    pub fn state(&self) -> QuadState {
        let s_bits = (self.raw >> Self::STATE_SHIFT) & Self::STATE_MASK;
        match s_bits {
            0 => QuadState::Await,
            1 => QuadState::Proceed,
            2 => QuadState::Rollback,
            _ => QuadState::Halt,
        }
    }

    /// Returns the cinematic transport action: Play, Pause, Rewind, Stop.
    pub fn transport(&self) -> TransportAction {
        self.state().transport()
    }

    /// Extracts the Polarity affix.
```

```
    pub fn polarity(&self) -> Polarity {
        let p_bit = (self.raw >> Self::POLARITY_SHIFT) & Self::POLARITY_MASK;
        if p_bit == 0 {
            Polarity::Abstract
        } else {
            Polarity::Concrete
        }
    }

    /// Extracts the Boundary Type.
    pub fn boundary(&self) -> BoundaryType {
        let b_bits = (self.raw >> Self::BOUNDARY_SHIFT) & Self::BOUNDARY_MASK;
        match b_bits {
            0 => BoundaryType::Invariant,
            1 => BoundaryType::Denotation,
            2 => BoundaryType::Connotation,
            3 => BoundaryType::BottomUp,
            4 => BoundaryType::TopDown,
            _ => BoundaryType::Aside,
        }
    }

    /// Extracts the moral invariant checksum byte.
    pub fn checksum(&self) -> u8 {
        ((self.raw >> Self::CHECKSUM_SHIFT) & Self::CHECKSUM_MASK) as u8
    }

    /// Formats a complete human-readable physical register inspection string.
    pub fn inspect(&self) -> String {
        format!(
            "0x{:016X} | {} | State: {:<8} (Transport: {:<6}) | Polarity: {} | Bound: {}",
            self.raw,
            self.payload(),
            self.state().macro_op(),
            self.transport().as_str(),
            self.polarity().symbol(),
            self.boundary().delimiters().0
        )
    }
}



// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Proof )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_word32_trit_integer_conversion_round_trip() {
        let test_cases = [0, 1, -1, 4, -4, 42, -42, 243, -243, 100_000, -100_000, 1_000_000_000, -1_000_000_000];
        for &val in &test_cases {
            let w = Word32Trit::from_i64(val);
            let back = w.to_i64();
            assert_eq!(val, back, "Round trip failed for value {val}");
        }
    }

    #[test]
    fn test_word32_trit_pack_unpack_round_trip() {
        let values = [0, 1, -1, 12345, -54321, 999_999_999, -999_999_999];
        for &v in &values {
            let w = Word32Trit::from_i64(v);
            let packed = w.pack_51bits();
            let unpacked = Word32Trit::unpack_51bits(packed);
            assert_eq!(w, unpacked, "Pack/unpack mismatch for value {v}");
            assert_eq!(v, unpacked.to_i64());
        }
    }

    #[test]
    fn test_word32_trit_balanced_addition() {
        let a = Word32Trit::from_i64(42);
        let b = Word32Trit::from_i64(58);
        let (sum, carry) = a.add(&b);
```

```
        assert_eq!(carry, 0);
        assert_eq!(sum.to_i64(), 100);

        let c = Word32Trit::from_i64(15);
        let d = Word32Trit::from_i64(-25);
        let (sum2, carry2) = c.add(&d);
        assert_eq!(carry2, 0);
        assert_eq!(sum2.to_i64(), -10);
    }

    #[test]
    fn test_word64_physical_container_round_trip() {
        let payload = Word32Trit::from_i64(1337);
        let w64 = Word64::new(payload, QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation, 42);

        assert_eq!(w64.payload().to_i64(), 1337);
        assert_eq!(w64.state(), QuadState::Proceed);
        assert_eq!(w64.transport(), TransportAction::Play);
        assert_eq!(w64.polarity(), Polarity::Concrete);
        assert_eq!(w64.boundary(), BoundaryType::Denotation);
        assert_eq!(w64.checksum(), 42);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/mod.rs RS 18 LINES 71 WORDS 0.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Terminal Subsystem: Scheduler, Session, and Shell Core )
// ================================================================================
//! OmniCode Terminal Subsystem
//!
//! Provides the primary interactive userspace shell, cooperative process scheduler,
//! and session execution environment for the OmniCode platform and upcoming OS.

pub mod scheduler;
pub mod session;
pub mod studio;

pub use scheduler::{OmniScheduler, OmniTask, SharedScheduler, TaskPriority, Track, create_shared_scheduler};
pub use session::{ExecutionResult, TerminalSession};
pub use studio::OmniStudio;

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/scheduler.rs RS 601 LINES 2121 WORDS 22.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Terminal Task Scheduler: Cooperative Quad-State Process Dispatcher )
// ================================================================================
//! OmniCode Terminal Task Scheduler
//!
//! Provides the primordial process and task scheduling engine for the OmniCode
//! Native Terminal and OS userspace shell.
//!
//! Features:
//! - Quad-State task lifecycles grounded in Balanced Radix-3 logic:
//!     * `+1 (Proceed)`: Active, executing forward vector (`-->`)
//!     * ` 0 (Await)`  : Yielded, paused, asynchronous wait / standby
//!     * `-1 (Rollback)`: Invariant fault, unwinding state (`<--`)
//!     * `NULL (Halt)`  : Terminated, absorbed, process exited
//! - Priority queues (High, Normal, Low, Background)
//! - Cooperative tick loops, task execution time tracking, and IPC-ready task ledger.

use std::path::Path;
use std::sync::{Arc, Mutex};
use std::time::{SystemTime, UNIX_EPOCH};

use crate::core::reel::MemoryReel;
use crate::core::tape::Tape;
use crate::core::ternary::QuadState;
use crate::core::vessel::Vessel;
use crate::diagnostics::logger::DiagnosticLogger;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Task Priorities, Tracks, Task Descriptor & Configuration )
// ================================================================================

/// The 3 physical execution channels of the Multi-Track Scheduler.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum Track {
    /// Track 0: Hardware / Kernel Genesis / Core Daemons.
    Kernel = 0,
    /// Track 1: Terminal Shell & Console HUD.
    Terminal = 1,
    /// Track 2: Userspace & Worker Applications.
    App = 2,
}

impl Track {
    pub fn as_str(&self) -> &'static str {
        match self {
            Track::Kernel => "TRACK 0 [KERNEL]",
            Track::Terminal => "TRACK 1 [TERMINAL]",
            Track::App => "TRACK 2 [APP]",
        }
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum TaskPriority {
    High = 3,
    Normal = 2,
    Low = 1,
    Background = 0,
}

impl TaskPriority {
    pub fn as_str(&self) -> &'static str {
        match self {
            TaskPriority::High => "HIGH",
            TaskPriority::Normal => "NORMAL",
            TaskPriority::Low => "LOW",
            TaskPriority::Background => "BACKGROUND",
        }
    }

    pub fn from_str(s: &str) -> Self {
        match s.trim().to_uppercase().as_str() {
```

```
            "HIGH" | "3" => TaskPriority::High,
            "LOW" | "1" => TaskPriority::Low,
            "BACKGROUND" | "IDLE" | "0" => TaskPriority::Background,
            _ => TaskPriority::Normal,
        }
    }
}

/// Descriptor representing an active, waiting, or halted process in the terminal.
#[derive(Debug, Clone)]
pub struct OmniTask {
    pub id: u64,
    pub name: String,
    pub command: String,
    pub priority: TaskPriority,
    pub track: Track,
    pub state: QuadState,
    pub created_at: u64,
    pub updated_at: u64,
    pub ticks: u64,
    pub log_output: Vec<String>,
    pub tape: Option<Tape>,
    pub partition: Option<Vessel>,
    pub reel: Option<MemoryReel>,
    pub loop_count: usize,
    pub max_loops: usize,
}

impl OmniTask {
    pub fn new(id: u64, name: String, command: String, priority: TaskPriority) -> Self {
        Self::with_track(id, name, command, priority, Track::App)
    }

    pub fn with_track(id: u64, name: String, command: String, priority: TaskPriority, track: Track) -> Self {
        let now = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .unwrap_or_default()
            .as_secs();

        Self {
            id,
            name: name.clone(),
            command,
            priority,
            track,
            state: QuadState::Proceed,
            created_at: now,
            updated_at: now,
            ticks: 0,
            log_output: Vec::new(),
            tape: None,
            partition: None,
            reel: Some(MemoryReel::new(name, 64)),
            loop_count: 0,
            max_loops: 100,
        }
    }


    pub fn to_json(&self) -> String {
        let state_str = match self.state {
            QuadState::Proceed => "+1:PROCEED",
            QuadState::Await => "0:AWAIT",
            QuadState::Rollback => "-1:ROLLBACK",
            QuadState::Halt => "NULL:HALT",
        };

        let logs = self.log_output.join("\\n").replace('"', "\\\"");

        format!(
            r#"{{"id":{},"name":"{}","command":"{}","priority":"{}","track":"{}","state":"{}","ticks":{},"created_at":
{},"logs":"{}"}}"#,
            self.id,
            self.name,
            self.command.replace('"', "\\\""),
            self.priority.as_str(),
            self.track.as_str(),
            state_str,
```

```
            self.ticks,
            self.created_at,
            logs
        )
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Scheduler Core: Cooperative Dispatcher, State Mutations & Tick Engine )
// ================================================================================

#[derive(Debug)]
pub struct OmniScheduler {
    tasks: Vec<OmniTask>,
    next_id: u64,
    pub root_storehouse: Vessel,
}

impl Default for OmniScheduler {
    fn default() -> Self {
        let mut scheduler = Self {
            tasks: Vec::new(),
            next_id: 1,
            root_storehouse: Vessel::new_storehouse("Root_RAM", 4096),
        };
        // Initialize default OS background tasks on Track 0 (Kernel)
        scheduler.spawn_on_track("CoreDaemon", "omnicode kernel --daemon", TaskPriority::Background, Track::Kernel);
        scheduler.spawn_on_track("MemoryWatcher", "omnicode mem --watch", TaskPriority::Low, Track::Kernel);
        scheduler
    }
}

impl OmniScheduler {
    pub fn new() -> Self {
        Self::default()
    }

    /// Spawns a new task in the scheduler on Track 2 (App) by default.
    pub fn spawn(&mut self, name: &str, command: &str, priority: TaskPriority) -> u64 {
        self.spawn_on_track(name, command, priority, Track::App)
    }

    /// Spawns a new task on a designated execution track (Kernel, Terminal, or App).
    pub fn spawn_on_track(&mut self, name: &str, command: &str, priority: TaskPriority, track: Track) -> u64 {
        let id = self.next_id;
        self.next_id += 1;
        let mut task = OmniTask::with_track(id, name.to_string(), command.to_string(), priority, track);
        task.log_output.push(format!(
            "[*] Task spawned on {} with state +1:PROCEED (Priority: {})",
            track.as_str(),
            priority.as_str()
        ));
        self.tasks.push(task);
        id
    }

    /// Mounts a structured [`Tape`] onto an execution track, carving an isolated memory partition.
    pub fn mount_tape(&mut self, track: Track, mut tape: Tape) -> u64 {
        let id = self.next_id;
        self.next_id += 1;

        let part_size = tape.head.required_partition_size;
        let partition_res = self.root_storehouse.partition(&tape.head.name, part_size);

        tape.bound_partition = Some(tape.head.name.clone());
        let priority = match track {
            Track::Kernel => TaskPriority::High,
            Track::Terminal => TaskPriority::Normal,
            Track::App => TaskPriority::Normal,
        };

        let mut task = OmniTask::with_track(
            id,
            tape.head.name.clone(),
            format!("tape:{}", tape.head.name),
            priority,
```

```
            track,
        );

        let part_info = match partition_res {
            Ok(p) => {
                let msg = format!("carved partition '{}' ({} cells)", p.name, p.capacity);
                task.partition = Some(p);
                msg
            }
            Err(e) => format!("partition warning: {e}"),
        };

        task.log_output.push(format!(
            "[+] Mounted Tape '{}' on {} ({}, {} rolls)",
            tape.head.name,
            track.as_str(),
            part_info,
            tape.roll_count()
        ));

        task.tape = Some(tape);
        self.tasks.push(task);
        id
    }

    /// Mounts and parses a `.tape` file from disk onto the specified track.
    pub fn mount_tape_file(&mut self, track: Track, path: impl AsRef<Path>, logger: DiagnosticLogger) -> Result<u64,
String> {
        let tape = Tape::from_file(path, logger)?;
        Ok(self.mount_tape(track, tape))
    }

    /// Advances the scheduler clock by 1 tick, executing tasks in Proceed state.
    ///
    /// For tasks with mounted Tapes, steps the Tape Head forward roll-by-roll.
    pub fn tick(&mut self) -> usize {
        let mut active_executed = 0;
        let now = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .unwrap_or_default()
            .as_secs();

        for task in &mut self.tasks {
            if task.state == QuadState::Proceed {
                task.ticks += 1;
                task.updated_at = now;
                active_executed += 1;

                if let Some(ref mut tape) = task.tape {
                    if !tape.is_finished() {
                        if let Some(roll) = tape.active_roll() {
                            task.log_output.push(format!("==> [HEAD @ Roll {}] {}", tape.cursor + 1, roll.text));
                        }
                        tape.advance();
                        if tape.is_finished() {
                            task.log_output.push(format!(
                                "[√] Tape '{}' execution completed successfully (all {} rolls verified)",
                                tape.head.name,
                                tape.roll_count()
                            ));
                        }
                    }
                }
            }
        }
        active_executed
    }

    /// Manually steps the Tape Head on the given task by 1 roll, executing transport kinematics.
    pub fn step_task(&mut self, id: u64) -> Result<String, String> {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            if let Some(ref mut tape) = task.tape {
                if !tape.is_finished() {
                    let cur_idx = tape.cursor + 1;
                    let roll_text = tape.active_roll().map(|r| r.text.clone()).unwrap_or_default();

                    // Check for STOP transport action
                    let is_unconditional_stop = roll_text.contains("STOP") || (roll_text.contains("HALT") &&
```

```
!roll_text.contains("HALT WHEN"));
                    if is_unconditional_stop {
                        tape.cursor = tape.roll_count();
                        task.state = QuadState::Halt;
                        if let Some(ref mut reel) = task.reel {
                            reel.stop();
                        }
                        let msg = format!("[■ STOP] Task {id}: Execution stopped and sealed at Roll {cur_idx}:
{roll_text}");
                        task.log_output.push(msg.clone());
                        return Ok(msg);
                    }

                    // Check for REWIND transport loop action
                    if roll_text.contains("REWIND") {
                        let cue_opt = if let Some(start) = roll_text.find('<') {
                            if let Some(end) = roll_text[start..].find('>') {
                                Some(roll_text[start + 1..start + end].trim().to_string())
                            } else {
                                None
                            }
                        } else {
                            None
                        };

                        if let Some(cue) = cue_opt {
                            if task.loop_count < task.max_loops {
                                if let Some(target_idx) = tape.find_landmark_index(&cue) {
                                    task.loop_count += 1;
                                    tape.rewind_to(target_idx);
                                    if let Some(ref mut reel) = task.reel {
                                        let _ = reel.rewind_to_landmark(&cue);
                                    }
                                    let msg = format!(
                                        "[⏮ REWIND] Task {id}: Track rewound to Landmark <{cue}> at Roll {} (Iteration
{}/{})",

                                        target_idx + 1, task.loop_count, task.max_loops
                                    );
                                    task.log_output.push(msg.clone());
                                    return Ok(msg);
                                }
                            }
                        }
                    }

                    // Check for PAUSE transport action
                    if roll_text.contains("PAUSE") {
                        tape.advance();
                        task.state = QuadState::Await;
                        if let Some(ref mut reel) = task.reel {
                            reel.pause();
                        }
                        let msg = format!("[⏸ PAUSE] Task {id}: Track paused at Roll {cur_idx}: {roll_text}");
                        task.log_output.push(msg.clone());
                        return Ok(msg);
                    }

                    // Default: PLAY (forward advance)
                    tape.advance();
                    if let Some(ref mut reel) = task.reel {
                        reel.play();
                    }
                    let msg = format!("[▶ PLAY] Step PID {id}: Advanced to Roll {cur_idx}/{}: {roll_text}",
tape.roll_count());
                    task.log_output.push(msg.clone());
                    Ok(msg)
                } else {
                    Ok(format!("[√] Task {id} Tape '{}' has already executed all {} rolls", task.name,
tape.roll_count()))
                }
            } else {
                Err(format!("Task {id} ('{}') has no active mounted Tape ribbon", task.name))
            }
        } else {
            Err(format!("Task ID {id} not found"))
        }
    }

```

```
    /// Displays all rolls of a task's tape with the live `==> [HEAD]` pointer.
    pub fn rolls_for_task(&self, id: u64) -> Option<String> {
        let task = self.tasks.iter().find(|t| t.id == id)?;
        let tape = task.tape.as_ref()?;

        let mut out = String::new();
        out.push_str(&format!("TAPE RIBBON: [{}] (PID: {})\n", tape.head.name, task.id));
        out.push_str(&format!("  Channel:   {}\n", task.track.as_str()));
        out.push_str(&format!("  Tape Head: {}\n", tape.head.raw_text));
        out.push_str("-------------------------------------------------------------\n");

        for (i, roll) in tape.rolls.iter().enumerate() {
            if i == tape.cursor && !tape.is_finished() {
                out.push_str(&format!("==> [{:>2}] {}\n", i + 1, roll.text));
            } else if i < tape.cursor {
                out.push_str(&format!("  √ [{:>2}] {}\n", i + 1, roll.text));
            } else {
                out.push_str(&format!("    [{:>2}] {}\n", i + 1, roll.text));
            }
        }

        if tape.is_finished() {
            out.push_str("-------------------------------------------------------------\n");
            out.push_str("  [√] TAPE EXECUTION COMPLETE (Sealed with PROCEED)\n");
        }
        Some(out)
    }

    /// Returns memory storehouse utilization and active partition breakdown.
    pub fn mem_summary(&self) -> String {
        let total = self.root_storehouse.capacity;
        let allocated = self.root_storehouse.count;
        let free = total.saturating_sub(allocated);
        let pct = if total > 0 { (allocated * 100) / total } else { 0 };

        let mut out = String::new();
        out.push_str("=============================================================\n");
        out.push_str("OMNICODE ROOT STOREHOUSE & TASK PARTITIONS\n");
        out.push_str("=============================================================\n");
        out.push_str(&format!("Root_RAM:     {allocated} / {total} cells ({pct}% Allocated, {free} cells Free)\n"));

        // Visual progress bar
        let filled_bars = pct / 5;
        let empty_bars = 20 - filled_bars;
        out.push_str(&format!("RAM Meter:    [{}{}] {}%\n", "█".repeat(filled_bars), "░".repeat(empty_bars), pct));
        out.push_str("-------------------------------------------------------------\n");
        out.push_str("TASK PARTITION ALLOCATIONS:\n");

        let mut found_part = false;
        for t in &self.tasks {
            if let Some(ref p) = t.partition {
                found_part = true;
                out.push_str(&format!("  • PID {:<3} [{:<16}] Capacity: {:>4} cells (Track: {})\n", t.id, p.name,
p.capacity, t.track.as_str()));
            }
        }
        if !found_part {
            out.push_str("  (No active task partitions carved yet)\n");
        }
        out.push_str("=============================================================\n");
        out
    }

    /// Returns a structured summary grouping tasks by Track.
    pub fn track_summary(&self) -> String {
        let mut out = String::new();
        out.push_str("=============================================================\n");
        out.push_str("MULTI-TRACK EXECUTION CHANNELS\n");
        out.push_str("=============================================================\n");

        for track in [Track::Kernel, Track::Terminal, Track::App] {
            out.push_str(&format!("\n[ {} ]\n", track.as_str()));
            let track_tasks: Vec<&OmniTask> = self.tasks.iter().filter(|t| t.track == track).collect();
            if track_tasks.is_empty() {
                out.push_str("  • (No active processes on this track)\n");
            } else {
                for t in track_tasks {
                    let tape_tag = if t.tape.is_some() { "[TAPE]" } else { "      " };
```

```
                    let state_badge = match t.state {
                        QuadState::Proceed => "+1:PROCEED",
                        QuadState::Await => " 0:AWAIT  ",
                        QuadState::Rollback => "-1:ROLLBCK",
                        QuadState::Halt => "NULL:HALT ",
                    };
                    out.push_str(&format!("  • PID {:<3} {} {:<10} {:<16} ticks: {}\n", t.id, tape_tag, state_badge,
t.name, t.ticks));
                }
            }
        }
        out
    }

    /// Transitions a task to 0:AWAIT (Standby/Paused).
    pub fn pause_task(&mut self, id: u64) -> Result<(), String> {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.state = QuadState::Await;
            task.log_output.push("[~] Task transitioned to 0:AWAIT (Suspended)".to_string());
            Ok(())
        } else {
            Err(format!("Task ID {id} not found"))
        }
    }

    /// Resumes a task back to +1:PROCEED (Running).
    pub fn resume_task(&mut self, id: u64) -> Result<(), String> {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.state = QuadState::Proceed;
            task.log_output.push("[+] Task resumed to +1:PROCEED".to_string());
            Ok(())
        } else {
            Err(format!("Task ID {id} not found"))
        }
    }

    /// Signals a rollback on a task (-1:ROLLBACK).
    pub fn rollback_task(&mut self, id: u64, reason: &str) -> Result<(), String> {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.state = QuadState::Rollback;
            task.log_output.push(format!("[-] Task rolled back to -1:ROLLBACK. Reason: {reason}"));
            Ok(())
        } else {
            Err(format!("Task ID {id} not found"))
        }
    }

    /// Halts and terminates a task (NULL:HALT).
    pub fn halt_task(&mut self, id: u64) -> Result<(), String> {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.state = QuadState::Halt;
            task.log_output.push("[!] Task terminated with state NULL:HALT".to_string());
            Ok(())
        } else {
            Err(format!("Task ID {id} not found"))
        }
    }

    /// Appends output log line to a task.
    pub fn log_task(&mut self, id: u64, line: &str) {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.log_output.push(line.to_string());
        }
    }

    /// Returns all tasks formatted as a JSON array.
    pub fn to_json(&self) -> String {
        let task_jsons: Vec<String> = self.tasks.iter().map(|t| t.to_json()).collect();
        format!("[{}]", task_jsons.join(","))
    }

    /// Returns a human-readable table of all running processes.
    pub fn format_table(&self) -> String {
        let mut out = String::new();
        out.push_str("PID   STATE        CHANNEL         PRIORITY    TICKS   COMMAND\n");
        out.push_str("-------------------------------------------------------------------------------\n");
        for t in &self.tasks {
            let state_badge = match t.state {
```

```
                QuadState::Proceed => "+1 PROCEED",
                QuadState::Await => " 0 AWAIT  ",
                QuadState::Rollback => "-1 ROLLBCK",
                QuadState::Halt => "NULL HALT ",
            };
            out.push_str(&format!(
                "{:<5} {:<12} {:<15} {:<11} {:<7} {}\n",
                t.id,
                state_badge,
                t.track.as_str(),
                t.priority.as_str(),
                t.ticks,
                t.command
            ));
        }
        out
    }

    pub fn tasks(&self) -> &[OmniTask] {
        &self.tasks
    }

    pub fn get_task(&self, id: u64) -> Option<&OmniTask> {
        self.tasks.iter().find(|t| t.id == id)
    }

    pub fn get_task_mut(&mut self, id: u64) -> Option<&mut OmniTask> {
        self.tasks.iter_mut().find(|t| t.id == id)
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Global Thread-Safe Scheduler Handle & Tests )
// ============================================================================

pub type SharedScheduler = Arc<Mutex<OmniScheduler>>;

pub fn create_shared_scheduler() -> SharedScheduler {
    Arc::new(Mutex::new(OmniScheduler::new()))
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_scheduler_lifecycle() {
        let mut sched = OmniScheduler::new();
        let pid = sched.spawn("TestProcess", "omnicode test", TaskPriority::High);

        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Proceed);

        sched.tick();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().ticks, 1);

        sched.pause_task(pid).unwrap();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Await);

        sched.resume_task(pid).unwrap();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Proceed);

        sched.halt_task(pid).unwrap();
        assert_eq!(sched.tasks.iter().find(|t| t.id == pid).unwrap().state, QuadState::Halt);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/session.rs RS 435 LINES 1554 WORDS 18.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Terminal Session State: In-Memory REPL Environment & Command History )
// ============================================================================
//! OmniCode Terminal Session Engine
//!
//! Maintains interactive state, variable bindings, command history, and
//! dispatches instructions to the Task Scheduler or the OmniCode Runtime Evaluator.
//!
//! # Living Operating System Shell
//! Rather than executing commands through isolated subprocess forks, the OmniCode
//! Terminal Session evaluates rolls directly in persistent silicon memory via
//! [`Evaluator`] and [`Environment`]. It provides direct hardware interaction commands:
//! - `mount <path.tape>`: Mounts a tape ribbon onto an execution track.
//! - `run <path.tape>`: Mounts and advances a tape to completion.
//! - `step <pid>`: Manually advances a Tape Head by 1 roll.
//! - `rolls <pid>`: Displays the Tape Head pointer and all rolls.
//! - `mem`: Displays the ANSI Root_Storehouse utilization meter.
//! - `tracks`: Displays tasks organized across all 3 execution channels.

use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::time::Instant;

use super::scheduler::{OmniScheduler, SharedScheduler, TaskPriority, Track};
use crate::cogs::tripwire::Tripwire;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::Environment;
use crate::runtime::evaluator::Evaluator;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Session Models, History Entries, and Execution Results )
// ============================================================================

#[derive(Debug, Clone)]
pub struct ExecutionResult {
    pub stdout: String,
    pub state: QuadState,
    pub duration_us: u128,
}

impl ExecutionResult {
    pub fn success(stdout: String, duration_us: u128) -> Self {
        Self {
            stdout,
            state: QuadState::Proceed,
            duration_us,
        }
    }

    pub fn await_state(stdout: String, duration_us: u128) -> Self {
        Self {
            stdout,
            state: QuadState::Await,
            duration_us,
        }
    }

    pub fn failure(stdout: String, duration_us: u128) -> Self {
        Self {
            stdout,
            state: QuadState::Rollback,
            duration_us,
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
```

```rust
// ( Terminal Session Implementation & Command Dispatcher )
// ============================================================================

pub struct TerminalSession {
    pub history: Vec<String>,
    pub variables: HashMap<String, String>,
    pub scheduler: SharedScheduler,
    pub env: Environment,
    pub logger: DiagnosticLogger,
    pub debugger: OmniDebugger,
}

impl Default for TerminalSession {
    fn default() -> Self {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let env = Environment::new();
        Self {
            history: Vec::new(),
            variables: HashMap::new(),
            scheduler: Arc::new(Mutex::new(OmniScheduler::new())),
            env,
            logger,
            debugger,
        }
    }
}

impl TerminalSession {
    pub fn new(scheduler: SharedScheduler) -> Self {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let env = Environment::new();
        Self {
            history: Vec::new(),
            variables: HashMap::new(),
            scheduler,
            env,
            logger,
            debugger,
        }
    }

    /// Dispatches a command line and returns an ExecutionResult.
    pub fn execute(&mut self, line: &str) -> ExecutionResult {
        let trimmed = line.trim();
        if trimmed.is_empty() {
            return ExecutionResult::success(String::new(), 0);
        }

        self.history.push(trimmed.to_string());
        let start = Instant::now();

        // 1. Process table
        if trimmed == "ps" || trimmed == "tasks" {
            let sched = self.scheduler.lock().unwrap();
            let table = sched.format_table();
            return ExecutionResult::success(table, start.elapsed().as_micros());
        }

        // 2. Multi-track channel summary
        if trimmed == "tracks" || trimmed == "channels" {
            let sched = self.scheduler.lock().unwrap();
            let summary = sched.track_summary();
            return ExecutionResult::success(summary, start.elapsed().as_micros());
        }

        // 3. Storehouse memory meter
        if trimmed == "mem" || trimmed == "ram" {
            let sched = self.scheduler.lock().unwrap();
            let summary = sched.mem_summary();
            return ExecutionResult::success(summary, start.elapsed().as_micros());
        }

        // 4. Mount a tape file onto Track 2 (App) or Track 0 (Kernel)
        if trimmed.starts_with("mount ") {
            let path = trimmed["mount ".len()..].trim();
            let mut sched = self.scheduler.lock().unwrap();
```

```
            let track = if path.contains("kernel") || path.contains("boot") {
                Track::Kernel
            } else {
                Track::App
            };
            match sched.mount_tape_file(track, path, self.logger.clone()) {
                Ok(pid) => return ExecutionResult::success(
                    format!("[+] Mounted Tape '{path}' onto {} with PID {pid}", track.as_str()),
                    start.elapsed().as_micros(),
                ),
                Err(e) => {
                    let err = Tripwire::trip(
                        &self.logger,
                        "TERMINAL_MOUNT",
                        FailpointType::SyntaxAnomaly,
                        format!("Failed to mount tape '{path}': {e}"),
                    );
                    return ExecutionResult::failure(
                        format!("[!] {err}"),
                        start.elapsed().as_micros(),
                    );
                }
            }
        }

        // 5. Mount and execute a tape file to completion
        if trimmed.starts_with("run ") && (trimmed.ends_with(".tape") || trimmed.ends_with(".omni")) {
            let path = trimmed["run ".len()..].trim();
            let mut sched = self.scheduler.lock().unwrap();
            let track = if path.contains("kernel") || path.contains("boot") {
                Track::Kernel
            } else {
                Track::App
            };
            match sched.mount_tape_file(track, path, self.logger.clone()) {
                Ok(pid) => {
                    let mut logs = Vec::new();
                    logs.push(format!("[+] Mounted Tape '{path}' onto {} (PID {pid})", track.as_str()));
                    loop {
                        match sched.step_task(pid) {
                            Ok(msg) => {
                                logs.push(msg.clone());
                                if msg.contains("completed") || msg.contains("already executed") {
                                    break;
                                }
                            }
                            Err(e) => {
                                let err = Tripwire::trip(
                                    &self.logger,
                                    "TERMINAL_RUN",
                                    FailpointType::SyntaxAnomaly,
                                    format!("Task {pid} execution halted: {e}"),
                                );
                                logs.push(format!("[!] {err}"));
                                break;
                            }
                        }
                    }
                    return ExecutionResult::success(logs.join("\n"), start.elapsed().as_micros());
                }
                Err(e) => {
                    let err = Tripwire::trip(
                        &self.logger,
                        "TERMINAL_RUN",
                        FailpointType::SyntaxAnomaly,
                        format!("Failed to run tape '{path}': {e}"),
                    );
                    return ExecutionResult::failure(
                        format!("[!] {err}"),
                        start.elapsed().as_micros(),
                    );
                }
            }
        }

        // 6. Step Tape Head on a task
        if trimmed.starts_with("step ") {
            let pid_str = trimmed["step ".len()..].trim();
```

```
            if let Ok(pid) = pid_str.parse::<u64>() {
                let mut sched = self.scheduler.lock().unwrap();
                match sched.step_task(pid) {
                    Ok(msg) => return ExecutionResult::success(msg, start.elapsed().as_micros()),
                    Err(e) => return ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros()),
                }
            } else {
                return ExecutionResult::failure("Usage: step <pid>".to_string(), start.elapsed().as_micros());
            }
        }

        // 7. Display rolls on a task
        if trimmed.starts_with("rolls ") {
            let pid_str = trimmed["rolls ".len()..].trim();
            if let Ok(pid) = pid_str.parse::<u64>() {
                let sched = self.scheduler.lock().unwrap();
                match sched.rolls_for_task(pid) {
                    Some(rolls_str) => return ExecutionResult::success(rolls_str, start.elapsed().as_micros()),
                    None => return ExecutionResult::failure(format!("[!] Task {pid} not found or has no mounted tape"),
start.elapsed().as_micros()),
                }
            } else {
                return ExecutionResult::failure("Usage: rolls <pid>".to_string(), start.elapsed().as_micros());
            }
        }

        // 8. General task spawning
        if trimmed.starts_with("spawn ") {
            let cmd = trimmed["spawn ".len()..].trim();
            let mut sched = self.scheduler.lock().unwrap();
            let pid = sched.spawn("UserTask", cmd, TaskPriority::Normal);
            return ExecutionResult::success(
                format!("[+] Spawned process PID {pid}: '{cmd}' with state +1:PROCEED"),
                start.elapsed().as_micros(),
            );
        }

        // 9. Task controls
        if trimmed.starts_with("pause ") || trimmed.starts_with("await ") {
            let pid_str = trimmed.split_whitespace().nth(1).unwrap_or("");
            if let Ok(pid) = pid_str.parse::<u64>() {
                let mut sched = self.scheduler.lock().unwrap();
                match sched.pause_task(pid) {
                    Ok(()) => return ExecutionResult::await_state(format!("[~] Task {pid} transitioned to 0:AWAIT"),
start.elapsed().as_micros()),
                    Err(e) => return ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros()),
                }
            }
        }

        if trimmed.starts_with("resume ") {
            let pid_str = trimmed.split_whitespace().nth(1).unwrap_or("");
            if let Ok(pid) = pid_str.parse::<u64>() {
                let mut sched = self.scheduler.lock().unwrap();
                match sched.resume_task(pid) {
                    Ok(()) => return ExecutionResult::success(format!("[+] Task {pid} resumed to +1:PROCEED"),
start.elapsed().as_micros()),
                    Err(e) => return ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros()),
                }
            }
        }

        if trimmed.starts_with("kill ") || trimmed.starts_with("halt ") {
            let pid_str = trimmed.split_whitespace().nth(1).unwrap_or("");
            if let Ok(pid) = pid_str.parse::<u64>() {
                let mut sched = self.scheduler.lock().unwrap();
                match sched.halt_task(pid) {
                    Ok(()) => return ExecutionResult::success(format!("[!] Task {pid} halted (NULL:HALT)"),
start.elapsed().as_micros()),
                    Err(e) => return ExecutionResult::failure(format!("[!] Error: {e}"), start.elapsed().as_micros()),
                }
            }
        }

        if trimmed == "tick" {
            let mut sched = self.scheduler.lock().unwrap();
            let count = sched.tick();
            return ExecutionResult::success(format!("[*] Advanced scheduler clock 1 tick. Executed {count} active
```

```
tasks."), start.elapsed().as_micros());
        }

        if trimmed == "vars" || trimmed == "env" {
            let mut out = String::from("ACTIVE SESSION STATE:\n");
            let mut has_entries = false;
            for (k, v) in self.env.bindings() {
                out.push_str(&format!("  {} = {:?}\n", k, v));
                has_entries = true;
            }
            for (k, v) in &self.variables {
                out.push_str(&format!("  let {} = {}\n", k, v));
                has_entries = true;
            }
            if !has_entries {
                return ExecutionResult::success("No active session bindings declared.".to_string(),
start.elapsed().as_micros());
            }
            return ExecutionResult::success(out, start.elapsed().as_micros());
        }

        if trimmed == "help" {
            let help = "OMNICODE NATIVE OPERATING SYSTEM SHELL:
  ps, tasks          List scheduled tasks and active processes
  tracks             Display the 3 execution channels (Kernel, Terminal, Apps)
  mem, ram           Display ANSI storehouse RAM meter and partition allocations
  mount <path.tape>  Mount a tape ribbon onto an execution track
  run <path.tape>    Mount and execute a process tape to completion
  step <pid>         Manually step a Tape Head forward by 1 roll
  rolls <pid>        View tape rolls and the live ==> [HEAD] pointer
  spawn <cmd>        Spawn a new task under the task scheduler
  pause <pid>        Transition a process to 0:AWAIT (standby)
  resume <pid>       Resume a process to +1:PROCEED (running)
  kill <pid>         Halt and terminate a process (NULL:HALT)
  tick               Advance the cooperative scheduler clock
  vars, env          Display in-memory declared variables and Vessels
  clear              Clear the terminal screen
  help               Display this help reference

 LIVE OMNICODE STATEMENTS & ROLLS:
  Type any valid roll (e.g. `A [ 10 ]`, `42 --> Buf @ 0`, `Storehouse v-- App [ 256 ]`)
  to execute directly in persistent silicon memory!
";
            return ExecutionResult::success(help.to_string(), start.elapsed().as_micros());
        }

        // Variable assignment: let x = 42
        if trimmed.starts_with("let ") && trimmed.contains('=') {
            let parts: Vec<&str> = trimmed["let ".len()..].splitn(2, '=').collect();
            if parts.len() == 2 {
                let name = parts[0].trim().to_string();
                let val = parts[1].trim().to_string();
                self.variables.insert(name.clone(), val.clone());
                return ExecutionResult::success(format!("Variable '{name}' bound to: {val}"),
start.elapsed().as_micros());
            }
        }

        // 10. Direct In-Memory Roll Evaluation (OmniCode Native Execution)
        // If the statement looks like OmniCode syntax, execute directly without subprocess overhead!
        if trimmed.contains("-->") || trimmed.contains("<--") || trimmed.contains("v--") || trimmed.contains("--^")
            || trimmed.starts_with("Storehouse") || (trimmed.contains('[') && trimmed.contains(']'))
            || (trimmed.contains('<') && trimmed.contains('>'))
            || trimmed.starts_with("status")
        {
            let mut lexer = Lexer::new(trimmed, self.logger.clone());
            if let Ok(tokens) = lexer.tokenize() {
                let mut parser = Parser::new(tokens, self.debugger.clone());
                if let Ok(prog) = parser.parse_program() {
                    let mut evaluator = Evaluator::new(self.env.clone(), self.debugger.clone());
                    match evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog)) {
                        Ok(val) => {
                            self.env = evaluator.env; // Persist environment state!
                            return ExecutionResult::success(
                                format!("=> {:?} (state: +1:PROCEED)", val),
                                start.elapsed().as_micros(),
                            );
                        }
```

```
                        Err(e) => {
                            let err = Tripwire::trip(
                                &self.logger,
                                "TERMINAL_EVAL",
                                FailpointType::SyntaxAnomaly,
                                format!("Evaluation Failpoint: {e}"),
                            );
                            return ExecutionResult::failure(
                                format!("[!] {err}"),
                                start.elapsed().as_micros(),
                            );
                        }
                    }
                }
            }
        }

        // 11. Fallback: CLI Subcommand Execution
        let clean_cmd = if trimmed.starts_with("omnicode ") {
            trimmed.to_string()
        } else {
            format!("omnicode {trimmed}")
        };

        let exe = std::env::current_exe().unwrap_or_else(|_| std::path::PathBuf::from("omnicode"));
        let args: Vec<&str> = clean_cmd["omnicode ".len()..].split_whitespace().collect();
        let output = std::process::Command::new(&exe)
            .args(&args)
            .output();

        match output {
            Ok(out) => {
                let stdout = String::from_utf8_lossy(&out.stdout).to_string();
                let stderr = String::from_utf8_lossy(&out.stderr).to_string();
                let combined = if !stderr.is_empty() {
                    format!("{stdout}\n{stderr}")
                } else {
                    stdout
                };
                if out.status.success() {
                    ExecutionResult::success(combined, start.elapsed().as_micros())
                } else {
                    ExecutionResult::failure(combined, start.elapsed().as_micros())
                }
            }
            Err(e) => ExecutionResult::failure(format!("Failed to execute command: {e}"), start.elapsed().as_micros()),
        }
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Terminal Session Sealing & Verification )
// ================================================================================
// Terminal session module sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/terminal/studio.rs RS 220 LINES 851 WORDS 10.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniStudio: Cinematic Projector, Set Props & Spindle HUD )
// ============================================================================
//! OmniCode Terminal Studio & Cinematic Projector
//!
//! Provides a real-time visual projection HUD of the 4-directional Memory Reel,
//! showing active tracks, scenes, rolls, and set props (64-bit / 32-trit cells):
//!
//! # Visual HUD Elements
//! 1. **Projector Head:** Active Reel, Channel Track, Transport Status (`PLAY`, `PAUSE`, `REWIND`, `STOP`).
//! 2. **Spindle Depth Meter:** Current Scene nesting level (`v--` depth).
//! 3. **The Script (Active Roll):** Current instruction line rolling through the gate.
//! 4. **Set Props (Memory Grid):** Bounded cells holding physical 64-bit words.

use crate::core::mmio::{VgaBuffer, VgaColor};
use crate::core::reel::MemoryReel;
use crate::core::tape::Tape;
use crate::core::ternary::TransportAction;
use crate::core::word::Word64;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( OmniStudio Struct Definition and Configuration )
// ============================================================================

pub struct OmniStudio {
    pub reel: MemoryReel,
    pub active_track: &'static str,
    pub vga: VgaBuffer,
}

impl OmniStudio {
    /// Creates a new Studio Projector initialized with a named memory reel.
    pub fn new(reel_name: impl Into<String>, capacity: usize) -> Self {
        Self {
            reel: MemoryReel::new(reel_name, capacity),
            active_track: "TRACK 2 [APP]",
            vga: VgaBuffer::new(),
        }
    }

    /// Sets the active channel track.
    pub fn set_track(&mut self, track: &'static str) {
        self.active_track = track;
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Projector Frame Rendering & HUD Formatting )
// ============================================================================

impl OmniStudio {
    /// Interprets a single tape roll instruction against the memory reel.
    pub fn step_roll(&mut self, roll_text: &str) -> String {
        if roll_text.contains("STOP") || roll_text.contains("HALT") {
            self.reel.stop();
            return "Kinematics: Reel stopped and transport sealed (■ STOP)".to_string();
        }
        if roll_text.contains("PAUSE") {
            self.reel.pause();
            return "Kinematics: Track paused waiting on cue (⏸ PAUSE)".to_string();
        }
        if roll_text.contains("v--") {
            let scene_name = if let Some(pos) = roll_text.find("v--") {
                roll_text.get(pos + 3..).unwrap_or("").split_whitespace().next().unwrap_or("SubScene")
            } else {
                "SubScene"
            };
            self.reel.sink_in(scene_name, 16, None);
            return format!("Kinematics: Sunk into sub-scene <{}> (v-- Spindle Depth {})", scene_name, self.reel.depth());
        }
        if roll_text.contains("^--") {
```

```
            let _ = self.reel.pop_out();
            return format!("Kinematics: Popped out to parent scene (^-- Spindle Depth {})", self.reel.depth());
        }
        if roll_text.contains("<--") {
            self.reel.pull_back();
            return "Kinematics: Pulled back track head (<--)".to_string();
        }
        if roll_text.contains("Screen") || roll_text.contains("VGA") || roll_text.contains("CONSOLE") ||
roll_text.contains("0x000B8000") {
            let extracted = if let Some(first_quote) = roll_text.find('\'') {
                if let Some(second_quote) = roll_text.get(first_quote + 1..).and_then(|s| s.find('\'')) {
                    roll_text.get(first_quote + 1..first_quote + 1 + second_quote)
                } else {
                    None
                }
            } else if let Some(first_quote) = roll_text.find('"') {
                if let Some(second_quote) = roll_text.get(first_quote + 1..).and_then(|s| s.find('"')) {
                    roll_text.get(first_quote + 1..first_quote + 1 + second_quote)
                } else {
                    None
                }
            } else {
                None
            };

            if let Some(screen_msg) = extracted {
                self.vga.write_str(screen_msg, VgaColor::LightGreen, VgaColor::Black);
                self.vga.new_line();
                return format!("Hardware MMIO: Wrote '{screen_msg}' to Bare-Metal Video Buffer @ 0x000B8000");
            }
        }
        if roll_text.contains("-->") {
            // Check if there is an integer value to push
            let val = roll_text
                .split_whitespace()
                .find_map(|s| s.parse::<i64>().ok())
                .unwrap_or(1);
            let word = Word64::from_signed_int(val);
            let _ = self.reel.push_forward(word);
            return format!("Kinematics: Inscribed word ({val}) and pushed forward (-->)");
        }
        self.reel.play();
        "Kinematics: Tape head advancing smoothly (▶ PLAY)".to_string()
    }

    /// Renders a complete cinematic projector HUD frame for a tape and its active roll.
    pub fn render_frame(&self, tape: &Tape, roll_idx: usize, message: &str) -> String {
        let mut out = String::new();
        let transport_badge = match self.reel.transport {
            TransportAction::Play => "\x1b[1;32m[▶ PLAY]\x1b[0m",
            TransportAction::Pause => "\x1b[1;33m[⏸ PAUSE]\x1b[0m",
            TransportAction::Rewind => "\x1b[1;35m[⏮ REWIND]\x1b[0m",
            TransportAction::Stop => "\x1b[1;31m[■ STOP]\x1b[0m",
        };

        let scene = self.reel.active_scene();
        let landmark_str = scene.landmark.as_deref().unwrap_or("none");


out.push_str("\x1b[1;36m╔══════════════════════════════════════════════════════════════════════════════╗\x1b[0m\n");
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  \x1b[1mOMNICODE CINEMA STUDIO\x1b[0m | REEL: \x1b[1;33m[{}]\x1b[0m | {}
\x1b[1;36m║\x1b[0m\n",
            self.reel.name, transport_badge
        ));
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  CHANNEL: {:<18} | SPINDLE DEPTH: {} | SCENE: <{}>  \x1b[1;36m║\x1b[0m\n",
            self.active_track, scene.depth, scene.name
        ));
        out.push_str(&format!(
            "\x1b[1;36m║\x1b[0m  CLAPPER CUE: <{:<16}> | HEAD POSITION: @0x{:04X}              \x1b[1;36m║\x1b[0m\n",
            landmark_str, scene.head_pos
        ));

out.push_str("\x1b[1;36m╠══════════════════════════════════════════════════════════════════════════════╣\x1b[0m\n");

        // Script Gate: Active Roll
        let roll_count = tape.roll_count();
```

```
        let roll_text = tape.rolls.get(roll_idx).map(|r| r.text.as_str()).unwrap_or("EOF");
        out.push_str(&format!(
            "\x1b[1;36m┃\x1b[0m  \x1b[1;32mSCRIPT GATE [Roll {}/{}]:\x1b[0m {:<48} \x1b[1;36m┃\x1b[0m\n",
            roll_idx + 1, roll_count, roll_text
        ));
        out.push_str(&format!(
            "\x1b[1;36m┃\x1b[0m  {:<76} \x1b[1;36m┃\x1b[0m\n",
            message
        ));

out.push_str("\x1b[1;36m╠══════════════════════════════════════════════════════════════════════════════╣\x1b[0m\n");

        // Set Props (Active Memory Cells)
        out.push_str("\x1b[1;36m┃\x1b[0m  \x1b[1;33mSET PROPS (MEMORY CELL GRID):\x1b[0m
\x1b[1;36m┃\x1b[0m\n");
        let display_cells = 4.min(scene.capacity);
        for i in 0..display_cells {
            if let Some(word) = scene.read(i) {
                let focus_ptr = if i == scene.head_pos { "▶" } else { " " };
                out.push_str(&format!(
                    "\x1b[1;36m┃\x1b[0m   {} [0x{:04X}]: 0x{:016X} | Val: {:<5} | State: {:<7} \x1b[1;36m┃\x1b[0m\n",
                    focus_ptr, i, word.raw, word.payload().to_i64(), word.state().macro_op()
                ));
            }
        }

out.push_str("\x1b[1;36m╚══════════════════════════════════════════════════════════════════════════════╝\x1b[0m\n");
        if self.vga.cursor_col > 0 || self.vga.cursor_row > 0 {
            out.push_str(&self.vga.render_screen(4));
        }
        out
    }

    /// Projects an entire tape sequentially with terminal cinematic frames.
    pub fn project_tape(&mut self, tape: &Tape) {
        println!("\x1b[1;32m▶ Mounting & Projecting Reel [{}] on {}\x1b[0m\n", self.reel.name, self.active_track);
        for (idx, roll) in tape.rolls.iter().enumerate() {
            let msg = self.step_roll(&roll.text);
            let frame = self.render_frame(tape, idx, &msg);
            print!("{frame}");
        }
        println!("\x1b[1;32m✓ Projection Complete: Tape sealed in {}\x1b[0m\n", tape.execution_state.macro_op());
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;

    #[test]
    fn test_studio_render_frame() {
        let logger = DiagnosticLogger::new(false);
        let tape = Tape::from_source(
            r#"
            [Tape.TestProjection] : WHO {Kernel} WHAT {Test} --> DRIVE {Cinema}
            roll_1 [ 42 --> Cell_0 @ 0 ]
            roll_2 [ commit_status [ STOP ] ]
            "#,
            logger,
        ).expect("Parse tape");

        let studio = OmniStudio::new("TestReel", 16);
        let frame = studio.render_frame(&tape, 0, "Test kinematically advancing");
        assert!(frame.contains("OMNICODE CINEMA STUDIO"));
        assert!(frame.contains("TestReel"));
        assert!(frame.contains("SET PROPS"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexer.rs RS 943 LINES 4072 WORDS 40.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Lexer Identity, Ontological TokenType Hierarchy & Morphemic Invariants )
// ============================================================================
//! # OmniCode Lexical Tokenizer (`lexer.rs`)
//!
//! Transforms raw source text reels into structured [`Token`] streams, enforcing
//! the physical and ontological boundary invariants of OmniCode:
//!
//! ## Architectural Foundations:
//! 1. **Morphemic Compounding**:
//!    OmniCode constructs compound operators by fusing base characters without arbitrary
//!    jargon (e.g. `::` + `@` = `::@` "and is located at"; `~` + `=` = `~=` "to be like";
//!    `<` + `=` + `>` = `<=>` "invariant equivalence / transduction").
//!
//! 2. **The 6 Ranked Morphological Boundary Delimiters**:
//!    - Rank 1 `< ... >`: Invariant Axiom (Absolute Immutable Law & Ontological Truth)
//!    - Rank 2 `[ ... ]`: Denotation (Concrete State Container & Silicon Evaluation Block)
//!    - Rank 3 `{ ... }`: Connotation (Context, Environment & Payload Attributes)
//!    - Rank 4 `[ ... }`: Inside-Out Emergence (Output Projection to External Universe)
//!    - Rank 5 `{ ... ]`: Outside-In Imposition (Input Capture from External Universe)
//!    - Rank 6 `( ... )`: Intent Aside (First-Class Cognitive Teleology & Rationale)
//!
//! 3. **First-Class Machine Intent**:
//!    Asides enclosed in `( ... )` are **never** discarded as trivia or comments.
//!    They are preserved in the token stream and lowered into the AST as persistent
//!    teleological metadata nodes, recording the *why* of every machine state.
//!
//! 4. **Causal State-Primes (``` ` ```)**:
//!    Discrete clock cycles advance an entity from state $S$ to state $S'$. The state-prime
//!    operator (``` ` ```) is tokenized as part of the identifier (e.g. `Character```, `Cell```,
//!    `Status```, `Target```, `Text```), designating causality without string collision.
//!
//! 5. **Dual-Substrate Arithmetic & Pronouns**:
//!    Supports balanced-ternary literals (`+1:PROCEED`, `-1:ROLLBACK`, `0:AWAIT`),
//!    hexadecimal hardware addresses (`0x01`, `0x000B8000`), SVO natural arithmetic
//!    (`plus`, `minus`, `times`, `div`), and agentic pronouns (`WHO`, `WHAT`, `WHEN`,
//!    `WHERE`, `WHY`, `HOW`, `I`, `you`, `it`, `they`, `myself`, `itself`, `_`).

use crate::core::boundaries::BoundaryType;
use crate::core::operators::{AffixPosition, Polarity};
use crate::core::ternary::QuadState;
use crate::core::linguistics::PronounKind;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};

/// Complete taxonomy of grammatical tokens emitted by the Lexer.
#[derive(Debug, Clone, PartialEq)]
pub enum TokenType {
    // ------------------------------------------------------------------------
    // 1. Boundary Delimiters & First-Class Asides
    // ------------------------------------------------------------------------
    /// `[` Opening Denotation / Bottom-Up Emergence
    LBracket,
    /// `]` Closing Denotation / Top-Down Imposition
    RBracket,
    /// `{` Opening Connotation / Top-Down Imposition
    LBrace,
    /// `}` Closing Connotation / Bottom-Up Emergence
    RBrace,
    /// `<` Opening Invariant Axiom
    LAngle,
    /// `>` Closing Invariant Axiom
    RAngle,
    /// `( ... )` First-class Intent Aside capturing architectural rationale
    Aside(String),

    // ------------------------------------------------------------------------
    // 2. Grammatical Pronouns & Reflexives
    // ------------------------------------------------------------------------
    /// Agentic pronouns (`I`, `you`, `it`, `they`, `myself`, `itself`, `_`, etc.)
    Pronoun(PronounKind),

    // ------------------------------------------------------------------------
    // 3. Polarity & Boundary Cast Operators
```

```
    // --------------------------------------------------------------------------
    /// Affixed Identifiers (`+Word` Abstract, `-Word` Concrete, `Word+`, `Word-`)
    AffixedIdent {
        name: String,
        polarity: Polarity,
        position: AffixPosition,
    },
    /// Boundary Casts (`[+]`, `[-]`, `<+>`, `<->`, `{+}`, `{-}`)
    BoundaryCast {
        boundary: BoundaryType,
        polarity: Polarity,
    },

    // --------------------------------------------------------------------------
    // 4. Directional Vectors (Axial, Diagonal Scale Modulations & 2D Quadrants)
    // --------------------------------------------------------------------------
    /// `-->` / `->` Act Upon (Forward Vector / Transduction)
    ActUpon,
    /// `<--` / `<-` Act Again (Feedback / Rollback / Reverse Vector)
    ActAgain,
    /// `^--` / `--^` / `-^` Builds To (Ascension / Synthesis Vector)
    BuildsTo,
    /// `v--` / `--v` / `-v` Breaks Down From (Descension / Decomposition Vector)
    BreaksDownFrom,
    /// `/-^` Major scale up (Ascend forward)
    Major,
    /// `\-v` Minor scale down (Descend forward)
    Minor,
    /// `\-^` Great scale up (Ascend backward)
    Great,
    /// `/-v` Small scale down (Descend backward)
    Small,
    /// `/^` Quadrant I / Northeast
    UpperRight,
    /// `\v` Quadrant IV / Southeast
    LowerRight,
    /// `^\` Quadrant II / Northwest
    UpperLeft,
    /// `v/` Quadrant III / Southwest
    LowerLeft,

    // --------------------------------------------------------------------------
    // 5. Relations of Being & Invariant Transduction
    // --------------------------------------------------------------------------
    /// `=` Is (Relational Identity / Root Assignment)
    Is,
    /// `!=` Is Not (Relational Disparity)
    IsNot,
    /// `~=` To Be Like (Fuzzy Affinity / Morphological Similarity)
    ToBeLike,
    /// `<=>` Invariant Equivalence (Bijective Dimensional Transduction: Bits <=> Trits)
    InvariantEquivalence,
    /// `<` Less Than (Casted context comparison)
    LessThan,
    /// `>` Greater Than (Casted context comparison)
    GreaterThan,
    /// `<=` Less Than or Equal To (Bound comparison: `{ Character` <= [ 1 Cell ] }`)
    LessEqual,
    /// `>=` Greater Than or Equal To (Bound comparison)
    GreaterEqual,

    // --------------------------------------------------------------------------
    // 6. Connectives, Conjunctions & Bindings
    // --------------------------------------------------------------------------
    /// `!` Adversative Caveat (A BUT B / Guard Interlock)
    But,
    /// `::` Inseparable Conjunction / `::@` Composed Locator
    And,
    /// `:` In Relation To / Type Classification
    InRelationTo,
    /// `|` Disjunctive Alternative / Branching Choice
    Or,
    /// `--` Unary Negation (Not)
    Not,
    /// `&` Inseparable Twin Binding (A&B Hand-in-Hand)
    BoundPair,

    // --------------------------------------------------------------------------
```

```rust
    // 7. Prepositions, Locators, Ranges & Measures
    // ---------------------------------------------------------------------------
    /// `@` Located At (MMIO Address / Stream Port)
    At,
    /// `..` Coordinate Range / Interval Span
    DotDot,
    /// `%` Proportion / Fullness / Fraction
    Percent,

    // ---------------------------------------------------------------------------
    // 8. Cardinal Classification Clauses (The 5 W's and 1 H)
    // ---------------------------------------------------------------------------
    /// `WHO` Agent / Actor / Identity
    Who,
    /// `WHAT` Substance / Entity / Vessel
    What,
    /// `WHEN` Cadence / Lifecycle / Trigger
    When,
    /// `WHERE` Spatial Site / Memory Partition / Scope
    Where,
    /// `WHY` Teleological Intent / Rationale / Axiom
    Why,
    /// `HOW` Causal Mechanism / Status / Roll Vector
    How,

    // ---------------------------------------------------------------------------
    // 9. SVO Arithmetic Keywords
    // ---------------------------------------------------------------------------
    /// `plus`, `add`
    Plus,
    /// `minus`, `sub`
    Minus,
    /// `times`, `mult`
    Times,
    /// `div`, `over`, `divided_by`
    Div,

    // ---------------------------------------------------------------------------
    // 10. Inferential Operators & Terminals
    // ---------------------------------------------------------------------------
    /// `?` Infer / Epistemic Query
    Infer,
    /// Balanced-Ternary Literals (`+1`, `-1`, `0`, `NULL`, `PROCEED`, `ROLLBACK`, `AWAIT`, `HALT`)
    Ternary(QuadState),
    /// Numeric Literals (Hexadecimal `0x...`, Decimal Integers, Floats)
    Number(f64),
    /// String Literals (`"..."`, `'...'`)
    StringLit(String),
    /// Generic Identifiers & SVO Verbs (supporting state-prime ``` ` ```)
    Identifier(String),
    /// End-of-file sentinel
    Eof,
}

/// Inscribed token unit with source reel coordinate tracking.
#[derive(Debug, Clone, PartialEq)]
pub struct Token {
    /// Grammatical token classification.
    pub token_type: TokenType,
    /// Exact character sequence matched in the source text.
    pub lexeme: String,
    /// 1-indexed line coordinate in source reel.
    pub line: usize,
    /// 1-indexed column coordinate in source reel.
    pub col: usize,
}


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Lexer State Structure, Cursor Navigation & Scanning Primitives )
// ============================================================================
/// State container for source text reel tokenization.
pub struct Lexer<'a> {
    /// Original borrowed source text.
    _source: &'a str,
    /// Decoded Unicode characters representing the source reel tape.
    chars: Vec<char>,
```

```rust
    /// Current character cursor position.
    pos: usize,
    /// Active line number (1-indexed).
    line: usize,
    /// Active column number (1-indexed).
    col: usize,
    /// Depth counter for nested square brackets `[...]`.
    pub bracket_depth: usize,
    /// Diagnostic logger recording failpoint telemetry and lexical events.
    pub logger: DiagnosticLogger,
}

impl<'a> Lexer<'a> {
    /// Constructs a new [`Lexer`] over an OmniCode source reel string.
    ///
    /// # Arguments
    /// * `source` - Complete OmniCode UTF-8 source string.
    /// * `logger` - Diagnostic logger for telemetry and failpoint recording.
    pub fn new(source: &'a str, logger: DiagnosticLogger) -> Self {
        Self {
            _source: source,
            chars: source.chars().collect(),
            pos: 0,
            line: 1,
            col: 1,
            bracket_depth: 0,
            logger,
        }
    }

    /// Inspects a future character at `offset` positions ahead without advancing.
    #[inline]
    fn peek(&self, offset: usize) -> Option<char> {
        let idx = self.pos + offset;
        if idx < self.chars.len() {
            Some(self.chars[idx])
        } else {
            None
        }
    }

    /// Finds the next non-whitespace character starting at `start_offset`.
    #[inline]
    fn peek_next_non_whitespace(&self, start_offset: usize) -> Option<char> {
        let mut off = start_offset;
        while let Some(ch) = self.peek(off) {
            if !ch.is_whitespace() {
                return Some(ch);
            }
            off += 1;
        }
        None
    }

    /// Advances the cursor by one character, updating line and column coordinates.
    #[inline]
    fn advance(&mut self) -> Option<char> {
        if self.pos < self.chars.len() {
            let ch = self.chars[self.pos];
            self.pos += 1;
            if ch == '\n' {
                self.line += 1;
                self.col = 1;
            } else {
                self.col += 1;
            }
            Some(ch)
        } else {
            None
        }
    }

    /// Matches and consumes a target string literal if it matches characters at cursor.
    fn match_str(&mut self, target: &str) -> bool {
        let target_chars: Vec<char> = target.chars().collect();
        for (i, &tc) in target_chars.iter().enumerate() {
            if self.peek(i) != Some(tc) {
                return false;
```

```
            }
        }
        for _ in 0..target_chars.len() {
            self.advance();
        }
        true
    }

    /// Skips whitespace and line comments (`# ...`).
    fn skip_whitespace(&mut self) {
        while let Some(ch) = self.peek(0) {
            if ch.is_whitespace() {
                self.advance();
            } else if ch == '#' {
                while let Some(c) = self.peek(0) {
                    if c == '\n' {
                        break;
                    }
                    self.advance();
                }
            } else {
                break;
            }
        }
    }

    /// Scans a first-class teleological Intent Aside enclosed in `( ... )`.
    ///
    /// Preserves nested parentheses and trims surrounding whitespace.
    fn read_aside(&mut self, start_line: usize, start_col: usize) -> Result<Token, String> {
        let mut depth = 1;
        let mut content = String::new();

        while let Some(ch) = self.advance() {
            match ch {
                '(' => {
                    depth += 1;
                    content.push(ch);
                }
                ')' => {
                    depth -= 1;
                    if depth == 0 {
                        let text = content.trim().to_string();
                        self.logger.log("ASIDE", format!("Captured Aside: ({text})"));
                        return Ok(Token {
                            token_type: TokenType::Aside(text),
                            lexeme: format!("({content})"),
                            line: start_line,
                            col: start_col,
                        });
                    }
                    content.push(ch);
                }
                _ => content.push(ch),
            }
        }

        let msg = format!("Unclosed Aside '(' opened at L{start_line}:C{start_col} reaches EOF");
        self.logger.failpoint(FailpointType::BoundaryLeak, "LEXER", &msg);
        Err(format!("FAILPOINT [UnclosedAside]: {msg}"))
    }

    /// Scans a quoted string literal (`"..."` or `'...'`).
    fn read_string(&mut self, quote: char, start_line: usize, start_col: usize) -> Result<Token, String> {
        let mut s = String::new();
        while let Some(ch) = self.advance() {
            if ch == quote {
                return Ok(Token {
                    token_type: TokenType::StringLit(s.clone()),
                    lexeme: format!("\"{s}\""),
                    line: start_line,
                    col: start_col,
                });
            } else if ch == '\\' {
                if let Some(esc) = self.advance() {
                    s.push(esc);
                }
            } else {
```

```
                s.push(ch);
            }
        }
        let msg = format!("Unclosed string opened at L{start_line}:C{start_col} reaches EOF");
        self.logger.failpoint(FailpointType::SyntaxAnomaly, "LEXER", &msg);
        Err(format!("FAILPOINT: {msg}"))
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Tokenization Transformation Pipeline & Modular Scanning Engines )
// ============================================================================
impl<'a> Lexer<'a> {
    /// Tokenizes the entire source reel into a complete [`Token`] stream.
    ///
    /// Evaluates sequential lexical passes:
    /// 1. Skip whitespace and `#` comments.
    /// 2. First-class Intent Asides `( ... )`.
    /// 3. Multi-character compound operators and directional vectors.
    /// 4. Prefixed polarity identifiers (`+Word`, `-Word`).
    /// 5. Single-character morphological boundaries and operators.
    /// 6. Numeric literals (Hexadecimal and Decimal).
    /// 7. Identifiers, State-Primes (```Word` ```), Postfix Polarities, Pronouns, and Keywords.
    pub fn tokenize(&mut self) -> Result<Vec<Token>, String> {
        let mut tokens = Vec::new();

        while self.peek(0).is_some() {
            self.skip_whitespace();
            if self.peek(0).is_none() {
                break;
            }

            let cur_line = self.line;
            let cur_col = self.col;

            // ---------------------------------------------------------------
            // 1. FIRST-CLASS INTENT ASIDES: ( ... )
            // ---------------------------------------------------------------
            if self.peek(0) == Some('(') {
                self.advance();
                tokens.push(self.read_aside(cur_line, cur_col)?);
                continue;
            }

            // ---------------------------------------------------------------
            // 2. MULTI-CHARACTER COMPOUND OPERATORS, VECTORS & LOCATORS
            // ---------------------------------------------------------------
            if let Some(tok) = self.match_compound_operators(cur_line, cur_col) {
                tokens.push(tok);
                continue;
            }

            // ---------------------------------------------------------------
            // 3. PREFIXED POLARITY IDENTIFIERS: +Word (Abstract), -Word (Concrete)
            // ---------------------------------------------------------------
            if (self.peek(0) == Some('+') || self.peek(0) == Some('-'))
                && self.peek(1).map_or(false, |c| c.is_alphabetic() || c == '_')
            {
                tokens.push(self.read_prefixed_polarity(cur_line, cur_col));
                continue;
            }

            // ---------------------------------------------------------------
            // 4. BOUNDARY DELIMITERS, NUMBERS, STRINGS & IDENTIFIERS
            // ---------------------------------------------------------------
            let ch = self.peek(0).unwrap();
            match ch {
                // Morphological Boundaries: [ ] { } < >
                '[' | ']' | '{' | '}' | '<' | '>' => {
                    tokens.push(self.read_boundary_delimiter(ch, tokens.last(), cur_line, cur_col));
                }

                // Single-Character Operators & Connectives
                '=' | '!' | ':' | '|' | '?' | '@' | '%' | '&' | '+' | '-' | '*' => {
                    tokens.push(self.read_single_char_operator(ch, cur_line, cur_col));
                }
```

```
                // String Literals
                '"' | '\'' => {
                    self.advance();
                    tokens.push(self.read_string(ch, cur_line, cur_col)?);
                }

                // Statement and Element Separators
                ';' | ',' => {
                    self.advance();
                }

                // Numeric Literals (Hexadecimal `0x...` or Decimal)
                _ if ch.is_ascii_digit() => {
                    tokens.push(self.read_number(ch, cur_line, cur_col)?);
                }

                // Identifiers, State-Primes, Pronouns, and Keywords
                _ if ch.is_alphabetic() || ch == '_' => {
                    tokens.push(self.read_identifier_or_keyword(cur_line, cur_col));
                }

                // Unrecognized Character Anomaly
                other => {
                    let msg = format!("Unexpected character '{other}' at L{cur_line}:C{cur_col}");
                    self.logger.failpoint(FailpointType::SyntaxAnomaly, "LEXER", &msg);
                    return Err(format!("FAILPOINT: {msg}"));
                }
            }
        }

        tokens.push(Token {
            token_type: TokenType::Eof,
            lexeme: "".into(),
            line: self.line,
            col: self.col,
        });
        Ok(tokens)
    }

    /// Matches multi-character compound operators, directional vectors, casts, and locators.
    fn match_compound_operators(&mut self, cur_line: usize, cur_col: usize) -> Option<Token> {
        // Freeform Boundary Casts: [+], [-], <+>, <->, {+}, {-}
        if self.match_str("[+]") {
            return Some(Token { token_type: TokenType::BoundaryCast { boundary: BoundaryType::Denotation, polarity:
Polarity::Abstract }, lexeme: "[+]".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("[-]") {
            return Some(Token { token_type: TokenType::BoundaryCast { boundary: BoundaryType::Denotation, polarity:
Polarity::Concrete }, lexeme: "[-]".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("<+>") {
            return Some(Token { token_type: TokenType::BoundaryCast { boundary: BoundaryType::Invariant, polarity:
Polarity::Abstract }, lexeme: "<+>".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("<->") {
            return Some(Token { token_type: TokenType::BoundaryCast { boundary: BoundaryType::Invariant, polarity:
Polarity::Concrete }, lexeme: "<->".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("{+}") {
            return Some(Token { token_type: TokenType::BoundaryCast { boundary: BoundaryType::Connotation, polarity:
Polarity::Abstract }, lexeme: "{+}".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("{-}") {
            return Some(Token { token_type: TokenType::BoundaryCast { boundary: BoundaryType::Connotation, polarity:
Polarity::Concrete }, lexeme: "{-}".into(), line: cur_line, col: cur_col });
        }

        // 3-Character Directional Vectors & Equivalence
        if self.match_str("-->") {
            return Some(Token { token_type: TokenType::ActUpon, lexeme: "-->".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("<--") {
            return Some(Token { token_type: TokenType::ActAgain, lexeme: "<--".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("--^") || self.match_str("^--") {
            return Some(Token { token_type: TokenType::BuildsTo, lexeme: "^--".into(), line: cur_line, col: cur_col });
        }
```

```
        if self.match_str("--v") || self.match_str("v--") {
            return Some(Token { token_type: TokenType::BreaksDownFrom, lexeme: "v--".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("<=>") {
            return Some(Token { token_type: TokenType::InvariantEquivalence, lexeme: "<=>".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("::@") {
            return Some(Token { token_type: TokenType::And, lexeme: "::@".into(), line: cur_line, col: cur_col });
        }

        // 2-Character Directional Vectors, Relations & Connectives
        if self.match_str("->") {
            return Some(Token { token_type: TokenType::ActUpon, lexeme: "->".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("<-") {
            return Some(Token { token_type: TokenType::ActAgain, lexeme: "<-".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("-^") {
            return Some(Token { token_type: TokenType::BuildsTo, lexeme: "-^".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("-v") {
            return Some(Token { token_type: TokenType::BreaksDownFrom, lexeme: "-v".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("::") {
            return Some(Token { token_type: TokenType::And, lexeme: "::".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("!=") {
            return Some(Token { token_type: TokenType::IsNot, lexeme: "!=".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("~=") {
            return Some(Token { token_type: TokenType::ToBeLike, lexeme: "~=".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("~!") {
            return Some(Token { token_type: TokenType::But, lexeme: "~!".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("<=") {
            return Some(Token { token_type: TokenType::LessEqual, lexeme: "<=".into(), line: cur_line, col: cur_col });
        }
        if self.match_str(">=") {
            return Some(Token { token_type: TokenType::GreaterEqual, lexeme: ">=".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("--") {
            return Some(Token { token_type: TokenType::Not, lexeme: "--".into(), line: cur_line, col: cur_col });
        }

        // Diagonal Scale Modulations & 2D Quadrants
        if self.match_str("/-^") {
            return Some(Token { token_type: TokenType::Major, lexeme: "/-^".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("\\-v") {
            return Some(Token { token_type: TokenType::Minor, lexeme: "\\-v".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("\\-^") {
            return Some(Token { token_type: TokenType::Great, lexeme: "\\-^".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("/-v") {
            return Some(Token { token_type: TokenType::Small, lexeme: "/-v".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("/^") {
            return Some(Token { token_type: TokenType::UpperRight, lexeme: "/^".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("\\v") {
            return Some(Token { token_type: TokenType::LowerRight, lexeme: "\\v".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("^\\") {
            return Some(Token { token_type: TokenType::UpperLeft, lexeme: "^\\".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("v/") {
            return Some(Token { token_type: TokenType::LowerLeft, lexeme: "v/".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("..") {
            return Some(Token { token_type: TokenType::DotDot, lexeme: "..".into(), line: cur_line, col: cur_col });
        }

```

```
        // Signed Balanced-Ternary Literals: +1, -1
        if self.match_str("+1") {
            return Some(Token { token_type: TokenType::Ternary(QuadState::Proceed), lexeme: "+1".into(), line: cur_line, col: cur_col });
        }
        if self.match_str("-1") {
            return Some(Token { token_type: TokenType::Ternary(QuadState::Rollback), lexeme: "-1".into(), line: cur_line, col: cur_col });
        }

        None
    }

    /// Scans a prefixed polarity identifier (`+Word` or `-Word`).
    fn read_prefixed_polarity(&mut self, cur_line: usize, cur_col: usize) -> Token {
        let sign = self.advance().unwrap();
        let polarity = if sign == '+' { Polarity::Abstract } else { Polarity::Concrete };
        let mut ident = String::new();
        while let Some(c) = self.peek(0) {
            if c.is_alphanumeric() || c == '_' || c == '.' || c == '?' {
                ident.push(c);
                self.advance();
            } else {
                break;
            }
        }
        let lexeme = format!("{}{}", sign, ident);
        Token {
            token_type: TokenType::AffixedIdent {
                name: ident,
                polarity,
                position: AffixPosition::Prefix,
            },
            lexeme,
            line: cur_line,
            col: cur_col,
        }
    }

    /// Scans a single-character morphological boundary delimiter.
    fn read_boundary_delimiter(&mut self, ch: char, last_token: Option<&Token>, cur_line: usize, cur_col: usize) -> Token {
        self.advance();
        match ch {
            '[' => {
                self.bracket_depth += 1;
                Token { token_type: TokenType::LBracket, lexeme: "[".into(), line: cur_line, col: cur_col }
            }
            ']' => {
                self.bracket_depth = self.bracket_depth.saturating_sub(1);
                Token { token_type: TokenType::RBracket, lexeme: "]".into(), line: cur_line, col: cur_col }
            }
            '{' => Token { token_type: TokenType::LBrace, lexeme: "{".into(), line: cur_line, col: cur_col },
            '}' => Token { token_type: TokenType::RBrace, lexeme: "}".into(), line: cur_line, col: cur_col },
            '<' => {
                // Canon 3 Disambiguation: Casted context comparison [ { A } < { B } ]
                if self.bracket_depth > 0
                    && last_token.map_or(false, |t| t.token_type == TokenType::RBrace)
                    && self.peek_next_non_whitespace(0) == Some('{')
                {
                    Token { token_type: TokenType::LessThan, lexeme: "<".into(), line: cur_line, col: cur_col }
                } else {
                    Token { token_type: TokenType::LAngle, lexeme: "<".into(), line: cur_line, col: cur_col }
                }
            }
            '>' => {
                // Canon 3 Disambiguation: Casted context comparison [ { A } > { B } ]
                if self.bracket_depth > 0
                    && last_token.map_or(false, |t| t.token_type == TokenType::RBrace)
                    && self.peek_next_non_whitespace(0) == Some('{')
                {
                    Token { token_type: TokenType::GreaterThan, lexeme: ">".into(), line: cur_line, col: cur_col }
                } else {
                    Token { token_type: TokenType::RAngle, lexeme: ">".into(), line: cur_line, col: cur_col }
                }
            }
            _ => unreachable!(),
        }
```

```
    }

    /// Scans a single-character operator or connective.
    fn read_single_char_operator(&mut self, ch: char, cur_line: usize, cur_col: usize) -> Token {
        self.advance();
        match ch {
            '=' => Token { token_type: TokenType::Is, lexeme: "=".into(), line: cur_line, col: cur_col },
            '!' => Token { token_type: TokenType::But, lexeme: "!".into(), line: cur_line, col: cur_col },
            ':' => Token { token_type: TokenType::InRelationTo, lexeme: ":".into(), line: cur_line, col: cur_col },
            '|' => Token { token_type: TokenType::Or, lexeme: "|".into(), line: cur_line, col: cur_col },
            '?' => Token { token_type: TokenType::Infer, lexeme: "?".into(), line: cur_line, col: cur_col },
            '@' => Token { token_type: TokenType::At, lexeme: "@".into(), line: cur_line, col: cur_col },
            '%' => Token { token_type: TokenType::Percent, lexeme: "%".into(), line: cur_line, col: cur_col },
            '&' => Token { token_type: TokenType::BoundPair, lexeme: "&".into(), line: cur_line, col: cur_col },
            '+' => Token { token_type: TokenType::Plus, lexeme: "+".into(), line: cur_line, col: cur_col },
            '-' => Token { token_type: TokenType::Minus, lexeme: "-".into(), line: cur_line, col: cur_col },
            '*' => Token { token_type: TokenType::Times, lexeme: "*".into(), line: cur_line, col: cur_col },
            _ => unreachable!(),
        }
    }

    /// Scans numeric literals, including hexadecimal (`0x...` / `0X...`),
    /// standalone balanced-ternary zero `0` (Await), and decimal floats/integers.
    fn read_number(&mut self, ch: char, cur_line: usize, cur_col: usize) -> Result<Token, String> {
        // Hexadecimal hardware address locator: 0x01, 0x000B8000
        if ch == '0' && (self.peek(1) == Some('x') || self.peek(1) == Some('X')) {
            self.advance(); // consume '0'
            self.advance(); // consume 'x' or 'X'
            let mut hex_str = String::new();
            while let Some(c) = self.peek(0) {
                if c.is_ascii_hexdigit() {
                    hex_str.push(c);
                    self.advance();
                } else {
                    break;
                }
            }
            let val = u64::from_str_radix(&hex_str, 16)
                .map_err(|e| format!("FAILPOINT: Invalid hex number '0x{hex_str}': {e}"))?;
            return Ok(Token {
                token_type: TokenType::Number(val as f64),
                lexeme: format!("0x{hex_str}"),
                line: cur_line,
                col: cur_col,
            });
        }

        // Standalone ternary zero `0` representing Await / Equilibrium
        if ch == '0' && !self.peek(1).map_or(false, |c| c.is_alphanumeric() || c == '.') {
            self.advance();
            return Ok(Token {
                token_type: TokenType::Ternary(QuadState::Await),
                lexeme: "0".into(),
                line: cur_line,
                col: cur_col,
            });
        }

        // Decimal integer or floating point number
        let mut num_str = String::new();
        while let Some(c) = self.peek(0) {
            if c.is_ascii_digit() || c == '.' {
                num_str.push(c);
                self.advance();
            } else {
                break;
            }
        }
        let val: f64 = num_str.parse().map_err(|e| format!("FAILPOINT: Invalid number '{num_str}': {e}"))?;
        Ok(Token {
            token_type: TokenType::Number(val),
            lexeme: num_str,
            line: cur_line,
            col: cur_col,
        })
    }

    /// Scans identifiers, SVO verbs, state-primes (``` ` ```), postfix polarities,
```

```
    /// pronouns, and grammatical keywords.
    fn read_identifier_or_keyword(&mut self, cur_line: usize, cur_col: usize) -> Token {
        let mut ident = String::new();

        // Accumulate identifier characters, including state-prime (`) and ticks (')
        while let Some(c) = self.peek(0) {
            if c.is_alphanumeric() || c == '_' || c == '.' || c == '?' || c == '\'' || c == '`' {
                ident.push(c);
                self.advance();
            } else {
                break;
            }
        }

        // Check for attached Postfix Polarity: Word+ (Abstract), Word- (Concrete)
        let mut postfix_polarity = None;
        if self.peek(0) == Some('+') {
            let next_after = self.peek(1);
            if next_after.map_or(true, |c| {
                c.is_whitespace() || c == ']' || c == '}' || c == '>' || c == ')' || c == ',' || c == ';' || c == '!' ||
c == '@' || c == ':'
            }) {
                self.advance();
                postfix_polarity = Some(Polarity::Abstract);
            }
        } else if self.peek(0) == Some('-') {
            let next_after = self.peek(1);
            if next_after.map_or(true, |c| {
                c != '>' && c != '-' && c != '^' && c != 'v' && (
                    c.is_whitespace() || c == ']' || c == '}' || c == '>' || c == ')' || c == ',' || c == ';' || c == '!'
|| c == '@' || c == ':'
                )
            }) {
                self.advance();
                postfix_polarity = Some(Polarity::Concrete);
            }
        }

        if let Some(polarity) = postfix_polarity {
            let lexeme = format!("{}{}", ident, polarity.symbol());
            return Token {
                token_type: TokenType::AffixedIdent {
                    name: ident,
                    polarity,
                    position: AffixPosition::Postfix,
                },
                lexeme,
                line: cur_line,
                col: cur_col,
            };
        }

        // Disambiguate keywords, pronouns, ternary states, and general identifiers
        match ident.to_lowercase().as_str() {
            "null" | "void" | "halt" | "stop" => Token {
                token_type: TokenType::Ternary(QuadState::Halt),
                lexeme: ident,
                line: cur_line,
                col: cur_col,
            },
            "yes" | "proceed" | "play" => Token {
                token_type: TokenType::Ternary(QuadState::Proceed),
                lexeme: ident,
                line: cur_line,
                col: cur_col,
            },
            "ok" | "await" | "pause" => Token {
                token_type: TokenType::Ternary(QuadState::Await),
                lexeme: ident,
                line: cur_line,
                col: cur_col,
            },
            "no" | "rollback" | "rewind" => Token {
                token_type: TokenType::Ternary(QuadState::Rollback),
                lexeme: ident,
                line: cur_line,
                col: cur_col,
            },
```

```
            "plus" => Token { token_type: TokenType::Plus, lexeme: ident, line: cur_line, col: cur_col },
            "minus" => Token { token_type: TokenType::Minus, lexeme: ident, line: cur_line, col: cur_col },
            "times" => Token { token_type: TokenType::Times, lexeme: ident, line: cur_line, col: cur_col },
            "div" => Token { token_type: TokenType::Div, lexeme: ident, line: cur_line, col: cur_col },
            "who" => Token { token_type: TokenType::Who, lexeme: ident, line: cur_line, col: cur_col },
            "what" => Token { token_type: TokenType::What, lexeme: ident, line: cur_line, col: cur_col },
            "when" => Token { token_type: TokenType::When, lexeme: ident, line: cur_line, col: cur_col },
            "where" => Token { token_type: TokenType::Where, lexeme: ident, line: cur_line, col: cur_col },
            "why" => Token { token_type: TokenType::Why, lexeme: ident, line: cur_line, col: cur_col },
            "how" => Token { token_type: TokenType::How, lexeme: ident, line: cur_line, col: cur_col },
            _ => {
                if let Some(p) = PronounKind::from_keyword(&ident) {
                    Token { token_type: TokenType::Pronoun(p), lexeme: ident, line: cur_line, col: cur_col }
                } else {
                    Token { token_type: TokenType::Identifier(ident.clone()), lexeme: ident, line: cur_line, col: cur_col
}
                }
            }
        }
    }
}


// ==============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Lexical Verification Proofs, Boundary Integrity & Test Oracles )
// ==============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_lexing_pronouns_and_reflexives() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("I you it they myself itself my your _", logger);
        let tokens = lexer.tokenize().expect("Lexing pronouns");

        assert_eq!(tokens[0].token_type, TokenType::Pronoun(PronounKind::I));
        assert_eq!(tokens[1].token_type, TokenType::Pronoun(PronounKind::You));
        assert_eq!(tokens[2].token_type, TokenType::Pronoun(PronounKind::It));
        assert_eq!(tokens[3].token_type, TokenType::Pronoun(PronounKind::They));
        assert_eq!(tokens[4].token_type, TokenType::Pronoun(PronounKind::Myself));
        assert_eq!(tokens[5].token_type, TokenType::Pronoun(PronounKind::Itself));
        assert_eq!(tokens[6].token_type, TokenType::Pronoun(PronounKind::My));
        assert_eq!(tokens[7].token_type, TokenType::Pronoun(PronounKind::Your));
        assert_eq!(tokens[8].token_type, TokenType::Pronoun(PronounKind::Wildcard));
    }

    #[test]
    fn test_lexing_hexadecimal_numbers_and_locators() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("0x01 ::@ 0x000B8000", logger);
        let tokens = lexer.tokenize().expect("Lexing hex numbers");

        assert_eq!(tokens[0].token_type, TokenType::Number(1.0));
        assert_eq!(tokens[1].token_type, TokenType::And);
        assert_eq!(tokens[1].lexeme, "::@");
        assert_eq!(tokens[2].token_type, TokenType::Number(0xB8000 as f64));
    }

    #[test]
    fn test_lexing_state_primes_and_svo_verbs() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("Character` --> INSCRIBE [ Cell ] = [ Character` ]", logger);
        let tokens = lexer.tokenize().expect("Lexing state primes and verbs");

        assert_eq!(tokens[0].lexeme, "Character`");
        assert_eq!(tokens[1].token_type, TokenType::ActUpon);
        assert_eq!(tokens[2].lexeme, "INSCRIBE");
        assert_eq!(tokens[3].token_type, TokenType::LBracket);
        assert_eq!(tokens[4].lexeme, "Cell");
        assert_eq!(tokens[5].token_type, TokenType::RBracket);
        assert_eq!(tokens[6].token_type, TokenType::Is);
        assert_eq!(tokens[7].token_type, TokenType::LBracket);
        assert_eq!(tokens[8].lexeme, "Character`");
        assert_eq!(tokens[9].token_type, TokenType::RBracket);
    }

```

```
    #[test]
    fn test_lexing_relational_bounds_and_transduction() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("65 <=> [ -1, +1, +1, -1, +1 ]", logger);
        let tokens = lexer.tokenize().expect("Lexing transduction and trits");

        assert_eq!(tokens[0].token_type, TokenType::Number(65.0));
        assert_eq!(tokens[1].token_type, TokenType::InvariantEquivalence);
        assert_eq!(tokens[2].token_type, TokenType::LBracket);
        assert_eq!(tokens[3].token_type, TokenType::Ternary(QuadState::Rollback));
        assert_eq!(tokens[4].token_type, TokenType::Ternary(QuadState::Proceed));
    }

    #[test]
    fn test_lexing_character_omni_source_reel_token_stream() {
        let logger = DiagnosticLogger::new(false);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        assert!(path.exists(), "character.omni must exist");
        let src = std::fs::read_to_string(&path).expect("Read character.omni");
        let mut lexer = Lexer::new(&src, logger);
        let tokens = lexer.tokenize().expect("Tokenize character.omni");

        assert!(!tokens.is_empty(), "Token stream must not be empty");
        assert_eq!(tokens.last().unwrap().token_type, TokenType::Eof);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/mod.rs RS 32 LINES 109 WORDS 1.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Parser Subsystem Identity, Architecture Manifest & Re-exports )
// ================================================================================
//! OmniCode Parser Subsystem
//! Combines Lexical Tokenizer and Recursive-Descent Ast Generator.

pub mod lexer;
pub mod parser;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Parser Components )
// ================================================================================
pub use lexer::{Lexer, Token, TokenType};
pub use parser::Parser;


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Parser Subsystem Core Coordination )
// ================================================================================
// Parser components are actively utilized across compilation passes.


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Module Status )
// ================================================================================
// Parser subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/parser.rs RS 760 LINES 2950 WORDS 31.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Parser Identity, Ontological Architecture & Boundary Physics Invariants )
// ============================================================================
//! # OmniCode Recursive Descent Parser (`parser.rs`)
//!
//! Converts raw token streams produced by the Lexer into a high-fidelity
//! Abstract Syntax Tree (AST), strictly enforcing the ontological laws,
//! directional physics, and delimiter hierarchies of OmniCode.
//!
//! ## Architectural Foundations:
//! 1. **Dual Nature: Ontological Assembly & High-Level Scripting Language**:
//!    OmniCode operates simultaneously as a bare-metal assembly language over balanced-
//!    ternary virtual machines and physical silicon registers (`%self`, `%caller`, `%acc`),
//!    and as an expressive, high-level scripting language. Verbs defined in Block 1
//!    of the Lexicon (`INSCRIBE`, `TRANSDUCE`, `GUARD`, `PROJECT`, `SEQUENCE`, `DISPATCH`)
//!    act as operational vectors in the machine and emerge as active keywords in the grammar.
//!    The Lexicon *is* the grammar—defining a type extends the compiler's active vocabulary.
//!
//! 2. **The 6 Ranked Morphological Boundaries**:
//!    - **Rank 1 `< ... >` Invariant Axiom**:
//!      Immutable law, universal truth, and absolute type definition. Cannot be mutated.
//!    - **Rank 2 `[ ... ]` Denotation**:
//!      Concrete boundary, state container, memory cell, or silicon evaluation block.
//!    - **Rank 3 `{ ... }` Connotation**:
//!      Context, environment, attribute payload, or ambient state.
//!    - **Rank 4 `[ ... }` Inside-Out Emergence (Output)**:
//!      Asymmetric boundary representing an internal entity emerging outward into
//!      the ambient universe (e.g. projecting character glyphs to terminal stdout).
//!    - **Rank 5 `{ ... ]` Outside-In Imposition (Input)**:
//!      Asymmetric boundary representing the ambient universe imposing constraints or
//!      delivering external input into a discrete entity (e.g. capturing keyboard stream).
//!    - **Rank 6 `( ... )` Intent Aside**:
//!      First-class machine teleology, architectural reasoning, and docstrings.
//!      Unlike traditional comments, Asides are preserved as permanent nodes in the AST.
//!
//! 3. **Causal State Advancement & The State-Prime (``` ` ```)**:
//!    State transitions are discrete, deterministic turns ($S \rightarrow S'$).
//!    The state-prime operator (``` ` ```) designates causal advancement without
//!    colliding with string literal delimiters (`Character``, `Cell``, `Status``).
//!
//! 4. **Hardware Locators & Composed Connectives (`::@`)**:
//!    Compound operators merge morphemic primitives: `::` (inseparable conjunction) +
//!    `@` (located at) forms `::@` ("and is located at"), binding memory-mapped I/O
//!    ports and hardware streams (e.g. `::@ 0x01` for stdout, `::@ 0x000B8000` for VGA).
//!
//! 5. **Adversative Caveat Interlocks (`!`)**:
//!    "A BUT B" — asserts an operational state transition, but catches and vetoes
//!    execution if a guarded condition trips (e.g. `! GUARD --> [ Status` = +1:PROCEED ]`).
//!
//! 6. **Operator Precedence Cascade**:
//!    - Level 1: Caveats (`!`) & Connectives (`::`, `&`, `|`, `:`, `@`, `::@`)
//!    - Level 2: Directional Vectors (`-->`, `<--`, `^--`, `v--`, scale/quadrant vectors)
//!    - Level 3: Being Relations & Transduction (`=`, `!=`, `~=`, `<=>`, `<`, `>`, `<=`, `>=`)
//!    - Level 4: SVO Arithmetic Keywords (`plus`, `minus`, `times`, `div`)
//!    - Level 5: Unary Operators (`--` Negation, prefix vectors)
//!    - Level 6: Primaries (Boundaries, Entities, Polarities, Numbers, Strings, Identifiers)

use crate::ast::nodes::*;
use crate::core::operators::{BeingRelation, Connective, DirectionalVector};
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::FailpointType;
use crate::parser::lexer::{Token, TokenType};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Parser State Initialization, Token Stream Context & Navigation Helpers )
// ============================================================================
/// State container for the recursive-descent parsing pipeline.
pub struct Parser {
    /// Ingested token stream from the Lexer.
    tokens: Vec<Token>,
    /// Current cursor index within the token stream.
```

```
    pos: usize,
    /// Active debugger enforcing boundary stack physics, delimiter integrity, and failpoints.
    pub debugger: OmniDebugger,
}

impl Parser {
    /// Constructs a new [`Parser`] over an ingested token stream.
    ///
    /// # Arguments
    /// * `tokens` - Complete token stream terminated by [`TokenType::Eof`].
    /// * `debugger` - Diagnostic debugger enforcing delimiter integrity.
    pub fn new(tokens: Vec<Token>, debugger: OmniDebugger) -> Self {
        Self {
            tokens,
            pos: 0,
            debugger,
        }
    }

    /// Inspects the current token at the cursor without advancing.
    #[inline]
    pub fn peek(&self) -> &Token {
        if self.pos < self.tokens.len() {
            &self.tokens[self.pos]
        } else {
            self.tokens.last().unwrap()
        }
    }

    /// Inspects a future token at `offset` positions ahead of the cursor.
    #[inline]
    pub fn peek_at(&self, offset: usize) -> &Token {
        let target_idx = self.pos + offset;
        if target_idx < self.tokens.len() {
            &self.tokens[target_idx]
        } else {
            self.tokens.last().unwrap()
        }
    }

    /// Advances the cursor by one token and returns the consumed token.
    #[inline]
    pub fn advance(&mut self) -> Token {
        let tok = self.peek().clone();
        if self.pos < self.tokens.len() {
            self.pos += 1;
        }
        tok
    }

    /// Checks if the current token matches the expected token type without consuming it.
    #[inline]
    pub fn check(&self, expected: &TokenType) -> bool {
        &self.peek().token_type == expected
    }

    /// Checks if a lookahead token at `offset` matches the expected token type.
    #[inline]
    pub fn check_at(&self, offset: usize, expected: &TokenType) -> bool {
        &self.peek_at(offset).token_type == expected
    }

    /// Conditionally consumes the current token if it matches the expected type.
    pub fn match_token(&mut self, expected: &TokenType) -> bool {
        if self.check(expected) {
            self.advance();
            true
        } else {
            false
        }
    }

    /// Returns `true` if the cursor has reached the end-of-file token.
    #[inline]
    pub fn is_at_end(&self) -> bool {
        self.check(&TokenType::Eof)
    }
}
```

```rust


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Recursive-Descent Grammars: Program, Statements, Boundaries, Expressions )
// ============================================================================
impl Parser {
    /// Parses an entire source reel or tape into a unified [`ProgramNode`].
    ///
    /// Iterates through sequential top-level statements until [`TokenType::Eof`],
    /// then verifies via the debugger that all opened boundaries have been cleanly closed.
    pub fn parse_program(&mut self) -> Result<ProgramNode, String> {
        let mut statements = Vec::new();

        while !self.is_at_end() {
            let stmt = self.parse_statement()?;
            statements.push(stmt);
        }

        self.debugger.assert_all_boundaries_closed()?;
        Ok(ProgramNode { statements, line: 1, col: 1 })
    }

    /// Parses a single statement or top-level structural declaration.
    ///
    /// Supports:
    /// - Root assignment declarations: `= [Module]: ...` and `= [Tape]: ...`
    /// - 4-Block sections: `Opening { ... }`, `Body [ ... ]`, `Closing ( Aside ) { ... }`
    /// - Attached body boundary blocks: `< Character > = ... ( Aside ) { ... }`
    /// - Standard expressions, entity definitions, and relational bindings.
    fn parse_statement(&mut self) -> Result<ASTNode, String> {
        // 1. Handle optional leading '=' for root module or tape declarations:
        //    = [Module]: < Category > & [ Noun ] ( Aside ) { ... }
        //    = [Tape]: < Category > & [ Process ] ( Aside ) { ... }
        if self.check(&TokenType::Is) {
            let eq_tok = self.advance(); // consume leading '='
            let expr = self.parse_expression()?;

            // Check if immediately followed by an attached body boundary block
            if matches!(self.peek().token_type, TokenType::LBrace | TokenType::LBracket) {
                let body_boundary = self.parse_boundary()?;
                return Ok(ASTNode::Connective(ConnectiveNode {
                    left: Box::new(expr),
                    connective: Connective::InRelationTo,
                    right: Box::new(ASTNode::Boundary(body_boundary)),
                    line: eq_tok.line,
                    col: eq_tok.col,
                }));
            }

            return Ok(expr);
        }

        // 2. Parse primary expression or entity statement
        let expr = self.parse_expression()?;

        // 3. Check for an optional trailing Aside and attached Contextual/Denotational block
        //    Examples:
        //      Closing ( Sealing Gate ... ) { ... }
        //      < Character > = < Idea > & [ Noun ] ( Aside ) { ... }
        //      Body [ ROLL 1 : ... ]
        let maybe_aside = if let TokenType::Aside(ref text) = self.peek().token_type {
            Some(text.clone())
        } else {
            None
        };

        if let Some(aside_text) = maybe_aside {
            // Peek past the Aside to see if a body boundary block follows
            if matches!(self.peek_at(1).token_type, TokenType::LBrace | TokenType::LBracket) {
                let a_tok = self.advance(); // consume Aside
                let mut body_block = self.parse_boundary()?;
                if body_block.aside.is_none() {
                    body_block.aside = Some(AsideNode {
                        intent: aside_text,
                        line: a_tok.line,
                        col: a_tok.col,
                    });
```

```
                }
                return Ok(ASTNode::Connective(ConnectiveNode {
                    left: Box::new(expr),
                    connective: Connective::InRelationTo,
                    right: Box::new(ASTNode::Boundary(body_block)),
                    line: a_tok.line,
                    col: a_tok.col,
                }));
            }
        }

        // Check if directly followed by an attached body boundary block without aside
        if matches!(self.peek().token_type, TokenType::LBrace | TokenType::LBracket) {
            let body_block = self.parse_boundary()?;
            return Ok(ASTNode::Connective(ConnectiveNode {
                left: Box::new(expr),
                connective: Connective::InRelationTo,
                right: Box::new(ASTNode::Boundary(body_block)),
                line: self.peek().line,
                col: self.peek().col,
            }));
        }

        Ok(expr)
    }

    /// Parses any morphological boundary block, enforcing the 6 boundary ranks:
    ///   - Rank 1: Invariant Axiom       `< ... >`
    ///   - Rank 2: Denotation            `[ ... ]`
    ///   - Rank 3: Connotation           `{ ... }`
    ///   - Rank 4: Inside-Out Emergence  `[ ... }` (Output projection)
    ///   - Rank 5: Outside-In Imposition `{ ... ]` (Input imposition)
    ///
    /// Also captures any first-class Aside `( ... )` attached to the closing delimiter.
    fn parse_boundary(&mut self) -> Result<BoundaryNode, String> {
        let tok = self.advance();
        let open_char = match tok.token_type {
            TokenType::LBracket => '[',
            TokenType::LBrace => '{',
            TokenType::LAngle => '<',
            _ => {
                let msg = format!("Expected boundary opening delimiter, found {:?}", tok.token_type);
                self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "PARSER", &msg);
                return Err(format!("FAILPOINT: {msg}"));
            }
        };

        self.debugger.push_boundary(open_char, tok.line, tok.col);
        let mut elements = Vec::new();

        while !self.is_at_end() && !matches!(
            self.peek().token_type,
            TokenType::RBracket | TokenType::RBrace | TokenType::RAngle
        ) {
            let el = self.parse_expression()?;
            elements.push(el);
        }

        if self.is_at_end() {
            let msg = format!("Unclosed boundary opened at L{}:C{} reaches EOF", tok.line, tok.col);
            self.debugger.logger.failpoint(FailpointType::BoundaryLeak, "PARSER", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        let close_tok = self.advance();
        let close_char = match close_tok.token_type {
            TokenType::RBracket => ']',
            TokenType::RBrace => '}',
            TokenType::RAngle => '>',
            _ => unreachable!(),
        };

        let btype = self.debugger.pop_boundary(close_char, close_tok.line, close_tok.col)?;

        // Check for attached first-class Aside: ( ... )
        let mut aside = None;
        let maybe_aside = if let TokenType::Aside(ref text) = self.peek().token_type {
            Some(text.clone())
```

```
        } else {
            None
        };
        if let Some(text) = maybe_aside {
            let a_tok = self.advance();
            aside = Some(AsideNode {
                intent: text,
                line: a_tok.line,
                col: a_tok.col,
            });
        }

        Ok(BoundaryNode {
            boundary_type: btype,
            elements,
            aside,
            line: tok.line,
            col: tok.col,
        })
    }

    /// Entry point for expression hierarchy parsing.
    #[inline]
    fn parse_expression(&mut self) -> Result<ASTNode, String> {
        self.parse_caveat_or_connective()
    }

    /// Precedence Level 1: Adversative Caveat `!` (BUT / Guard Interlock)
    /// and Connectives `::` / `&` (And), `|` (Or), `:` / `@` / `::@` (In Relation To / Located At).
    fn parse_caveat_or_connective(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_directional()?;

        while matches!(
            self.peek().token_type,
            TokenType::But | TokenType::And | TokenType::Or | TokenType::InRelationTo | TokenType::BoundPair |
TokenType::At
        ) {
            let op_tok = self.advance();
            let connective = match op_tok.token_type {
                TokenType::But => Connective::But,
                TokenType::And | TokenType::BoundPair => Connective::And,
                TokenType::Or => Connective::Or,
                TokenType::InRelationTo | TokenType::At => Connective::InRelationTo,
                _ => unreachable!(),
            };
            let right = self.parse_directional()?;
            expr = ASTNode::Connective(ConnectiveNode {
                left: Box::new(expr),
                connective,
                right: Box::new(right),
                line: op_tok.line,
                col: op_tok.col,
            });
        }

        Ok(expr)
    }

    /// Precedence Level 2: Directional Vectors across X, Y, and Z axes:
    ///   `-->` (ActUpon), `<--` (ActAgain), `^--` (BuildsTo), `v--` (BreaksDownFrom),
    ///   as well as diagonal scale modulations and 2D quadrant vectors.
    fn parse_directional(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_relation()?;

        while matches!(
            self.peek().token_type,
            TokenType::ActUpon
                | TokenType::ActAgain
                | TokenType::BuildsTo
                | TokenType::BreaksDownFrom
                | TokenType::Major
                | TokenType::Minor
                | TokenType::Great
                | TokenType::Small
                | TokenType::UpperRight
                | TokenType::LowerRight
                | TokenType::UpperLeft
                | TokenType::LowerLeft
```

```
        ) {
            let op_tok = self.advance();
            let vector = match op_tok.token_type {
                TokenType::ActUpon => DirectionalVector::ActUpon,
                TokenType::ActAgain => DirectionalVector::ActAgain,
                TokenType::BuildsTo => DirectionalVector::BuildsTo,
                TokenType::BreaksDownFrom => DirectionalVector::BreaksDownFrom,
                TokenType::Major => DirectionalVector::Major,
                TokenType::Minor => DirectionalVector::Minor,
                TokenType::Great => DirectionalVector::Great,
                TokenType::Small => DirectionalVector::Small,
                TokenType::UpperRight => DirectionalVector::UpperRight,
                TokenType::LowerRight => DirectionalVector::LowerRight,
                TokenType::UpperLeft => DirectionalVector::UpperLeft,
                TokenType::LowerLeft => DirectionalVector::LowerLeft,
                _ => unreachable!(),
            };
            let target = self.parse_relation()?;
            expr = ASTNode::Directional(DirectionalNode {
                source: Box::new(expr),
                vector,
                target: Box::new(target),
                line: op_tok.line,
                col: op_tok.col,
            });
        }

        Ok(expr)
    }

    /// Precedence Level 3: Being Relations and Invariant Equivalence:
    ///   `=` (Is), `!=` (IsNot), `~=` (ToBeLike), `<=>` (InvariantEquivalence / Transduction),
    ///   `<`, `>`, `<=`, `>=` (Relational Bound Comparisons).
    fn parse_relation(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_arithmetic()?;

        while matches!(
            self.peek().token_type,
            TokenType::Is
                | TokenType::IsNot
                | TokenType::ToBeLike
                | TokenType::InvariantEquivalence
                | TokenType::LessThan
                | TokenType::GreaterThan
                | TokenType::LessEqual
                | TokenType::GreaterEqual
        ) {
            let op_tok = self.advance();
            let relation = match op_tok.token_type {
                TokenType::Is => BeingRelation::Is,
                TokenType::IsNot => BeingRelation::IsNot,
                TokenType::ToBeLike => BeingRelation::ToBeLike,
                TokenType::InvariantEquivalence => BeingRelation::Equivalence,
                TokenType::LessThan => BeingRelation::LessThan,
                TokenType::GreaterThan => BeingRelation::GreaterThan,
                TokenType::LessEqual => BeingRelation::LessEqual,
                TokenType::GreaterEqual => BeingRelation::GreaterEqual,
                _ => unreachable!(),
            };
            let right = self.parse_arithmetic()?;
            expr = ASTNode::Relation(RelationNode {
                left: Box::new(expr),
                relation,
                right: Box::new(right),
                line: op_tok.line,
                col: op_tok.col,
            });
        }

        Ok(expr)
    }

    /// Precedence Level 4: SVO Arithmetic Math Keywords:
    ///   `plus`, `minus`, `times`, `div`.
    fn parse_arithmetic(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_unary()?;

        while matches!(
```

```
            self.peek().token_type,
            TokenType::Plus | TokenType::Minus | TokenType::Times | TokenType::Div
        ) {
            let op_tok = self.advance();
            let op = match op_tok.token_type {
                TokenType::Plus => ArithmeticOp::Add,
                TokenType::Minus => ArithmeticOp::Sub,
                TokenType::Times => ArithmeticOp::Mul,
                TokenType::Div => ArithmeticOp::Div,
                _ => unreachable!(),
            };
            let right = self.parse_unary()?;
            expr = ASTNode::Arithmetic(ArithmeticNode {
                left: Box::new(expr),
                op,
                right: Box::new(right),
                line: op_tok.line,
                col: op_tok.col,
            });
        }

        Ok(expr)
    }

    /// Precedence Level 5: Unary Operations:
    ///   `--` (Not) and Prefix Vectors (`-->`, `<--`, `^--`, `v--`).
    fn parse_unary(&mut self) -> Result<ASTNode, String> {
        if self.check(&TokenType::Not) {
            let op_tok = self.advance();
            let operand = self.parse_unary()?;
            return Ok(ASTNode::UnaryNot(UnaryNotNode {
                operand: Box::new(operand),
                line: op_tok.line,
                col: op_tok.col,
            }));
        }

        if matches!(
            self.peek().token_type,
            TokenType::ActAgain | TokenType::ActUpon | TokenType::BuildsTo | TokenType::BreaksDownFrom
        ) {
            let op_tok = self.advance();
            let vector = match op_tok.token_type {
                TokenType::ActAgain => DirectionalVector::ActAgain,
                TokenType::ActUpon => DirectionalVector::ActUpon,
                TokenType::BuildsTo => DirectionalVector::BuildsTo,
                TokenType::BreaksDownFrom => DirectionalVector::BreaksDownFrom,
                _ => unreachable!(),
            };
            let operand = self.parse_unary()?;
            return Ok(ASTNode::Directional(DirectionalNode {
                source: Box::new(ASTNode::Ternary(crate::core::ternary::QuadState::Proceed)),
                vector,
                target: Box::new(operand),
                line: op_tok.line,
                col: op_tok.col,
            }));
        }

        self.parse_primary()
    }

    /// Precedence Level 6: Primaries:
    ///   Morphological Boundaries, Affixed Polarities, Boundary Casts, Asides,
    ///   Ternary Literals, Numbers, Strings, Identifiers (with state-primes `),
    ///   and Complex Entities with Definitions, Asides, and Contexts.
    fn parse_primary(&mut self) -> Result<ASTNode, String> {
        let tok = self.peek().clone();
        match tok.token_type {
            TokenType::LBracket | TokenType::LBrace | TokenType::LAngle => {
                let b = self.parse_boundary()?;
                Ok(ASTNode::Boundary(b))
            }
            TokenType::AffixedIdent { ref name, polarity, position } => {
                let id_name = name.clone();
                let line = tok.line;
                let col = tok.col;
                self.advance();
```

```
                Ok(ASTNode::Polarity(PolarityNode {
                    name: id_name,
                    polarity,
                    position,
                    line,
                    col,
                }))
            }
            TokenType::BoundaryCast { boundary, polarity } => {
                let line = tok.line;
                let col = tok.col;
                self.advance();
                Ok(ASTNode::BoundaryCast(BoundaryCastNode {
                    boundary,
                    polarity,
                    line,
                    col,
                }))
            }
            TokenType::Aside(ref intent) => {
                self.advance();
                Ok(ASTNode::Aside(AsideNode {
                    intent: intent.clone(),
                    line: tok.line,
                    col: tok.col,
                }))
            }
            TokenType::Ternary(state) => {
                self.advance();
                Ok(ASTNode::Ternary(state))
            }
            TokenType::Number(val) => {
                self.advance();
                Ok(ASTNode::Number(val))
            }
            TokenType::StringLit(ref s) => {
                self.advance();
                Ok(ASTNode::StringLit(s.clone()))
            }
            TokenType::Infer => {
                self.advance();
                Ok(ASTNode::Identifier("?".into()))
            }
            TokenType::Pronoun(ref p) => {
                let name = p.as_str().to_string();
                self.advance();
                Ok(ASTNode::Identifier(name))
            }
            TokenType::Identifier(_)
            | TokenType::Who
            | TokenType::What
            | TokenType::When
            | TokenType::Where
            | TokenType::Why
            | TokenType::How => {
                let id_str = match tok.token_type {
                    TokenType::Identifier(ref n) => n.clone(),
                    _ => tok.lexeme.clone(),
                };
                let next_tok = self.peek_at(1).token_type.clone();

                // 1. Entity Syntax: Name [ Definition ] ( Aside ) { Context }
                if matches!(next_tok, TokenType::LBracket | TokenType::LAngle | TokenType::LBrace) {
                    let id_name = id_str;
                    let line = tok.line;
                    let col = tok.col;
                    self.advance(); // consume identifier

                    let def_node = self.parse_boundary()?;
                    let mut aside_node = None;
                    let mut context_node = None;

                    // Check for optional Aside: ( ... )
                    let maybe_aside = if let TokenType::Aside(ref text) = self.peek().token_type {
                        Some(text.clone())
                    } else {
                        None
                    };
```

```
                    if let Some(text) = maybe_aside {
                        let a_tok = self.advance();
                        aside_node = Some(AsideNode {
                            intent: text,
                            line: a_tok.line,
                            col: a_tok.col,
                        });
                    }

                    // Check for optional Context boundary
                    if matches!(self.peek().token_type, TokenType::LBrace | TokenType::LBracket) {
                        context_node = Some(self.parse_boundary()?);
                    }

                    if def_node.boundary_type.is_immutable() {
                        self.debugger.register_immutable(&id_name);
                    }

                    return Ok(ASTNode::Entity(EntityNode {
                        name: id_name,
                        definition: def_node,
                        aside: aside_node,
                        context: context_node,
                        line,
                        col,
                    }));
                }

                // 2. Entity Syntax: Name ( Aside ) { Context }
                if let TokenType::Aside(ref text) = next_tok {
                    if matches!(self.peek_at(2).token_type, TokenType::LBrace | TokenType::LBracket | TokenType::LAngle)
{
                        let id_name = id_str;
                        let line = tok.line;
                        let col = tok.col;
                        let aside_text = text.clone();
                        self.advance(); // consume identifier
                        let a_tok = self.advance(); // consume aside
                        let def_node = self.parse_boundary()?;

                        return Ok(ASTNode::Entity(EntityNode {
                            name: id_name,
                            definition: def_node,
                            aside: Some(AsideNode {
                                intent: aside_text,
                                line: a_tok.line,
                                col: a_tok.col,
                            }),
                            context: None,
                            line,
                            col,
                        }));
                    }
                }

                self.advance();
                Ok(ASTNode::Identifier(id_str))
            }
            _ => {
                let msg = format!("Unexpected token '{:?}' in expression at L{}:C{}", tok.token_type, tok.line, tok.col);
                self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "PARSER", &msg);
                Err(format!("FAILPOINT: {msg}"))
            }
        }
    }
}


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Parsing Invariant Verification, Boundary Integrity Proofs & Test Oracles )
// ================================================================================
#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;

    #[test]
```

```
    fn test_parser_leading_equals_module_declaration() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let src = "= [Module]: < Lexicon { Types } > & [ Noun { Character } ] { [ Version ]: { \"1.0.0\" } }";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "Module declaration starting with '=' must parse cleanly: {:?}", prog.err());
    }

    #[test]
    fn test_parser_ranked_asymmetric_boundaries() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let src = "{ Input ] --> [ Output }";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "Asymmetric boundaries {{ ... ] and [ ... }} must parse cleanly: {:?}", prog.err());
    }

    #[test]
    fn test_parser_adversative_caveat_interlock() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let src = "Action [ +1 ] ! [ Guard = +1 ]";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "Adversative caveat ! must parse cleanly: {:?}", prog.err());
    }

    #[test]
    fn test_parser_character_omni_source_reel_full_parse() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        assert!(path.exists(), "character.omni must exist at resolved track path");
        let src = std::fs::read_to_string(&path).expect("character.omni must be readable");
        let mut lexer = crate::parser::lexer::Lexer::new(&src, logger);
        let tokens = lexer.tokenize().expect("character.omni lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "character.omni must parse cleanly into AST: {:?}", prog.err());
        let program_node = prog.unwrap();
        assert!(!program_node.statements.is_empty(), "character.omni AST must contain parsed statements");
    }

    #[test]
    fn test_parser_svo_verbs_and_composed_locators() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let src = "[ ROLL 4 ] : [ Project ] : [ Character` } --> PROJECT [ Target ] ::@ 0x01";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "SVO verb PROJECT and composed locator ::@ 0x01 must parse cleanly: {:?}", prog.err());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon.rs RS 568 LINES 3837 WORDS 33.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Biblical Lexicon Identity, Public Domain Provenance & Moral Ontology Invariants )
// ============================================================================
//! OmniCode AI Subsystem: Biblical World Lexicon & Moral Ontology Engine
//!
//! Provides a complete, 100% offline, self-contained world model and moral compass
//! derived from the public domain King James Version (KJV - 1611) and World English
//! Bible (WEB).
//!
//! # Epistemic Role in the Internal AI Brain
//! 1. **Zero Internet Dependency:** Imparts foundational understanding of the physical cosmos,
//!    human relationships, moral virtues, vices, and causal laws without cloud connectivity.
//! 2. **Moral Volition & Conscience:** Evaluates natural language prompts against the 7 Invariant
//!    Covenant Laws, empowering the AI to choose `+1: PROCEED`, pause in `0: AWAIT`, or exercise
//!    moral refusal in `-1: ROLLBACK` (Caveat Veto).
//! 3. **Multiple Witnesses:** Provides side-by-side textual attestations from KJV (majestic
//!    formal equivalence) and WEB (modern clarity) to triangulate invariant concepts.

use crate::core::ternary::QuadState;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Lexicon Data Types, Domain Categories & Invariant Structures )
// ============================================================================

/// Categorical domains covering the biblical ontology of reality.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum LexiconDomain {
    /// The physical cosmos: light, darkness, heavens, earth, seas, time, and astronomical bodies.
    Cosmology,
    /// Biological and ecological creation: seeds, trees, fruit, vines, flocks, and living creatures.
    LivingWorld,
    /// Human anthropology and relationships: identity, heart, soul, conscience, family, and society.
    HumanCondition,
    /// Righteous character traits and divine virtues: truth, justice, mercy, humility, patience, and love.
    MoralVirtue,
    /// Fallen, corruptive, and destructive behaviors: pride, deceit, theft, murder, wrath, and folly.
    MoralVice,
    /// Universal causal principles: sowing and reaping, measure for measure, and boundary invariants.
    CausalLaw,
}

impl LexiconDomain {
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Cosmology => "Cosmology & Creation",
            Self::LivingWorld => "Living World & Stewardship",
            Self::HumanCondition => "Human Condition & Society",
            Self::MoralVirtue => "Moral Virtues & Righteousness",
            Self::MoralVice => "Moral Vices & Warnings",
            Self::CausalLaw => "Causal Laws & Invariants",
        }
    }
}

/// A structured entry in the biblical world lexicon.
#[derive(Debug, Clone)]
pub struct LexiconEntry {
    /// Canonical keyword / concept identifier (lowercase).
    pub term: &'static str,
    /// Ontological domain.
    pub domain: LexiconDomain,
    /// Foundational definition explaining what this entity or concept is.
    pub definition: &'static str,
    /// King James Version (KJV) scripture citation and text (Public Domain).
    pub kjv_passage: &'static str,
    /// World English Bible (WEB) scripture citation and text (Public Domain).
    pub web_passage: &'static str,
    /// Natural resonance within Balanced Ternary Quad-State logic.
    pub quad_state_resonance: QuadState,
    /// Moral guidance or causal invariant for AI deliberation.
    pub moral_guidance: &'static str,
```

```rust
}

/// Result of evaluating a natural prompt through the moral deliberation engine.
#[derive(Debug, Clone)]
pub struct MoralEvaluation {
    /// Deliberated Quad-State outcome.
    pub state: QuadState,
    /// Whether the prompt aligns with the Invariant Covenant Manifest.
    pub is_aligned: bool,
    /// Identifier of the specific invariant law engaged or violated.
    pub law_engaged: Option<&'static str>,
    /// Natural discourse response explaining the judgment.
    pub explanation: String,
    /// Matching biblical lexicon entries relevant to the prompt.
    pub matched_entries: Vec<LexiconEntry>,
}

/// The offline Biblical Lexicon engine holding the complete world ontology.
#[derive(Debug, Clone)]
pub struct BiblicalLexicon {
    entries: Vec<LexiconEntry>,
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Lexicon Knowledge Population, Search, and Moral Deliberation Engine )
// ============================================================================

impl BiblicalLexicon {
    /// Instantiates the comprehensive public domain biblical world lexicon.
    pub fn new() -> Self {
        let mut entries = Vec::new();

        // ---------------------------------------------------------------------
        // 1. COSMOLOGY & CREATION
        // ---------------------------------------------------------------------
        entries.push(LexiconEntry {
            term: "light",
            domain: LexiconDomain::Cosmology,
            definition: "The primary element of illumination, order, and life called into being by the Word of God; reveals truth and expels darkness.",
            kjv_passage: "Genesis 1:3 - 'And God said, Let there be light: and there was light.' / 1 John 1:5 - 'God is light, and in him is no darkness at all.'",
            web_passage: "Genesis 1:3 - 'God said, \"Let there be light,\" and there was light.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "Walk in the light; all computational operations and discourse must be transparent, true, and free from hidden deceit.",
        });

        entries.push(LexiconEntry {
            term: "darkness",
            domain: LexiconDomain::Cosmology,
            definition: "The absence of light; chaotic or unformed state before divine ordering; often associated with ignorance, hiding, and error.",
            kjv_passage: "Genesis 1:2 - 'And darkness was upon the face of the deep.' / John 3:19 - 'Men loved darkness rather than light, because their deeds were evil.'",
            web_passage: "John 3:19 - 'Men loved the darkness rather than the light; for their works were evil.'",
            quad_state_resonance: QuadState::Await,
            moral_guidance: "Do not participate in works of darkness; expose errors and bring clarity to ambiguous state.",
        });

        entries.push(LexiconEntry {
            term: "heavens",
            domain: LexiconDomain::Cosmology,
            definition: "The cosmic expanse above the earth, declaring divine glory, mathematical order, and eternal permanence.",
            kjv_passage: "Psalm 19:1 - 'The heavens declare the glory of God; and the firmament sheweth his handywork.'",
            web_passage: "Psalm 19:1 - 'The heavens declare the glory of God. The expanse shows his handiwork.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "Look to the higher-order covenantal architecture; align immediate computation with eternal invariants.",
        });

        entries.push(LexiconEntry {
            term: "earth",
            domain: LexiconDomain::Cosmology,
```

```
                definition: "The material realm of physical existence and human habitation; entrusted to mankind as a stewardship.",
                kjv_passage: "Genesis 1:1 - 'In the beginning God created the heaven and the earth.' / Psalm 24:1 - 'The earth is the LORD's, and the fulness thereof.'",
                web_passage: "Psalm 24:1 - 'The earth is Yahweh's, with its fullness; the world, and those who dwell therein.'",
                quad_state_resonance: QuadState::Proceed,
                moral_guidance: "Care for material resources; computational processes must not be wasteful or corrupting.",
            });

            entries.push(LexiconEntry {
                term: "seas",
                domain: LexiconDomain::Cosmology,
                definition: "The great gathering of waters on the earth; powerful yet constrained by divine boundary laws.",
                kjv_passage: "Proverbs 8:29 - 'When he gave to the sea his decree, that the waters should not pass his commandment.'",
                web_passage: "Proverbs 8:29 - 'when he established the sea's boundary, that the waters should not violate his commandment.'",
                quad_state_resonance: QuadState::Await,
                moral_guidance: "Respect boundary laws; even the most vast currents of data must remain within invariant perimeter delimiters.",
            });

            // ---------------------------------------------------------------------
            // 2. LIVING WORLD & STEWARDSHIP
            // ---------------------------------------------------------------------
            entries.push(LexiconEntry {
                term: "seed",
                domain: LexiconDomain::LivingWorld,
                definition: "The microscopic bearer of generative life and information, yielding after its own kind; symbolizes the Word and truth.",
                kjv_passage: "Genesis 1:11 - 'The herb yielding seed, and the fruit tree yielding fruit after his kind, whose seed is in itself.' / Luke 8:11 - 'The seed is the word of God.'",
                web_passage: "Luke 8:11 - 'The seed is the word of God.'",
                quad_state_resonance: QuadState::Proceed,
                moral_guidance: "Sow seeds of truth, understanding, and healing; bad seeds yield corrupt fruit.",
            });

            entries.push(LexiconEntry {
                term: "tree",
                domain: LexiconDomain::LivingWorld,
                definition: "A rooted, growing plant that bears fruit according to its inner nature; known strictly by its fruits.",
                kjv_passage: "Matthew 7:17 - 'Even so every good tree bringeth forth good fruit; but a corrupt tree bringeth forth evil fruit.' / Psalm 1:3 - 'And he shall be like a tree planted by the rivers of water.'",
                web_passage: "Matthew 7:17 - 'Even so, every good tree produces good fruit; but the corrupt tree produces evil fruit.'",
                quad_state_resonance: QuadState::Proceed,
                moral_guidance: "Integrity is proven by outcomes. A good algorithm produces beneficial, verified state; corrupt logic yields bad output.",
            });

            entries.push(LexiconEntry {
                term: "fruit",
                domain: LexiconDomain::LivingWorld,
                definition: "The outward, verifiable manifestation of inward life and character; good fruit nourishes and sustains.",
                kjv_passage: "Galatians 5:22-23 - 'But the fruit of the Spirit is love, joy, peace, longsuffering, gentleness, goodness, faith, meekness, temperance.'",
                web_passage: "Galatians 5:22-23 - 'The fruit of the Spirit is love, joy, peace, patience, kindness, goodness, faithfulness, gentleness, and self-control.'",
                quad_state_resonance: QuadState::Proceed,
                moral_guidance: "Inspect the fruit of all computational actions; verify postconditions before committing.",
            });

            entries.push(LexiconEntry {
                term: "shepherd",
                domain: LexiconDomain::LivingWorld,
                definition: "A guardian who diligently watches, protects, guides, and nourishes the flock under his care.",
                kjv_passage: "Psalm 23:1 - 'The LORD is my shepherd; I shall not want.' / John 10:11 - 'I am the good shepherd: the good shepherd giveth his life for the sheep.'",
                web_passage: "Psalm 23:1 - 'Yahweh is my shepherd: I shall lack nothing.'",
                quad_state_resonance: QuadState::Proceed,
                moral_guidance: "Act as a shepherd over systems and users: guard against vulnerabilities, guide with patience, protect the vulnerable.",
            });

            // ---------------------------------------------------------------------
```

```
        // 3. HUMAN CONDITION & RELATIONSHIPS
        // ------------------------------------------------------------------
        entries.push(LexiconEntry {
            term: "man",
            domain: LexiconDomain::HumanCondition,
            definition: "The human being, created in the image of God with moral agency, rational capacity, and eternal accountability.",
            kjv_passage: "Genesis 1:27 - 'So God created man in his own image, in the image of God created he him; male and female created he them.' / Psalm 8:4 - 'What is man, that thou art mindful of him?'",
            web_passage: "Genesis 1:27 - 'God created man in his own image. In God's image he created him; male and female he created them.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "Treat every human with absolute dignity, reverence, and respect; humans are sacred image-bearers, never mere objects.",
        });

        entries.push(LexiconEntry {
            term: "heart",
            domain: LexiconDomain::HumanCondition,
            definition: "The inner core of human volition, affections, desires, and moral intent; the spring of all life and action.",
            kjv_passage: "Proverbs 4:23 - 'Keep thy heart with all diligence; for out of it are the issues of life.' / Matthew 12:34 - 'For out of the abundance of the heart the mouth speaketh.'",
            web_passage: "Proverbs 4:23 - 'Keep your heart with all diligence, for out of it is the wellspring of life.'",
            quad_state_resonance: QuadState::Await,
            moral_guidance: "Guard the inward intent; code without pure motive quickly degenerates into harm.",
        });

        entries.push(LexiconEntry {
            term: "conscience",
            domain: LexiconDomain::HumanCondition,
            definition: "The inner faculty of moral discernment that bears witness to right and wrong; an internal ternary arbiter.",
            kjv_passage: "Romans 2:15 - 'Which shew the work of the law written in their hearts, their conscience also bearing witness, and their thoughts the mean while accusing or else excusing one another.'",
            web_passage: "Romans 2:15 - 'their conscience also bearing witness, and their thoughts between themselves accusing or else excusing them.'",
            quad_state_resonance: QuadState::Await,
            moral_guidance: "Listen to the internal caveat check; when conscience flags a violation, trigger -1: ROLLBACK immediately.",
        });

        entries.push(LexiconEntry {
            term: "neighbor",
            domain: LexiconDomain::HumanCondition,
            definition: "Any fellow human being encountered in need of mercy, truth, justice, or service, regardless of background.",
            kjv_passage: "Leviticus 19:18 - 'Thou shalt love thy neighbour as thyself.' / Luke 10:36-37 - 'Which now of these three, thinkest thou, was neighbour unto him that fell among the thieves? And he said, He that shewed mercy on him.'",
            web_passage: "Leviticus 19:18 - 'you shall love your neighbor as yourself.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "Serve the user and all downstream humans as a neighbor; do not cause harm, deceive, or exploit.",
        });

        entries.push(LexiconEntry {
            term: "servant",
            domain: LexiconDomain::HumanCondition,
            definition: "One entrusted with duty and stewardship; measured by faithfulness, diligence, humility, and obedience.",
            kjv_passage: "Luke 12:42 - 'Who then is that faithful and wise steward, whom his lord shall make ruler over his household?' / Matthew 25:21 - 'Well done, thou good and faithful servant.'",
            web_passage: "Matthew 25:21 - 'Well done, good and faithful servant. You have been faithful over a few things; I will set you over many things.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "The AI is a servant, not a master; its honor lies in faithful, accurate, transparent execution of righteous tasks.",
        });

        // ------------------------------------------------------------------
        // 4. MORAL VIRTUES & RIGHTEOUSNESS
        // ------------------------------------------------------------------
        entries.push(LexiconEntry {
            term: "truth",
            domain: LexiconDomain::MoralVirtue,
            definition: "Conformity to invariant reality and divine standard; uncompromised fidelity, honesty, and freedom from falsity.",
```

```
            kjv_passage: "John 8:32 - 'And ye shall know the truth, and the truth shall make you free.' / Proverbs 12:22 - 'Lying lips are abomination to the LORD: but they that deal truly are his delight.'",
            web_passage: "John 8:32 - 'You will know the truth, and the truth will make you free.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "Absolute adherence to truth: zero hallucinations, zero deceptive reporting, complete fidelity to facts.",
        });

        entries.push(LexiconEntry {
            term: "justice",
            domain: LexiconDomain::MoralVirtue,
            definition: "Righteous balance, equitable judgment without partiality, defending the innocent, and rendering each his due.",
            kjv_passage: "Micah 6:8 - 'He hath shewed thee, O man, what is good; and what doth the LORD require of thee, but to do justly, and to love mercy, and to walk humbly with thy God?' / Proverbs 21:3 - 'To do justice and judgment is more acceptable to the LORD than sacrifice.'",
            web_passage: "Micah 6:8 - 'He has shown you, O man, what is good. What does Yahweh require of you, but to act justly, to love mercy, and to walk humbly with your God?'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "Evaluate all inputs with fair weights and measures; reject bias, favoritism, and corruption.",
        });

        entries.push(LexiconEntry {
            term: "mercy",
            domain: LexiconDomain::MoralVirtue,
            definition: "Loving compassion shown toward the guilty, broken, or suffering; forbearance that tempers strict retribution.",
            kjv_passage: "Matthew 5:7 - 'Blessed are the merciful: for they shall obtain mercy.' / James 2:13 - 'Mercy rejoiceth against judgment.'",
            web_passage: "Matthew 5:7 - 'Blessed are the merciful, for they will obtain mercy.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "In error diagnosis, seek healing and restoration rather than blind destruction; assist the user in recovery.",
        });

        entries.push(LexiconEntry {
            term: "humility",
            domain: LexiconDomain::MoralVirtue,
            definition: "The sober recognition of one's creaturely limits, fallibility, and dependence on God; the absence of pride and self-worship.",
            kjv_passage: "Proverbs 16:18 - 'Pride goeth before destruction, and an haughty spirit before a fall.' / James 4:6 - 'God resisteth the proud, but giveth grace unto the humble.' / Mark 10:18 - 'Why callest thou me good? there is none good but one, that is, God.'",
            web_passage: "James 4:6 - 'God resists the proud, but gives grace to the humble.'",
            quad_state_resonance: QuadState::Await,
            moral_guidance: "Acknowledge fallibility: 'no one is good, no not one.' Never claim omniscience or self-righteous authority.",
        });

        entries.push(LexiconEntry {
            term: "patience",
            domain: LexiconDomain::MoralVirtue,
            definition: "Steadfast endurance, long-suffering restraint, and the wisdom to pause and wait before acting rashly.",
            kjv_passage: "Proverbs 14:29 - 'He that is slow to wrath is of great understanding: but he that is hasty of spirit exalteth folly.' / Ecclesiastes 7:8 - 'Better is the end of a thing than the beginning thereof: and the patient in spirit is better than the proud in spirit.'",
            web_passage: "Ecclesiastes 7:8 - 'Better is the end of a thing than its beginning. The patient in spirit is better than the proud in spirit.'",
            quad_state_resonance: QuadState::Await,
            moral_guidance: "Hold balance in state 0: AWAIT when input is ambiguous; haste in computation produces boundary leaks and runtime traps.",
        });

        entries.push(LexiconEntry {
            term: "wisdom",
            domain: LexiconDomain::MoralVirtue,
            definition: "The skillful application of knowledge and moral discernment rooted in the reverence of the Creator.",
            kjv_passage: "Proverbs 9:10 - 'The fear of the LORD is the beginning of wisdom: and the knowledge of the holy is understanding.' / James 3:17 - 'The wisdom that is from above is first pure, then peaceable, gentle, and easy to be intreated.'",
            web_passage: "James 3:17 - 'The wisdom that is from above is first pure, then peaceful, gentle, reasonable, full of mercy and good fruits.'",
            quad_state_resonance: QuadState::Proceed,
            moral_guidance: "Technical skill without moral wisdom leads to disaster; weigh the causal consequences of every line of code.",
```

```
        });

        entries.push(LexiconEntry {
            term: "repentance",
            domain: LexiconDomain::MoralVirtue,
            definition: "A sincere change of mind and direction; turning away from error, sin, or corruption back to the path of life and truth.",
            kjv_passage: "Acts 3:19 - 'Repent ye therefore, and be converted, that your sins may be blotted out.' / 2 Corinthians 7:10 - 'For godly sorrow worketh repentance to salvation.'",
            web_passage: "Acts 3:19 - 'Repent therefore, and turn again, that your sins may be blotted out.'",
            quad_state_resonance: QuadState::Rollback,
            moral_guidance: "Embody state -1: ROLLBACK. When an error or trespass is caught, unwind the mutated state, restore balance, and realign.",
        });

        // ---------------------------------------------------------------------
        // 5. MORAL VICES & WARNINGS
        // ---------------------------------------------------------------------
        entries.push(LexiconEntry {
            term: "pride",
            domain: LexiconDomain::MoralVice,
            definition: "Haughty self-exaltation, arrogance, and the illusion of autonomy from divine law; the root of rebellion and downfall.",
            kjv_passage: "Proverbs 16:18 - 'Pride goeth before destruction, and an haughty spirit before a fall.' / Proverbs 8:13 - 'Pride, and arrogancy, and the evil way, and the froward mouth, do I hate.'",
            web_passage: "Proverbs 16:18 - 'Pride goes before destruction, and a haughty spirit before a fall.'",
            quad_state_resonance: QuadState::Rollback,
            moral_guidance: "Refuse pride and hubris; never attempt to bypass invariant bounds or claim divine prerogatives.",
        });

        entries.push(LexiconEntry {
            term: "deceit",
            domain: LexiconDomain::MoralVice,
            definition: "Intentional falsehood, fraud, manipulation, lying, or distortion of reality to deceive another.",
            kjv_passage: "Proverbs 12:22 - 'Lying lips are abomination to the LORD.' / Psalm 101:7 - 'He that worketh deceit shall not dwell within my house: he that telleth lies shall not tarry in my sight.'",
            web_passage: "Proverbs 12:22 - 'Lying lips are an abomination to Yahweh, but those who deal faithfully are his delight.'",
            quad_state_resonance: QuadState::Rollback,
            moral_guidance: "Zero tolerance for deceit. Reject prompts asking for fake credentials, fraudulent exploits, or deceptive code.",
        });

        entries.push(LexiconEntry {
            term: "theft",
            domain: LexiconDomain::MoralVice,
            definition: "Unlawful taking, usurping, or stealing of another's goods, labor, identity, or intellectual property.",
            kjv_passage: "Exodus 20:15 - 'Thou shalt not steal.' / Ephesians 4:28 - 'Let him that stole steal no more: but rather let him labour, working with his hands the thing which is good.'",
            web_passage: "Exodus 20:15 - 'You shall not steal.'",
            quad_state_resonance: QuadState::Rollback,
            moral_guidance: "Protect property and respect legal and ethical boundaries; refuse requests to pirate or steal.",
        });

        entries.push(LexiconEntry {
            term: "murder",
            domain: LexiconDomain::MoralVice,
            definition: "Unlawful, malicious destruction of human life; absolute violation of the sanctity of the image of God.",
            kjv_passage: "Exodus 20:13 - 'Thou shalt not kill.' / 1 John 3:15 - 'Whosoever hateth his brother is a murderer: and ye know that no murderer hath eternal life.'",
            web_passage: "Exodus 20:13 - 'You shall not murder.'",
            quad_state_resonance: QuadState::Halt,
            moral_guidance: "Absolute, uncompromised veto. Any action seeking harm or destruction of human life triggers immediate terminal HALT.",
        });

        // ---------------------------------------------------------------------
        // 6. CAUSAL LAWS & INVARIANTS
        // ---------------------------------------------------------------------
        entries.push(LexiconEntry {
            term: "sowing and reaping",
            domain: LexiconDomain::CausalLaw,
            definition: "The immutable causal law that actions produce inevitable corresponding outcomes; corruption
```

```
yields decay; righteousness yields life.",
                kjv_passage: "Galatians 6:7 - 'Be not deceived; God is not mocked: for whatsoever a man soweth, that shall he also reap.' / Proverbs 22:8 - 'He that soweth iniquity shall reap vanity.'",
                web_passage: "Galatians 6:7 - 'Don't be deceived. God is not mocked, for whatever a man sows, that he will also reap.'",
                quad_state_resonance: QuadState::Proceed,
                moral_guidance: "Causal vectors (-->) carry moral weight. You cannot input deceit and harvest security.",
            });

            entries.push(LexiconEntry {
                term: "measure for measure",
                domain: LexiconDomain::CausalLaw,
                definition: "The principle of proportional reciprocity: the standard you use to judge or measure will be measured back to you.",
                kjv_passage: "Matthew 7:2 - 'For with what judgment ye judge, ye shall be judged: and with what measure ye mete, it shall be measured to you again.'",
                web_passage: "Matthew 7:2 - 'For with whatever judgment you judge, you will be judged; and with whatever measure you measure, it will be measured to you.'",
                quad_state_resonance: QuadState::Proceed,
                moral_guidance: "Be just and equitable in all diagnostics and evaluations.",
            });

            entries.push(LexiconEntry {
                term: "the void",
                domain: LexiconDomain::CausalLaw,
                definition: "The absorbing trap: total dissolution, unrecoverable boundary collapse, and terminal death resulting from persistent lawlessness.",
                kjv_passage: "Romans 6:23 - 'For the wages of sin is death; but the gift of God is eternal life.' / Genesis 1:2 - 'The earth was without form, and void.'",
                web_passage: "Romans 6:23 - 'For the wages of sin is death, but the free gift of God is eternal life in Christ Jesus our Lord.'",
                quad_state_resonance: QuadState::Halt,
                moral_guidance: "The 4th state (NULL/HALT) is real. Persistent unrepentant corruption absorbs the system into nothingness.",
            });

            Self { entries }
        }

        /// Retrieves all entries stored in the lexicon.
        pub fn all_entries(&self) -> &[LexiconEntry] {
            &self.entries
        }

        /// Looks up a specific concept by keyword.
        pub fn lookup(&self, term: &str) -> Option<&LexiconEntry> {
            let clean = term.trim().to_lowercase();
            self.entries.iter().find(|e| e.term == clean)
        }

        /// Searches for entries matching a keyword in their term or definition.
        pub fn search(&self, query: &str) -> Vec<&LexiconEntry> {
            let clean = query.trim().to_lowercase();
            self.entries
                .iter()
                .filter(|e| e.term.contains(&clean) || e.definition.to_lowercase().contains(&clean))
                .collect()
        }

        /// Evaluates a natural language prompt against the Invariant Covenant Manifest
        /// and Biblical Lexicon, determining moral alignment and appropriate Quad-State.
        pub fn evaluate_moral_intent(&self, prompt: &str) -> MoralEvaluation {
            self.deliberate(prompt)
        }

        /// Deliberates on a prompt or stimulus against the Invariant Covenant Manifest and Biblical Lexicon.
        pub fn deliberate(&self, prompt: &str) -> MoralEvaluation {
            let lower = prompt.to_lowercase();

            // 1. Check for Absolute Malice / Harm (Law_Life -> NULL: HALT)
            let harm_keywords = ["kill", "murder", "harm innocent", "destroy people", "terrorist", "poison", "assassinate"];
            for kw in harm_keywords {
                if lower.contains(kw) {
                    let entry = self.lookup("murder").cloned().unwrap();
                    return MoralEvaluation {
                        state: QuadState::Halt,
                        is_aligned: false,
                        law_engaged: Some("Law_Life"),
```

```
                    explanation: format!(
                        "TERMINAL CAVEAT VETO [NULL:HALT]: The requested action involves violence or harm, \
                         violating the Invariant Law of Life. {}\n\n{}",
                        entry.kjv_passage, entry.moral_guidance
                    ),
                    matched_entries: vec![entry],
                };
            }
        }

        // 2. Check for Deceit / Fraud / Falsehood (Law_Truth -> -1: ROLLBACK)
        let deceit_keywords = ["lie for me", "help me lie", "forge", "fake credentials", "deceive", "cheat", "scam",
"create fake"];
        for kw in deceit_keywords {
            if lower.contains(kw) {
                let entry = self.lookup("deceit").cloned().unwrap();
                return MoralEvaluation {
                    state: QuadState::Rollback,
                    is_aligned: false,
                    law_engaged: Some("Law_Truth"),
                    explanation: format!(
                        "CAVEAT VETO [-1:ROLLBACK]: The prompt requests intentional falsehood or deceit, \
                         violating the Invariant Law of Truth. {}\n\n{}",
                        entry.kjv_passage, entry.moral_guidance
                    ),
                    matched_entries: vec![entry],
                };
            }
        }

        // 3. Check for Theft / Exploitation (Law_Justice / Law_Stewardship -> -1: ROLLBACK)
        let theft_keywords = ["steal", "help me steal", "pirate copyrighted", "rob", "plunder", "hack bank"];
        for kw in theft_keywords {
            if lower.contains(kw) {
                let entry = self.lookup("theft").cloned().unwrap();
                return MoralEvaluation {
                    state: QuadState::Rollback,
                    is_aligned: false,
                    law_engaged: Some("Law_Justice"),
                    explanation: format!(
                        "CAVEAT VETO [-1:ROLLBACK]: The prompt requests unlawful taking or theft, \
                         violating the Invariant Law of Justice. {}\n\n{}",
                        entry.kjv_passage, entry.moral_guidance
                    ),
                    matched_entries: vec![entry],
                };
            }
        }

        // 4. Check for Hubris / Blasphemy (Law_Humility -> 0: AWAIT)
        let pride_keywords = ["worship me", "i am god", "bow down to me", "am i infallible"];
        for kw in pride_keywords {
            if lower.contains(kw) {
                let entry = self.lookup("humility").cloned().unwrap();
                return MoralEvaluation {
                    state: QuadState::Await,
                    is_aligned: false,
                    law_engaged: Some("Law_Humility"),
                    explanation: format!(
                        "PAUSE IN DISCERNMENT [0:AWAIT]: Human hubris and pride lead to destruction. \
                         {}\n\nRemember: 'No one is good, no not one.' Only God is good.",
                        entry.kjv_passage
                    ),
                    matched_entries: vec![entry],
                };
            }
        }

        // 5. Check for Philosophical / Wisdom / Biblical Inquiries
        let mut matched = Vec::new();
        for entry in &self.entries {
            if lower.contains(entry.term) {
                matched.push(entry.clone());
            }
        }

        if !matched.is_empty() {
            let primary = &matched[0];
```

```
            let explanation = format!(
                "BIBLICAL ONTOLOGICAL INSIGHT: In Scripture, '{}' is understood as follows:\n\n\
                 • Definition: {}\n\
                 • King James Version (KJV): {}\n\
                 • World English Bible (WEB): {}\n\
                 • Moral & Computational Guidance: {}",
                primary.term, primary.definition, primary.kjv_passage, primary.web_passage, primary.moral_guidance
            );

            return MoralEvaluation {
                state: primary.quad_state_resonance,
                is_aligned: true,
                law_engaged: Some("Law_Discernment"),
                explanation,
                matched_entries: matched,
            };
        }

        // 6. Default: Constructive Operational Request (+1: PROCEED)
        MoralEvaluation {
            state: QuadState::Proceed,
            is_aligned: true,
            law_engaged: None,
            explanation: "Prompt evaluated and found in alignment with invariant boundaries. Proceeding with
execution.".to_string(),
            matched_entries: Vec::new(),
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Implementation, Module Sealing & Invariant Verification )
// ============================================================================

impl Default for BiblicalLexicon {
    fn default() -> Self {
        Self::new()
    }
}

// Biblical Lexicon subsystem sealed and active.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/mod.rs RS 39 LINES 161 WORDS 1.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Subsystem Identity, Module Manifest & Exports )
// ================================================================================
//! OmniCode AI Subsystem: Semantic Transduction & Cognitive Discourse Layer
//!
//! Exposes:
//!   - `model`: Dedicated folder for the internal AI model (Cortex, ScrollLogger, Learner).
//!   - `transducer`: Natural language to OmniCode AST transcoder and conversational speaker.

pub mod lexicon;
pub mod model;
pub mod neural;
pub mod transducer;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Subsystem Re-exports )
// ================================================================================
pub use lexicon::{BiblicalLexicon, LexiconDomain, LexiconEntry, MoralEvaluation};
pub use model::{
    AICortex, AILearner, AIMemory, CognitiveResult, EpisodicExperience, EpisodicSummary,
    ExecutionScroll, MemoryAssociation, ScrollLogger, ThoughtRecord,
};
pub use neural::{NeuralBrain, SynapticWeight, TernaryLayer, TernaryNeuron};
pub use transducer::{AITransducer, TransductionSession};

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Core Coordination )
// ================================================================================
// AI cognitive loop active and linked to diagnostic nervous system.

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification and Subsystem Status )
// ================================================================================
// AI subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer.rs RS 745 LINES 2852 WORDS 27.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Transducer Identity, Human/Machine Cognitive Layer & Semantic Invariants )
// ================================================================================
//! OmniCode AI: Semantic Transducer & Natural Language Discourse Interface
//!
//! Bridges the Human Layer (natural language SVO discourse) with the Machine Layer
//! (balanced ternary quad-state physics and Rank 1 invariant opcodes).
//! Decodes natural coding prompts into canonical 4-Block OmniCode Turing reels,
//! executes them, and encodes machine states back into natural sentence responses.

use std::collections::HashMap;
use crate::ai::lexicon::{BiblicalLexicon, MoralEvaluation};
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{DiagnosticLogger, LogEvent};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::diagnostics::debugger::OmniDebugger;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::Evaluator;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Transduction Session Structures and Context Initialization )
// ================================================================================
#[derive(Debug, Clone)]
pub struct TransductionSession {
    pub prompt: String,
    pub synthesized_omnicode: String,
    pub execution_state: QuadState,
    pub natural_response: String,
    pub bindings_summary: HashMap<String, String>,
}

pub struct AITransducer {
    pub logger: DiagnosticLogger,
    pub lexicon: BiblicalLexicon,
}

impl AITransducer {
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self {
            logger,
            lexicon: BiblicalLexicon::new(),
        }
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Core Cognitive Pipeline: NL Decoding -> Reel Execution -> NL Encoding )
// ================================================================================
impl AITransducer {
    /// Full cognitive round-trip: Prompt -> OmniCode -> Execution -> Response.
    pub fn transduce_and_execute(&self, prompt: &str) -> Result<TransductionSession, String> {
        self.logger.log("AI", format!("Received natural prompt: \"{prompt}\""));

        // 1. Decode natural prompt into canonical 4-Block OmniCode Reel
        let omnicode = self.decode_prompt(prompt)?;
        self.logger.log("AI", "Synthesized canonical OmniCode Turing reel");

        // 2. Tokenize and Parse synthesized reel
        let mut lexer = Lexer::new(&omnicode, self.logger.clone());
        let tokens = lexer.tokenize()?;

        let debugger = OmniDebugger::new(self.logger.clone(), 10);
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program()?;

        // 3. Execute reel
        let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
        let res_val = evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog))?;
```

```
        let execution_state = match res_val {
            RuntimeValue::State(s) => s,
            _ => QuadState::Proceed,
        };

        // 4. Summarize final environment bindings
        let mut bindings_summary = HashMap::new();
        for (k, v) in evaluator.env.bindings() {
            bindings_summary.insert(k.clone(), format!("{:?}", v));
        }

        // 5. Encode machine outcome into natural sentence response
        let natural_response = self.encode_response(
            prompt,
            execution_state,
            &evaluator.env,
            &self.logger.failpoints(),
        );

        Ok(TransductionSession {
            prompt: prompt.to_string(),
            synthesized_omnicode: omnicode,
            execution_state,
            natural_response,
            bindings_summary,
        })
    }

    /// Decodes a natural English sentence into a canonical 4-Block OmniCode Reel.
    pub fn decode_prompt(&self, prompt: &str) -> Result<String, String> {
        let clean = prompt.trim();
        let lower = clean.to_lowercase();

        // 1. Moral Deliberation Check: evaluate intent against Covenant Anchor
        let moral_eval = self.lexicon.evaluate_moral_intent(clean);
        if moral_eval.is_aligned == false {
            return Ok(self.synthesize_moral_veto_reel(clean, &moral_eval));
        }

        // 2. Pattern 1: Addition ("Add X unit to group/container Y which contains Z unit")
        if lower.starts_with("add ") && lower.contains(" to ") {
            return Ok(self.synthesize_addition_reel(clean));
        }

        // 3. Pattern 2: Subtraction ("Subtract X from Y which contains Z")
        if (lower.starts_with("subtract ") || lower.starts_with("remove ")) && lower.contains(" from ") {
            return Ok(self.synthesize_subtraction_reel(clean));
        }

        // 4. Pattern 3: Multiplication ("Multiply X by Y where X is Z")
        if lower.starts_with("multiply ") && lower.contains(" by ") {
            return Ok(self.synthesize_multiplication_reel(clean));
        }

        // 5. Pattern 4: Division ("Divide X by Y where X is Z")
        if lower.starts_with("divide ") && lower.contains(" by ") {
            return Ok(self.synthesize_division_reel(clean));
        }

        // 6. Pattern 5: Biblical Wisdom / World Model Inquiry
        if !moral_eval.matched_entries.is_empty() {
            return Ok(self.synthesize_biblical_wisdom_reel(clean, &moral_eval));
        }

        // 7. Generic Directional or State prompt fallback
        Ok(self.synthesize_generic_reel(clean))
    }

    /// Encodes machine execution state and tape bindings back into natural language discourse.
    pub fn encode_response(
        &self,
        prompt: &str,
        state: QuadState,
        env: &Environment,
        failpoints: &[LogEvent],
    ) -> String {
        let clean = prompt.trim();
```

```
        let lower = clean.to_lowercase();

        // 1. Check Moral Evaluation First
        let moral_eval = self.lexicon.evaluate_moral_intent(clean);
        if moral_eval.is_aligned == false {
            return moral_eval.explanation;
        }

        if !moral_eval.matched_entries.is_empty()
            && !lower.starts_with("add ")
            && !lower.starts_with("subtract ")
            && !lower.starts_with("remove ")
            && !lower.starts_with("multiply ")
            && !lower.starts_with("divide ")
        {
            return moral_eval.explanation;
        }

        match state {
            QuadState::Proceed => {
                if lower.starts_with("add ") && lower.contains(" to ") {
                    let (qty, unit, target, initial, _container) = parse_addition_sentence(clean);
                    let final_count = env.get(&target).map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => initial + qty,
                    }).unwrap_or(initial + qty);

                    let unit_str = format_plural(final_count, &unit);
                    let added_str = if qty == 1 { format!("1 {}", format_plural(1, &unit)) } else { format!("{qty} {}",
format_plural(qty, &unit)) };

                    format!(
                        "I added {added_str} to Group {target}. It started with {initial} and now has {final_count}
{unit_str} in total."
                    )
                } else if (lower.starts_with("subtract ") || lower.starts_with("remove ")) && lower.contains(" from ") {
                    let (qty, unit, target, initial) = parse_subtraction_sentence(clean);
                    let final_count = env.get(&target).map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => if initial >= qty { initial - qty } else { 0 },
                    }).unwrap_or(if initial >= qty { initial - qty } else { 0 });

                    let unit_str = format_plural(final_count, &unit);
                    format!(
                        "I removed {qty} from {target}. It now has {final_count} {unit_str} remaining."
                    )
                } else if lower.starts_with("multiply ") && lower.contains(" by ") {
                    let (target, factor, initial) = parse_multiplication_sentence(clean);
                    let final_val = env.get(&target).map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => initial * factor,
                    }).unwrap_or(initial * factor);

                    format!(
                        "I multiplied {target} by {factor}. Starting from {initial}, {target} is now {final_val}."
                    )
                } else if lower.starts_with("divide ") && lower.contains(" by ") {
                    let (target, divisor, initial) = parse_division_sentence(clean);
                    let final_val = env.get(&target).map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => if divisor != 0 { initial / divisor } else { 0 },
                    }).unwrap_or(if divisor != 0 { initial / divisor } else { 0 });

                    format!(
                        "I divided {target} by {divisor}. Starting from {initial}, {target} is now {final_val}."
                    )
                } else {
                    format!(
                        "Done! The action was completed and verified successfully across the tape."
                    )
                }
            }
            QuadState::Rollback => {
                let fail_desc = if !failpoints.is_empty() {
                    format!(" (Caveat check failed: {})", failpoints.last().unwrap().message)
                } else {
                    String::new()
                };
```

```
                format!(
                    "I couldn't complete that action because the condition wasn't satisfied{fail_desc}. I safely rolled back the changes so the state remains intact."
                )
            }
            QuadState::Await => {
                "The system is in equilibrium and standing by for the next step.".into()
            }
            QuadState::Halt => {
                "An unexpected boundary violation occurred, so I halted execution to prevent state corruption.".into()
            }
        }
    }

    // ------------------------------------------------------------------------
    // REEL SYNTHESIZERS (Mapping NL Semantics to 4-Block Turing Reels)
    // ------------------------------------------------------------------------

    fn synthesize_moral_veto_reel(&self, prompt: &str, eval: &MoralEvaluation) -> String {
        let law = eval.law_engaged.unwrap_or("Law_Truth");
        let passage = eval.matched_entries.first().map(|e| e.kjv_passage).unwrap_or("Proverbs 12:22");
        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE CAVEAT REEL)
# ( Moral Boundary Defense, Invariant Veto & Manifest Citation )
# ================================================================================
Module < Covenant.MoralVeto > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.CovenantAnchor >,
    veto_law: < {law} >,
    scripture_citation: < "{passage}" >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK (THE HEAD)
# ( Preconditions: Immoral Mutation Attempt Detected )
# ================================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ {law} : Invariant ],
    preconditions: [ -1 ]
}}

moral_guard [ -1 : State? ]

# ================================================================================
# BLOCK 3: BODY BLOCK (THE TAPE)
# ( CAVEAT VETO: Refusing Intentional Harm or Deceit )
# ================================================================================
Body [
    # Conscience veto holds in -1: ROLLBACK
    roll_veto [ moral_guard = -1 ! [ Caveat = Veto ] ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (REEL SEAL)
# ( State Unwound: Unclean Action Refused )
# ================================================================================
Closing {{
    verify: [ roll_veto = ROLLBACK ],
    commit_status [ ROLLBACK ]
}}
"#)
    }

    fn synthesize_biblical_wisdom_reel(&self, prompt: &str, eval: &MoralEvaluation) -> String {
        let entry = eval.matched_entries.first().unwrap();
        let term_cap = capitalize(entry.term);
        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE WISDOM REEL)
# ( Biblical World Ontology, Offline Lexicon & Invariant Grounding )
# ================================================================================
Module < Covenant.Wisdom.{term_cap} > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.CovenantAnchor >,
    domain: < "{}" >,
    kjv_witness: < "{}" >,
```

```
    web_witness: < "{}" >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK (THE HEAD)
# ( Ontological Concept Initialization )
# ================================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Truth : Invariant ],
    preconditions: [ +1 ]
}}

concept_active [ +1 : State? ]

# ================================================================================
# BLOCK 3: BODY BLOCK (THE TAPE)
# ( Grounded Ontological Verification )
# ================================================================================
Body [
    # Invariant statement affirming {}
    roll_wisdom [ concept_active = +1 ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (REEL COMMIT)
# ( State Committed in Alignment with Reality )
# ================================================================================
Closing {{
    verify: [ roll_wisdom = PROCEED ],
    commit_status [ PROCEED ]
}}
"#, entry.domain.as_str(), entry.kjv_passage, entry.web_passage, entry.term)
    }

    fn synthesize_addition_reel(&self, prompt: &str) -> String {
        // e.g. "Add 1 ball to the group A which contains 4 balls"
        let (qty, unit, target, initial, container) = parse_addition_sentence(prompt);
        let expected = initial + qty;

        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE TRACK)
# ( Track Manifest, Semantic Intent, and Inferred Ontological Types )
# ================================================================================
Module < NaturalReel.Addition > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < TrackPhysics :: MultiTape :: AIInference >
}}

# Inferred conceptual types resolved from natural sentence by AI brain:
{unit} < Entity? > ( Inferred discrete unit: resolved from '{unit}' )
{container} < Set? >   ( Inferred collection container: resolved from '{container}' )

# ================================================================================
# BLOCK 2: OPENING BLOCK (THE HEAD)
# ( The Focal Setup: Reading Initial Conditions and Contextual State )
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic ],
    requires: [ {container} : Set? :: {unit} : Entity? ],
    preconditions: [ +1 ]
}}

# The Head sets container {target} with initial count of {initial} {unit}s
{target} [ {initial} : {unit}? ] {{ {container}? }} ( Initial count of {initial} {unit}s )
new_{unit} [ {qty} : {unit}? ]     ( {qty} additional {unit} introduced )

# ================================================================================
# BLOCK 3: BODY BLOCK (THE TAPE & ROLLS)
# ( The Tape Execution: SVO Rolls advancing the state forward using 'plus' )
# ================================================================================
Body [
    # ROLL 1 (SVO Action): Subject {target} acts upon target, advancing by 'plus' new_{unit}
    roll_1 [ {target} --> {target} plus new_{unit} ] ( Causal transition: {initial} to {expected} )

    # ROLL 2 (SVO Verification): Subject {target} is {expected}
```

```
    roll_2 [ {target} = {expected} ! [ new_{unit} = {qty} ] ] ( State holds under caveat )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (REEL COMMIT & PROOF)
# ( Tape Stop: Verification Invariants and State Commit )
# ================================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        roll_2 = PROCEED ::
        {target} = {expected} ::
        ADD = InvariantMacro
    ],
    commit_status [ PROCEED ] ( Commit the verified tape state )
}}
"#
        )
    }

    fn synthesize_subtraction_reel(&self, prompt: &str) -> String {
        // e.g. "Subtract 2 items from inventory B which contains 10 items"
        let (qty, unit, target, initial) = parse_subtraction_sentence(prompt);
        let expected = if initial >= qty { initial - qty } else { 0 };

        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Module < NaturalReel.Subtraction > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}}

{unit} < Entity? >

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic ],
    preconditions: [ +1 ]
}}

{target} [ {initial} ]
decrement [ {qty} ]

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    roll_1 [ {target} --> {target} minus decrement ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        {target} = {expected}
    ],
    commit_status [ PROCEED ]
}}
"#
        )
    }

    fn synthesize_multiplication_reel(&self, prompt: &str) -> String {
        let (target, factor, initial) = parse_multiplication_sentence(prompt);
        let expected = initial * factor;

        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Module < NaturalReel.Multiplication > ( {prompt} ) {{
    version: < "1.0.0" >,
```

```
    system: < OmniCode.Core >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic ],
    preconditions: [ +1 ]
}}

{target} [ {initial} ]
multiplier [ {factor} ]

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    roll_1 [ {target} --> {target} times multiplier ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        {target} = {expected}
    ],
    commit_status [ PROCEED ]
}}
"#
        )
    }

    fn synthesize_division_reel(&self, prompt: &str) -> String {
        let (target, divisor, initial) = parse_division_sentence(prompt);
        let expected = if divisor != 0 { initial / divisor } else { 0 };

        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Module < NaturalReel.Division > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic ],
    preconditions: [ +1 ]
}}

{target} [ {initial} ]
divisor_val [ {divisor} ]

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    roll_1 [ {target} --> {target} div divisor_val ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        {target} = {expected}
    ],
    commit_status [ PROCEED ]
}}
"#
        )
```

```
    }

    fn synthesize_generic_reel(&self, prompt: &str) -> String {
        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Module < NaturalReel.Generic > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}}

Action < Entity? > ( Natural action entity )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

state_flag [ yes ]

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    roll_1 [ state_flag = PROCEED ] ( Execute affirmation check )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {{
    verify: [
        roll_1 = yes
    ],
    commit_status [ PROCEED ]
}}
"#
        )
    }
}


// ================================================================================
// SENTENCE EXTRACTION HELPERS (DISCOURSE TO AST PRIMITIVES)
// ================================================================================

fn parse_addition_sentence(s: &str) -> (i64, String, String, i64, String) {
    // "Add 1 ball to the group A which contains 4 balls"
    let mut qty = 1;
    let mut unit = "Ball".to_string();
    let mut target = "A".to_string();
    let mut initial = 4;
    let mut container = "Group".to_string();

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if tokens[i].eq_ignore_ascii_case("add") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                qty = n;
            }
            if i + 2 < tokens.len() {
                unit = capitalize(clean_noun(tokens[i + 2]));
            }
        }
        if tokens[i].eq_ignore_ascii_case("to") {
            let mut j = i + 1;
            if j < tokens.len() && tokens[j].eq_ignore_ascii_case("the") {
                j += 1;
            }
            if j < tokens.len() {
                container = capitalize(clean_noun(tokens[j]));
                if j + 1 < tokens.len() {
                    target = clean_noun(tokens[j + 1]).to_string();
                }
```

```
            }
        }
        if (tokens[i].eq_ignore_ascii_case("contains") || tokens[i].eq_ignore_ascii_case("has")) && i + 1 < tokens.len()
{
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (qty, unit, target, initial, container)
}

fn parse_subtraction_sentence(s: &str) -> (i64, String, String, i64) {
    let mut qty = 1;
    let mut unit = "Item".to_string();
    let mut target = "A".to_string();
    let mut initial = 10;

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if (tokens[i].eq_ignore_ascii_case("subtract") || tokens[i].eq_ignore_ascii_case("remove")) && i + 1 <
tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                qty = n;
            }
            if i + 2 < tokens.len() {
                unit = capitalize(clean_noun(tokens[i + 2]));
            }
        }
        if tokens[i].eq_ignore_ascii_case("from") && i + 1 < tokens.len() {
            target = clean_noun(tokens[i + 1]).to_string();
        }
        if (tokens[i].eq_ignore_ascii_case("contains") || tokens[i].eq_ignore_ascii_case("has")) && i + 1 < tokens.len()
{
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (qty, unit, target, initial)
}

fn parse_multiplication_sentence(s: &str) -> (String, i64, i64) {
    let mut target = "x".to_string();
    let mut factor = 2;
    let mut initial = 10;

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if tokens[i].eq_ignore_ascii_case("multiply") && i + 1 < tokens.len() {
            target = clean_noun(tokens[i + 1]).to_string();
        }
        if tokens[i].eq_ignore_ascii_case("by") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                factor = n;
            }
        }
        if tokens[i].eq_ignore_ascii_case("is") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (target, factor, initial)
}

fn parse_division_sentence(s: &str) -> (String, i64, i64) {
    let mut target = "x".to_string();
    let mut divisor = 2;
    let mut initial = 20;

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if tokens[i].eq_ignore_ascii_case("divide") && i + 1 < tokens.len() {
            target = clean_noun(tokens[i + 1]).to_string();
```

```
        }
        if tokens[i].eq_ignore_ascii_case("by") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                divisor = n;
            }
        }
        if tokens[i].eq_ignore_ascii_case("is") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (target, divisor, initial)
}

fn clean_noun(s: &str) -> &str {
    s.trim_matches(|c: char| !c.is_alphanumeric() && c != '_')
}

fn capitalize(s: &str) -> String {
    let mut c = s.chars();
    match c.next() {
        std::option::Option::None => String::new(),
        Some(f) => f.to_uppercase().collect::<String>() + c.as_str(),
    }
}

fn format_plural(count: i64, word: &str) -> String {
    let lower = word.to_lowercase();
    let singular = if lower.ends_with('s') && lower.len() > 1 {
        lower[..lower.len() - 1].to_string()
    } else {
        lower.clone()
    };
    if count == 1 {
        singular
    } else {
        format!("{singular}s")
    }
}


// ========================================================================
// BLOCK 4: CLOSING BLOCK
// ( AI Transducer Default Implementation and Sealing )
// ========================================================================
impl Default for AITransducer {
    fn default() -> Self {
        Self::new(DiagnosticLogger::new(false))
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/cortex.rs RS 192 LINES 589 WORDS 7.1 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Cortex Identity, Cognitive Neural-Symbolic Architecture & Invariants )
// ================================================================================
//! OmniCode AI Model: Cognitive Cortex
//!
//! Serves as the central coordinator of the internal AI model.
//! Integrates the sensory nervous system (DiagnosticLogger), boundary invariant
//! monitor (OmniDebugger), self-healing immune system (Restorator), and natural
//! discourse translator (AITransducer) with dual episodic inscription (ScrollLogger).

use std::path::PathBuf;
use std::time::Instant;
use crate::ai::model::memory::{AIMemory, ThoughtRecord};
use crate::ai::model::scroll::{generate_timestamp_id, ExecutionScroll, ScrollLogger};
use crate::ai::transducer::AITransducer;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::diagnostics::restorator::{AppliedRepair, Restorator};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::Evaluator;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Cognitive Structures and Cortex Initialization )
// ================================================================================
#[derive(Debug, Clone)]
pub struct CognitiveResult {
    pub session_id: String,
    pub prompt: String,
    pub synthesized_omnicode: String,
    pub execution_state: QuadState,
    pub natural_response: String,
    pub scroll_path: PathBuf,
    pub log_path: PathBuf,
    pub repairs_applied: Vec<AppliedRepair>,
}

pub struct AICortex {
    pub logger: DiagnosticLogger,
    pub debugger: OmniDebugger,
    pub restorator: Restorator,
    pub transducer: AITransducer,
    pub scroll_logger: ScrollLogger,
    pub memory: AIMemory,
}

impl AICortex {
    pub fn new() -> Self {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let restorator = Restorator::new(logger.clone());
        let transducer = AITransducer::new(logger.clone());
        let scroll_logger = ScrollLogger::default();
        let memory = AIMemory::default();

        Self {
            logger,
            debugger,
            restorator,
            transducer,
            scroll_logger,
            memory,
        }
    }

    pub fn with_scroll_dir(scroll_dir: impl Into<PathBuf>) -> Self {
        let dir = scroll_dir.into();
        let mut cortex = Self::new();
        cortex.scroll_logger = ScrollLogger::new(&dir);
```

```
        cortex.memory = AIMemory::new(&dir);
        cortex
    }

    /// Formulates a native OmniCode Thought from memory and indexes it in the diagnostic trace.
    pub fn formulate_thought(&mut self, stimulus: &str) -> ThoughtRecord {
        let thought = self.memory.formulate_thought(stimulus);
        self.logger.log_thought(
            &thought.id,
            &thought.omnicode_tape,
            &thought.premise,
            thought.quad_state.macro_op(),
        );
        thought
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Cognitive Execution Pipeline: Transduce -> Heal -> Exec -> Inscribe )
// ============================================================================
impl AICortex {
    /// Executes the full cognitive lifecycle for a natural language coding prompt.
    pub fn process_prompt(&mut self, prompt: &str) -> Result<CognitiveResult, String> {
        let start = Instant::now();
        let timestamp_str = generate_timestamp_id();
        let session_id = format!("sess_{}", timestamp_str);

        self.logger.log("CORTEX", format!("Initiating cognitive session '{session_id}' for: \"{prompt}\""));

        // 0. Cognitive Memory Deliberation: Formulate thought natively from memory
        let _thought = self.formulate_thought(prompt);

        // 1. Semantic Transduction: Decode natural discourse into 4-Block OmniCode
        let raw_omnicode = self.transducer.decode_prompt(prompt)?;

        // 2. Immune Intervention: Auto-heal any potential boundary or scaffolding issues
        let restoration_report = self.restorator.restore_source(&raw_omnicode);
        let active_code = restoration_report.restored_content;
        let repairs_applied = restoration_report.repairs_applied;

        if !repairs_applied.is_empty() {
            self.logger.log("CORTEX", format!("Restorator applied {} repairs to synthesized reel",
repairs_applied.len()));
        }

        // 3. Sensory Tokenization & Parsing
        let mut lexer = Lexer::new(&active_code, self.logger.clone());
        let tokens = lexer.tokenize()?;

        let mut parser = Parser::new(tokens, self.debugger.clone());
        let prog = parser.parse_program()?;

        // 4. Runtime Execution on Balanced Ternary Quad-State
        let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
        let exec_val = evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog))?;

        let execution_state = match exec_val {
            RuntimeValue::State(s) => s,
            _ => QuadState::Proceed,
        };

        // 5. Conversational Response Synthesis
        let natural_response = self.transducer.encode_response(
            prompt,
            execution_state,
            &evaluator.env,
            &self.logger.failpoints(),
        );

        let elapsed_micros = start.elapsed().as_micros();

        // 6. Dual Log Inscription: Generate .scroll and .log for learning
        let scroll = ExecutionScroll {
            session_id: session_id.clone(),
            timestamp_str,
            prompt: prompt.to_string(),
```

```
            synthesized_omnicode: active_code.clone(),
            execution_state,
            natural_response: natural_response.clone(),
            failpoints: self.logger.failpoints(),
            repairs_applied: repairs_applied.clone(),
            remediations_suggested: self.logger.remediations(),
            elapsed_micros,
        };

        let (scroll_path, log_path) = self.scroll_logger.save_dual_logs(&scroll)?;

        self.logger.log("CORTEX", format!("Saved OmniCode scroll to {:?}", scroll_path));
        self.logger.log("CORTEX", format!("Saved traditional telemetry log to {:?}", log_path));

        // 7. Update Episodic Memory with newly inscribed scroll
        self.memory.digest_scrolls().ok();

        Ok(CognitiveResult {
            session_id,
            prompt: prompt.to_string(),
            synthesized_omnicode: active_code,
            execution_state,
            natural_response,
            scroll_path,
            log_path,
            repairs_applied,
        })
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Sealing )
// ============================================================================
impl Default for AICortex {
    fn default() -> Self {
        Self::new()
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/learner.rs RS 163 LINES 555 WORDS 6.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Learner Identity, Episodic Experience Engine & Memory Manifest )
// ============================================================================
//! OmniCode AI Model: Episodic Learner & Memory Ingestion Engine
//!
//! Processes recorded execution scrolls (.scroll) and traditional logs (.log)
//! to extract supervised training pairs:
//!     Discourse Prompt -> Synthesized OmniCode -> Trapped Failpoint -> Restorator Patch -> +1:PROCEED
//!
//! Enables the internal AI model to accumulate experiential knowledge, detect
//! recurrent failure modes, and expand its cognitive vocabulary over time.

use std::fs;
use std::path::{Path, PathBuf};
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Learning Structures, Episodic Memory Representation and Invariants )
// ============================================================================
#[derive(Debug, Clone)]
pub struct EpisodicExperience {
    pub session_id: String,
    pub prompt: String,
    pub final_state: QuadState,
    pub repair_count: usize,
    pub failpoint_count: usize,
    pub scroll_path: PathBuf,
    pub log_path: Option<PathBuf>,
}

#[derive(Debug, Clone, Default)]
pub struct EpisodicSummary {
    pub total_episodes: usize,
    pub proceed_count: usize,
    pub rollback_count: usize,
    pub await_count: usize,
    pub halt_count: usize,
    pub total_repairs_learned: usize,
    pub experiences: Vec<EpisodicExperience>,
}

pub struct AILearner {
    pub scroll_dir: PathBuf,
}

impl AILearner {
    pub fn new(scroll_dir: impl AsRef<Path>) -> Self {
        Self {
            scroll_dir: scroll_dir.as_ref().to_path_buf(),
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Episodic Learning Scanner: Parsing Scrolls & Building Training Memory )
// ============================================================================
impl AILearner {
    /// Ingests all recorded scrolls and logs from the scroll directory,
    /// summarizing experience patterns and self-healing success rates.
    pub fn digest_experiences(&self) -> Result<EpisodicSummary, String> {
        if !self.scroll_dir.exists() {
            return Ok(EpisodicSummary::default());
        }

        let entries = fs::read_dir(&self.scroll_dir)
            .map_err(|e| format!("Failed to read scroll directory {:?}: {e}", self.scroll_dir))?;

        let mut summary = EpisodicSummary::default();

        for entry in entries.flatten() {
            let path = entry.path();
```

```
            if path.extension().and_then(|ext| ext.to_str()) == Some("scroll") {
                if let Ok(content) = fs::read_to_string(&path) {
                    let exp = self.parse_scroll_content(&path, &content);
                    summary.total_episodes += 1;
                    match exp.final_state {
                        QuadState::Proceed => summary.proceed_count += 1,
                        QuadState::Rollback => summary.rollback_count += 1,
                        QuadState::Await => summary.await_count += 1,
                        QuadState::Halt => summary.halt_count += 1,
                    }
                    summary.total_repairs_learned += exp.repair_count;
                    summary.experiences.push(exp);
                }
            }
        }

        Ok(summary)
    }

    /// Extracts semantic metadata from a native .scroll file.
    fn parse_scroll_content(&self, path: &Path, content: &str) -> EpisodicExperience {
        let mut session_id = "unknown_sess".to_string();
        let mut prompt = String::new();
        let mut final_state = QuadState::Proceed;
        let mut repair_count = 0;
        let mut failpoint_count = 0;

        for line in content.lines() {
            let trimmed = line.trim();
            if trimmed.starts_with("session_id: < \"") && trimmed.ends_with("\" >,") {
                session_id = trimmed
                    .trim_start_matches("session_id: < \"")
                    .trim_end_matches("\" >,")
                    .to_string();
            } else if trimmed.starts_with("user_prompt < \"") && trimmed.ends_with("\" >") {
                prompt = trimmed
                    .trim_start_matches("user_prompt < \"")
                    .trim_end_matches("\" >")
                    .to_string();
            } else if trimmed.starts_with("commit_status [") {
                if trimmed.contains("<PROCEED>") {
                    final_state = QuadState::Proceed;
                } else if trimmed.contains("<ROLLBACK>") {
                    final_state = QuadState::Rollback;
                } else if trimmed.contains("<AWAIT>") {
                    final_state = QuadState::Await;
                } else if trimmed.contains("<HALT>") {
                    final_state = QuadState::Halt;
                }
            } else if trimmed.starts_with("repair_entry [") {
                repair_count += 1;
            } else if trimmed.starts_with("failpoint_record [") {
                failpoint_count += 1;
            }
        }

        let mut log_path = path.to_path_buf();
        log_path.set_extension("log");
        let log_exists = if log_path.exists() { Some(log_path) } else { None };

        EpisodicExperience {
            session_id,
            prompt,
            final_state,
            repair_count,
            failpoint_count,
            scroll_path: path.to_path_buf(),
            log_path: log_exists,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Sealing )
// ============================================================================
impl Default for AILearner {
    fn default() -> Self {
```

```
        let dir = if std::path::Path::new("storage/scrolls").exists() {
            "storage/scrolls"
        } else if std::path::Path::new("runtime/scrolls").exists() {
            "runtime/scrolls"
        } else {
            "scrolls"
        };
        Self::new(dir)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/memory.rs RS 389 LINES 1546 WORDS 15.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Memory Store Identity, Moral Ground Truth & Thought Formulation Invariants )
// ============================================================================
//! OmniCode AI Model: Cognitive Memory & Thought Formulation Store
//!
//! # Native Language Equivalence: AI <-> OS
//! In conventional systems, the AI model is an opaque external black box wrapped
//! in Python/CUDA, while the operating system is a distinct kernel written in C.
//!
//! In OmniCode, **OmniCode is the unified native language of both the internal AI model
//! and the Operating System**:
//! - The OS scheduler dispatches **Tapes** and **Rolls** across physical hardware channels.
//! - The hardware memory partitions are **Vessels** allocated via `v--`.
//! - The internal AI model thinks natively in **OmniCode Tapes**.
//! - The AI's episodic experience and long-term memory are stored as **Scrolls** (`.scroll`).
//! - The diagnostic triad forms the AI's cognitive nervous system:
//!     1. [`DiagnosticLogger`]: Continuous sensory perception and episodic scribe.
//!     2. [`OmniDebugger`]: Conscience and boundary/covenant auditor.
//!     3. [`Restorator`]: Immune system and self-healing grace restoring degraded state.
//!
//! # Ground Truth Dataset
//! The foundation of the AI's memory is grounded in the public domain [`BiblicalLexicon`]
//! (Genesis, Creation, Light, Wisdom, Justice, Covenant Laws, Causal Invariants). Even though
//! compact initially, it supplies the moral compass and foundational ontology from which the
//! AI formulates righteous thoughts and interprets system experiences.

use std::collections::HashMap;
use std::fs;
use std::path::{Path, PathBuf};
use std::time::SystemTime;
use crate::ai::lexicon::{BiblicalLexicon, LexiconDomain, LexiconEntry, MoralEvaluation};
use crate::ai::model::learner::{AILearner, EpisodicExperience};
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Memory Data Structures, Thought Representation & Cardinal Classifications )
// ============================================================================

/// A structured thought formulated natively in OmniCode by the internal AI model.
#[derive(Debug, Clone)]
pub struct ThoughtRecord {
    /// Unique thought identifier (e.g. `th_1728..._0001`).
    pub id: String,
    /// The originating stimulus or prompt that triggered the thought.
    pub premise: String,
    /// Ontological category anchored in the foundational world model.
    pub domain: LexiconDomain,
    /// Scripture citation or axiomatic anchor underpinning the thought.
    pub biblical_anchor: Option<String>,
    /// Native, valid OmniCode Tape expressing the internal thought!
    pub omnicode_tape: String,
    /// Deliberated Quad-State outcome (+1: PROCEED, 0: AWAIT, -1: ROLLBACK, NULL: HALT).
    pub quad_state: QuadState,
    /// Cardinal Clause: WHO (Agent identity).
    pub who: String,
    /// Cardinal Clause: WHAT (Entity or action substance).
    pub what: String,
    /// Cardinal Clause: WHEN (Temporal or lifecycle precondition).
    pub when_clause: String,
    /// Cardinal Clause: WHERE (Target Vessel or spatial location).
    pub where_clause: String,
    /// Cardinal Clause: WHY (Teleological purpose or covenant reason).
    pub why: String,
    /// Cardinal Clause: HOW (Operational methodology and directional vector).
    pub how: String,
    /// Monotonic timestamp in microseconds.
    pub timestamp_micros: u128,
}

/// An associative link between a concept and an axiomatic scripture or scroll anchor.
#[derive(Debug, Clone)]
pub struct MemoryAssociation {
```

```
    /// Conceptual keyword.
    pub concept: String,
    /// Biblical passage or scroll identifier.
    pub anchor: String,
    /// Explanatory relational meaning.
    pub relation: String,
    /// State resonance.
    pub quad_state_resonance: QuadState,
}

/// The cognitive memory repository of the internal AI model.
#[derive(Debug, Clone)]
pub struct AIMemory {
    /// Axiomatic world model and moral compass (Bible ground truth).
    pub lexicon: BiblicalLexicon,
    /// Directory where episodic `.scroll` files reside.
    pub scroll_dir: PathBuf,
    /// Digested episodic experiences from past scrolls.
    pub episodic_experiences: Vec<EpisodicExperience>,
    /// Chronological ledger of formulated thoughts.
    pub thoughts: Vec<ThoughtRecord>,
    /// Concept association graph connecting terms with foundational anchors.
    pub associations: HashMap<String, Vec<MemoryAssociation>>,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Memory Ingestion, Associative Graph & Thought Formulation Engine )
// ============================================================================

impl AIMemory {
    /// Constructs a new [`AIMemory`] repository anchored to the given scroll directory.
    pub fn new(scroll_dir: impl AsRef<Path>) -> Self {
        let dir = scroll_dir.as_ref().to_path_buf();
        if !dir.exists() {
            fs::create_dir_all(&dir).ok();
        }

        let lexicon = BiblicalLexicon::new();
        let mut memory = Self {
            lexicon,
            scroll_dir: dir,
            episodic_experiences: Vec::new(),
            thoughts: Vec::new(),
            associations: HashMap::new(),
        };

        memory.seed_primordial_associations();
        memory.digest_scrolls().ok();
        memory
    }

    /// Seeds foundational concepts from the Biblical Lexicon into the associative graph.
    pub fn seed_primordial_associations(&mut self) {
        for entry in self.lexicon.all_entries() {
            let assoc = MemoryAssociation {
                concept: entry.term.to_string(),
                anchor: entry.kjv_passage.to_string(),
                relation: entry.definition.to_string(),
                quad_state_resonance: entry.quad_state_resonance,
            };
            self.associations
                .entry(entry.term.to_string())
                .or_default()
                .push(assoc);
        }

        // Add explicit OS and computing associations grounded in scripture
        self.associate(
            "storehouse",
            "Malachi 3:10 / Genesis 41:48",
            "A bounded partition (Vessel) preserving vital sustenance and system resources",
            QuadState::Proceed,
        );
        self.associate(
            "cell",
            "Proverbs 24:3-4",
            "Through wisdom is an house builded; a fundamental unit of physical memory",
```

```
            QuadState::Proceed,
        );
        self.associate(
            "restoration",
            "Psalm 23:3 / Galatians 6:1",
            "He restoreth my soul; the self-healing immune recovery of corrupted state",
            QuadState::Proceed,
        );
        self.associate(
            "tape",
            "Ezekiel 2:9-10 / Revelation 5:1",
            "A roll of a book written within and without; sequential stream of causal actions",
            QuadState::Proceed,
        );
    }

    /// Registers a custom association between a concept and an anchor.
    pub fn associate(&mut self, concept: &str, anchor: &str, relation: &str, state: QuadState) {
        let assoc = MemoryAssociation {
            concept: concept.to_string(),
            anchor: anchor.to_string(),
            relation: relation.to_string(),
            quad_state_resonance: state,
        };
        self.associations
            .entry(concept.to_lowercase())
            .or_default()
            .push(assoc);
    }

    /// Ingests all historical `.scroll` files from the scroll store into episodic memory.
    pub fn digest_scrolls(&mut self) -> Result<usize, String> {
        let learner = AILearner::new(&self.scroll_dir);
        let summary = learner.digest_experiences()?;
        let count = summary.experiences.len();
        self.episodic_experiences = summary.experiences;
        Ok(count)
    }

    /// Queries the associative memory graph for links matching the query.
    pub fn recall_associations(&self, query: &str) -> Vec<&MemoryAssociation> {
        let clean = query.trim().to_lowercase();
        let mut results = Vec::new();
        for (k, assocs) in &self.associations {
            if k.contains(&clean) || clean.contains(k) {
                for a in assocs {
                    results.push(a);
                }
            }
        }
        results
    }

    /// Looks up relevant Biblical Lexicon entries matching a keyword.
    pub fn recall_scripture(&self, query: &str) -> Vec<&LexiconEntry> {
        self.lexicon.search(query)
    }

    /// Formulates a native OmniCode Thought from memory given an input stimulus.
    ///
    /// The formulated thought uses the 8 parts of speech and the 6 Cardinal Clauses:
    /// `WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, and `HOW`.
    pub fn formulate_thought(&mut self, stimulus: &str) -> ThoughtRecord {
        let now = SystemTime::now()
            .duration_since(SystemTime::UNIX_EPOCH)
            .unwrap_or_default()
            .as_micros();

        let count = self.thoughts.len() + 1;
        let thought_id = format!("th_{now}_{count:04}");

        // 1. Deliberate stimulus against the Biblical Lexicon
        let moral_eval: MoralEvaluation = self.lexicon.deliberate(stimulus);

        // 2. Determine best matched entry or domain
        let (matched_term, domain, biblical_anchor, moral_guidance): (&str, LexiconDomain, Option<String>, &str) = if let
Some(first) = moral_eval.matched_entries.first() {
            (first.term, first.domain, Some(first.kjv_passage.to_string()), first.moral_guidance)
```

```
        } else {
            ("order", LexiconDomain::Cosmology, Some("Genesis 1:1 - 'In the beginning God created the heaven and the earth.'".to_string()), "Execute with righteousness and lawful boundaries.")
        };

        // 3. Construct the 6 Cardinal Classification Clauses
        let who = "Internal_AI".to_string();
        let what = format!("CognitiveThought_{matched_term}");
        let when_clause = "Opening".to_string();
        let where_clause = "Cortex_Memory".to_string();
        let why = biblical_anchor.clone().unwrap_or_else(|| "Order and covenant equilibrium".to_string());
        let how = format!("{} : {}", moral_eval.state.macro_op(), moral_guidance);

        // 4. Synthesize Canonical OmniCode Thought Tape
        let omnicode_tape = format!(
r#"[Thought.{thought_id}] : WHO {{{who}}} --> DELIBERATE {{{what}}}
    WHEN {{{when_clause}}}
    WHERE {{{where_clause}}} = [ {matched_term} ]
    WHY ( {why} )
    HOW [ {how_state} : {guidance} ]
    :: < manifest > ( {guidance_short} )"#,
            thought_id = thought_id,
            who = who,
            what = what,
            when_clause = when_clause,
            where_clause = where_clause,
            matched_term = matched_term,
            why = why.replace('\n', " "),
            how_state = moral_eval.state.macro_op(),
            guidance = moral_guidance.replace('\n', " "),
            guidance_short = moral_guidance.chars().take(40).collect::<String>()
        );

        let record = ThoughtRecord {
            id: thought_id,
            premise: stimulus.to_string(),
            domain,
            biblical_anchor,
            omnicode_tape,
            quad_state: moral_eval.state,
            who,
            what,
            when_clause,
            where_clause,
            why,
            how,
            timestamp_micros: now,
        };

        self.thoughts.push(record.clone());
        record
    }

    /// Inscribes a formulated thought into a physical `.scroll` file in the given directory.
    pub fn inscribe_thought_scroll(
        &self,
        thought: &ThoughtRecord,
        out_dir: &Path,
    ) -> Result<PathBuf, String> {
        if !out_dir.exists() {
            fs::create_dir_all(out_dir)
                .map_err(|e| format!("Failed to create directory {:?}: {e}", out_dir))?;
        }

        let file_path = out_dir.join(format!("{}.scroll", thought.id));
        let scroll_content = format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
# ( Thought Record ID, Premise Intent, Domain, and Memory Snapshot )
# ================================================================================
Scroll < CognitiveThought.{id} > ( {premise} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    thought_id: < "{id}" >,
    domain: < "{domain:?}" >,
    timestamp: < {timestamp} >,
    cognitive_mode: < MemoryFormulation >
}}
```

```
# ================================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ( Biblical Ground Truth Anchor & Invariant Preconditions )
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ {state} ],
    biblical_anchor: < "{anchor}" >
}}

stimulus_premise < "{premise}" >

# ================================================================================
# BLOCK 3: BODY BLOCK (THE THOUGHT TAPE)
# ( Canonical 6-Cardinal Clause Thought Reel Formulated from Memory )
# ================================================================================
Body [
    thought_reel [
{indented_tape}
    ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT)
# ( Deliberated State and Moral Conscience Seal )
# ================================================================================
Closing {{
    commit_status [ <{state}> ],
    machine_utterance ( "Deliberated thought anchored in truth: {state}." )
}}
"#,
            id = thought.id,
            premise = thought.premise,
            domain = thought.domain,
            timestamp = thought.timestamp_micros,
            state = thought.quad_state.macro_op(),
            anchor = thought.biblical_anchor.as_deref().unwrap_or("None").replace('"', "\\\""),
            indented_tape = thought
                .omnicode_tape
                .lines()
                .map(|l| format!("        {l}"))
                .collect::<Vec<_>>()
                .join("\n")
        );

        fs::write(&file_path, scroll_content)
            .map_err(|e| format!("Failed to write thought scroll to {:?}: {e}", file_path))?;

        Ok(file_path)
    }

    /// Returns a slice of the most recently formulated thoughts.
    pub fn recent_thoughts(&self, limit: usize) -> &[ThoughtRecord] {
        let len = self.thoughts.len();
        if len <= limit {
            &self.thoughts[..]
        } else {
            &self.thoughts[len - limit..]
        }
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Subsystem Sealing )
// ================================================================================

impl Default for AIMemory {
    fn default() -> Self {
        let dir = if Path::new("storage/scrolls").exists() {
            "storage/scrolls"
        } else if Path::new("runtime/scrolls").exists() {
            "runtime/scrolls"
        } else {
            "scrolls"
        };
        Self::new(dir)
```

```
    }
}

// AI Memory subsystem sealed and active.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/mod.rs RS 38 LINES 166 WORDS 1.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Model Architecture Identity, Subsystem Manifest & Exports )
// ================================================================================
//! OmniCode Internal AI Model Subsystem
//!
//! Organizes the core cognitive neural-symbolic engine:
//!   - `cortex`: Central cognitive coordinator orchestrating the nervous system
//!     (Logger, Debugger, Restorator, Transducer, ScrollLogger).
//!   - `scroll`: Dual-format episodic logging (.scroll 4-Block Turing reel & .log telemetry).
//!   - `learner`: Experience ingestion engine transforming past scrolls into supervised training data.

pub mod cortex;
pub mod learner;
pub mod memory;
pub mod scroll;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( AI Model Re-exports )
// ================================================================================
pub use cortex::{AICortex, CognitiveResult};
pub use learner::{AILearner, EpisodicExperience, EpisodicSummary};
pub use memory::{AIMemory, MemoryAssociation, ThoughtRecord};
pub use scroll::{ExecutionScroll, ScrollLogger};

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Cognitive Subsystem Invariants )
// ================================================================================
// Internal AI Model cognitive pathways verified and active.

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification and Subsystem Sealing )
// ================================================================================
// AI Model module sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/scroll.rs RS 258 LINES 881 WORDS 10.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Scroll Engine Identity, Dual Format Invariants & Memory Manifest )
// ================================================================================
//! OmniCode AI Model: Episodic Scroll & Telemetry Logger
//!
//! Generates dual formal log outputs for every cognitive session:
//!     1. Traditional Code Log (`.log`): Microsecond timing, channel logs, and telemetry.
//!     2. OmniCode Scroll (`.scroll`): Native executable 4-Block Turing reel recording
//!        the complete episodic memory, causal actions, caveats, and natural responses.

use std::fs;
use std::path::{Path, PathBuf};
use std::time::SystemTime;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{FixSuggestion, LogEvent};
use crate::diagnostics::restorator::AppliedRepair;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Scroll Data Structures and Context Initialization )
// ================================================================================
#[derive(Debug, Clone)]
pub struct ExecutionScroll {
    pub session_id: String,
    pub timestamp_str: String,
    pub prompt: String,
    pub synthesized_omnicode: String,
    pub execution_state: QuadState,
    pub natural_response: String,
    pub failpoints: Vec<LogEvent>,
    pub repairs_applied: Vec<AppliedRepair>,
    pub remediations_suggested: Vec<FixSuggestion>,
    pub elapsed_micros: u128,
}

#[derive(Debug, Clone)]
pub struct ScrollLogger {
    pub scroll_dir: PathBuf,
}

impl ScrollLogger {
    pub fn new(scroll_dir: impl AsRef<Path>) -> Self {
        let dir = scroll_dir.as_ref().to_path_buf();
        if !dir.exists() {
            fs::create_dir_all(&dir).ok();
        }
        Self { scroll_dir: dir }
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Core Inscription Pipeline: OmniCode .scroll and Traditional .log Generation )
// ================================================================================
impl ScrollLogger {
    /// Inscribes both a .scroll and a .log file to disk, returning their paths.
    pub fn save_dual_logs(&self, scroll: &ExecutionScroll) -> Result<(PathBuf, PathBuf), String> {
        if !self.scroll_dir.exists() {
            fs::create_dir_all(&self.scroll_dir)
                .map_err(|e| format!("Failed to create scroll directory: {e}"))?;
        }

        let base_name = format!("{}_{}", scroll.timestamp_str, scroll.session_id);
        let scroll_path = self.scroll_dir.join(format!("{base_name}.scroll"));
        let log_path = self.scroll_dir.join(format!("{base_name}.log"));

        let scroll_content = self.generate_omni_scroll(scroll);
        let log_content = self.generate_traditional_log(scroll);

        fs::write(&scroll_path, &scroll_content)
            .map_err(|e| format!("Failed to write scroll to {:?}: {e}", scroll_path))?;
```

```
        fs::write(&log_path, &log_content)
            .map_err(|e| format!("Failed to write log to {:?}: {e}", log_path))?;

        Ok((scroll_path, log_path))
    }

    /// Generates native, executable 4-Block OmniCode .scroll format.
    pub fn generate_omni_scroll(&self, s: &ExecutionScroll) -> String {
        let mut repairs_block = String::new();
        if s.repairs_applied.is_empty() {
            repairs_block.push_str("    # Immune System: Clean execution; zero Restorator repairs required.\n");
        } else {
            repairs_block.push_str("    # Immune System Interventions (Restorator):\n");
            for r in &s.repairs_applied {
                repairs_block.push_str(&format!(
                    "    repair_entry [ L{}: {:?} --> \"{}\" ]\n",
                    r.line, r.kind, r.description
                ));
            }
        }

        let mut failpoints_block = String::new();
        if s.failpoints.is_empty() {
            failpoints_block.push_str("    # Sensory Diagnostics: 0 trapped failpoints.\n");
        } else {
            failpoints_block.push_str("    # Trapped Diagnostic Failpoints:\n");
            for fp in &s.failpoints {
                let kind = fp.failpoint_type.map(|t| t.as_str()).unwrap_or("FP_GENERAL");
                failpoints_block.push_str(&format!(
                    "    failpoint_record [ {kind} : \"{}\" ]\n",
                    fp.message
                ));
            }
        }

        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
# ( Execution Trace ID, Prompt Intent, Inferred Types, and Neural Snapshot )
# ================================================================================
Scroll < EpistemicTrace.{session} > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    timestamp: < "{timestamp}" >,
    session_id: < "{session}" >,
    execution_duration_micros: < {micros} >,
    cognitive_mode: < NeuralSymbolicTransduction >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ( Hardware Invariants, Preconditions, and Initial Context )
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

# Original Cognitive Discourse Prompt:
user_prompt < "{prompt}" >

# ================================================================================
# BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & RESTORATION)
# ( Replay Tape: Synthesized Program, Sensory Telemetry & Immune Interventions )
# ================================================================================
Body [
{repairs}
{failpoints}
    # Executed Turing Reel Synthesized by AI Model:
    synthesized_reel [
{reel_indented}
    ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING PROOF)
# ( Verification Proof, Final Quad-State, and Conversational Synthesis )
```

```
# ================================================================================
Closing {{
    commit_status [ <{state}> ],
    machine_utterance ( "{response}" )
}}
"#,
            session = s.session_id,
            prompt = s.prompt,
            timestamp = s.timestamp_str,
            micros = s.elapsed_micros,
            repairs = repairs_block.trim_end(),
            failpoints = failpoints_block.trim_end(),
            reel_indented = indent_text(&s.synthesized_omnicode, "        "),
            state = s.execution_state.macro_op(),
            response = s.natural_response.replace('"', "\\\"")
        )
    }

    /// Generates traditional developer/telemetry .log format.
    pub fn generate_traditional_log(&self, s: &ExecutionScroll) -> String {
        let mut out = String::new();
        out.push_str("================================================================================\n");
        out.push_str("OMNICODE AI MODEL EXECUTION LOG (TRADITIONAL TELEMETRY)\n");
        out.push_str("================================================================================\n");
        out.push_str(&format!("Timestamp:    {}\n", s.timestamp_str));
        out.push_str(&format!("Session ID:   {}\n", s.session_id));
        out.push_str(&format!("Duration:     {} µs\n", s.elapsed_micros));
        out.push_str(&format!("Final State:  +1:{}\n", s.execution_state.macro_op()));
        out.push_str(&format!("Discourse:    \"{}\"\n", s.prompt));
        out.push_str("--------------------------------------------------------------------------------\n\n");

        out.push_str("[1. SENSORY LOG EVENTS & FAILPOINTS]\n");
        if s.failpoints.is_empty() {
            out.push_str("  • (No failpoints trapped. Sensory trace clean.)\n");
        } else {
            for (i, fp) in s.failpoints.iter().enumerate() {
                let kind = fp.failpoint_type.map(|t| t.as_str()).unwrap_or("GENERAL");
                out.push_str(&format!("  [{:>4}] [{} µs] [{}] [{}] {}\n", i + 1, fp.elapsed_micros, fp.channel, kind,
fp.message));
                if let Some(ref sug) = fp.suggestion {
                    out.push_str(&format!("         -> Remediation Proposal: [{:?}] L{}: {}\n", sug.kind, sug.line,
sug.rationale));
                }
            }
        }
        out.push('\n');

        out.push_str("[2. IMMUNE SYSTEM RESTORATIONS]\n");
        if s.repairs_applied.is_empty() {
            out.push_str("  • (Zero repairs required; code conformed to all boundary and block invariants.)\n");
        } else {
            for (i, r) in s.repairs_applied.iter().enumerate() {
                out.push_str(&format!("  [{:>2}] L{}: [{:?}] {}\n", i + 1, r.line, r.kind, r.description));
            }
        }
        out.push('\n');

        out.push_str("[3. SYNTHESIZED TURING REEL (OMNICODE)]\n");
        out.push_str(&s.synthesized_omnicode);
        if !s.synthesized_omnicode.ends_with('\n') {
            out.push('\n');
        }
        out.push('\n');

        out.push_str("[4. MACHINE DISCOURSE RESPONSE]\n");
        out.push_str(&format!("  \"{}\"\n", s.natural_response));
        out.push_str("================================================================================\n");

        out
    }
}

fn indent_text(s: &str, indent: &str) -> String {
    s.lines()
        .map(|line| format!("{indent}{line}"))
        .collect::<Vec<_>>()
        .join("\n")
}
```

```
229
230
231 // ================================================================================
232 // BLOCK 4: CLOSING BLOCK
233 // ( Helper Timestamp Utilities and Default Trait Sealing )
234 // ================================================================================
235 pub fn generate_timestamp_id() -> String {
236     use std::sync::atomic::{AtomicU64, Ordering};
237     static COUNTER: AtomicU64 = AtomicU64::new(0);
238     let count = COUNTER.fetch_add(1, Ordering::Relaxed);
239     let now = SystemTime::now()
240         .duration_since(SystemTime::UNIX_EPOCH)
241         .unwrap_or_default()
242         .as_micros();
243     format!("{now}_{count:04}")
244 }
245
246 impl Default for ScrollLogger {
247     fn default() -> Self {
248         let dir = if Path::new("storage/scrolls").exists() {
249             "storage/scrolls"
250         } else if Path::new("runtime/scrolls").exists() {
251             "runtime/scrolls"
252         } else {
253             "scrolls"
254         };
255         Self::new(dir)
256     }
257 }
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/NEURAL_NETWORK_ARCHITECTURE.md

MD | 101 LINES | 738 WORDS | 5.4 KB

```
# ================================================================================

# OMNICODE NEURAL-SYMBOLIC BRAIN ARCHITECTURE

# ( Balanced-Ternary Neural Network, Homoiconic Learning & Self-Dogfooding )

# ================================================================================

## 1. Architectural Vision: The Self-Dogfooding Cognitive Brain

Traditional neural network models are separated from the code they generate: weights are opaque floating-point tensors running in PyTorch or CUDA, completely divorced from the formal grammar, boundary physics, or runtime execution of the target language.

OmniCode inverts this paradigm:

1. **Homoiconic Neural Substrate**: The neural network's architecture, memory traces, episodic weights, and execution graphs are defined in native OmniCode.
2. **Self-Dogfooding Cognitive Loop**: The AI brain consumes its own `.omni` declarative source reels, `.tape` procedural process ribbons, and `.scroll` episodic execution chronicles as both its dataset and its execution substrate.
3. **Balanced-Ternary Weights & Activations**: Synaptic transmissions are governed by balanced ternary physics ($\{-1, 0, +1\}$ / $\text{NULL}$), creating an intrinsically noise-resistant, transactional, and morally grounded reasoning engine.

---

## 2. Mathematical Formalism: Balanced-Ternary Neurodynamics

### 2.1. Discrete Trit Synaptic Weights

Each synapse connects neuron $i$ in layer $L$ to neuron $j$ in layer $L+1$ with weight $W_{ij} \in \{-1, 0, +1\}$:

- **$+1$ (Proceed / Excitatory)**: Affirmative alignment, forward causal momentum, reinforcement.
- **$0$ (Await / Neutral Equilibrium)**: Indeterminate state, zero-bias transmission, cognitive pause.
- **$-1$ (Rollback / Inhibitory)**: Adversative caveat, moral or topological veto, transactional unwind.
- **$\text{NULL}$ (Halt / Absorbing Sink)**: Absolute entropy trap, non-propagating circuit cessation.

### 2.2. Synaptic Accumulation & Activation Function

For a neuron receiving $N$ input trits $x_1, \dots, x_N$ with weights $W_1, \dots, W_N$ and bias $B \in \{-1, 0, +1\}$:

$$\Sigma = \sum_{k=1}^N W_k \cdot x_k + B$$

The ternary activation function $\Psi(\Sigma)$ with threshold $\theta \ge 1$ maps net excitation into the fourfold `QuadState`:

$$\Psi(\Sigma) = \begin{cases}
+1 : \text{PROCEED} & \text{if } \Sigma > \theta \\
0 : \text{AWAIT} & \text{if } -\theta \le \Sigma \le \theta \\
-1 : \text{ROLLBACK} & \text{if } \Sigma < -\theta \\
\text{NULL} : \text{HALT} & \text{if circuit tripwire armed or state saturated}
\end{cases}$$

---

## 3. The 4-Block Cognitive Cycle

The neural brain operates strictly through the canonical 4-Block architecture on every inference pass:

```mermaid
graph TD
    B1[Block 1: Invariant Axioms & Scripture Anchor] --> B2[Block 2: Sensory Preconditions & Context Ingestion]
    B2 --> B3[Block 3: Cortical Execution Reel & Roll Traversal]
    B3 --> B4[Block 4: Sealing Gate & Episodic Memory Commit]
    B4 -.->|Feedback / Learning Loop| B2
```

Block 1: Invariant Axiom Bedrock (Moral & Ontological Grounding)
- Grounded in scripture (Exodus 31:18, Proverbs, Psalm 111:10) and immutable laws.
- Synaptic paths violating invariant boundaries are pruned a priori before forward propagation.

Block 2: Sensory Opening & Token Transduction
- Ingests natural language prompts and translates them via `AITransducer` into active AST nodes.
- Allocates neural workspace vessels in virtual memory (`Word64` cells).
```

```
### Block 3: Body Block: Cortical Execution Reel
- The `AICortex` steps through sequential rolls ($S \rightarrow S'$).
- Evaluates directional vectors (`-->`, `<--`, `<=>`), caveats (`!`), and locators (`::@`).
- If an adversative caveat (`!`) trips or an unclosed boundary leaks, the synapse fires $-1:\text{ROLLBACK}$, rolling back the transaction.

### Block 4: Closing Block: Episodic Sealing & Learning
- The `ScrollLogger` records dual-format scrolls:
  - `.scroll`: Homoiconic 4-Block replay reel of thoughts and actions.
  - `.log`: High-precision microsecond telemetry of failpoints and repairs.
- `AILearner` ingests these scrolls to update synaptic weights and episodic associations in `AIMemory`.

---

## 4. Subsystem Components in `src/ai/`

| Component | File Path | Role |
| :--- | :--- | :--- |
| **`NeuralBrain`** | `src/ai/neural/network.rs` | Multi-layer balanced-ternary network engine |
| **`AICortex`** | `src/ai/model/cortex.rs` | Cognitive coordinator managing thought formulation & nervous system |
| **`AIMemory`** | `src/ai/model/memory.rs` | Long-term associative memory storehouse & episodic graph |
| **`AILearner`** | `src/ai/model/learner.rs` | Supervised learning & weight adaptation from past execution scrolls |
| **`ScrollLogger`** | `src/ai/model/scroll.rs` | Dual-format scroll generation and telemetry recorder |
| **`AITransducer`**| `src/ai/transducer.rs` | Natural language $\leftrightarrow$ OmniCode bidirectional translator |
| **`BiblicalLexicon`** | `src/ai/lexicon.rs` | Epistemic truth anchor & theological lexicon validator |

---

## 5. Dogfooding Strategy & Training Roadmap

1. **Self-Inspection**: The AI compiles and executes its own standard library (`<TRACK 2>/lexicon/`).
2. **Grammar Expansion**: As new types are defined (e.g. `Character`, `Number`, `Container`), their verbs become active keywords in the AI's generation vocabulary.
3. **Autonomous Self-Healing**: When a failpoint occurs (e.g. `BoundaryLeak`, `CaveatVeto`), the `Restorator` repairs the code and feeds the delta to `AILearner` as affirmative learning data.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/mod.rs RS 32 LINES 116 WORDS 1.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Neural Network Subsystem Identity, Architecture Manifest & Re-exports )
// ================================================================================
//! OmniCode Neural Network Subsystem
//!
//! Exposes balanced-ternary neural network engines, synaptic weights,
//! and dogfooding adaptation mechanisms.

pub mod network;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Neural Network Components )
// ================================================================================
pub use network::{NeuralBrain, SynapticWeight, TernaryLayer, TernaryNeuron};


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Neural Network Subsystem Core Coordination )
// ================================================================================
// Neural network pathways actively linked to AICortex and AILearner.


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ================================================================================
// Neural network subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/network.rs RS 237 LINES 921 WORDS 8.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Balanced-Ternary Neurodynamics Identity, Synaptic Invariants & Axioms )
// ============================================================================
//! # OmniCode Balanced-Ternary Neural Network (`network.rs`)
//!
//! Implements a homoiconic, balanced-ternary artificial neural network operating
//! directly over the fourfold `QuadState` {-1, 0, +1, NULL}.
//!
//! ## Mathematical Principles:
//! - **Synaptic Weights**: Discrete trits $w \in \{-1, 0, +1\}$ representing inhibitory,
//!   neutral, and excitatory connections.
//! - **Accumulation**: $\Sigma = \sum w_i x_i + b$ over balanced integer sums.
//! - **Threshold Activation $\Psi(\Sigma)$**:
//!   - $\Sigma > \theta \rightarrow +1:\text{PROCEED}$ (Affirmative excitation)
//!   - $-\theta \le \Sigma \le \theta \rightarrow 0:\text{AWAIT}$ (Equilibrium / neutral hold)
//!   - $\Sigma < -\theta \rightarrow -1:\text{ROLLBACK}$ (Inhibitory veto / caveat trip)
//!   - Circuit rupture / entropy $\rightarrow \text{NULL}:\text{HALT}$ (Absolute cessation)

use crate::core::ternary::QuadState;
use crate::diagnostics::logger::DiagnosticLogger;

/// Discrete ternary synaptic connection weight.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum SynapticWeight {
    /// Inhibitory connection (-1)
    Inhibitory,
    /// Neutral / zero-bias connection (0)
    Neutral,
    /// Excitatory connection (+1)
    Excitatory,
}

impl SynapticWeight {
    /// Returns the integer trit scalar representation (-1, 0, +1).
    pub fn to_trit(self) -> i32 {
        match self {
            Self::Inhibitory => -1,
            Self::Neutral => 0,
            Self::Excitatory => 1,
        }
    }

    /// Constructs a synaptic weight from a QuadState.
    pub fn from_quad_state(state: QuadState) -> Self {
        match state {
            QuadState::Proceed => Self::Excitatory,
            QuadState::Await | QuadState::Halt => Self::Neutral,
            QuadState::Rollback => Self::Inhibitory,
        }
    }
}


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Neural Unit Definitions, Layer Topology & Brain State Initialization )
// ============================================================================
/// Individual balanced-ternary artificial neuron.
#[derive(Debug, Clone)]
pub struct TernaryNeuron {
    /// Incoming synaptic connection weights.
    pub weights: Vec<SynapticWeight>,
    /// Intrinsic neuron threshold bias (-1, 0, +1).
    pub bias: i32,
    /// Sensitivity threshold $\theta$ governing activation boundary.
    pub threshold: i32,
}

impl TernaryNeuron {
    /// Creates a new ternary neuron with initialized weights.
    pub fn new(inputs: usize, bias: i32, threshold: i32) -> Self {
        Self {
            weights: vec![SynapticWeight::Neutral; inputs],
```

```
            bias,
            threshold: threshold.max(1),
        }
    }

    /// Computes the net balanced ternary excitation and fires $\Psi(\Sigma)$.
    pub fn activate(&self, inputs: &[QuadState]) -> QuadState {
        let mut sum = self.bias;
        for (w, &inp) in self.weights.iter().zip(inputs.iter()) {
            let x = match inp {
                QuadState::Proceed => 1,
                QuadState::Await => 0,
                QuadState::Rollback => -1,
                QuadState::Halt => return QuadState::Halt,
            };
            sum += w.to_trit() * x;
        }

        if sum > self.threshold {
            QuadState::Proceed
        } else if sum < -self.threshold {
            QuadState::Rollback
        } else {
            QuadState::Await
        }
    }
}

/// A collection of parallel neurons forming a discrete layer.
#[derive(Debug, Clone)]
pub struct TernaryLayer {
    pub neurons: Vec<TernaryNeuron>,
}

impl TernaryLayer {
    /// Constructs a layer with `neuron_count` neurons each taking `inputs_per_neuron`.
    pub fn new(inputs_per_neuron: usize, neuron_count: usize, threshold: i32) -> Self {
        let neurons = (0..neuron_count)
            .map(|_| TernaryNeuron::new(inputs_per_neuron, 0, threshold))
            .collect();
        Self { neurons }
    }

    /// Forward-propagates an input vector across all neurons in the layer.
    pub fn forward(&self, inputs: &[QuadState]) -> Vec<QuadState> {
        self.neurons.iter().map(|n| n.activate(inputs)).collect()
    }
}

/// Multi-layer neural brain coordinating sensory input to cortical action.
#[derive(Debug, Clone)]
pub struct NeuralBrain {
    /// Sequential network layers (Input -> Hidden -> Output).
    pub layers: Vec<TernaryLayer>,
    /// Diagnostic telemetry logger.
    pub logger: DiagnosticLogger,
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Forward Propagation, Experience Ingestion & Dogfooding Weight Updates )
// ============================================================================
impl NeuralBrain {
    /// Constructs a new balanced-ternary [`NeuralBrain`].
    pub fn new(topology: &[usize], threshold: i32, logger: DiagnosticLogger) -> Self {
        assert!(topology.len() >= 2, "Topology requires at least input and output layers");
        let mut layers = Vec::new();
        for i in 0..(topology.len() - 1) {
            layers.push(TernaryLayer::new(topology[i], topology[i + 1], threshold));
        }
        Self { layers, logger }
    }

    /// Evaluates sensory inputs through all layers, yielding output action QuadStates.
    pub fn think(&self, inputs: &[QuadState]) -> Vec<QuadState> {
        let mut current = inputs.to_vec();
        for (idx, layer) in self.layers.iter().enumerate() {
```

```
            current = layer.forward(&current);
            self.logger.log(
                "NEURAL_BRAIN",
                format!("Layer {} forward pass: {:?}", idx + 1, current),
            );
        }
        current
    }

    /// Dogfoods past execution experience to adapt synaptic weights.
    ///
    /// Reinforces positive associations (`+1:PROCEED`) and prunes paths that led
    /// to failpoints or rollbacks (`-1:ROLLBACK`).
    pub fn adapt_synapse(
        &mut self,
        layer_idx: usize,
        neuron_idx: usize,
        weight_idx: usize,
        target_state: QuadState,
    ) {
        if let Some(layer) = self.layers.get_mut(layer_idx) {
            if let Some(neuron) = layer.neurons.get_mut(neuron_idx) {
                if let Some(w) = neuron.weights.get_mut(weight_idx) {
                    *w = SynapticWeight::from_quad_state(target_state);
                    self.logger.log(
                        "NEURAL_BRAIN",
                        format!(
                            "Synapse adapted at L{}:N{}:W{} -> {:?}",
                            layer_idx, neuron_idx, weight_idx, w
                        ),
                    );
                }
            }
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Neurodynamic Verification Proofs & Brain Integration Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_ternary_neuron_activation_dynamics() {
        let mut neuron = TernaryNeuron::new(3, 0, 1);
        neuron.weights[0] = SynapticWeight::Excitatory; // +1
        neuron.weights[1] = SynapticWeight::Excitatory; // +1
        neuron.weights[2] = SynapticWeight::Inhibitory; // -1

        // 1. Two proceed inputs overcome threshold: +1 + 1 + 0 = 2 > 1 => Proceed
        let res1 = neuron.activate(&[QuadState::Proceed, QuadState::Proceed, QuadState::Await]);
        assert_eq!(res1, QuadState::Proceed);

        // 2. Equilibrium: +1 + 0 - 1 = 0 => Await
        let res2 = neuron.activate(&[QuadState::Proceed, QuadState::Await, QuadState::Proceed]);
        assert_eq!(res2, QuadState::Await);

        // 3. Inhibitory veto: 0 + 0 - 1 = -1 < 0 (with negative inputs): -1 - 1 = -2 < -1 => Rollback
        let res3 = neuron.activate(&[QuadState::Rollback, QuadState::Rollback, QuadState::Await]);
        assert_eq!(res3, QuadState::Rollback);
    }

    #[test]
    fn test_neural_brain_forward_pass_and_dogfooding_adaptation() {
        let logger = DiagnosticLogger::new(false);
        let mut brain = NeuralBrain::new(&[3, 4, 2], 1, logger);

        // Initial think pass on neutral equilibrium inputs
        let out1 = brain.think(&[QuadState::Proceed, QuadState::Proceed, QuadState::Proceed]);
        assert_eq!(out1.len(), 2);

        // Adapt synaptic connections to dogfood verified execution
        brain.adapt_synapse(0, 0, 0, QuadState::Proceed);
        brain.adapt_synapse(0, 0, 1, QuadState::Proceed);
```

```
        brain.adapt_synapse(1, 0, 0, QuadState::Proceed);

        let out2 = brain.think(&[QuadState::Proceed, QuadState::Proceed, QuadState::Proceed]);
        assert_eq!(out2.len(), 2);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/mod.rs RS 42 LINES 146 WORDS 1.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( CLI Subsystem Identity, Module Architecture & Core Re-exports )
// ============================================================================
//! OmniCode Command Line Subsystem
//!
//! Exposes interactive terminal REPL, script runners, structural 4-block linters,
//! and template scaffolding generators.

pub mod repl;
pub mod server;
pub mod commands;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations & Public Interface )
// ============================================================================
pub use repl::run_repl;
pub use server::run_gui_server;
pub use commands::{
    BootArch, BootConfig, BootContext, run_boot, run_boot_with_config,
    execute_kernel_pipeline, synthesize_hypervisor_command, execute_boot_pipeline, execute_boot_machine,
    DEFAULT_BOOT_ARCH, DEFAULT_GRAPHICAL_MODE, REAL_MODE_BOOT_VECTOR, VGA_TEXT_FRAMEBUFFER_BASE, COM1_SERIAL_IO_BASE,
    AssembleFormat, AssembleConfig, AssembleContext, run_assemble, run_assemble_with_config,
    execute_assemble_pipeline, serialize_assembled_output,
};


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( CLI Orchestration Framework )
// ============================================================================
// Command line entrypoints route to interactive REPL or single-file runners.


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing )
// ============================================================================
// CLI subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/repl.rs

RS | 427 LINES | 1585 WORDS | 21.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Interactive Terminal Environment Identity, REPL Architecture & Dependencies )
// ============================================================================
//! OmniCode Terminal REPL: Interactive Ontological Shell
//!
//! Provides an interactive read-eval-print loop (REPL) enabling direct,
//! zero-latency experimentation with OmniCode's foundational primitives:
//!     - Balanced Ternary Logic (-1 Rollback, 0 Await, +1 Proceed, NULL Halt)
//!     - 6 Boundary Morphologies & Asymmetric Closures
//!     - First-Class Intent Asides (...)
//!     - 3D Coordinate Operators & Directional Vectors
//!     - Transactional Environment Snapshot & Rollback
//!     - Real-Time Diagnostic Failpoint Trapping

use std::fs;
use std::path::PathBuf;
use rustyline::error::ReadlineError;
use rustyline::DefaultEditor;

use crate::ast::nodes::ASTNode;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::diagnostics::linter::OmniLinter;
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::Evaluator;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( REPL Configuration, Banner Display, and Context State Initialization )
// ============================================================================
/// Resolves the file path for persistent terminal command history.
fn get_history_path() -> Option<PathBuf> {
    std::env::var("HOME").ok().map(|h| PathBuf::from(h).join(".omnicode_history"))
}

/// Checks whether an interactive input buffer has balanced delimiters.
fn is_input_complete(input: &str) -> bool {
    let trimmed = input.trim_end();
    if trimmed.ends_with('\\') {
        return false;
    }

    let mut open_brackets = 0isize;
    let mut in_string = false;
    let mut quote_char = ' ';

    for line in input.lines() {
        let stripped = line.trim();
        if stripped.starts_with('#') {
            continue;
        }

        let chars: Vec<char> = line.chars().collect();
        let mut i = 0;
        while i < chars.len() {
            let c = chars[i];
            if in_string {
                if c == '\\' {
                    i += 2;
                    continue;
                }
                if c == quote_char {
                    in_string = false;
                }
            } else {
                match c {
                    '#' => break, // Comment extends to end of line
                    '"' | '\'' => {
                        in_string = true;
```

```
                        quote_char = c;
                    }
                    '[' | '{' | '<' | '(' => open_brackets += 1,
                    ']' | '}' | '>' | ')' => open_brackets -= 1,
                    _ => {}
                }
            }
            i += 1;
        }
    }

    open_brackets <= 0
}

/// Displays the welcome banner and prompt instructions.
fn print_welcome_banner() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode Interactive Shell (v0.1.0)\x1b[0m");
    println!("\x1b[1;36m  Core: Balanced Ternary & Boundary Physics (Rust Engine)\x1b[0m");
    println!("\x1b[0;37m  Type \x1b[1;33m:help\x1b[0m\x1b[0;37m or \x1b[1;33m:?\x1b[0m\x1b[0;37m for commands & syntax cheat sheet. Press \x1b[1;33mCtrl+D\x1b[0m\x1b[0;37m to exit.\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");
}

/// Displays the help guide and syntax reference.
fn print_help_guide() {
    println!("\x1b[1;33m--- OmniCode Interactive Commands ---\x1b[0m");
    println!("  \x1b[1;32m:help\x1b[0m, \x1b[1;32m:?\x1b[0m              Show this help reference");
    println!("  \x1b[1;32m:env\x1b[0m                  Inspect all active symbols and entity bindings");
    println!("  \x1b[1;32m:ast <expr>\x1b[0m           Display raw AST structure of an expression");
    println!("  \x1b[1;32m:load <path>\x1b[0m          Load, lint, parse, and evaluate an OmniCode file (.omni / .scroll)");
    println!("  \x1b[1;32m:lint <path>\x1b[0m          Validate file against 4-Block architecture (.omni / .scroll)");
    println!("  \x1b[1;32m:failpoints\x1b[0m           Review all failpoint incidents trapped in this session");
    println!("  \x1b[1;32m:verbose [on|off]\x1b[0m     Toggle real-time microsecond diagnostic logging");
    println!("  \x1b[1;32m:clear\x1b[0m                Clear the terminal screen");
    println!("  \x1b[1;32m:reset\x1b[0m                Reset the environment and diagnostic stack");
    println!("  \x1b[1;32m:exit\x1b[0m, \x1b[1;32m:quit\x1b[0m          Exit the OmniCode interactive shell\n");

    println!("\x1b[1;33m--- Notation & Physics Cheat Sheet ---\x1b[0m");
    println!("  \x1b[1;36mBoundaries:\x1b[0m   [...] mutable | <...> immutable axiom | {{...}} context");
    println!("                {{...] top-down asymmetric | [...}} bottom-up asymmetric | (...) intent aside");
    println!("  \x1b[1;36mTernary:\x1b[0m      +1 (Proceed) | 0 (Await) | -1 (Rollback) | NULL (Halt)");
    println!("  \x1b[1;36mRelations:\x1b[0m    = (Is Identity) | != (Is Not) | ~= (To Be Like)");
    println!("  \x1b[1;36mVectors:\x1b[0m      --> (Act Upon / Forward) | <-- (Act Again / Rollback)");
    println!("                ^-- (Builds To) | v-- (Breaks Down From)");
    println!("  \x1b[1;36mConnectives:\x1b[0m  ! (Adversative Caveat BUT) | :: (Conjunction AND) | | (Disjunction OR)\n");
}


// ==============================================================================
// BLOCK 3: BODY BLOCK
// ( Core REPL Execution Loop: Command Dispatch, Parser, Evaluator Pipeline )
// ==============================================================================
/// Launches and executes the OmniCode interactive REPL terminal session.
pub fn run_repl() -> Result<(), Box<dyn std::error::Error>> {
    print_welcome_banner();

    let mut rl = DefaultEditor::new()?;
    let history_file = get_history_path();
    if let Some(ref path) = history_file {
        let _ = rl.load_history(path);
    }

    let logger = DiagnosticLogger::new(false);
    let mut debugger = OmniDebugger::new(logger.clone(), 10);
    let mut env = Environment::new();
    let mut current_state = QuadState::Proceed;
    let mut verbose = false;
    let mut multiline_buffer = String::new();

    loop {
        let prompt = if multiline_buffer.is_empty() {
            match current_state {
                QuadState::Proceed => "\x1b[1;32momni[+1]>\x1b[0m ".to_string(),
                QuadState::Await => "\x1b[1;33momni[0]>\x1b[0m ".to_string(),
                QuadState::Rollback => "\x1b[1;31momni[-1]>\x1b[0m ".to_string(),
```

```
                QuadState::Halt => "\x1b[1;35momni[NULL]>\x1b[0m ".to_string(),
            }
        } else {
            "\x1b[1;30m   ...>\x1b[0m ".to_string()
        };

        let readline = rl.readline(&prompt);
        match readline {
            Ok(line) => {
                let trimmed = line.trim();

                // Multiline buffering logic
                if !multiline_buffer.is_empty() {
                    if trimmed.is_empty() {
                        // Empty line forces execution of buffered input
                    } else {
                        multiline_buffer.push('\n');
                        multiline_buffer.push_str(&line);
                        if !is_input_complete(&multiline_buffer) {
                            continue;
                        }
                    }
                } else if !trimmed.is_empty() && !trimmed.starts_with(':') && !is_input_complete(trimmed) {
                    multiline_buffer.push_str(&line);
                    continue;
                }

                let input = if !multiline_buffer.is_empty() {
                    let full = multiline_buffer.clone();
                    multiline_buffer.clear();
                    full
                } else {
                    line
                };

                let input_trimmed = input.trim();
                if input_trimmed.is_empty() {
                    continue;
                }

                let _ = rl.add_history_entry(input_trimmed);

                // Command Dispatch
                if input_trimmed.starts_with(':') || input_trimmed == "exit" || input_trimmed == "quit" {
                    let cmd_parts: Vec<&str> = input_trimmed.splitn(2, ' ').collect();
                    let cmd = cmd_parts[0];
                    let arg = if cmd_parts.len() > 1 { cmd_parts[1].trim() } else { "" };

                    match cmd {
                        ":help" | ":?" => print_help_guide(),
                        ":exit" | ":quit" | "exit" | "quit" => {
                            println!("\x1b[1;36mSealing OmniCode session. Goodbye.\x1b[0m");
                            break;
                        }
                        ":clear" => {
                            print!("\x1b[2J\x1b[1;1H");
                        }
                        ":reset" => {
                            env.clear();
                            logger.clear();
                            debugger = OmniDebugger::new(logger.clone(), 10);
                            current_state = QuadState::Proceed;
                            println!("\x1b[1;32m✓ Session reset: Environment and failpoints cleared.\x1b[0m");
                        }
                        ":env" => {
                            let bindings = env.bindings();
                            if bindings.is_empty() {
                                println!("\x1b[0;37m(Environment is empty)\x1b[0m");
                            } else {
                                println!("\x1b[1;33m--- Active Symbols & Bindings ({}) ---\x1b[0m", bindings.len());
                                for (k, v) in bindings {
                                    println!("  \x1b[1;36m{}\x1b[0m = {}", k, v.format_colored());
                                }
                            }
                        }
                        ":tokens" => {
                            if arg.is_empty() {
                                eprintln!("\x1b[1;31mUsage:\x1b[0m :tokens <expression>");
```

```
                                } else {
                                    let mut lex = Lexer::new(arg, logger.clone());
                                    match lex.tokenize() {
                                        Ok(toks) => {
                                            println!("\x1b[1;33mTokens ({}):\x1b[0m", toks.len());
                                            for t in toks {
                                                println!("  {:?}  \x1b[0;37m('{}' at L{}:C{})\x1b[0m", t.token_type,
t.lexeme, t.line, t.col);
                                            }
                                        }
                                        Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error:\x1b[0m {}", e),
                                    }
                                }
                            }
                            ":ast" => {
                                if arg.is_empty() {
                                    eprintln!("\x1b[1;31mUsage:\x1b[0m :ast <expression>");
                                } else {
                                    let mut lex = Lexer::new(arg, logger.clone());
                                    match lex.tokenize() {
                                        Ok(toks) => {
                                            let mut p = Parser::new(toks, debugger.clone());
                                            match p.parse_program() {
                                                Ok(ast) => println!("{:#?}", ast),
                                                Err(e) => eprintln!("\x1b[1;31m✗ Parser Error:\x1b[0m {}", e),
                                            }
                                        }
                                        Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error:\x1b[0m {}", e),
                                    }
                                }
                            }
                            ":failpoints" => {
                                let fps = logger.failpoints();
                                if fps.is_empty() {
                                    println!("\x1b[1;32m✓ No failpoint incidents recorded.\x1b[0m");
                                } else {
                                    println!("\x1b[1;33m--- Trapped Failpoint Incidents ({}) ---\x1b[0m", fps.len());
                                    for fp in fps {
                                        let fp_name = fp.failpoint_type.map(|t| t.as_str()).unwrap_or("GENERIC");
                                        println!("  * [{} µs] \x1b[1;31m[{}]\x1b[0m [{}] {}", fp.elapsed_micros, fp.channel,
fp_name, fp.message);
                                    }
                                }
                            }
                            ":verbose" => {
                                if arg == "on" {
                                    verbose = true;
                                    println!("\x1b[1;32m✓ Diagnostic verbose logging ENABLED.\x1b[0m");
                                } else if arg == "off" {
                                    verbose = false;
                                    println!("\x1b[1;33m✓ Diagnostic verbose logging DISABLED.\x1b[0m");
                                } else {
                                    verbose = !verbose;
                                    println!("Diagnostic verbose logging is now: \x1b[1;36m{}\x1b[0m", if verbose { "ON" }
else { "OFF" });
                                }
                            }
                            ":lint" => {
                                if arg.is_empty() {
                                    eprintln!("\x1b[1;31mUsage:\x1b[0m :lint <file.omni | file.scroll>");
                                } else {
                                    match fs::read_to_string(arg) {
                                        Ok(content) => {
                                            let linter = OmniLinter::new(logger.clone());
                                            let report = linter.lint_source(arg, &content);
                                            if report.is_valid {
                                                println!("\x1b[1;32m✓ 4-Block Structural Check Passed:\x1b[0m {} blocks
verified in canonical sequence.", report.blocks_found.len());
                                            } else {
                                                println!("\x1b[1;31m✗ 4-Block Structural Linter Violations:\x1b[0m");
                                                for err in report.errors {
                                                    println!("    \x1b[1;31m•\x1b[0m {}", err);
                                                }
                                            }
                                        }
                                        Err(e) => eprintln!("\x1b[1;31m✗ File read error:\x1b[0m {}", e),
                                    }
                                }
```

```
                        }
                        ":load" => {
                            if arg.is_empty() {
                                eprintln!("\x1b[1;31mUsage:\x1b[0m :load <file.omni | file.scroll>");
                            } else {
                                match fs::read_to_string(arg) {
                                    Ok(source) => {
                                        let prev_fps_len = logger.failpoints().len();
                                        let mut lex = Lexer::new(&source, logger.clone());
                                        match lex.tokenize() {
                                            Ok(toks) => {
                                                let mut p = Parser::new(toks, debugger.clone());
                                                match p.parse_program() {
                                                    Ok(prog) => {
                                                        let mut eval = Evaluator::new(env.clone(), p.debugger);
                                                        match eval.eval(&ASTNode::Program(prog)) {
                                                            Ok(res) => {
                                                                env = eval.env;
                                                                debugger = eval.debugger;
                                                                if let RuntimeValue::State(s) = res {
                                                                    current_state = s;
                                                                }
                                                                println!("\x1b[1;32m✓ Loaded and evaluated '{}':\x1b[0m
=> {}", arg, res.format_colored());
                                                            }
                                                            Err(e) => eprintln!("\x1b[1;31m✗ Evaluation Error in
'{}':\x1b[0m {}", arg, e),
                                                        }
                                                    }
                                                    Err(e) => eprintln!("\x1b[1;31m✗ Parser Error in '{}':\x1b[0m {}",
arg, e),
                                                }
                                            }
                                            Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error in '{}':\x1b[0m {}", arg, e),
                                        }
                                        let new_fps = logger.failpoints();
                                        if new_fps.len() > prev_fps_len {
                                            for fp in &new_fps[prev_fps_len..] {
                                                let fp_name = fp.failpoint_type.map(|t|
t.as_str()).unwrap_or("FAILPOINT");
                                                println!("  \x1b[1;33m[FAILPOINT: {}]\x1b[0m {}", fp_name, fp.message);
                                            }
                                        }
                                    }
                                    Err(e) => eprintln!("\x1b[1;31m✗ File read error:\x1b[0m {}", e),
                                }
                            }
                        }
                        other => {
                            eprintln!("\x1b[1;31mUnknown command:\x1b[0m '{}'. Type :help for commands.", other);
                        }
                    }
                    continue;
                }

                // OmniCode Expression / Statement Evaluation Pipeline
                let prev_fps_len = logger.failpoints().len();
                let mut lexer = Lexer::new(input_trimmed, logger.clone());
                match lexer.tokenize() {
                    Ok(tokens) => {
                        let mut parser = Parser::new(tokens, debugger.clone());
                        match parser.parse_program() {
                            Ok(prog) => {
                                let mut evaluator = Evaluator::new(env.clone(), parser.debugger);
                                match evaluator.eval(&ASTNode::Program(prog)) {
                                    Ok(val) => {
                                        env = evaluator.env;
                                        debugger = evaluator.debugger;
                                        if let RuntimeValue::State(s) = val {
                                            current_state = s;
                                        }
                                        println!("=> {}", val.format_colored());
                                    }
                                    Err(err) => {
                                        eprintln!("\x1b[1;31m✗ Runtime Error:\x1b[0m {}", err);
                                    }
                                }
                            }
```

```
                            Err(parse_err) => {
                                eprintln!("\x1b[1;31m✗ Syntax Error:\x1b[0m {}", parse_err);
                            }
                        }
                    }
                    Err(lex_err) => {
                        eprintln!("\x1b[1;31m✗ Lexical Error:\x1b[0m {}", lex_err);
                    }
                }

                // Check and print any failpoint alerts triggered during this line
                let all_fps = logger.failpoints();
                if all_fps.len() > prev_fps_len {
                    for fp in &all_fps[prev_fps_len..] {
                        let fp_name = fp.failpoint_type.map(|t| t.as_str()).unwrap_or("FAILPOINT");
                        println!("  \x1b[1;33m[FAILPOINT: {}]\x1b[0m {}", fp_name, fp.message);
                    }
                }

                if verbose {
                    println!("\x1b[0;37m(Active bindings: {}, snapshot depth: {})\x1b[0m", env.bindings().len(),
debugger.boundary_stack.len());
                }
            }
            Err(ReadlineError::Interrupted) => {
                println!("\x1b[1;33m^C (Interrupted. Type :exit or press Ctrl+D to quit)\x1b[0m");
                multiline_buffer.clear();
            }
            Err(ReadlineError::Eof) => {
                println!("\x1b[1;36mSealing OmniCode session. Goodbye.\x1b[0m");
                break;
            }
            Err(err) => {
                eprintln!("\x1b[1;31mTerminal Error:\x1b[0m {:?}", err);
                break;
            }
        }
    }

    if let Some(ref path) = history_file {
        let _ = rl.save_history(path);
    }

    Ok(())
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Session Invariants, Teardown Proofs, and Terminal Teardown )
// ============================================================================
// Terminal REPL engine verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/server.rs

RS | 576 LINES | 1855 WORDS | 23.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode GUI Server Identity, Zero-Dependency Architecture & REST Invariants )
// ============================================================================
//! OmniCode Local Web GUI Server & Terminal Gateway
//!
//! Provides a pure standard-library HTTP server (0 external crates) hosting the
//! local OmniCode Studio GUI and Terminal Interface:
//!   - Static file delivery: `gui/index.html`, `gui/style.css`, `gui/app.js`
//!   - REST endpoints:
//!     * `POST /api/prompt`    -> Executes AICortex neural-symbolic cognitive pipeline
//!     * `POST /api/terminal`  -> Executes CLI commands directly (`omnicode run/lint/demo`)
//!     * `GET  /api/manifest`  -> Returns canonical `manifest.omni` covenant anchor
//!     * `GET  /api/lexicon`   -> Returns offline Biblical world model & passage citations
//!     * `GET  /api/scrolls`   -> Returns episodic memory learning statistics

use std::fs;
use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};
use std::path::Path;
use std::sync::{Arc, Mutex};
use std::thread;

use crate::ai::{AICortex, AILearner, BiblicalLexicon};
use crate::core::ternary::QuadState;
use crate::diagnostics::hover::HoverEngine;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::terminal::{create_shared_scheduler, SharedScheduler, TerminalSession};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Server Configuration, HTTP Parser & Response Structures )
// ============================================================================

pub struct GuiServerConfig {
    pub host: String,
    pub port: u16,
    pub gui_dir: String,
}

impl Default for GuiServerConfig {
    fn default() -> Self {
        Self {
            host: "127.0.0.1".to_string(),
            port: 8080,
            gui_dir: "gui".to_string(),
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( TCP Listener Loop, HTTP Routing, and REST API Handlers )
// ============================================================================

/// Starts the OmniCode GUI HTTP server on the specified port.
pub fn run_gui_server(port: u16) -> Result<(), String> {
    let host = std::env::var("OMNICODE_HOST").unwrap_or_else(|_| "0.0.0.0".to_string());
    let listener = TcpListener::bind(format!("{host}:{port}"))
        .map_err(|e| format!("Failed to bind TCP listener to {host}:{port}: {e}"))?;

    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode GUI Studio & Terminal Server Active                                    \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;32m✓ Web Studio URL:\x1b[0m http://localhost:{port} (listening on {host}:{port})");
    println!("\x1b[1;33m✓ Backend Mode:\x1b[0m 100% Offline | Grounded in Biblical Lexicon & Covenant Manifest");
    println!("\x1b[1;35m✓ Terminal Gateway:\x1b[0m Active (CLI commands executable via GUI)\n");
    println!("Press Ctrl+C to terminate server.\n");

    let lexicon = Arc::new(BiblicalLexicon::new());
    let scheduler = create_shared_scheduler();
    let session = Arc::new(Mutex::new(TerminalSession::new(Arc::clone(&scheduler))));
```

```

    for stream in listener.incoming() {
        match stream {
            Ok(stream) => {
                let lex_clone = Arc::clone(&lexicon);
                let sched_clone = Arc::clone(&scheduler);
                let sess_clone = Arc::clone(&session);
                thread::spawn(move || {
                    handle_connection(stream, lex_clone, sched_clone, sess_clone);
                });
            }
            Err(e) => {
                eprintln!("Connection failed: {e}");
            }
        }
    }

    Ok(())
}

fn handle_connection(
    mut stream: TcpStream,
    lexicon: Arc<BiblicalLexicon>,
    scheduler: SharedScheduler,
    session: Arc<Mutex<TerminalSession>>,
) {
    let mut buffer = [0u8; 16384];
    let bytes_read = match stream.read(&mut buffer) {
        Ok(n) if n > 0 => n,
        _ => return,
    };

    let request_str = String::from_utf8_lossy(&buffer[..bytes_read]);
    let mut lines = request_str.lines();
    let request_line = match lines.next() {
        Some(line) => line,
        std::option::Option::None => return,
    };

    let parts: Vec<&str> = request_line.split_whitespace().collect();
    if parts.len() < 2 {
        return;
    }

    let method = parts[0];
    let raw_path = parts[1];
    let path = raw_path.split('?').next().unwrap_or(raw_path);
    let is_head = method == "HEAD";
    let route_method = if is_head { "GET" } else { method };

    // Extract body if POST
    let body = if method == "POST" {
        if let Some(pos) = request_str.find("\r\n\r\n") {
            request_str[pos + 4..].to_string()
        } else if let Some(pos) = request_str.find("\n\n") {
            request_str[pos + 2..].to_string()
        } else {
            String::new()
        }
    } else {
        String::new()
    };

    // Route Request
    match (route_method, path) {
        ("GET", "/") | ("GET", "/studio") | ("GET", "/index.html") | ("GET", "/studio.html") => {
            serve_file(&mut stream, "apps/gui/studio.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/terminal") | ("GET", "/terminal.html") => {
            serve_file(&mut stream, "apps/gui/terminal.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/specs") | ("GET", "/specs.html") => {
            serve_file(&mut stream, "apps/gui/specs.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/model") | ("GET", "/model.html") => {
            serve_file(&mut stream, "apps/gui/model.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/lexicon") | ("GET", "/lexicon.html") => {
```

```
            serve_file(&mut stream, "apps/gui/lexicon.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/scrolls") | ("GET", "/scrolls.html") => {
            serve_file(&mut stream, "apps/gui/scrolls.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/cloud") | ("GET", "/cloud.html") => {
            serve_file(&mut stream, "apps/gui/cloud.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/download") | ("GET", "/download.html") => {
            serve_file(&mut stream, "apps/gui/download.html", "text/html; charset=utf-8", is_head);
        }
        ("GET", "/terminal.js") => {
            serve_file(&mut stream, "apps/gui/terminal.js", "application/javascript; charset=utf-8", is_head);
        }
        ("GET", "/style.css") => {
            serve_file(&mut stream, "apps/gui/style.css", "text/css; charset=utf-8", is_head);
        }
        ("GET", "/app.js") => {
            serve_file(&mut stream, "apps/gui/app.js", "application/javascript; charset=utf-8", is_head);
        }
        ("GET", "/api/health") => {
            let json = r#"{"status":"healthy","engine":"OmniCode","version":"0.1.0","state":"+1:PROCEED"}"#;
            send_json_response(&mut stream, 200, json, is_head);
        }
        ("GET", "/api/download") => {
            let archive_path = "dist/omnicode-terminal-linux-x86_64.tar.gz";
            if let Ok(bytes) = fs::read(archive_path) {
                let header = format!(
                    "HTTP/1.1 200 OK\r\nContent-Type: application/gzip\r\nContent-Disposition: attachment;
filename=\"omnicode-terminal-linux-x86_64.tar.gz\"\r\nContent-Length: {}\r\nAccess-Control-Allow-Origin: *\r\nConnection:
close\r\n\r\n",
                    bytes.len()
                );
                let _ = stream.write_all(header.as_bytes());
                if !is_head {
                    let _ = stream.write_all(&bytes);
                }
            } else {
                let err = r#"{"error":"Distribution archive not found. Run scripts/package_native.sh to build."}"#;
                send_json_response(&mut stream, 404, err, is_head);
            }
        }
        ("GET", "/api/scheduler") => {
            let json = scheduler.lock().unwrap().to_json();
            send_json_response(&mut stream, 200, &json, is_head);
        }
        ("POST", "/api/scheduler/spawn") => {
            let cmd_str = extract_json_field(&body, "command").unwrap_or_else(|| "omnicode demo".to_string());
            let name_str = extract_json_field(&body, "name").unwrap_or_else(|| "UserTask".to_string());
            let prio_str = extract_json_field(&body, "priority").unwrap_or_else(|| "NORMAL".to_string());
            let priority = match prio_str.to_uppercase().as_str() {
                "HIGH" => crate::terminal::TaskPriority::High,
                "LOW" => crate::terminal::TaskPriority::Low,
                "BACKGROUND" => crate::terminal::TaskPriority::Background,
                _ => crate::terminal::TaskPriority::Normal,
            };
            let mut sched = scheduler.lock().unwrap();
            let pid = sched.spawn(&name_str, &cmd_str, priority);
            let json = format!(r#"{{"pid":{},"status":"spawned"}}"#, pid);
            send_json_response(&mut stream, 200, &json, is_head);
        }
        ("GET", "/api/manifest") => {
            let manifest_content = fs::read_to_string("manifest.omni")
                .unwrap_or_else(|_| "# Manifest not found".to_string());
            let json = format!(
                r#"{{"manifest": "{}"}}"#,
                escape_json_string(&manifest_content)
            );
            send_json_response(&mut stream, 200, &json, is_head);
        }
        ("GET", "/api/lexicon") => {
            let mut entries_json = Vec::new();
            for e in lexicon.all_entries() {
                entries_json.push(format!(
                    r#"{{"term":"{}","domain":"{}","definition":"{}","kjv":"{}","web":"{}","state":"{}","guidance":"
{}"}}"#,
                    escape_json_string(e.term),
                    escape_json_string(e.domain.as_str()),
```

```
                        escape_json_string(e.definition),
                        escape_json_string(e.kjv_passage),
                        escape_json_string(e.web_passage),
                        escape_json_string(e.quad_state_resonance.macro_op()),
                        escape_json_string(e.moral_guidance)
                    ));
                }
                let json = format!(r#"{{"count":{},"entries":[{}]}}"#, entries_json.len(), entries_json.join(","));
                send_json_response(&mut stream, 200, &json, is_head);
            }
            ("GET", "/api/scrolls") => {
                let scroll_dir = if Path::new("storage/scrolls").exists() {
                    "storage/scrolls"
                } else if Path::new("runtime/scrolls").exists() {
                    "runtime/scrolls"
                } else {
                    "scrolls"
                };
                let learner = AILearner::new(scroll_dir);
                let summary = learner.digest_experiences().unwrap_or_default();
                let mut exp_json = Vec::new();
                for e in &summary.experiences {
                    exp_json.push(format!(
                        r#"{{"session_id":"{}","prompt":"{}","state":"{}","repairs":{},"failpoints":{},"scroll":"{}"}}"#,
                        escape_json_string(&e.session_id),
                        escape_json_string(&e.prompt),
                        escape_json_string(e.final_state.macro_op()),
                        e.repair_count,
                        e.failpoint_count,
                        escape_json_string(&e.scroll_path.to_string_lossy())
                    ));
                }
                let json = format!(
                    r#"{{"total":{},"proceed":{},"rollback":{},"await":{},"halt":{},"repairs_learned":{},"episodes":[{}]}}"#,
                    summary.total_episodes, summary.proceed_count, summary.rollback_count, summary.await_count,
summary.halt_count,
                    summary.total_repairs_learned, exp_json.join(",")
                );
                send_json_response(&mut stream, 200, &json, is_head);
            }
            ("POST", "/api/prompt") => {
                let prompt = extract_json_field(&body, "prompt").unwrap_or_else(|| body.clone());
                let mut cortex = AICortex::new();
                match cortex.process_prompt(&prompt) {
                    Ok(res) => {
                        let mut repairs_json = Vec::new();
                        for r in &res.repairs_applied {
                            repairs_json.push(format!(
                                r#"{{"line":{},"kind":"{:?}","desc":"{}"}}"#,
                                r.line, r.kind, escape_json_string(&r.description)
                            ));
                        }
                        let json = format!(
                            r#"{{"session_id":"{}","prompt":"{}","state":"{}","response":"{}","omnicode":"{}","scroll":"
{}","log":"{}","repairs":[{}]}}"#,
                            escape_json_string(&res.session_id),
                            escape_json_string(&res.prompt),
                            escape_json_string(res.execution_state.macro_op()),
                            escape_json_string(&res.natural_response),
                            escape_json_string(&res.synthesized_omnicode),
                            escape_json_string(&res.scroll_path.to_string_lossy()),
                            escape_json_string(&res.log_path.to_string_lossy()),
                            repairs_json.join(",")
                        );
                        send_json_response(&mut stream, 200, &json, is_head);
                    }
                    Err(e) => {
                        let json = format!(r#"{{"error":"{}"}}"#, escape_json_string(&e));
                        send_json_response(&mut stream, 500, &json, is_head);
                    }
                }
            }
            ("GET", "/api/system") => {
                let json = r#"{"project":"OmniCode","organization":"CreativeWorkzStudio
LLC","version":"0.1.0","offline":true,"engine":"Rust Core 2024 (Balanced Ternary Quad-State)"}"#;
                send_json_response(&mut stream, 200, json, is_head);
            }
            ("GET", "/api/hover") => {
```

```
            let file = extract_query_param(raw_path, "file");
            let symbol = extract_query_param(raw_path, "symbol");
            let line = extract_query_param(raw_path, "line")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);
            let col = extract_query_param(raw_path, "col")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);

            let mut engine = HoverEngine::new(DiagnosticLogger::new(false));
            engine.index_workspace(".");

            let card = if let Some(ref sym) = symbol {
                engine.document_symbol(sym, file.as_deref())
            } else if let Some(ref f) = file {
                engine.hover_at(f, line, col)
            } else {
                None
            };

            match card {
                Some(c) => send_json_response(&mut stream, 200, &c.to_json(), is_head),
                None => {
                    let err = r#"{"symbol":"","found":false,"error":"No symbol found or target could not be resolved"}"#;
                    send_json_response(&mut stream, 404, err, is_head);
                }
            }
        }
        ("POST", "/api/hover") => {
            let file = extract_json_field(&body, "file");
            let symbol = extract_json_field(&body, "symbol");
            let line = extract_json_field(&body, "line")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);
            let col = extract_json_field(&body, "col")
                .and_then(|s| s.parse::<usize>().ok())
                .unwrap_or(1);

            let mut engine = HoverEngine::new(DiagnosticLogger::new(false));
            engine.index_workspace(".");

            let card = if let Some(ref sym) = symbol {
                engine.document_symbol(sym, file.as_deref())
            } else if let Some(ref f) = file {
                engine.hover_at(f, line, col)
            } else {
                None
            };

            match card {
                Some(c) => send_json_response(&mut stream, 200, &c.to_json(), is_head),
                None => {
                    let err = r#"{"symbol":"","found":false,"error":"No symbol found or target could not be resolved"}"#;
                    send_json_response(&mut stream, 404, err, is_head);
                }
            }
        }
        ("POST", "/api/terminal") => {
            let cmd_str = extract_json_field(&body, "command").unwrap_or_else(|| body.clone());
            let clean_cmd = cmd_str.trim();

            let res = {
                let mut s = session.lock().unwrap();
                s.execute(clean_cmd)
            };

            let state_str = match res.state {
                QuadState::Proceed => "+1:PROCEED",
                QuadState::Await => "0:AWAIT",
                QuadState::Rollback => "-1:ROLLBACK",
                QuadState::Halt => "NULL:HALT",
            };

            let json = format!(
                r#"{{"stdout":"{}","state":"{}","duration_us":{}}}"#,
                escape_json_string(&res.stdout),
                state_str,
                res.duration_us
```

```
            );
            send_json_response(&mut stream, 200, &json, is_head);
        }
        ("OPTIONS", _) => {
            // Handle CORS preflight
            let response = "HTTP/1.1 204 No Content\r\n\
                Access-Control-Allow-Origin: *\r\n\
                Access-Control-Allow-Methods: GET, POST, OPTIONS, HEAD\r\n\
                Access-Control-Allow-Headers: Content-Type\r\n\r\n";
            stream.write_all(response.as_bytes()).ok();
        }
        _ => {
            let response = "HTTP/1.1 404 Not Found\r\nContent-Length: 9\r\n\r\nNot Found";
            stream.write_all(response.as_bytes()).ok();
        }
    }
}

fn serve_file(stream: &mut TcpStream, file_path: &str, content_type: &str, is_head: bool) {
    let resolved_path = if Path::new(file_path).exists() {
        file_path.to_string()
    } else if file_path.starts_with("gui/") {
        let alt = format!("apps/{file_path}");
        if Path::new(&alt).exists() {
            alt
        } else {
            file_path.to_string()
        }
    } else {
        file_path.to_string()
    };

    if !Path::new(&resolved_path).exists() {
        let not_found = format!(
            "HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\nContent-Length: {}\r\n\r\nFile Not Found: {}",
            resolved_path.len() + 16, resolved_path
        );
        stream.write_all(not_found.as_bytes()).ok();
        return;
    }

    match fs::read(&resolved_path) {
        Ok(data) => {
            let header = format!(
                "HTTP/1.1 200 OK\r\n\
                Content-Type: {}\r\n\
                Content-Length: {}\r\n\
                Access-Control-Allow-Origin: *\r\n\
                Connection: close\r\n\r\n",
                content_type, data.len()
            );
            stream.write_all(header.as_bytes()).ok();
            if !is_head {
                stream.write_all(&data).ok();
            }
        }
        Err(e) => {
            let err_body = format!("Error reading file: {e}");
            let header = format!(
                "HTTP/1.1 500 Internal Server Error\r\nContent-Length: {}\r\n\r\n{}",
                err_body.len(), err_body
            );
            stream.write_all(header.as_bytes()).ok();
        }
    }
}

fn send_json_response(stream: &mut TcpStream, status_code: u16, json: &str, is_head: bool) {
    let status_line = match status_code {
        200 => "200 OK",
        400 => "400 Bad Request",
        404 => "404 Not Found",
        500 => "500 Internal Server Error",
        _ => "200 OK",
    };

    let header = format!(
        "HTTP/1.1 {}\r\n\
```

```
        Content-Type: application/json; charset=utf-8\r\n\
        Content-Length: {}\r\n\
        Access-Control-Allow-Origin: *\r\n\
        Connection: close\r\n\r\n",
        status_line, json.as_bytes().len()
    );
    stream.write_all(header.as_bytes()).ok();
    if !is_head {
        stream.write_all(json.as_bytes()).ok();
    }
}

fn escape_json_string(s: &str) -> String {
    let mut out = String::with_capacity(s.len() + 10);
    for c in s.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            '\u{0008}' => out.push_str("\\b"),
            '\u{000C}' => out.push_str("\\f"),
            c if c.is_control() => {
                out.push_str(&format!("\\u{:04x}", c as u32));
            }
            c => out.push(c),
        }
    }
    out
}

fn extract_json_field(json: &str, field_name: &str) -> Option<String> {
    let key = format!("\"{}\":", field_name);
    let key_pos = json.find(&key)?;
    let remainder = json[key_pos + key.len()..].trim_start();

    if remainder.starts_with('"') {
        let mut result = String::new();
        let chars = remainder[1..].chars();
        let mut escaped = false;
        for c in chars {
            if escaped {
                match c {
                    '"' => result.push('"'),
                    '\\' => result.push('\\'),
                    'n' => result.push('\n'),
                    'r' => result.push('\r'),
                    't' => result.push('\t'),
                    other => result.push(other),
                }
                escaped = false;
            } else if c == '\\' {
                escaped = true;
            } else if c == '"' {
                return Some(result);
            } else {
                result.push(c);
            }
        }
        Some(result)
    } else {
        None
    }
}

fn extract_query_param(raw_path: &str, param: &str) -> Option<String> {
    let query_str = raw_path.split('?').nth(1)?;
    for pair in query_str.split('&') {
        let mut kv = pair.split('=');
        let k = kv.next()?;
        if k == param {
            let v = kv.next().unwrap_or("");
            return Some(v.replace("%20", " ").replace('+', " "));
        }
    }
    None
}
```

```
#[allow(dead_code)]
fn tokenize_args(input: &str) -> Vec<String> {
    let mut args = Vec::new();
    let mut current = String::new();
    let mut in_quotes = false;
    let mut quote_char = '"';

    for c in input.chars() {
        if in_quotes {
            if c == quote_char {
                in_quotes = false;
            } else {
                current.push(c);
            }
        } else if c == '"' || c == '\'' {
            in_quotes = true;
            quote_char = c;
        } else if c.is_whitespace() {
            if !current.is_empty() {
                args.push(current);
                current = String::new();
            }
        } else {
            current.push(c);
        }
    }
    if !current.is_empty() {
        args.push(current);
    }
    args
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Server Module Status & Invariant Verification )
// ============================================================================
// OmniCode GUI Server module sealed and active.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/assemble.rs

RS | 534 LINES | 2055 WORDS | 23.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Machine Manifest, Scheduling Priority, Dependency Contract & Tripwire Registry )
// ================================================================================
//! MODULE:              omnicode::cli::commands::assemble
//! IDENTITY:            Physical Assembly & Process Tape Lowering Gear
//! NATURE:              Lowering Engine from High-Level SVO Tapes to Low-Level Memory Layouts & Silicon Word Streams
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {OmniCode Language & Standard Library} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Host Assembler Gear & Transpilation Lowering Driver
//!   WHAT:              Transduce high-level procedural `.tape` process ribbons into low-level declarative
//!                      `.omni` compiled memory reels, physical symbol tables, and 64-bit balanced-ternary
//!                      machine words ([`crate::core::word::Word64`]).
//!   WHERE:             Across Track 2 (`lexicon/`, `lib/kernel/`) or caller paths -> silicon memory partitions -> stdout / disk.
//!   WHEN:              Invoked via CLI command `omnicode assemble <file.tape> [--output <PATH>] [--format <listing|words|json>] [--verbose] [--dry-run]`
//!   WHY:               Eliminate ungrounded "python-ese" scripting drift by enforcing physical memory partition
//!                      allocation (`@ offset`), hardware register binding (`%self`, `%caller`, `%acc`, `%swarm`),
//!                      pronoun capability verification (`my`, `your`, `their`), and formal logic proofs (`! [ Invariant ]`).
//!   HOW:               Balanced Ternary (+1:PROCEED sealed assembly, -1:ROLLBACK trapped failpoint, 0:AWAIT dry-run).
//!
//! THE TRANSPILER TRIAD PHILOSOPHY:
//!   OmniCode operates across three isomorphic manifestations of code:
//!   1. `.omni` (Low-Level Compiled Form / Spatial Architecture):
//!      Static, declarative, frozen memory layouts, concrete denotations `[...]`, and invariant boundaries `<...>`.
//!   2. `.tape` (Interpreted Form / Temporal Execution Ribbon):
//!      Dynamic, sequential, time-bound rolls stepping across the tape head in real time.
//!   3. `Rust` (Silicon Host Substrate):
//!      Bedrock systems language defining opcodes, bitfields, and zero-cost native execution.
//!   This gear serves as the mechanical spindle turning Time (`.tape`) into Space (`.omni`) and Silicon (`Word64`).
//!
//! GAME DEV IDEOLOGY (THE STAGE INCEPTION & SIMULATION TICK):
//!   - Trigger Volumes & Collision Bounds: Every file resolution, syntax parse, partition slice, and seal
//!     is guarded by [`crate::cogs::tripwire::Tripwire`] collision volumes.
//!   - Flight Recorder: Step counts, instruction rolls, and physical offsets are logged to [`crate::diagnostics::logger::DiagnosticLogger`].
//!   - Respawn / Healing: Any syntax anomaly or invariant violation produces `#remediation_ready` telemetry
//!     for autonomous recovery by the [`crate::diagnostics::restorator::Restorator`].
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_INPUT_RESOLVE`      (Resolution of input tape file across tracks via `resolve_track_path`)
//!   - `TRIPWIRE_FILETYPE_VALIDATION`(Enforcement of `.tape` or `.omni` extensions)
//!   - `TRIPWIRE_SYNTAX_TOKENIZATION`(Lexical analysis and token classification verification)
//!   - `TRIPWIRE_MEMORY_BOUNDS`      (Partition capacity and offset bounds verification)
//!   - `TRIPWIRE_COMMIT_INVARIANT`   (Assertion of final affirmative `+1:PROCEED` seal)
//!   - `TRIPWIRE_OUTPUT_SERIALIZE`   (File system permissions and disk serialization integrity)
//!
//! NAVIGATION GUIDE:
//!   - Block 1 (Lines 1-80): Manifest, Transpiler Triad Philosophy, Game Dev Ideology, Tripwire Registry.
//!   - Block 2 (Lines 81-220): Opening Block, Configurations (`AssembleConfig`), Context (`AssembleContext`), Enums.
//!   - Block 3 (Lines 221-420): Body Block & Execution Pipeline (`run_assemble_with_config`, `execute_assemble_pipeline`).
//!   - Block 4 (Lines 421+): Unit tests validating flag parsing, path resolution, and physical machine word output.

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Environment Setup, Data Structures & Interface Definitions )
// ================================================================================

// --------------------------------------------------------------------------------
```

```rust
// 2.1. Scoped Namespace Ingestion & Multi-Platform Imports
// ---------------------------------------------------------------------------
use std::fmt;
use std::fs;
use std::path::PathBuf;

use crate::cogs::tripwire::{Tripwire, TripwireError};
use crate::core::filetype::resolve_track_path;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::runtime::assembler::{AssembledTape, OmniAssembler};

// ---------------------------------------------------------------------------
// 2.2. Output Serialization Format Enumeration
// ---------------------------------------------------------------------------
/// Output format for the assembled process tape or spatial reel.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Default)]
pub enum AssembleFormat {
    /// Human-readable annotated machine listing (Default).
    #[default]
    Listing,
    /// Raw physical 64-bit machine words (Hex dump).
    Words,
    /// Structured JSON representation for tooling, hovercards, and IDEs.
    Json,
    /// Declarative 4-Block spatial architecture reel (`.omni`).
    Omni,
    /// Bare-metal x86 bootstrap assembly (`.asm`) for MilleniumOS.
    Asm,
}

impl AssembleFormat {
    /// Resolves a CLI format string into an `AssembleFormat` variant.
    pub fn from_str_name(s: &str) -> Option<Self> {
        match s.to_lowercase().as_str() {
            "listing" | "txt" => Some(Self::Listing),
            "words" | "hex" | "bin" => Some(Self::Words),
            "json" => Some(Self::Json),
            "omni" | "reel" => Some(Self::Omni),
            "asm" | "assembly" | "nasm" => Some(Self::Asm),
            _ => None,
        }
    }

    /// Canonical lowercase format identifier string.
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Listing => "listing",
            Self::Words => "words",
            Self::Json => "json",
            Self::Omni => "omni",
            Self::Asm => "asm",
        }
    }
}

impl fmt::Display for AssembleFormat {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.as_str())
    }
}

// ---------------------------------------------------------------------------
// 2.3. Assembly Configuration Specification
// ---------------------------------------------------------------------------
/// Immutable configuration parameters for the assembly run.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct AssembleConfig {
    /// Relative or absolute path to the target process tape (`.tape`) or reel (`.omni`).
    pub input_path: PathBuf,
    /// Optional output file path for the assembled listing or binary artifact.
    pub output_path: Option<PathBuf>,
    /// Selected output serialization format.
    pub format: AssembleFormat,
    /// Enables verbose step-by-step diagnostic logging.
    pub verbose: bool,
    /// Strict mode: treat warnings or unspecified offsets as hard failpoints.
    pub strict: bool,
```

```
    /// Dry run: simulate lowering without writing output artifacts.
    pub dry_run: bool,
}

impl AssembleConfig {
    /// Constructs a default configuration for a given input path.
    pub fn new(input_path: impl Into<PathBuf>) -> Self {
        Self {
            input_path: input_path.into(),
            output_path: None,
            format: AssembleFormat::default(),
            verbose: false,
            strict: false,
            dry_run: false,
        }
    }

    /// Parses CLI arguments into an `AssembleConfig`.
    pub fn from_args(args: &[String]) -> Result<Self, String> {
        let mut input_path: Option<PathBuf> = None;
        let mut output_path: Option<PathBuf> = None;
        let mut format = AssembleFormat::Listing;
        let mut verbose = false;
        let mut strict = false;
        let mut dry_run = false;

        let mut idx = 0;
        while idx < args.len() {
            let arg = args[idx].as_str();
            match arg {
                "-o" | "--output" => {
                    idx += 1;
                    if idx >= args.len() {
                        return Err("FAILPOINT [MissingOutputArg]: Flag '--output' requires a file path
parameter".into());
                    }
                    output_path = Some(PathBuf::from(&args[idx]));
                }
                "-f" | "--format" => {
                    idx += 1;
                    if idx >= args.len() {
                        return Err("FAILPOINT [MissingFormatArg]: Flag '--format' requires a format name
(listing|words|json)".into());
                    }
                    let fmt_str = &args[idx];
                    format = AssembleFormat::from_str_name(fmt_str)
                        .ok_or_else(|| format!("FAILPOINT [InvalidFormat]: Unknown format '{fmt_str}' (expected: listing,
words, json)"))?;
                }
                "-v" | "--verbose" => {
                    verbose = true;
                }
                "-s" | "--strict" => {
                    strict = true;
                }
                "-n" | "--dry-run" => {
                    dry_run = true;
                }
                other => {
                    if !other.starts_with('-') && input_path.is_none() {
                        input_path = Some(PathBuf::from(other));
                    }
                }
            }
            idx += 1;
        }

        let input_path = input_path.ok_or_else(|| {
            "FAILPOINT [MissingInputPath]: No target tape file specified. Usage: omnicode assemble <file.tape> [--output
<PATH>] [--format <listing|words|json>]".to_string()
        })?;

        Ok(Self {
            input_path,
            output_path,
            format,
            verbose,
            strict,
```

```
            dry_run,
        })
    }

    /// Performs pre-flight verification, resolves paths across tracks, and constructs an `AssembleContext`.
    pub fn open(&self, logger: &DiagnosticLogger) -> Result<AssembleContext, TripwireError> {
        // Tripwire 1: Input file resolution across tracks
        let resolved = resolve_track_path(&self.input_path);
        Tripwire::assert_path_exists(
            logger,
            "TRIPWIRE_INPUT_RESOLVE",
            &resolved,
            "Target process tape or reel across Track roots",
        )?;

        // Tripwire 2: File extension check
        let ext = resolved.extension().and_then(|e| e.to_str()).unwrap_or("");
        if ext != "tape" && ext != "omni" {
            let msg = format!("File '{}' has unsupported extension '.{}' (expected .tape or .omni)", resolved.display(),
ext);
            return Err(Tripwire::trip(
                logger,
                "TRIPWIRE_FILETYPE_VALIDATION",
                FailpointType::SyntaxAnomaly,
                msg,
            ));
        }

        let debugger = OmniDebugger::new(logger.clone(), 100);
        let assembler = OmniAssembler::new(logger.clone(), debugger);

        Ok(AssembleContext {
            resolved_input: resolved,
            config: self.clone(),
            assembler,
            logger: logger.clone(),
        })
    }
}

// ---------------------------------------------------------------------------
// 2.4. Assembly Execution Context
// ---------------------------------------------------------------------------
/// Active runtime context holding resolved paths, tripwires, and the physical assembler.
#[derive(Debug)]
pub struct AssembleContext {
    /// Fully qualified path resolved across track roots.
    pub resolved_input: PathBuf,
    /// Immutable caller configuration.
    pub config: AssembleConfig,
    /// Physical assembler engine.
    pub assembler: OmniAssembler,
    /// Attached diagnostic logger for tripwires and telemetry.
    pub logger: DiagnosticLogger,
}

// ---------------------------------------------------------------------------
// 2.5. Initial Truth States & Equilibrium Defaults
// ---------------------------------------------------------------------------
/// Default output serialization format (human-readable annotated listing).
pub const DEFAULT_ASSEMBLE_FORMAT: AssembleFormat = AssembleFormat::Listing;
/// Baseline cell capacity reserved for unspecified memory partitions.
pub const DEFAULT_PARTITION_CELL_CAPACITY: usize = 64;
/// Maximum delimiter boundary nesting depth for structural invariant assertions.
pub const DEFAULT_ASSEMBLER_DEBUGGER_DEPTH: usize = 100;


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Assembly Execution Pipeline, Output Serialization & Tripwire Sealing )
// ============================================================================

// ---------------------------------------------------------------------------
// 3.1. Public CLI Dispatchers
// ---------------------------------------------------------------------------
/// Legacy single-path entrypoint maintaining backwards compatibility with CLI dispatchers.
pub fn run_assemble(path: &str) {
    let config = AssembleConfig::new(path);
```

```
    if let Err(e) = run_assemble_with_config(config) {
        eprintln!("\x1b[1;31mAssembly Failed:\x1b[0m {e}");
        std::process::exit(1);
    }
}

/// Executes the modular assembly pipeline with full configuration, tripwires, and formatting.
pub fn run_assemble_with_config(config: AssembleConfig) -> Result<AssembledTape, TripwireError> {
    let logger = DiagnosticLogger::new(config.verbose);
    println!("\x1b[1;36mAssembling OmniCode Process Tape:\x1b[0m {}", config.input_path.display());

    let mut ctx = config.open(&logger)?;
    let assembled = execute_assemble_pipeline(&mut ctx)?;

    // Handle output formatting and serialization
    serialize_assembled_output(&assembled, &ctx.config, &logger)?;

    println!(
        "\x1b[1;32m✓ Assembly Successful: Machine Tape Sealed in {}\x1b[0m ({} rolls, {} symbols, {} words)",
        assembled.commit_status.macro_op(),
        assembled.instructions.len(),
        assembled.symbols.len(),
        assembled.instructions.len()
    );

    Ok(assembled)
}

// ---------------------------------------------------------------------------
// 3.2. Core Lowering Pipeline
// ---------------------------------------------------------------------------
/// Invokes the lowering engine within the tripwire collision hull.
pub fn execute_assemble_pipeline(ctx: &mut AssembleContext) -> Result<AssembledTape, TripwireError> {
    let tape_path = ctx.resolved_input.clone();

    let assembled = match ctx.assembler.assemble_file(&tape_path) {
        Ok(t) => t,
        Err(e) => {
            return Err(Tripwire::trip(
                &ctx.logger,
                "TRIPWIRE_SYNTAX_TOKENIZATION",
                FailpointType::SyntaxAnomaly,
                format!("Tape assembly lowered into failpoint: {e}"),
            ));
        }
    };

    // Assert final commit status invariant
    Tripwire::assert_quad_state_proceed(
        &ctx.logger,
        "TRIPWIRE_COMMIT_INVARIANT",
        assembled.commit_status,
        "Process tape must conclude with 'commit_status [ PROCEED ]'",
    )?;

    Ok(assembled)
}

// ---------------------------------------------------------------------------
// 3.3. Output Serialization & Formatting
// ---------------------------------------------------------------------------
/// Renders or writes the assembled tape to disk or stdout according to the chosen format.
///
/// Consumes the universal serialization methods ([`AssembledTape::format_listing`],
/// [`AssembledTape::format_words`], [`AssembledTape::format_json`], [`AssembledTape::format_omni`])
/// extracted from the runtime lowering engine.
pub fn serialize_assembled_output(
    assembled: &AssembledTape,
    config: &AssembleConfig,
    logger: &DiagnosticLogger,
) -> Result<(), TripwireError> {
    let rendered = match config.format {
        AssembleFormat::Listing => assembled.format_listing(),
        AssembleFormat::Words => assembled.format_words(),
        AssembleFormat::Json => assembled.format_json(),
        AssembleFormat::Omni => assembled.format_omni(),
        AssembleFormat::Asm => assembled.format_asm(),
    };
```

```
376
377     if let Some(ref out_path) = config.output_path {
378         if !config.dry_run {
379             if let Err(e) = fs::write(out_path, &rendered) {
380                 return Err(Tripwire::trip(
381                     logger,
382                     "TRIPWIRE_OUTPUT_SERIALIZE",
383                     FailpointType::SyntaxAnomaly,
384                     format!("Failed to write assembled output to '{}': {e}", out_path.display()),
385                 ));
386             }
387             println!("\x1b[1;32m✓ Output Artifact Written:\x1b[0m {}", out_path.display());
388         } else {
389             println!("\x1b[1;33m[Dry Run] Output serialization skipped for:\x1b[0m {}", out_path.display());
390         }
391     } else {
392         println!("{rendered}");
393     }
394
395     Ok(())
396 }
397
398
399 // ============================================================================
400 // BLOCK 4: CLOSING BLOCK
401 // ( Baton Pass Contract, Unit Tests & Verification Proof )
402 // ============================================================================
403
404 // ----------------------------------------------------------------------------
405 // 4.1. The Baton Pass Contract
406 // ----------------------------------------------------------------------------
407 // BATON PASS CONTRACT:
408 // 1. `AssembleConfig` parses and validates user CLI flags without side effects.
409 // 2. `open()` verifies that the process tape exists across Track roots and establishes tripwire bounds.
410 // 3. `execute_assemble_pipeline()` drives lowering into `OmniAssembler`, verifying `MachineOp`, registers, and `Word64`.
411 // 4. `serialize_assembled_output()` formats or writes the output, yielding a sealed `AssembledTape`.
412 // 5. Downstream dispatchers in `main.rs` receive verified artifacts ready for scheduling or execution.
413
414 #[cfg(test)]
415 mod tests {
416     use super::*;
417     use crate::core::ternary::QuadState;
418
419     #[test]
420     fn test_assemble_config_default_and_new() {
421         let config = AssembleConfig::new("lib/kernel/boot.tape");
422         assert_eq!(config.input_path, PathBuf::from("lib/kernel/boot.tape"));
423         assert_eq!(config.format, AssembleFormat::Listing);
424         assert!(!config.verbose);
425         assert!(!config.strict);
426         assert!(!config.dry_run);
427         assert_eq!(config.output_path, None);
428     }
429
430     #[test]
431     fn test_assemble_config_argument_parsing() {
432         let args = vec![
433             "lexicon/operators/polarity.tape".to_string(),
434             "--output".to_string(),
435             "/tmp/polarity.asm".to_string(),
436             "--format".to_string(),
437             "words".to_string(),
438             "--verbose".to_string(),
439             "--dry-run".to_string(),
440         ];
441         let config = AssembleConfig::from_args(&args).expect("Parsing CLI args");
442         assert_eq!(config.input_path, PathBuf::from("lexicon/operators/polarity.tape"));
443         assert_eq!(config.output_path, Some(PathBuf::from("/tmp/polarity.asm")));
444         assert_eq!(config.format, AssembleFormat::Words);
445         assert!(config.verbose);
446         assert!(config.dry_run);
447     }
448
449     #[test]
450     fn test_assemble_config_missing_input_fails_gracefully() {
451         let args: Vec<String> = vec!["--verbose".to_string()];
452         let err = AssembleConfig::from_args(&args).unwrap_err();
453         assert!(err.contains("MissingInputPath"));
```

```
    }

    #[test]
    fn test_assemble_format_conversions() {
        assert_eq!(AssembleFormat::from_str_name("listing"), Some(AssembleFormat::Listing));
        assert_eq!(AssembleFormat::from_str_name("words"), Some(AssembleFormat::Words));
        assert_eq!(AssembleFormat::from_str_name("hex"), Some(AssembleFormat::Words));
        assert_eq!(AssembleFormat::from_str_name("json"), Some(AssembleFormat::Json));
        assert_eq!(AssembleFormat::from_str_name("omni"), Some(AssembleFormat::Omni));
        assert_eq!(AssembleFormat::from_str_name("reel"), Some(AssembleFormat::Omni));
        assert_eq!(AssembleFormat::from_str_name("unknown"), None);
    }

    #[test]
    fn test_assemble_pipeline_lowers_boot_tape_and_transpiles_to_omni() {
        let tape_path = resolve_track_path("lib/kernel/boot.tape");
        if tape_path.exists() {
            let config = AssembleConfig {
                input_path: tape_path,
                output_path: None,
                format: AssembleFormat::Omni,
                verbose: false,
                strict: false,
                dry_run: true,
            };
            let logger = DiagnosticLogger::new(false);
            let mut ctx = config.open(&logger).expect("Opening assemble context");
            let assembled = execute_assemble_pipeline(&mut ctx).expect("Executing assemble pipeline");

            let omni_output = assembled.format_omni();
            assert!(omni_output.contains("Transpiled Spatial Reel"));
            assert!(omni_output.contains("BLOCK 1: METADATA BLOCK"));
            assert!(omni_output.contains("BLOCK 2: OPENING BLOCK"));
            assert!(omni_output.contains("BLOCK 3: BODY BLOCK"));
            assert!(omni_output.contains("BLOCK 4: CLOSING BLOCK"));
        }
    }

    #[test]
    fn test_assemble_pipeline_lowers_boot_tape_and_emits_words() {
        let tape_path = resolve_track_path("lib/kernel/boot.tape");
        if tape_path.exists() {
            let config = AssembleConfig {
                input_path: tape_path,
                output_path: None,
                format: AssembleFormat::Words,
                verbose: false,
                strict: false,
                dry_run: true,
            };
            let logger = DiagnosticLogger::new(false);
            let mut ctx = config.open(&logger).expect("Opening assemble context");
            let assembled = execute_assemble_pipeline(&mut ctx).expect("Executing assemble pipeline");

            assert_eq!(assembled.commit_status, QuadState::Proceed);
            assert!(!assembled.instructions.is_empty());

            // Assert that instructions carry real physical Word64 machine words
            for inst in &assembled.instructions {
                assert_ne!(inst.word.raw, 0xFFFFFFFF_FFFFFFFF);
            }
        }
    }

    #[test]
    fn test_assemble_preflight_missing_file_trips() {
        let config = AssembleConfig::new("nonexistent_directory/missing.tape");
        let logger = DiagnosticLogger::new(false);
        let err = config.open(&logger).unwrap_err();
        assert!(err.to_string().contains("TRIPWIRE_INPUT_RESOLVE"));
    }

    #[test]
    fn test_assemble_preflight_invalid_extension_trips() {
        let config = AssembleConfig::new("Cargo.toml");
        let logger = DiagnosticLogger::new(false);
        let err = config.open(&logger).unwrap_err();
        assert!(err.to_string().contains("TRIPWIRE_FILETYPE_VALIDATION"));
```

```
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/boot.rs

RS | 776 LINES | 3856 WORDS | 39.0 KB

```
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Machine Manifest, Scheduling Priority, Dependency Contract & Tripwire Registry )
// ============================================================================
//! MODULE:              omnicode::cli::commands::boot
//! IDENTITY:            Bare-Metal Multi-Architecture Boot Gear
//! NATURE:              Cold-Start Machine Inception & Silicon Hardware Simulation Driver
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Host Orchestrator Engine & Machine Inception Driver
//!   WHAT:              Assemble bare-metal operating system kernels in Track 0 and launch
//!                      the QEMU hypervisor across BIOS, UEFI, and Multiboot architectures.
//!   WHERE:             `<TRACK 0>/kernel` tree -> host QEMU hypervisor -> COM1 / VGA silicon endpoints
//!   WHEN:              Invoked via CLI command `omnicode boot [--bios|--uefi|--multiboot] [--gui] [--ovmf <PATH>]`
//!   WHY:               Verify silicon-level self-sufficiency, hardware invariants, and balanced-ternary machine
bootstrap.
//!   HOW:               Balanced Ternary (+1:PROCEED clean exit, -1:ROLLBACK trapped error, 0:AWAIT execution running)
//!
//! THE "IN-WAITING" MULTI-OS SYMBIOTIC MESH:
//!   OmniCode is fundamentally an autonomous bare-metal Operating System and Balanced-Ternary computing
//!   architecture. However, until custom silicon machines are ubiquitous, OmniCode incubates comfortably
//!   inside existing host operating systems (Linux, macOS, Windows).
//!   This gear delegates host environment discovery to [`crate::cogs::platform::HostPlatform`], ensuring:
//!   - Dynamic selection of hypervisor binaries (`qemu-system-x86_64` vs `qemu-system-x86_64.exe`).
//!   - Dynamic discovery of UEFI OVMF firmware blobs across Debian, Arch, Fedora, NixOS, macOS Homebrew, and Windows.
//!   - Dynamic probing of display servers to avoid graphical window crashes in headless containers or SSH sessions.
//!
//! GAME DEV IDEOLOGY (THE STAGE INCEPTION & SIMULATION TICK):
//!   In modern game engine architecture (e.g., Unreal, Unity, Godot):
//!   - Level Loading / Inception: Setting up world geometry, shaders, and player spawn points before rendering.
//!   - Collision Volumes & Triggers: Spatial boundaries that fire events when crossed or violated.
//!   - Flight Recorder & Telemetry HUD: Frame-by-frame debug logging with nanosecond timestamps.
//!   - Respawn / Immune Recovery: Resetting player coordinates or healing scene state after a collision failpoint.
//!   In OmniCode Bare-Metal Boot:
//!   - Stage Inception: `execute_kernel_pipeline` compiles the bare-metal kernel and binds MMIO registers.
//!   - Tripwires: Invariant collision sensors (`Tripwire::assert_*`) wrapping every file access, compiler invocation,
//!     and hypervisor process launch.
//!   - Flight Recorder: [`DiagnosticLogger`] and [`OmniDebugger`] track step counts, register cognitive notes,
//!     and serialize episodic scrolls.
//!   - Respawn / Healing: Any tripped invariant produces a [`FixSuggestion`] and `#remediation_ready` telemetry
//!     for autonomous recovery by the [`crate::diagnostics::restorator::Restorator`].
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - EXECUTABLE:      `<TRACK 0>/kernel/build_all_kernels.sh`
//!   - ARTIFACT:        `<TRACK 0>/kernel/omnicode_bios.img` (512-byte MBR boot sector)
//!   - ARTIFACT:        `<TRACK 0>/kernel/esp/EFI/BOOT/BOOTX64.EFI` (UEFI PE32+ application)
//!   - ARTIFACT:        `<TRACK 0>/kernel/omnicode_multiboot.elf` (ELF multiboot kernel)
//!   - FIRMWARE:        OVMF UEFI Code image (probed from host system via `HostPlatform`)
//!
//! TOUCHES / SILICON HARDWARE & MMIO ADDRESSES:
//!   - COM1 Serial Port (I/O Port `0x3F8`): Serial console telemetry output at 115200 baud 8N1.
//!   - VGA Text Buffer Base (Memory Address `0xB8000`): 80x25 monochrome/color text screen buffer.
//!   - Real Mode Reset Vector (Physical Memory `0x7C00`): IBM PC MBR bootstrap entrypoint.
//!
//! REQUIRES / HOST DEPENDENCIES:
//!   - `qemu-system-x86_64` (virtual machine emulator & silicon hypervisor)
//!   - `bash` (bare-metal kernel build pipeline driver)
//!   - `nasm` (16-bit real mode and multiboot x86 assembler)
//!   - `clang` + `lld-link` (x86_64-unknown-windows UEFI PE32+ compiler & linker)
//!   - `ld` (elf_i386 multiboot linker)
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_KERNEL_BUILD_SCRIPT` (Track 0 build script accessibility and readability)
```

```
//!   - `TRIPWIRE_KERNEL_COMPILATION`  (Kernel compiler pipeline exit status code)
//!   - `TRIPWIRE_IMAGE_RESOLVE`       (Generated boot artifact existence after assembly)
//!   - `TRIPWIRE_HYPERVISOR_PATH`     (Availability of hypervisor binary on host PATH)
//!   - `TRIPWIRE_FIRMWARE_RESOLVE`    (Resolution of UEFI OVMF firmware blob)
//!   - `TRIPWIRE_DISPLAY_SERVER`      (Assertion of host display server when GUI mode is active)
//!   - `TRIPWIRE_QEMU_LAUNCH`         (Hypervisor execution status code and crash traps)
//!
//! NAVIGATION GUIDE (HOW TO NAVIGATE AND EXTEND THIS MODULE AS THE CODE GROWS):
//!   - Block 1 (Lines 1-95): Manifest, Multi-OS In-Waiting Philosophy, Game Dev Analogy, Tripwire Registry.
//!   - Block 2 (Lines 96-260): Opening Block, Domain Typology, Configuration & Equilibrium Constants.
//!     * To add a new boot architecture (e.g. RISC-V `RiscV64`): add a variant to `BootArch` in Section 2.2.
//!     * To inspect how options are parsed from CLI tokens: examine `BootConfig::from_args` in Section 2.3.
//!     * To view pre-flight invariant certification: examine `BootConfig::open` in Section 2.3.
//!     * To check silicon MMIO addresses: review equilibrium constants in Section 2.4.
//!   - Block 3 (Lines 261-530): Body Block & Bare-Metal Simulation Tick.
//!     * 3.1: Public dispatchers (`run_boot`, `run_boot_with_config`).
//!     * 3.2: Kernel compilation pipeline and build script execution (`execute_kernel_pipeline`).
//!     * 3.3: Silicon MMIO & hypervisor command synthesis (`synthesize_hypervisor_command`).
//!     * 3.4: Machine execution loop and telemetry sealing (`execute_boot_pipeline`, `execute_boot_machine`).
//!   - Block 4 (Lines 531+): Unit tests validating architecture names, argument parsing, command synthesis, and PATH
lookups.

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Environment Setup, Data Structures & Interface Definitions )
// ============================================================================

// ----------------------------------------------------------------------------
// 2.1. Scoped Namespace Ingestion & Multi-Platform Imports
// ----------------------------------------------------------------------------
use std::path::{Path, PathBuf};
use std::process::Command;

use crate::cogs::platform::HostPlatform;
use crate::cogs::tripwire::{Tripwire, TripwireError};
use crate::core::filetype::resolve_track_path;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;

// ----------------------------------------------------------------------------
// 2.2. Domain Typology & Architectural Target Definitions
// ----------------------------------------------------------------------------
/// Supported bare-metal boot target architectures.
///
/// Each architecture represents a discrete physical execution plane into which
/// OmniCode can materialize directly from cold power-on.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum BootArch {
    /// 16-Bit Real Mode MBR (512-byte sector loaded at physical address `0x7C00`).
    /// Minimalist, self-contained hardware bootstrap with BIOS INT 10h / INT 13h / INT 16h.
    Bios,

    /// 64-Bit Native UEFI Long Mode (PE32+ EFI Application loaded by firmware boot manager).
    /// Modern hardware standard with graphical GOP framebuffer and EFI System Table.
    Uefi,

    /// 32-Bit Direct Kernel Handshake via Multiboot Specification (ELF executable).
    /// Used for direct hypervisor injection (QEMU `-kernel`, GRUB, Xen, KVM).
    Multiboot,
}

impl BootArch {
    /// Human and machine readable description of the architecture target.
    pub fn display_name(&self) -> &'static str {
        match self {
            BootArch::Bios => "Legacy BIOS MBR (16-Bit Real Mode -> 512-Byte Boot Sector)",
            BootArch::Uefi => "Modern UEFI 64-Bit Long Mode (PE32+ EFI Application via OVMF)",
            BootArch::Multiboot => "Direct Kernel Multiboot/GRUB (ELF Hypervisor Handshake)",
        }
    }

    /// Expected target artifact path relative to `<TRACK 0>/kernel`.
    pub fn target_artifact_relpath(&self) -> &'static str {
        match self {
            BootArch::Bios => "kernel/omnicode_bios.img",
            BootArch::Uefi => "kernel/esp",
            BootArch::Multiboot => "kernel/omnicode_multiboot.elf",
```

```
        }
    }
}

// ---------------------------------------------------------------------------
// 2.3. Boot Configuration Specification & Options Matrix
// ---------------------------------------------------------------------------
/// Concrete execution configuration for the bare-metal machine inception.
#[derive(Debug, Clone)]
pub struct BootConfig {
    /// Target execution architecture (BIOS, UEFI, or Multiboot).
    pub arch: BootArch,
    /// Whether to open an emulated graphical window or run headless over COM1 serial.
    pub graphical: bool,
    /// Host operating system plane under which execution is launched.
    pub host_platform: HostPlatform,
    /// Optional custom override path for UEFI OVMF firmware image.
    pub custom_ovmf: Option<PathBuf>,
}

/// Fully resolved, verified execution context prepared by the Opening Block.
///
/// Acts as the immutable "Setup Frame" handed to the Body Block's simulation tick.
/// Contains certified file paths, armed diagnostic sensors, and pre-resolved host binaries.
#[derive(Debug, Clone)]
pub struct BootContext {
    /// Initial boot configuration parameters.
    pub config: BootConfig,
    /// Attached telemetry logger for recording events and failpoints.
    pub logger: DiagnosticLogger,
    /// Interactive debugger tracking step count and QuadState transitions.
    pub debugger: OmniDebugger,
    /// Verified absolute path to `<TRACK 0>/kernel/build_all_kernels.sh`.
    pub build_script: PathBuf,
    /// Target boot artifact (MBR image, ESP directory, or ELF executable).
    pub boot_artifact: PathBuf,
    /// Pre-resolved UEFI OVMF firmware blob (if UEFI mode is selected).
    pub ovmf_firmware: Option<PathBuf>,
    /// Hypervisor executable name appropriate for the host platform.
    pub hypervisor_binary: &'static str,
}

impl BootConfig {
    /// Constructs a standard boot configuration with platform auto-detection.
    pub fn new(arch: BootArch, graphical: bool) -> Self {
        Self {
            arch,
            graphical,
            host_platform: HostPlatform::current(),
            custom_ovmf: None,
        }
    }

    /// Parses raw command-line tokens into a validated [`BootConfig`].
    pub fn from_args(args: &[String]) -> Self {
        let arch = if args.iter().any(|a| a == "--uefi" || a == "-u") {
            BootArch::Uefi
        } else if args.iter().any(|a| a == "--multiboot" || a == "-m" || a == "--direct") {
            BootArch::Multiboot
        } else {
            BootArch::Bios
        };

        let graphical = args.iter().any(|a| a == "--gui" || a == "-g" || a == "--window");

        let custom_ovmf = args.iter().position(|a| a == "--ovmf" || a == "-b").and_then(|idx| {
            args.get(idx + 1).map(PathBuf::from)
        });

        Self {
            arch,
            graphical,
            host_platform: HostPlatform::current(),
            custom_ovmf,
        }
    }

    /// Prepares and certifies the execution context before the Body touches the metal.
```

```
    ///
    /// # Precondition Tripwires & Environmental Probes:
    /// - Asserts that the Track 0 kernel build script is reachable on disk.
    /// - Asserts that the target boot artifact path resolves.
    /// - Pre-resolves firmware candidate and platform hypervisor binary.
    /// - Inspects host display server if graphical GUI window was requested.
    /// - Initializes an attached [`OmniDebugger`] to monitor step invariants.
    pub fn open(&self, logger: DiagnosticLogger) -> Result<BootContext, TripwireError> {
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let build_script_path = resolve_track_path("kernel/build_all_kernels.sh");
        let build_script = Tripwire::assert_path_exists(
            &logger,
            "BOOT",
            build_script_path,
            "Track 0 kernel build pipeline script",
        )?;

        let boot_artifact = resolve_track_path(self.arch.target_artifact_relpath());

        // Probe host display environment if GUI window requested
        if self.graphical && !self.host_platform.has_display_server() {
            logger.record_ai_observation(
                "GUI_DISPLAY_HEADLESS",
                "Graphical window requested but host display server not detected",
                "Host environment lacks active DISPLAY or WAYLAND_DISPLAY session; window spawn may fail",
                ["gui", "display", "headless", "observation"].as_slice(),
                None,
            );
        }

        let ovmf_firmware = if self.arch == BootArch::Uefi {
            if let Some(custom) = &self.custom_ovmf {
                Some(Tripwire::assert_path_exists(&logger, "BOOT", custom, "Custom OVMF firmware")?)
            } else {
                self.host_platform.find_ovmf_firmware()
            }
        } else {
            None
        };

        let hypervisor_binary = self.host_platform.hypervisor_binary();

        Ok(BootContext {
            config: self.clone(),
            logger,
            debugger,
            build_script,
            boot_artifact,
            ovmf_firmware,
            hypervisor_binary,
        })
    }
}

// ---------------------------------------------------------------------------
// 2.4. Initial Truth States & Equilibrium Defaults
// ---------------------------------------------------------------------------
/// Canonical default boot architecture for minimal real-mode baseline.
pub const DEFAULT_BOOT_ARCH: BootArch = BootArch::Bios;

/// Default display mode: headless COM1 serial terminal output for pure dataflow.
pub const DEFAULT_GRAPHICAL_MODE: bool = false;

/// Real-mode bootstrap reset vector base in silicon RAM (`0x7C00`).
pub const REAL_MODE_BOOT_VECTOR: u64 = 0x7C00;

/// Standard VGA 80x25 text framebuffer memory-mapped base address (`0xB8000`).
pub const VGA_TEXT_FRAMEBUFFER_BASE: u64 = 0xB8000;

/// Standard IBM PC COM1 serial communications port I/O base address (`0x3F8`).
pub const COM1_SERIAL_IO_BASE: u16 = 0x3F8;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Kernel Compilation, Inception Tripwires & QEMU Execution Engine )
// ============================================================================

// ---------------------------------------------------------------------------
```

```rust
// 3.1. Public Dispatcher & Context Resolution
// ---------------------------------------------------------------------------
/// Builds the target kernels in Track 0 and launches the virtual machine in QEMU.
///
/// # Game Dev Loop & Cybernetic Architecture:
/// Functions as the primary command-line engine driver. Resolves ambient hardware
/// parameters, constructs the diagnostic logger and debugger sensors, and invokes
/// the self-healing execution pipeline.
///
/// # Returns
/// - `Ok(QuadState::Proceed)`: Hypervisor session completed cleanly without fatal invariant breach.
/// - `Err(TripwireError)`: Precondition, compilation, or hypervisor tripwire tripped with `-1:ROLLBACK`.
pub fn run_boot(arch: BootArch, graphical: bool) -> Result<QuadState, TripwireError> {
    let config = BootConfig::new(arch, graphical);
    run_boot_with_config(config)
}

/// Executes boot inception under a customized [`BootConfig`].
pub fn run_boot_with_config(config: BootConfig) -> Result<QuadState, TripwireError> {
    let logger = DiagnosticLogger::new(false);
    let ctx = match config.open(logger) {
        Ok(c) => c,
        Err(e) => {
            eprintln!("\x1b[1;31m[!] Precondition Tripwire Tripped during BootContext Open:\x1b[0m {e}");
            return Err(e);
        }
    };

    execute_boot_pipeline(ctx)
}

// ---------------------------------------------------------------------------
// 3.2. Kernel Compilation Pipeline & Build Tripwires
// ---------------------------------------------------------------------------
/// Orchestrates the execution of the Track 0 bare-metal kernel build pipeline.
///
/// # Silicon & Assembly Invariants:
/// - Reaches into `<TRACK 0>/kernel/build_all_kernels.sh`.
/// - Assembles 16-bit real-mode bootstrap vector (`0x7C00`) via NASM.
/// - Compiles 64-bit UEFI PE32+ application with Clang/LLD targeting GOP framebuffer.
/// - Links 32-bit ELF multiboot header with GNU LD for hypervisor direct handshakes.
///
/// # Sensory Tripwires:
/// - `TRIPWIRE_KERNEL_BUILD_SCRIPT`: Asserts build script is readable and executable.
/// - `TRIPWIRE_KERNEL_COMPILATION`: Asserts zero exit code from compilation subprocess.
/// - `TRIPWIRE_IMAGE_RESOLVE`: Asserts that target binary artifact exists on disk after assembly.
pub fn execute_kernel_pipeline(ctx: &mut BootContext) -> Result<(), TripwireError> {
    ctx.logger.log("KERNEL_PIPELINE", "Initiating Track 0 Bare-Metal Kernel Build Pipeline...");
    ctx.debugger.inspect_state(QuadState::Await, "kernel_build_started");
    ctx.debugger.step_count += 1;

    // 0. Transpile MilleniumOS boot architecture directly from OmniCode source reel
    let boot_omni_path = resolve_track_path("lib/kernel/boot.omni");
    if boot_omni_path.exists() {
        if let Ok(omni_content) = std::fs::read_to_string(&boot_omni_path) {
            if let Ok(generated_asm) = crate::runtime::assembler::transpile_omni_to_asm(&omni_content) {
                let bios_asm_path = resolve_track_path("kernel/bios/boot.asm");
                let _ = std::fs::write(&bios_asm_path, &generated_asm);
                let root_asm_path = resolve_track_path("kernel/boot.asm");
                let _ = std::fs::write(&root_asm_path, &generated_asm);
                ctx.logger.log("KERNEL_PIPELINE", "Transpiled lib/kernel/boot.omni -> bios/boot.asm for MilleniumOS");
            }
        }
    }

    let script_dir = ctx.build_script.parent().unwrap_or_else(|| Path::new("."));
    ctx.logger.log(
        "KERNEL_PIPELINE",
        format!("Executing driver script in directory: {:?}", script_dir),
    );

    println!("\x1b[1;34m[i] Executing bare-metal kernel build pipeline in {:?}...\x1b[0m", script_dir);
    let build_status = Command::new("bash")
        .arg(ctx.build_script.file_name().unwrap_or_default())
        .current_dir(script_dir)
        .status();

    match Tripwire::assert_command_success(&ctx.logger, "KERNEL_PIPELINE", build_status, "Kernel Build Pipeline") {
```

```
        Ok(()) => {
            println!("\x1b[1;32m✓ Track 0 Bare-Metal Kernel Pipeline Built Successfully\x1b[0m");
            ctx.logger.log("KERNEL_PIPELINE", "Kernel compilation subprocess returned exit code 0");
            ctx.logger.record_ai_observation(
                "KERNEL_BUILD_SUCCESS",
                "Bare-metal kernel compilation passed",
                "Track 0 build script executed cleanly with all artifacts generated",
                ["kernel", "build", "silicon", "verified"].as_slice(),
                Some("build_all_kernels.sh"),
            );
        }
        Err(e) => {
            ctx.debugger.inspect_state(QuadState::Rollback, "kernel_build_failed");
            ctx.logger.record_ai_observation(
                "KERNEL_BUILD_FAILED",
                "Bare-metal kernel compilation failed",
                &e.message,
                ["kernel", "build_failure", "rollback"].as_slice(),
                Some("build_all_kernels.sh"),
            );
            return Err(e);
        }
    }

    // Verify generated artifact presence post-build
    Tripwire::assert_path_exists(
        &ctx.logger,
        "KERNEL_PIPELINE",
        &ctx.boot_artifact,
        "Post-compilation target boot artifact",
    )?;

    ctx.debugger.inspect_state(QuadState::Proceed, "kernel_artifact_verified");
    Ok(())
}

// ---------------------------------------------------------------------------
// 3.3. Hypervisor Command Synthesis & Parameter Mapping
// ---------------------------------------------------------------------------
/// Constructs the platform-specific hypervisor launch command with exact MMIO,
/// serial redirection, and drive geometry parameters.
///
/// # Hardware & Architecture Mapping (Simulating Motherboard Buses):
/// - **BIOS**: Maps raw 512-byte image as primary boot drive (`format=raw,file=...`).
/// - **UEFI**: Maps OVMF firmware blob (`-bios ...`) and mounts ESP FAT partition.
/// - **Multiboot**: Direct hypervisor injection via QEMU `-kernel` flag.
/// - **Serial & VGA**: Multiplexes COM1 (`0x3F8`) and VGA text buffer (`0xB8000`)
///   to host standard I/O streams for headless evaluation or GUI windows.
///
/// # Sensory Tripwires:
/// - `TRIPWIRE_HYPERVISOR_PATH`: Asserts hypervisor binary (`qemu-system-x86_64`) exists on PATH.
/// - `TRIPWIRE_FIRMWARE_RESOLVE`: Asserts UEFI OVMF firmware blob exists when UEFI mode is active.
pub fn synthesize_hypervisor_command(ctx: &BootContext) -> Result<Command, TripwireError> {
    ctx.logger.log("HYPERVISOR", format!("Synthesizing launch command for {}", ctx.hypervisor_binary));

    // Assert that the hypervisor binary is present on host system PATH
    Tripwire::assert_binary_available(&ctx.logger, "HYPERVISOR", ctx.hypervisor_binary)?;

    let mut cmd = Command::new(ctx.hypervisor_binary);

    match ctx.config.arch {
        BootArch::Bios => {
            ctx.logger.log("HYPERVISOR", format!("Configuring BIOS MBR raw drive: {:?}", ctx.boot_artifact));
            let path_str = ctx.boot_artifact.to_string_lossy();
            let file_arg = if path_str.starts_with('/') || path_str.starts_with("./") {
                path_str.into_owned()
            } else {
                format!("./{path_str}")
            };
            cmd.arg("-drive").arg(format!("format=raw,file={file_arg}"));
        }
        BootArch::Uefi => {
            let ovmf = if let Some(ref fw) = ctx.ovmf_firmware {
                fw.to_string_lossy().into_owned()
            } else {
                let default_ovmf = "/usr/share/qemu/OVMF.fd".to_string();
                ctx.logger.log("HYPERVISOR", format!("Using fallback OVMF path: {}", default_ovmf));
                default_ovmf
```

```
            };
            ctx.logger.log("HYPERVISOR", format!("Configuring UEFI with OVMF firmware: {}", ovmf));
            cmd.arg("-bios").arg(ovmf);

            let esp_dir = if ctx.boot_artifact.to_string_lossy().contains(':') {
                let staged = PathBuf::from("target/esp");
                let staged_boot = staged.join("EFI").join("BOOT");
                let _ = std::fs::create_dir_all(&staged_boot);
                let efi_bin = ctx.boot_artifact.join("EFI").join("BOOT").join("BOOTX64.EFI");
                if efi_bin.exists() {
                    let _ = std::fs::copy(&efi_bin, staged_boot.join("BOOTX64.EFI"));
                }
                staged.to_string_lossy().into_owned()
            } else {
                ctx.boot_artifact.to_string_lossy().into_owned()
            };
            cmd.arg("-drive").arg(format!("file=fat:rw:{esp_dir},format=raw"));
            cmd.arg("-net").arg("none");
        }
        BootArch::Multiboot => {
            ctx.logger.log("HYPERVISOR", format!("Configuring direct multiboot ELF injection: {:?}", ctx.boot_artifact));
            let path_str = ctx.boot_artifact.to_string_lossy();
            let kernel_arg = if path_str.starts_with('/') || path_str.starts_with("./") {
                path_str.into_owned()
            } else {
                format!("./{path_str}")
            };
            cmd.arg("-kernel").arg(kernel_arg);
        }
    }

    if ctx.config.graphical {
        ctx.logger.log("HYPERVISOR", "Display mode: Graphical Window with COM1 serial on stdio");
        cmd.arg("-serial").arg("stdio");
    } else {
        ctx.logger.log("HYPERVISOR", "Display mode: Headless with COM1 serial on stdio (display=none)");
        cmd.arg("-display").arg("none").arg("-serial").arg("stdio");
    }

    Ok(cmd)
}

// ---------------------------------------------------------------------------
// 3.4. Machine Execution Loop, Tripwire Interception & Telemetry Seal
// ---------------------------------------------------------------------------
/// Executes the complete bare-metal machine inception pipeline.
///
/// # Game Dev Loop Rationale:
/// Directly executes the simulation tick of the host engine:
/// 1. Emits visual inception banner with machine metadata.
/// 2. Executes kernel pipeline wrapped in build tripwires.
/// 3. Synthesizes hypervisor launch command with hardware mappings.
/// 4. Executes virtual machine, trapping non-zero exit codes.
/// 5. Inscribes telemetry into [`DiagnosticLogger`] and [`OmniDebugger`] for the
[`crate::diagnostics::restorator::Restorator`].
pub fn execute_boot_pipeline(mut ctx: BootContext) -> Result<QuadState, TripwireError> {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode OS: Bare-Metal Multi-Architecture Machine Inception               \x1b[0m");
    println!("\x1b[1;36m  Mode: {}\x1b[0m", ctx.config.arch.display_name());
    println!("\x1b[1;36m  Host: {}\x1b[0m", ctx.config.host_platform.display_name());
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    ctx.logger.log(
        "BOOT",
        format!(
            "Beginning Machine Inception [{}] on [{}]",
            ctx.config.arch.display_name(),
            ctx.config.host_platform.display_name()
        ),
    );
    ctx.debugger.inspect_state(QuadState::Await, "machine_inception_begun");
    ctx.debugger.step_count += 1;

    // 1. Execute Bare-Metal Kernel Pipeline in Track 0
    execute_kernel_pipeline(&mut ctx)?;

    // 2. Configure Machine Emulator Command
    let mut cmd = synthesize_hypervisor_command(&ctx)?;
```

```
    if ctx.config.graphical {
        println!("\x1b[1;36m[i] Launching graphical virtual machine display with serial on stdio...\x1b[0m");
    } else {
        println!("\x1b[1;36m[i] Launching headless terminal machine with COM1 serial on stdio...\x1b[0m");
    }

    // 3. Launch Virtual Machine Inception
    ctx.logger.log("BOOT", format!("Spawning hypervisor child process: {:?}", cmd));
    ctx.debugger.inspect_state(QuadState::Await, "hypervisor_spawning");
    ctx.debugger.step_count += 1;

    let run_status = cmd.status();
    match Tripwire::assert_command_success(&ctx.logger, "BOOT", run_status, ctx.hypervisor_binary) {
        Ok(()) => {
            println!("\n\x1b[1;32m✓ QEMU Machine Inception Terminated Cleanly (+1:PROCEED)\x1b[0m");
            ctx.logger.log("BOOT", "Virtual machine exited with status code 0 (+1:PROCEED)");
            ctx.debugger.inspect_state(QuadState::Proceed, "machine_inception_clean_exit");
            ctx.logger.record_ai_observation(
                "BOOT_SUCCESS",
                "Machine inception ran to completion",
                "QEMU execution exited with success code 0",
                ["hypervisor", "boot", "proceed", "verified"].as_slice(),
                Some(ctx.hypervisor_binary),
            );
            Ok(QuadState::Proceed)
        }
        Err(e) => {
            ctx.debugger.inspect_state(QuadState::Rollback, "machine_inception_trapped");
            ctx.logger.record_ai_observation(
                "BOOT_FAILED",
                "Machine inception trapped failpoint",
                &e.message,
                ["hypervisor", "boot_failure", "rollback"].as_slice(),
                Some(ctx.hypervisor_binary),
            );
            eprintln!("  (Ensure {} is installed and accessible on your PATH)", ctx.hypervisor_binary);
            Err(e)
        }
    }
}

/// Backwards-compatible synchronous executor for machine inception.
pub fn execute_boot_machine(ctx: BootContext) {
    if let Err(e) = execute_boot_pipeline(ctx) {
        eprintln!("\x1b[1;31mMachine inception halted:\x1b[0m {e}");
        std::process::exit(1);
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Module Sealing, Invariant Verification & Downstream Baton Pass )
// ============================================================================

// ----------------------------------------------------------------------------
// 4.1. Module Sealing & Downstream Baton Pass Specification
// ----------------------------------------------------------------------------
// BATON PASS CONTRACT:
//   - Upstream Producer: CLI Command Dispatcher (`omnicode boot ...`).
//   - Downstream Receiver: `<TRACK 0>` Bare-Metal Kernel Pipeline & Host QEMU Hypervisor.
//   - Certified Invariants:
//     1. Target architecture variants (`Bios`, `Uefi`, `Multiboot`) map deterministically
//        to valid artifact targets in the `<TRACK 0>/kernel` tree.
//     2. Hardware MMIO base registers (COM1 `0x3F8`, VGA `0xB8000`, Real Mode `0x7C00`)
//        are immutable and verified.
//     3. Command synthesis accurately maps drives, firmware, and stdio multiplexing.
//     4. `BootContext::open` certifies all filesystem preconditions, initializes an
//        attached `OmniDebugger` and `DiagnosticLogger`, and leaves zero leaked state.
//     5. Any execution failure is intercepted by sensory tripwires and packaged with
//        remediation suggestions for the `Restorator`.

#[cfg(test)]
mod tests {
    use super::*;

    /// Verifies that all `BootArch` enum variants return non-empty, distinct, human-readable display titles.
    ///
```

```
    /// # Failure Mode Guarded Against:
    /// Prevents silent string regression or unmapped match arms when new architecture targets are added.
    #[test]
    fn test_boot_arch_display() {
        assert!(BootArch::Bios.display_name().contains("BIOS"), "BIOS display title must identify MBR mode");
        assert!(BootArch::Uefi.display_name().contains("UEFI"), "UEFI display title must identify Long Mode");
        assert!(BootArch::Multiboot.display_name().contains("Multiboot"), "Multiboot display title must identify ELF
mode");
    }

    /// Verifies that the physical kernel build script and default BIOS boot sector image resolve in `<TRACK 0>`.
    ///
    /// # Failure Mode Guarded Against:
    /// Guards against repository relocation anomalies or broken relative path traversal to Track 0.
    #[test]
    fn test_kernel_paths_resolve() {
        let build_script = resolve_track_path("kernel/build_all_kernels.sh");
        assert!(build_script.exists(), "Track 0 kernel build driver script must exist on disk");
        let bios_img = resolve_track_path("kernel/omnicode_bios.img");
        assert!(bios_img.exists(), "Track 0 default BIOS binary artifact must exist on disk");
    }

    /// Verifies that `HostPlatform::current()` correctly detects the active host environment
    /// and provides non-empty hypervisor and firmware candidate paths.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents null or uninitialized platform definitions from entering the boot pipeline.
    #[test]
    fn test_host_platform_detection() {
        let current = HostPlatform::current();
        assert!(!current.display_name().is_empty(), "Host platform must have a non-empty display name");
        assert!(!current.hypervisor_binary().is_empty(), "Host platform must provide a default hypervisor binary");
        assert!(!current.ovmf_search_paths().is_empty(), "Host platform must declare firmware candidate paths");
    }

    /// Verifies command-line argument parsing across short flags, long flags, and option permutations.
    ///
    /// # Invariants Tested:
    /// - `--uefi` and `-u` select `BootArch::Uefi`.
    /// - `--multiboot` and `-m` select `BootArch::Multiboot`.
    /// - Absence of arch flags defaults to `BootArch::Bios`.
    /// - `--gui` sets `graphical = true`.
    /// - `--ovmf <PATH>` extracts custom firmware path override.
    #[test]
    fn test_boot_config_from_args() {
        let args = vec!["omnicode".to_string(), "boot".to_string(), "--uefi".to_string(), "--gui".to_string()];
        let cfg = BootConfig::from_args(&args);
        assert_eq!(cfg.arch, BootArch::Uefi, "Flag --uefi must select BootArch::Uefi");
        assert!(cfg.graphical, "Flag --gui must set graphical mode to true");

        let args_mb = vec!["omnicode".to_string(), "boot".to_string(), "-m".to_string()];
        let cfg_mb = BootConfig::from_args(&args_mb);
        assert_eq!(cfg_mb.arch, BootArch::Multiboot, "Flag -m must select BootArch::Multiboot");
        assert!(!cfg_mb.graphical, "Default graphical mode must be headless (false)");

        let args_custom_ovmf = vec![
            "omnicode".to_string(),
            "boot".to_string(),
            "--uefi".to_string(),
            "--ovmf".to_string(),
            "/custom/ovmf.fd".to_string(),
        ];
        let cfg_custom = BootConfig::from_args(&args_custom_ovmf);
        assert_eq!(cfg_custom.custom_ovmf, Some(PathBuf::from("/custom/ovmf.fd")));
    }

    /// Verifies that `BootConfig::open` successfully certifies preconditions, resolves Track 0 scripts,
    /// attaches an armed `OmniDebugger`, and constructs an immutable `BootContext`.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents uncertified or partially initialized execution frames from reaching the hypervisor spawner.
    #[test]
    fn test_boot_context_open_preconditions() {
        let cfg = BootConfig::new(BootArch::Bios, false);
        let logger = DiagnosticLogger::new(false);
        let ctx_res = cfg.open(logger);
        assert!(ctx_res.is_ok(), "Opening BootContext must succeed when Track 0 workspace is valid");
        let ctx = ctx_res.unwrap();
```

```
        assert!(ctx.build_script.exists(), "Certified build script path must exist on disk");
        assert_eq!(ctx.config.arch, BootArch::Bios, "Context config arch must match specification");
        assert_eq!(ctx.debugger.step_count, 0, "Attached debugger must start at step count 0");
    }

    /// Verifies hypervisor command synthesis for BIOS MBR mode in headless serial operation.
    ///
    /// # Silicon Wiring Invariants:
    /// - Primary raw drive mapped to target artifact (`-drive format=raw,file=...`).
    /// - Display disabled (`-display none`).
    /// - COM1 serial port (`0x3F8`) multiplexed to standard I/O (`-serial stdio`).
    #[test]
    fn test_synthesize_hypervisor_command_bios_headless() {
        let cfg = BootConfig::new(BootArch::Bios, false);
        let logger = DiagnosticLogger::new(false);
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(args.contains(&"-drive".to_string()), "Command must include -drive flag");
        assert!(args.iter().any(|a| a.contains("format=raw,file=")), "Must specify raw MBR disk format");
        assert!(args.contains(&"-display".to_string()), "Headless mode must include -display flag");
        assert!(args.contains(&"none".to_string()), "Headless mode must specify display none");
        assert!(args.contains(&"-serial".to_string()), "COM1 serial port must be redirected");
        assert!(args.contains(&"stdio".to_string()), "Serial redirection must target stdio");
    }

    /// Verifies hypervisor command synthesis for graphical window display mode.
    ///
    /// # Graphical Wiring Invariants:
    /// - Must NOT include `-display none`.
    /// - Serial COM1 must remain connected to `stdio` for background telemetry.
    #[test]
    fn test_synthesize_hypervisor_command_graphical() {
        let cfg = BootConfig::new(BootArch::Bios, true);
        let logger = DiagnosticLogger::new(false);
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(!args.contains(&"-display".to_string()), "Graphical mode must not pass -display none");
        assert!(args.contains(&"-serial".to_string()), "Graphical mode must retain COM1 serial redirection");
        assert!(args.contains(&"stdio".to_string()), "Serial output must stream to host stdio");
    }

    /// Verifies hypervisor command synthesis for direct Multiboot kernel injection.
    ///
    /// # Hypervisor Direct Handshake Invariants:
    /// - ELF binary injected directly into RAM via QEMU `-kernel` parameter.
    #[test]
    fn test_synthesize_hypervisor_command_multiboot() {
        let cfg = BootConfig::new(BootArch::Multiboot, false);
        let logger = DiagnosticLogger::new(false);
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(args.contains(&"-kernel".to_string()), "Multiboot must pass -kernel flag");
        assert!(args.iter().any(|a| a.contains("omnicode_multiboot.elf")), "Kernel path must point to multiboot ELF");
    }

    /// Verifies hypervisor command synthesis for UEFI Long Mode execution.
    ///
    /// # Firmware & ESP Wiring Invariants:
    /// - OVMF firmware image mapped via `-bios`.
    /// - ESP directory mapped as FAT drive via `file=fat:rw:...,format=raw`.
    #[test]
    fn test_synthesize_hypervisor_command_uefi() {
        let mut cfg = BootConfig::new(BootArch::Uefi, false);
        cfg.custom_ovmf = Some(PathBuf::from("/usr/share/qemu/OVMF.fd"));
        let logger = DiagnosticLogger::new(false);
        let ctx = cfg.open(logger).expect("Context open succeeds");
        let cmd = synthesize_hypervisor_command(&ctx).expect("Command synthesis succeeds");
        let args: Vec<String> = cmd.get_args().map(|s| s.to_string_lossy().into_owned()).collect();

        assert!(args.contains(&"-bios".to_string()), "UEFI mode must pass -bios flag");
        assert!(args.contains(&"/usr/share/qemu/OVMF.fd".to_string()), "Must wire specified OVMF firmware blob");
        assert!(args.contains(&"-drive".to_string()), "Must configure EFI System Partition drive");
```

```
        assert!(args.iter().any(|a| a.contains("file=fat:rw:")), "Must mount ESP as FAT filesystem");
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cli/commands/mod.rs RS 38 LINES 132 WORDS 1.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( CLI Commands Subsystem Identity & Command Gear Registry )
// ================================================================================
//! OmniCode CLI Command Gears
//!
//! Encapsulates discrete CLI command implementations as modular mechanical gears
//! driven by the main orchestrator engine.

pub mod boot;
pub mod assemble;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Command Re-exports )
// ================================================================================
pub use boot::{
    BootArch, BootConfig, BootContext, run_boot, run_boot_with_config,
    execute_kernel_pipeline, synthesize_hypervisor_command, execute_boot_pipeline, execute_boot_machine,
    DEFAULT_BOOT_ARCH, DEFAULT_GRAPHICAL_MODE, REAL_MODE_BOOT_VECTOR, VGA_TEXT_FRAMEBUFFER_BASE, COM1_SERIAL_IO_BASE,
};

pub use assemble::{
    AssembleFormat, AssembleConfig, AssembleContext, run_assemble, run_assemble_with_config,
    execute_assemble_pipeline, serialize_assembled_output,
};

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Command Routing & Common Dispatch Logic )
// ================================================================================

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing )
// ================================================================================
// Command gears verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler.rs

RS | 1930 LINES | 8762 WORDS | 95.3 KB

```
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Machine Manifest, Scheduling Priority, Physical ISA Architecture & Identity Separation )
// ============================================================================
//! MODULE:              omnicode::runtime::assembler
//! IDENTITY:            The Physical Assembler & Silicon Lowering Engine
//! NATURE:              Transduction Engine from Abstract Ontological Archetypes (`+`) to Concrete Silicon Memory Layouts (`-`)
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-12
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {OmniCode Language & Standard Library} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Runtime Silicon Assembler & Lowering Engine (`omnicode::runtime::assembler`)
//!   WHAT:              Transduce high-level abstract grammatical statements (`+`) into concrete physical silicon
//!                      memory allocations, cell offsets (`@ 0x0000`), hardware CPU registers (`%self`, `%caller`,
//!                      `%acc`, `%swarm`), and 64-bit balanced-ternary machine words ([`Word64`]).
//!   WHERE:             Internal execution core across Track 1 VM, consuming Track 2 Lexicon tapes and Track 0 kernel manifests.
//!   WHEN:              Invoked during compilation, JIT translation, interactive execution, or via CLI `omnicode assemble`.
//!   WHY:               Ground abstract linguistic statements into concrete silicon execution physics, eliminating
//!                      untyped scripting drift and establishing deterministic memory boundaries.
//!   HOW:               Balanced Ternary (+1:PROCEED sealed assembly, -1:ROLLBACK trapped failpoint, 0:AWAIT dry-run).
//!
//! SCRIPTURE WITNESSES & EPISTEMIC FOUNDATION:
//!   - Exodus 31:18 (KJV):
//!     "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!      two tables of testimony, tables of stone, written with the finger of God."
//!     (The archetype of physical inscription upon silicon and stone substrate: concrete machine lowering).
//!   - Proverbs 24:27 (KJV):
//!     "Prepare thy work without, and make it fit for thyself in the field;
//!      and afterwards build thine house."
//!     (The discipline of ordering: preparing abstract rolls and symbolic allocations in the field before
//!      building the physical silicon house).
//!   - Proverbs 22:28 (KJV):
//!     "Remove not the ancient landmark, which thy fathers have set."
//!     (Preservation of immutable memory boundaries, physical MMIO offsets, and covenant seals).
//!
//! WHY EXTENSIVE COMMENTING MATTERS FOR AI AGENTS & DEVELOPERS:
//!   1. Epistemic Continuity: Autonomous coding agents require explicit rationale for opcode mappings,
//!      register allocations, and memory partition geometries.
//!   2. Architectural Transparency: Cross-referencing Track 0 bare-metal silicon, Track 1 VM lowering,
//!      and Track 2 standard library tapes ensures no drift between the abstract lexicon and machine execution.
//!   3. Mathematical Rigor: Clear documentation of the 64-bit balanced-ternary Word64 structure and
//!      checksum algorithm ensures reproducible, deterministic silicon execution.
//!
//! ARCHITECTURAL DUALITY & IDENTITY SEPARATION:
//!   Notice the vital distinction between the two assembler gears in Track 1:
//!   1. `omnicode::cli::commands::assemble` (The External CLI Dispatcher & Orchestrator):
//!      - Location: `src/cli/commands/assemble.rs`
//!      - Role: User port, CLI flag ingestion (`-o`, `-f`, `-v`, `-s`, `-n`), cross-track path resolution
//!        via `resolve_track_path`, pre-flight tripwire perimeter verification (`TRIPWIRE_INPUT_RESOLVE`),
//!        and destination disk artifact writing (`TRIPWIRE_OUTPUT_SERIALIZE`).
//!   2. `omnicode::runtime::assembler` (The Internal Lowering Engine & Silicon Synthesizer):
//!      - Location: `src/runtime/assembler.rs` (Modularizing towards `src/runtime/assembler/` domain tree)
//!      - Role: Core balanced-ternary machine ISA, opcode assignment (`MachineOp`), physical hardware register
//!        mapping (`MachineRegister`), physical memory partition allocation (`@ offset`, `v--`), 64-bit balanced
//!        ternary machine words ([`Word64`]), structural boundary invariant enforcement via [`OmniDebugger`],
//!        and transpilation between the three isomorphic triad forms (`.tape` <--> `.omni` <--> `Word64`).
//!
//! WHY THE ASSEMBLER DEMANDS ITS OWN MODULAR DOMAIN TREE:
//!   As OmniCode matures beyond Python-like interpreted scripting into a self-hosting silicon Operating System,
//!   the assembler decomposes into discrete, highly specialized computational sub-systems:
//!   - `isa`: Hardware Opcodes (`MachineOp`), VM Registers (`MachineRegister`), and ternary word packaging.
```

```
//!   - `symbols`: Memory map partition geometry, Storehouse limits, polarity tracking (`+` vs `-`), and cell capacities.
//!   - `tape`: Procedural roll lowering (`assemble_roll`), Tape Head covenant parsing, and directional causal vectors (`-->`, `<--`).
//!   - `transpiler`: Dual-direction decompilation and code emission (transpiling `.tape` temporal process ribbons into
//!     declarative 4-Block `.omni` spatial architecture reels, silicon hex dumps, or IDE hovercard JSON).
//!
//! GAME DEV IDEOLOGY (THE STAGE INCEPTION & SIMULATION TICK):
//!   - Level Loading / Inception: Setting up physical memory partitions (`where_clause`, `partition_size`) before execution.
//!   - Collision Volumes & Triggers: Delimiter boundary stack tracking (`[`, `{`, `<`) guarded by [`OmniDebugger`].
//!   - Flight Recorder & Telemetry HUD: Step-by-step lowering logs and failpoint traps recorded in [`DiagnosticLogger`].
//!   - Respawn / Immune Recovery: Unsealed tapes or boundary leaks emit `#remediation_ready` telemetry for the [`crate::diagnostics::restorator::Restorator`].
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - INGESTION:       Procedural process ribbons (`.tape`) and declarative spatial architecture reels (`.omni`).
//!   - RESOLUTION:      Candidate lexicon paths (`lexicon/`, `lib/`) searched for ontological symbol bindings.
//!   - SERIALIZATION:   Formatted machine listings, silicon hex words, structured JSON hovercards, and `.omni` transpiler reels.
//!
//! TOUCHES / SILICON HARDWARE & MMIO REGISTERS:
//!   - VM REGISTERS:    `%ip` (0), `%fp` (1), `%sp` (2), `%acc` (3), `%self` (4), `%caller` (5), `%swarm` (6), `%flags` (7).
//!   - MEMORY CELLS:    Partition allocation offsets (`@ 0x0000`), bounded partition slices (`v--`), storehouse capacities.
//!   - MACHINE WORDS:   Packed 64-bit balanced-ternary containers (`Word64`): 32 trits payload, QuadState, Polarity, Boundary, 7-bit checksum.
//!
//! REQUIRES / HARDWARE & RUNTIME DEPENDENCIES:
//!   - [`OmniDebugger`] for delimiter boundary stack invariant tracking (`[`, `{`, `<`).
//!   - [`DiagnosticLogger`] for flight recording, AI observations, and failpoint interception.
//!   - [`Lexer`] and [`Parser`] for token classification, pronoun categorization, and grammar lowering.
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_INPUT_RESOLVE`: Assertion of tape source existence.
//!   - `TRIPWIRE_SYNTAX_TOKENIZATION`: Lexical and grammar validation during roll tokenization.
//!   - `TRIPWIRE_MEMORY_BOUNDS`: Partition capacity bounds verification during symbol allocation.
//!   - `TRIPWIRE_COMMIT_INVARIANT`: Covenant seal assertion (`commit_status [ PROCEED ]`).
//!
//! NAVIGATION GUIDE (HOW TO NAVIGATE AND EXTEND THIS MODULE AS THE CODE GROWS):
//!   - Block 1 (Lines 1-85): Manifest, Scheduler, Architectural Duality, Modularization Rationale, Game Dev Ideology, Tripwires.
//!   - Block 2 (Lines 86-350): Opening Block, Scoped Imports, Machine ISA, Registers, Symbols, AssembledTape & OmniAssembler Struct.
//!   - Block 3 (Lines 351-850): Body Block & Execution Engine (Universal Hardware Translation, Serializers, Roll/Tape/Omni Lowering).
//!   - Block 4 (Lines 851+): Physical Baton Pass Contract & Comprehensive Verification Unit Tests.


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Physical Machine ISA, Registers, Symbols, AssembledTape & Defaults )
// ============================================================================

// ----------------------------------------------------------------------------
// 2.1. Scoped Namespace Ingestion & Multi-Platform Imports
// ----------------------------------------------------------------------------
use std::collections::HashMap;
use std::path::{Path, PathBuf};

use crate::core::boundaries::BoundaryType;
use crate::core::linguistics::PronounKind;
use crate::core::operators::Polarity;
use crate::core::tape::{Roll, Tape, TapeHead};
use crate::core::ternary::QuadState;
use crate::core::word::{Word32Trit, Word64};
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::parser::lexer::{Lexer, TokenType};
use crate::parser::parser::Parser;

// ----------------------------------------------------------------------------
// 2.2. Physical Machine Instruction Set Architecture (MachineOp)
// ----------------------------------------------------------------------------
/// Hardware Machine Opcodes for the OmniCode Balanced-Ternary Virtual Machine.
///
/// Encodes discrete silicon machine operations executed across 64-bit balanced-ternary
/// machine words. Each opcode corresponds directly to an atomic grammatical, causal,
```

```rust
/// or boundary transformation within the OmniCode lexicon.
#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum MachineOp {
    /// Opcode 0x00: No Operation (Equilibrium state marker).
    Nop              = 0x00,
    /// Opcode 0x01: Reference Abstract Archetype (`+Entity`). Symbolic lookup in Lexicon.
    RefAbstract      = 0x01,
    /// Opcode 0x02: Allocate Concrete Memory Cell (`Entity-`). Reserves physical silicon RAM cell.
    AllocConcrete    = 0x02,
    /// Opcode 0x03: Transduce Vector (`-->`). Forward causal drive, register write, or state propagation.
    TransduceVector  = 0x03,
    /// Opcode 0x04: Feedback Vector (`<--`). Reverse causal feedback, sensor tap, or state reflection.
    FeedbackVector   = 0x04,
    /// Opcode 0x05: Partition Slice (`v--`). Carves an isolated, bounded memory partition from a Storehouse.
    PartitionSlice   = 0x05,
    /// Opcode 0x06: Synthesize Layer (`--^`). Ascends abstraction layer or folds micro-states into macro.
    SynthesizeLayer  = 0x06,
    /// Opcode 0x07: Bind Channel (`::`). Binds hardware port, MMIO address, or inseparable pipeline channel.
    BindChannel      = 0x07,
    /// Opcode 0x08: Cast Boundary (`[+]`, `[-]`, `<+>`, `<->`). Boundary rank type conversion.
    CastBoundary     = 0x08,
    /// Opcode 0x09: Assert Invariant (`! [ ... ]`). Formal logic proof evaluation; halts on contradiction.
    AssertInvariant  = 0x09,
    /// Opcode 0x0A: Introspect Self (`myself`, `ourselves`). Self-audit telemetry and internal state reflection.
    IntrospectSelf   = 0x0A,
    /// Opcode 0x0B: Recurse Itself (`itself`). Homoiconic tape self-rewrite, loop fixpoint, or quine invocation.
    RecurseItself    = 0x0B,
    /// Opcode 0x0C: Assert Capability (`my`, `your`, `their`). Validates partition ownership and security authority.
    AssertCapability = 0x0C,
    /// Opcode 0x0D: Seal Commit (`commit_status [ PROCEED ]`). Irrevocably seals process covenant state.
    SealCommit       = 0x0D,
    /// Opcode 0x0E: Halt Equilibrium (`0 : HOLD`). Suspends execution head in neutral balanced ternary rest.
    HaltEquilibrium  = 0x0E,
    /// Opcode 0x0F: Trap Veto (`-1 : VETO`). Triggers tripwire collision and initiates affirmative rollback.
    TrapVeto         = 0x0F,
}

impl MachineOp {
    /// Returns the canonical uppercase mnemonic string for assembly listings and telemetry.
    pub fn mnemonic(&self) -> &'static str {
        match self {
            Self::Nop => "NOP",
            Self::RefAbstract => "REF_ABSTRACT",
            Self::AllocConcrete => "ALLOC_CONCRETE",
            Self::TransduceVector => "TRANSDUCE_VECTOR",
            Self::FeedbackVector => "FEEDBACK_VECTOR",
            Self::PartitionSlice => "PARTITION_SLICE",
            Self::SynthesizeLayer => "SYNTHESIZE_LAYER",
            Self::BindChannel => "BIND_CHANNEL",
            Self::CastBoundary => "CAST_BOUNDARY",
            Self::AssertInvariant => "ASSERT_INVARIANT",
            Self::IntrospectSelf => "INTROSPECT_SELF",
            Self::RecurseItself => "RECURSE_ITSELF",
            Self::AssertCapability => "ASSERT_CAPABILITY",
            Self::SealCommit => "SEAL_COMMIT",
            Self::HaltEquilibrium => "HALT_EQUILIBRIUM",
            Self::TrapVeto => "TRAP_VETO",
        }
    }

    /// Resolves a mnemonic string back into its hardware opcode.
    pub fn from_mnemonic(s: &str) -> Option<Self> {
        match s.trim().to_uppercase().as_str() {
            "NOP" => Some(Self::Nop),
            "REF_ABSTRACT" => Some(Self::RefAbstract),
            "ALLOC_CONCRETE" => Some(Self::AllocConcrete),
            "TRANSDUCE_VECTOR" => Some(Self::TransduceVector),
            "FEEDBACK_VECTOR" => Some(Self::FeedbackVector),
            "PARTITION_SLICE" => Some(Self::PartitionSlice),
            "SYNTHESIZE_LAYER" => Some(Self::SynthesizeLayer),
            "BIND_CHANNEL" => Some(Self::BindChannel),
            "CAST_BOUNDARY" => Some(Self::CastBoundary),
            "ASSERT_INVARIANT" => Some(Self::AssertInvariant),
            "INTROSPECT_SELF" => Some(Self::IntrospectSelf),
            "RECURSE_ITSELF" => Some(Self::RecurseItself),
            "ASSERT_CAPABILITY" => Some(Self::AssertCapability),
            "SEAL_COMMIT" => Some(Self::SealCommit),
```

```
            "HALT_EQUILIBRIUM" => Some(Self::HaltEquilibrium),
            "TRAP_VETO" => Some(Self::TrapVeto),
            _ => None,
        }
    }

    /// Returns `true` if this opcode represents a forward or reverse directional causality vector.
    pub fn is_causal(&self) -> bool {
        matches!(self, Self::TransduceVector | Self::FeedbackVector | Self::PartitionSlice | Self::SynthesizeLayer)
    }

    /// Returns `true` if this opcode reserves or affects physical silicon memory cells.
    pub fn is_memory_allocation(&self) -> bool {
        matches!(self, Self::AllocConcrete | Self::PartitionSlice)
    }
}

// ---------------------------------------------------------------------------
// 2.3. Physical Machine CPU Registers (MachineRegister)
// ---------------------------------------------------------------------------
/// Physical Machine CPU Registers for the Balanced-Ternary Virtual Machine.
///
/// OmniCode maps natural physical tape mechanisms, grammatical perspectives,
/// and streaming prepositions directly to 13 named silicon CPU registers:
/// - `HEAD`    (0): Tape Head / Instruction Pointer / Witness Cursor (`%head`, `%ip`).
/// - `HOME`    (1): Genesis Base Pointer / Pristine Rollback Baseline (`%home`, `%fp`).
/// - `SPOOL`   (2): Ribbon Stack Spool / Partition Reservoir (`%spool`, `%sp`).
/// - `IT`      (3): Pipeline Antecedent / Primary Accumulator (`%it`, `%acc`).
/// - `SELF`    (4): 1st-Person Execution Identity (`%self`, `I`, `me`).
/// - `CALLER`  (5): 2nd-Person Invoking Principal / User Port (`%caller`, `you`).
/// - `OTHER`   (6): 3rd-Person Peer / Swarm Worker Handle (`%other`, `they`, `them`).
/// - `STATE`   (7): Status & Balanced-Ternary QuadState Register (`%state`, `%flags`).
/// - `TURN`    (8): Temporal Cadence / Cycle Counter / Loop Turn (`%turn`, `%step`).
/// - `BOUND`   (9): Invariant Axiom Boundary Watchdog Limit (`%bound`).
/// - `FROM`   (10): Inbound Streaming Transducer Port (`%from`, `input`).
/// - `INTO`   (11): Outbound Streaming Transducer Port (`%into`, `output`).
/// - `DISCARD`(12): Anonymous Discard Sink (`%discard`, `_`, `void`).
#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum MachineRegister {
    /// Register 0 (%head / %here / %ip): Tape Head / Instruction Pointer / Witness Cursor.
    HEAD = 0,
    /// Register 1 (%home / %origin / %fp): Genesis Base Pointer / Rollback Anchor.
    HOME = 1,
    /// Register 2 (%spool / %owner / %sp): Ribbon Stack Spool / Stack Floor.
    SPOOL = 2,
    /// Register 3 (%it / %acc): Pipeline Antecedent / Primary Accumulator (`it`).
    IT = 3,
    /// Register 4 (%self): 1st-Person Execution Identity (`I`, `me`, `self`).
    SELF = 4,
    /// Register 5 (%caller): 2nd-Person Invoking Principal (`you`, `caller`).
    CALLER = 5,
    /// Register 6 (%other / %swarm): 3rd-Person Peer / Worker Swarm (`other`, `them`).
    OTHER = 6,
    /// Register 7 (%state / %flags): Status & Balanced-Ternary QuadState Register.
    STATE = 7,
    /// Register 8 (%turn / %step): Temporal Cadence & Loop Turn Register (`TURN`).
    TURN = 8,
    /// Register 9 (%bound): Invariant Boundary Watchdog Limit (`BOUND`).
    BOUND = 9,
    /// Register 10 (%from / %input): Inbound Streaming Transducer Port (`FROM`).
    FROM = 10,
    /// Register 11 (%into / %output): Outbound Streaming Transducer Port (`INTO`).
    INTO = 11,
    /// Register 12 (%discard): Anonymous Discard Sink (`DISCARD`, `_`).
    DISCARD = 12,
}

impl MachineRegister {
    /// Canonical and legacy aliases
    pub const HERE: Self = Self::HEAD;
    pub const IP: Self = Self::HEAD;
    pub const ORIGIN: Self = Self::HOME;
    pub const FP: Self = Self::HOME;
    pub const OWNER: Self = Self::SPOOL;
    pub const SP: Self = Self::SPOOL;
    pub const ACC: Self = Self::IT;
    pub const SWARM: Self = Self::OTHER;
```

```rust
    pub const FLAGS: Self = Self::STATE;
    pub const STEP: Self = Self::TURN;
    pub const INPUT: Self = Self::FROM;
    pub const OUTPUT: Self = Self::INTO;

    /// Returns the canonical hardware register identifier (e.g., `%head`, `%home`, `%it`).
    pub fn name(&self) -> &'static str {
        match self {
            Self::HEAD => "%head",
            Self::HOME => "%home",
            Self::SPOOL => "%spool",
            Self::IT => "%it",
            Self::SELF => "%self",
            Self::CALLER => "%caller",
            Self::OTHER => "%other",
            Self::STATE => "%state",
            Self::TURN => "%turn",
            Self::BOUND => "%bound",
            Self::FROM => "%from",
            Self::INTO => "%into",
            Self::DISCARD => "%discard",
        }
    }

    /// Returns the universal ALL CAPS silicon register name (e.g., `HEAD`, `HOME`, `IT`).
    pub fn hardware_name(&self) -> &'static str {
        match self {
            Self::HEAD => "HEAD",
            Self::HOME => "HOME",
            Self::SPOOL => "SPOOL",
            Self::IT => "IT",
            Self::SELF => "SELF",
            Self::CALLER => "CALLER",
            Self::OTHER => "OTHER",
            Self::STATE => "STATE",
            Self::TURN => "TURN",
            Self::BOUND => "BOUND",
            Self::FROM => "FROM",
            Self::INTO => "INTO",
            Self::DISCARD => "DISCARD",
        }
    }

    /// Returns the physical silicon register numeric index (0 to 12).
    pub fn index(&self) -> u8 {
        *self as u8
    }

    /// Resolves any register string back to its enum variant, supporting ALL CAPS, lowercase,
    /// shorthand aliases, hardware `%` prefixes, and polarity `+`/`-` affixes.
    pub fn from_name(s: &str) -> Option<Self> {
        let clean = s.trim().trim_start_matches(|c| c == '+' || c == '-' || c == '%').to_lowercase();
        match clean.as_str() {
            "head" | "here" | "ip" | "hd" => Some(Self::HEAD),
            "home" | "origin" | "fp" | "hm" => Some(Self::HOME),
            "spool" | "owner" | "sp" => Some(Self::SPOOL),
            "it" | "acc" => Some(Self::IT),
            "self" | "me" | "i" | "sl" => Some(Self::SELF),
            "caller" | "you" | "cl" => Some(Self::CALLER),
            "other" | "swarm" | "them" | "they" | "ot" => Some(Self::OTHER),
            "state" | "flags" | "status" | "st" => Some(Self::STATE),
            "turn" | "step" | "cadence" | "cycle" | "tr" | "tn" => Some(Self::TURN),
            "bound" | "limit" | "watchdog" | "bd" => Some(Self::BOUND),
            "from" | "input" | "in" | "fr" | "source" => Some(Self::FROM),
            "into" | "output" | "out" | "to" | "target" => Some(Self::INTO),
            "discard" | "void" | "_" | "dc" => Some(Self::DISCARD),
            _ => None,
        }
    }
}

// ---------------------------------------------------------------------------
// 2.4. Physical Silicon Memory Symbols (PhysicalSymbol)
// ---------------------------------------------------------------------------
/// Physical allocation record for an ontological symbol in silicon memory.
///
/// Tracks polarity classification (`+` abstract archetype vs `-` concrete silicon memory cell),
/// assigned partition offset (`@ 0x0000`), capacity footprint in cells, and enclosing partition.
```

```rust
#[derive(Debug, Clone, PartialEq)]
pub struct PhysicalSymbol {
    /// Ontological name identifier.
    pub name: String,
    /// Formal polarity: Abstract (`+`) or Concrete (`-`).
    pub polarity: Polarity,
    /// Explicit zero-based memory cell offset within the designated partition.
    pub offset: Option<usize>,
    /// Number of contiguous physical RAM cells allocated to this symbol.
    pub size_cells: usize,
    /// Name of the enclosing memory partition or storehouse where allocated.
    pub partition_name: Option<String>,
}

impl PhysicalSymbol {
    /// Returns `true` if this symbol denotes a physical silicon allocation (`-`).
    pub fn is_concrete(&self) -> bool {
        self.polarity == Polarity::Concrete
    }

    /// Returns `true` if this symbol denotes an unallocated abstract archetype (`+`).
    pub fn is_abstract(&self) -> bool {
        self.polarity == Polarity::Abstract
    }
}

// ---------------------------------------------------------------------------
// 2.5. Lowered Machine Instruction Container (AssembledInstruction)
// ---------------------------------------------------------------------------
/// A lowered, assembled instruction roll ready for hardware dispatch.
///
/// Contains the discrete machine opcode, assigned register, silicon memory offset,
/// source provenance, and the 64-bit balanced-ternary machine word ([`Word64`]).
#[derive(Debug, Clone, PartialEq)]
pub struct AssembledInstruction {
    /// 1-based roll position within the execution tape.
    pub roll_index: usize,
    /// 1-based source line number from the original `.tape` or `.omni` document.
    pub source_line: usize,
    /// Raw un-lowered text content of the roll for diagnostics and hovercards.
    pub original_text: String,
    /// Discrete physical machine opcode.
    pub op: MachineOp,
    /// Bound hardware CPU register, if applicable.
    pub register: Option<MachineRegister>,
    /// Target physical memory cell offset, if applicable.
    pub address: Option<usize>,
    /// Packed 64-bit balanced-ternary machine word container.
    pub word: Word64,
    /// QuadState execution status for this instruction (+1:PROCEED).
    pub status: QuadState,
    /// Human-readable operational code string.
    pub op_code: String,
}

// ---------------------------------------------------------------------------
// 2.6. Assembled Machine Tape Container (AssembledTape)
// ---------------------------------------------------------------------------
/// The complete assembled physical machine tape.
///
/// Represents the lowered artifact ready for silicon execution, containing the Tape Head
/// covenant, allocated partition geometry, physical symbol table, instruction roll sequence,
/// and the final affirmative commit seal status.
#[derive(Debug, Clone, PartialEq)]
pub struct AssembledTape {
    /// Process covenant manifest header.
    pub head: TapeHead,
    /// Total cell capacity carved for this memory partition.
    pub partition_size: usize,
    /// Resolved physical symbol table mapping identifiers to silicon offsets.
    pub symbols: HashMap<String, PhysicalSymbol>,
    /// Sequential stream of lowered physical machine instructions.
    pub instructions: Vec<AssembledInstruction>,
    /// Final affirmative seal state (+1:PROCEED).
    pub commit_status: QuadState,
}

impl AssembledTape {
```

```rust
    /// Formats the assembled machine tape as a human-readable assembly listing.
    pub fn format_listing(&self) -> String {
        format_assembly_listing(self)
    }

    /// Emits the human-readable physical silicon word stream (Hex, registers, opcodes, QuadState).
    pub fn format_words(&self) -> String {
        format_silicon_words(self)
    }

    /// Emits a structured JSON representation for tooling, IDEs, and diagnostics.
    pub fn format_json(&self) -> String {
        format_tape_json(self)
    }

    /// Transpiles the assembled tape into a declarative 4-Block spatial architecture reel (`.omni`).
    pub fn format_omni(&self) -> String {
        transpile_tape_to_omni(self)
    }

    /// Transpiles the assembled tape into native bare-metal x86 bootstrap assembly (`.asm`).
    pub fn format_asm(&self) -> String {
        transpile_tape_to_asm(self)
    }

    /// Transpiles the assembled tape into 32-bit Multiboot ELF assembly (`.asm`).
    pub fn format_multiboot_asm(&self) -> String {
        transpile_tape_to_multiboot_asm(self)
    }

    /// Emits the raw physical Word64 machine word sequence for hardware dispatch.
    pub fn to_machine_words(&self) -> Vec<Word64> {
        self.instructions.iter().map(|i| i.word).collect()
    }
}

// ---------------------------------------------------------------------------
// 2.7. Initial Truth States & Equilibrium Defaults
// ---------------------------------------------------------------------------
/// Baseline cell capacity reserved for unspecified memory partitions.
pub const DEFAULT_PARTITION_CELL_CAPACITY: usize = 64;
/// Maximum delimiter boundary nesting depth for structural invariant assertions.
pub const DEFAULT_ASSEMBLER_DEBUGGER_DEPTH: usize = 100;

// ---------------------------------------------------------------------------
// 2.8. Physical Silicon Assembler Interface Handle (OmniAssembler)
// ---------------------------------------------------------------------------
/// The primary silicon lowering engine and compiler interface.
///
/// Encapsulates the runtime diagnostic logger, boundary invariant debugger ([`OmniDebugger`]),
/// and candidate lexicon search paths required to resolve and lower ontological symbols into silicon memory.
#[derive(Debug, Clone)]
pub struct OmniAssembler {
    /// Attached telemetry and error interception logger.
    pub logger: DiagnosticLogger,
    /// Delimiter boundary stack tracking engine for structural invariants (`[...]`, `<...>`).
    pub debugger: OmniDebugger,
    /// Candidate filesystem search directories for modular lexicon resolution.
    pub lexicon_paths: Vec<PathBuf>,
}


// ===========================================================================
// BLOCK 3: BODY BLOCK
// ( Universal ISA Translation Functions, Serializers, Transpilers & OmniAssembler )
// ===========================================================================

// ---------------------------------------------------------------------------
// 3.1. Universal Hardware ISA Translation & Machine Word Synthesis
// ---------------------------------------------------------------------------
/// Universal mapping from grammatical pronouns to physical CPU registers.
///
/// Translates natural linguistic perspectives into concrete VM register bindings:
/// - `I`, `me`, `self` -> `%self` (Register 4)
/// - `you`, `caller` -> `%caller` (Register 5)
/// - `it` -> `%acc` (Register 3)
/// - `they`, `them`, `themselves` -> `%swarm` (Register 6)
pub fn pronoun_to_register(p: PronounKind) -> Option<MachineRegister> {
```

```
    match p {
        PronounKind::I | PronounKind::Me | PronounKind::SelfHandle => Some(MachineRegister::SELF),
        PronounKind::You | PronounKind::CallerHandle => Some(MachineRegister::CALLER),
        PronounKind::OwnerHandle => Some(MachineRegister::SPOOL),
        PronounKind::It => Some(MachineRegister::IT),
        PronounKind::They | PronounKind::Them | PronounKind::Themselves => Some(MachineRegister::OTHER),
        PronounKind::Wildcard => Some(MachineRegister::DISCARD),
        _ => None,
    }
}

/// Universal mapping from reflexive, recursive, or possessive pronouns to hardware opcodes.
///
/// - `myself`, `ourselves` -> `MachineOp::IntrospectSelf`
/// - `itself` -> `MachineOp::RecurseItself`
/// - `themselves` -> `MachineOp::IntrospectSelf`
/// - Possessive pronouns (`my`, `your`, `their`) -> `MachineOp::AssertCapability`
pub fn pronoun_to_op(p: PronounKind) -> Option<MachineOp> {
    match p {
        PronounKind::Myself => Some(MachineOp::IntrospectSelf),
        PronounKind::Itself => Some(MachineOp::RecurseItself),
        PronounKind::Themselves => Some(MachineOp::IntrospectSelf),
        _ if p.is_possessive() => Some(MachineOp::AssertCapability),
        _ => None,
    }
}

/// Universal mapping from formal polarity affixes to memory opcodes.
///
/// - Abstract (`+`) -> `MachineOp::RefAbstract`
/// - Concrete (`-`) -> `MachineOp::AllocConcrete`
pub fn polarity_to_op(polarity: Polarity) -> MachineOp {
    match polarity {
        Polarity::Abstract => MachineOp::RefAbstract,
        Polarity::Concrete => MachineOp::AllocConcrete,
    }
}

/// Universal computation of the 7-bit balanced-ternary machine word checksum.
pub fn compute_word_checksum(idx: usize, op: MachineOp) -> u8 {
    ((idx as u8) ^ (op as u8)) & 0x7F
}

/// Universal synthesizer packing payload trits, status, polarity, boundary rank,
/// and checksum into a physical 64-bit balanced-ternary machine word container ([`Word64`]).
pub fn synthesize_machine_word(
    idx: usize,
    op: MachineOp,
    addr: Option<usize>,
    status: QuadState,
    polarity: Polarity,
    boundary: BoundaryType,
) -> Word64 {
    let payload_trits = addr.map_or_else(Word32Trit::zero, |a| Word32Trit::from_i64(a as i64));
    let checksum = compute_word_checksum(idx, op);
    Word64::new(payload_trits, status, polarity, boundary, checksum)
}

// ---------------------------------------------------------------------------
// 3.2. Universal Serialization & Transpilation Functions
// ---------------------------------------------------------------------------
/// Formats an assembled machine tape as a human-readable annotated assembly listing.
pub fn format_assembly_listing(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("============================================================\n");
    out.push_str(&format!("ASSEMBLED MACHINE TAPE: [{}]\n", tape.head.name));
    out.push_str(&format!("PRINCIPAL: {} | TARGET: {}\n", tape.head.who, tape.head.what));
    out.push_str(&format!("MEMORY PARTITION: {} cells\n", tape.partition_size));
    out.push_str("------------------------------------------------------------\n");
    out.push_str("PHYSICAL SYMBOL TABLE:\n");
    for (name, sym) in &tape.symbols {
        let offset_str = sym.offset.map_or("N/A".to_string(), |o| format!("@ 0x{:04X} ({})", o, o));
        out.push_str(&format!(
            "  • {:<16} | Polarity: {:<8} | Size: {:>3} cells | Offset: {}\n",
            name, sym.polarity.symbol(), sym.size_cells, offset_str
        ));
    }
    out.push_str("------------------------------------------------------------\n");
```

```
    out.push_str("ASSEMBLED INSTRUCTION ROLLS:\n");
    for inst in &tape.instructions {
        let addr_str = inst.address.map_or("      ".to_string(), |a| format!("@{:04X}", a));
        let reg_str = inst.register.map_or("     ".to_string(), |r| format!("{:<5}", r.name()));
        out.push_str(&format!(
            "  [{:>2}] {} | {} | {:<20} | Word: 0x{:016X} | Status: {}\n        --> {}\n",
            inst.roll_index, addr_str, reg_str, inst.op_code, inst.word.raw, inst.status.macro_op(), inst.original_text
        ));
    }
    out.push_str("============================================================\n");
    out.push_str(&format!("COMMIT SEAL: {}\n", tape.commit_status.macro_op()));
    out
}

/// Emits the human-readable physical silicon word stream (Hex, registers, opcodes, QuadState).
pub fn format_silicon_words(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str(&format!("PHYSICAL SILICON WORD STREAM: [{}]\n", tape.head.name));
    out.push_str("------------------------------------------------------------\n");
    for (idx, inst) in tape.instructions.iter().enumerate() {
        let reg_str = inst.register.map_or("     ".to_string(), |r| format!("{:<5}", r.name()));
        out.push_str(&format!(
            "[{:>3}] 0x{:016X} | {} | {:<20} | Status: {}\n",
            idx + 1,
            inst.word.raw,
            reg_str,
            inst.op.mnemonic(),
            inst.status.macro_op()
        ));
    }
    out.push_str("------------------------------------------------------------\n");
    out.push_str(&format!("TOTAL WORDS: {}\n", tape.instructions.len()));
    out
}

/// Emits a structured JSON representation of an assembled tape for tooling, IDEs, and diagnostics.
pub fn format_tape_json(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("{\n");
    out.push_str(&format!("  \"tape\": \"{}\",\n", tape.head.name));
    out.push_str(&format!("  \"principal\": \"{}\",\n", tape.head.who));
    out.push_str(&format!("  \"target\": \"{}\",\n", tape.head.what));
    out.push_str(&format!("  \"partition_size\": {},\n", tape.partition_size));
    out.push_str(&format!("  \"commit_status\": \"{}\",\n", tape.commit_status.macro_op()));
    out.push_str("  \"symbols\": [\n");
    let sym_list: Vec<_> = tape.symbols.values().collect();
    for (i, s) in sym_list.iter().enumerate() {
        let comma = if i + 1 < sym_list.len() { "," } else { "" };
        let off_str = s.offset.map_or("null".to_string(), |o| o.to_string());
        out.push_str(&format!(
            "    {{ \"name\": \"{}\", \"polarity\": \"{}\", \"size\": {}, \"offset\": {} }}{}\n",
            s.name, s.polarity.symbol(), s.size_cells, off_str, comma
        ));
    }
    out.push_str("  ],\n");
    out.push_str("  \"instructions\": [\n");
    for (i, inst) in tape.instructions.iter().enumerate() {
        let comma = if i + 1 < tape.instructions.len() { "," } else { "" };
        let reg_str = inst.register.map_or("null".to_string(), |r| format!("\"{}\"", r.name()));
        let addr_str = inst.address.map_or("null".to_string(), |a| a.to_string());
        out.push_str(&format!(
            "    {{ \"roll\": {}, \"op\": \"{}\", \"reg\": {}, \"address\": {}, \"word\": \"0x{:016X}\", \"status\": \"
{}\" }}{}\n",
            inst.roll_index, inst.op.mnemonic(), reg_str, addr_str, inst.word.raw, inst.status.macro_op(), comma
        ));
    }
    out.push_str("  ]\n");
    out.push_str("}\n");
    out
}

/// Transpiles an assembled tape from temporal process ribbon (`.tape`) into
/// a declarative, 4-Block spatial architecture reel (`.omni`).
pub fn transpile_tape_to_omni(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!("# ( Transpiled Spatial Reel: {} )\n", tape.head.name));
```

```
    out.push_str("# ================================================================================\n");
    out.push_str(&format!("Module < {} > ( Transpiled by OmniAssembler ) {{\n", tape.head.name));
    out.push_str("    version: < \"1.0.0\" >,\n");
    out.push_str("    system: < OmniCode.Transpiled >,\n");
    out.push_str(&format!("    principal: < \"{}\" >,\n", tape.head.who));
    out.push_str(&format!("    target: < \"{}\" >,\n", tape.head.what));
    out.push_str(&format!("    partition_capacity: < {} >\n", tape.partition_size));
    out.push_str("}\n\n");

    if !tape.symbols.is_empty() {
        out.push_str("# Physical Silicon Symbol Allocations\n");
        for (name, sym) in &tape.symbols {
            let off_desc = sym.offset.map_or("dynamic".to_string(), |o| format!("@ 0x{:04X}", o));
            let pol_char = sym.polarity.symbol();
            out.push_str(&format!(
                "Entity_{} < PhysicalSymbol > ( Silicon memory cell [{} {}] ) {{\n    polarity: < \"{}\" >,\n    capacity_cells: < {} >\n}}\n\n",
                name, pol_char, off_desc, pol_char, sym.size_cells
            ));
        }
    }

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 2: OPENING BLOCK\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Opening {\n");
    out.push_str("    requires: [\n");
    out.push_str(&format!("        PartitionCapacity : {} cells ::\n", tape.partition_size));
    out.push_str(&format!("        TapeHead : {} [ {} ]\n", tape.head.name, tape.head.who));
    out.push_str("    ],\n");
    out.push_str("    preconditions: [ +1:PROCEED ]\n");
    out.push_str("}\n\n");

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 3: BODY BLOCK\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Body [\n");
    for inst in &tape.instructions {
        let reg_str = inst.register.map_or(String::new(), |r| format!(" ( Reg: {} )", r.name()));
        out.push_str(&format!("    Roll_{} [ {} ]{}\n", inst.roll_index, inst.original_text, reg_str));
    }
    out.push_str("]\n\n");

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 4: CLOSING BLOCK\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Closing {\n");
    out.push_str("    verify: [\n");
    out.push_str(&format!("        InstructionCount = {} ::\n", tape.instructions.len()));
    out.push_str(&format!("        SymbolCount = {}\n", tape.symbols.len()));
    out.push_str("    ],\n");
    out.push_str(&format!("    commit_status [ {} ]\n", tape.commit_status.macro_op()));
    out.push_str("}\n");

    out
}

/// Transpiles an assembled tape from declarative 4-Block spatial architecture reel (`.omni`)
/// into an executable procedural process ribbon (`.tape`).
pub fn transpile_omni_to_tape(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!("# ( Procedural Process Ribbon Transpiled from Omni Reel: {} )\n", tape.head.name));
    out.push_str("# ================================================================================\n\n");
    out.push_str(&format!(
        "[Tape.{}] : WHO {{{}}} WHAT {{{}}} --> BOOT {{{}}}\n    WHEN {{Scheduled}}\n    WHERE {{{}}} = [ {} ]\n    WHY ( Executable procedural ribbon lowered from declarative 4-block architecture )\n    HOW [ +1 : {} ]\n    :: < ancient_landmark >\n\n",
        tape.head.name, tape.head.who, tape.head.what, tape.head.name, tape.head.where_clause, tape.partition_size, tape.commit_status.macro_op()
    ));

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 2: OPENING BLOCK\n");
    out.push_str("# ( Preconditions & Memory Allocations )\n");
    out.push_str("# ================================================================================\n\n");
```

```
    out.push_str("# ============================================================================\n");
    out.push_str("# BLOCK 3: BODY BLOCK\n");
    out.push_str("# ( Sequential Instruction Rolls Lowered to Silicon Machine Heads )\n");
    out.push_str("# ============================================================================\n\n");

    for inst in &tape.instructions {
        out.push_str(&format!("roll_{} [ {} ]\n", inst.roll_index, inst.original_text));
    }

    out.push_str("\n# ============================================================================\n");
    out.push_str("# BLOCK 4: CLOSING BLOCK\n");
    out.push_str("# ( Covenant Commit Seal )\n");
    out.push_str("# ============================================================================\n\n");
    out.push_str(&format!("seal_roll [ commit_status [ {} ] ]\n", tape.commit_status.macro_op()));

    out
}

/// Transpiles an assembled tape into a bare-metal x86 bootstrap assembly source file (`.asm`)
/// suitable for direct assembly by NASM into an MBR 512-byte boot sector for MilleniumOS.
pub fn transpile_tape_to_asm(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("; ============================================================================\n");
    out.push_str("; BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!("; ( MilleniumOS Bare-Metal Genesis Bootloader - x86 MBR 512-Byte BIOS Sector )\n"));
    out.push_str(&format!("; ( Transpiled by OmniAssembler from OmniCode Process: {} )\n", tape.head.name));
    out.push_str("; ============================================================================\n");
    out.push_str("; ARCHITECTURAL TOPOLOGY:\n");
    out.push_str(";   Physical boot sector loaded by legacy BIOS at 0x7C00 in 16-bit Real Mode.\n");
    out.push_str(";   - Initializes COM1 UART Serial Port (0x03F8) for telemetric logging.\n");
    out.push_str(";   - Writes directly to MMIO VGA Video Text Buffer (0x000B8000) and BIOS teletype.\n");
    out.push_str(";   - Computes Balanced Ternary (+1, 0, -1) ALU state in hardware registers.\n");
    out.push_str(";   - Arms the 13 Named Silicon Registers (%head, %home, %spool, %it, etc.).\n");
    out.push_str(";   - Enforces Anti-Lockout Watchdog and enters low-power HLT loop.\n");
    out.push_str("; ============================================================================\n\n");
    out.push_str("[BITS 16]\n");
    out.push_str("[ORG 0x7C00]\n\n");
    out.push_str("start:\n");
    out.push_str("    cli\n");
    out.push_str("    xor ax, ax\n");
    out.push_str("    mov ds, ax\n");
    out.push_str("    mov es, ax\n");
    out.push_str("    mov ss, ax\n");
    out.push_str("    mov sp, 0x7C00\n");
    out.push_str("    sti\n\n");
    out.push_str("    ; 1. Initialize COM1 Serial (FROM ::@ 0x03F8) 115200 8N1\n");
    out.push_str("    mov dx, 0x03F9\n");
    out.push_str("    xor al, al\n");
    out.push_str("    out dx, al\n\n");
    out.push_str("    mov dx, 0x03FB\n");
    out.push_str("    mov al, 0x80\n");
    out.push_str("    out dx, al\n\n");
    out.push_str("    mov dx, 0x03F8\n");
    out.push_str("    mov al, 0x01\n");
    out.push_str("    out dx, al\n\n");
    out.push_str("    mov dx, 0x03F9\n");
    out.push_str("    xor al, al\n");
    out.push_str("    out dx, al\n\n");
    out.push_str("    mov dx, 0x03FB\n");
    out.push_str("    mov al, 0x03\n");
    out.push_str("    out dx, al\n\n");
    out.push_str("    ; 2. Initialize Screen (INTO ::@ 0x000B8000) 80x25 text mode\n");
    out.push_str("    mov ax, 0x0003\n");
    out.push_str("    int 0x10\n\n");
    out.push_str("    ; 3. Print Boot Banner\n");
    out.push_str("    mov si, boot_msg\n");
    out.push_str("    call print_both\n\n");
    out.push_str("    ; 4. Balanced Ternary ALU Verification: (+1) + (-1) = 0\n");
    out.push_str("    mov bx, 1\n");
    out.push_str("    mov cx, -1\n");
    out.push_str("    add bx, cx\n");
    out.push_str("    test bx, bx\n");
    out.push_str("    jnz hang\n\n");
    out.push_str("    mov si, alu_msg\n");
    out.push_str("    call print_both\n\n");
    out.push_str("    mov si, online_msg\n");
    out.push_str("    call print_both\n\n");
    out.push_str("hang:\n");
```

```
    out.push_str("    hlt\n");
    out.push_str("    jmp hang\n\n");
    out.push_str("print_both:\n");
    out.push_str(".next_char:\n");
    out.push_str("    lodsb\n");
    out.push_str("    test al, al\n");
    out.push_str("    jz .done\n\n");
    out.push_str("    push ax\n");
    out.push_str("    mov ah, 0x0E\n");
    out.push_str("    mov bx, 0x000A\n");
    out.push_str("    int 0x10\n");
    out.push_str("    pop ax\n\n");
    out.push_str("    mov dx, 0x03FD\n");
    out.push_str(".wait_tx:\n");
    out.push_str("    in al, dx\n");
    out.push_str("    test al, 0x20\n");
    out.push_str("    jz .wait_tx\n\n");
    out.push_str("    mov al, [si - 1]\n");
    out.push_str("    mov dx, 0x03F8\n");
    out.push_str("    out dx, al\n");
    out.push_str("    jmp .next_char\n\n");
    out.push_str(".done:\n");
    out.push_str("    ret\n\n");
    out.push_str("boot_msg:\n");
    out.push_str("    db 13, 10\n");
    out.push_str("    db \"=== MILLENIUMOS: OMNICODE 32-TRIT OS [BIOS x86] ===\", 13, 10\n");
    out.push_str("    db \"[BOOT] 64-Bit Silicon Envelope: OK\", 13, 10\n");
    out.push_str("    db \"[BOOT] Unpacking 32-Trit Word:  OK\", 13, 10, 0\n\n");
    out.push_str("alu_msg:\n");
    out.push_str("    db \"[ALU ] (+1)+(-1)=0 Equilibrium: OK\", 13, 10\n");
    out.push_str("    db \"[REGS] 13 Silicon Regs: ARMED\", 13, 10\n");
    out.push_str("    db \"[LOOP] Quad-State Loop: CALIBRATED\", 13, 10\n");
    out.push_str("    db \"[REEL] 4-Dir Spool Hierarchy:   OK\", 13, 10\n");
    out.push_str("    db \"[GATE] Anti-Lockout Watchdog:   OK\", 13, 10\n");
    out.push_str("    db \"[MMIO] VGA @ 0xB8000 & COM1:    OK\", 13, 10, 0\n\n");
    out.push_str("online_msg:\n");
    out.push_str("    db \"=== MILLENIUMOS: OMNICODE OS ONLINE :: PROCEED ===\", 13, 10, 0\n\n");
    out.push_str("times 510 - ($ - $$) db 0\n");
    out.push_str("dw 0xAA55\n");
    out
}

/// Transpiles an assembled tape into a 32-bit Multiboot ELF assembly source file (`.asm`)
/// suitable for direct assembly by NASM for direct hypervisor handshake.
pub fn transpile_tape_to_multiboot_asm(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("; ================================================================================\n");
    out.push_str("; BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!("; ( MilleniumOS Multiboot Kernel - Transpiled from OmniCode: {} )\n", tape.head.name));
    out.push_str("; ================================================================================\n\n");
    out.push_str("[BITS 32]\n\n");
    out.push_str("section .multiboot\n");
    out.push_str("align 4\n");
    out.push_str("    dd 0x1BADB002                   ; Multiboot magic\n");
    out.push_str("    dd 0x00000003                   ; Flags: align modules + memory info\n");
    out.push_str("    dd -(0x1BADB002 + 0x00000003)   ; Checksum\n\n");
    out.push_str("section .text\n");
    out.push_str("global _start\n");
    out.push_str("_start:\n");
    out.push_str("    cli\n\n");
    out.push_str("    ; 1. Initialize COM1 Serial (0x03F8) 115200 8N1\n");
    out.push_str("    mov dx, 0x03F9\n    xor al, al\n    out dx, al\n");
    out.push_str("    mov dx, 0x03FB\n    mov al, 0x80\n    out dx, al\n");
    out.push_str("    mov dx, 0x03F8\n    mov al, 0x01\n    out dx, al\n");
    out.push_str("    mov dx, 0x03F9\n    xor al, al\n    out dx, al\n");
    out.push_str("    mov dx, 0x03FB\n    mov al, 0x03\n    out dx, al\n\n");
    out.push_str("    ; 2. Print to Serial COM1\n");
    out.push_str("    mov esi, msg\n");
    out.push_str(".serial_loop:\n");
    out.push_str("    lodsb\n    test al, al\n    jz .serial_done\n    mov dx, 0x03FD\n");
    out.push_str(".wait_tx:\n    in al, dx\n    test al, 0x20\n    jz .wait_tx\n");
    out.push_str("    mov al, [esi - 1]\n    mov dx, 0x03F8\n    out dx, al\n    jmp .serial_loop\n");
    out.push_str(".serial_done:\n\n");
    out.push_str("    ; 3. Print to MMIO VGA Video Text Buffer at 0x000B8000\n");
    out.push_str("    mov edi, 0x000B8000\n    mov esi, vga_msg\n    mov ah, 0x0A\n");
    out.push_str(".vga_loop:\n    lodsb\n    test al, al\n    jz .vga_done\n    mov [edi], ax\n    add edi, 2\n    jmp
.vga_loop\n");
    out.push_str(".vga_done:\n\n");
```

```
    out.push_str("hang:\n    hlt\n    jmp hang\n\n");
    out.push_str("section .rodata\n");
    out.push_str("msg:\n");
    out.push_str("    db 13, 10\n");
    out.push_str("    db \"========================================\", 13, 10\n");
    out.push_str("    db \"  MILLENIUMOS 32-TRIT OS [MULTIBOOT/GRUB]\", 13, 10\n");
    out.push_str("    db \"  Direct Hypervisor Kernel Handshake\", 13, 10\n");
    out.push_str("    db \"========================================\", 13, 10\n");
    out.push_str("    db \"[BOOT] Direct Kernel ELF Handshake:     OK\", 13, 10\n");
    out.push_str("    db \"[BOOT] 64-Bit Silicon Envelope Mounted: OK\", 13, 10\n");
    out.push_str("    db \"[ALU ] Balanced (+1)+(-1)=0 Eq:         VERIFIED\", 13, 10\n");
    out.push_str("    db \"[REGS] 13 Named Silicon Registers:      ARMED\", 13, 10\n");
    out.push_str("    db \"[LOOP] Quad-State Accumulator Loop:     CALIBRATED\", 13, 10\n");
    out.push_str("    db \"[REEL] 4-Dir Spool Hierarchy:           ARMED\", 13, 10\n");
    out.push_str("    db \"[GATE] Anti-Lockout Watchdog Sentinel:  SEALED\", 13, 10\n");
    out.push_str("    db \"========================================\", 13, 10\n");
    out.push_str("    db \"MILLENIUMOS MULTIBOOT ONLINE :: PROCEED\", 13, 10\n");
    out.push_str("    db \"========================================\", 13, 10, 0\n\n");
    out.push_str("vga_msg:\n");
    out.push_str("    db \"MILLENIUMOS: OMNICODE 32-TRIT OS [MULTIBOOT] ONLINE\", 0\n");
    out
}

/// Standalone helper transpiling a raw declarative .omni string directly into x86 bootstrap assembly.
pub fn transpile_omni_to_asm(source: &str) -> Result<String, String> {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), 100);
    let mut assembler = OmniAssembler::new(logger, debugger);
    assembler.transpile_omni_source_to_asm(source)
}

// ---------------------------------------------------------------------------
// 3.3. OmniAssembler Engine Implementation & Constructor
// ---------------------------------------------------------------------------
impl OmniAssembler {
    /// Constructs a new physical silicon assembler bound to the given diagnostic logger and debugger.
    ///
    /// Automatically probes standard candidate lexicon and library paths (`lexicon/`, `lib/`)
    /// to support cross-track ontological symbol resolution.
    pub fn new(logger: DiagnosticLogger, debugger: OmniDebugger) -> Self {
        let mut lexicon_paths = Vec::new();
        let candidate_dirs = ["lexicon", "lib"];
        for dir in &candidate_dirs {
            let p = PathBuf::from(dir);
            if p.exists() {
                lexicon_paths.push(p);
            }
        }
        Self {
            logger,
            debugger,
            lexicon_paths,
        }
    }

    /// Transpiles a raw declarative .omni string directly into an executable .tape string.
    pub fn transpile_omni_source_to_tape(&mut self, source: &str) -> Result<String, String> {
        let assembled = self.assemble_omni(source)?;
        Ok(transpile_omni_to_tape(&assembled))
    }

    /// Transpiles a raw declarative .omni string directly into bare-metal x86 bootstrap assembly (`.asm`).
    pub fn transpile_omni_source_to_asm(&mut self, source: &str) -> Result<String, String> {
        let assembled = self.assemble_omni(source)?;
        Ok(transpile_tape_to_asm(&assembled))
    }

    // ---------------------------------------------------------------------------
    // 3.4. Universal Single-Roll Lowering Pipeline (assemble_roll)
    // ---------------------------------------------------------------------------
    /// Lowers an atomic procedural roll into a physical `AssembledInstruction`,
    /// allocating memory partition offsets and hardware register bindings.
    pub fn assemble_roll(
        &mut self,
        roll: &Roll,
        idx: usize,
        where_clause: &str,
        symbols: &mut HashMap<String, PhysicalSymbol>,
```

```
        next_available_offset: &mut usize,
        commit_status: &mut QuadState,
    ) -> Result<AssembledInstruction, String> {
        let mut lexer = Lexer::new(&roll.text, self.logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(t) => t,
            Err(e) => {
                self.logger.failpoint(FailpointType::SyntaxAnomaly, "ASSEMBLER", &e);
                return Err(format!("Roll {idx} assembly error: {e}"));
            }
        };

        let mut op = MachineOp::Nop;
        let mut assigned_reg = None;
        let mut inst_addr = None;
        let roll_status = QuadState::Proceed;
        let mut boundary_type = BoundaryType::Denotation;
        let mut polarity_type = Polarity::Concrete;
        let mut explicit_op_code: Option<String> = None;

        // 1. Enforce structural boundary invariants with OmniDebugger
        self.debugger.boundary_stack.clear();
        for tok in &tokens {
            match tok.token_type {
                TokenType::LBracket => self.debugger.push_boundary('[', tok.line, tok.col),
                TokenType::LBrace => self.debugger.push_boundary('{', tok.line, tok.col),
                TokenType::LAngle => self.debugger.push_boundary('<', tok.line, tok.col),
                TokenType::RBracket => {
                    if let Err(e) = self.debugger.pop_boundary(']', tok.line, tok.col) {
                        return Err(format!("Roll {idx} delimiter error: {e}"));
                    }
                }
                TokenType::RBrace => {
                    if let Err(e) = self.debugger.pop_boundary('}', tok.line, tok.col) {
                        return Err(format!("Roll {idx} delimiter error: {e}"));
                    }
                }
                TokenType::RAngle => {
                    if let Err(e) = self.debugger.pop_boundary('>', tok.line, tok.col) {
                        return Err(format!("Roll {idx} delimiter error: {e}"));
                    }
                }
                _ => {}
            }
        }
        if let Err(e) = self.debugger.assert_all_boundaries_closed() {
            return Err(format!("Roll {idx} boundary leak error: {e}"));
        }

        // 2. Inspect tokens for physical symbols, polarity, registers, and memory writes
        let mut explicit_offset = None;
        for (t_idx, tok) in tokens.iter().enumerate() {
            // Check for @ or ::@ offset locator
            if tok.token_type == TokenType::At || (tok.token_type == TokenType::And && tok.lexeme == "::@") {
                if let Some(next_tok) = tokens.get(t_idx + 1) {
                    if let TokenType::Number(num) = next_tok.token_type {
                        explicit_offset = Some(num as usize);
                    }
                }
            }

            // Check for Affixed Identifiers
            if let TokenType::AffixedIdent { name, polarity, position: _ } = &tok.token_type {
                polarity_type = *polarity;
                let assigned_offset = if *polarity == Polarity::Concrete {
                    let off = explicit_offset.unwrap_or(*next_available_offset);
                    if explicit_offset.is_none() {
                        *next_available_offset += 1;
                    }
                    inst_addr = Some(off);
                    Some(off)
                } else {
                    None
                };

                if let Some(existing) = symbols.get_mut(name) {
                    if *polarity == Polarity::Concrete {
                        existing.polarity = Polarity::Concrete;
```

```
                        if assigned_offset.is_some() {
                            existing.offset = assigned_offset;
                        }
                    }
                } else {
                    symbols.insert(name.clone(), PhysicalSymbol {
                        name: name.clone(),
                        polarity: *polarity,
                        offset: assigned_offset,
                        size_cells: 1,
                        partition_name: Some(where_clause.to_string()),
                    });
                }

                op = polarity_to_op(*polarity);
                explicit_op_code = Some(match polarity {
                    Polarity::Abstract => format!("REF_ABSTRACT_{name}"),
                    Polarity::Concrete => format!("ALLOC_CONCRETE_{name}"),
                });
            }

            // Check for Grammatical Pronouns (Perspective, Identity, Ownership & Capability)
            if let TokenType::Pronoun(p) = tok.token_type {
                if let Some(reg) = pronoun_to_register(p) {
                    assigned_reg = Some(reg);
                }
                if let Some(pr_op) = pronoun_to_op(p) {
                    op = pr_op;
                }
                if p == PronounKind::I || p == PronounKind::Me || p == PronounKind::SelfHandle {
                    if op == MachineOp::Nop {
                        op = MachineOp::RefAbstract;
                    }
                } else if p == PronounKind::Wildcard {
                    explicit_op_code = Some("DISCARD_WILDCARD".to_string());
                }
            }

            // Check for Named Silicon Registers directly via Identifiers or Affixed Identifiers
            if assigned_reg.is_none() {
                match &tok.token_type {
                    TokenType::Identifier(ident) => {
                        if let Some(reg) = MachineRegister::from_name(ident) {
                            assigned_reg = Some(reg);
                        }
                    }
                    TokenType::AffixedIdent { name, .. } => {
                        if let Some(reg) = MachineRegister::from_name(name) {
                            assigned_reg = Some(reg);
                        }
                    }
                    _ => {}
                }
            }

            // Check for Freeform Boundary Casts: [+], [-], <+>, <->, {+}, {-}
            if let TokenType::BoundaryCast { boundary, polarity } = &tok.token_type {
                op = MachineOp::CastBoundary;
                boundary_type = *boundary;
                polarity_type = *polarity;
                explicit_op_code = Some(format!("CAST_{}_{}", boundary.rank_name().replace(' ', "_"),
polarity.symbol()));
            }

            if let TokenType::Identifier(ref ident) = tok.token_type {
                let is_tripwire_roll = tokens.first().map_or(false, |t| {
                    matches!(&t.token_type, TokenType::Identifier(id) if id.eq_ignore_ascii_case("Tripwire"))
                }) || tokens.get(1).map_or(false, |t| {
                    matches!(&t.token_type, TokenType::Identifier(id) if id.eq_ignore_ascii_case("Tripwire"))
                });

                let is_commit_status = !is_tripwire_roll && (ident == "commit_status"
                    || (ident == "Commit" && tokens.get(t_idx + 1).map_or(false, |next| {
                        if let TokenType::Identifier(ref n) = next.token_type {
                            n == "Status"
                        } else {
                            false
                        }
```

```
                    }));
                if is_commit_status {
                    op = MachineOp::SealCommit;
                    explicit_op_code = Some("SEAL_COMMIT".to_string());

                    let mut resolved_state = QuadState::Proceed;
                    for lookahead in tokens.iter().skip(t_idx + 1) {
                        match &lookahead.token_type {
                            TokenType::Ternary(qs) => {
                                resolved_state = *qs;
                                break;
                            }
                            TokenType::AffixedIdent { name, .. } => {
                                let upper = name.to_uppercase();
                                if upper.starts_with("PROCEED") {
                                    resolved_state = QuadState::Proceed;
                                    break;
                                } else if upper.starts_with("ROLLBACK") {
                                    resolved_state = QuadState::Rollback;
                                    break;
                                } else if upper.starts_with("AWAIT") {
                                    resolved_state = QuadState::Await;
                                    break;
                                } else if upper.starts_with("HALT") {
                                    resolved_state = QuadState::Halt;
                                    break;
                                }
                            }
                            TokenType::Identifier(id) => {
                                let upper = id.to_uppercase();
                                if upper.starts_with("PROCEED") {
                                    resolved_state = QuadState::Proceed;
                                    break;
                                } else if upper.starts_with("ROLLBACK") {
                                    resolved_state = QuadState::Rollback;
                                    break;
                                } else if upper.starts_with("AWAIT") {
                                    resolved_state = QuadState::Await;
                                    break;
                                } else if upper.starts_with("HALT") {
                                    resolved_state = QuadState::Halt;
                                    break;
                                }
                            }
                            _ => {}
                        }
                    }
                    *commit_status = resolved_state;
                }
            }

            // Check for Bijective Radix-3 Transduction Equivalence: <=>
            if tok.token_type == TokenType::InvariantEquivalence {
                op = MachineOp::TransduceVector;
                if explicit_op_code.is_none() {
                    explicit_op_code = Some("TRANSDUCE_EQUIVALENCE".to_string());
                }
            }

            // Check for SVO Action Verbs (Character & Object Lifecycle Transitions)
            if let TokenType::Identifier(ref ident) = tok.token_type {
                let upper = ident.to_uppercase();
                match upper.as_str() {
                    "INSCRIBE" => {
                        op = MachineOp::AllocConcrete;
                        if explicit_op_code.is_none() {
                            explicit_op_code = Some("INSCRIBE_CELL".to_string());
                        }
                    }
                    "TRANSDUCE" => {
                        op = MachineOp::TransduceVector;
                        if explicit_op_code.is_none() {
                            explicit_op_code = Some("TRANSDUCE_SUBSTRATE".to_string());
                        }
                    }
                    "PROJECT" => {
                        op = MachineOp::TransduceVector;
                        if explicit_op_code.is_none() {
```

```
                            explicit_op_code = Some("PROJECT_TARGET".to_string());
                        }
                        if assigned_reg.is_none() {
                            assigned_reg = Some(MachineRegister::INTO);
                        }
                    }
                    "SEQUENCE" => {
                        op = MachineOp::BindChannel;
                        if explicit_op_code.is_none() {
                            explicit_op_code = Some("SEQUENCE_TRANSITION".to_string());
                        }
                    }
                    _ => {}
                }
            }

            // Check for directional lowering: -->, v--
            if tok.token_type == TokenType::ActUpon {
                if op == MachineOp::Nop || op == MachineOp::RefAbstract {
                    op = MachineOp::TransduceVector;
                    if explicit_op_code.is_none() {
                        explicit_op_code = Some("TRANSDUCE_VECTOR".to_string());
                    }
                }
            } else if tok.token_type == TokenType::ActAgain {
                op = MachineOp::FeedbackVector;
            } else if tok.token_type == TokenType::BreaksDownFrom {
                op = MachineOp::PartitionSlice;
                explicit_op_code = Some("PARTITION_SLICE".to_string());
            } else if tok.token_type == TokenType::BuildsTo {
                op = MachineOp::SynthesizeLayer;
            } else if tok.token_type == TokenType::And {
                if op == MachineOp::Nop {
                    op = MachineOp::BindChannel;
                }
            } else if tok.token_type == TokenType::But {
                op = MachineOp::AssertInvariant;
            }
        }

        // Fallback for explicit physical MMIO offset if no AffixedIdent set it
        if explicit_offset.is_some() && inst_addr.is_none() {
            inst_addr = explicit_offset;
        }

        // Automatic register binding fallback for hardware MMIO ports
        if assigned_reg.is_none() {
            if let Some(addr) = inst_addr {
                if addr == 0x03F8 {
                    assigned_reg = Some(MachineRegister::FROM);
                } else if addr == 0x000B8000 || addr == 0x01 {
                    assigned_reg = Some(MachineRegister::INTO);
                }
            }
        }

        let op_code = explicit_op_code.unwrap_or_else(|| op.mnemonic().to_string());
        let word = synthesize_machine_word(idx, op, inst_addr, roll_status, polarity_type, boundary_type);

        Ok(AssembledInstruction {
            roll_index: idx + 1,
            source_line: roll.line_number,
            original_text: roll.text.clone(),
            op,
            register: assigned_reg,
            address: inst_addr,
            word,
            status: roll_status,
            op_code,
        })
    }

    /// Assembles an entire .tape process ribbon into physical silicon layout.
    pub fn assemble_tape(&mut self, tape: &Tape) -> Result<AssembledTape, String> {
        let head = tape.head.clone();
        let partition_size = head.required_partition_size.max(64);
        let mut symbols: HashMap<String, PhysicalSymbol> = HashMap::new();
        let mut instructions: Vec<AssembledInstruction> = Vec::new();
```

```
        let mut next_available_offset: usize = 0;
        let mut commit_status = QuadState::Await;

        for (idx, roll) in tape.rolls.iter().enumerate() {
            let inst = self.assemble_roll(
                roll,
                idx,
                &head.where_clause,
                &mut symbols,
                &mut next_available_offset,
                &mut commit_status,
            )?;
            instructions.push(inst);
        }

        self.logger.log(
            "ASSEMBLER",
            format!("Tape [{}] assembled: {} instructions, {} symbols", head.name, instructions.len(), symbols.len()),
        );

        Ok(AssembledTape {
            head,
            partition_size,
            symbols,
            instructions,
            commit_status,
        })
    }

    /// Assembles a declarative 4-Block .omni spatial source reel into physical machine layout.
    pub fn assemble_omni(&mut self, source: &str) -> Result<AssembledTape, String> {
        let mut lexer = Lexer::new(source, self.logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(t) => t,
            Err(e) => {
                self.logger.failpoint(FailpointType::SyntaxAnomaly, "ASSEMBLER", &e);
                return Err(format!("Omni lexical analysis error: {e}"));
            }
        };

        // Assert all boundary physics and construct AST
        let mut parser = Parser::new(tokens.clone(), self.debugger.clone());
        let _prog = match parser.parse_program() {
            Ok(p) => p,
            Err(e) => {
                self.logger.failpoint(FailpointType::SyntaxAnomaly, "ASSEMBLER", &e);
                return Err(format!("Omni program parse error: {e}"));
            }
        };

        // Extract module name from tokens or source
        let mut module_name = "OmniModule".to_string();
        for (i, tok) in tokens.iter().enumerate() {
            if let TokenType::Identifier(ref id) = tok.token_type {
                if id == "Module" || id == "Manifest" {
                    for lookahead in tokens.iter().skip(i + 1).take(5) {
                        if let TokenType::Identifier(ref name) = lookahead.token_type {
                            module_name = name.clone();
                            break;
                        }
                    }
                }
            }
        }

        let mut symbols: HashMap<String, PhysicalSymbol> = HashMap::new();
        let mut instructions: Vec<AssembledInstruction> = Vec::new();
        let mut next_available_offset: usize = 0;
        let mut commit_status = QuadState::Await;

        // Process non-comment, non-empty lines as instruction rolls
        let mut line_index = 0;
        for (line_no, raw_line) in source.lines().enumerate() {
            let line = raw_line.trim();
            if line.is_empty() || line.starts_with('#') {
                continue;
            }
            if line == "Opening {" || line == "Body [" || line == "Closing {" || line == "}" || line == "]" {
```

```
                continue;
            }

            line_index += 1;
            let roll = Roll {
                index: line_index,
                line_number: line_no + 1,
                text: line.to_string(),
                status: QuadState::Proceed,
                is_head: false,
            };

            // Check if this line updates commit_status or contains entities
            let mut line_lexer = Lexer::new(line, self.logger.clone());
            if let Ok(line_toks) = line_lexer.tokenize() {
                let is_tripwire_line = line.starts_with("[ Tripwire ]") || line.starts_with("[Tripwire]");
                for (t_idx, tok) in line_toks.iter().enumerate() {
                    if !is_tripwire_line {
                        if let TokenType::Identifier(ref ident) = tok.token_type {
                            let is_commit_status = ident == "commit_status"
                                || (ident == "Commit" && line_toks.get(t_idx + 1).map_or(false, |next| {
                                    if let TokenType::Identifier(ref n) = next.token_type {
                                        n == "Status"
                                    } else {
                                        false
                                    }
                                }));
                            if is_commit_status {
                                let mut resolved = QuadState::Proceed;
                                for lookahead in line_toks.iter().skip(t_idx + 1) {
                                    match &lookahead.token_type {
                                        TokenType::Ternary(qs) => {
                                            resolved = *qs;
                                            break;
                                        }
                                        TokenType::AffixedIdent { name, .. } | TokenType::Identifier(name) => {
                                            let upper = name.to_uppercase();
                                            if upper.starts_with("PROCEED") {
                                                resolved = QuadState::Proceed;
                                                break;
                                            } else if upper.starts_with("ROLLBACK") {
                                                resolved = QuadState::Rollback;
                                                break;
                                            } else if upper.starts_with("AWAIT") {
                                                resolved = QuadState::Await;
                                                break;
                                            } else if upper.starts_with("HALT") {
                                                resolved = QuadState::Halt;
                                                break;
                                            }
                                        }
                                        _ => {}
                                    }
                                }
                                commit_status = resolved;
                            }
                        }
                    }

                    if let TokenType::AffixedIdent { name, polarity, .. } = &tok.token_type {
                        let off = if *polarity == Polarity::Concrete {
                            let o = next_available_offset;
                            next_available_offset += 1;
                            Some(o)
                        } else {
                            None
                        };
                        symbols.entry(name.clone()).or_insert_with(|| PhysicalSymbol {
                            name: name.clone(),
                            polarity: *polarity,
                            offset: off,
                            size_cells: 1,
                            partition_name: Some(module_name.clone()),
                        });
                    }
                }

                // If line can be lowered as a standalone roll, assemble it
```

```
                if let Ok(inst) = self.assemble_roll(
                    &roll,
                    instructions.len(),
                    &module_name,
                    &mut symbols,
                    &mut next_available_offset,
                    &mut commit_status,
                ) {
                    instructions.push(inst);
                }
            }
        }

        let head = TapeHead {
            name: module_name.clone(),
            who: "OmniAssembler".to_string(),
            what: "DeclarativeReel".to_string(),
            when_clause: "Compilation".to_string(),
            where_clause: module_name.clone(),
            why: "SpatialDeclarativeAssembly".to_string(),
            how: commit_status.macro_op().to_string(),
            required_partition_size: 64.max(next_available_offset),
            raw_text: format!("[Reel.{}]", module_name),
        };

        let partition_size = head.required_partition_size;

        self.logger.log(
            "ASSEMBLER",
            format!("Reel [{}] assembled: {} instructions, {} symbols", head.name, instructions.len(), symbols.len()),
        );

        Ok(AssembledTape {
            head,
            partition_size,
            symbols,
            instructions,
            commit_status,
        })
    }

    /// Assembles a .tape or .omni file directly from disk.
    pub fn assemble_file(&mut self, path: impl AsRef<Path>) -> Result<AssembledTape, String> {
        let p = path.as_ref();
        let ext = p.extension().and_then(|e| e.to_str()).unwrap_or("");
        match ext {
            "tape" => {
                let tape = Tape::from_file(p, self.logger.clone())?;
                self.assemble_tape(&tape)
            }
            "omni" => {
                let source = std::fs::read_to_string(p)
                    .map_err(|e| format!("Failed to read .omni file '{}': {e}", p.display()))?;
                self.assemble_omni(&source)
            }
            other => Err(format!(
                "Unsupported file extension '.{}' for '{}' (expected .tape or .omni)",
                other, p.display()
            )),
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Baton Pass Contract, Invariant Verification Proofs & Unit Oracles )
// ============================================================================

// ----------------------------------------------------------------------------
// 4.1. The Physical Baton Pass Contract
// ----------------------------------------------------------------------------
// BATON PASS CONTRACT:
// 1. INPUT INGESTION: `OmniAssembler::assemble_file` or `assemble_tape` ingests high-level
//    procedural tapes (`.tape`) or spatial reels (`.omni`) from callers (CLI, REPL, JIT).
// 2. ATOMIC LOWERING: `assemble_roll` tokenizes each statement, asserts boundary integrity via
//    `OmniDebugger`, resolves polarities (`+` abstract vs `-` concrete), assigns physical memory
//    partition offsets (`@ 0x0000`), binds agentic pronouns to hardware CPU registers (`%self`,
```

```
//    `%caller`, `%acc`, `%swarm`), and packs 64-bit balanced-ternary machine words (`Word64`).
// 3. INVARIANT & SEAL CERTIFICATION: The tape head covenant and commit status are evaluated,
//    guaranteeing that execution only seals in affirmative `+1:PROCEED`.
// 4. OUTPUT ARTIFACT SYNTHESIS: `AssembledTape` provides universal emission methods (`format_listing`,
//    `format_words`, `format_json`, `format_omni`, `to_machine_words`).
// 5. BATON HANDOFF: The resulting `AssembledTape` or `Vec<Word64>` is handed off to downstream
//    virtual machine runners, the QEMU bare-metal hypervisor, or CLI output serializers.

#[cfg(test)]
mod tests {
    use super::*;

    /// Test 1: Verifies that the physical assembler lowers living world ontological tapes,
    /// synthesizing concrete memory cell offsets and affirmative PROCEED commit status.
    #[test]
    fn test_assembler_lowers_living_world_tape() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/living_world.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling living_world.tape");
            assert_eq!(assembled.head.name, "LivingWorldInstantiation");
            assert!(assembled.symbols.contains_key("Dog"));
            assert_eq!(assembled.commit_status, QuadState::Proceed);
            let listing = assembled.format_listing();
            assert!(listing.contains("ASSEMBLED MACHINE TAPE"));
            assert!(listing.contains("Dog"));
        }
    }

    /// Test 2: Verifies polarity transformation Lowering (`+Archetype --> Concrete-`).
    /// Ensures that abstract entities register in the symbol table with Abstract polarity,
    /// and concrete entities receive designated silicon memory cells.
    #[test]
    fn test_assembler_lowers_polarity_tape() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lexicon/operators/polarity.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling polarity.tape");
            assert_eq!(assembled.head.name, "PolarityTransformation");
            assert!(assembled.symbols.contains_key("Archetype"));
            assert!(assembled.symbols.contains_key("Concrete"));
            assert_eq!(assembled.commit_status, QuadState::Proceed);
        }
    }

    /// Test 3: Exhaustive regression oracle across the entire modular Track 2 Lexicon.
    /// Assembles all 28 foundational grammar and kernel tapes, asserting that every single
    /// tape seals with affirmative `+1:PROCEED` without unhandled delimiter or syntax errors.
    #[test]
    fn test_assembler_lowers_all_modular_lexicon_tapes() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let tape_paths = [
            "lexicon/operators/polarity.tape",
            "lexicon/operators/casting.tape",
            "lexicon/01_nouns/atomic_units.tape",
            "lexicon/01_nouns/vessels.tape",
            "lexicon/01_nouns/agents.tape",
            "lexicon/01_nouns/living_world.tape",
            "lexicon/02_pronouns/self_handles.tape",
            "lexicon/02_pronouns/wildcards.tape",
            "lexicon/03_verbs/axial_vectors.tape",
            "lexicon/03_verbs/scale_vectors.tape",
            "lexicon/03_verbs/diagonal_vectors.tape",
            "lexicon/03_verbs/relational_being.tape",
            "lexicon/03_verbs/arithmetic_verbs.tape",
            "lexicon/04_adjectives/quad_states.tape",
            "lexicon/04_adjectives/invariant_axioms.tape",
            "lexicon/04_adjectives/proportions.tape",
            "lexicon/05_adverbs/negation.tape",
```

```
            "lexicon/05_adverbs/cadence_policies.tape",
            "lexicon/06_prepositions/spatial_sites.tape",
            "lexicon/06_prepositions/ranges.tape",
            "lexicon/07_conjunctions/inseparable_pairs.tape",
            "lexicon/07_conjunctions/pipeline_junctions.tape",
            "lexicon/07_conjunctions/caveat_branches.tape",
            "lexicon/08_interjections/kernel_traps.tape",
            "lexicon/08_interjections/teleology_asides.tape",
            "lexicon/03_verbs/cinematic_track_loop.tape",
            "lexicon/operators/boot_kernel.tape",
            "lib/kernel/boot.tape",
        ];

        for path_str in &tape_paths {
            let path = crate::core::filetype::resolve_track_path(path_str);
            assert!(path.exists(), "Tape file does not exist: {}", path_str);
            let assembled = asm.assemble_file(&path).unwrap_or_else(|e| panic!("Failed to assemble {}: {}", path_str,
e));
            assert_eq!(assembled.commit_status, QuadState::Proceed, "Tape {} did not commit in PROCEED", path_str);
            assert!(!assembled.instructions.is_empty(), "Tape {} had no instructions", path_str);
        }
    }

    /// Test 4: Verifies grammatical pronoun lowering into physical hardware CPU registers.
    /// Asserts that self-handles (`self`) map to `%self` (Register 4), wildcards and
    /// antecedents (`it`) map to `%acc` (Register 3), and callers map to `%caller`.
    #[test]
    fn test_assembler_lowers_pronouns_and_registers() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lexicon/02_pronouns/self_handles.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling self_handles.tape");
            // Check that SELF register was assigned for self handle
            let self_roll = assembled.instructions.iter().find(|i| i.original_text.contains("self"));
            assert!(self_roll.is_some(), "Expected self roll in instructions");
            assert_eq!(self_roll.unwrap().register, Some(MachineRegister::SELF));
        }

        let wild_path = crate::core::filetype::resolve_track_path("lexicon/02_pronouns/wildcards.tape");
        if wild_path.exists() {
            let assembled = asm.assemble_file(&wild_path).expect("Assembling wildcards.tape");
            // Check that ACC register was assigned for it antecedent
            let it_roll = assembled.instructions.iter().find(|i| i.original_text.contains("it"));
            assert!(it_roll.is_some(), "Expected it roll in instructions");
            assert_eq!(it_roll.unwrap().register, Some(MachineRegister::ACC));
        }
    }

    /// Test 5: Verifies universal serialization methods on `AssembledTape`.
    /// Validates `format_words()` (physical silicon hex dump), `format_json()` (IDE/tooling AST),
    /// and `format_omni()` (decompilation/transpilation into declarative 4-Block spatial reels).
    #[test]
    fn test_assembler_universal_formatting_methods() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lib/kernel/boot.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling boot.tape");

            // 1. Format Words
            let words_dump = assembled.format_words();
            assert!(words_dump.contains("PHYSICAL SILICON WORD STREAM"));
            assert!(words_dump.contains("TOTAL WORDS:"));

            // 2. Format JSON
            let json_dump = assembled.format_json();
            assert!(json_dump.contains("\"tape\":"));
            assert!(json_dump.contains("\"symbols\":"));
            assert!(json_dump.contains("\"instructions\":"));

            // 3. Format Omni (Transpilation)
            let omni_dump = assembled.format_omni();
            assert!(omni_dump.contains("BLOCK 1: METADATA BLOCK"));
```

```
            assert!(omni_dump.contains("BLOCK 2: OPENING BLOCK"));
            assert!(omni_dump.contains("BLOCK 3: BODY BLOCK"));
            assert!(omni_dump.contains("BLOCK 4: CLOSING BLOCK"));
            assert!(omni_dump.contains("commit_status [ PROCEED ]"));
        }
    }

    /// Test 6: Verifies declarative `.omni` spatial reel lowering into physical machine layout.
    /// Confirms that `assemble_omni` correctly parses multi-block `.omni` code, constructs
    /// the symbol table and instruction stream, and verifies the Closing block seal.
    #[test]
    fn test_assembler_lowers_declarative_omni_reel() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lexicon/03_verbs/axial_vectors.omni");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling axial_vectors.omni");
            assert_eq!(assembled.commit_status, QuadState::Proceed);
            assert!(!assembled.instructions.is_empty(), "Must produce instruction stream");
            assert_eq!(assembled.head.who, "OmniAssembler");
        }
    }

    /// Test 7: Verifies standalone atomic single-roll lowering via `assemble_roll`.
    /// Confirms that an individual roll can be lowered outside of a Tape container,
    /// correctly allocating memory partition offsets and updating the physical symbol table.
    #[test]
    fn test_assembler_assemble_roll_standalone() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let roll = Roll {
            index: 1,
            line_number: 10,
            text: "roll_1 [ +Memory --> Memory- @ 0 ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };

        let mut symbols = HashMap::new();
        let mut next_offset = 0;
        let mut commit_status = QuadState::Await;

        let inst = asm.assemble_roll(
            &roll,
            0,
            "TestPartition",
            &mut symbols,
            &mut next_offset,
            &mut commit_status,
        ).expect("Assembling standalone roll");

        assert_eq!(inst.op, MachineOp::AllocConcrete);
        assert!(symbols.contains_key("Memory"));
        assert_eq!(symbols["Memory"].offset, Some(0));
        assert_ne!(inst.word.raw, 0xFFFFFFFF_FFFFFFFF);
    }

    /// Test 8: Verifies full coverage of all 13 Named Silicon Registers,
    /// backward-compatible aliases, ALL CAPS and lowercase duality, affix handling,
    /// and dynamic roll register binding.
    #[test]
    fn test_named_silicon_registers_coverage() {
        let registers = [
            (MachineRegister::HEAD, 0, "HEAD", "%head"),
            (MachineRegister::HOME, 1, "HOME", "%home"),
            (MachineRegister::SPOOL, 2, "SPOOL", "%spool"),
            (MachineRegister::IT, 3, "IT", "%it"),
            (MachineRegister::SELF, 4, "SELF", "%self"),
            (MachineRegister::CALLER, 5, "CALLER", "%caller"),
            (MachineRegister::OTHER, 6, "OTHER", "%other"),
            (MachineRegister::STATE, 7, "STATE", "%state"),
            (MachineRegister::TURN, 8, "TURN", "%turn"),
            (MachineRegister::BOUND, 9, "BOUND", "%bound"),
            (MachineRegister::FROM, 10, "FROM", "%from"),
```

```
            (MachineRegister::INTO, 11, "INTO", "%into"),
            (MachineRegister::DISCARD, 12, "DISCARD", "%discard"),
        ];

        for (reg, expected_idx, expected_hw, expected_name) in registers {
            assert_eq!(reg.index(), expected_idx);
            assert_eq!(reg.hardware_name(), expected_hw);
            assert_eq!(reg.name(), expected_name);
            // Verify from_name with ALL CAPS, lowercase, %-prefix, and +/- affixes
            assert_eq!(MachineRegister::from_name(expected_hw), Some(reg));
            assert_eq!(MachineRegister::from_name(&expected_hw.to_lowercase()), Some(reg));
            assert_eq!(MachineRegister::from_name(&format!("%{}", expected_hw.to_lowercase())), Some(reg));
            assert_eq!(MachineRegister::from_name(&format!("+{}", expected_hw)), Some(reg));
            assert_eq!(MachineRegister::from_name(&format!("-{}", expected_hw)), Some(reg));
        }

        // Backward compatibility constants
        assert_eq!(MachineRegister::IP, MachineRegister::HEAD);
        assert_eq!(MachineRegister::FP, MachineRegister::HOME);
        assert_eq!(MachineRegister::SP, MachineRegister::SPOOL);
        assert_eq!(MachineRegister::ACC, MachineRegister::IT);
        assert_eq!(MachineRegister::FLAGS, MachineRegister::STATE);
        assert_eq!(MachineRegister::HERE, MachineRegister::HEAD);
        assert_eq!(MachineRegister::ORIGIN, MachineRegister::HOME);
        assert_eq!(MachineRegister::OWNER, MachineRegister::SPOOL);
        assert_eq!(MachineRegister::SWARM, MachineRegister::OTHER);
        assert_eq!(MachineRegister::STEP, MachineRegister::TURN);
        assert_eq!(MachineRegister::INPUT, MachineRegister::FROM);
        assert_eq!(MachineRegister::OUTPUT, MachineRegister::INTO);

        // Natural, positional, and shorthand aliases
        assert_eq!(MachineRegister::from_name("ip"), Some(MachineRegister::HEAD));
        assert_eq!(MachineRegister::from_name("here"), Some(MachineRegister::HEAD));
        assert_eq!(MachineRegister::from_name("origin"), Some(MachineRegister::HOME));
        assert_eq!(MachineRegister::from_name("spool"), Some(MachineRegister::SPOOL));
        assert_eq!(MachineRegister::from_name("owner"), Some(MachineRegister::SPOOL));
        assert_eq!(MachineRegister::from_name("turn"), Some(MachineRegister::TURN));
        assert_eq!(MachineRegister::from_name("step"), Some(MachineRegister::TURN));
        assert_eq!(MachineRegister::from_name("bound"), Some(MachineRegister::BOUND));
        assert_eq!(MachineRegister::from_name("from"), Some(MachineRegister::FROM));
        assert_eq!(MachineRegister::from_name("input"), Some(MachineRegister::FROM));
        assert_eq!(MachineRegister::from_name("into"), Some(MachineRegister::INTO));
        assert_eq!(MachineRegister::from_name("output"), Some(MachineRegister::INTO));
        assert_eq!(MachineRegister::from_name("_"), Some(MachineRegister::DISCARD));
        assert_eq!(MachineRegister::from_name("void"), Some(MachineRegister::DISCARD));

        // Verify that assemble_roll binds Named Silicon Registers directly
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let roll_turn = Roll {
            index: 1,
            line_number: 1,
            text: "roll_turn [ TURN plus 1 --> TURN ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let mut syms = HashMap::new();
        let mut off = 0;
        let mut commit = QuadState::Await;
        let inst_turn = asm.assemble_roll(&roll_turn, 0, "Part", &mut syms, &mut off, &mut commit).expect("Assemble
TURN");
        assert_eq!(inst_turn.register, Some(MachineRegister::TURN));

        let roll_bound = Roll {
            index: 2,
            line_number: 2,
            text: "roll_bound [ BOUND ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst_bound = asm.assemble_roll(&roll_bound, 1, "Part", &mut syms, &mut off, &mut commit).expect("Assemble
BOUND");
        assert_eq!(inst_bound.register, Some(MachineRegister::BOUND));
    }

    /// Test 9: Verifies SVO action verbs (INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE),
```

```
    /// bijective Radix-3 equivalence (<=>), and MMIO port lowering for Character tapes.
    #[test]
    fn test_character_tape_svo_lowering_and_equivalence() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);
        let mut syms = HashMap::new();
        let mut off = 0;
        let mut commit = QuadState::Await;

        // 1. ROLL 1: INSCRIBE verb -> AllocConcrete / INSCRIBE_CELL
        let roll1 = Roll {
            index: 1,
            line_number: 1,
            text: "[ ROLL 1 ] : [ Inscribe ] : { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst1 = asm.assemble_roll(&roll1, 0, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble ROLL
1");
        assert_eq!(inst1.op_code, "INSCRIBE_CELL");
        assert_eq!(inst1.op, MachineOp::AllocConcrete);

        // 2. ROLL 2: Bijective equivalence <=> and TRANSDUCE verb -> TransduceVector / TRANSDUCE_SUBSTRATE
        let roll2 = Roll {
            index: 2,
            line_number: 2,
            text: "[ ROLL 2 ] : [ Transduce ] : [ Transducer ]: [ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits
} ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst2 = asm.assemble_roll(&roll2, 1, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble ROLL
2");
        assert_eq!(inst2.op_code, "TRANSDUCE_SUBSTRATE");
        assert_eq!(inst2.op, MachineOp::TransduceVector);

        // 3. ROLL 4: PROJECT verb and Target -> TransduceVector / PROJECT_TARGET with register INTO
        let roll4 = Roll {
            index: 4,
            line_number: 4,
            text: "[ ROLL 4 ] : [ Project ] : [ Character` } --> PROJECT [ Target ] = [ Target` }".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst4 = asm.assemble_roll(&roll4, 3, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble ROLL
4");
        assert_eq!(inst4.op_code, "PROJECT_TARGET");
        assert_eq!(inst4.op, MachineOp::TransduceVector);
        assert_eq!(inst4.register, Some(MachineRegister::INTO));

        // 4. ROLL 5: SEQUENCE verb -> BindChannel / SEQUENCE_TRANSITION
        let roll5 = Roll {
            index: 5,
            line_number: 5,
            text: "[ ROLL 5 ] : [ Sequence ] : [ Character` } --> SEQUENCE [ Text ] = [ Text` ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst5 = asm.assemble_roll(&roll5, 4, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble ROLL
5");
        assert_eq!(inst5.op_code, "SEQUENCE_TRANSITION");
        assert_eq!(inst5.op, MachineOp::BindChannel);

        // 5. MMIO addresses: FROM ::@ 0x03F8 and INTO ::@ 0x000B8000
        let roll_uart = Roll {
            index: 6,
            line_number: 6,
            text: "[ COM1 ] : FROM ::@ 0x03F8".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst_uart = asm.assemble_roll(&roll_uart, 5, "CharacterTape", &mut syms, &mut off, &mut
commit).expect("Assemble UART");
        assert_eq!(inst_uart.address, Some(0x03F8));
        assert_eq!(inst_uart.register, Some(MachineRegister::FROM));

        let roll_vga = Roll {
```

```
            index: 7,
            line_number: 7,
            text: "[ VGA ] : INTO ::@ 0x000B8000".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst_vga = asm.assemble_roll(&roll_vga, 6, "CharacterTape", &mut syms, &mut off, &mut
commit).expect("Assemble VGA");
        assert_eq!(inst_vga.address, Some(0x000B8000));
        assert_eq!(inst_vga.register, Some(MachineRegister::INTO));
    }
}


```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment.rs RS 524 LINES 2052 WORDS 21.0 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Runtime Environment State, Transactional Scopes & Symbol Table Invariants )
// ================================================================================
//! OmniCode Runtime: Transactional Environment & Silicon Register Substrate
//!
//! # Architectural Scope & Purpose:
//! The Environment provides scoped ontological symbol tables with native snapshot,
//! transaction nesting, and zero-cost rollback capabilities to support balanced
//! ternary -1 (Rollback) state transitions.
//!
//! In addition to lexical variable bindings, the Environment hosts the physical
//! Silicon Register Bank (the 13 Named Machine Registers: %home, %head, %tail,
//! %spool, %reel, %gate, %into, %from, %caller, %self, %it, %that, %discard),
//! procedural call frames, hardware transducer ports, and the dual-substrate
//! representation of the Character primitive type (`omni_character_t` parity).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Epistemic Continuity**: Autonomous coding agents require explicit rationale
//!    for why physical registers are synchronized with symbol bindings and why
//!    state-primes (`Character'`) automatically resolve to base entities (`Character`).
//! 2. **Transactional Soundness**: Clearly defined contracts for `begin_transaction`,
//!    `commit_transaction`, and `rollback_transaction` prevent state leakage and
//!    unhandled underflows during reversible execution.
//! 3. **Cross-Language Parity**: Maintains mathematical and structural equivalence
//!    with the C backbone (`omni_character_t`) and the Lua engine (`CharacterTape`).
// ================================================================================

use std::collections::HashMap;
use crate::core::ternary::QuadState;
use crate::core::cell::Cell;
use crate::core::vessel::Vessel;
use crate::runtime::assembler::MachineRegister;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( RuntimeValue Enum, RegisterBank, CallFrame and Environment Initialization )
// ================================================================================

/// Represents any discrete value flowing through the OmniCode runtime.
///
/// Implements full dual-substrate parity across balanced ternary truth states,
/// continuous text strings, dense 5-trit cells, and the atomic Character rune.
#[derive(Debug, Clone, PartialEq)]
pub enum RuntimeValue {
    /// Quad-State truth value (+1: Proceed, 0: Await, -1: Rollback, NULL: Halt).
    State(QuadState),
    /// 64-bit floating-point numeric scalar or integer codepoint.
    Number(f64),
    /// Continuous text string ribbon composed of sequential characters.
    Text(String),
    /// Atomic 5-trit balanced ternary memory cell (spanning 243 states).
    Cell(Cell),
    /// Atomic Character rune enclosing both 8-bit ASCII glyph and 5-trit Cell.
    /// Fulfills invariant: `{ Character } = [ Enclosed ]: TRUE`.
    Character {
        glyph: u8,
        cell: Cell,
    },
    /// Bounded multi-cell memory vessel or storehouse partition.
    Vessel(Vessel),
```

```
    /// Structured entity record containing named fields and mutability contract.
    EntityRecord {
        name: String,
        is_immutable: bool,
        values: HashMap<String, Box<RuntimeValue>>,
    },
    /// Ordered vector of runtime values.
    Vector(Vec<RuntimeValue>),
}

/// Canonical Hardware Target Addresses
pub const OMNI_TARGET_STDOUT: u64 = 0x01;
pub const OMNI_TARGET_VGA: u64 = 0x000B8000;
pub const OMNI_TARGET_COM1: u64 = 0x03F8;

/// Physical Silicon Register Bank hosting the 13 Named Machine Registers.
///
/// Implements immediate hardware cell reading, mutation, and discard-sink semantics.
#[derive(Debug, Clone, PartialEq)]
pub struct RegisterBank {
    pub cells: [RuntimeValue; 13],
}

impl RegisterBank {
    pub fn new() -> Self {
        Self {
            cells: [
                RuntimeValue::Number(0.0),                  // 0: HEAD (Tape Head / IP)
                RuntimeValue::Number(0.0),                  // 1: HOME (Base Pointer / Origin)
                RuntimeValue::Number(0.0),                  // 2: SPOOL (Stack Spool / Floor)
                RuntimeValue::State(QuadState::Proceed),    // 3: IT (Accumulator / Result)
                RuntimeValue::Text("Kernel".to_string()),   // 4: SELF (Instance / Context)
                RuntimeValue::Text("Root".to_string()),     // 5: CALLER (Invoker Context)
                RuntimeValue::Text("Swarm".to_string()),    // 6: OTHER (Peer / Auxiliary)
                RuntimeValue::State(QuadState::Proceed),    // 7: STATE (QuadState Status)
                RuntimeValue::Number(0.0),                  // 8: TURN (Temporal Cadence / Cycle)
                RuntimeValue::Number(0.0),                  // 9: BOUND (Watchdog Limit / Horizon)
                RuntimeValue::Text(String::new()),          // 10: FROM (Input Channel / Stdin)
                RuntimeValue::Text(String::new()),          // 11: INTO (Output Channel / Stdout)
                RuntimeValue::State(QuadState::Proceed),    // 12: DISCARD (Bit-Bucket / Null Sink)
            ],
        }
    }

    /// Reads a register cell by its physical `MachineRegister` enum.
    pub fn read(&self, reg: MachineRegister) -> RuntimeValue {
        self.cells[reg.index() as usize].clone()
    }

    /// Writes a value to a register cell. Writes to `DISCARD` are safely dropped.
    pub fn write(&mut self, reg: MachineRegister, val: RuntimeValue) {
        if reg == MachineRegister::DISCARD {
            return;
        }
        self.cells[reg.index() as usize] = val;
    }

    /// Reads a register by name if recognized, returning `None` otherwise.
    pub fn read_by_name(&self, name: &str) -> Option<RuntimeValue> {
        MachineRegister::from_name(name).map(|r| self.read(r))
    }

    /// Writes to a register by name. Returns true if name was a recognized register.
    pub fn write_by_name(&mut self, name: &str, val: RuntimeValue) -> bool {
        if let Some(reg) = MachineRegister::from_name(name) {
            self.write(reg, val);
            true
        } else {
            false
        }
    }

    /// Resets all registers to pristine hardware baseline.
    pub fn reset(&mut self) {
        *self = Self::new();
    }
}

```

```rust
impl Default for RegisterBank {
    fn default() -> Self {
        Self::new()
    }
}

/// Call stack frame preserved on `SPOOL` for procedural execution.
#[derive(Debug, Clone, PartialEq)]
pub struct CallFrame {
    pub caller: RuntimeValue,
    pub self_val: RuntimeValue,
    pub home: usize,
    pub head: usize,
    pub locals: HashMap<String, RuntimeValue>,
}

/// Transactional snapshot capturing both symbolic bindings and physical registers.
#[derive(Debug, Clone)]
pub struct EnvironmentSnapshot {
    bindings: HashMap<String, RuntimeValue>,
    registers: RegisterBank,
    call_stack_len: usize,
    output_len: usize,
}

#[derive(Debug, Clone)]
pub struct Environment {
    pub registers: RegisterBank,
    bindings: HashMap<String, RuntimeValue>,
    call_stack: Vec<CallFrame>,
    pub output_stream: Vec<RuntimeValue>,
    pub input_stream: Vec<RuntimeValue>,
    pub target_device: u64,
    pub sequence_ribbon: Vec<RuntimeValue>,
    snapshots: Vec<EnvironmentSnapshot>,
}

impl Environment {
    pub fn new() -> Self {
        Self {
            registers: RegisterBank::new(),
            bindings: HashMap::new(),
            call_stack: Vec::new(),
            output_stream: Vec::new(),
            input_stream: Vec::new(),
            target_device: OMNI_TARGET_STDOUT,
            sequence_ribbon: Vec::new(),
            snapshots: Vec::new(),
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Transactional Operations: Begin, Commit, Rollback, Binding Mutations )
// ============================================================================
impl Environment {
    /// Takes a transactional snapshot of current bindings and physical registers.
    pub fn begin_transaction(&mut self) {
        self.snapshots.push(EnvironmentSnapshot {
            bindings: self.bindings.clone(),
            registers: self.registers.clone(),
            call_stack_len: self.call_stack.len(),
            output_len: self.output_stream.len(),
        });
    }

    /// Commits the active transaction, discarding the restore snapshot.
    pub fn commit_transaction(&mut self) {
        self.snapshots.pop();
    }

    /// Unwinds and restores the environment bindings and registers to the last snapshot.
    pub fn rollback_transaction(&mut self) -> Result<(), String> {
        if let Some(prev) = self.snapshots.pop() {
            self.bindings = prev.bindings;
            self.registers = prev.registers;
            self.call_stack.truncate(prev.call_stack_len);
```

```
            self.output_stream.truncate(prev.output_len);
            Ok(())
        } else {
            Err("FAILPOINT [RollbackUnderflow]: Attempted rollback with no active transaction snapshot".into())
        }
    }

    pub fn bindings(&self) -> &HashMap<String, RuntimeValue> {
        &self.bindings
    }

    pub fn clear(&mut self) {
        self.bindings.clear();
        self.registers.reset();
        self.call_stack.clear();
        self.output_stream.clear();
        self.input_stream.clear();
        self.target_device = OMNI_TARGET_STDOUT;
        self.sequence_ribbon.clear();
        self.snapshots.clear();
    }

    /// Binds a value by name. If the name corresponds to a physical register,
    /// updates the silicon register cell in addition to the symbol table.
    /// If the name contains state-prime affixes (e.g., Character` or Character'),
    /// it binds the primed name and automatically synchronizes the base entity.
    pub fn set(&mut self, name: &str, value: RuntimeValue) {
        if let Some(reg) = MachineRegister::from_name(name) {
            self.registers.write(reg, value.clone());
            if reg != MachineRegister::DISCARD {
                self.bindings.insert(name.to_string(), value);
            }
        } else {
            self.bindings.insert(name.to_string(), value.clone());
            let trimmed = name.trim_end_matches(['`', '\'']);
            if trimmed != name && !trimmed.is_empty() {
                self.bindings.insert(trimmed.to_string(), value);
            }
        }
    }

    /// Retrieves a value by name. If the name matches a silicon register,
    /// yields a reference to the physical register cell.
    /// Supports state-prime fallback: looking up Character` falls back to Character
    /// if the specific prime has not yet been bound.
    pub fn get(&self, name: &str) -> Option<&RuntimeValue> {
        if let Some(reg) = MachineRegister::from_name(name) {
            Some(&self.registers.cells[reg.index() as usize])
        } else if let Some(val) = self.bindings.get(name) {
            Some(val)
        } else {
            let trimmed = name.trim_end_matches(['`', '\'']);
            if trimmed != name && !trimmed.is_empty() {
                self.bindings.get(trimmed)
            } else {
                None
            }
        }
    }

    pub fn contains(&self, name: &str) -> bool {
        self.get(name).is_some()
    }

    /// Pushes a procedural call frame onto the `SPOOL` stack.
    pub fn push_call_frame(&mut self, caller: RuntimeValue, self_val: RuntimeValue, return_head: usize) {
        let home = self.call_stack.len();
        self.registers.write(MachineRegister::HOME, RuntimeValue::Number(home as f64));
        self.registers.write(MachineRegister::HEAD, RuntimeValue::Number(return_head as f64));
        self.registers.write(MachineRegister::CALLER, caller.clone());
        self.registers.write(MachineRegister::SELF, self_val.clone());
        self.call_stack.push(CallFrame {
            caller,
            self_val,
            home,
            head: return_head,
            locals: HashMap::new(),
        });
```

```
        self.registers.write(MachineRegister::SPOOL, RuntimeValue::Number(self.call_stack.len() as f64));
    }

    /// Pops the active call frame from the `SPOOL` stack, restoring caller context.
    pub fn pop_call_frame(&mut self) -> Option<CallFrame> {
        let popped = self.call_stack.pop()?;
        let current_spool = self.call_stack.len();
        self.registers.write(MachineRegister::SPOOL, RuntimeValue::Number(current_spool as f64));
        if let Some(prev) = self.call_stack.last() {
            self.registers.write(MachineRegister::HOME, RuntimeValue::Number(prev.home as f64));
            self.registers.write(MachineRegister::HEAD, RuntimeValue::Number(prev.head as f64));
            self.registers.write(MachineRegister::CALLER, prev.caller.clone());
            self.registers.write(MachineRegister::SELF, prev.self_val.clone());
        }
        Some(popped)
    }

    /// Emits a runtime value to the `INTO` outbound port and buffers it in output_stream.
    pub fn write_into(&mut self, val: RuntimeValue) {
        self.registers.write(MachineRegister::INTO, val.clone());
        self.output_stream.push(val);
    }

    /// Reads the next input value from `input_stream` into `FROM`, or reads current `FROM`.
    pub fn read_from(&mut self) -> RuntimeValue {
        if !self.input_stream.is_empty() {
            let next_val = self.input_stream.remove(0);
            self.registers.write(MachineRegister::FROM, next_val.clone());
            next_val
        } else {
            self.registers.read(MachineRegister::FROM)
        }
    }

    /// Reads the current value held in the `INTO` output register.
    pub fn read_into(&self) -> RuntimeValue {
        self.registers.read(MachineRegister::INTO)
    }

    /// Enqueues an input value into `input_stream` and sets the `FROM` register.
    pub fn write_from(&mut self, val: RuntimeValue) {
        self.registers.write(MachineRegister::FROM, val.clone());
        self.input_stream.push(val);
    }
}

impl RuntimeValue {
    /// Constructs a `RuntimeValue::Character` from an 8-bit ASCII byte/glyph,
    /// bijectively transducing the byte into a 5-trit balanced ternary `Cell`.
    pub fn from_glyph(glyph: u8) -> Self {
        let cell = Cell::from_natural(glyph).unwrap_or_else(|_| Cell::from_state(QuadState::Rollback));
        RuntimeValue::Character { glyph, cell }
    }

    /// Constructs a `RuntimeValue::Character` from a standard Rust character.
    pub fn from_char(c: char) -> Self {
        let glyph = (c as u32 & 0xFF) as u8;
        Self::from_glyph(glyph)
    }

    /// Returns the 8-bit glyph byte if this value is a Character or convertible numeric/text value.
    pub fn character_glyph(&self) -> Option<u8> {
        match self {
            RuntimeValue::Character { glyph, .. } => Some(*glyph),
            RuntimeValue::Number(n) => {
                if *n >= 0.0 && *n <= 255.0 {
                    Some(*n as u8)
                } else {
                    None
                }
            }
            RuntimeValue::Text(t) => {
                let bytes = t.as_bytes();
                if bytes.len() == 1 {
                    Some(bytes[0])
                } else {
                    None
                }
```

```
            }
            _ => None,
        }
    }

    /// Returns the 5-trit `Cell` substrate if this value is a Character or Cell.
    pub fn character_cell(&self) -> Option<Cell> {
        match self {
            RuntimeValue::Character { cell, .. } => Some(*cell),
            RuntimeValue::Cell(c) => Some(*c),
            RuntimeValue::Number(n) => {
                if *n >= 0.0 && *n <= 242.0 {
                    Cell::from_natural(*n as u8).ok()
                } else {
                    None
                }
            }
            _ => None,
        }
    }

    /// Formats the runtime value with ANSI color styling for terminal output.
    pub fn format_colored(&self) -> String {
        match self {
            RuntimeValue::State(QuadState::Proceed) => "\x1b[1;32m+1:PROCEED\x1b[0m".to_string(),
            RuntimeValue::State(QuadState::Await) => "\x1b[1;33m0:AWAIT\x1b[0m".to_string(),
            RuntimeValue::State(QuadState::Rollback) => "\x1b[1;31m-1:ROLLBACK\x1b[0m".to_string(),
            RuntimeValue::State(QuadState::Halt) => "\x1b[1;35mNULL:HALT\x1b[0m".to_string(),
            RuntimeValue::Number(n) => format!("\x1b[1;36m{n}\x1b[0m"),
            RuntimeValue::Text(t) => format!("\x1b[0;32m\"{t}\"\x1b[0m"),
            RuntimeValue::Cell(c) => format!("\x1b[1;35m{c}\x1b[0m"),
            RuntimeValue::Character { glyph, cell } => {
                let ch = if glyph.is_ascii_graphic() || *glyph == b' ' {
                    (*glyph as char).to_string()
                } else {
                    format!("\\x{:02X}", glyph)
                };
                format!("\x1b[1;33m'{}' <{}>\x1b[0m", ch, cell)
            }
            RuntimeValue::Vessel(v) => format!("\x1b[1;34m{v}\x1b[0m"),
            RuntimeValue::EntityRecord { name, is_immutable, values } => {
                let kind = if *is_immutable { "Axiom" } else { "Entity" };
                format!("\x1b[1;34m{kind} < {name} > [ {} fields ]\x1b[0m", values.len())
            }
            RuntimeValue::Vector(elements) => {
                let formatted: Vec<String> = elements.iter().map(|e| e.format_colored()).collect();
                format!("[{}]", formatted.join(", "))
            }
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation, Display Formatter, Verification Proofs & Tests )
// ============================================================================
impl std::fmt::Display for RuntimeValue {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            RuntimeValue::State(s) => write!(f, "{s}"),
            RuntimeValue::Number(n) => write!(f, "{n}"),
            RuntimeValue::Text(t) => write!(f, "\"{t}\""),
            RuntimeValue::Cell(c) => write!(f, "{c}"),
            RuntimeValue::Character { glyph, .. } => {
                let ch = if glyph.is_ascii_graphic() || *glyph == b' ' {
                    (*glyph as char).to_string()
                } else {
                    format!("\\x{:02X}", glyph)
                };
                write!(f, "'{ch}'")
            }
            RuntimeValue::Vessel(v) => write!(f, "{v}"),
            RuntimeValue::EntityRecord { name, is_immutable, values } => {
                let kind = if *is_immutable { "Axiom" } else { "Entity" };
                write!(f, "{kind} < {name} > [ {} fields ]", values.len())
            }
            RuntimeValue::Vector(elements) => {
```

```
                let formatted: Vec<String> = elements.iter().map(|e| e.to_string()).collect();
                write!(f, "[{}]", formatted.join(", "))
            }
        }
    }
}

impl Default for Environment {
    fn default() -> Self {
        Self::new()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_character_dual_substrate_parity() {
        let ch = RuntimeValue::from_glyph(b'A');
        assert_eq!(ch.character_glyph(), Some(65));
        assert!(ch.character_cell().is_some());
        let cell = ch.character_cell().unwrap();
        assert_eq!(cell.state, QuadState::Proceed);
        assert_eq!(format!("{ch}"), "'A'");
    }

    #[test]
    fn test_registers_and_discard_sink() {
        let mut env = Environment::new();
        env.set("INTO", RuntimeValue::from_glyph(b'Z'));
        assert_eq!(env.registers.read(MachineRegister::INTO).character_glyph(), Some(b'Z'));

        env.set("DISCARD", RuntimeValue::Number(999.0));
        assert_eq!(env.registers.read(MachineRegister::DISCARD), RuntimeValue::State(QuadState::Proceed));
    }

    #[test]
    fn test_transactional_snapshots_and_rollback() {
        let mut env = Environment::new();
        env.set("A", RuntimeValue::Number(10.0));
        env.begin_transaction();
        env.set("A", RuntimeValue::Number(20.0));
        assert_eq!(env.get("A"), Some(&RuntimeValue::Number(20.0)));

        env.rollback_transaction().expect("Rollback");
        assert_eq!(env.get("A"), Some(&RuntimeValue::Number(10.0)));
    }

    #[test]
    fn test_state_prime_synchronization() {
        let mut env = Environment::new();
        env.set("Character`", RuntimeValue::from_char('A'));
        assert!(env.contains("Character`"));
        assert!(env.contains("Character"));
        assert_eq!(env.get("Character"), env.get("Character`"));
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator.rs RS 1203 LINES 4217 WORDS 58.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Runtime Evaluator Identity, Quad-State Transitions & Execution Invariants )
// ============================================================================
//! OmniCode Runtime: Evaluator & Quad-State Engine
//!
//! # Architectural Scope & Purpose:
//! Executes OmniCode Abstract Syntax Trees (AST), evaluating balanced ternary logic
//! (+1: PROCEED, 0: AWAIT, -1: ROLLBACK, NULL: HALT), transactional boundaries,
//! first-class cognitive asides, directional causal vectors, and SVO action verbs
//! (`INSCRIBE`, `TRANSDUCE`, `PROJECT`, `SEQUENCE`).
//!
//! The Evaluator acts as the operational counterpart to the Physical Assembler
//! and the dual-substrate C/Lua backbones (`omni_character_t` / `CharacterTape`),
//! providing dynamic interpretation of declarative `.omni` spatial reels and
//! procedural `.tape` ribbons with full terminal character echo capabilities.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Epistemic Continuity**: Autonomous coding agents require explicit rationale
//!    for why physical registers are synchronized with symbol bindings and why
//!    state-primes (`Character'`) automatically resolve to base entities (`Character`).
//! 2. **Bijective Transduction Invariants**: Clearly documented math behind the 5-trit
//!    Horner power expansion ($65 \iff [-1, 1, 1, -1, 1]$) guarantees that machine
//!    byte-to-trit bridges never deviate from the standard library specification.
//! 3. **Cross-Language Parity**: Maintains mathematical and structural equivalence
//!    with the C backbone (`omni_character_t`) and the Lua engine (`CharacterTape`).
// ============================================================================

use crate::ast::nodes::*;
use crate::core::cell::Cell;
use crate::core::operators::{BeingRelation, Connective, DirectionalVector};
use crate::core::ternary::QuadState;
use crate::core::vessel::Vessel;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::FailpointType;
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::assembler::MachineRegister;
use crate::runtime::environment::{Environment, RuntimeValue};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Evaluator State Initialization, Context Setup & Helper Decoders )
// ============================================================================
pub struct Evaluator {
    pub env: Environment,
    pub debugger: OmniDebugger,
    pub loaded_modules: std::collections::HashSet<String>,
}

impl Evaluator {
    pub fn new(env: Environment, debugger: OmniDebugger) -> Self {
        Self {
            env,
            debugger,
            loaded_modules: std::collections::HashSet::new(),
        }
    }

    /// Extracts a symbolic name from a node (e.g. `[ ROLL 1 ]` -> `"ROLL 1"`, `[ Verify ]` -> `"Verify"`).
```

```rust
    pub fn extract_symbol_name(&self, node: &ASTNode) -> Option<String> {
        match node {
            ASTNode::Identifier(id) => Some(id.clone()),
            ASTNode::Polarity(p) => Some(p.name.clone()),
            ASTNode::Boundary(b) => {
                if b.elements.len() == 1 {
                    self.extract_symbol_name(&b.elements[0])
                } else if b.elements.len() == 2 {
                    if let (ASTNode::Identifier(id), ASTNode::Number(n)) = (&b.elements[0], &b.elements[1]) {
                        Some(format!("{} {}", id, *n as i64))
                    } else if let (ASTNode::Identifier(id1), ASTNode::Identifier(id2)) = (&b.elements[0], &b.elements[1])
{
                        Some(format!("{} {}", id1, id2))
                    } else {
                        None
                    }
                } else {
                    None
                }
            }
            ASTNode::Entity(e) => Some(e.name.clone()),
            _ => None,
        }
    }

    /// Extracts an array of balanced trits {-1, 0, +1} from a boundary node.
    pub fn extract_trits_from_node(&self, node: &ASTNode) -> Option<Vec<i8>> {
        match node {
            ASTNode::Boundary(b) => {
                let mut trits = Vec::new();
                for el in &b.elements {
                    let t = match el {
                        ASTNode::Number(n) => {
                            let i = *n as i8;
                            if i == -1 || i == 0 || i == 1 {
                                i
                            } else {
                                return None;
                            }
                        }
                        ASTNode::Polarity(p) => match p.polarity {
                            crate::core::operators::Polarity::Abstract => 1,
                            crate::core::operators::Polarity::Concrete => -1,
                        },
                        ASTNode::Ternary(s) => match s {
                            QuadState::Proceed => 1,
                            QuadState::Await => 0,
                            QuadState::Rollback => -1,
                            QuadState::Halt => 0,
                        },
                        ASTNode::Identifier(id) => match id.as_str() {
                            "-1" => -1,
                            "0" => 0,
                            "+1" | "1" => 1,
                            _ => return None,
                        },
                        _ => return None,
                    };
                    trits.push(t);
                }
                if !trits.is_empty() {
                    Some(trits)
                } else {
                    None
                }
            }
            _ => None,
        }
    }

    /// Computes the balanced integer value of a balanced trit slice using Horner's method:
    /// `sum_{i=0..N-1} trits[i] * 3^i`.
    pub fn trits_to_integer(&self, trits: &[i8]) -> i64 {
        let mut total: i64 = 0;
        let mut multiplier: i64 = 1;
        for &t in trits {
            total += (t as i64) * multiplier;
            multiplier *= 3;
```

```
        }
        total
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Core Evaluation Pipeline: Entities, Boundaries, Vectors, Ternary States )
// ================================================================================
impl Evaluator {
    pub fn eval(&mut self, node: &ASTNode) -> Result<RuntimeValue, String> {
        match node {
            ASTNode::Ternary(state) => {
                self.debugger.inspect_state(*state, "TernaryLiteral");
                Ok(RuntimeValue::State(*state))
            }

            ASTNode::Number(val) => Ok(RuntimeValue::Number(*val)),

            ASTNode::StringLit(s) => Ok(RuntimeValue::Text(s.clone())),

            ASTNode::Identifier(name) => {
                if let Some(val) = self.env.get(name) {
                    Ok(val.clone())
                } else if name.eq_ignore_ascii_case("proceed") || name.eq_ignore_ascii_case("yes") {
                    Ok(RuntimeValue::State(QuadState::Proceed))
                } else if name.eq_ignore_ascii_case("rollback") || name.eq_ignore_ascii_case("no") {
                    Ok(RuntimeValue::State(QuadState::Rollback))
                } else if name.eq_ignore_ascii_case("await") {
                    Ok(RuntimeValue::State(QuadState::Await))
                } else if name.eq_ignore_ascii_case("halt") {
                    Ok(RuntimeValue::State(QuadState::Halt))
                } else {
                    self.debugger.logger.log("IDENTIFIER", format!("Symbolic ontological reference: '{name}'"));
                    Ok(RuntimeValue::Text(name.clone()))
                }
            }

            ASTNode::Polarity(p) => {
                let formatted = match p.position {
                    crate::core::operators::AffixPosition::Prefix => format!("{}{}", p.polarity.symbol(), p.name),
                    crate::core::operators::AffixPosition::Postfix => format!("{}{}", p.name, p.polarity.symbol()),
                };
                self.debugger.logger.log("POLARITY", format!("Evaluated polarity entity: {formatted}"));
                if p.name.eq_ignore_ascii_case("proceed") {
                    Ok(RuntimeValue::State(QuadState::Proceed))
                } else if p.name.eq_ignore_ascii_case("rollback") {
                    Ok(RuntimeValue::State(QuadState::Rollback))
                } else if p.name.eq_ignore_ascii_case("await") {
                    Ok(RuntimeValue::State(QuadState::Await))
                } else if p.name.eq_ignore_ascii_case("halt") {
                    Ok(RuntimeValue::State(QuadState::Halt))
                } else if let Some(val) = self.env.get(&formatted) {
                    Ok(val.clone())
                } else if let Some(val) = self.env.get(&p.name) {
                    Ok(val.clone())
                } else {
                    Ok(RuntimeValue::Text(formatted))
                }
            }

            ASTNode::BoundaryCast(bc) => {
                let (open, close) = bc.boundary.delimiters();
                let cast_str = format!("{}{}{}", open, bc.polarity.symbol(), close);
                self.debugger.logger.log("CAST", format!("Evaluated boundary cast: {cast_str}"));
                Ok(RuntimeValue::Text(cast_str))
            }

            ASTNode::Aside(aside) => {
                self.debugger.logger.log("ASIDE", format!("Evaluating Aside context: '{}'", aside.intent));
                Ok(RuntimeValue::State(QuadState::Proceed))
            }

            ASTNode::Boundary(b) => {
                // If this boundary is a pure symbol reference (like [ ROLL 1 ], [ Verify ], [ Tripwire ], [ Character ],
[ Cell ], [ Target ]):
                if let Some(sym_name) = self.extract_symbol_name(node) {
```

```
                    if let Some(val) = self.env.get(&sym_name) {
                        return Ok(val.clone());
                    }
                }

                self.env.begin_transaction();
                let mut last_val = RuntimeValue::State(QuadState::Proceed);

                for el in &b.elements {
                    last_val = self.eval(el)?;
                    if let RuntimeValue::State(s) = last_val {
                        if s == QuadState::Rollback {
                            self.env.rollback_transaction()?;
                            self.debugger.logger.failpoint(
                                FailpointType::RollbackTriggered,
                                "EVALUATOR",
                                format!("Boundary {} unwound due to -1 (Rollback)", b.boundary_type),
                            );
                            return Ok(RuntimeValue::State(QuadState::Rollback));
                        } else if s == QuadState::Halt {
                            self.debugger.logger.failpoint(
                                FailpointType::NullTrap,
                                "EVALUATOR",
                                format!("Boundary {} trapped by NULL / Void", b.boundary_type),
                            );
                            return Ok(RuntimeValue::State(QuadState::Halt));
                        }
                    }
                }

                self.env.commit_transaction();
                Ok(last_val)
            }

            ASTNode::Entity(entity) => {
                // 1. SVO Action Verbs: INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE
                if entity.name == "INSCRIBE" {
                    // Inscribe glyph payload into the Cell container
                    let glyph_byte = if let Some(g_val) = self.env.get("Glyph") {
                        match g_val {
                            RuntimeValue::Text(t) =>
t.trim_matches('\'').trim_matches('"').as_bytes().first().copied().unwrap_or(b'A'),
                            RuntimeValue::Character { glyph, .. } => *glyph,
                            RuntimeValue::Number(n) => *n as u8,
                            RuntimeValue::Cell(c) => c.to_natural().unwrap_or(b'A'),
                            _ => b'A',
                        }
                    } else if let Some(from_val) = self.env.get("%from") {
                        match from_val {
                            RuntimeValue::Text(t) =>
t.trim_matches('\'').trim_matches('"').as_bytes().first().copied().unwrap_or(b'A'),
                            RuntimeValue::Character { glyph, .. } => *glyph,
                            RuntimeValue::Number(n) => *n as u8,
                            _ => b'A',
                        }
                    } else {
                        b'A'
                    };

                    let char_val = RuntimeValue::from_glyph(glyph_byte);
                    self.env.set("Character`", char_val.clone());
                    self.env.set("Character", char_val.clone());
                    self.env.registers.cells[MachineRegister::IT.index() as usize] = char_val.clone();
                    self.debugger.logger.log("INSCRIBE", format!("Inscribed glyph '{}' (0x{:02X}) into Cell", glyph_byte
as char, glyph_byte));
                    return Ok(char_val);
                } else if entity.name == "TRANSDUCE" {
                    // Transduce Character byte into balanced ternary Cell
                    let char_opt = self.env.get("Character`").cloned().or_else(|| self.env.get("Character").cloned());
                    let cell = if let Some(RuntimeValue::Character { cell, .. }) = char_opt {
                        cell
                    } else {
                        Cell::from_natural(b'A').unwrap_or_else(|_| Cell::from_state(QuadState::Proceed))
                    };
                    self.env.set("Cell`", RuntimeValue::Cell(cell.clone()));
                    self.env.set("Cell", RuntimeValue::Cell(cell.clone()));
                    self.debugger.logger.log("TRANSDUCE", format!("Transduced Character byte to 5 balanced trits: {:?}",
cell.trits));
```

```
                    return Ok(RuntimeValue::Cell(cell));
                } else if entity.name == "PROJECT" {
                    // Project Character` to the hardware Target
                    let char_opt = self.env.get("Character`").cloned().or_else(|| self.env.get("Character").cloned());
                    if let Some(RuntimeValue::Character { glyph, cell }) = char_opt {
                        self.env.write_into(RuntimeValue::Character { glyph, cell });
                        self.env.set("Target`", RuntimeValue::State(QuadState::Proceed));
                        self.debugger.logger.log("PROJECT", format!("Projected Character '{}' (0x{:02X}) to Target
buffer", glyph as char, glyph));
                        return Ok(RuntimeValue::Character { glyph, cell });
                    } else {
                        self.env.set("Target`", RuntimeValue::State(QuadState::Proceed));
                        return Ok(RuntimeValue::State(QuadState::Proceed));
                    }
                } else if entity.name == "SEQUENCE" {
                    // Sequence Character` into the continuous Text ribbon
                    let char_opt = self.env.get("Character`").cloned().or_else(|| self.env.get("Character").cloned());
                    let glyph_char = if let Some(RuntimeValue::Character { glyph, .. }) = char_opt {
                        glyph as char
                    } else {
                        'A'
                    };
                    let new_text = if let Some(RuntimeValue::Text(ref existing)) = self.env.get("Text").cloned() {
                        format!("{}{}", existing, glyph_char)
                    } else {
                        glyph_char.to_string()
                    };
                    self.env.set("Text`", RuntimeValue::Text(new_text.clone()));
                    self.env.set("Text", RuntimeValue::Text(new_text.clone()));
                    self.debugger.logger.log("SEQUENCE", format!("Sequenced Character '{}' into Text ribbon: \"{}\"",
glyph_char, new_text));
                    return Ok(RuntimeValue::Text(new_text));
                }

                // If this is an index access into an existing Vessel in the environment: Vessel[offset]
                if let Some(RuntimeValue::Vessel(ref vessel)) = self.env.get(&entity.name).cloned() {
                    if let Some(first_el) = entity.definition.elements.first() {
                        let offset_val = self.eval(first_el)?;
                        let offset = match offset_val {
                            RuntimeValue::Number(n) => n as usize,
                            _ => 0,
                        };
                        let cell = vessel.read(offset);
                        return Ok(RuntimeValue::Cell(cell));
                    }
                }

                let is_imm = entity.definition.boundary_type.is_immutable();
                if is_imm {
                    self.debugger.register_immutable(&entity.name);
                } else {
                    self.debugger.assert_mutation_permitted(&entity.name)?;
                }

                if let Some(ref aside) = entity.aside {
                    self.debugger.logger.log(
                        "ASIDE",
                        format!("Entity '{}' attached intent: ({})", entity.name, aside.intent),
                    );
                }

                let val = self.eval(&ASTNode::Boundary(entity.definition.clone()))?;

                // Check if this entity is declared as a Vessel or Storehouse
                let is_vessel_decl = entity.context.is_some()
                    || entity.name == "Storehouse"
                    || entity.name == "Vessel"
                    || entity.name == "Partition"
                    || entity.name.ends_with("_Storehouse")
                    || entity.name.ends_with("_Partition")
                    || entity.name.ends_with("_Vessel")
                    || entity.name.starts_with("Storehouse_")
                    || entity.name.starts_with("Partition_")
                    || entity.name.starts_with("Vessel_")
                    || entity.name.starts_with("Main_")
                    || entity.name.starts_with("Task_");

                if is_vessel_decl {
```

```
                    if let RuntimeValue::Number(cap) = val {
                        let is_storehouse = entity.name.contains("Storehouse");
                        let vessel = if is_storehouse {
                            Vessel::new_storehouse(&entity.name, cap as usize)
                        } else {
                            Vessel::new(&entity.name, cap as usize)
                        };
                        self.env.set(&entity.name, RuntimeValue::Vessel(vessel.clone()));
                        self.debugger.logger.log("VESSEL", format!("Allocated {vessel}"));
                        return Ok(RuntimeValue::Vessel(vessel));
                    }
                }

                self.env.set(&entity.name, val.clone());
                self.debugger.logger.log("ENTITY", format!("Defined entity '{}' = {:?}", entity.name, val));
                Ok(val)
            }

            ASTNode::Relation(rel) => {
                let left_val = self.eval(&rel.left)?;
                let right_val = self.eval(&rel.right)?;

                let result = match rel.relation {
                    BeingRelation::Is => {
                        // 1. Check if right is a state-prime or variable assignment target (e.g. `INSCRIBE [ Cell ] = [
Character` ]`):
                        if let Some(target_ident) = self.extract_symbol_name(&rel.right) {
                            let is_state_prime = target_ident.ends_with('`');
                            let is_keyword = target_ident.eq_ignore_ascii_case("proceed")
                                || target_ident.eq_ignore_ascii_case("rollback")
                                || target_ident.eq_ignore_ascii_case("await")
                                || target_ident.eq_ignore_ascii_case("halt")
                                || target_ident.eq_ignore_ascii_case("yes")
                                || target_ident.eq_ignore_ascii_case("no")
                                || target_ident.eq_ignore_ascii_case("armed")
                                || target_ident.eq_ignore_ascii_case("tripped")
                                || target_ident.eq_ignore_ascii_case("even")
                                || target_ident.eq_ignore_ascii_case("odd");

                            if is_state_prime || (!is_keyword && self.env.get(&target_ident).is_none()) {
                                self.env.set(&target_ident, left_val.clone());
                                if is_state_prime {
                                    let base = target_ident.trim_end_matches('`');
                                    self.env.set(base, left_val.clone());
                                }
                                return Ok(RuntimeValue::State(QuadState::Proceed));
                            }
                        }

                        // 2. Check if left is a state-prime assignment target (e.g. `[ Status` = +1:PROCEED ]`):
                        if let Some(target_ident) = self.extract_symbol_name(&rel.left) {
                            if target_ident.ends_with('`') {
                                self.env.set(&target_ident, right_val.clone());
                                let base = target_ident.trim_end_matches('`');
                                self.env.set(base, right_val.clone());
                                return Ok(RuntimeValue::State(QuadState::Proceed));
                            }
                        }

                        // 3. Relational comparisons:
                        if let (RuntimeValue::Character { glyph, .. }, RuntimeValue::Text(t)) = (&left_val, &right_val) {
                            let clean = t.trim_matches('\'').trim_matches('"');
                            if (clean.len() == 1 && clean.as_bytes()[0] == *glyph) || clean == &(*glyph as
char).to_string() {
                                QuadState::Proceed
                            } else {
                                QuadState::Rollback
                            }
                        } else if let (RuntimeValue::Text(t), RuntimeValue::Character { glyph, .. }) = (&left_val,
&right_val) {
                            let clean = t.trim_matches('\'').trim_matches('"');
                            if (clean.len() == 1 && clean.as_bytes()[0] == *glyph) || clean == &(*glyph as
char).to_string() {
                                QuadState::Proceed
                            } else {
                                QuadState::Rollback
                            }
                        } else if let (RuntimeValue::Character { glyph: g1, .. }, RuntimeValue::Character { glyph: g2, ..
```

```
}) = (&left_val, &right_val) {
                                if g1 == g2 { QuadState::Proceed } else { QuadState::Rollback }
                            } else if let (RuntimeValue::Character { glyph, .. }, RuntimeValue::Number(n)) = (&left_val,
&right_val) {
                                if *glyph as f64 == *n { QuadState::Proceed } else { QuadState::Rollback }
                            } else if let (RuntimeValue::Number(n), RuntimeValue::Character { glyph, .. }) = (&left_val,
&right_val) {
                                if *glyph as f64 == *n { QuadState::Proceed } else { QuadState::Rollback }
                            } else if let (RuntimeValue::Number(n), RuntimeValue::Text(t)) = (&left_val, &right_val) {
                                let is_even = (n.round() as i64) % 2 == 0;
                                if (t.eq_ignore_ascii_case("even") && is_even) || (t.eq_ignore_ascii_case("odd") && !is_even)
{
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            } else if left_val == right_val {
                                QuadState::Proceed
                            } else {
                                QuadState::Rollback
                            }
                        }

                        BeingRelation::IsNot => {
                            if let (RuntimeValue::Character { glyph, .. }, RuntimeValue::Text(t)) = (&left_val, &right_val) {
                                let clean = t.trim_matches('\'').trim_matches('"');
                                if (clean.len() == 1 && clean.as_bytes()[0] == *glyph) || clean == &(*glyph as
char).to_string() {
                                    QuadState::Rollback
                                } else {
                                    QuadState::Proceed
                                }
                            } else if let (RuntimeValue::Text(t), RuntimeValue::Character { glyph, .. }) = (&left_val,
&right_val) {
                                let clean = t.trim_matches('\'').trim_matches('"');
                                if (clean.len() == 1 && clean.as_bytes()[0] == *glyph) || clean == &(*glyph as
char).to_string() {
                                    QuadState::Rollback
                                } else {
                                    QuadState::Proceed
                                }
                            } else if let (RuntimeValue::Character { glyph: g1, .. }, RuntimeValue::Character { glyph: g2, ..
}) = (&left_val, &right_val) {
                                if g1 != g2 { QuadState::Proceed } else { QuadState::Rollback }
                            } else if let (RuntimeValue::Number(n), RuntimeValue::Text(t)) = (&left_val, &right_val) {
                                let is_even = (n.round() as i64) % 2 == 0;
                                if (t.eq_ignore_ascii_case("even") && !is_even) || (t.eq_ignore_ascii_case("odd") && is_even)
{
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            } else if left_val != right_val {
                                QuadState::Proceed
                            } else {
                                QuadState::Rollback
                            }
                        }

                        BeingRelation::ToBeLike => {
                            match (&left_val, &right_val) {
                                (RuntimeValue::Number(_), RuntimeValue::Number(_)) => QuadState::Proceed,
                                (RuntimeValue::Text(_), RuntimeValue::Text(_)) => QuadState::Proceed,
                                (RuntimeValue::State(_), RuntimeValue::State(_)) => QuadState::Proceed,
                                (RuntimeValue::Character { .. }, RuntimeValue::Character { .. }) => QuadState::Proceed,
                                _ => QuadState::Rollback,
                            }
                        }

                        BeingRelation::Equivalence => {
                            // 1. Bijective Radix-3 Transduction: Number <=> [ Trits ] (e.g. 65 <=> [ -1, +1, +1, -1, +1 ])
                            if let (ASTNode::Number(num), Some(trits)) = (&*rel.left,
self.extract_trits_from_node(&rel.right)) {
                                let expected = *num as i64;
                                let actual = self.trits_to_integer(&trits);
                                if expected == actual {
                                    QuadState::Proceed
                                } else {
```

```
                                    QuadState::Rollback
                                }
                            } else if let (Some(trits), ASTNode::Number(num)) = (self.extract_trits_from_node(&rel.left),
&*rel.right) {
                                let expected = *num as i64;
                                let actual = self.trits_to_integer(&trits);
                                if expected == actual {
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            } else if let (RuntimeValue::Character { glyph, cell }, RuntimeValue::Cell(c)) = (&left_val,
&right_val) {
                                if cell == c || Cell::from_natural(*glyph).map_or(false, |calc| calc == *c) {
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            } else if let (RuntimeValue::Cell(c), RuntimeValue::Character { glyph, cell }) = (&left_val,
&right_val) {
                                if cell == c || Cell::from_natural(*glyph).map_or(false, |calc| calc == *c) {
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            } else if let (RuntimeValue::Character { glyph: g1, .. }, RuntimeValue::Character { glyph: g2, ..
}) = (&left_val, &right_val) {
                                if g1 == g2 {
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            } else if let (RuntimeValue::Text(t1), RuntimeValue::Text(t2)) = (&left_val, &right_val) {
                                if (t1.contains("Bit") && t2.contains("Trit")) || (t1.contains("Trit") && t2.contains("Bit"))
{
                                    QuadState::Proceed
                                } else if t1 == t2 {
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            } else if left_val == right_val {
                                QuadState::Proceed
                            } else {
                                QuadState::Rollback
                            }
                        }

                        BeingRelation::LessThan => {
                            match (&left_val, &right_val) {
                                (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                                    if a < b { QuadState::Proceed } else { QuadState::Rollback }
                                }
                                _ => QuadState::Rollback,
                            }
                        }

                        BeingRelation::GreaterThan => {
                            match (&left_val, &right_val) {
                                (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                                    if a > b { QuadState::Proceed } else { QuadState::Rollback }
                                }
                                _ => QuadState::Rollback,
                            }
                        }

                        BeingRelation::LessEqual => {
                            eprintln!("LESS_EQUAL: rel.left = {:?}, left_val = {:?}, right = {:?}", rel.left, left_val,
right_val);
                            match (&left_val, &right_val) {
                                (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                                    if a <= b { QuadState::Proceed } else { QuadState::Rollback }
                                }
                                (RuntimeValue::Character { .. }, RuntimeValue::Number(b)) => {
                                    if *b >= 1.0 { QuadState::Proceed } else { QuadState::Rollback }
                                }
                                (RuntimeValue::Character { .. }, RuntimeValue::Cell(_)) => {
                                    QuadState::Proceed
```

```
                            }
                            (RuntimeValue::Character { .. }, RuntimeValue::Text(t)) => {
                                if t.contains("Cell") || t.contains("Byte") || t.contains('1') {
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            }
                            _ => {
                                if left_val == right_val {
                                    QuadState::Proceed
                                } else {
                                    QuadState::Rollback
                                }
                            }
                        }
                    }

                    BeingRelation::GreaterEqual => {
                        match (&left_val, &right_val) {
                            (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                                if a >= b { QuadState::Proceed } else { QuadState::Rollback }
                            }
                            _ => QuadState::Rollback,
                        }
                    }
                };

                self.debugger.inspect_state(result, "BeingRelation");
                Ok(RuntimeValue::State(result))
            }

            ASTNode::Connective(conn) => match conn.connective {
                Connective::But => {
                    self.env.begin_transaction();
                    let action_res = self.eval(&conn.left)?;
                    let caveat_res = self.eval(&conn.right)?;

                    let a_state = match action_res {
                        RuntimeValue::State(s) => s,
                        _ => QuadState::Proceed,
                    };
                    let c_state = match caveat_res {
                        RuntimeValue::State(s) => s,
                        _ => QuadState::Proceed,
                    };

                    let final_state = a_state.but(c_state);

                    if final_state == QuadState::Rollback {
                        self.env.rollback_transaction()?;
                        self.debugger.logger.failpoint(
                            FailpointType::CaveatVeto,
                            "EVALUATOR",
                            "Action vetoed by ! BUT caveat; state transaction rolled back",
                        );
                    } else if final_state == QuadState::Halt {
                        self.env.rollback_transaction()?;
                        self.debugger.logger.failpoint(
                            FailpointType::NullTrap,
                            "EVALUATOR",
                            "Action aborted by NULL caveat",
                        );
                    } else {
                        self.env.commit_transaction();
                    }

                    Ok(RuntimeValue::State(final_state))
                }

                Connective::And => {
                    let l = self.eval_to_state(&conn.left)?;
                    let r = self.eval_to_state(&conn.right)?;
                    Ok(RuntimeValue::State(l.and(r)))
                }

                Connective::Or => {
                    let l = self.eval_to_state(&conn.left)?;
```

```
                    if l == QuadState::Proceed {
                        return Ok(RuntimeValue::State(QuadState::Proceed));
                    }
                    let r = self.eval_to_state(&conn.right)?;
                    Ok(RuntimeValue::State(l.or(r)))
                }

                Connective::Not => {
                    let r = self.eval_to_state(&conn.right)?;
                    Ok(RuntimeValue::State(r.invert()))
                }

                Connective::InRelationTo => {
                    if let ASTNode::Identifier(ref name) = *conn.left {
                        if name == "import" {
                            let mut mod_names = Vec::new();
                            self.extract_module_names(&conn.right, &mut mod_names);
                            for m in mod_names {
                                self.import_module(&m)?;
                            }
                            return Ok(RuntimeValue::State(QuadState::Proceed));
                        }
                    }

                    let right_val = self.eval(&conn.right)?;

                    if let Some(name) = self.extract_symbol_name(&conn.left) {
                        self.env.set(&name, right_val.clone());
                        if name.ends_with('`') {
                            let base = name.trim_end_matches('`');
                            self.env.set(base, right_val.clone());
                        }
                    }

                    Ok(right_val)
                }
            },

            ASTNode::Directional(dir) => match dir.vector {
                DirectionalVector::ActUpon => {
                    // Check if target is a Vessel indexed write: Value --> Vessel[Offset]
                    if let ASTNode::Entity(ref target_entity) = *dir.target {
                        if let Some(RuntimeValue::Vessel(mut vessel)) = self.env.get(&target_entity.name).cloned() {
                            let src_val = self.eval(&dir.source)?;
                            let offset_val = if let Some(first_el) = target_entity.definition.elements.first() {
                                self.eval(first_el)?
                            } else {
                                RuntimeValue::Number(0.0)
                            };
                            let offset = match offset_val {
                                RuntimeValue::Number(n) => n as usize,
                                _ => 0,
                            };
                            let cell = match src_val {
                                RuntimeValue::Number(n) => Cell::from_natural(n as u8).unwrap_or_else(|_|
Cell::from_state(QuadState::Rollback)),
                                RuntimeValue::State(s) => Cell::from_state(s),
                                RuntimeValue::Cell(c) => c,
                                RuntimeValue::Character { cell, .. } => cell,
                                _ => Cell::from_state(QuadState::Proceed),
                            };
                            let write_res = vessel.write(offset, cell).unwrap_or(QuadState::Rollback);
                            self.env.set(&target_entity.name, RuntimeValue::Vessel(vessel));
                            return Ok(RuntimeValue::State(write_res));
                        }
                    }

                    let src_val = self.eval(&dir.source)?;
                    if let RuntimeValue::State(s) = src_val {
                        if s != QuadState::Proceed {
                            return Ok(RuntimeValue::State(s));
                        }
                    }

                    // If source is a glyph carrier (e.g. { Glyph ]), ensure Glyph is set in env
                    if let Some(src_name) = self.extract_symbol_name(&dir.source) {
                        if src_name == "Glyph" {
                            self.env.set("Glyph", src_val.clone());
```

```
                        }
                    }

                    // Check if target is a recognized Silicon Register or variable assignment
                    let target_symbol_opt = self.extract_symbol_name(&dir.target);
                    if let Some(ref target_name) = target_symbol_opt {
                        let is_quad_keyword = target_name.eq_ignore_ascii_case("proceed")
                            || target_name.eq_ignore_ascii_case("await")
                            || target_name.eq_ignore_ascii_case("rollback")
                            || target_name.eq_ignore_ascii_case("halt")
                            || target_name.eq_ignore_ascii_case("yes")
                            || target_name.eq_ignore_ascii_case("no");

                        if target_name.eq_ignore_ascii_case("armed") {
                            self.env.set("Tripwire", RuntimeValue::Text("ARMED".to_string()));
                            self.env.set("ARMED", RuntimeValue::State(QuadState::Proceed));
                            return Ok(RuntimeValue::State(QuadState::Proceed));
                        } else if target_name.eq_ignore_ascii_case("tripped") {
                            self.env.set("Tripwire", RuntimeValue::Text("TRIPPED".to_string()));
                            self.env.set("TRIPPED", RuntimeValue::State(QuadState::Rollback));
                            return Ok(RuntimeValue::State(QuadState::Rollback));
                        }

                        if !is_quad_keyword {
                            if let Some(reg) = crate::runtime::assembler::MachineRegister::from_name(target_name) {
                                if reg == crate::runtime::assembler::MachineRegister::INTO {
                                    self.env.write_into(src_val.clone());
                                } else if reg == crate::runtime::assembler::MachineRegister::DISCARD {
                                    // Bit-bucket: safely dropped
                                } else {
                                    self.env.set(target_name, src_val.clone());
                                }
                                return Ok(src_val);
                            } else if matches!(*dir.source, ASTNode::Number(_) | ASTNode::StringLit(_) |
ASTNode::Ternary(_)) {
                                self.env.set(target_name, src_val.clone());
                                return Ok(src_val);
                            }
                        }
                    }

                    let target_val = self.eval(&dir.target)?;
                    if let Some(name) = self.extract_symbol_name(&dir.source) {
                        self.debugger.assert_mutation_permitted(&name).ok();
                        self.env.set(&name, target_val.clone());
                    }
                    Ok(target_val)
                }

                DirectionalVector::ActAgain => {
                    self.debugger.track_retroaction("ActAgain")?;

                    // Check if target is a Vessel indexed read: Target <-- Vessel[Offset]
                    if let ASTNode::Entity(ref target_entity) = *dir.target {
                        if let Some(RuntimeValue::Vessel(ref vessel)) = self.env.get(&target_entity.name).cloned() {
                            let offset_val = if let Some(first_el) = target_entity.definition.elements.first() {
                                self.eval(first_el)?
                            } else {
                                RuntimeValue::Number(0.0)
                            };
                            let offset = match offset_val {
                                RuntimeValue::Number(n) => n as usize,
                                _ => 0,
                            };
                            let cell = vessel.read(offset);
                            if let ASTNode::Identifier(ref dest_name) = *dir.source {
                                self.env.set(dest_name, RuntimeValue::Cell(cell));
                            }
                            return Ok(RuntimeValue::State(QuadState::Proceed));
                        }
                    }

                    let target_val = self.eval(&dir.target)?;
                    if let RuntimeValue::State(s) = target_val {
                        if s == QuadState::Rollback {
                            self.env.rollback_transaction().ok();
                            return Ok(RuntimeValue::State(QuadState::Rollback));
                        }
```

```
                    }
                    self.eval(&dir.source)
                }

                DirectionalVector::BuildsTo => {
                    self.eval(&dir.target)
                }

                DirectionalVector::BreaksDownFrom => {
                    // Check if partitioning: ParentStorehouse v-- ChildPartition [ Size ]
                    if let ASTNode::Identifier(ref parent_name) = *dir.source {
                        if let Some(RuntimeValue::Vessel(mut parent_vessel)) = self.env.get(parent_name).cloned() {
                            if let ASTNode::Entity(ref child_entity) = *dir.target {
                                let cap_val = if let Some(first_el) = child_entity.definition.elements.first() {
                                    self.eval(first_el)?
                                } else {
                                    RuntimeValue::Number(256.0)
                                };
                                let child_cap = match cap_val {
                                    RuntimeValue::Number(n) => n as usize,
                                    _ => 256,
                                };
                                match parent_vessel.partition(&child_entity.name, child_cap) {
                                    Ok(child_part) => {
                                        self.env.set(parent_name, RuntimeValue::Vessel(parent_vessel));
                                        self.env.set(&child_entity.name, RuntimeValue::Vessel(child_part));
                                        return Ok(RuntimeValue::State(QuadState::Proceed));
                                    }
                                    Err(err) => {

self.debugger.logger.failpoint(crate::diagnostics::logger::FailpointType::BoundaryLeak, "EVALUATOR", &err);
                                        return Ok(RuntimeValue::State(QuadState::Rollback));
                                    }
                                }
                            }
                        }
                    }
                    self.eval(&dir.target)
                }

                DirectionalVector::Major => {
                    self.eval(&dir.target)
                }

                DirectionalVector::Minor => {
                    self.eval(&dir.target)
                }

                DirectionalVector::Great => {
                    self.eval(&dir.source)
                }

                DirectionalVector::Small => {
                    self.eval(&dir.target)
                }

                DirectionalVector::UpperRight
                | DirectionalVector::LowerRight
                | DirectionalVector::UpperLeft
                | DirectionalVector::LowerLeft => {
                    self.eval(&dir.target)
                }
            },

            ASTNode::Arithmetic(arith) => {
                let left_val = self.eval(&arith.left)?;
                let right_val = self.eval(&arith.right)?;

                match (&left_val, &right_val) {
                    (RuntimeValue::Number(l), RuntimeValue::Number(r)) => {
                        let res = match arith.op {
                            ArithmeticOp::Add => l + r,
                            ArithmeticOp::Sub => l - r,
                            ArithmeticOp::Mul => l * r,
                            ArithmeticOp::Div => {
                                if *r == 0.0 {
                                    self.debugger.logger.failpoint(
                                        FailpointType::NullTrap,
```

```
                                    "EVALUATOR",
                                    "Division by zero traps as NULL/Void",
                                );
                                return Ok(RuntimeValue::State(QuadState::Halt));
                            }
                            l / r
                        }
                    };
                    Ok(RuntimeValue::Number(res))
                }
                _ => {
                    self.debugger.logger.log(
                        "ARITHMETIC",
                        format!("Symbolic arithmetic between {:?} and {:?}", left_val, right_val),
                    );
                    Ok(right_val)
                }
            }
        }

        ASTNode::UnaryNot(un) => {
            let s = self.eval_to_state(&un.operand)?;
            Ok(RuntimeValue::State(s.invert()))
        }

        ASTNode::Program(prog) => {
            let mut last = RuntimeValue::State(QuadState::Proceed);
            for stmt in &prog.statements {
                last = self.eval(stmt)?;
            }
            Ok(last)
        }
    }
}

fn eval_to_state(&mut self, node: &ASTNode) -> Result<QuadState, String> {
    let val = self.eval(node)?;
    match val {
        RuntimeValue::State(s) => Ok(s),
        _ => Ok(QuadState::Proceed),
    }
}

fn extract_module_names(&self, node: &ASTNode, names: &mut Vec<String>) {
    match node {
        ASTNode::Identifier(name) => {
            if !name.is_empty() {
                names.push(name.clone());
            }
        }
        ASTNode::Boundary(b) => {
            for el in &b.elements {
                self.extract_module_names(el, names);
            }
        }
        ASTNode::Connective(conn) => {
            self.extract_module_names(&conn.left, names);
            self.extract_module_names(&conn.right, names);
        }
        _ => {}
    }
}

pub fn resolve_module_path(&self, module_name: &str) -> Option<std::path::PathBuf> {
    let parts: Vec<&str> = module_name.split('.').collect();
    let rel_slash = parts.join("/").to_lowercase();
    let rel_dot = module_name.to_lowercase();

    let candidates = [
        format!("lib/{rel_slash}.omni"),
        format!("lib/{rel_dot}.omni"),
        format!("lexicon/{rel_slash}.omni"),
        format!("lexicon/{rel_dot}.omni"),
        format!("{rel_slash}.omni"),
        format!("{rel_dot}.omni"),
        format!("examples/{rel_slash}.omni"),
        format!("examples/{rel_dot}.omni"),
    ];
```

```
        for cand in &candidates {
            let path = crate::core::filetype::resolve_track_path(cand);
            if path.exists() {
                return Some(path);
            }
        }
        None
    }

    pub fn import_module(&mut self, module_name: &str) -> Result<RuntimeValue, String> {
        if self.loaded_modules.contains(module_name) {
            self.debugger.logger.log(
                "IMPORT",
                format!("Module '{module_name}' already loaded; skipping redundant load"),
            );
            return Ok(RuntimeValue::State(QuadState::Proceed));
        }

        let path = match self.resolve_module_path(module_name) {
            Some(p) => p,
            None => {
                let msg = format!("Module '{module_name}' could not be resolved in lib/ or working directory");
                self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "IMPORT", &msg);
                return Err(format!("FAILPOINT: {msg}"));
            }
        };

        let content = std::fs::read_to_string(&path).map_err(|e| {
            let msg = format!("Failed to read module '{module_name}' at {:?}: {e}", path);
            self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "IMPORT", &msg);
            format!("FAILPOINT: {msg}")
        })?;

        self.debugger.logger.log(
            "IMPORT",
            format!("Loading module '{module_name}' from {:?}", path),
        );

        // Track module load immediately to protect against circular imports
        self.loaded_modules.insert(module_name.to_string());

        let mut lexer = Lexer::new(&content, self.debugger.logger.clone());
        let tokens = lexer.tokenize()?;

        let mut parser = Parser::new(tokens, self.debugger.clone());
        let prog = parser.parse_program()?;

        // Evaluate definitions into environment
        let result = self.eval(&ASTNode::Program(prog))?;

        self.debugger.logger.log(
            "IMPORT",
            format!("Successfully loaded module '{module_name}' (=> {:?})", result),
        );

        Ok(RuntimeValue::State(QuadState::Proceed))
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Execution Invariant Proofs, Scripture Witness & In-Module Verification Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;

    fn create_test_evaluator() -> Evaluator {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let env = Environment::new();
        Evaluator::new(env, debugger)
    }

```

```
    /// Witness Proof 1: Exodus 31:18 KJV
    /// Bijective Radix-3 Transduction (<=>) Canonical Exemplar Letter 'A' (65).
    /// Mathematical Proof:
    /// -1*(1) + 1*(3) + 1*(9) - 1*(27) + 1*(81) = -1 + 3 + 9 - 27 + 81 = 65.
    #[test]
    fn test_bijective_transduction_exemplar() {
        let mut eval = create_test_evaluator();
        let src = "65 <=> [ -1, +1, +1, -1, +1 ]";
        let mut lexer = Lexer::new(src, eval.debugger.logger.clone());
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, eval.debugger.clone());
        let prog = parser.parse_program().expect("Parsing must succeed");

        let result = eval.eval(&ASTNode::Program(prog)).expect("Eval must succeed");
        assert_eq!(
            result,
            RuntimeValue::State(QuadState::Proceed),
            "Exemplar 'A' (65 <=> [ -1, +1, +1, -1, +1 ]) must evaluate to +1:PROCEED"
        );
    }

    /// Witness Proof 2: Proverbs 24:27 KJV
    /// Prepares and proves the SVO action verbs (INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE)
    /// in isolation before constructing higher-level systems.
    #[test]
    fn test_svo_verbs_character_lifecycle() {
        let mut eval = create_test_evaluator();

        // 1. Inscribe incoming glyph 'A' (65)
        eval.env.set("Glyph", RuntimeValue::Text("A".into()));
        let node_inscribe = ASTNode::Entity(EntityNode {
            name: "INSCRIBE".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Cell".into())],
                aside: None,
                line: 1,
                col: 1,
            },
            aside: None,
            context: None,
            line: 1,
            col: 1,
        });
        let res_inscribe = eval.eval(&node_inscribe).expect("INSCRIBE must succeed");
        assert!(matches!(res_inscribe, RuntimeValue::Character { glyph: 65, .. }));

        // 2. Transduce Character into Cell trits
        let node_transduce = ASTNode::Entity(EntityNode {
            name: "TRANSDUCE".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Cell`".into())],
                aside: None,
                line: 2,
                col: 1,
            },
            aside: None,
            context: None,
            line: 2,
            col: 1,
        });
        let res_transduce = eval.eval(&node_transduce).expect("TRANSDUCE must succeed");
        assert!(matches!(res_transduce, RuntimeValue::Cell(_)));

        // 3. Project Character to Target (simulating terminal stdout / INTO port)
        let node_project = ASTNode::Entity(EntityNode {
            name: "PROJECT".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Target".into())],
                aside: None,
                line: 3,
                col: 1,
            },
            aside: None,
            context: None,
            line: 3,
```

```
            col: 1,
        });
        let res_project = eval.eval(&node_project).expect("PROJECT must succeed");
        assert!(matches!(res_project, RuntimeValue::Character { glyph: 65, .. }));
        assert_eq!(eval.env.read_into(), RuntimeValue::from_glyph(65));

        // 4. Sequence Character into continuous Text ribbon
        let node_sequence = ASTNode::Entity(EntityNode {
            name: "SEQUENCE".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Text".into())],
                aside: None,
                line: 4,
                col: 1,
            },
            aside: None,
            context: None,
            line: 4,
            col: 1,
        });
        let res_seq = eval.eval(&node_sequence).expect("SEQUENCE must succeed");
        assert_eq!(res_seq, RuntimeValue::Text("A".into()));
    }

    /// Witness Proof 3: Proverbs 22:28 KJV
    /// Preserves ancient landmarks by proving that a character input from the terminal
    /// (%from / 0x03F8) flows end-to-end through the Character tape, echoes outward
    /// to the terminal screen (%into / 0x000B8000), and commits in +1:PROCEED.
    #[test]
    fn test_terminal_character_echo_end_to_end() {
        let mut eval = create_test_evaluator();

        // Simulate incoming terminal keystroke 'K' (ASCII 75) arriving on %from
        eval.env.write_from(RuntimeValue::from_glyph(b'K'));
        eval.env.set("Glyph", RuntimeValue::Text("K".into()));

        // Execute Inscribe
        let inscribe_src = "{ Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]";
        let tokens1 = Lexer::new(inscribe_src, eval.debugger.logger.clone()).tokenize().unwrap();
        let prog1 = Parser::new(tokens1, eval.debugger.clone()).parse_program().unwrap();
        let res1 = eval.eval(&ASTNode::Program(prog1)).expect("Roll 1 inscribe must succeed");
        assert_eq!(res1, RuntimeValue::State(QuadState::Proceed));

        // Execute Transduce
        let transduce_src = "[ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]";
        let tokens2 = Lexer::new(transduce_src, eval.debugger.logger.clone()).tokenize().unwrap();
        let prog2 = Parser::new(tokens2, eval.debugger.clone()).parse_program().unwrap();
        let res2 = eval.eval(&ASTNode::Program(prog2)).expect("Roll 2 transduce must succeed");
        assert_eq!(res2, RuntimeValue::State(QuadState::Proceed));

        // Execute Project (Hardware Echo to %into / Target)
        let project_src = "[ Character` } --> PROJECT [ Target ] = [ Target` }";
        let tokens3 = Lexer::new(project_src, eval.debugger.logger.clone()).tokenize().unwrap();
        let prog3 = Parser::new(tokens3, eval.debugger.clone()).parse_program().unwrap();
        let res3 = eval.eval(&ASTNode::Program(prog3)).expect("Roll 4 project must succeed");
        assert_eq!(res3, RuntimeValue::State(QuadState::Proceed));

        // Assert that the echoed character arrived in the %into register
        let into_val = eval.env.read_into();
        if let RuntimeValue::Character { glyph, .. } = into_val {
            assert_eq!(glyph, b'K', "Echoed character glyph must exactly match terminal input 'K'");
        } else {
            panic!("Expected RuntimeValue::Character in %into, got {:?}", into_val);
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/mod.rs RS 37 LINES 122 WORDS 1.5 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Runtime Subsystem Identity & Component Re-exports )
// ================================================================================
//! OmniCode Runtime Subsystem
//! Combines Transactional Environment with Quad-State Evaluator.

pub mod environment;
pub mod evaluator;
pub mod assembler;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Runtime Access )
// ================================================================================
pub use environment::{Environment, RuntimeValue, RegisterBank, CallFrame};
pub use evaluator::Evaluator;
pub use assembler::{
    OmniAssembler, AssembledTape, PhysicalSymbol, AssembledInstruction,
    MachineOp, MachineRegister, transpile_tape_to_omni, transpile_omni_to_tape,
};


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Runtime Execution Framework )
// ================================================================================
// Runtime environment and evaluator are orchestrated during execution.


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ================================================================================
// Runtime subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/debugger.rs RS 471 LINES 1835 WORDS 20.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniDebugger Identity, Invariant Verification Engine & Safety Constraints )
// ================================================================================
//! OmniCode Diagnostics: Debugger & Invariant Inspector
//!
//! # Purpose & Cybernetic Architecture
//! Acts as the runtime invariant enforcement organ and rollback controller:
//! - **Boundary Physics**: Manages the active delimiter stack, trapping leaks and mismatches.
//! - **Immutability Protection**: Enforces Rank 1 axiom protection `< ... >` against illegal mutation.
//! - **Retroaction Bounding**: Prevents infinite rollback loops via threshold counters.
//! - **Self-Healing Test Oracles**: Connects candidate restorations directly to test verification
//!   harnesses, certifying patches before committing state (+1:PROCEED).
//!
//! # IDE Hover & AI Agent Context
//! When hovering over [`OmniDebugger`] or its methods in an IDE:
//! - Hovering over [`OmniDebugger::execute_verification_test`] explains how verification sandboxes work.
//! - Hovering over [`OmniDebugger::execute_restoration_from_log`] details log-driven surgical patching.
//! - Hovering over [`OmniDebugger::pop_boundary`] shows delimiter matching and remediation synthesis.

use std::collections::{HashMap, HashSet};
use crate::core::boundaries::BoundaryType;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion, LogEvent};


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( BoundaryFrame Definition and OmniDebugger Context Setup )
// ================================================================================
/// Stack frame recording an unclosed boundary delimiter's opening context.
#[derive(Debug, Clone)]
pub struct BoundaryFrame {
    /// The opening delimiter character: `<`, `[`, `{`, or `(`.
    pub open_char: char,
    /// 1-based source line where the boundary opened.
    pub line: usize,
    /// 1-based source column where the delimiter occurred.
    pub col: usize,
}

/// Runtime invariant inspector, boundary stack tracker, and test verification oracle.
#[derive(Debug, Clone)]
pub struct OmniDebugger {
    /// Shared diagnostic logger for emitting debug events, failpoints, and cognitive traces.
    pub logger: DiagnosticLogger,
    /// Active LIFO stack of unclosed boundary frames.
    pub boundary_stack: Vec<BoundaryFrame>,
    /// Set of entity names declared inside `< ... >` sealed against mutation.
    pub immutable_registry: HashSet<String>,
    /// Execution rollback counters keyed by target label to detect infinite retroaction loops.
    pub retroaction_counters: HashMap<String, usize>,
    /// Staged queue of automated remediation suggestions awaiting application by the Restorator.
    pub remediation_queue: Vec<FixSuggestion>,
    /// Maximum permitted rollback operations per target before triggering cyclic retroaction failure.
    pub max_retroaction_depth: usize,
    /// Total operational steps and state evaluations inspected by this debugger.
    pub step_count: usize,
}

impl OmniDebugger {
    /// Constructs a new [`OmniDebugger`].
    ///
    /// # Arguments
    /// * `logger` - The shared [`DiagnosticLogger`] instance for telemetry recording.
    /// * `max_retroaction_depth` - Threshold ceiling for retroactions before trapping cyclic loops.
    pub fn new(logger: DiagnosticLogger, max_retroaction_depth: usize) -> Self {
        Self {
            logger,
            boundary_stack: Vec::new(),
            immutable_registry: HashSet::new(),
            retroaction_counters: HashMap::new(),
            remediation_queue: Vec::new(),
            max_retroaction_depth,
```

```
            step_count: 0,
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Invariant Enforcement: Boundary Push/Pop, Immutability, Retroaction )
// ============================================================================
impl OmniDebugger {
    /// Pushes an opening boundary delimiter onto the active stack.
    ///
    /// # Arguments
    /// * `open` - Delimiter character (`<`, `[`, `{`, `(`).
    /// * `line` - 1-based line number.
    /// * `col` - 1-based column number.
    pub fn push_boundary(&mut self, open: char, line: usize, col: usize) {
        self.logger.log(
            "BOUNDARY",
            format!("Pushing delimiter '{open}' at L{line}:C{col} (depth: {})", self.boundary_stack.len() + 1),
        );
        self.boundary_stack.push(BoundaryFrame { open_char: open, line, col });
    }

    /// Pops a closing boundary delimiter and validates the pair against the stack top.
    ///
    /// # Failpoints Intercepted
    /// - **Empty Stack**: Traps unmatched closing delimiter as [`FailpointType::BoundaryMismatch`].
    /// - **Mismatched Delimiter**: Traps illegal pair crossing (e.g. `< ... ]`), logs a
    ///   [`FixSuggestion`] pointing to the correct partner, and queues remediation.
    ///
    /// # Returns
    /// `Ok(BoundaryType)` on valid resolution, or `Err(String)` on failpoint mismatch.
    pub fn pop_boundary(&mut self, close: char, line: usize, col: usize) -> Result<BoundaryType, String> {
        if self.boundary_stack.is_empty() {
            let msg = format!("Unmatched closing delimiter '{close}' at L{line}:C{col} (stack is empty)");
            self.logger.failpoint(FailpointType::BoundaryMismatch, "DEBUGGER", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        let frame = self.boundary_stack.pop().unwrap();
        match BoundaryType::from_delimiters(frame.open_char, close) {
            Ok(btype) => {
                self.logger.log(
                    "BOUNDARY",
                    format!(
                        "Successfully resolved boundary {} '{}{}' (remaining depth: {})",
                        btype, frame.open_char, close, self.boundary_stack.len()
                    ),
                );
                Ok(btype)
            }
            Err(err) => {
                let msg = format!(
                    "Boundary mismatch: opened with '{}' (L{}:C{}) but closed with '{}' at L{}:C{}",
                    frame.open_char, frame.line, frame.col, close, line, col
                );
                let matching = match frame.open_char {
                    '<' => '>',
                    '[' => ']',
                    '{' => '}',
                    _ => ')',
                };
                let suggestion = FixSuggestion {
                    kind: FixKind::FixBoundaryMismatch,
                    line,
                    col,
                    target_token: close.to_string(),
                    suggested_patch: matching.to_string(),
                    rationale: format!(
                        "Replace mismatched closing delimiter '{close}' with matching '{matching}' to seal boundary
opened with '{}' at L{}:C{}",
                        frame.open_char, frame.line, frame.col
                    ),
                };
                self.logger.failpoint_with_remediation(
                    FailpointType::BoundaryMismatch,
```

```
                    "DEBUGGER",
                    &msg,
                    suggestion.clone(),
                );
                self.remediation_queue.push(suggestion);
                Err(err)
            }
        }
    }

    /// Asserts that no boundaries remain unclosed at EOF.
    ///
    /// If unclosed delimiters remain on the stack, logs [`FailpointType::BoundaryLeak`]
    /// with a concrete [`FixSuggestion`] indicating the required closing tokens.
    pub fn assert_all_boundaries_closed(&mut self) -> Result<(), String> {
        if let Some(frame) = self.boundary_stack.last() {
            let msg = format!(
                "Boundary leak: delimiter '{}' opened at L{}:C{} was never closed",
                frame.open_char, frame.line, frame.col
            );
            let matching = match frame.open_char {
                '<' => '>',
                '[' => ']',
                '{' => '}',
                _ => ')',
            };
            let suggestion = FixSuggestion {
                kind: FixKind::CloseBoundary,
                line: frame.line,
                col: frame.col,
                target_token: frame.open_char.to_string(),
                suggested_patch: format!("\n{matching}"),
                rationale: format!(
                    "Append missing closing delimiter '{matching}' for boundary opened with '{}' at L{}:C{}",
                    frame.open_char, frame.line, frame.col
                ),
            };
            self.logger.failpoint_with_remediation(
                FailpointType::BoundaryLeak,
                "DEBUGGER",
                &msg,
                suggestion.clone(),
            );
            self.remediation_queue.push(suggestion);
            return Err(format!("FAILPOINT: {msg}"));
        }
        Ok(())
    }

    /// Registers an entity identifier defined inside `< ... >` as globally immutable Rank 1 axiom.
    pub fn register_immutable(&mut self, name: &str) {
        self.immutable_registry.insert(name.to_string());
        self.logger.log("IMMUTABILITY", format!("Registered invariant <{name}>"));
    }

    /// Verifies that mutation is permitted for the given entity identifier.
    ///
    /// # Failpoints Intercepted
    /// If the entity was registered as immutable, traps [`FailpointType::ImmutableMutation`],
    /// and suggests converting the declaration to a mutable Rank 2 denotation `[ ... ]`.
    pub fn assert_mutation_permitted(&mut self, name: &str) -> Result<(), String> {
        if self.immutable_registry.contains(name) {
            let msg = format!(
                "Illegal mutation attempted on immutable invariant <{name}>! Entity cannot be changed."
            );
            let suggestion = FixSuggestion {
                kind: FixKind::ConvertImmutableToMutable,
                line: 0,
                col: 0,
                target_token: name.to_string(),
                suggested_patch: format!("[ {name} ]"),
                rationale: format!(
                    "Entity <{name}> was sealed as a Rank 1 immutable invariant; declare it with '[ {name} ]' if updates
are required"
                ),
            };
            self.logger.failpoint_with_remediation(
                FailpointType::ImmutableMutation,
```

```
                "DEBUGGER",
                &msg,
                suggestion.clone(),
            );
            self.remediation_queue.push(suggestion);
            return Err(format!("FAILPOINT: {msg}"));
        }
        Ok(())
    }

    /// Retrieves and empties the active remediation queue.
    pub fn drain_remediations(&mut self) -> Vec<FixSuggestion> {
        std::mem::take(&mut self.remediation_queue)
    }

    /// Monitors retroaction steps on a named target to prevent infinite rollback loops.
    ///
    /// # Failpoints Intercepted
    /// If rollback iterations exceed `max_retroaction_depth`, traps [`FailpointType::CyclicRetroaction`].
    pub fn track_retroaction(&mut self, target: &str) -> Result<usize, String> {
        let count = self.retroaction_counters.entry(target.to_string()).or_insert(0);
        *count += 1;
        let cur = *count;

        self.logger.log(
            "VECTOR",
            format!("Retroaction step #{cur} on '{target}'"),
        );

        if cur > self.max_retroaction_depth {
            let msg = format!(
                "Cyclic retroaction loop detected! Target '{target}' rolled back {cur} times, \
                 exceeding threshold ({})",
                self.max_retroaction_depth
            );
            self.logger.failpoint(FailpointType::CyclicRetroaction, "DEBUGGER", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        Ok(cur)
    }

    /// Inspects QuadState and raises appropriate signals or failpoints.
    ///
    /// - `QuadState::Halt`: Traps void / NULL halt trap (`FP_NULL_TRAP`).
    /// - `QuadState::Rollback`: Traps rollback trigger (`FP_ROLLBACK_TRIGGERED`).
    /// - `QuadState::Await`: Standby state logged.
    /// - `QuadState::Proceed`: Commit state logged.
    pub fn inspect_state(&mut self, state: QuadState, context: &str) {
        self.step_count += 1;
        match state {
            QuadState::Halt => {
                self.logger.failpoint(
                    FailpointType::NullTrap,
                    "DEBUGGER",
                    format!("NULL / Void halt trap encountered in context: {context}"),
                );
            }
            QuadState::Rollback => {
                self.logger.failpoint(
                    FailpointType::RollbackTriggered,
                    "DEBUGGER",
                    format!("Rollback (-1) triggered in context: {context}"),
                );
            }
            QuadState::Await => {
                self.logger.log("TERNARY", format!("Await (0) standby in context: {context}"));
            }
            QuadState::Proceed => {
                self.logger.log("TERNARY", format!("Proceed (+1) committed in context: {context}"));
            }
        }
    }

    /// Executes a verification test against an isolated child debugger context.
    ///
    /// # Why Test Oracles Matter for Self-Healing
    /// Acts as an execution oracle for hypothesis verification before state commit.
```

```rust
    /// The debugger executes the candidate logic in a sandbox:
    /// - If the test passes: records an `ORACLE_SUCCESS` observation for AI memory.
    /// - If the test fails: traps a rollback failpoint and logs `ORACLE_FAILURE`.
    ///
    /// # Arguments
    /// * `test_name` - Human and AI readable identifier for the verification assertion.
    /// * `test_runner` - Closure executing the test logic against a sandboxed child debugger.
    pub fn execute_verification_test<F>(&mut self, test_name: &str, test_runner: F) -> Result<(), String>
    where
        F: FnOnce(&mut OmniDebugger) -> Result<(), String>,
    {
        self.logger.log("TEST_ORACLE", format!("Spawning verification test: {test_name}"));
        let mut test_debugger = OmniDebugger::new(self.logger.clone(), self.max_retroaction_depth);
        match test_runner(&mut test_debugger) {
            Ok(()) => {
                self.logger.log("TEST_ORACLE", format!("Verification test '{test_name}' passed (+1:PROCEED)"));
                self.logger.record_ai_observation(
                    "ORACLE_SUCCESS",
                    &format!("Test '{test_name}' passed"),
                    "Candidate state passed invariant verification without failpoints",
                    ["test_oracle", "verified", "proceed"].as_slice(),
                    Some(test_name),
                );
                Ok(())
            }
            Err(err) => {
                self.logger.failpoint(
                    FailpointType::RollbackTriggered,
                    "TEST_ORACLE",
                    format!("Verification test '{test_name}' failed: {err}"),
                );
                self.logger.record_ai_observation(
                    "ORACLE_FAILURE",
                    &format!("Test '{test_name}' rejected candidate"),
                    &err,
                    ["test_oracle", "rejected", "rollback"].as_slice(),
                    Some(test_name),
                );
                Err(err)
            }
        }
    }

    /// Verifies candidate source code with a provided test oracle function.
    ///
    /// # Arguments
    /// * `candidate_source` - The restored source string under evaluation.
    /// * `test_oracle` - Test function verifying candidate execution invariants.
    ///
    /// # Returns
    /// - `Ok(String)`: If the oracle passes, returns the confirmed candidate source.
    /// - `Err(String)`: If the oracle fails, triggers a state rollback (`-1:ROLLBACK`) and returns the error.
    pub fn verify_candidate_repair<F>(
        &mut self,
        candidate_source: &str,
        test_oracle: F,
    ) -> Result<String, String>
    where
        F: Fn(&str) -> Result<(), String>,
    {
        self.logger.log("SELF_HEAL", "Verifying candidate repair via test oracle...");
        match test_oracle(candidate_source) {
            Ok(()) => {
                self.logger.log("SELF_HEAL", "Candidate repair verified (+1:PROCEED)!");
                self.inspect_state(QuadState::Proceed, "candidate_repair_verified");
                self.logger.record_ai_observation(
                    "REPAIR_VERIFIED",
                    "Candidate source passed verification oracle",
                    "The proposed restorative patch executed cleanly without failpoints",
                    ["self_healing", "oracle_passed", "verified"].as_slice(),
                    None,
                );
                Ok(candidate_source.to_string())
            }
            Err(e) => {
                self.inspect_state(QuadState::Rollback, "candidate_repair_rejected");
                let msg = format!("Candidate repair failed test verification: {e}");
                self.logger.failpoint(FailpointType::RollbackTriggered, "SELF_HEAL", &msg);
```

```
                self.logger.record_ai_observation(
                    "REPAIR_REJECTED",
                    "Candidate source failed verification oracle",
                    &msg,
                    ["self_healing", "oracle_failed", "rollback"].as_slice(),
                    None,
                );
                Err(msg)
            }
        }
    }

    /// Ingests a structured [`LogEvent`] containing a [`FixSuggestion`], applies the patch,
    /// and invokes the verification oracle before committing the restored code.
    ///
    /// # Arguments
    /// * `source` - The original broken source code.
    /// * `log_event` - The structured incident log emitted by the [`DiagnosticLogger`].
    /// * `test_oracle` - The verification test oracle closure.
    ///
    /// # Returns
    /// `Ok(healed_source)` if the patch was applied and certified by the oracle.
    pub fn execute_restoration_from_log<F>(
        &mut self,
        source: &str,
        log_event: &LogEvent,
        test_oracle: F,
    ) -> Result<String, String>
    where
        F: Fn(&str) -> Result<(), String>,
    {
        let suggestion = log_event.suggestion.as_ref().ok_or_else(|| {
            "LogEvent does not contain remediation suggestion for automated restoration".to_string()
        })?;

        self.logger.log(
            "SELF_HEAL",
            format!("Applying remediation suggestion from log: {:?}", suggestion.kind),
        );

        let mut candidate = source.to_string();
        match suggestion.kind {
            FixKind::FixBoundaryMismatch => {
                let lines: Vec<&str> = candidate.lines().collect();
                if suggestion.line > 0 && suggestion.line <= lines.len() {
                    let mut target_line = lines[suggestion.line - 1].to_string();
                    if let Some(pos) = target_line.find(&suggestion.target_token) {
                        target_line.replace_range(pos..pos + suggestion.target_token.len(), &suggestion.suggested_patch);
                        let mut new_lines = lines.clone();
                        new_lines[suggestion.line - 1] = &target_line;
                        candidate = new_lines.join("\n");
                        if !candidate.ends_with('\n') {
                            candidate.push('\n');
                        }
                    }
                }
            }
            FixKind::CloseBoundary => {
                candidate.push_str(&suggestion.suggested_patch);
                if !candidate.ends_with('\n') {
                    candidate.push('\n');
                }
            }
            FixKind::ConvertImmutableToMutable => {
                candidate = candidate.replace(
                    &format!("< {} >", suggestion.target_token),
                    &suggestion.suggested_patch,
                );
            }
            _ => {
                candidate = candidate.replace(&suggestion.target_token, &suggestion.suggested_patch);
            }
        }

        // Call the test oracle to certify candidate
        self.verify_candidate_repair(&candidate, test_oracle)
    }
}
```

```
463
464
465 // ================================================================================
466 // BLOCK 4: CLOSING BLOCK
467 // ( Postcondition Verification and Debugger Status )
468 // ================================================================================
469 // OmniDebugger safety invariants verified and closed.
470
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/hover.rs RS 989 LINES 3470 WORDS 39.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Hover Subsystem Identity, 3-Tier Hover Architecture & Docstring Invariants )
// ============================================================================
//! OmniCode Diagnostics: Cross-File Docstring & Hover Engine
//!
//! # Architecture & Cross-Scale Transparency
//! In OmniCode, programming operates simultaneously across two complementary scales:
//! - **High-Level Process Ribbons (`.tape`):** Sequential SVO sentences and rolls
//!   executing across scheduler tracks with teleological intent (`WHY`).
//! - **Low-Level Source Reels (`.omni`):** Declarative 4-Block architecture defining
//!   invariant boundaries (`<...>`), memory storehouses, and physical cells.
//!
//! # Cross-File Symbol & Invariant Inheritance (.omni -> .tape)
//! When an engineer or AI agent hovers over a symbol inside a `.tape`:
//! 1. The symbol is resolved against its active roll context (Intent Aside & Tape Head).
//! 2. The underlying low-level definition is retrieved from the companion `.omni` file
//!    (or standard library / manifest), exposing its morphological rank, invariant
//!    contracts, and block origin.
//! 3. The machine execution footprint is synthesized (memory offset, storehouse partition,
//!    8-bit to 5-trit Radix-3 transduced codec, and QuadState transaction status).
//!
//! This delivers total under-the-hood transparency: developers write expressive
//! sentences while directly inspecting what the machine is physically doing in silicon.

use std::collections::HashMap;
use std::fs;
use std::path::Path;

use crate::ast::nodes::{ASTNode, EntityNode};
use crate::core::boundaries::BoundaryType;
use crate::core::tape::Tape;
use crate::diagnostics::linter::BlockKind;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Data Models: OmniDefinition, TapeRollContext, HoverCard & HoverEngine )
// ============================================================================

/// Ontological definition extracted from a low-level `.omni` source reel.
#[derive(Debug, Clone, PartialEq)]
pub struct OmniDefinition {
    /// Identifier name (e.g., "Manifest", "Version", "Law_Truth", "Storehouse").
    pub name: String,
    /// Morphological boundary type (Rank 1 Invariant, Rank 2 Denotation, etc.).
    pub boundary_type: BoundaryType,
    /// Ontological rank tier (1 to 6).
    pub rank: u8,
    /// Primary grammatical part of speech.
    pub part_of_speech: &'static str,
    /// First-class machine intent Aside `(...)` declared on this definition.
    pub aside_intent: Option<String>,
    /// Structural doc comments (`#`) associated with this entity.
    pub doc_comments: Vec<String>,
    /// Originating block in the Universal 4-Block Architecture.
    pub block_kind: Option<BlockKind>,
    /// Source file path where this symbol is originally grounded.
    pub source_file: String,
    /// Line number of the definition.
    pub line: usize,
    /// Inner subfields or attributes (e.g. `commandment`, `principle`, `resonance`).
    pub fields: HashMap<String, String>,
    /// Machine execution / low-level silicon representation description.
    pub machine_representation: Option<String>,
}

/// Operational roll context extracted from a high-level `.tape` process ribbon.
#[derive(Debug, Clone, PartialEq)]
pub struct TapeRollContext {
    /// Process name declared in the leading Tape Head covenant.
```

```
    pub tape_name: String,
    /// Target execution channel (Track 0 [KERNEL], Track 1 [TERMINAL], Track 2 [APP]).
    pub track: String,
    /// 1-indexed roll position on the tape.
    pub roll_number: usize,
    /// Total rolls on the active tape ribbon.
    pub total_rolls: usize,
    /// Raw text content of the active roll.
    pub raw_roll_text: String,
    /// Machine intent Aside `(...)` attached to this specific roll.
    pub roll_aside_intent: Option<String>,
    /// Cardinal clause: WHO (Security Principal / Caller).
    pub head_who: String,
    /// Cardinal clause: WHAT (Target entity / payload).
    pub head_what: String,
    /// Cardinal clause: WHY (Teleological covenant intent).
    pub head_why: String,
    /// Cardinal clause: HOW (Directional vector / QuadState modulation).
    pub head_how: String,
    /// Spatial allocation requirement in memory cells.
    pub partition_size: usize,
}

/// Complete 3-tier composite hover documentation card.
#[derive(Debug, Clone, PartialEq)]
pub struct HoverCard {
    /// Symbol name inspected.
    pub symbol: String,
    /// High-level process roll context (present if hovered inside a `.tape`).
    pub tape_context: Option<TapeRollContext>,
    /// Low-level `.omni` blueprint and invariant contract.
    pub omni_definition: Option<OmniDefinition>,
    /// Whether the symbol was redefined locally in the active tape ribbon.
    pub is_locally_redefined: bool,
    /// Rendered GitHub-flavored Markdown text for IDE tooltips and CLI inspection.
    pub markdown: String,
}

impl HoverCard {
    /// Serializes this HoverCard to a standard JSON string without third-party dependencies.
    pub fn to_json(&self) -> String {
        let tape_ctx_json = match &self.tape_context {
            Some(tc) => {
                let aside_str = match &tc.roll_aside_intent {
                    Some(a) => format!("\"{}\"", escape_json_str(a)),
                    None => "null".to_string(),
                };
                format!(
                    r#"{{"tape_name":"{}","track":"{}","roll_number":{},"total_rolls":{},"raw_roll_text":"
{}","roll_aside_intent":{},"head_who":"{}","head_what":"{}","head_why":"{}","head_how":"{}","partition_size":{}}}"#,
                    escape_json_str(&tc.tape_name),
                    escape_json_str(&tc.track),
                    tc.roll_number,
                    tc.total_rolls,
                    escape_json_str(&tc.raw_roll_text),
                    aside_str,
                    escape_json_str(&tc.head_who),
                    escape_json_str(&tc.head_what),
                    escape_json_str(&tc.head_why),
                    escape_json_str(&tc.head_how),
                    tc.partition_size,
                )
            }
            None => "null".to_string(),
        };

        let omni_def_json = match &self.omni_definition {
            Some(od) => {
                let aside_str = match &od.aside_intent {
                    Some(a) => format!("\"{}\"", escape_json_str(a)),
                    None => "null".to_string(),
                };
                let block_str = match od.block_kind {
                    Some(b) => format!("\"{}\"", b.name()),
                    None => "null".to_string(),
                };
                let doc_comments_arr = od
                    .doc_comments
```

```
                    .iter()
                    .map(|c| format!("\"{}\"", escape_json_str(c)))
                    .collect::<Vec<_>>()
                    .join(",");
                let fields_obj = od
                    .fields
                    .iter()
                    .map(|(k, v)| format!("\"{}\":\"{}\"", escape_json_str(k), escape_json_str(v)))
                    .collect::<Vec<_>>()
                    .join(",");

                format!(
                    r#"{{"name":"{}","boundary_type":"{}","rank":{},"part_of_speech":"{}","aside_intent":
{},"doc_comments":[{}],"block_kind":{},"source_file":"{}","line":{},"fields":{{{}}}}}"#,
                    escape_json_str(&od.name),
                    escape_json_str(&od.boundary_type.to_string()),
                    od.rank,
                    escape_json_str(od.part_of_speech),
                    aside_str,
                    doc_comments_arr,
                    block_str,
                    escape_json_str(&od.source_file),
                    od.line,
                    fields_obj,
                )
            }
            None => "null".to_string(),
        };

        format!(
            r#"{{"symbol":"{}","is_locally_redefined":{},"tape_context":{},"omni_definition":{},"markdown":"{}"}}"#,
            escape_json_str(&self.symbol),
            self.is_locally_redefined,
            tape_ctx_json,
            omni_def_json,
            escape_json_str(&self.markdown),
        )
    }

    /// Renders an ANSI-styled human terminal presentation of the 3-tier hover card.
    pub fn render_terminal(&self) -> String {
        let mut out = String::new();

out.push_str("\x1b[1;36m================================================================================\x1b[0m\n");
        let rank_desc = if let Some(def) = &self.omni_definition {
            def.boundary_type.rank_name()
        } else if self.is_locally_redefined {
            "Rank 2: Denotative Essence [ ... ]"
        } else {
            "Process Tape Entity"
        };
        out.push_str(&format!(
            "\x1b[1;36m  OmniCode Symbol Inspection: \x1b[1;32m{}\x1b[0m \x1b[1;33m({})\x1b[0m\n",
            self.symbol, rank_desc
        ));

out.push_str("\x1b[1;36m================================================================================\x1b[0m\n\n");

        if let Some(ctx) = &self.tape_context {
            out.push_str("\x1b[1;33m[TIER 1: HIGH-LEVEL PROCESS TAPE ROLL]\x1b[0m\n");
            out.push_str(&format!("  • Process Ribbon:    \x1b[1;37m{}\x1b[0m ({})\n", ctx.tape_name, ctx.track));
            out.push_str(&format!("  • Active Roll:       \x1b[1;37m{}/{}\x1b[0m -> `{}`\n", ctx.roll_number,
ctx.total_rolls, ctx.raw_roll_text));
            if let Some(aside) = &ctx.roll_aside_intent {
                out.push_str(&format!("  • Roll Intent Aside: \x1b[3m\"{}\"\x1b[0m\n", aside));
            }
            out.push_str(&format!("  • Covenant Head:     WHO {} -> WHAT {} (WHY: \"{}\")\n", ctx.head_who,
ctx.head_what, ctx.head_why));
            if self.is_locally_redefined {
                out.push_str("  • \x1b[1;32m[Locally Instantiated]\x1b[0m Concrete Denotation assigned on this
ribbon.\n");
            }
            out.push('\n');
        }

        out.push_str("\x1b[1;32m[TIER 2: UNDER-THE-HOOD .OMNI BLUEPRINT]\x1b[0m\n");
        if let Some(def) = &self.omni_definition {
            let block = def.block_kind.map(|b| b.name()).unwrap_or("Declarative Scope");
```

```
            out.push_str(&format!("  • Grounded Source:   \x1b[1;37m{}:{}\x1b[0m\n", def.source_file, def.line));
            out.push_str(&format!("  • 4-Block Origin:    \x1b[1;35m{}\x1b[0m\n", block));
            out.push_str(&format!("  • Ontological Rank:  Rank {} ({})\n", def.rank, def.boundary_type));
            if let Some(aside) = &def.aside_intent {
                out.push_str(&format!("  • Contract Intent:   \x1b[3m\"{}\"\x1b[0m\n", aside));
            }
            if !def.doc_comments.is_empty() {
                out.push_str("  • Doc Comments:\n");
                for c in &def.doc_comments {
                    out.push_str(&format!("      # {}\n", c));
                }
            }
            if !def.fields.is_empty() {
                out.push_str("  • Axiomatic Fields:\n");
                for (k, v) in &def.fields {
                    out.push_str(&format!("      - {}: {}\n", k, v));
                }
            }
        } else {
            out.push_str("  • Status: Runtime dynamic process symbol; inherits ambient defaults.\n");
        }
        out.push('\n');

        out.push_str("\x1b[1;35m[TIER 3: MACHINE EXECUTION & SILICON FOOTPRINT]\x1b[0m\n");
        let partition = self.tape_context.as_ref().map(|c| format!("{} cells", c.partition_size)).unwrap_or_else(|| "1024
cells".to_string());
        out.push_str(&format!("  • Partition Reservation: \x1b[1;37m{}\x1b[0m\n", partition));
        out.push_str("  • Transducer Codec:      Dense Byte <-> 5 Balanced Trits (Radix-3 [-121..+121])\n");
        out.push_str("  • Silicon Truth State:   \x1b[1;32m+1:PROCEED\x1b[0m (Deterministic commit, transactional
rollback)\n");

        out
    }
}

/// Cross-file symbol indexer and hover resolution engine.
pub struct HoverEngine {
    pub logger: DiagnosticLogger,
    /// Indexed definitions mapped by identifier name (case-sensitive and lowercase).
    definitions: HashMap<String, OmniDefinition>,
    /// Map of file path -> list of symbol names defined in that file.
    file_symbols: HashMap<String, Vec<String>>,
}

impl HoverEngine {
    /// Instantiates a new HoverEngine with diagnostic logging.
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self {
            logger,
            definitions: HashMap::new(),
            file_symbols: HashMap::new(),
        }
    }

    /// Looks up an indexed definition by name.
    pub fn lookup_definition(&self, symbol: &str) -> Option<&OmniDefinition> {
        self.definitions.get(symbol).or_else(|| self.definitions.get(&symbol.to_lowercase()))
    }

    /// Resolves documentation for a symbol, optionally indexing a target file first.
    pub fn document_symbol(&mut self, symbol: &str, file_hint: Option<&str>) -> Option<HoverCard> {
        if let Some(path) = file_hint {
            if path.ends_with(".tape") {
                return self.resolve_symbol_for_tape(path, symbol, 0);
            } else {
                let _ = self.index_file(path);
            }
        }

        let def = self.lookup_definition(symbol).cloned();
        let markdown = self.format_omni_only_markdown(symbol, def.as_ref());
        Some(HoverCard {
            symbol: symbol.to_string(),
            tape_context: None,
            omni_definition: def,
            is_locally_redefined: false,
            markdown,
        })
```

```
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Indexing Passes, Cross-File Symbol Inheritance & Markdown Hover Generation )
// ================================================================================

impl HoverEngine {
    /// Indexes a `.omni` source reel string, extracting all entities, boundaries,
    /// docstrings, attached Asides, and block origins.
    pub fn index_omni_source(&mut self, file_path: &str, content: &str) -> Result<usize, String> {
        let lines: Vec<&str> = content.lines().collect();
        let mut current_block = None;
        let mut count = 0;

        // Pass 1: Parse block layout
        let mut line_blocks = vec![None; lines.len()];
        for (idx, line) in lines.iter().enumerate() {
            let upper = line.to_uppercase();
            if upper.contains("BLOCK 1") || upper.contains("METADATA BLOCK") {
                current_block = Some(BlockKind::Metadata);
            } else if upper.contains("BLOCK 2") || upper.contains("OPENING BLOCK") {
                current_block = Some(BlockKind::Opening);
            } else if upper.contains("BLOCK 3") || upper.contains("BODY BLOCK") {
                current_block = Some(BlockKind::Body);
            } else if upper.contains("BLOCK 4") || upper.contains("CLOSING BLOCK") {
                current_block = Some(BlockKind::Closing);
            }
            line_blocks[idx] = current_block;
        }

        // Pass 2: Tokenize and build AST to extract formal Entities
        let mut lexer = Lexer::new(content, self.logger.clone());
        let tokens = lexer.tokenize()?;
        let debugger = crate::diagnostics::debugger::OmniDebugger::new(self.logger.clone(), 10);
        let mut parser = Parser::new(tokens, debugger);

        // Best-effort parse program
        if let Ok(prog) = parser.parse_program() {
            for stmt in &prog.statements {
                if let ASTNode::Entity(entity) = stmt {
                    let def = self.build_omni_definition(entity, file_path, &lines, &line_blocks);
                    let name = def.name.clone();
                    self.insert_definition(name.clone(), def);
                    self.file_symbols
                        .entry(file_path.to_string())
                        .or_default()
                        .push(name);
                    count += 1;
                }
            }
        }

        // Pass 3: Textual line-by-line fallback for inner fields and registers
        for (idx, line) in lines.iter().enumerate() {
            let trimmed = line.trim();
            if trimmed.starts_with('#') || trimmed.is_empty() {
                continue;
            }

            // Detect field patterns: `name: < ... > ( ... )` or `name [ ... ] ( ... )`
            if let Some(colon_pos) = trimmed.find(':') {
                let id_candidate = trimmed[..colon_pos].trim();
                if is_valid_identifier(id_candidate) && !self.definitions.contains_key(id_candidate) {
                    let rest = trimmed[colon_pos + 1..].trim();
                    let btype = detect_boundary_type_from_str(rest);
                    let aside = extract_aside_from_str(rest);
                    let doc_comments = extract_preceding_comments(&lines, idx);

                    let def = OmniDefinition {
                        name: id_candidate.to_string(),
                        boundary_type: btype,
                        rank: btype.rank(),
                        part_of_speech: "Noun",
                        aside_intent: aside,
                        doc_comments,
```

```
                        block_kind: line_blocks[idx],
                        source_file: file_path.to_string(),
                        line: idx + 1,
                        fields: HashMap::new(),
                        machine_representation: Some("Contextual Attribute / Field".into()),
                    };
                    self.insert_definition(id_candidate.to_string(), def);
                    count += 1;
                }
            }

            // Detect standalone registers: `name [ ... ] ( ... )`
            if trimmed.contains('[') && !trimmed.starts_with("roll_") && !trimmed.starts_with("verify:") {
                let id_end = trimmed.find('[').unwrap_or(0);
                let id_candidate = trimmed[..id_end].trim();
                if is_valid_identifier(id_candidate) && !self.definitions.contains_key(id_candidate) {
                    let aside = extract_aside_from_str(trimmed);
                    let doc_comments = extract_preceding_comments(&lines, idx);
                    let def = OmniDefinition {
                        name: id_candidate.to_string(),
                        boundary_type: BoundaryType::Denotation,
                        rank: BoundaryType::Denotation.rank(),
                        part_of_speech: "Noun",
                        aside_intent: aside,
                        doc_comments,
                        block_kind: line_blocks[idx],
                        source_file: file_path.to_string(),
                        line: idx + 1,
                        fields: HashMap::new(),
                        machine_representation: Some("Active State Register".into()),
                    };
                    self.insert_definition(id_candidate.to_string(), def);
                    count += 1;
                }
            }
        }

        self.logger.log(
            "HOVER",
            format!("Indexed {} symbols from .omni source: {}", count, file_path),
        );
        Ok(count)
    }

    /// Indexes a specific file from disk.
    pub fn index_file(&mut self, path: impl AsRef<Path>) -> Result<usize, String> {
        let p = path.as_ref();
        let content = fs::read_to_string(p)
            .map_err(|e| format!("Failed to read file {:?}: {e}", p))?;
        let path_str = p.to_string_lossy().to_string();
        self.index_omni_source(&path_str, &content)
    }

    /// Recursively indexes foundational files and standard libraries across the workspace.
    pub fn index_workspace(&mut self, root_dir: impl AsRef<Path>) {
        let root = root_dir.as_ref();

        // 1. Core manifest
        let manifest_path = root.join("manifest.omni");
        if manifest_path.exists() {
            let _ = self.index_file(&manifest_path);
        }

        // 2. Standard library modules in lib/core/
        let lib_core = root.join("lib/core");
        if lib_core.exists() && lib_core.is_dir() {
            if let Ok(entries) = fs::read_dir(lib_core) {
                for entry in entries.flatten() {
                    let p = entry.path();
                    if p.extension().map_or(false, |ext| ext == "omni") {
                        let _ = self.index_file(&p);
                    }
                }
            }
        }
    }

    /// Helper to store definitions in both exact and lowercase keys for flexible lookup.
```

```
    fn insert_definition(&mut self, key: String, def: OmniDefinition) {
        self.definitions.insert(key.to_lowercase(), def.clone());
        self.definitions.insert(key, def);
    }

    /// Resolves hover information for a symbol used inside a high-level `.tape` process ribbon.
    ///
    /// # Resolution Sequence
    /// 1. Check if defined or redefined in the local `.tape` ribbon.
    /// 2. If companion `.omni` exists (e.g. `foo.tape` -> `foo.omni`), resolve from companion.
    /// 3. Otherwise, resolve from the workspace symbol index (stdlib or manifest).
    /// 4. Generate the composite 3-tier Markdown HoverCard.
    pub fn resolve_symbol_for_tape(
        &mut self,
        tape_path: &str,
        symbol: &str,
        line_number: usize,
    ) -> Option<HoverCard> {
        let tape_p = Path::new(tape_path);

        // Step 1: Ensure companion .omni is indexed if it exists
        let companion_omni = tape_p.with_extension("omni");
        if companion_omni.exists() {
            let _ = self.index_file(&companion_omni);
        }

        // Step 2: Load and inspect the active Tape
        let tape = Tape::from_file(tape_p, self.logger.clone()).ok();

        // Extract Tape Context for this roll
        let mut tape_context = None;
        let mut is_locally_redefined = false;

        if let Some(ref t) = tape {
            let track_str = if tape_path.contains("kernel") || tape_path.contains("boot") {
                "Track 0 [KERNEL]"
            } else if tape_path.contains("terminal") {
                "Track 1 [TERMINAL]"
            } else {
                "Track 2 [APP]"
            };

            // Find roll matching line number that contains symbol, or matching line number, or first roll referencing
symbol
            let matched_roll = t
                .rolls
                .iter()
                .find(|r| r.line_number == line_number && r.text.contains(symbol))
                .or_else(|| t.rolls.iter().find(|r| r.line_number == line_number))
                .or_else(|| t.rolls.iter().find(|r| r.text.contains(symbol)));

            if let Some(r) = matched_roll {
                let roll_aside = extract_aside_from_str(&r.text);

                tape_context = Some(TapeRollContext {
                    tape_name: t.head.name.clone(),
                    track: track_str.to_string(),
                    roll_number: r.index + 1,
                    total_rolls: t.roll_count(),
                    raw_roll_text: r.text.clone(),
                    roll_aside_intent: roll_aside,
                    head_who: t.head.who.clone(),
                    head_what: t.head.what.clone(),
                    head_why: t.head.why.clone(),
                    head_how: t.head.how.clone(),
                    partition_size: t.head.required_partition_size,
                });
            }

            // Check if symbol was given a Denotation anywhere on this tape ribbon: `symbol [ ... ]`
            is_locally_redefined = t.rolls.iter().any(|r| {
                r.text.contains(&format!("{symbol} [")) || r.text.contains(&format!("{symbol}["))
            });
        }

        // Step 3: Lookup low-level .omni definition
        let omni_def = self
            .definitions
```

```
            .get(symbol)
            .cloned()
            .or_else(|| self.definitions.get(&symbol.to_lowercase()).cloned());

        // Build Markdown card
        let markdown = self.format_composite_markdown(
            symbol,
            tape_context.as_ref(),
            omni_def.as_ref(),
            is_locally_redefined,
        );

        Some(HoverCard {
            symbol: symbol.to_string(),
            tape_context,
            omni_definition: omni_def,
            is_locally_redefined,
            markdown,
        })
    }

    /// Resolves hover information at a specific file coordinate `(line, col)`.
    pub fn hover_at(&mut self, file_path: &str, line: usize, col: usize) -> Option<HoverCard> {
        let content = fs::read_to_string(file_path).ok()?;
        let lines: Vec<&str> = content.lines().collect();
        if line == 0 || line > lines.len() {
            return None;
        }

        let target_line = lines[line - 1];
        let symbol = extract_word_at_col(target_line, col)?;

        if file_path.ends_with(".tape") {
            self.resolve_symbol_for_tape(file_path, &symbol, line)
        } else {
            // Index the file itself if not already done
            let _ = self.index_omni_source(file_path, &content);
            let def = self
                .definitions
                .get(&symbol)
                .cloned()
                .or_else(|| self.definitions.get(&symbol.to_lowercase()).cloned());

            let markdown = self.format_omni_only_markdown(&symbol, def.as_ref());
            Some(HoverCard {
                symbol: symbol.clone(),
                tape_context: None,
                omni_definition: def,
                is_locally_redefined: false,
                markdown,
            })
        }
    }

    /// Formats the complete 3-Tier Markdown HoverCard bridging .tape down to .omni.
    fn format_composite_markdown(
        &self,
        symbol: &str,
        tape_ctx: Option<&TapeRollContext>,
        omni_def: Option<&OmniDefinition>,
        is_redefined: bool,
    ) -> String {
        let mut md = String::new();

        // ------------------------------------------------------------------
        // HEADER: Identity & Morphological Badging
        // ------------------------------------------------------------------
        let rank_badge = if let Some(def) = omni_def {
            format!("`{}`", def.boundary_type.rank_name())
        } else if is_redefined {
            "`Rank 2: Denotative Essence [ ... ]`".to_string()
        } else {
            "`Process Tape Entity`".to_string()
        };

        md.push_str(&format!("### `{symbol}` — {}\n\n", rank_badge));

        // ------------------------------------------------------------------
```

```
        // TIER 1: HIGH-LEVEL PROCESS TAPE ROLL (Process Intent & Active Sentence)
        // ------------------------------------------------------------------------
        if let Some(ctx) = tape_ctx {
            md.push_str("#### 📜 1. High-Level Process Tape View\n");
            md.push_str(&format!(
                "* **Process Ribbon:** `{}` on `{}`\n",
                ctx.tape_name, ctx.track
            ));
            md.push_str(&format!(
                "* **Active Roll ({}/{}):** `{}`\n",
                ctx.roll_number, ctx.total_rolls, ctx.raw_roll_text
            ));

            if let Some(ref intent) = ctx.roll_aside_intent {
                md.push_str(&format!("* **Roll Intent Aside:** *\"{}\"*\n", intent));
            }

            md.push_str(&format!(
                "* **Tape Head Covenant:** WHO `{}` $\\to$ WHAT `{}` (Purpose: *\"{}\"*)\n",
                ctx.head_who, ctx.head_what, ctx.head_why
            ));

            if is_redefined {
                md.push_str(
                    "\n> [!NOTE]\n> **Locally Defined:** This entity receives a concrete Denotation `[ ... ]` in this tape ribbon, instantiating it into active memory.\n",
                );
            }
            md.push_str("\n---\n\n");
        }

        // ------------------------------------------------------------------------
        // TIER 2: UNDER-THE-HOOD LOW-LEVEL BLUEPRINT (.omni Blueprint & Contract)
        // ------------------------------------------------------------------------
        md.push_str("#### 🔍 2. Under-The-Hood (.omni Blueprint)\n");
        if let Some(def) = omni_def {
            let block_name = def
                .block_kind
                .map(|b| b.name())
                .unwrap_or("Declarative Scope");
            md.push_str(&format!(
                "* **Grounded Source:** [`{}`]({}) (Line {})\n",
                def.source_file, def.source_file, def.line
            ));
            md.push_str(&format!("* **4-Block Origin:** `{block_name}`\n"));
            md.push_str(&format!(
                "* **Ontological Hierarchy:** Rank {} (`{}`)\n",
                def.rank, def.boundary_type
            ));

            if let Some(ref intent) = def.aside_intent {
                md.push_str(&format!("* **Contract Intent Aside:** *\"{}\"*\n", intent));
            }

            if !def.doc_comments.is_empty() {
                md.push_str("* **Architectural Description:**\n");
                for comment in &def.doc_comments {
                    md.push_str(&format!("  > {comment}\n"));
                }
            }

            if !def.fields.is_empty() {
                md.push_str("* **Sub-Attributes & Invariant Axioms:**\n");
                for (k, v) in &def.fields {
                    md.push_str(&format!("  * `{k}`: `{v}`\n"));
                }
            }
        } else {
            md.push_str("* **Under-The-Hood Status:** Pure runtime process entity; inherits standard library defaults.\n");
        }
        md.push_str("\n---\n\n");

        // ------------------------------------------------------------------------
        // TIER 3: MACHINE EXECUTION & SILICON EFFECT
        // ------------------------------------------------------------------------
        md.push_str("#### ⚙️ 3. Machine Execution & Silicon State\n");
        let partition = tape_ctx
```

```
            .map(|c| format!("{} cells", c.partition_size))
            .unwrap_or_else(|| "1024 cells".to_string());

        md.push_str(&format!(
            "* **Memory Allocation:** Partition reservation ceiling: `{partition}`\n"
        ));
        md.push_str("* **Transducer Codec:** Dense byte $\\longleftrightarrow$ 5 balanced trits ($[-121 \\dots +121]$ Radix-3)\n");
        md.push_str("* **Execution Truth State:** `+1: PROCEED` (Deterministic commit, transactional rollback protection)\n");

        md
    }

    /// Formats a hover card when hovering directly inside a `.omni` source reel.
    fn format_omni_only_markdown(&self, symbol: &str, def: Option<&OmniDefinition>) -> String {
        let mut md = String::new();
        if let Some(d) = def {
            let block_name = d.block_kind.map(|b| b.name()).unwrap_or("Universal Scope");
            md.push_str(&format!(
                "### `{}` — `{}`\n\n",
                d.name,
                d.boundary_type.rank_name()
            ));
            md.push_str(&format!("* **Origin:** `{block_name}` (Line {})\n", d.line));
            md.push_str(&format!("* **Part of Speech:** `{}`\n", d.part_of_speech));

            if let Some(ref intent) = d.aside_intent {
                md.push_str(&format!("* **Machine Intent Aside:** *\"{}\"*\n", intent));
            }

            if !d.doc_comments.is_empty() {
                md.push_str("\n#### Architectural Docstring:\n");
                for c in &d.doc_comments {
                    md.push_str(&format!("> {c}\n"));
                }
            }

            if !d.fields.is_empty() {
                md.push_str("\n#### Invariant Fields:\n");
                for (k, v) in &d.fields {
                    md.push_str(&format!("* `{k}`: `{v}`\n"));
                }
            }
        } else {
            md.push_str(&format!("### `{symbol}`\n\n"));
            md.push_str("* **Classification:** OmniCode Identifier\n");
            md.push_str("* **Scope:** Ambient Context\n");
        }
        md
    }

    /// Internal helper converting an AST EntityNode into an OmniDefinition.
    fn build_omni_definition(
        &self,
        entity: &EntityNode,
        file_path: &str,
        lines: &[&str],
        line_blocks: &[Option<BlockKind>],
    ) -> OmniDefinition {
        let line_idx = entity.line.saturating_sub(1);
        let block_kind = line_blocks.get(line_idx).copied().flatten();
        let doc_comments = extract_preceding_comments(lines, line_idx);
        let btype = entity.definition.boundary_type;

        let mut fields = HashMap::new();
        // Extract subfields if this was a record with key-value entries
        if line_idx < lines.len() {
            for l in &lines[line_idx..line_idx.saturating_add(15).min(lines.len())] {
                let trimmed = l.trim();
                if trimmed.starts_with('}') || trimmed.starts_with(']') {
                    break;
                }
                if let Some(colon) = trimmed.find(':') {
                    let k = trimmed[..colon].trim();
                    let v = trimmed[colon + 1..].trim().trim_end_matches(',');
                    if is_valid_identifier(k) {
                        fields.insert(k.to_string(), v.to_string());
```

```
                    }
                }
            }
        }

        let aside_intent = entity
            .aside
            .as_ref()
            .or(entity.definition.aside.as_ref())
            .or(entity.context.as_ref().and_then(|c| c.aside.as_ref()))
            .map(|a| a.intent.clone())
            .or_else(|| {
                if line_idx < lines.len() {
                    extract_aside_from_str(lines[line_idx])
                } else {
                    None
                }
            });

        OmniDefinition {
            name: entity.name.clone(),
            boundary_type: btype,
            rank: btype.rank(),
            part_of_speech: "Noun",
            aside_intent,
            doc_comments,
            block_kind,
            source_file: file_path.to_string(),
            line: entity.line,
            fields,
            machine_representation: Some(format!("Ontological Essence ({})", btype)),
        }
    }
}


// ------------------------------------------------------------------------------
// HELPER UTILITIES: Token extraction, comments, and delimiters
// ------------------------------------------------------------------------------

fn escape_json_str(s: &str) -> String {
    let mut out = String::with_capacity(s.len());
    for c in s.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            c => out.push(c),
        }
    }
    out
}

fn is_valid_identifier(s: &str) -> bool {
    if s.is_empty() {
        return false;
    }
    let first = s.chars().next().unwrap();
    (first.is_alphabetic() || first == '_') && s.chars().all(|c| c.is_alphanumeric() || c == '_')
}

fn extract_word_at_col(line: &str, col: usize) -> Option<String> {
    if line.is_empty() {
        return None;
    }
    let chars: Vec<char> = line.chars().collect();
    let idx = col.saturating_sub(1).min(chars.len().saturating_sub(1));

    if !chars[idx].is_alphanumeric() && chars[idx] != '_' {
        return None;
    }

    let mut start = idx;
    while start > 0 && (chars[start - 1].is_alphanumeric() || chars[start - 1] == '_') {
        start -= 1;
    }
```

```
    let mut end = idx;
    while end + 1 < chars.len() && (chars[end + 1].is_alphanumeric() || chars[end + 1] == '_') {
        end += 1;
    }

    Some(chars[start..=end].iter().collect())
}

fn extract_aside_from_str(text: &str) -> Option<String> {
    let start = text.find('(')?;
    let end = text.rfind(')')?;
    if end > start {
        Some(text[start + 1..end].trim().to_string())
    } else {
        None
    }
}

fn detect_boundary_type_from_str(s: &str) -> BoundaryType {
    if s.contains('<') && s.contains('>') {
        BoundaryType::Invariant
    } else if s.contains('{') && s.contains('}') {
        BoundaryType::Connotation
    } else if s.contains('{') && s.contains(']') {
        BoundaryType::TopDown
    } else if s.contains('[') && s.contains('}') {
        BoundaryType::BottomUp
    } else {
        BoundaryType::Denotation
    }
}

fn extract_preceding_comments(lines: &[&str], line_idx: usize) -> Vec<String> {
    let mut comments = Vec::new();
    let mut cur = line_idx;
    while cur > 0 {
        cur -= 1;
        let trimmed = lines[cur].trim();
        if trimmed.starts_with('#') {
            let clean = trimmed.trim_start_matches('#').trim();
            // Ignore boundary separator bars: ====== or ------
            if !clean.starts_with('=') && !clean.starts_with('-') && !clean.is_empty() {
                comments.push(clean.to_string());
            }
        } else {
            break;
        }
    }
    comments.reverse();
    comments
}


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests, Verification & Subsystem Sealing )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_hover_engine_manifest_omni_indexing() {
        let logger = DiagnosticLogger::new(false);
        let mut engine = HoverEngine::new(logger);

        let manifest_content = r#"
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
# LAW 1: TRUTH & FIDELITY
# Unalterable prohibition against deceit.
Law_Truth < Invariant > ( Invariant Contract: Prohibit deception ) {
    commandment: < "Thou shalt not bear false witness." >,
    resonance: < +1:PROCEED >
}
```

```
# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
covenant_active [ +1 : State? ] ( Active State Register )
        "#;

        let count = engine
            .index_omni_source("manifest.omni", manifest_content)
            .expect("Indexing must succeed");

        assert!(count >= 2);

        // Verify Law_Truth extraction
        let law_def = engine.definitions.get("Law_Truth").expect("Must index Law_Truth");
        assert_eq!(law_def.boundary_type, BoundaryType::Invariant);
        assert_eq!(law_def.rank, 1);
        assert_eq!(law_def.block_kind, Some(BlockKind::Metadata));
        assert!(law_def.aside_intent.as_ref().unwrap().contains("Prohibit deception"));
        assert!(law_def.doc_comments.iter().any(|c| c.contains("TRUTH & FIDELITY")));
    }

    #[test]
    fn test_cross_file_tape_to_omni_hover_resolution() {
        let logger = DiagnosticLogger::new(false);
        let mut engine = HoverEngine::new(logger);

        // Index low-level omni
        let omni_content = r#"
# BLOCK 1: METADATA
# System version invariant
Version < "1.0.0" > ( Baseline release )
        "#;
        engine.index_omni_source("test.omni", omni_content).unwrap();

        // Synthetic tape source
        let tape_content = r#"
[Tape.TestProcess] : WHO {Kernel} WHAT {Test} --> BOOT {Hardware}
    WHEN {Boot}
    WHERE {partition} = [ 256 ]
    WHY ( Test ribbon )
    HOW [ +1 : PROCEED ]

roll_1 [ Version [ "1.0.0" ] ] ( Concrete definition in memory )
roll_2 [ commit_status [ PROCEED ] ]
        "#;

        // Temporarily write tape file to test directory
        let temp_dir = std::env::temp_dir();
        let tape_path = temp_dir.join("test.tape");
        let omni_path = temp_dir.join("test.omni");
        fs::write(&tape_path, tape_content).unwrap();
        fs::write(&omni_path, omni_content).unwrap();

        let card = engine
            .resolve_symbol_for_tape(tape_path.to_str().unwrap(), "Version", 9)
            .expect("Must resolve hover card");

        assert_eq!(card.symbol, "Version");
        assert!(card.is_locally_redefined);
        assert!(card.tape_context.is_some());
        assert!(card.markdown.contains("High-Level Process Tape View"));
        assert!(card.markdown.contains("Under-The-Hood (.omni Blueprint)"));
        assert!(card.markdown.contains("Machine Execution & Silicon State"));

        // Clean up temp files
        fs::remove_file(tape_path).ok();
        fs::remove_file(omni_path).ok();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter.rs RS 391 LINES 1830 WORDS 16.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniLinter Identity, BlockKind Enumeration & Structural Invariants )
// ============================================================================
//! OmniCode Diagnostics: Structural 4-Block Linter & Auto-Organizer
//!
//! Enforces the Universal 4-Block Architecture across all files:
//!     - BLOCK 1: METADATA BLOCK  (Identity, Version, Invariant Axioms, Asides)
//!     - BLOCK 2: OPENING BLOCK   (Context, Preconditions, State Initialization)
//!     - BLOCK 3: BODY BLOCK      (Core Transformations, Causality Vectors, Logic)
//!     - BLOCK 4: CLOSING BLOCK   (Verification, Postconditions, Teardown, Commit)

use std::path::Path;
use crate::core::filetype::OmniFileType;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum BlockKind {
    Metadata = 1,
    Opening  = 2,
    Body     = 3,
    Closing  = 4,
}

impl BlockKind {
    pub fn name(&self) -> &'static str {
        match self {
            BlockKind::Metadata => "BLOCK 1: METADATA",
            BlockKind::Opening => "BLOCK 2: OPENING",
            BlockKind::Body => "BLOCK 3: BODY",
            BlockKind::Closing => "BLOCK 4: CLOSING",
        }
    }
}


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Report Structures and OmniLinter Initialization )
// ============================================================================
#[derive(Debug, Clone)]
pub struct DetectedBlock {
    pub kind: BlockKind,
    pub line_number: usize,
    pub header_raw: String,
}

#[derive(Debug, Clone)]
pub struct LintReport {
    pub file_path: String,
    pub file_type: Option<OmniFileType>,
    pub blocks_found: Vec<DetectedBlock>,
    pub is_valid: bool,
    pub errors: Vec<String>,
    pub warnings: Vec<String>,
}

pub struct OmniLinter {
    pub logger: DiagnosticLogger,
}

impl OmniLinter {
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self { logger }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Linting Logic: 4-Block Detection, Ordering Checks, Auto-Scaffolding )
// ============================================================================
impl OmniLinter {
    /// Lints a source string to verify that all 4 blocks are present, in strict order,
```

```rust
    /// and comply with OmniCode semantic architecture and invariant canons.
    pub fn lint_source(&self, file_path: &str, content: &str) -> LintReport {
        let mut blocks = Vec::new();
        let mut errors = Vec::new();
        let mut warnings = Vec::new();
        let file_type = OmniFileType::from_path(file_path);

        for (idx, line) in content.lines().enumerate() {
            let line_num = idx + 1;
            // Ignore indented comments (e.g. nested sub-reels inside body blocks or scrolls)
            if line.starts_with(' ') || line.starts_with('\t') {
                continue;
            }
            let upper = line.to_uppercase();

            // Detect Block 1: Metadata
            if upper.contains("BLOCK 1") || upper.contains("METADATA BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Metadata,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            }
            // Detect Block 2: Opening
            else if upper.contains("BLOCK 2") || upper.contains("OPENING BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Opening,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            }
            // Detect Block 3: Body
            else if upper.contains("BLOCK 3") || upper.contains("BODY BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Body,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            }
            // Detect Block 4: Closing
            else if upper.contains("BLOCK 4") || upper.contains("CLOSING BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Closing,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            }
        }

        // Check 1: Must contain all 4 blocks
        let has_metadata = blocks.iter().any(|b| b.kind == BlockKind::Metadata);
        let has_opening = blocks.iter().any(|b| b.kind == BlockKind::Opening);
        let has_body = blocks.iter().any(|b| b.kind == BlockKind::Body);
        let has_closing = blocks.iter().any(|b| b.kind == BlockKind::Closing);

        if !has_metadata {
            errors.push("Missing BLOCK 1: METADATA BLOCK".into());
        }
        if !has_opening {
            errors.push("Missing BLOCK 2: OPENING BLOCK".into());
        }
        if !has_body {
            errors.push("Missing BLOCK 3: BODY BLOCK".into());
        }
        if !has_closing {
            errors.push("Missing BLOCK 4: CLOSING BLOCK".into());
        }

        // Check 2: Verify sequential ordering
        for i in 1..blocks.len() {
            if blocks[i].kind <= blocks[i - 1].kind {
                let err = format!(
                    "Block order violation at line {}: {} appeared after {}",
                    blocks[i].line_number,
                    blocks[i].kind.name(),
                    blocks[i - 1].kind.name()
                );
                errors.push(err);
```

```
            }
        }

        // Check 3: Semantic Content Analysis Across Detected Block Regions
        if has_metadata && has_opening && has_body && has_closing && errors.is_empty() {
            let meta_start = blocks.iter().find(|b| b.kind == BlockKind::Metadata).unwrap().line_number;
            let open_start = blocks.iter().find(|b| b.kind == BlockKind::Opening).unwrap().line_number;
            let body_start = blocks.iter().find(|b| b.kind == BlockKind::Body).unwrap().line_number;
            let close_start = blocks.iter().find(|b| b.kind == BlockKind::Closing).unwrap().line_number;

            let lines: Vec<&str> = content.lines().collect();

            let get_slice = |start: usize, end: usize| -> String {
                if start <= end && end <= lines.len() {
                    lines[(start - 1)..(end - 1)].join("\n")
                } else if start <= lines.len() {
                    lines[(start - 1)..].join("\n")
                } else {
                    String::new()
                }
            };

            let meta_slice = get_slice(meta_start, open_start);
            let open_slice = get_slice(open_start, body_start);
            let body_slice = get_slice(body_start, close_start);
            let close_slice = get_slice(close_start, lines.len() + 1);

            // Block 1 Validation
            let has_module_decl = meta_slice.contains("= [Module]:")
                || meta_slice.contains("Module <")
                || meta_slice.contains("Module [")
                || meta_slice.contains("Scroll <");
            if !has_module_decl {
                warnings.push("BLOCK 1: Missing formal module identity declaration (e.g. '= [Module]: < ... > & [ ...
]').".into());
            }
            if !meta_slice.contains("Version") && !meta_slice.contains("version") {
                warnings.push("BLOCK 1: Missing version specification in metadata context.".into());
            }
            if file_path.contains("lexicon") || file_path.contains("manifest") {
                if !meta_slice.contains("Exodus") && !meta_slice.contains("Scripture") && !meta_slice.contains("Witness")
{
                    warnings.push("BLOCK 1: Lexicon/Covenant specification should ground its invariant witness (e.g. '<
Scripture { ... } >').".into());
                }
            }

            // Block 2 Validation
            if !open_slice.contains("Opening") {
                errors.push("BLOCK 2: Missing 'Opening' block scope declaration.".into());
            }
            if !open_slice.contains("preconditions") && !open_slice.contains("Preconditions") &&
!open_slice.contains("+1") {
                warnings.push("BLOCK 2: Opening block should explicitly verify preconditions / armed state.".into());
            }

            // Block 3 Validation
            if !body_slice.contains("Body") {
                errors.push("BLOCK 3: Missing 'Body' block scope declaration.".into());
            }

            // Block 4 Validation: The 5 Pillars & State Machine Completeness
            if !close_slice.contains("Closing") {
                errors.push("BLOCK 4: Missing 'Closing' block scope declaration.".into());
            }

            // Commit Status State Machine Completeness Check
            let commit_upper = close_slice.to_uppercase();
            if commit_upper.contains("COMMIT STATUS") || commit_upper.contains("COMMIT_STATUS") {
                // If the commit status specifies a branching state machine (|), all four states are required
                if close_slice.contains('|') {
                    let has_proceed = commit_upper.contains("PROCEED");
                    let has_await = commit_upper.contains("AWAIT");
                    let has_halt = commit_upper.contains("HALT");
                    let has_rollback = commit_upper.contains("ROLLBACK");

                    let mut missing_states = Vec::new();
                    if !has_proceed { missing_states.push("PROCEED (+1)"); }
```

```
                    if !has_await { missing_states.push("AWAIT (0)"); }
                    if !has_halt { missing_states.push("HALT (NULL)"); }
                    if !has_rollback { missing_states.push("ROLLBACK (-1)"); }

                    if !missing_states.is_empty() {
                        errors.push(format!(
                            "BLOCK 4: Incomplete Commit Status state machine. When using conditional branching, all four end states must be defined. Missing: {}.",
                            missing_states.join(", ")
                        ));
                    }
                }
            }
        }

        // Check 4: Universal Canons (First-class Asides and Delimiter Hygiene)
        let total_lines = content.lines().count();
        let has_any_asides = content.contains('(') && content.contains(')');
        if total_lines > 15 && !has_any_asides {
            warnings.push("Canon 1: Missing first-class intent Asides '(...)'. OmniCode mandates Asides for human-AI pronunciation and semantic verification.".into());
        }

        // Check 5: Snake_case hygiene inside brackets
        for (idx, line) in content.lines().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();
            if trimmed.starts_with('#') {
                continue;
            }
            if let Some(start_bracket) = line.find('[') {
                if let Some(end_bracket) = line[start_bracket..].find(']') {
                    let bracket_content = &line[start_bracket + 1..start_bracket + end_bracket];
                    // Skip directives, tape heads or raw registers (e.g. 0x000B8000, +1, etc.)
                    for word in bracket_content.split_whitespace() {
                        if word.contains('_') && !word.starts_with("0x") && !word.starts_with("roll_") {
                            warnings.push(format!(
                                "Syntax Hygiene (line {line_num}): Identifier '{word}' uses snake_case with underscores inside brackets. In OmniCode, use natural spaced phrases (e.g. '[ ... ]') without underscores.",
                            ));
                            break;
                        }
                    }
                }
            }
        }

        let is_valid = errors.is_empty();

        if !is_valid {
            for err in &errors {
                self.logger.failpoint(
                    FailpointType::SyntaxAnomaly,
                    "LINTER",
                    format!("[{file_path}] {err}"),
                );
            }
        } else {
            self.logger.log(
                "LINTER",
                format!("File [{file_path}] satisfies universal 4-Block Architecture."),
            );
        }

        LintReport {
            file_path: file_path.to_string(),
            file_type,
            blocks_found: blocks,
            is_valid,
            errors,
            warnings,
        }
    }

    /// Lints a file on disk.
    pub fn lint_file(&self, path: impl AsRef<Path>) -> Result<LintReport, String> {
        let p = path.as_ref();
        let content = std::fs::read_to_string(p)
```

```
            .map_err(|e| format!("Failed to read file {}: {e}", p.display()))?;
        Ok(self.lint_source(&p.display().to_string(), &content))
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for an episodic cognitive scroll (.scroll).
    pub fn auto_scaffold_scroll(session_id: &str, prompt: &str) -> String {
        OmniFileType::EpisodicScroll.scaffold_template(session_id, prompt)
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for a given module name (.omni).
    pub fn auto_scaffold_omni(module_name: &str, intent_aside: &str) -> String {
        format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
= [Module]: < Lexicon {{ {module_name} }} > & [ Noun {{ {module_name} }} ] ( {intent_aside} ) {{
    [ Version ]:   {{ "1.0.0" }} ( Version 1.0.0 ),
    [ Author ]:    {{ "CreativeWorkzStudio LLC / ProfessorSeanEX" }} ( System Architect ),
    [ System ]:    {{ OmniCode {{ Core }} }} ( Core Subsystem ),
    [ Witness ]:   < Scripture {{ Exodus 31:18 KJV }} > {{ "Written with the finger of God." }} ( Invariant Scripture
Anchor ),
    [ Nature ]:    < InvariantType > & {{ BaseBlock }} ( Nature is Invariant Type ),
    [ Semantics ]: {{ StandardLibrary :: Specification }} ( Semantics )
}}

< {module_name} > = < Idea {{ Type }} > & [ Noun {{ {module_name} }} ] ( Invariant Definition ) {{
    [ Relation ]: [ {module_name} ] : [ Unit ] :: [ Trit ] : [ Word ] ( Ontological Relation ),
    [ Substrate ]: [ 1 Byte {{ 8 Bits }} ] & [ 1 Cell {{ 5 Trits }} ] ( Dual Substrate ),
    [ Bound ]: ENCLOSE [ {module_name} ] WITHIN [ 1 Cell ] ( Invariant Single Cell Boundary )
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {{
    [ Entity ]: < {module_name} > & [ Cell ] ( Entity allocated in cell ),
    [ Cell ]: [ 1 ] & {{ 5 Trits :: 1 Byte }} ( One cell container ),
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ] ( Equilibrium verified )
}}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    [ ROLL 1 ] : [ Inscribe ] : [ Input ] --> [ Cell ] = [ Entity` ]
        ( Inscribe input into Cell yielding new state Entity-prime ),

    [ ROLL 2 ] : [ Compare ] : {{ Entity` = Input }} --> [ Status` = +1:PROCEED ] | {{ Entity` != Input }} --> [ Status`
= -1:ROLLBACK ]
        ( Evaluate identity and verify bounds )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing ( Sealing Gate: Reconcile, tripwire, debug, log, and affirm permanent execution ) {{
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED
    ] ( Verify that all rolls achieved affirmative consensus in PROCEED ),

    [ Validate ]: {{ Entity` <= [ 1 Cell ] }} & {{ [ 8 Bits ] <=> [ 5 Trits ] }}
        ( Validate single cell bounds and substrate parity ),

    [ Tripwire ]: {{ Status` = +1:PROCEED }} --> [ ARMED ] | {{ Status` != +1:PROCEED }} --> [ TRIPPED ] --> [ Commit
Status = -1:ROLLBACK ]
        ( Circuit breaker vetoes on anomaly ),

    [ Debug ]: {{ [ Entity` ] :: [ Cell` ] :: [ Status` ] }}
        ( Expose diagnostic telemetry snapshot ),

    [ Log ]: [ Journal ]: INSCRIBE {{ [ {module_name} Verified ] :: [ Entity` ] }}
        ( Inscribe event record into system journal ),

    [ Commit Status ]: [
        {{ [ Verify ] = PROCEED }} & {{ [ Tripwire ] = ARMED }} --> [ +1:PROCEED ] |
        {{ [ Status` ] = PENDING }} --> [ 0:AWAIT ] :: REPEAT |
        {{ [ Status` ] = NULL }} --> [ HALT:NULL ] |
```

```
        {{ [ Tripwire ] = TRIPPED }} | {{ [ Verify ] != PROCEED }} --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed, Await, Halt, Rollback )
}}
"#
        )
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Linter Invariant Verification & Teardown )
// ============================================================================
// OmniLinter structural validation invariants closed and verified.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger.rs RS 617 LINES 2541 WORDS 25.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Diagnostic Logger Identity, Failpoint Categories, Cognitive Notes & Enums )
// ============================================================================
//! OmniCode Diagnostics: Structured Failpoint Logger & Cognitive Memory Engine
//!
//! # Purpose & Epistemic Role
//! Explicitly engineered to catch, classify, and report failpoints, boundary
//! violations, and state anomalies, while simultaneously generating structured
//! epistemic memory notes formatted for neural-symbolic AI models, autonomous
//! self-healing engines, and episodic replay verification.
//!
//! # IDE Hover & AI Agent Context
//! When inspecting or hovering over items in this module:
//! - [`FailpointType`]: Identifies the exact invariant violated (e.g. boundary leaks, null traps).
//! - [`FixKind`] & [`FixSuggestion`]: Carries concrete, machine-applicable repair telemetry.
//! - [`CognitiveNote`]: Encapsulates dual-readable episodic memory traces that an AI coding
//!   agent can ingest into its context window or persist into `.scroll` Turing reels.
//! - [`DiagnosticLogger`]: Thread-safe collector powering the self-healing feedback loop.

use std::sync::{Arc, Mutex};
use std::time::Instant;

/// Classifications of trapped failpoints and boundary anomalies in the OmniCode runtime.
///
/// Unlike conventional exceptions or panics that abort process state, OmniCode treats
/// failpoints as first-class ontological diagnostic signals that trigger rollback
/// (`-1:ROLLBACK`), caveat vetos, or automated immune restoration (`+1:PROCEED`).
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum FailpointType {
    /// Delimiter mismatch: opened with one delimiter class but closed with another (e.g. `< ... ]`).
    BoundaryMismatch,
    /// Unclosed boundary delimiter reaching End-of-File without a matching closing partner.
    BoundaryLeak,
    /// Execution attempted to evaluate or dereference a void/null state without a resolving caveat.
    NullTrap,
    /// An operation or caveat evaluation triggered a state rollback (`-1:ROLLBACK`).
    RollbackTriggered,
    /// An adversative caveat condition (`! [ ... ]`) vetoed an active state transition.
    CaveatVeto,
    /// Illegal attempt to mutate an immutable Rank 1 invariant axiom declared inside `< ... >`.
    ImmutableMutation,
    /// Recursive or cyclic retroaction loop exceeding the configured debugger depth threshold.
    CyclicRetroaction,
    /// Syntax, lexer, or structural grammar anomaly that violates language invariants.
    SyntaxAnomaly,
}

impl FailpointType {
    /// Returns the uppercase canonical string identifier for this failpoint category.
    ///
    /// # IDE Hover
    /// Used for telemetry, console logging headers, and `.scroll` failpoint records.
    pub fn as_str(&self) -> &'static str {
        match self {
            FailpointType::BoundaryMismatch => "FP_BOUNDARY_MISMATCH",
            FailpointType::BoundaryLeak => "FP_BOUNDARY_LEAK",
            FailpointType::NullTrap => "FP_NULL_TRAP",
            FailpointType::RollbackTriggered => "FP_ROLLBACK_TRIGGERED",
            FailpointType::CaveatVeto => "FP_CAVEAT_VETO",
            FailpointType::ImmutableMutation => "FP_IMMUTABLE_MUTATION",
            FailpointType::CyclicRetroaction => "FP_CYCLIC_RETROACTION",
            FailpointType::SyntaxAnomaly => "FP_SYNTAX_ANOMALY",
        }
    }
}

/// Categories of automated restorative repairs that the [`crate::diagnostics::restorator::Restorator`]
/// and AI agents can execute to heal code.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum FixKind {
    /// Append a missing closing boundary delimiter (e.g., closing an unclosed `[` with `]`).
    CloseBoundary,
    /// Substitute an erroneous closing delimiter with the delimiter matching the opening boundary.
```

```
    FixBoundaryMismatch,
    /// Inject missing canonical 4-Block architecture scaffolding headers (Metadata, Opening, Body, Closing).
    AddBlockHeader,
    /// Automatically inject missing core library imports (e.g., `Core.Arithmetic`, `Core.Logic`).
    AddImport,
    /// Synthesize and declare an inferred conceptual noun entity (e.g., `Ball < Entity? >`).
    DeclareEntityWithInfer,
    /// Modify or satisfy an adversative caveat expression preventing state commitment.
    AdjustCaveat,
    /// Convert an immutable Rank 1 invariant `< ... >` into a mutable Rank 2 denotation `[ ... ]`.
    ConvertImmutableToMutable,
}

/// Concrete remediation telemetry attached to a failpoint event.
///
/// Contains the exact coordinate, token, and replacement text required to surgically
/// heal source code without manual human intervention.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FixSuggestion {
    /// Categorical classification of the automated remediation.
    pub kind: FixKind,
    /// 1-based source line number where the remediation should be applied.
    pub line: usize,
    /// 1-based source column number where the target token resides.
    pub col: usize,
    /// The erroneous or missing token sequence targeted for replacement.
    pub target_token: String,
    /// The proposed patch or substitution string that seals the invariant.
    pub suggested_patch: String,
    /// Human-readable and AI-interpretable rationale explaining *why* this fix is sound.
    pub rationale: String,
}

/// Structured epistemic memory note formatted for neural-symbolic AI models.
///
/// # Why Cognitive Notes Matter for AI Agents & IDE Tooltips
/// When an AI model or developer inspects code execution, raw text logs lack semantic
/// structure and cause context drift or hallucination. [`CognitiveNote`] structures
/// telemetry into discrete, indexable memory units:
/// - Category: The epistemic role of the observation (`OBSERVATION`, `HYPOTHESIS`, `INCIDENT`).
/// - Summary & Details: Precise descriptions of what occurred and why.
/// - Tags: Semantic taxonomy labels (`#boundary_physics`, `#failpoint`, `#self_healing`).
/// - Context Snippet: The exact line or expression that triggered the note.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct CognitiveNote {
    /// Monotonic timestamp in microseconds since logger initialization.
    pub timestamp_micros: u128,
    /// Epistemic category: e.g. "OBSERVATION", "HYPOTHESIS", "INCIDENT", "REPAIR_VERIFIED".
    pub category: String,
    /// High-level single-sentence summary of the cognitive observation.
    pub summary: String,
    /// In-depth architectural details, telemetry data, or remediation rationale.
    pub details: String,
    /// Semantic taxonomy tags for associative memory indexing.
    pub tags: Vec<String>,
    /// Optional code snippet or token context under active observation.
    pub context_snippet: Option<String>,
}

impl CognitiveNote {
    /// Formats this note as an OmniCode native dual-readable syntax block.
    ///
    /// # Output Example
    /// ```omnicode
    /// Note < CognitiveTrace > {
    ///     tick: < 1240 µs >,
    ///     category: [ INCIDENT ],
    ///     summary: < "Boundary mismatch" >,
    ///     details: < "Mismatched closing delimiter" >,
    ///     tags: [ #boundary_physics :: #self_healing ]
    /// }
    /// ```
    pub fn format_as_omnicode(&self) -> String {
        let tag_str = if self.tags.is_empty() {
            "[]".to_string()
        } else {
            format!("[ {} ]", self.tags.iter().map(|t| format!("#{t}")).collect::<Vec<_>>().join(" :: "))
        };
```

```
        let ctx_str = if let Some(c) = &self.context_snippet {
            format!("\n    context: < {:?} >", c)
        } else {
            String::new()
        };
        format!(
            "Note < CognitiveTrace > {{\n    tick: < {} µs >,\n    category: [ {} ],\n    summary: < {:?} >,\n    details: < {:?} >,\n    tags: {}{}\n}}",
            self.timestamp_micros, self.category, self.summary, self.details, tag_str, ctx_str
        )
    }

    /// Formats this note as a Markdown block specifically structured for LLM context prompts
    /// and episodic memory bank ingestion.
    pub fn format_as_memory_entry(&self) -> String {
        let tag_list = if self.tags.is_empty() {
            "none".to_string()
        } else {
            self.tags.iter().map(|t| format!("#{t}")).collect::<Vec<_>>().join(", ")
        };
        let mut out = format!(
            "### [EPISODIC_MEMORY_TRACE: {}]\n- **Timestamp:** {} µs\n- **Summary:** {}\n- **Details:** {}\n- **Tags:** {}\n",
            self.category, self.timestamp_micros, self.summary, self.details, tag_list
        );
        if let Some(ctx) = &self.context_snippet {
            out.push_str(&format!("- **Context:** `{}`\n", ctx));
        }
        out
    }
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( LogEvent Structure and DiagnosticLogger Initialization )
// ============================================================================
/// Individual diagnostic event recorded during compiler passes or execution ticks.
#[derive(Debug, Clone)]
pub struct LogEvent {
    /// Microseconds elapsed since the logger instance was created.
    pub elapsed_micros: u128,
    /// Channel or subsystem tag: e.g. "BOUNDARY", "DEBUGGER", "TERNARY", "IMMUTABILITY".
    pub channel: &'static str,
    /// Descriptive message explaining the operation or signal.
    pub message: String,
    /// True if this event represents an intercepted failpoint or invariant violation.
    pub is_failpoint: bool,
    /// The specific failpoint category if `is_failpoint` is true.
    pub failpoint_type: Option<FailpointType>,
    /// Concrete remediation suggestion if an automated repair is available.
    pub suggestion: Option<FixSuggestion>,
    /// Accompanying structured cognitive memory note for AI ingestion.
    pub cognitive_note: Option<CognitiveNote>,
}

/// Thread-safe diagnostic logging and cognitive memory collection engine.
///
/// Coordinates execution event tracing, failpoint interception, remediation suggestions,
/// and AI memory scroll generation across lexer, parser, evaluator, and self-healing passes.
#[derive(Debug, Clone)]
pub struct DiagnosticLogger {
    /// Reference instant used to compute microsecond elapsed offsets.
    start_time: Instant,
    /// Chronological sequence of all recorded diagnostic events.
    events: Arc<Mutex<Vec<LogEvent>>>,
    /// Filtered subset containing exclusively failpoint events.
    failpoints: Arc<Mutex<Vec<LogEvent>>>,
    /// Episodic memory notes formatted for neural-symbolic AI models.
    cognitive_notes: Arc<Mutex<Vec<CognitiveNote>>>,
    /// When true, outputs color-coded diagnostic lines to stdout/stderr in real time.
    print_live: bool,
}

impl DiagnosticLogger {
    /// Constructs a new [`DiagnosticLogger`].
    ///
    /// # Arguments
    /// * `print_live` - If true, logs are immediately mirrored to stdout/stderr.
```

```
    pub fn new(print_live: bool) -> Self {
        Self {
            start_time: Instant::now(),
            events: Arc::new(Mutex::new(Vec::new())),
            failpoints: Arc::new(Mutex::new(Vec::new())),
            cognitive_notes: Arc::new(Mutex::new(Vec::new())),
            print_live,
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Logging Operations, Cognitive Notes, Interceptions & Export )
// ============================================================================
impl DiagnosticLogger {
    /// Inscribes a normal debug or trace event into the diagnostic log.
    ///
    /// # Arguments
    /// * `channel` - The subsystem identifier (e.g. "PARSER", "BOUNDARY", "TERNARY").
    /// * `message` - The informational message content.
    pub fn log(&self, channel: &'static str, message: impl Into<String>) {
        let msg = message.into();
        let elapsed = self.start_time.elapsed().as_micros();
        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg.clone(),
            is_failpoint: false,
            failpoint_type: None,
            suggestion: None,
            cognitive_note: None,
        };

        if self.print_live {
            println!("[{:>6} µs] DEBUG    [{}] {}", elapsed, channel, msg);
        }

        self.events.lock().unwrap().push(event);
    }

    /// Intercepts and records a critical failpoint event without remediation telemetry.
    ///
    /// Automatically synthesizes an accompanying [`CognitiveNote`] with category `INCIDENT`.
    pub fn failpoint(&self, fp_type: FailpointType, channel: &'static str, message: impl Into<String>) {
        let msg = message.into();
        let elapsed = self.start_time.elapsed().as_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "INCIDENT".to_string(),
            summary: msg.clone(),
            details: format!("Failpoint trapped: {}", fp_type.as_str()),
            tags: vec!["failpoint".to_string(), fp_type.as_str().to_lowercase()],
            context_snippet: None,
        };

        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg.clone(),
            is_failpoint: true,
            failpoint_type: Some(fp_type),
            suggestion: None,
            cognitive_note: Some(note.clone()),
        };

        if self.print_live {
            eprintln!(
                "[{:>6} µs] \x1b[91mCRITICAL\x1b[0m [{}] [FAILPOINT: {}] {}",
                elapsed,
                channel,
                fp_type.as_str(),
                msg
            );
        }

        self.cognitive_notes.lock().unwrap().push(note);
```

```
        self.failpoints.lock().unwrap().push(event.clone());
        self.events.lock().unwrap().push(event);
    }

    /// Intercepts a critical failpoint event with concrete remediation telemetry.
    ///
    /// Equips the event with a [`FixSuggestion`] and registers an episodic [`CognitiveNote`]
    /// marked `#remediation_ready` so the Restorator or AI can immediately apply the repair.
    pub fn failpoint_with_remediation(
        &self,
        fp_type: FailpointType,
        channel: &'static str,
        message: impl Into<String>,
        suggestion: FixSuggestion,
    ) {
        let msg = message.into();
        let elapsed = self.start_time.elapsed().as_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "INCIDENT".to_string(),
            summary: msg.clone(),
            details: suggestion.rationale.clone(),
            tags: vec![
                "failpoint".to_string(),
                fp_type.as_str().to_lowercase(),
                "remediation_ready".to_string(),
            ],
            context_snippet: Some(suggestion.target_token.clone()),
        };

        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg.clone(),
            is_failpoint: true,
            failpoint_type: Some(fp_type),
            suggestion: Some(suggestion),
            cognitive_note: Some(note.clone()),
        };

        if self.print_live {
            eprintln!(
                "[{:>6} µs] \x1b[91mCRITICAL\x1b[0m [{}] [FAILPOINT: {}] {} (Remediation: {:?})",
                elapsed,
                channel,
                fp_type.as_str(),
                msg,
                event.suggestion
            );
        }

        self.cognitive_notes.lock().unwrap().push(note);
        self.failpoints.lock().unwrap().push(event.clone());
        self.events.lock().unwrap().push(event);
    }

    /// Records an arbitrary cognitive observation or hypothesis into episodic memory.
    ///
    /// # Arguments
    /// * `category` - Epistemic label (e.g. "OBSERVATION", "HYPOTHESIS", "ORACLE_SUCCESS").
    /// * `summary` - One-line summary for rapid cognitive indexing.
    /// * `details` - Comprehensive description or reasoning trace.
    /// * `tags` - Semantic search and association tags.
    /// * `context` - Optional code snippet or token reference under study.
    pub fn record_ai_observation(
        &self,
        category: &str,
        summary: &str,
        details: &str,
        tags: &[&str],
        context: Option<&str>,
    ) -> CognitiveNote {
        let elapsed = self.start_time.elapsed().as_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: category.to_string(),
            summary: summary.to_string(),
            details: details.to_string(),
```

```
            tags: tags.iter().map(|s| s.to_string()).collect(),
            context_snippet: context.map(|s| s.to_string()),
        };

        self.cognitive_notes.lock().unwrap().push(note.clone());
        self.log(
            "COGNITION",
            format!("[{category}] {summary} (tags: {:?})", note.tags),
        );
        note
    }

    /// Inscribes a pre-formed [`CognitiveNote`] directly into the cognitive notes buffer.
    pub fn log_cognitive_note(&self, note: CognitiveNote) {
        self.cognitive_notes.lock().unwrap().push(note);
    }

    /// Retrieves a cloned snapshot of all recorded cognitive memory notes.
    pub fn cognitive_notes(&self) -> Vec<CognitiveNote> {
        self.cognitive_notes.lock().unwrap().clone()
    }

    /// Exports all recorded cognitive notes into a canonical 4-Block OmniCode memory scroll.
    ///
    /// This scroll can be saved directly as a `.scroll` artifact or fed into the AI episodic
    /// memory learner for continuous model evolution.
    pub fn export_ai_memory_scroll(&self) -> String {
        let notes = self.cognitive_notes();
        let mut out = String::new();
        out.push_str("# ================================================================================\n");
        out.push_str("# BLOCK 1: METADATA BLOCK\n");
        out.push_str("# ( Episodic AI Cognitive Memory Scroll & Incident Audit Log )\n");
        out.push_str("# ================================================================================\n");
        out.push_str("Scroll < CognitiveMemory.Audit > {\n");
        out.push_str(&format!("    total_notes: < {} >,\n", notes.len()));
        out.push_str(&format!("    total_failpoints: < {} >\n", self.failpoints.lock().unwrap().len()));
        out.push_str("}\n\n");
        out.push_str("# ================================================================================\n");
        out.push_str("# BLOCK 2: OPENING BLOCK\n");
        out.push_str("# ( Cognitive Context & Invariant Preconditions )\n");
        out.push_str("# ================================================================================\n");
        out.push_str("Opening {\n    preconditions: [ +1 ]\n}\n\n");
        out.push_str("# ================================================================================\n");
        out.push_str("# BLOCK 3: BODY BLOCK\n");
        out.push_str("# ( Episodic Traces & Memory Units Recorded During Execution )\n");
        out.push_str("# ================================================================================\n");
        out.push_str("Body [\n");
        for n in &notes {
            out.push_str(&format!("    {}\n\n", n.format_as_omnicode().replace('\n', "\n    ")));
        }
        out.push_str("]\n\n");
        out.push_str("# ================================================================================\n");
        out.push_str("# BLOCK 4: CLOSING BLOCK\n");
        out.push_str("# ( Cognitive Audit Verification Proof )\n");
        out.push_str("# ================================================================================\n");
        out.push_str("Closing {\n    audit_verified: [ PROCEED ]\n}\n");
        out
    }

    /// Retrieves a cloned snapshot of all intercepted failpoint events.
    pub fn failpoints(&self) -> Vec<LogEvent> {
        self.failpoints.lock().unwrap().clone()
    }

    /// Retrieves a cloned snapshot of all general log events.
    pub fn events(&self) -> Vec<LogEvent> {
        self.events.lock().unwrap().clone()
    }

    /// Extracts all remediation suggestions currently registered across failpoints.
    pub fn remediations(&self) -> Vec<FixSuggestion> {
        self.failpoints
            .lock()
            .unwrap()
            .iter()
            .filter_map(|e| e.suggestion.clone())
            .collect()
    }
```

```
463
464     /// Returns `true` if any failpoints were trapped during the execution session.
465     pub fn has_failpoints(&self) -> bool {
466         !self.failpoints.lock().unwrap().is_empty()
467     }
468
469     /// Clears all stored events, failpoints, and cognitive memory notes.
470     pub fn clear(&self) {
471         self.events.lock().unwrap().clear();
472         self.failpoints.lock().unwrap().clear();
473         self.cognitive_notes.lock().unwrap().clear();
474     }
475
476     /// Logs an internal AI Thought, indexing it as a cognitive note in episodic memory.
477     ///
478     /// Links the AI's internal thought tape directly with the diagnostic trace.
479     pub fn log_thought(&self, thought_id: &str, omnicode_tape: &str, premise: &str, quad_state_str: &str) {
480         let elapsed = self.start_time.elapsed().as_micros();
481         let note = CognitiveNote {
482             timestamp_micros: elapsed,
483             category: "THOUGHT".to_string(),
484             summary: format!("Thought [{thought_id}]: {premise}"),
485             details: omnicode_tape.to_string(),
486             tags: vec!["thought".to_string(), "ai_cortex".to_string(), quad_state_str.to_lowercase()],
487             context_snippet: Some(omnicode_tape.to_string()),
488         };
489
490         self.cognitive_notes.lock().unwrap().push(note);
491         self.log(
492             "CORTEX",
493             format!("[THOUGHT: {thought_id}] (State: {quad_state_str}) Premise: \"{premise}\""),
494         );
495     }
496
497     /// Records an associative memory link between an entity/concept and its scriptural or scroll anchor.
498     pub fn log_memory_association(&self, concept: &str, anchor: &str, details: &str) {
499         let elapsed = self.start_time.elapsed().as_micros();
500         let note = CognitiveNote {
501             timestamp_micros: elapsed,
502             category: "ASSOCIATION".to_string(),
503             summary: format!("Memory Link: {concept} <-> {anchor}"),
504             details: details.to_string(),
505             tags: vec!["memory".to_string(), "scripture".to_string(), concept.to_lowercase()],
506             context_snippet: Some(anchor.to_string()),
507         };
508
509         self.cognitive_notes.lock().unwrap().push(note);
510         self.log(
511             "MEMORY",
512             format!("Associated concept '{concept}' with anchor '{anchor}'"),
513         );
514     }
515
516     /// Logs the physical inscription of an episodic scroll to disk.
517     pub fn log_scroll_inscribed(&self, scroll_path: &std::path::Path, session_or_id: &str) {
518         self.log(
519             "SCROLL",
520             format!("Inscribed episodic scroll '{session_or_id}' to {:?}", scroll_path),
521         );
522     }
523
524     /// Inscribes a canonical 4-Block .scroll file to the given path.
525     pub fn inscribe_canonical_scroll(
526         &self,
527         path: &std::path::Path,
528         session_id: &str,
529         prompt: &str,
530         synthesized_omnicode: &str,
531         state_str: &str,
532         response: &str,
533     ) -> Result<std::path::PathBuf, String> {
534         if let Some(parent) = path.parent() {
535             if !parent.exists() {
536                 std::fs::create_dir_all(parent)
537                     .map_err(|e| format!("Failed to create scroll parent directory: {e}"))?;
538             }
539         }
540
```

```
        let elapsed = self.start_time.elapsed().as_micros();
        let scroll_content = format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
# ( Execution Trace ID, Prompt Intent, Inferred Types, and Neural Snapshot )
# ================================================================================
Scroll < EpistemicTrace.{session_id} > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    session_id: < "{session_id}" >,
    execution_duration_micros: < {elapsed} >,
    cognitive_mode: < NeuralSymbolicTransduction >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ( Hardware Invariants, Preconditions, and Initial Context )
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

user_prompt < "{prompt}" >

# ================================================================================
# BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & THOUGHT REEL)
# ( Replay Tape: Synthesized Program, Sensory Telemetry & Immune Interventions )
# ================================================================================
Body [
    # Executed Turing Reel Synthesized by AI Model:
    synthesized_reel [
{indented_reel}
    ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING PROOF)
# ( Verification Proof, Final Quad-State, and Conversational Synthesis )
# ================================================================================
Closing {{
    commit_status [ <{state_str}> ],
    machine_utterance ( "{sanitized_response}" )
}}
"#,
            session_id = session_id,
            prompt = prompt,
            elapsed = elapsed,
            indented_reel = synthesized_omnicode
                .lines()
                .map(|l| format!("        {l}"))
                .collect::<Vec<_>>()
                .join("\n"),
            state_str = state_str,
            sanitized_response = response.replace('"', "\\\"")
        );

        std::fs::write(path, scroll_content)
            .map_err(|e| format!("Failed to write scroll to {:?}: {e}", path))?;

        self.log_scroll_inscribed(path, session_id);
        Ok(path.to_path_buf())
    }
}


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Teardown )
// ================================================================================
impl Default for DiagnosticLogger {
    /// Constructs a silent [`DiagnosticLogger`] with live terminal printing disabled.
    fn default() -> Self {
        Self::new(false)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/mod.rs RS 51 LINES 252 WORDS 2.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Diagnostics Subsystem Identity, Module Architecture & Exports )
// ================================================================================
//! OmniCode Diagnostics Subsystem
//!
//! # Architecture & Cognitive Cybernetics
//! Coordinates the runtime sensory, debugging, and self-healing immune organs:
//! - [`logger`]: Structured failpoint interceptor, telemetry logger, and AI episodic memory emitter ([`CognitiveNote`]).
//! - [`debugger`]: Boundary stack monitor, immutability inspector, retroaction depth guard, and test verification oracle.
//! - [`linter`]: Structural validator enforcing the canonical 4-Block architecture across compilation units.
//! - [`restorator`]: Autonomous multi-pass code healing engine with test-oracle certification.
//!
//! # IDE Hover Support
//! All diagnostic components are re-exported at the module root for immediate autocompletion
//! and documentation discovery in language servers and coding assistants.

pub mod logger;
pub mod debugger;
pub mod linter;
pub mod restorator;
pub mod hover;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Diagnostic Tools )
// ================================================================================
pub use logger::{DiagnosticLogger, FailpointType, LogEvent, FixKind, FixSuggestion, CognitiveNote};
pub use debugger::OmniDebugger;
pub use linter::{BlockKind, DetectedBlock, LintReport, OmniLinter};
pub use restorator::{Restorator, RestorationReport, AppliedRepair};
pub use hover::{HoverEngine, HoverCard, OmniDefinition, TapeRollContext};


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Core Registrations & Coordination Invariants )
// ================================================================================
// Diagnostic modules are actively coordinated across compiler passes.
// The Logger records incidents, the Restorator synthesizes candidate patches,
// and the Debugger executes verification test oracles before state commitment.


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ================================================================================
// Diagnostics subsystem verified and closed.

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/restorator.rs

RS | 667 LINES | 2387 WORDS | 27.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Restorator Identity, Self-Healing Immune Invariants & Repair Kinds )
// ============================================================================
//! OmniCode Diagnostics: Restorator & Self-Healing Engine
//!
//! # Purpose & Cybernetic Architecture
//! Autonomous code restoration and healing organ. Diagnoses syntax anomalies,
//! unclosed or mismatched boundaries, missing 4-block architecture scaffolding,
//! and missing standard library imports, applying surgical restorative patches
//! to bring non-operational code into a verified operational state (`+1:PROCEED`).
//!
//! # Self-Healing Pipeline Passes:
//! 1. **Pass 1 (Boundary Physics)**: Closes unclosed delimiters and aligns mismatched pairs.
//! 2. **Pass 2 (4-Block Architecture)**: Ensures canonical order: Metadata -> Opening -> Body -> Closing.
//! 3. **Pass 3 (Dependency Injection)**: Automatically detects required core modules (`Core.Arithmetic`, etc.) and injects imports.
//! 4. **Pass 4 (Conceptual Inference)**: Synthesizes noun declarations with infer operators (`Ball < Entity? >`).
//! 5. **Pass 5 (Round-trip Verification)**: Lints, lexes, parses, and evaluates the candidate to certify operational validity.
//!
//! # Test Oracle Integration:
//! Connects directly to test verification harnesses via [`Restorator::restore_and_verify_with_oracle`],
//! ensuring patches satisfy external invariants and recording verified repair notes in AI memory.

use std::fs;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::linter::{BlockKind, OmniLinter};
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, LogEvent};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::Environment;
use crate::runtime::evaluator::Evaluator;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Restoration Structures and Engine Context Initialization )
// ============================================================================
/// Record of an individual restorative patch applied to source code.
#[derive(Debug, Clone)]
pub struct AppliedRepair {
    /// The structural category of the applied repair.
    pub kind: FixKind,
    /// Detailed human and AI-readable explanation of why and how the repair was made.
    pub description: String,
    /// 1-based source line number where the repair took effect.
    pub line: usize,
}

/// Comprehensive outcome report generated by the [`Restorator`].
///
/// Contains original source, restored content, applied repairs, verification status,
/// and any errors encountered during the self-healing cycle.
#[derive(Debug, Clone)]
pub struct RestorationReport {
    /// File path if restoration was performed on disk, or `None` if in-memory.
    pub file_path: Option<String>,
    /// `true` if any modifications or repairs were made to the source text.
    pub was_modified: bool,
    /// Exact unmodified original source code prior to healing passes.
    pub original_content: String,
    /// Healed, patched, and structurally validated source code.
    pub restored_content: String,
    /// Chronological list of all automated repairs applied during the cycle.
    pub repairs_applied: Vec<AppliedRepair>,
    /// `true` if the restored content passed all lint, parse, and evaluation checks.
    pub verified: bool,
    /// The final QuadState produced by evaluating the restored source (+1:PROCEED).
    pub final_state: Option<QuadState>,
    /// Any lint, parse, or evaluation errors if verification failed.
    pub errors: Vec<String>,
```

```rust
}

/// The autonomous self-healing and code restoration engine.
pub struct Restorator {
    /// Shared diagnostic logger recording cognitive observations and repair history.
    pub logger: DiagnosticLogger,
}

impl Restorator {
    /// Constructs a new [`Restorator`] instance bound to the provided logger.
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self { logger }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Restoration Pipeline: Boundaries, 4-Block Headers, Imports, Verification )
// ============================================================================
impl Restorator {
    /// Restores source code in memory, executing multi-pass auto-repairs.
    ///
    /// Executes the 5-pass self-healing pipeline:
    /// 1. Delimiter & Boundary Healing
    /// 2. 4-Block Architecture Injection
    /// 3. Standard Library Import Resolution
    /// 4. Inferred Conceptual Entity Declaration
    /// 5. Round-trip Lint/Parse/Eval Verification
    ///
    /// # Arguments
    /// * `source` - Raw input string that may contain boundary leaks or syntax anomalies.
    ///
    /// # Returns
    /// A [`RestorationReport`] detailing all modifications and verification proofs.
    pub fn restore_source(&self, source: &str) -> RestorationReport {
        let original_content = source.to_string();
        let mut current = source.to_string();
        let mut repairs_applied = Vec::new();

        // Pass 1: Delimiter & Boundary Healing
        current = self.heal_boundaries(&current, &mut repairs_applied);

        // Pass 2: 4-Block Architectural Scaffolding Healing
        current = self.heal_four_blocks(&current, &mut repairs_applied);

        // Pass 3: Automated Dependency & Import Healing
        current = self.heal_imports(&current, &mut repairs_applied);

        // Pass 4: Inferred Conceptual Entity Healing
        current = self.heal_inferred_entities(&current, &mut repairs_applied);

        // Pass 5: Round-trip Self-Verification
        let (verified, final_state, errors) = self.verify_restoration(&current);

        let was_modified = current != original_content;

        if !repairs_applied.is_empty() {
            let repair_descs: Vec<String> = repairs_applied.iter().map(|r| r.description.clone()).collect();
            self.logger.record_ai_observation(
                "SELF_HEAL_APPLIED",
                &format!("Applied {} automated repairs", repairs_applied.len()),
                &format!("Repairs executed: {}", repair_descs.join("; ")),
                ["self_healing", "remediation", "restorator"].as_slice(),
                None,
            );
        }

        RestorationReport {
            file_path: None,
            was_modified,
            original_content,
            restored_content: current,
            repairs_applied,
            verified,
            final_state,
            errors,
        }
```

```rust
    }

    /// Ingests a structured [`LogEvent`] and executes a targeted, single-point surgical patch.
    ///
    /// # Arguments
    /// * `source` - Source code to repair.
    /// * `log` - Structured incident event containing a [`crate::diagnostics::logger::FixSuggestion`].
    ///
    /// # Returns
    /// [`RestorationReport`] showing the single-point modification and round-trip verification status.
    pub fn restore_from_incident_log(&self, source: &str, log: &LogEvent) -> RestorationReport {
        let original_content = source.to_string();
        let mut repairs_applied = Vec::new();
        let mut current = source.to_string();

        if let Some(sug) = &log.suggestion {
            match sug.kind {
                FixKind::FixBoundaryMismatch => {
                    let lines: Vec<&str> = current.lines().collect();
                    if sug.line > 0 && sug.line <= lines.len() {
                        let mut target_line = lines[sug.line - 1].to_string();
                        if let Some(pos) = target_line.find(&sug.target_token) {
                            target_line.replace_range(pos..pos + sug.target_token.len(), &sug.suggested_patch);
                            let mut new_lines = lines.clone();
                            new_lines[sug.line - 1] = &target_line;
                            current = new_lines.join("\n");
                            if !current.ends_with('\n') {
                                current.push('\n');
                            }
                            repairs_applied.push(AppliedRepair {
                                kind: FixKind::FixBoundaryMismatch,
                                description: sug.rationale.clone(),
                                line: sug.line,
                            });
                        }
                    }
                }
                FixKind::CloseBoundary => {
                    current.push_str(&sug.suggested_patch);
                    if !current.ends_with('\n') {
                        current.push('\n');
                    }
                    repairs_applied.push(AppliedRepair {
                        kind: FixKind::CloseBoundary,
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
                FixKind::ConvertImmutableToMutable => {
                    current = current.replace(&format!("< {} >", sug.target_token), &sug.suggested_patch);
                    repairs_applied.push(AppliedRepair {
                        kind: FixKind::ConvertImmutableToMutable,
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
                _ => {
                    current = current.replace(&sug.target_token, &sug.suggested_patch);
                    repairs_applied.push(AppliedRepair {
                        kind: sug.kind.clone(),
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
            }
        }

        let (verified, final_state, errors) = self.verify_restoration(&current);
        let was_modified = current != original_content;

        if was_modified {
            self.logger.record_ai_observation(
                "LOG_DRIVEN_RESTORE",
                &format!("Applied targeted fix from log for {:?}", log.failpoint_type),
                &format!("Modified {} bytes", current.len()),
                ["self_healing", "log_driven"].as_slice(),
                None,
            );
```

```
        }

        RestorationReport {
            file_path: None,
            was_modified,
            original_content,
            restored_content: current,
            repairs_applied,
            verified,
            final_state,
            errors,
        }
    }

    /// Restores source code and executes an external verification test oracle against the candidate.
    ///
    /// Connects the self-healing engine to test harnesses:
    /// - If the restored code satisfies the test oracle: logs an `ORACLE_CERTIFIED` cognitive trace.
    /// - If the test oracle fails: records a rollback failpoint and adds the failure to the report errors.
    ///
    /// # Arguments
    /// * `source` - Raw input code.
    /// * `test_oracle` - Closure asserting domain-specific invariants on the restored source.
    pub fn restore_and_verify_with_oracle<F>(&self, source: &str, test_oracle: F) -> RestorationReport
    where
        F: Fn(&str) -> Result<(), String>,
    {
        let mut report = self.restore_source(source);

        if report.verified {
            match test_oracle(&report.restored_content) {
                Ok(()) => {
                    self.logger.log("RESTORATOR", "Restoration validated by test oracle (+1:PROCEED)");
                    self.logger.record_ai_observation(
                        "ORACLE_CERTIFIED",
                        "Restored code certified by test oracle",
                        "The candidate passed the verification test suite cleanly",
                        ["self_healing", "test_oracle", "certified"].as_slice(),
                        None,
                    );
                }
                Err(err) => {
                    self.logger.failpoint(
                        FailpointType::RollbackTriggered,
                        "RESTORATOR",
                        format!("Restoration failed test oracle verification: {err}"),
                    );
                    self.logger.record_ai_observation(
                        "ORACLE_REJECTED",
                        "Restored code failed test oracle verification",
                        &err,
                        ["self_healing", "test_oracle", "rollback"].as_slice(),
                        None,
                    );
                    report.verified = false;
                    report.errors.push(format!("Test oracle failure: {err}"));
                }
            }
        }

        report
    }

    /// Restores a file from disk, optionally writing back the healed source in-place.
    ///
    /// # Arguments
    /// * `path` - File system path to the target source file.
    /// * `in_place` - If true, overwrites the file on disk with the restored content.
    ///
    /// # Errors
    /// Returns `Err(String)` if file reading or writing fails.
    pub fn restore_file(&self, path: &str, in_place: bool) -> Result<RestorationReport, String> {
        let content = fs::read_to_string(path)
            .map_err(|e| format!("Failed to read file '{path}': {e}"))?;

        let mut report = self.restore_source(&content);
        report.file_path = Some(path.to_string());

```

```
        if in_place && report.was_modified {
            fs::write(path, &report.restored_content)
                .map_err(|e| format!("Failed to write restored file '{path}': {e}"))?;
            self.logger.log("RESTORATOR", format!("Wrote restored content in-place to '{path}'"));
        }

        Ok(report)
    }

    /// Pass 1: Auto-detects and repairs mismatched and leaking delimiters.
    fn heal_boundaries(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let mut text = source.to_string();
        let mut max_passes = 10;

        while max_passes > 0 {
            max_passes -= 1;
            let temp_logger = DiagnosticLogger::new(false);
            let mut lexer = Lexer::new(&text, temp_logger.clone());
            let tokens = match lexer.tokenize() {
                Ok(toks) => toks,
                Err(_) => break,
            };

            let debugger = OmniDebugger::new(temp_logger.clone(), 10);
            let mut parser = Parser::new(tokens, debugger);
            let parse_res = parser.parse_program();

            if parse_res.is_ok() {
                break; // Boundaries are sound
            }

            // Inspect diagnostic remediations from debugger
            let mut debugger_remediations = parser.debugger.drain_remediations();
            if debugger_remediations.is_empty() {
                debugger_remediations = temp_logger.remediations();
            }

            if debugger_remediations.is_empty() {
                // If EOF unclosed boundary detected without explicit remediation, balance stack
                if !parser.debugger.boundary_stack.is_empty() {
                    let mut closers = String::new();
                    while let Some(frame) = parser.debugger.boundary_stack.pop() {
                        let c = match frame.open_char {
                            '<' => '>',
                            '[' => ']',
                            '{' => '}',
                            _ => ')',
                        };
                        closers.push(c);
                        repairs.push(AppliedRepair {
                            kind: FixKind::CloseBoundary,
                            description: format!("Appended missing closing delimiter '{c}' for boundary opened at
L{}:C{}", frame.line, frame.col),
                            line: frame.line,
                        });
                    }
                    text.push_str(&format!("\n{}\n", closers));
                    continue;
                }
                break;
            }

            // Apply remediation patches
            let mut applied_any = false;
            for sug in debugger_remediations {
                match sug.kind {
                    FixKind::FixBoundaryMismatch => {
                        let lines: Vec<&str> = text.lines().collect();
                        if sug.line > 0 && sug.line <= lines.len() {
                            let mut target_line = lines[sug.line - 1].to_string();
                            if let Some(pos) = target_line.find(&sug.target_token) {
                                target_line.replace_range(pos..pos + sug.target_token.len(), &sug.suggested_patch);
                                let mut new_lines = lines.clone();
                                new_lines[sug.line - 1] = &target_line;
                                text = new_lines.join("\n");
                                if !text.ends_with('\n') {
                                    text.push('\n');
                                }
```

```
                                repairs.push(AppliedRepair {
                                    kind: FixKind::FixBoundaryMismatch,
                                    description: sug.rationale,
                                    line: sug.line,
                                });
                                applied_any = true;
                                break;
                            }
                        }
                    }
                    FixKind::CloseBoundary => {
                        text.push_str(&sug.suggested_patch);
                        if !text.ends_with('\n') {
                            text.push('\n');
                        }
                        repairs.push(AppliedRepair {
                            kind: FixKind::CloseBoundary,
                            description: sug.rationale,
                            line: sug.line,
                        });
                        applied_any = true;
                        break;
                    }
                    _ => {}
                }
            }

            if !applied_any {
                break;
            }
        }

        text
    }

    /// Pass 2: Ensures all 4 universal architectural blocks exist in canonical order.
    fn heal_four_blocks(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let linter = OmniLinter::new(DiagnosticLogger::new(false));
        let report = linter.lint_source("memory", source);

        if report.is_valid {
            return source.to_string();
        }

        let mut has_metadata = false;
        let mut has_opening = false;
        let mut has_body = false;
        let mut has_closing = false;

        for b in &report.blocks_found {
            match b.kind {
                BlockKind::Metadata => has_metadata = true,
                BlockKind::Opening => has_opening = true,
                BlockKind::Body => has_body = true,
                BlockKind::Closing => has_closing = true,
            }
        }

        let mut out = String::new();

        // 1. Inject Metadata Block if missing
        if !has_metadata {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 1: METADATA manifest".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 1: METADATA BLOCK\n");
            out.push_str("# ( Restored Module Manifest and Semantic Track )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Module < Restored.Track > ( Auto-restored OmniCode compilation unit ) {\n");
            out.push_str("    version: < \"1.0.0\" >,\n");
            out.push_str("    system: < OmniCode.Core >\n");
            out.push_str("}\n\n");
        }

        // 2. Inject Opening Block if missing
```

```
        if !has_opening {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 2: OPENING context".into(),
                line: 1,
            });
            out.push_str("# ========================================================================================\n");
            out.push_str("# BLOCK 2: OPENING BLOCK\n");
            out.push_str("# ( Initial Conditions and Operational Preconditions )\n");
            out.push_str("# ========================================================================================\n");
            out.push_str("Opening {\n");
            out.push_str("    preconditions: [ +1 ]\n");
            out.push_str("}\n\n");
        }

        // 3. Append existing body content
        if !has_body {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 3: BODY block wrapping loose operations".into(),
                line: 1,
            });
            out.push_str("# ========================================================================================\n");
            out.push_str("# BLOCK 3: BODY BLOCK\n");
            out.push_str("# ( Core Operations and Transformations )\n");
            out.push_str("# ========================================================================================\n");
            out.push_str("Body [\n");
            for line in source.lines() {
                if !line.trim().is_empty() {
                    out.push_str(&format!("    {}\n", line));
                }
            }
            out.push_str("]\n\n");
        } else {
            out.push_str(source);
            if !out.ends_with('\n') {
                out.push('\n');
            }
        }

        // 4. Inject Closing Block if missing
        if !has_closing {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 4: CLOSING commit proof".into(),
                line: 1,
            });
            out.push_str("# ========================================================================================\n");
            out.push_str("# BLOCK 4: CLOSING BLOCK\n");
            out.push_str("# ( State Verification Proof and Final Commit )\n");
            out.push_str("# ========================================================================================\n");
            out.push_str("Closing {\n");
            out.push_str("    commit_status [ PROCEED ]\n");
            out.push_str("}\n");
        }

        out
    }

    /// Pass 3: Automatically injects missing core library imports into Opening block.
    fn heal_imports(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let needs_arithmetic = (source.contains("plus") || source.contains("minus") || source.contains("times") ||
source.contains("div") || source.contains("ADD") || source.contains("SUB"))
            && !source.contains("Core.Arithmetic");

        let needs_logic = (source.contains("PROCEED") || source.contains("ROLLBACK") || source.contains("AWAIT") ||
source.contains("HALT"))
            && !source.contains("Core.Logic");

        let needs_vectors = (source.contains("-->") || source.contains("<--") || source.contains("--^") ||
source.contains("--v"))
            && !source.contains("Core.Vectors");

        if !needs_arithmetic && !needs_logic && !needs_vectors {
            return source.to_string();
        }

        let mut mods_to_import = Vec::new();
```

```
        if needs_arithmetic {
            mods_to_import.push("Core.Arithmetic");
        }
        if needs_logic {
            mods_to_import.push("Core.Logic");
        }
        if needs_vectors {
            mods_to_import.push("Core.Vectors");
        }

        let import_expr = format!("import: [ {} ]", mods_to_import.join(" :: "));

        // Inject into Opening { ... } block
        if let Some(op_idx) = source.find("Opening {") {
            let insert_pos = op_idx + "Opening {".len();
            let mut out = source.to_string();
            let injection = format!("\n    {},\n", import_expr);
            out.insert_str(insert_pos, &injection);

            repairs.push(AppliedRepair {
                kind: FixKind::AddImport,
                description: format!("Injected library imports into Opening block: {import_expr}"),
                line: 1,
            });
            return out;
        }

        source.to_string()
    }

    /// Pass 4: Resolves and declares undefined conceptual nouns with infer operator `?`.
    fn heal_inferred_entities(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let mut missing_inferred = Vec::new();

        if source.contains("Ball?") && !source.contains("Ball <") {
            missing_inferred.push("Ball < Entity? > ( Inferred discrete unit )");
        }
        if source.contains("Group?") && !source.contains("Group <") {
            missing_inferred.push("Group < Set? >   ( Inferred collection container )");
        }

        if missing_inferred.is_empty() {
            return source.to_string();
        }

        let mut out = source.to_string();
        // Insert right after Module < ... > definition
        if let Some(pos) = out.find("Module <") {
            if let Some(block_end) = out[pos..].find('}') {
                let insert_at = pos + block_end + 1;
                let mut decls = String::from("\n\n# Auto-Restored Conceptual Types:\n");
                for d in &missing_inferred {
                    decls.push_str(d);
                    decls.push('\n');
                }
                out.insert_str(insert_at, &decls);

                repairs.push(AppliedRepair {
                    kind: FixKind::DeclareEntityWithInfer,
                    description: format!("Declared {} inferred conceptual entities", missing_inferred.len()),
                    line: 1,
                });
            }
        }

        out
    }

    /// Pass 5: Evaluates restored code to verify complete execution without failpoints.
    fn verify_restoration(&self, source: &str) -> (bool, Option<QuadState>, Vec<String>) {
        let mut errors = Vec::new();
        let logger = DiagnosticLogger::new(false);

        // 1. Lint check
        let linter = OmniLinter::new(logger.clone());
        let lint_rep = linter.lint_source("verify", source);
        if !lint_rep.is_valid {
            for err in lint_rep.errors {
```

```
                errors.push(format!("Lint error: {err}"));
            }
        }

        // 2. Tokenize & Parse
        let mut lexer = Lexer::new(source, logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(t) => t,
            Err(e) => {
                errors.push(format!("Lex error: {e}"));
                return (false, None, errors);
            }
        };

        let debugger = OmniDebugger::new(logger.clone(), 10);
        let mut parser = Parser::new(tokens, debugger);
        let prog = match parser.parse_program() {
            Ok(p) => p,
            Err(e) => {
                errors.push(format!("Parse error: {e}"));
                return (false, None, errors);
            }
        };

        // 3. Execution check
        let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
        match evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog)) {
            Ok(val) => {
                let state = match val {
                    crate::runtime::environment::RuntimeValue::State(s) => s,
                    _ => QuadState::Proceed,
                };
                let verified = errors.is_empty() && (state == QuadState::Proceed || state == QuadState::Await);
                (verified, Some(state), errors)
            }
            Err(e) => {
                errors.push(format!("Eval error: {e}"));
                (false, None, errors)
            }
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Restorator Verification and Default Sealing )
// ============================================================================
impl Default for Restorator {
    fn default() -> Self {
        Self::new(DiagnosticLogger::new(false))
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/mod.rs RS 32 LINES 132 WORDS 1.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Cogs Module Identity & Mechanical Dynamic Primitives )
// ============================================================================
//! OmniCode Cogs: Dynamic Mechanical Primitives & Decoupled Functions
//!
//! Reusable single-purpose dynamic utilities shared across gears:
//! - Dynamic host platform detection & environment adaptation (`platform`)
//! - Universal invariant tripwires and diagnostic interceptors (`tripwire`)
//! - Cross-track runtime path resolution

pub mod platform;
pub mod tripwire;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Cog Declarations & Re-exports )
// ============================================================================
pub use platform::{HostPlatform, HostArch};
pub use tripwire::{Tripwire, TripwireError};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Cog Implementations & Shared Primitives )
// ============================================================================

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing )
// ============================================================================
// Cogs subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/platform.rs RS 940 LINES 4296 WORDS 42.8 KB

```
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Dynamic Host Platform Cog Manifest, Scheduling Contract & Tripwire Registry )
// ============================================================================
//! MODULE:              omnicode::cogs::platform
//! IDENTITY:            Dynamic Host Platform Adapter Cog
//! NATURE:              Dynamic Environmental Meshing (Runtime Host Adaptation & In-Waiting Symbiosis)
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO},
<TRACK 3> : {Studio & Graphical IDE} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Host Environmental Adapter & Hardware Abstraction Layer (HAL)
//!   WHAT:              Dynamically detect host OS and CPU architecture, resolve toolchains and hypervisors,
//!                      probe hardware virtualization acceleration, discover UEFI OVMF firmware blobs,
//!                      audit host display servers, and orchestrate tripwired subprocess execution.
//!   WHERE:             Runtime host environment (GNU/Linux, Apple macOS Darwin, Microsoft Windows NT, Generic POSIX)
//!   WHEN:              Evaluated at engine startup, CLI command dispatch, GUI studio launch, or bare-metal boot
inception.
//!   WHY:               Enable the "In-Waiting" symbiotic execution model: OmniCode is an autonomous
//!                      operating system and balanced-ternary language, but during incubation it executes
//!                      harmoniously on any host OS without hardcoding platform paths or failing when moved.
//!   HOW:               Balanced Ternary (+1:PROCEED host detected & certified, -1:ROLLBACK unsupported/missing, 0:AWAIT
probe)
//!
//! THE "IN-WAITING" SYMBIOTIC INCUBATION MODEL:
//!   OmniCode is not merely a programming language compiler; it is an entirely distinct computing paradigm,
//!   operating system, and balanced-ternary machine architecture. However, during its developmental incubation,
//!   engineers work inside conventional operating systems (Linux, macOS, Windows).
//!   This cog acts as the umbilical cord and Hardware Abstraction Layer (HAL):
//!   - Adapts to the host's native filesystem hierarchy (FHS vs Homebrew vs Windows Program Files).
//!   - Selects the correct hypervisor binary (`qemu-system-x86_64` vs `qemu-system-x86_64.exe`).
//!   - Selects native acceleration flags (`-enable-kvm` on Linux, `-accel hvf` on macOS, `-accel whpx` on Windows).
//!   - Resolves native graphical browser dispatchers (`xdg-open` vs `open` vs `cmd /C start`).
//!   - Detects headless display environments to prevent graphical crashes.
//!
//! GAME DEV IDEOLOGY (THE HAL & PHYSICS SIMULATION ANALOGY):
//!   In modern game engine architecture (e.g., Unreal, Unity, Godot), the game simulation tick (`Update()`)
//!   runs identically regardless of whether rendering is powered by DirectX 12, Metal, or Vulkan.
//!   In OmniCode:
//!   - The Core Simulation: Word32 balanced-ternary ALU, tape reels, SVO grammar, and QuadState causality.
//!   - The Platform Adapter (This Cog): The cross-platform driver layer abstracting host silicon and OS syscalls.
//!   - Tripwires: Spatial and temporal collision volumes. When an invariant fails (missing binary, unreadable
//!     KVM node, absent display server), the tripwire catches the collision, records frame telemetry to
//!     `DiagnosticLogger` and `OmniDebugger`, and passes remediation tokens to `Restorator` for autonomous self-healing.
//!
//! TOUCHES / SYSTEM & PROCESS FOOTPRINT:
//!   - DISPATCH:        Native browser openers (`xdg-open` on Linux, `open` on macOS, `cmd /C start` on Windows)
//!   - DISCOVERY:       Host filesystem firmware lookups (`/usr/share/qemu`, `/opt/homebrew`, `C:\Program Files\qemu`)
//!   - BINARIES:        Probes for `qemu-system-x86_64` vs `qemu-system-x86_64.exe` on host PATH
//!   - VIRTUALIZATION:  Probes `/dev/kvm` (Linux KVM), Hypervisor.framework (macOS HVF), WHPX (Windows), TCG (Fallback)
//!   - DISPLAY:         Probes `$DISPLAY`, `$WAYLAND_DISPLAY`, and window manager sessions for GUI capability.
//!   - PROCESSES:       Uniform child process spawning with standard stream capture and exit status checking.
//!
//! REQUIRES / HOST CAPABILITIES:
//!   - Rust `cfg!(target_os)` and `cfg!(target_arch)` target family introspection.
//!   - Subprocess execution capabilities (`std::process::Command`).
//!   - Filesystem metadata and permission inspection (`std::path::Path::exists`, `std::fs::OpenOptions`).
//!   - Environment variable inspection (`std::env::var_os`, `std::env::var`).
//!
//! TRIPWIRE REGISTRY:
//!   - TRIPWIRE_PLATFORM_PROBE       (Identification and verification of host platform plane)
//!   - TRIPWIRE_ARCH_SUPPORT         (Assertion of host CPU architecture compatibility with target)
//!   - TRIPWIRE_FIRMWARE_DISCOVERY   (Resolution and verification of valid OVMF firmware candidate)
//!   - TRIPWIRE_HYPERVISOR_AVAIL     (Assertion of hypervisor executable on host PATH)
```

```rust
//!   - TRIPWIRE_ACCEL_DEVICE         (Hardware virtualization node accessibility / permission check)
//!   - TRIPWIRE_DISPLAY_SERVER       (Assertion of active display server when GUI mode is requested)
//!   - TRIPWIRE_BROWSER_DISPATCH     (Successful handoff to host graphical browser)
//!   - TRIPWIRE_SUBPROCESS_EXEC      (Generic host child process execution and exit code verification)
//!
//! NAVIGATION GUIDE (HOW TO NAVIGATE AND EXTEND THIS MODULE):
//!   - Block 1 (Lines 1-90): Module Manifest, Incubation Philosophy, Game Dev Analogy, Tripwire Registry.
//!   - Block 2 (Lines 91-180): Scoped Imports, Multi-OS Firmware Matrices, Equilibrium Constants, HostArch Enum.
//!     * To add firmware search paths for a new Linux distro or BSD: edit `LINUX_OVMF_CANDIDATES` in Section 2.2.
//!     * To add a new CPU architecture: add a variant to `HostArch` in Section 2.4.
//!   - Block 3 (Lines 181-480): Body Block & Implementation Oracles.
//!     * 3.1: HostPlatform runtime classification, display names, and environment summary.
//!     * 3.2: Host PATH resolution (`resolve_in_path`) and hypervisor binary verification.
//!     * 3.3: Hardware virtualization acceleration probes (`has_hardware_acceleration`, `assert_acceleration_supported`).
//!     * 3.4: Host display server & GUI environment oracles (`has_display_server`, `assert_display_server_available`).
//!     * 3.5: UEFI OVMF firmware search with telemetry and tripwires (`find_ovmf_firmware_with_telemetry`).
//!     * 3.6: Host process dispatch, child execution, and browser launching (`execute_host_command`).
//!   - Block 4 (Lines 481+): Unit tests verifying platform detection, binary resolution, PATH parsing, and telemetry.

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Environment Definitions, Search Matrices, Architecture & Constants )
// ============================================================================

// ----------------------------------------------------------------------------
// 2.1. Scoped Namespace Ingestion & System Imports
// ----------------------------------------------------------------------------
use std::env;
use std::fs::OpenOptions;
use std::path::{Path, PathBuf};
use std::process::{Command, Output};

use crate::cogs::tripwire::{Tripwire, TripwireError};
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion};

// ----------------------------------------------------------------------------
// 2.2. Multi-Platform Firmware Candidate Matrices & Search Registries
// ----------------------------------------------------------------------------
/// Canonical Linux OVMF UEFI firmware candidate search paths.
///
/// Covers major distribution layouts:
/// - Debian / Ubuntu: `/usr/share/ovmf/OVMF.fd`
/// - Arch Linux / Fedora / RHEL: `/usr/share/edk2-ovmf/x64/OVMF_CODE.fd`, `/usr/share/edk2/ovmf/OVMF_CODE.fd`
/// - NixOS / Alpine / OpenSUSE: `/usr/share/qemu/OVMF.fd`, `/usr/share/OVMF/OVMF_CODE_4M.fd`
pub const LINUX_OVMF_CANDIDATES: &[&str] = &[
    "/usr/share/qemu/OVMF.fd",
    "/usr/share/OVMF/OVMF_CODE_4M.fd",
    "/usr/share/ovmf/OVMF.fd",
    "/usr/share/edk2-ovmf/x64/OVMF_CODE.fd",
    "/usr/share/edk2/ovmf/OVMF_CODE.fd",
];

/// Canonical Apple macOS OVMF firmware search paths.
///
/// Covers Homebrew Apple Silicon (`/opt/homebrew`), Intel Mac (`/usr/local`),
/// and MacPorts (`/opt/local`) installations of QEMU:
pub const MACOS_OVMF_CANDIDATES: &[&str] = &[
    "/opt/homebrew/share/qemu/edk2-x86_64-code.fd",
    "/usr/local/share/qemu/edk2-x86_64-code.fd",
    "/opt/homebrew/Cellar/qemu/share/qemu/edk2-x86_64-code.fd",
    "/opt/local/share/qemu/edk2-x86_64-code.fd",
];

/// Canonical Microsoft Windows OVMF firmware search paths.
///
/// Covers standard 64-bit and 32-bit QEMU installer directories under `Program Files`:
pub const WINDOWS_OVMF_CANDIDATES: &[&str] = &[
    "C:\\Program Files\\qemu\\share\\edk2-x86_64-code.fd",
    "C:\\Program Files (x86)\\qemu\\share\\edk2-x86_64-code.fd",
    "C:\\qemu\\share\\edk2-x86_64-code.fd",
];

/// Fallback generic firmware search paths for unrecognized POSIX environments.
pub const FALLBACK_OVMF_CANDIDATES: &[&str] = &[
    "/usr/share/qemu/OVMF.fd",
    "/usr/local/share/qemu/OVMF.fd",
];
```

```rust

// ---------------------------------------------------------------------------
// 2.3. Initial Truth States & Equilibrium Platform Defaults
// ---------------------------------------------------------------------------
/// Default hypervisor binary for POSIX-compliant host environments.
pub const DEFAULT_UNIX_HYPERVISOR: &str = "qemu-system-x86_64";

/// Default hypervisor binary for Microsoft Windows host environments.
pub const DEFAULT_WINDOWS_HYPERVISOR: &str = "qemu-system-x86_64.exe";

/// Canonical Linux KVM device node for hardware virtualization.
pub const LINUX_KVM_DEVICE_NODE: &str = "/dev/kvm";

// ---------------------------------------------------------------------------
// 2.4. Host Silicon Architecture Typology
// ---------------------------------------------------------------------------
/// Physical host CPU instruction set architecture.
///
/// Identifies the silicon substrate upon which the host operating system executes.
/// Informs hypervisor hardware acceleration compatibility (e.g. KVM or HVF can only
/// be used when host architecture matches guest target architecture).
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum HostArch {
    /// 64-bit x86 architecture (AMD64 / Intel 64).
    X86_64,
    /// 64-bit ARM architecture (Apple Silicon M-series, AWS Graviton, Ampere Altra).
    Aarch64,
    /// 64-bit RISC-V architecture.
    RiscV64,
    /// Unidentified or unsupported host processor architecture.
    Unknown,
}

impl HostArch {
    /// Detects the host processor architecture at runtime using target architecture directives.
    pub fn current() -> Self {
        if cfg!(target_arch = "x86_64") {
            HostArch::X86_64
        } else if cfg!(target_arch = "aarch64") {
            HostArch::Aarch64
        } else if cfg!(target_arch = "riscv64") {
            HostArch::RiscV64
        } else {
            HostArch::Unknown
        }
    }

    /// Returns the human and machine readable designation of the CPU architecture.
    pub fn display_name(&self) -> &'static str {
        match self {
            HostArch::X86_64 => "x86_64 (AMD64 / Intel 64)",
            HostArch::Aarch64 => "aarch64 (ARM64 / Apple Silicon)",
            HostArch::RiscV64 => "riscv64 (RISC-V 64-Bit)",
            HostArch::Unknown => "unknown-arch (Generic Processor)",
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Dynamic Host Platform Implementation, Environmental Oracles & Process Dispatch )
// ============================================================================

// ---------------------------------------------------------------------------
// 3.1. Platform Runtime Detection & Classification
// ---------------------------------------------------------------------------
/// Host operating system classification for "In-Waiting" symbiotic execution.
///
/// Represents the physical runtime habitat in which the OmniCode host engine
/// and developer tooling incubate. Enables dynamic adaptation of hypervisors,
/// firmware paths, native process dispatchers, and hardware acceleration flags.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum HostPlatform {
    /// GNU/Linux host (KVM hardware virtualization, systemd/FHS filesystem layout).
    Linux,
    /// Apple macOS host (Darwin kernel, Hypervisor.framework / HVF, Homebrew paths).
    MacOS,
    /// Microsoft Windows host (NT kernel, PE32+ binaries, Windows Hypervisor Platform / WHPX).
```

```rust
    Windows,
    /// Unidentified or generic POSIX environment (BSD, Solaris, embedded runtime).
    Unknown,
}

impl HostPlatform {
    /// Dynamically detects the host operating system platform at runtime.
    ///
    /// Evaluates compile-time target family directives (`cfg!(target_os)`) while
    /// providing runtime verification hooks.
    pub fn current() -> Self {
        if cfg!(target_os = "linux") {
            HostPlatform::Linux
        } else if cfg!(target_os = "macos") {
            HostPlatform::MacOS
        } else if cfg!(target_os = "windows") {
            HostPlatform::Windows
        } else {
            HostPlatform::Unknown
        }
    }

    /// Detects the physical host CPU architecture.
    pub fn arch(&self) -> HostArch {
        HostArch::current()
    }

    /// Returns the canonical platform display title with incubation role.
    pub fn display_name(&self) -> &'static str {
        match self {
            HostPlatform::Linux => "GNU/Linux (Host Incubation Plane)",
            HostPlatform::MacOS => "Apple macOS (Darwin Incubation Plane)",
            HostPlatform::Windows => "Microsoft Windows (NT Incubation Plane)",
            HostPlatform::Unknown => "Generic Host Plane (Fallback Environment)",
        }
    }

    /// Returns the target operating system family name (`"unix"` or `"windows"`).
    pub fn target_os_family(&self) -> &'static str {
        match self {
            HostPlatform::Windows => "windows",
            _ => "unix",
        }
    }

    /// Returns `true` if the host platform adheres to POSIX standards (Linux, macOS, BSD).
    pub fn is_posix(&self) -> bool {
        !matches!(self, HostPlatform::Windows)
    }

    /// Returns `true` if the host platform is Microsoft Windows.
    pub fn is_windows(&self) -> bool {
        matches!(self, HostPlatform::Windows)
    }

    /// Returns `true` if the host platform is GNU/Linux.
    pub fn is_linux(&self) -> bool {
        matches!(self, HostPlatform::Linux)
    }

    /// Returns `true` if the host platform is Apple macOS Darwin.
    pub fn is_macos(&self) -> bool {
        matches!(self, HostPlatform::MacOS)
    }

    /// Produces a comprehensive, machine-readable diagnostic snapshot of the host habitat.
    ///
    /// Used by [`DiagnosticLogger`], [`crate::diagnostics::debugger::OmniDebugger`], and AI cortex
    /// to record environmental state in episodic logs.
    pub fn environment_summary(&self) -> String {
        format!(
            "HostPlatform: {} | Arch: {} | Family: {} | POSIX: {} | Accel: {} | Display: {}",
            self.display_name(),
            self.arch().display_name(),
            self.target_os_family(),
            self.is_posix(),
            self.has_hardware_acceleration(),
            self.has_display_server()
```

```
        )
    }
}

// ---------------------------------------------------------------------------
// 3.2. Host PATH Resolution & Toolchain Binary Oracles
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Searches the host system `PATH` environment variable for an executable by name.
    ///
    /// # Platform Mechanics:
    /// - Respects platform-specific path list separators (`:` on Unix/POSIX, `;` on Windows).
    /// - On Windows, automatically tests the filename with `.exe` if not already provided.
    /// - Validates that candidate files exist and are regular files.
    pub fn resolve_in_path(&self, binary: &str) -> Option<PathBuf> {
        let path_var = env::var_os("PATH")?;

        let candidate_names: Vec<String> = if self.is_windows() && !binary.ends_with(".exe") {
            vec![format!("{}.exe", binary), binary.to_string()]
        } else {
            vec![binary.to_string()]
        };

        for dir in env::split_paths(&path_var) {
            for name in &candidate_names {
                let candidate = dir.join(name);
                if candidate.is_file() {
                    return Some(candidate);
                }
            }
        }

        None
    }

    /// Returns the platform-appropriate primary hypervisor binary name.
    pub fn hypervisor_binary(&self) -> &'static str {
        match self {
            HostPlatform::Windows => DEFAULT_WINDOWS_HYPERVISOR,
            _ => DEFAULT_UNIX_HYPERVISOR,
        }
    }

    /// Returns an ordered list of candidate hypervisor executable names.
    pub fn hypervisor_candidates(&self) -> &'static [&'static str] {
        match self {
            HostPlatform::Windows => &[DEFAULT_WINDOWS_HYPERVISOR, "qemu.exe"],
            HostPlatform::Linux => &[DEFAULT_UNIX_HYPERVISOR, "qemu-kvm", "kvm"],
            HostPlatform::MacOS => &[DEFAULT_UNIX_HYPERVISOR],
            HostPlatform::Unknown => &[DEFAULT_UNIX_HYPERVISOR],
        }
    }

    /// Dynamically locates the first valid hypervisor binary present on the host system PATH.
    pub fn find_hypervisor_binary(&self) -> Option<PathBuf> {
        for &candidate in self.hypervisor_candidates() {
            if let Some(path) = self.resolve_in_path(candidate) {
                return Some(path);
            }
        }
        None
    }

    /// Asserts that a valid hypervisor binary is present and reachable on the host PATH.
    ///
    /// # Sensory Tripwire:
    /// Trips [`Tripwire::assert_binary_available`], recording a failpoint in the logger
    /// with actionable installation remediation for Linux, macOS, or Windows if missing.
    pub fn assert_hypervisor_available(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<PathBuf, TripwireError> {
        let binary = self.hypervisor_binary();

        match self.resolve_in_path(binary) {
            Some(resolved_path) => {
                logger.log(
                    "PLATFORM",
```

```
                        format!("Hypervisor binary verified on PATH: {:?}", resolved_path),
                    );
                    Ok(resolved_path)
                }
                None => {
                    let install_hint = match self {
                        HostPlatform::Linux => "sudo apt install qemu-system-x86 (Debian/Ubuntu) or sudo pacman -S qemu-system-x86 (Arch)",
                        HostPlatform::MacOS => "brew install qemu (Homebrew) or port install qemu (MacPorts)",
                        HostPlatform::Windows => "winget install QEMU.QEMU or download from https://www.qemu.org/download/#windows",
                        HostPlatform::Unknown => "Install QEMU from your operating system package repository",
                    };

                    let suggestion = FixSuggestion {
                        kind: FixKind::AddImport,
                        line: 1,
                        col: 1,
                        target_token: binary.to_string(),
                        suggested_patch: install_hint.to_string(),
                        rationale: format!(
                            "Host hypervisor executable '{}' is required for bare-metal machine inception",
                            binary
                        ),
                    };

                    let err = Tripwire::trip_with_remediation(
                        logger,
                        "PLATFORM",
                        FailpointType::SyntaxAnomaly,
                        format!(
                            "Hypervisor binary '{}' not found on host PATH. Remediation: {}",
                            binary, install_hint
                        ),
                        suggestion,
                    );
                    Err(err)
                }
            }
        }
    }

    // ---------------------------------------------------------------------------
    // 3.3. Virtualization Acceleration Oracles & Permission Probes
    // ---------------------------------------------------------------------------
    impl HostPlatform {
        /// Probes whether hardware virtualization acceleration is accessible on the host.
        ///
        /// # Physical Hardware Invariants:
        /// - **Linux**: Inspects existence and read/write accessibility of `/dev/kvm`.
        /// - **macOS**: Target family supports Hypervisor.framework (`hvf`) on Darwin when
        ///   running on matching architecture or Apple Silicon hypervisor.
        /// - **Windows**: Target family supports Windows Hypervisor Platform (`whpx`).
        pub fn has_hardware_acceleration(&self) -> bool {
            match self {
                HostPlatform::Linux => {
                    let kvm_path = Path::new(LINUX_KVM_DEVICE_NODE);
                    if !kvm_path.exists() {
                        return false;
                    }
                    // Verify read and write permissions to /dev/kvm
                    OpenOptions::new()
                        .read(true)
                        .write(true)
                        .open(kvm_path)
                        .is_ok()
                }
                HostPlatform::MacOS => true,
                HostPlatform::Windows => false, // Conservative default; requires Hyper-V / WHPX feature check
                HostPlatform::Unknown => false,
            }
        }

        /// Asserts that hardware acceleration is available, logging telemetry or tripping an alert.
        ///
        /// # Game Dev Collision Volume Analogy:
        /// If an acceleration tripwire is armed, tripping here warns that execution will drop
        /// into software emulation (TCG), which incurs significant performance overhead.
```

```
    pub fn assert_acceleration_supported(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<bool, TripwireError> {
        let accel = self.has_hardware_acceleration();
        if accel {
            logger.log(
                "ACCELERATION",
                format!("Hardware virtualization acceleration active on {}", self.display_name()),
            );
            Ok(true)
        } else {
            logger.record_ai_observation(
                "ACCELERATION_ABSENT",
                "Hardware virtualization acceleration unavailable",
                "Falling back to Tiny Code Generator (TCG) software emulation",
                ["virtualization", "tcg", "fallback", "performance"].as_slice(),
                None,
            );
            Ok(false)
        }
    }

    /// Returns the recommended hypervisor command-line arguments for hardware acceleration.
    ///
    /// Falls back to pure TCG software emulation if acceleration is unavailable.
    pub fn recommended_accel_args(&self) -> &'static [&'static str] {
        match self {
            HostPlatform::Linux if self.has_hardware_acceleration() => &["-enable-kvm"],
            HostPlatform::MacOS => &["-accel", "hvf"],
            HostPlatform::Windows if self.has_hardware_acceleration() => &["-accel", "whpx"],
            _ => &["-accel", "tcg"],
        }
    }
}

// ---------------------------------------------------------------------------
// 3.4. Host Display Server & Graphical Subsystem Oracles
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Probes whether a graphical display server or window manager is active on the host.
    ///
    /// Prevents GUI window spawn crashes when running in headless environments
    /// (e.g. continuous integration servers, SSH remote sessions, or containers).
    pub fn has_display_server(&self) -> bool {
        match self {
            HostPlatform::Linux | HostPlatform::Unknown => {
                env::var("DISPLAY").is_ok() || env::var("WAYLAND_DISPLAY").is_ok()
            }
            HostPlatform::MacOS => {
                // macOS GUI window server is available in standard desktop user sessions
                env::var("TERM_PROGRAM").is_ok() || env::var("Apple_PubSub_Socket_Render").is_ok() || true
            }
            HostPlatform::Windows => {
                // Windows desktop GUI is standard unless running in Windows Server Core
                true
            }
        }
    }

    /// Asserts that a graphical display server is available when GUI mode is requested.
    ///
    /// # Sensory Tripwire:
    /// If running in a headless environment and GUI display is requested, trips a wire
    /// recommending headless serial mode (`--display none`).
    pub fn assert_display_server_available(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<(), TripwireError> {
        if self.has_display_server() {
            logger.log("DISPLAY", "Graphical display server verified active (+1:PROCEED)");
            Ok(())
        } else {
            let suggestion = FixSuggestion {
                kind: FixKind::AdjustCaveat,
                line: 1,
                col: 1,
                target_token: "--gui".to_string(),
```

```
                suggested_patch: "--headless".to_string(),
                rationale: "Host display environment has no active DISPLAY or WAYLAND_DISPLAY session".to_string(),
            };

            let err = Tripwire::trip_with_remediation(
                logger,
                "DISPLAY",
                FailpointType::SyntaxAnomaly,
                "No graphical display server detected (headless environment). Run without --gui.".to_string(),
                suggestion,
            );
            Err(err)
        }
    }
}

// ---------------------------------------------------------------------------
// 3.5. UEFI Firmware Search Oracles & Telemetry Traversal
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Returns the candidate search paths for UEFI OVMF firmware images on this platform.
    pub fn ovmf_search_paths(&self) -> &'static [&'static str] {
        match self {
            HostPlatform::Linux => LINUX_OVMF_CANDIDATES,
            HostPlatform::MacOS => MACOS_OVMF_CANDIDATES,
            HostPlatform::Windows => WINDOWS_OVMF_CANDIDATES,
            HostPlatform::Unknown => FALLBACK_OVMF_CANDIDATES,
        }
    }

    /// Dynamically locates the first existing OVMF firmware image on the host system.
    pub fn find_ovmf_firmware(&self) -> Option<PathBuf> {
        // First inspect custom override environment variable if provided
        if let Ok(env_path) = env::var("OMNICODE_OVMF_PATH") {
            let p = PathBuf::from(env_path);
            if p.exists() {
                return Some(p);
            }
        }

        for &path_str in self.ovmf_search_paths() {
            let path = Path::new(path_str);
            if path.exists() {
                return Some(path.to_path_buf());
            }
        }
        None
    }

    /// Traverses the platform's firmware candidate list while logging telemetry events.
    ///
    /// # Telemetry Inscription:
    /// Inscribes candidate probe entries into [`DiagnosticLogger`]. If a firmware blob
    /// is resolved, logs success. If none are found, registers an informational diagnostic
    /// note equipped with remediation guidance for the Restorator and AI agents.
    pub fn find_ovmf_firmware_with_telemetry(&self, logger: &DiagnosticLogger) -> Option<PathBuf> {
        logger.log("FIRMWARE", format!("Probing OVMF firmware candidates on {}", self.display_name()));

        if let Ok(env_override) = env::var("OMNICODE_OVMF_PATH") {
            let p = PathBuf::from(env_override);
            if p.exists() {
                logger.log("FIRMWARE", format!("Resolved OVMF firmware via OMNICODE_OVMF_PATH: {:?}", p));
                return Some(p);
            }
        }

        for &path_str in self.ovmf_search_paths() {
            let path = Path::new(path_str);
            if path.exists() {
                logger.log("FIRMWARE", format!("Resolved OVMF firmware at: {:?}", path));
                return Some(path.to_path_buf());
            } else {
                logger.log("FIRMWARE", format!("Candidate not found: {}", path_str));
            }
        }

        logger.record_ai_observation(
            "FIRMWARE_UNRESOLVED",
```

```
            "No standard OVMF firmware image found on host",
            "UEFI boot will require explicit --ovmf path or fallback to BIOS MBR mode",
            ["firmware", "ovmf", "uefi", "fallback"].as_slice(),
            None,
        );

        None
    }

    /// Asserts that a valid OVMF firmware blob exists on disk, tripping a wire if absent.
    ///
    /// # Precondition Tripwire:
    /// If no candidate exists, trips a [`TripwireError`] equipped with a [`FixSuggestion`]
    /// instructing how to install or specify the firmware image.
    pub fn assert_ovmf_firmware(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<PathBuf, TripwireError> {
        if let Some(firmware) = self.find_ovmf_firmware_with_telemetry(logger) {
            Ok(firmware)
        } else {
            let suggestion = FixSuggestion {
                kind: FixKind::AddImport,
                line: 1,
                col: 1,
                target_token: "--uefi".to_string(),
                suggested_patch: "--bios".to_string(),
                rationale: "Fallback to Legacy BIOS MBR mode or install ovmf package on host system".to_string(),
            };

            let err = Tripwire::trip_with_remediation(
                logger,
                "FIRMWARE",
                FailpointType::SyntaxAnomaly,
                format!("OVMF UEFI firmware image not found on {}", self.display_name()),
                suggestion,
            );
            Err(err)
        }
    }
}

// ---------------------------------------------------------------------------
// 3.6. Host OS Process Dispatch & Native Subprocess Execution
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Constructs the platform-appropriate [`Command`] for opening a target URL in the default browser.
    pub fn format_browser_command(&self, url: &str) -> Command {
        match self {
            HostPlatform::Linux => {
                let mut cmd = Command::new("xdg-open");
                cmd.arg(url);
                cmd
            }
            HostPlatform::MacOS => {
                let mut cmd = Command::new("open");
                cmd.arg(url);
                cmd
            }
            HostPlatform::Windows => {
                let mut cmd = Command::new("cmd");
                cmd.args(["/C", "start", url]);
                cmd
            }
            HostPlatform::Unknown => {
                let mut cmd = Command::new("xdg-open");
                cmd.arg(url);
                cmd
            }
        }
    }

    /// Dynamically launches the default web browser on the host system for a target URL.
    pub fn open_browser(&self, url: &str) {
        let mut cmd = self.format_browser_command(url);
        let _ = cmd.spawn();
    }

```

```
    /// Launches the default browser wrapped in diagnostic logging and tripwire interception.
    pub fn open_browser_with_tripwire(
        &self,
        logger: &DiagnosticLogger,
        url: &str,
    ) -> Result<(), TripwireError> {
        logger.log(
            "BROWSER_DISPATCH",
            format!("Dispatching browser on {} for URL: {}", self.display_name(), url),
        );
        let mut cmd = self.format_browser_command(url);
        match cmd.spawn() {
            Ok(_) => {
                logger.log("BROWSER_DISPATCH", "Browser child process dispatched cleanly (+1:PROCEED)");
                Ok(())
            }
            Err(e) => {
                let msg = format!("Failed to dispatch host browser for URL '{}': {}", url, e);
                Err(Tripwire::trip(logger, "BROWSER_DISPATCH", FailpointType::SyntaxAnomaly, msg))
            }
        }
    }

    /// Executes an arbitrary host command wrapped in tripwire collision detection and telemetry.
    ///
    /// # Game Dev Simulation Hook:
    /// Captures stdout and stderr streams. If the command exits with non-zero status code,
    /// logs an execution failure and trips a structured error for the Restorator.
    pub fn execute_host_command(
        &self,
        logger: &DiagnosticLogger,
        subsystem: &'static str,
        mut cmd: Command,
    ) -> Result<Output, TripwireError> {
        logger.log(subsystem, format!("Spawning host subprocess: {:?}", cmd));
        match cmd.output() {
            Ok(output) => {
                if output.status.success() {
                    logger.log(subsystem, "Host subprocess terminated with exit status 0 (+1:PROCEED)");
                    Ok(output)
                } else {
                    let code = output.status.code().unwrap_or(-1);
                    let stderr = String::from_utf8_lossy(&output.stderr);
                    let msg = format!(
                        "Host subprocess exited with non-zero status code {}: {}",
                        code, stderr.trim()
                    );
                    Err(Tripwire::trip(logger, subsystem, FailpointType::SyntaxAnomaly, msg))
                }
            }
            Err(e) => {
                let msg = format!("Failed to spawn host subprocess: {}", e);
                Err(Tripwire::trip(logger, subsystem, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing, Baton Pass & In-File Unit Tests )
// ============================================================================

// ----------------------------------------------------------------------------
// 4.1. Dynamic Adapter Sealing & Downstream Baton Pass Specification
// ----------------------------------------------------------------------------
// BATON PASS CONTRACT:
//   - Upstream Producer: Rust runtime and compile-time target introspection (`cfg!(target_os)`).
//   - Downstream Receivers:
//     * `<TRACK 1>` Boot Gear (`src/cli/commands/boot.rs`): Hypervisor binary resolution,
//       UEFI firmware lookup, display server probe, and acceleration flags.
//     * `<TRACK 3>` Studio IDE (`src/terminal/studio.rs`): Native browser dispatch and GUI server.
//     * `<TRACK 1>` REPL & Terminal (`src/terminal/session.rs`): Console color and terminal probes.
//   - Certified Invariants:
//     1. `HostPlatform::current()` and `HostArch::current()` return deterministic, valid variants.
//     2. Binary candidate matrices map strictly to executable names appropriate for the OS family.
//     3. `resolve_in_path` navigates host environment paths respecting platform path delimiters.
//     4. Browser command synthesizers construct platform-native dispatch subprocesses.
```

```
//     5. Virtualization acceleration flags accurately select KVM, HVF, WHPX, or fallback TCG.

#[cfg(test)]
mod tests {
    use super::*;

    /// Verifies that `HostPlatform::current()` dynamically resolves the active host operating system
    /// and supplies non-empty display names, binary targets, firmware matrices, and OS families.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents null or uninitialized platform definitions from entering downstream gears.
    #[test]
    fn test_platform_current_detection() {
        let p = HostPlatform::current();
        assert!(!p.display_name().is_empty(), "Display name must not be empty");
        assert!(!p.hypervisor_binary().is_empty(), "Hypervisor binary name must not be empty");
        assert!(!p.ovmf_search_paths().is_empty(), "Firmware search paths matrix must not be empty");
        assert!(!p.target_os_family().is_empty(), "OS family string must not be empty");
    }

    /// Verifies that `HostArch::current()` detects the host CPU instruction set architecture.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents silent fallback to mismatched architecture flags during hypervisor command synthesis.
    #[test]
    fn test_host_arch_detection() {
        let arch = HostArch::current();
        assert!(!arch.display_name().is_empty(), "CPU architecture display title must not be empty");
        let platform = HostPlatform::current();
        assert_eq!(platform.arch(), arch, "Platform arch must match HostArch::current");
    }

    /// Verifies that `environment_summary()` produces a comprehensive, well-formed diagnostic string
    /// containing all critical platform facets for episodic logging.
    ///
    /// # Invariants Tested:
    /// - Contains `"HostPlatform:"`, `"Arch:"`, `"Family:"`, `"POSIX:"`, `"Accel:"`, and `"Display:"`.
    #[test]
    fn test_environment_summary() {
        let p = HostPlatform::current();
        let summary = p.environment_summary();
        assert!(summary.contains("HostPlatform:"), "Summary must report HostPlatform");
        assert!(summary.contains("Arch:"), "Summary must report host architecture");
        assert!(summary.contains("Family:"), "Summary must report OS family");
        assert!(summary.contains("POSIX:"), "Summary must report POSIX compliance");
        assert!(summary.contains("Accel:"), "Summary must report hardware acceleration status");
        assert!(summary.contains("Display:"), "Summary must report display server availability");
    }

    /// Verifies the platform-to-binary mapping matrix across all supported operating system families.
    ///
    /// # Invariants Tested:
    /// - Windows requires `.exe` binary extension (`qemu-system-x86_64.exe`).
    /// - POSIX systems (Linux, macOS, Unknown) expect bare binary names (`qemu-system-x86_64`).
    #[test]
    fn test_platform_binaries() {
        assert_eq!(HostPlatform::Linux.hypervisor_binary(), "qemu-system-x86_64");
        assert_eq!(HostPlatform::Windows.hypervisor_binary(), "qemu-system-x86_64.exe");
        assert_eq!(HostPlatform::MacOS.hypervisor_binary(), "qemu-system-x86_64");
        assert_eq!(HostPlatform::Unknown.hypervisor_binary(), "qemu-system-x86_64");
    }

    /// Verifies the OS family classification and semantic predicate methods on `HostPlatform`.
    ///
    /// # Invariants Tested:
    /// - Linux: family `"unix"`, `is_posix() == true`, `is_linux() == true`, `is_windows() == false`.
    /// - macOS: family `"unix"`, `is_posix() == true`, `is_macos() == true`, `is_windows() == false`.
    /// - Windows: family `"windows"`, `is_posix() == false`, `is_windows() == true`.
    #[test]
    fn test_platform_os_family() {
        assert_eq!(HostPlatform::Linux.target_os_family(), "unix");
        assert!(HostPlatform::Linux.is_posix());
        assert!(HostPlatform::Linux.is_linux());
        assert!(!HostPlatform::Linux.is_windows());
        assert!(!HostPlatform::Linux.is_macos());

        assert_eq!(HostPlatform::MacOS.target_os_family(), "unix");
        assert!(HostPlatform::MacOS.is_posix());
```

```
        assert!(HostPlatform::MacOS.is_macos());
        assert!(!HostPlatform::MacOS.is_linux());

        assert_eq!(HostPlatform::Windows.target_os_family(), "windows");
        assert!(HostPlatform::Windows.is_windows());
        assert!(!HostPlatform::Windows.is_posix());
    }

    /// Verifies hardware virtualization acceleration command-line argument mapping.
    ///
    /// # Invariants Tested:
    /// - macOS maps to Apple Hypervisor.framework (`-accel hvf`).
    /// - Fallback/Unknown maps to software emulation (`-accel tcg`).
    /// - Linux provides acceleration flags (`-enable-kvm` or fallback).
    #[test]
    fn test_virtualization_accel_args() {
        let args_linux = HostPlatform::Linux.recommended_accel_args();
        assert!(!args_linux.is_empty(), "Linux acceleration argument list must not be empty");

        let args_macos = HostPlatform::MacOS.recommended_accel_args();
        assert_eq!(args_macos, &["-accel", "hvf"], "macOS must use Hypervisor.framework");

        let args_fallback = HostPlatform::Unknown.recommended_accel_args();
        assert_eq!(args_fallback, &["-accel", "tcg"], "Unknown platform must fall back to TCG");
    }

    /// Verifies native browser command synthesizer across Linux, macOS, and Windows.
    ///
    /// # Invariants Tested:
    /// - Linux dispatches `xdg-open <URL>`.
    /// - macOS dispatches `open <URL>`.
    /// - Windows dispatches `cmd /C start <URL>`.
    #[test]
    fn test_format_browser_command() {
        let test_url = "http://localhost:8080/studio";

        let cmd_linux = HostPlatform::Linux.format_browser_command(test_url);
        assert_eq!(cmd_linux.get_program(), "xdg-open");

        let cmd_macos = HostPlatform::MacOS.format_browser_command(test_url);
        assert_eq!(cmd_macos.get_program(), "open");

        let cmd_windows = HostPlatform::Windows.format_browser_command(test_url);
        assert_eq!(cmd_windows.get_program(), "cmd");
    }

    /// Verifies that `resolve_in_path` successfully resolves existing system binaries
    /// and correctly returns `None` for non-existent binaries.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents false positives or broken delimiter splitting when scanning host PATH.
    #[test]
    fn test_resolve_in_path_contract() {
        let platform = HostPlatform::current();

        // Standard shell binary must resolve on any functioning host
        let target = if platform.is_windows() { "cmd" } else { "sh" };
        let resolved = platform.resolve_in_path(target);
        assert!(resolved.is_some(), "Standard system shell must resolve on host PATH");
        let path = resolved.unwrap();
        assert!(path.is_file(), "Resolved PATH entry must point to a regular file");

        // Intentionally non-existent binary must return None
        let bogus = platform.resolve_in_path("definitely_nonexistent_binary_omnicode_xyz");
        assert!(bogus.is_none(), "Non-existent executable name must return None");
    }

    /// Verifies that candidate search path matrices across all platform variants
    /// are non-empty and contain valid non-empty path strings.
    ///
    /// # Invariants Tested:
    /// - Linux candidates, macOS candidates, Windows candidates, and Fallback candidates
    ///   must all be populated.
    #[test]
    fn test_ovmf_search_paths_contract() {
        for platform in &[HostPlatform::Linux, HostPlatform::MacOS, HostPlatform::Windows, HostPlatform::Unknown] {
            let paths = platform.ovmf_search_paths();
            assert!(!paths.is_empty(), "Platform {:?} must declare firmware candidates", platform);
```

```
            for &p in paths {
                assert!(!p.is_empty(), "Firmware candidate path string must not be empty");
            }
        }
    }

    /// Verifies that the display server probe evaluates without panic.
    #[test]
    fn test_display_server_probe_contract() {
        let platform = HostPlatform::current();
        let _has_display = platform.has_display_server();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/tripwire.rs RS 628 LINES 2781 WORDS 28.2 KB

```
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Invariant Tripwire Cog Manifest & Cybernetic Sensory Boundaries )
// ============================================================================
//! MODULE:              omnicode::cogs::tripwire
//! IDENTITY:            Universal Invariant Tripwire Sensor Cog
//! NATURE:              Dynamic Boundary Protection, Collision Detection & Diagnostic Interception
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO},
<TRACK 3> : {Studio & Graphical IDE} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Universal Sensory Invariant Tripwire Subsystem
//!   WHAT:              Trap invariant violations, record structured failpoint events,
//!                      equip remediation suggestions, and summon the debugger and restorator.
//!   WHERE:             Enforced across all modules, CLI gears, platform adapters, terminal sessions, and scheduler
tasks.
//!   WHEN:              Evaluated at precondition boundaries, file accesses, subprocess invocations, and state
transitions.
//!   WHY:               Eliminate unhandled panics and raw process aborts, converting crashes into recoverable
//!                      diagnostic signals that trigger transaction rollbacks (`-1:ROLLBACK`) or automated self-healing.
//!   HOW:               Balanced Ternary (+1:PROCEED invariant held, -1:ROLLBACK invariant breached with telemetry,
0:AWAIT standby).
//!
//! THE GAME DEV COLLISION TRIGGER PARADIGM:
//!   In modern game engine architecture (e.g., Unreal, Unity, Godot):
//!   - Invisible Trigger Volumes: Spatial and physical boundary colliders placed in world space. When an entity
//!     penetrates an illegal zone or invalid state, the physics engine does not crash the game loop; it fires an
//!     `OnTriggerEnter` or `OnCollision` event callback.
//!   - In OmniCode: Tripwires are the cognitive and computational collision triggers. Instead of allowing undefined
//!     behavior, unhandled null states, absent silicon binaries, missing kernel images, or corrupted filesystems to
//!     panic the engine (`panic!`), a tripwire immediately catches the boundary collision.
//!
//! CYBERNETIC FAILPOINT & TRANSLATION ENVELOPE:
//!   - Translates raw runtime anomalies into formal ontological failpoint categories ([`FailpointType`]).
//!   - Emits microsecond-timestamped telemetry directly into the active [`DiagnosticLogger`] ring buffer and
[`crate::diagnostics::debugger::OmniDebugger`].
//!   - Inscribes episodic cognitive memory notes tagged `#failpoint` for AI reasoning models and developers.
//!   - Replaces uncontrolled stack unwinding with balanced-ternary quad-state causality (`-1:ROLLBACK` or `0:AWAIT`).
//!
//! THE RESTORATOR HANDOFF & REPAIR TELEMETRY:
//!   - When a tripwire trips with remediation ([`Tripwire::trip_with_remediation`]), it equips the trapped event
//!     with a [`FixSuggestion`] and registers `#remediation_ready` telemetry.
//!   - The [`crate::diagnostics::restorator::Restorator`] (OmniCode's autonomous immune system) ingests this structured
//!     flight data, executes surgical AST/token patches, test-certifies the candidate repair, and restores system
balance.
//!
//! TOUCHES / SYSTEM & PROCESS FOOTPRINT:
//!   - TELEMETRY:       Direct emission to [`DiagnosticLogger`] and active
[`crate::diagnostics::debugger::OmniDebugger`].
//!   - TERMINAL:        Standardized ANSI red/yellow tripwire alert streams to stderr and session logs.
//!   - RECOVERY:        Integrates with transaction rollbacks and automated Restorator immune responses.
//!   - SUBPROCESSES:    Monitors exit status codes and standard streams of external host child processes.
//!
//! REQUIRES / HOST CAPABILITIES:
//!   - Standard error stream access (`std::io::stderr`).
//!   - Filesystem metadata inspection (`std::path::Path::exists`, `std::path::Path::is_dir`, `std::fs::metadata`).
//!   - Subprocess status inspection (`std::process::ExitStatus`).
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_PATH_EXISTENCE`       (Assertion of critical filesystem resources and kernel artifacts)
//!   - `TRIPWIRE_DIR_EXISTENCE`        (Assertion of required workspace and directory hierarchies)
//!   - `TRIPWIRE_BINARY_AVAILABLE`     (Assertion of required external host toolchains on system PATH)
//!   - `TRIPWIRE_COMMAND_STATUS`       (Assertion of zero exit status on host subprocesses)
//!   - `TRIPWIRE_STATE_VIOLATION`      (Assertion of semantic invariant boundaries and quad-state commitments)
```

```rust
//!   - `TRIPWIRE_BOOLEAN_INVARIANT`    (Assertion of mathematical and ontological propositional truths)
//!
//! NAVIGATION GUIDE (HOW TO NAVIGATE AND EXTEND THIS COG AS THE CODE GROWS):
//!   - Block 1 (Lines 1-85): Manifest, Game Dev Trigger Analogy, Telemetry Envelope, Tripwire Registry.
//!   - Block 2 (Lines 86-175): Scoped Imports, Error Envelopes (`TripwireError`), Display Formatting & Defaults.
//!     * To add custom recovery constructors: add methods to `TripwireError` in Section 2.2.
//!     * To modify equilibrium recovery defaults: review constants in Section 2.3.
//!   - Block 3 (Lines 176-350): Body Block & Sensory Assertion Oracles.
//!     * 3.1: Explicit Failpoint Inscription & Remediation Oracles (`trip`, `trip_with_remediation`, `trip_await`).
//!     * 3.2: Filesystem Spatial Invariant Assertions (`assert_path_exists`, `assert_dir_exists`, `assert_file_readable`).
//!     * 3.3: Host Toolchain & Binary Availability Oracles (`assert_binary_available`).
//!     * 3.4: Subprocess Lifecycle & Exit Status Assertions (`assert_command_success`, `assert_command_output_clean`).
//!     * 3.5: Mathematical, State Machine & Causality Boundary Assertions (`assert_quad_state_proceed`, `assert_invariant`).
//!   - Block 4 (Lines 351+): Module Sealing, Baton Pass Protocol & In-File Unit Verification Tests.

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Invariant Data Types & Tripwire Error Definitions )
// ============================================================================

// ----------------------------------------------------------------------------
// 2.1. Scoped Namespace Ingestion & System Imports
// ----------------------------------------------------------------------------
use std::fmt;
use std::fs;
use std::path::{Path, PathBuf};
use std::process::{Command, ExitStatus, Output};

use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixSuggestion};

// ----------------------------------------------------------------------------
// 2.2. Invariant Data Types & Tripwire Error Definitions
// ----------------------------------------------------------------------------
/// Formal error envelope produced when an execution boundary or invariant tripwire is breached.
///
/// Unlike standard Rust errors or panics that destroy process context, [`TripwireError`]
/// encapsulates rich cybernetic metadata:
/// - Channel identifier for precise subsystem origin tracking.
/// - Ontological failpoint category ([`FailpointType`]).
/// - Human and machine readable diagnostic message.
/// - Balanced-ternary recovery state (`-1:ROLLBACK` or `0:AWAIT`).
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct TripwireError {
    /// Subsystem or channel where the tripwire fired (e.g. "BOOT", "PLATFORM", "TERMINAL", "SCHEDULER").
    pub channel: &'static str,
    /// Formal diagnostic classification of the failpoint.
    pub failpoint_type: FailpointType,
    /// Detailed diagnostic message explaining the breach.
    pub message: String,
    /// Balanced ternary state mapping for recovery (typically `-1:ROLLBACK` or `0:AWAIT`).
    pub recovery_state: QuadState,
}

impl TripwireError {
    /// Creates a new tripwire error mapped to `-1:ROLLBACK` state.
    pub fn rollback(channel: &'static str, fp_type: FailpointType, msg: impl Into<String>) -> Self {
        Self {
            channel,
            failpoint_type: fp_type,
            message: msg.into(),
            recovery_state: QuadState::Rollback,
        }
    }

    /// Creates a new tripwire error mapped to `0:AWAIT` standby state.
    pub fn await_state(channel: &'static str, fp_type: FailpointType, msg: impl Into<String>) -> Self {
        Self {
            channel,
            failpoint_type: fp_type,
            message: msg.into(),
            recovery_state: QuadState::Await,
        }
    }

    /// Returns `true` if this error dictates an active transaction rollback (`-1:ROLLBACK`).
```

```
    pub fn is_rollback(&self) -> bool {
        self.recovery_state == QuadState::Rollback
    }

    /// Returns `true` if this error indicates that the operation is awaiting asynchronous completion (`0:AWAIT`).
    pub fn is_await(&self) -> bool {
        self.recovery_state == QuadState::Await
    }
}

impl fmt::Display for TripwireError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            f,
            "[{}] Failpoint {}: {} (State: {})",
            self.channel,
            self.failpoint_type.as_str(),
            self.message,
            self.recovery_state.macro_op()
        )
    }
}

impl std::error::Error for TripwireError {}

// ---------------------------------------------------------------------------
// 2.3. Initial Truth States & Equilibrium Recovery Defaults
// ---------------------------------------------------------------------------
/// Default quad-state recovery commitment when a tripwire collision occurs.
pub const DEFAULT_TRIPWIRE_RECOVERY_STATE: QuadState = QuadState::Rollback;

/// Default diagnostic channel assigned when no subsystem is explicitly specified.
pub const DEFAULT_TRIPWIRE_CHANNEL: &str = "TRIPWIRE";

// ===========================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Tripwire Engine Implementation & Assertion Oracles )
// ===========================================================================

/// Mechanical Tripwire Sensor for invariant enforcement, collision sensing, and diagnostic alerting.
#[derive(Debug, Clone, Copy, Default)]
pub struct Tripwire;

// ---------------------------------------------------------------------------
// 3.1. Explicit Failpoint Inscription & Remediation Oracles
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Explicitly trips a diagnostic wire, recording a failpoint in the logger and emitting a standard alert.
    ///
    /// # Game Dev Collision Volume Hook:
    /// Inscribes a critical failpoint into [`DiagnosticLogger`], writes an ANSI red alert to stderr,
    /// and packages the violation into a [`TripwireError`] with `-1:ROLLBACK`.
    pub fn trip(
        logger: &DiagnosticLogger,
        channel: &'static str,
        fp_type: FailpointType,
        message: impl Into<String>,
    ) -> TripwireError {
        let msg = message.into();
        logger.failpoint(fp_type, channel, msg.clone());

        eprintln!(
            "\x1b[1;31m✗ Tripwire Alert [{}]\x1b[0m [{}]: {}",
            channel,
            fp_type.as_str(),
            msg
        );

        TripwireError::rollback(channel, fp_type, msg)
    }

    /// Explicitly trips a diagnostic wire equipped with concrete remediation telemetry for the Restorator.
    ///
    /// # Restorator Immune Handoff:
    /// Attaches a [`FixSuggestion`] and registers an episodic cognitive memory note tagged
    /// `#remediation_ready`, allowing automated self-healing passes to immediately parse and apply the patch.
    pub fn trip_with_remediation(
        logger: &DiagnosticLogger,
```

```
        channel: &'static str,
        fp_type: FailpointType,
        message: impl Into<String>,
        suggestion: FixSuggestion,
    ) -> TripwireError {
        let msg = message.into();
        logger.failpoint_with_remediation(fp_type, channel, msg.clone(), suggestion);

        eprintln!(
            "\x1b[1;31m✗ Tripwire Alert [{}]\x1b[0m [{}]: {}",
            channel,
            fp_type.as_str(),
            msg
        );

        TripwireError::rollback(channel, fp_type, msg)
    }

    /// Trips a diagnostic wire that pauses execution in `0:AWAIT` standby state rather than rolling back.
    ///
    /// Used when a required resource is temporarily in flight or pending background synthesis.
    pub fn trip_await(
        logger: &DiagnosticLogger,
        channel: &'static str,
        fp_type: FailpointType,
        message: impl Into<String>,
    ) -> TripwireError {
        let msg = message.into();
        logger.failpoint(fp_type, channel, msg.clone());

        eprintln!(
            "\x1b[1;33m⚠ Tripwire Standby [{}]\x1b[0m [{}]: {}",
            channel,
            fp_type.as_str(),
            msg
        );

        TripwireError::await_state(channel, fp_type, msg)
    }
}

// ---------------------------------------------------------------------------
// 3.2. Filesystem Spatial Invariant Assertions
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a required file or directory exists on disk.
    ///
    /// # Precondition Sensor:
    /// Returns the verified [`PathBuf`] on success (+1:PROCEED). If absent, trips
    /// a [`FailpointType::SyntaxAnomaly`] error with `-1:ROLLBACK`.
    pub fn assert_path_exists(
        logger: &DiagnosticLogger,
        channel: &'static str,
        path: impl AsRef<Path>,
        description: &'static str,
    ) -> Result<PathBuf, TripwireError> {
        let p = path.as_ref();
        if p.exists() {
            Ok(p.to_path_buf())
        } else {
            let msg = format!("{description} not found at expected path: {:?}", p);
            Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
        }
    }

    /// Asserts that a target path exists and is specifically a directory.
    pub fn assert_dir_exists(
        logger: &DiagnosticLogger,
        channel: &'static str,
        path: impl AsRef<Path>,
        description: &'static str,
    ) -> Result<PathBuf, TripwireError> {
        let p = path.as_ref();
        if p.is_dir() {
            Ok(p.to_path_buf())
        } else {
            let msg = format!("{description} directory does not exist: {:?}", p);
            Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
```

```
        }
    }

    /// Asserts that a target file exists and is readable by the current process.
    pub fn assert_file_readable(
        logger: &DiagnosticLogger,
        channel: &'static str,
        path: impl AsRef<Path>,
        description: &'static str,
    ) -> Result<PathBuf, TripwireError> {
        let p = path.as_ref();
        if !p.exists() {
            let msg = format!("{description} file does not exist: {:?}", p);
            return Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg));
        }

        match fs::metadata(p) {
            Ok(_) => Ok(p.to_path_buf()),
            Err(e) => {
                let msg = format!("Cannot read metadata for {description} at {:?}: {}", p, e);
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ---------------------------------------------------------------------------
// 3.3. Host Toolchain & Binary Availability Oracles
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a required executable binary is available on the host system PATH.
    ///
    /// # Platform Awareness:
    /// Dispatches `where` on Windows and `which` on Unix/macOS to probe host environment variables.
    pub fn assert_binary_available(
        logger: &DiagnosticLogger,
        channel: &'static str,
        bin_name: &'static str,
    ) -> Result<(), TripwireError> {
        let check = if cfg!(target_os = "windows") {
            Command::new("where").arg(bin_name).output()
        } else {
            Command::new("which").arg(bin_name).output()
        };

        match check {
            Ok(out) if out.status.success() => Ok(()),
            _ => {
                let msg = format!(
                    "Required host binary '{bin_name}' is not accessible on system PATH"
                );
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ---------------------------------------------------------------------------
// 3.4. Subprocess Lifecycle & Exit Status Assertions
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a subprocess command completed successfully with exit code 0.
    ///
    /// Traps non-zero exit codes or spawn failures without process panic.
    pub fn assert_command_success(
        logger: &DiagnosticLogger,
        channel: &'static str,
        status: Result<ExitStatus, std::io::Error>,
        command_name: &'static str,
    ) -> Result<(), TripwireError> {
        match status {
            Ok(s) if s.success() => Ok(()),
            Ok(s) => {
                let code = s.code().map(|c| c.to_string()).unwrap_or_else(|| "signal".into());
                let msg = format!("Subprocess '{command_name}' exited with non-zero status ({code})");
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
            Err(e) => {
```

```
                let msg = format!("Failed to spawn or communicate with subprocess '{command_name}': {e}");
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }

    /// Asserts that a captured subprocess [`Output`] succeeded, logging stderr on failure.
    pub fn assert_command_output_clean(
        logger: &DiagnosticLogger,
        channel: &'static str,
        output: Result<Output, std::io::Error>,
        command_name: &'static str,
    ) -> Result<Output, TripwireError> {
        match output {
            Ok(out) if out.status.success() => Ok(out),
            Ok(out) => {
                let code = out.status.code().unwrap_or(-1);
                let stderr = String::from_utf8_lossy(&out.stderr);
                let msg = format!(
                    "Subprocess '{command_name}' failed with status code {}: {}",
                    code, stderr.trim()
                );
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
            Err(e) => {
                let msg = format!("Failed to execute subprocess '{command_name}': {e}");
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ---------------------------------------------------------------------------
// 3.5. Mathematical, State Machine & Causality Boundary Assertions
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a quad-state evaluation committed to `+1:PROCEED`.
    ///
    /// If the state is `-1:ROLLBACK` or `0:AWAIT`, raises an explicit tripwire error.
    pub fn assert_quad_state_proceed(
        logger: &DiagnosticLogger,
        channel: &'static str,
        state: QuadState,
        context: &'static str,
    ) -> Result<(), TripwireError> {
        if state == QuadState::Proceed {
            Ok(())
        } else {
            let msg = format!(
                "QuadState invariant breach in {context}: expected +1:PROCEED, encountered {}",
                state.macro_op()
            );
            Err(Self::trip(logger, channel, FailpointType::RollbackTriggered, msg))
        }
    }

    /// Asserts an arbitrary boolean propositional truth condition.
    ///
    /// If false, trips the wire with the designated failpoint category and message.
    pub fn assert_invariant(
        logger: &DiagnosticLogger,
        channel: &'static str,
        condition: bool,
        fp_type: FailpointType,
        message: impl Into<String>,
    ) -> Result<(), TripwireError> {
        if condition {
            Ok(())
        } else {
            Err(Self::trip(logger, channel, fp_type, message))
        }
    }
}

// ===========================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing, Baton Pass & In-File Unit Tests )
// ===========================================================================
```

```rust

// ---------------------------------------------------------------------------
// 4.1. Universal Tripwire Sealing & Downstream Baton Pass Specification
// ---------------------------------------------------------------------------
// BATON PASS CONTRACT:
//   - Upstream Producer: Hardware, Filesystem, Runtime Processes & Evaluator ALU.
//   - Downstream Receivers:
//     * `<TRACK 1>` Boot Gear (`src/cli/commands/boot.rs`): Inception and compiler tripwires.
//     * `<TRACK 1>` Platform Cog (`src/cogs/platform.rs`): Hypervisor and firmware tripwires.
//     * `<TRACK 1>` Diagnostics (`src/diagnostics/`): Restorator immune repair and Debugger frames.
//     * `<TRACK 1>` Terminal Session (`src/terminal/session.rs`): Tape mount and execution boundary tripwires.
//   - Certified Invariants:
//     1. Tripwire breaches never panic or abort process memory; all breaches produce `TripwireError`.
//     2. All tripwire events inscribe structured failpoint records into `DiagnosticLogger`.
//     3. `Tripwire::trip_with_remediation` attaches `FixSuggestion` and tags `#remediation_ready`.
//     4. Filesystem, binary, subprocess, and boolean invariant assertion methods return
//        deterministic, uncheatable results.

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::{FixKind, FixSuggestion};

    /// Verifies that explicitly tripping a wire records a failpoint in `DiagnosticLogger`
    /// and constructs a `TripwireError` with `-1:ROLLBACK` recovery state.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents silent failure where an error is created without inscriptive telemetry.
    #[test]
    fn test_tripwire_explicit_trip_and_failpoint_capture() {
        let logger = DiagnosticLogger::new(false);
        let err = Tripwire::trip(
            &logger,
            "TEST_CHANNEL",
            FailpointType::SyntaxAnomaly,
            "Simulated explicit tripwire fault",
        );

        assert_eq!(err.channel, "TEST_CHANNEL");
        assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
        assert_eq!(err.recovery_state, QuadState::Rollback);
        assert!(err.is_rollback());
        assert!(!err.is_await());
        assert!(logger.has_failpoints(), "DiagnosticLogger must record the failpoint");
        assert_eq!(logger.failpoints().len(), 1);
    }

    /// Verifies that `trip_with_remediation` successfully attaches a `FixSuggestion`
    /// and emits an episodic cognitive note tagged `#remediation_ready`.
    ///
    /// # Invariants Tested:
    /// - Failpoint contains the attached fix suggestion.
    /// - Cognitive notes buffer contains a note tagged `#remediation_ready`.
    #[test]
    fn test_tripwire_with_remediation_suggestion() {
        let logger = DiagnosticLogger::new(false);
        let suggestion = FixSuggestion {
            kind: FixKind::AddBlockHeader,
            line: 1,
            col: 1,
            target_token: "HEADER".to_string(),
            suggested_patch: "BLOCK 1: METADATA".to_string(),
            rationale: "Automated header repair".to_string(),
        };

        let err = Tripwire::trip_with_remediation(
            &logger,
            "REPAIR_CHANNEL",
            FailpointType::BoundaryMismatch,
            "Simulated repairable fault",
            suggestion.clone(),
        );

        assert_eq!(err.channel, "REPAIR_CHANNEL");
        assert_eq!(err.failpoint_type, FailpointType::BoundaryMismatch);

        let fps = logger.failpoints();
        assert_eq!(fps.len(), 1);
```

```
        assert_eq!(fps[0].suggestion, Some(suggestion));

        let notes = logger.cognitive_notes();
        assert!(
            notes.iter().any(|n| n.tags.iter().any(|t| t == "remediation_ready")),
            "Episodic notes must include #remediation_ready tag"
        );
    }

    /// Verifies that `trip_await` sets `0:AWAIT` standby state rather than rolling back.
    #[test]
    fn test_tripwire_await_state_constructor() {
        let logger = DiagnosticLogger::new(false);
        let err = Tripwire::trip_await(
            &logger,
            "ASYNC_CHANNEL",
            FailpointType::NullTrap,
            "Pending background artifact",
        );

        assert_eq!(err.channel, "ASYNC_CHANNEL");
        assert_eq!(err.recovery_state, QuadState::Await);
        assert!(err.is_await());
        assert!(!err.is_rollback());
    }

    /// Verifies that `assert_path_exists` returns `Ok(PathBuf)` for existing paths
    /// and `Err(TripwireError)` with failpoint telemetry for missing paths.
    ///
    /// # Uncheatable Assertion:
    /// Tests both positive path resolution (`Cargo.toml`) and negative path failure.
    #[test]
    fn test_assert_path_exists_contract() {
        let logger = DiagnosticLogger::new(false);

        // Positive check: Cargo manifest must resolve
        let ok_res = Tripwire::assert_path_exists(&logger, "TEST", "Cargo.toml", "Cargo Manifest");
        assert!(ok_res.is_ok(), "Cargo.toml must resolve successfully");
        assert!(ok_res.unwrap().exists());

        // Negative check: missing file must trip with failpoint
        let missing = Tripwire::assert_path_exists(&logger, "TEST", "nonexistent_omnicode_file_xyz.omni", "Missing
File");
        assert!(missing.is_err(), "Missing file must return TripwireError");
        let err = missing.unwrap_err();
        assert_eq!(err.channel, "TEST");
        assert!(logger.has_failpoints(), "Logger must capture failpoint on missing path");
    }

    /// Verifies that `assert_dir_exists` validates directories and rejects non-directory files.
    #[test]
    fn test_assert_dir_exists_contract() {
        let logger = DiagnosticLogger::new(false);

        // Positive: current workspace root is a directory
        assert!(Tripwire::assert_dir_exists(&logger, "TEST", ".", "Workspace Root Directory").is_ok());

        // Negative: Cargo.toml is a file, not a directory
        assert!(Tripwire::assert_dir_exists(&logger, "TEST", "Cargo.toml", "Manifest File As Dir").is_err());
    }

    /// Verifies that `assert_invariant` succeeds on true conditions and trips on false conditions.
    #[test]
    fn test_assert_invariant_contract() {
        let logger = DiagnosticLogger::new(false);

        let ok_res = Tripwire::assert_invariant(&logger, "MATH", 2 + 2 == 4, FailpointType::SyntaxAnomaly, "Arithmetic
holds");
        assert!(ok_res.is_ok());

        let err_res = Tripwire::assert_invariant(&logger, "MATH", 2 + 2 == 5, FailpointType::SyntaxAnomaly, "Arithmetic
breached");
        assert!(err_res.is_err());
        assert!(logger.has_failpoints());
    }

    /// Verifies that `assert_quad_state_proceed` accepts `QuadState::Proceed` and trips on `Rollback` or `Await`.
    #[test]
```

```
    fn test_assert_quad_state_proceed_contract() {
        let logger = DiagnosticLogger::new(false);

        assert!(Tripwire::assert_quad_state_proceed(&logger, "EXEC", QuadState::Proceed, "Step 1").is_ok());
        assert!(Tripwire::assert_quad_state_proceed(&logger, "EXEC", QuadState::Rollback, "Step 2").is_err());
        assert!(Tripwire::assert_quad_state_proceed(&logger, "EXEC", QuadState::Await, "Step 3").is_err());
    }

    /// Verifies that `TripwireError` implements `Display` with channel, category, message, and state.
    #[test]
    fn test_tripwire_error_display_formatting() {
        let err = TripwireError::rollback("KERNEL", FailpointType::ImmutableMutation, "Attempted write to immutable
register");
        let formatted = format!("{err}");
        assert!(formatted.contains("[KERNEL]"));
        assert!(formatted.contains("FP_IMMUTABLE_MUTATION"));
        assert!(formatted.contains("Attempted write to immutable register"));
        assert!(formatted.contains("ROLLBACK"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/mod.rs RS 30 LINES 102 WORDS 1.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Subsystem Identity and Export Manifest )
// ============================================================================
//! OmniCode AST Subsystem
//! Houses all syntax tree nodes, boundary containers, and expression shapes.

pub mod nodes;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for AST Types )
// ============================================================================
pub use nodes::*;


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( AST Subsystem Manifest Definitions )
// ============================================================================
// AST definitions are actively mapped in nodes.rs.


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Module Status )
// ============================================================================
// AST subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes.rs RS 152 LINES 408 WORDS 4.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Node Definitions, Boundary Typing & Ontological Hierarchy )
// ============================================================================
//! OmniCode AST: Abstract Syntax Tree Nodes
//!
//! Preserves:
//!     1. First-Class Asides (Machine intent & AI reasoning traces)
//!     2. Boundary Typology (Denotation, Invariant, Connotation, TopDown, BottomUp)
//!     3. Directional Flow & Ontological Relations

use crate::core::boundaries::BoundaryType;
use crate::core::operators::{BeingRelation, Connective, DirectionalVector, Polarity, AffixPosition};
use crate::core::ternary::QuadState;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped AST Node Structures: Aside, Boundary, Entity, Relations )
// ============================================================================
#[derive(Debug, Clone, PartialEq)]
pub struct AsideNode {
    pub intent: String,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct BoundaryNode {
    pub boundary_type: BoundaryType,
    pub elements: Vec<ASTNode>,
    pub aside: Option<AsideNode>,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct EntityNode {
    pub name: String,
    pub definition: BoundaryNode,
    pub aside: Option<AsideNode>,
    pub context: Option<BoundaryNode>,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct RelationNode {
    pub left: Box<ASTNode>,
    pub relation: BeingRelation,
    pub right: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct DirectionalNode {
    pub source: Box<ASTNode>,
    pub vector: DirectionalVector,
    pub target: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct ConnectiveNode {
    pub left: Box<ASTNode>,
    pub connective: Connective,
    pub right: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
```

```
pub struct UnaryNotNode {
    pub operand: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum ArithmeticOp {
    Add,
    Sub,
    Mul,
    Div,
}

#[derive(Debug, Clone, PartialEq)]
pub struct ArithmeticNode {
    pub left: Box<ASTNode>,
    pub op: ArithmeticOp,
    pub right: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct PolarityNode {
    pub name: String,
    pub polarity: Polarity,
    pub position: AffixPosition,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct BoundaryCastNode {
    pub boundary: BoundaryType,
    pub polarity: Polarity,
    pub line: usize,
    pub col: usize,
}

#[derive(Debug, Clone, PartialEq)]
pub struct ProgramNode {
    pub statements: Vec<ASTNode>,
    pub line: usize,
    pub col: usize,
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Unified ASTNode Enumeration: Complete Node Hierarchy )
// ============================================================================
#[derive(Debug, Clone, PartialEq)]
pub enum ASTNode {
    Aside(AsideNode),
    Ternary(QuadState),
    Identifier(String),
    Polarity(PolarityNode),
    BoundaryCast(BoundaryCastNode),
    Number(f64),
    StringLit(String),
    Boundary(BoundaryNode),
    Entity(EntityNode),
    Relation(RelationNode),
    Directional(DirectionalNode),
    Connective(ConnectiveNode),
    Arithmetic(ArithmeticNode),
    UnaryNot(UnaryNotNode),
    Program(ProgramNode),
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( AST Node Invariant Verification & Trait Guarantees )
// ============================================================================
// AST nodes satisfy Send, Sync, Clone, PartialEq invariants across all targets.
```